



DEITEL® DEVELOPER SERIES

Modern C++

Functional-Style Programming

Templates

Copy/Move Semantics

Concepts

Spaceship Operator

Smart Pointers

Text Formatting

Security

Modules

Standard Library

C++20

для

программистов

Метод готовых объектов

Executors

Design Patterns

Ranges/Views

Coroutines

Performance

Contracts

Open Source Libraries

jthread

Concurrency

Parallel Algorithms

Lambdas

ПОЛ ДЕЙТЕЛ • ХАРВИ ДЕЙТЕЛ



C++20 для программистов

Пол Дейтел, Харви Дейтел

Часть 1

Базовые возможности C++20:
быстрый старт. Процедурное
программирование

1. Введение и тест-драйв популярных
бесплатных компиляторов C++
Краткая справка по OOP. C++ в Windows*,
MacOS* и Linux (Visual C++*, g++*, Xcode*
и clang++)

2. Азы программирования на C++
Базовые понятия C++. Введение в метод
готовых объектов (MGO): быстрое создание
объектно-ориентированных программ
с помощью библиотек. MGO: работа
с объектами класса string

3. Управляющие инструкции (часть 1)
Основы форматирования текста в C++20.
MGO: операции с чрезвычайно большими
целыми числами с помощью бесплатных
библиотек с открытым кодом

4. Управляющие инструкции (часть 2)
MGO: библиотека miniz-cpp для записи
и чтения ZIP-файлов

5. Функции и шаблоны функций
MGO: `Unflynn Unqatmih Wjtz Qarvc:`
`Qwazkptm xzz Xndmwwghlz`
(зашифрованная фраза из нашего примера
шифрования с закрытым ключом)

- Язык C++ как средство создания критически важных систем.
- «Большая четверка» C++20: диапазоны, концепты, модули и корутины.
- Сотни примеров исходного кода с комментариями.
- Адрес для общения с авторами: deitel1@deitel.com

Часть 3

Современное объектно-ориентированное программирование и исключения

9. Пользовательские классы
MGO: сериализация объектов в формате JSON

10. OOP: наследование и динамический полиморфизм
Наследование интерфейса, идиома неvirtуального интерфейса (NVI), `std::variant` и `std::visit`

11. Перегрузка операторов, семантика копирования и перемещения, умные указатели и RAIIP
Создание уникальных классов: пользовательский класс `MyArray`. Оператор трехстороннего сравнения (`<=>`) в C++20, управление ресурсами с помощью RAIIP

12. Исключения и обзор контрактного программирования

Часть 4

Обобщенное программирование: шаблоны, концепты и метапрограммирование

13. Контейнеры и итераторы стандартной библиотеки
Работа с готовыми структурами данных

14. Алгоритмы стандартной библиотеки, диапазоны и представления C++20
Функциональное программирование

15. Шаблоны, концепты (C++20) и метапрограммирование
Динамический полиморфизм, шаблоны функций, сокращенные шаблоны функций в C++20, шаблоны классов, вариативные шаблоны, выражения свертки

Часть 5

Более сложные темы: модули, параллельные алгоритмы, конкурентность и корутины

16. Модули C++20: технология разработки больших программ
Импорт заголовков, заголовочные единицы, объявление модуля, фрагменты модуля, разделы

17. Параллельные алгоритмы и конкурентность: высокоуровневый подход
Программирование для многоядерных процессоров с параллельными алгоритмами C++17. Конкурентность, многопоточность

18. Корутины (C++20)
`co_await`, `co_yield`, `co_return`. Библиотеки поддержки корутин, генераторы, исполнители и задачи

Часть 6

Онлайн-главы
(на английском языке)

19. (Online) Stream I/O
and C++20 Text Formatting

20. (Online) Other Topics
and a Look Toward C++23

- Ресурсы для разработчиков: GitHub*, StackOverflow*, проекты с открытым исходным кодом и многие другие.
- Контейнеры Docker с компиляторами GNU g++ и Clang.
- Обзор ожидаемых возможностей C++23.
Блог: <https://deitel.com/blog>

4e DEITEL DEVELOPER SERIES

Modern C++
Concepts
Modules

C++20
для
программистов

Метод готовых объектов

Ranges Views
Coroutines
Concurrency

ПОЛ ДЕЙТЕЛ - ХАРВИ ДЕЙТЕЛ

- Инструменты статического анализа кода.
- Загрузка исходного кода по адресу: <https://deitel.com/cpp20fp>



DEITEL® DEVELOPER SERIES

C++20 for Programmers

An Objects-Natural Approach

Paul Deitel • Harvey Deitel



Pearson

Boston • Columbus • New York • San Francisco • Amsterdam • Cape Town
Dubai • London • Madrid • Milan • Munich • Paris • Montreal • Toronto • Delhi • Mexico City
São Paulo • Sydney • Hong Kong • Seoul • Singapore • Taipei • Tokyo



DEITEL® DEVELOPER SERIES

C++20

для

программистов

Метод готовых объектов

Пол Дейтел • Харви Дейтел



Санкт-Петербург • Москва • Минск

2024

ББК 32.973.2-018.1

УДК 004.43

Д27

Дейтел Пол, Дейтел Харви

Д27 С++20 для программистов. — СПб.: Питер, 2024. — 1056 с.: ил. — (Серия «Библиотека программиста»).

ISBN 978-5-4461-2359-9

Программисты с опытом работы на других языках высокого уровня смогут на практике освоить современный C++ и «большую четверку» его новых возможностей: диапазоны, концепты, модули и корутины.

200+ практических примеров реального исходного кода позволят быстро овладеть идиомами современного C++, используя популярные компиляторы: Visual C++®, GNU® g++, Apple® Xcode® и LLVM®/Clang. Знание базы позволит перейти к контейнерам стандартной библиотеки C++ `algorithm` и `vector`; функциональному программированию с диапазонами и представлениями C++20; строкам, файлам и регулярным выражениям; объектно-ориентированному программированию с классами, наследованием, динамическим и статическим полиморфизмом; перегрузке операторов, семантике копирования и перемещения, RAII и умным указателям; исключениям и ожидаемым в C++23 контрактам; контейнерам, итераторам и алгоритмам стандартной библиотеки; шаблонам, концептам C++20 и метапрограммированию; модулям C++ 20 и технологии разработки больших программ; конкурентности, параллелизму, параллельным алгоритмам стандартной библиотеки C++17 и C++20 и корутинам C++20.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ББК 32.973.2-018.1

УДК 004.43

Права на издание получены по соглашению с Pearson Education Inc. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полную приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. В книге возможны упоминания организаций, деятельность которых запрещена на территории Российской Федерации, таких как Meta Platforms Inc., Facebook, Instagram и др. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

ISBN 978-0136905691 англ.

Authorized translation from the English language edition, entitled C++20 FOR PROGRAMMERS, 3rd Edition by PAUL DEITEL; HARVEY DEITEL, published by Pearson Education, Inc. publishing as Pearson, © 2022

ISBN 978-5-4461-2359-9

© Перевод на русский язык ООО «Прогресс книга», 2024

© Издание на русском языке, оформление ООО «Прогресс книга», 2024

© Серия «Библиотека программиста», 2024

Краткое содержание

Предисловие	24
Об авторах	46
От издательства	47
Перед началом работы.....	48
Глава 1. Введение и тест-драйв популярных бесплатных компиляторов C++	56
Глава 2. Азы программирования на C++	79
Глава 3. Управляющие инструкции (часть 1)	101
Глава 4. Управляющие инструкции (часть 2)	137
Глава 5. Функции и шаблоны функций	175
Глава 6. Массивы, векторы, библиотека <code>ranges</code> и функциональное программирование	235
Глава 7. Указатели в современном C++ (фактор риска).....	277
Глава 8. Строки и их представления, текстовые файлы, CSV-файлы и регулярные выражения.....	315
Глава 9. Пользовательские классы	370
Глава 10. ООП: наследование и динамический полиморфизм.....	446
Глава 11. Перегрузка операторов, семантика копирования и перемещения, умные указатели	535
Глава 12. Исключения и обзор контрактного программирования	597
Глава 13. Контейнеры и итераторы стандартной библиотеки	642

Глава 14. Алгоритмы стандартной библиотеки, диапазоны и представления C++20	701
Глава 15. Шаблоны, концепты (C++20) и метапрограммирование	779
Глава 16. Модули (C++20): технология разработки больших программ	875
Глава 17. Параллельные алгоритмы и конкурентность: высокоуровневый подход.....	931
Глава 18. Корутин (C++20)	1024
Приложение А. Приоритеты и группировка операторов	1052
Приложение Б. Набор символов.....	1055

Оглавление

Предисловие.....	24
Современный C++.....	24
В курсе событий.....	25
Кому пригодится эта книга.....	25
Готовые файлы с исходным кодом программ.....	25
Три профессиональных компилятора на выбор.....	26
Учебный метод готовых объектов.....	26
Что такое готовые объекты?.....	27
Бесплатные классы.....	27
Проект Boost.....	27
Примеры кода с готовыми объектами.....	28
Краткое содержание книги.....	28
Часть 1. Быстрый старт и базовые возможности C++.....	29
Часть 2. Массивы, указатели и строки.....	30
Часть 3. Объектно-ориентированное программирование.....	31
Часть 4. Контейнеры, итераторы и алгоритмы стандартной библиотеки.....	33
Часть 5. Более сложные темы.....	34
Приложения.....	35
Дополнительные материалы на сайте deitel.com.....	35
C++ Core Guidelines.....	36
Библиотека Guidelines Support Library (GSL).....	36
Профессиональные статические анализаторы кода.....	37
Интернет-ресурсы для программистов.....	37
Stack Overflow.....	37
GitHub.....	38
Docker.....	39
Некоторые ключевые документы и ресурсы по C++.....	39
Документация.....	40
Блоги.....	40
Дополнительные ресурсы.....	41
Сайты, где ответят на ваши вопросы.....	41
Общение с авторами.....	41

Благодарности	41
Рецензенты	42
Артур О'Двайр	43
GitHub.....	43
Мэтт Годболт и Compiler Explorer.....	43
Дитмар Куль.....	44
Райнер Гримм	44
Брайан Гетц.....	44
Разработчики программ с открытым исходным кодом и блогеры	44
Поисковая система Google.....	45
Grammarly.....	45
Об авторах	46
От издательства	47
Перед началом работы	48
Условные обозначения	48
Загрузка примеров кода.....	48
Компиляторы, на которых мы тестировали исходный код	49
Не все примеры кода совместимы со всеми тремя компиляторами	49
Установка Visual Studio Community Edition в Windows	50
Установка Xcode в macOS	50
Установка актуальной версии GNU C++.....	50
Установка GNU Compiler Collection в Ubuntu Linux, запущенной в подсистеме Windows для Linux	51
Docker и контейнеры Docker	52
Установка Docker	52
Загрузка контейнера Docker, содержащего GNU Compiler Collection.....	52
Загрузка контейнера Docker, содержащего Clang.....	53
Ответы на ваши вопросы по C++	53
Онлайн-документация по C++	54
О библиотеке форматирования текста (fmt)	54
Средства статического анализа кода	54
Глава 1. Введение и тест-драйв популярных бесплатных компиляторов C++	56
1.1. Введение	57
1.2. Тест-драйв: компиляция приложения, написанного на C++20	58
1.2.1. Компиляция и запуск приложения в Windows с помощью Visual Studio 2022.....	59

1.2.2. Компиляция и запуск приложения в macOS с помощью Xcode	63
1.2.3. Компиляция и запуск приложения в Linux с помощью g++	67
1.2.4. Компиляция и запуск приложения в контейнере Docker с помощью g++	70
1.2.5. Компиляция и запуск приложения в контейнере Docker с помощью clang++	71
1.3. Закон Мура, многоядерные процессоры и многопоточное программирование	73
1.4. Что такое объектно-ориентированное программирование?	75
1.5. Итоги	78
Глава 2. Азы программирования на C++	79
2.1. Введение	80
2.2. Первая программа на C++: вывод текста на экран	80
2.3. Модификация нашей первой программы на C++	85
2.4. Вторая программа на C++: сложение целых чисел	86
2.5. Арифметические операторы	90
2.6. Принятие решений: операторы равенства и сравнения	92
2.7. Готовые объекты: создание и использование объектов класса string стандартной библиотеки	96
2.8. Итоги	100
Глава 3. Управляющие инструкции (часть 1)	101
3.1. Введение	102
3.2. Управляющие структуры	102
3.2.1. Последовательность	103
3.2.2. Инструкции выбора	104
3.2.3. Инструкции цикла	104
3.2.4. Комбинации управляющих инструкций	105
3.3. Инструкция одиночного выбора if	106
3.4. Инструкция двойного выбора if...else	107
3.4.1. Вложенные инструкции if...else	108
3.4.2. Блоки	109
3.4.3. Тернарный условный оператор ?	110
3.5. Инструкция цикла while	110
3.6. Цикл со счетчиком	111
3.6.1. Реализация цикла со счетчиком	112
3.6.2. Целочисленное деление с усечением	113
3.7. Цикл с контрольным значением	114
3.7.1. Реализация цикла с контрольным значением	114
3.7.2. Явное и неявное преобразование основных типов данных	117
3.7.3. Форматирование чисел с плавающей точкой	118

3.8. Вложенные управляющие инструкции	119
3.8.1. Постановка задачи	119
3.8.2. Реализация программы	120
3.8.3. Предотвращение сужающих преобразований с помощью скобочной инициализации	122
3.9. Составные операторы присваивания	123
3.10. Операторы инкремента и декремента	123
3.11. Несовместимость основных типов данных на разных платформах	127
3.12. Готовые объекты: работа с целыми числами произвольной величины	127
3.13. Форматирование текста функцией <code>format</code>	133
3.14. Итоги	135
Глава 4. Управляющие инструкции (часть 2)	137
4.1. Введение	138
4.2. Как работает цикл со счетчиком	138
4.3. Инструкция цикла <code>for</code>	139
4.4. Инкремент и декремент счетчика в цикле <code>for</code>	142
4.5. Программа для вычисления суммы четных чисел	143
4.6. Программа для расчета сложных процентов	144
4.7. Инструкция цикла <code>do...while</code>	148
4.8. Инструкция множественного выбора <code>switch</code>	150
4.9. Инструкции выбора с инициализаторами	156
4.10. Инструкции <code>break</code> и <code>continue</code>	157
4.11. Логические операторы	159
4.11.1. Оператор <code>&&</code> (логическое И)	159
4.11.2. Оператор <code> </code> (логическое ИЛИ)	160
4.11.3. Вычисление по короткой схеме	161
4.11.4. Оператор <code>!</code> (логическое НЕ)	162
4.11.5. Программное создание таблиц истинности для логических операторов	162
4.12. Типичные ошибки при работе с операторами равенства (<code>==</code>) и присваивания (<code>=</code>)	164
4.13. Готовые объекты: создание и чтение ZIP-файлов	166
4.14. Форматирование текста с указанием ширины полей и точности	171
4.15. Итоги	174
Глава 5. Функции и шаблоны функций	175
5.1. Введение	176
5.2. Функции и классы — основные компоненты программ	176
5.3. Функции математической библиотеки	177
5.4. Определения функций и прототипы функций	180

5.5. Порядок вычисления аргументов функции.....	183
5.6. Дополнительные сведения о прототипах функций и принудительном приведении аргументов	184
5.6.1. Сигнатуры функций и прототипы функций	184
5.6.2. Принудительное приведение аргументов.....	184
5.6.3. Правила продвижения аргументов и неявные преобразования.....	185
5.7. Заголовки стандартной библиотеки C++	187
5.8. Генерация случайных чисел.....	190
5.8.1. Бросаем игровой кубик	191
5.8.2. Бросаем кубик 60 миллионов раз.....	192
5.8.3. Инициализация генератора случайных чисел.....	194
5.8.4. Инициализация генератора случайных чисел с помощью <code>random_device</code>	195
5.9. Программа «Игра в кости» и знакомство с перечислениями.....	196
5.10. Правила областей видимости	202
5.11. Встроенные функции.....	207
5.12. Ссылки и ссылочные параметры	208
5.13. Аргументы по умолчанию	212
5.14. Оператор разрешения области видимости.....	214
5.15. Перегрузка функций.....	214
5.16. Шаблоны функций.....	218
5.17. Рекурсия.....	221
5.18. Пример рекурсивных вычислений: числа Фибоначчи.....	224
5.19. Рекурсия или итерация?	228
5.20. <code>Infylun Lhqtohm Wjtz Qarcv: Qjwazkrplm xzz Xndmwwqhlz</code>	230
5.21. Итоги	234
 Глава 6. Массивы, векторы, библиотека <code>ranges</code> и функциональное программирование	 235
6.1. Введение	236
6.2. Массивы.....	237
6.3. Объявление массива.....	237
6.4. Инициализация массива в цикле.....	238
6.5. Инициализация массива со списком инициализаторов.....	241
6.6. Цикл <code>for</code> для диапазона значений (C++11) и цикл <code>for</code> для диапазона значений с инициализатором (C++20).....	242
6.7. Заполнение массива и спецификатор <code>constexpr</code>	245
6.8. Суммирование элементов массива.....	247
6.9. Визуализация данных массива на простой диаграмме	248
6.10. Использование элементов массива в качестве счетчиков	250

6.11. Использование массива для подсчета результатов опроса	251
6.12. Сортировка массива и поиск в массиве.....	252
6.13. Многомерные массивы.....	255
6.14. Основы функционального программирования	259
6.14.1. «Что» и «как»	260
6.14.2. Передача функций в качестве аргументов другим функциям, основы лямбда-выражений.....	261
6.14.3. Фильтрация, преобразование и свертка с библиотекой ranges (C++20)	263
6.15. Готовые объекты: шаблон класса vector стандартной библиотеки C++	268
6.16. Итоги	275
Глава 7. Указатели в современном C++ [фактор риска]	277
7.1. Введение	278
7.2. Объявление и инициализация указателей	280
7.2.1. Объявление указателя.....	280
7.2.2. Инициализация указателя	280
7.2.3. Нулевые указатели до C++11	281
7.3. Операторы для работы с указателями	281
7.3.1. Оператор взятия адреса (&)	281
7.3.2. Оператор косвенного обращения (*)	282
7.3.3. Использование операторов взятия адреса (&) и косвенного обращения (*) ...	283
7.4. Передача аргументов по указателю	284
7.5. Традиционные массивы в стиле C	286
7.5.1. Объявление традиционного массива и доступ к его элементам.....	286
7.5.2. Инициализация традиционного массива	289
7.5.3. Передача традиционного массива функции	289
7.5.4. Объявление параметров функции, обрабатывающей традиционный массив ...	289
7.5.5. Пример с функциями sort, begin и end стандартной библиотеки (C++11)	290
7.5.6. Недостатки традиционных массивов.....	290
7.6. Функция to_array (C++20), преобразующая традиционные массивы в std::array ...	291
7.7. Использование const с указателями и данными, на которые они указывают.....	293
7.7.1. Неконстантный указатель на неконстантные данные.....	293
7.7.2. Неконстантный указатель на константные данные	293
7.7.3. Константный указатель на неконстантные данные	294
7.7.4. Константный указатель на константные данные.....	295
7.8. Оператор sizeof	296
7.9. Выражения и арифметические операции с указателями	299
7.9.1. Прибавление и вычитание целых чисел	300
7.9.2. Вычитание одного указателя из другого.....	301

7.9.3. Присваивание указателя.....	301
7.9.4. Невозможность разыменования указателя void*	302
7.9.5. Сравнение указателей	302
7.10. Готовые объекты: доступ к контейнерам через span (C++20)	302
7.11. Строки на основе указателей	309
7.11.1. Аргументы командной строки.....	310
7.11.2. Еще одна возможность функции to_array (C++20)	312
7.12. Другие темы, связанные с указателями	313
7.13. Итоги.....	314
Глава 8. Строки и их представления, текстовые файлы, CSV-файлы и регулярные выражения	315
8.1. Введение	316
8.2. Присваивание и конкатенация строк.....	317
8.3. Сравнение строк	319
8.4. Создание подстрок	321
8.5. Обмен значениями двух строк.....	321
8.6. Получение характеристик строки	322
8.7. Поиск подстрок и символов в строке	325
8.8. Замена и удаление символов в строке	327
8.9. Вставка символов в строку	329
8.10. Преобразования между числами и строками (C++11).....	330
8.11. Представления string_view (C++17)	332
8.12. Файлы и потоки.....	335
8.13. Создание файла с последовательным доступом	336
8.14. Чтение данных из файла с последовательным доступом.....	340
8.15. Чтение и запись текста в кавычках (C++14).....	342
8.16. Обновление данных в файле с последовательным доступом.....	344
8.17. Обработка строковых потоков	344
8.18. Необработанные строковые литералы	347
8.19. Готовые объекты: чтение и анализ CSV-файла с данными о катастрофе «Титаника»	348
8.19.1. Чтение CSV-файла с помощью библиотеки rapidcsv.....	349
8.19.2. Работа с набором данных «Титаник».....	351
8.20. Готовые объекты: основы регулярных выражений	359
8.20.1. Сравнение целых строк с шаблонами.....	361
8.20.2. Замена подстрок.....	365
8.20.3. Поиск в тексте по шаблонам	366
8.21. Итоги	369

Глава 9. Пользовательские классы	370
9.1. Введение	371
9.2. Тест-драйв объекта пользовательского класса Account.....	371
9.3. Класс Account с элементом данных, set-функцией и get-функцией	373
9.3.1. Определение класса	373
9.3.2. Спецификаторы доступа private и public.....	376
9.4. Конструктор класса Account.....	377
9.5. Разработка программ с set-функциями и get-функциями.....	381
9.6. Добавление «денежной» переменной в класс Account	383
9.7. Отделение интерфейса от реализации на примере класса Time.....	386
9.7.1. Интерфейс класса	388
9.7.2. Отделение интерфейса класса от реализации класса.....	388
9.7.3. Определение класса	389
9.7.4. Функции класса Time.....	390
9.7.5. Включение заголовка класса в файл исходного кода	391
9.7.6. Оператор разрешения области видимости (::).....	391
9.7.7. Функция setTime и исключения.....	392
9.7.8. Функции to24HourString и to12HourString.....	392
9.7.9. Неявное встраивание функций	393
9.7.10. Функции класса в сравнении с глобальными функциями	393
9.7.11. Использование класса Time	393
9.7.12. Размер объекта.....	395
9.8. Процесс компиляции и компоновки программы.....	395
9.9. Область видимости класса и доступ к элементам класса.....	397
9.10. Функции доступа и служебные функции	398
9.11. Конструкторы с аргументами по умолчанию.....	398
9.11.1. Усовершенствованный класс Time	398
9.11.2. Перегрузка конструкторов и делегирование конструкторов (C++11).....	404
9.12. Деструкторы	405
9.13. Вызов конструкторов и деструкторов.....	405
9.14. Коварная ловушка: возврат ссылки или указателя на скрытый элемент данных... ..	409
9.15. Оператор присваивания по умолчанию.....	412
9.16. Константные объекты и константные функции класса.....	414
9.17. Композиция: объекты как элементы классов.....	416
9.18. Дружественные функции и классы	421
9.19. Указатель this.....	423
9.19.1. Неявное и явное использование указателя this для доступа к элементам данных объекта	424
9.19.2. Использование указателя this для каскадного вызова функций	425

9.20. Статические элементы класса: данные и функции, общие для всех объектов класса.....	429
9.21. Агрегаты	434
9.21.1. Инициализация агрегата.....	435
9.21.2. Назначенные инициализаторы (C++20).....	435
9.22. Готовые объекты: сериализация в формате JSON	436
9.22.1. Сериализация вектора объектов, содержащего открытые данные.....	438
9.22.2. Сериализация вектора объектов, содержащего скрытые данные	442
9.23. Итоги	444
Глава 10. ООП: наследование и динамический полиморфизм	446
10.1. Введение.....	447
10.2. Базовые и производные классы	450
10.2.1. Иерархия класса CommunityMember.....	450
10.2.2. Иерархия класса Shape и открытое наследование	451
10.3. Взаимосвязь между базовым и производным классами	453
10.3.1. Объявление и использование класса SalariedEmployee.....	453
10.3.2. Создание иерархии наследования: SalariedEmployee и SalariedCommissionEmployee.....	456
10.4. Конструкторы и деструкторы в производных классах	462
10.5. Принцип динамического полиморфизма на примере объектов в видеоигре	463
10.6. Отношения между объектами в иерархии наследования	464
10.6.1. Вызов функций базового класса из объектов производного класса.....	465
10.6.2. Направление указателя производного класса на объект базового класса	468
10.6.3. Вызовы функций производного класса через указатель базового класса	469
10.7. Виртуальные функции и виртуальные деструкторы	470
10.7.1. Зачем нужны виртуальные функции.....	470
10.7.2. Объявление виртуальной функции	471
10.7.3. Вызов виртуальной функции	471
10.7.4. Виртуальные функции в иерархии SalariedEmployee.....	472
10.7.5. Виртуальные деструкторы.....	476
10.7.6. Финальные функции и классы	476
10.8. Абстрактные классы и чистые виртуальные функции	477
10.8.1. Чистые виртуальные функции.....	477
10.8.2. Драйверы устройств: полиморфизм в операционных системах	478
10.9. Полиморфизм в программе расчета зарплаты.....	479
10.9.1. Создание абстрактного базового класса Employee	480
10.9.2. Создание конкретного производного класса SalariedEmployee	482
10.9.3. Создание конкретного производного класса CommissionEmployee	485
10.9.4. Демонстрация полиморфных операций с объектами	487

10.10. Динамический полиморфизм, виртуальные функции и динамическое связывание «под капотом» компилятора	490
10.11. Идиома невиртуального интерфейса	493
10.12. Наследование интерфейса вместо наследования реализации	501
10.12.1. Переосмысление иерархии класса Employee: интерфейс CompensationModel	503
10.12.2. Класс Employee	503
10.12.3. Реализации CompensationModel	505
10.12.4. Тест-драйв новой иерархии	508
10.12.5. Преимущества внедрения зависимостей	509
10.13. Полиморфизм на основе шаблона класса std::variant и функции std::visit	509
10.14. Множественное наследование	516
10.14.1. Ромбовидное наследование	521
10.14.2. Устранение конфликта подобъектов с помощью виртуального наследования базового класса	523
10.15. Защищенные элементы класса	525
10.16. Открытое, защищенное и скрытое наследование	527
10.17. Другие методы динамического полиморфизма и статический полиморфизм	528
10.17.1. Другие методы динамического полиморфизма	529
10.17.2. Методы статического полиморфизма	530
10.17.3. Другие концепции полиморфизма	532
10.18. Итоги	533
Глава 11. Перегрузка операторов, семантика копирования и перемещения, умные указатели	535
11.1. Введение	536
11.2. Перегруженные операторы класса string стандартной библиотеки	538
11.3. Основные принципы перегрузки операторов	543
11.3.1. Перегрузка операторов не происходит автоматически	543
11.3.2. Операторы, которые не могут быть перегружены	544
11.3.3. Операторы, которые не нужно перегружать	544
11.3.4. Правила и ограничения перегрузки операторов	544
11.4. Управление динамической памятью с помощью операторов new и delete	545
11.5. Современные методы управления динамической памятью в C++: RAII и умные указатели	548
11.5.1. Умные указатели	549
11.5.2. Умный указатель unique_ptr	549
11.5.3. Владение объектами при работе с unique_ptr	551
11.5.4. Указатели unique_ptr и традиционные массивы	551

11.6. Создание класса <code>myArray</code> с перегрузкой операторов	552
11.6.1. Специальные функции класса	554
11.6.2. Работа с классом <code>myArray</code>	554
11.6.3. Определение класса <code>myArray</code>	565
11.6.4. Конструктор, задающий размер объекта <code>myArray</code>	566
11.6.5. Скобочная инициализация в конструкторе (C++11)	567
11.6.6. Копирующий конструктор и оператор копирующего присваивания	568
11.6.7. Перемещающий конструктор и оператор перемещающего присваивания	572
11.6.8. Деструктор	576
11.6.9. Функции <code>toString</code> и <code>size</code>	577
11.6.10. Перегрузка операторов равенства и неравенства	577
11.6.11. Перегрузка оператора индекса	579
11.6.12. Перегрузка унарного конвертирующего оператора <code>bool</code>	581
11.6.13. Перегрузка оператора префиксного инкремента	581
11.6.14. Перегрузка оператора постфиксного инкремента	582
11.6.15. Перегрузка оператора сложения с присваиванием	583
11.6.16. Перегрузка бинарных операторов извлечения из потока и вставки в поток	584
11.6.17. Дружественная функция <code>swap</code>	587
11.7. Оператор трехстороннего сравнения (C++20)	587
11.8. Преобразования между типами	591
11.9. Конструкторы и конвертирующие операторы со спецификатором <code>explicit</code>	592
11.10. Перегрузка оператора вызова функции	595
11.11. Итоги	595
Глава 12. Исключения и обзор контрактного программирования	597
12.1. Введение	598
12.2. Поток управления при обработке исключения	602
12.2.1. Определение класса исключений для представления типа проблемы, которая может возникнуть	602
12.2.2. Демонстрация обработки исключений	603
12.2.3. Включение кода в блок <code>try</code>	605
12.2.4. Определение обработчика <code>catch</code> для <code>DivideByZeroException</code>	605
12.2.5. Модель завершения обработки исключений	606
12.2.6. Поток управления при вводе ненулевого знаменателя	607
12.2.7. Поток управления при вводе нулевого знаменателя	607
12.3. Гарантии безопасности исключений и <code>noexcept</code>	608
12.4. Повторная генерация исключений	609
12.5. Размотка стека и необработанные исключения	611

12.6. Когда надо обрабатывать исключения	613
12.6.1. Макрос <code>assert</code>	616
12.6.2. Немедленное завершение	616
12.7. Конструкторы, деструкторы и обработка исключений	617
12.7.1. Генерация исключений в конструкторах	617
12.7.2. Перехват исключений в конструкторах с помощью функциональных блоков <code>try</code>	618
12.7.3. Исключения, деструкторы и <code>noexcept(false)</code>	620
12.8. Обработка исключений оператора <code>new</code>	621
12.8.1. Генерация исключения <code>bad_alloc</code> оператора <code>new</code>	622
12.8.2. Оператор <code>new</code> , возвращающий <code>nullptr</code> при сбое	623
12.8.3. Обработка исключений оператора <code>new</code> с помощью функции <code>set_new_handler</code>	624
12.9. Иерархия исключений стандартной библиотеки	625
12.10. Альтернатива блоку <code>finally</code> в языке C++: RAII («получение ресурсов есть инициализация»)	628
12.11. Библиотеки, поддерживающие и исключения, и коды ошибок	629
12.12. Журнал ошибок	630
12.13. Основы контрактного программирования	631
12.14. Итоги	640
Глава 13. Контейнеры и итераторы стандартной библиотеки	642
13.1. Введение	643
13.2. Основы контейнеров	645
13.2.1. Общие вложенные типы в последовательных и ассоциативных контейнерах	647
13.2.2. Общие функции контейнеров	648
13.2.3. Требования к элементам контейнера	651
13.3. Работа с итераторами	652
13.3.1. Использование <code>istream_iterator</code> для ввода и <code>ostream_iterator</code> для вывода	652
13.3.2. Категории итераторов	654
13.3.3. Поддерживаемые контейнеры	655
13.3.4. Предопределенные типы итераторов	656
13.3.5. Операторы для работы с итераторами	656
13.4. Несколько слов об алгоритмах для обработки контейнеров	658
13.5. Последовательные контейнеры	658
13.6. Последовательный контейнер <code>vector</code>	659
13.6.1. Использование векторов и итераторов	659
13.6.2. Функции для обработки элементов вектора	664

13.7. Последовательный контейнер <code>list</code>	668
13.8. Последовательный контейнер <code>deque</code>	673
13.9. Ассоциативные контейнеры	675
13.9.1. Ассоциативный контейнер <code>multiset</code>	676
13.9.2. Ассоциативный контейнер <code>set</code>	680
13.9.3. Ассоциативный контейнер <code>multimap</code>	682
13.9.4. Ассоциативный контейнер <code>map</code>	685
13.10. Адаптеры контейнеров	686
13.10.1. Адаптер <code>stack</code>	687
13.10.2. Адаптер <code>queue</code>	689
13.10.3. Адаптер <code>priority_queue</code>	690
13.11. Псевдоконтейнер <code>bitset</code>	692
13.12. Основы нотации «О-большое»	694
13.13. Основы хеш-таблиц	697
13.14. Итоги	700

Глава 14. Алгоритмы стандартной библиотеки, диапазоны и представления C++20

и представления C++20	701
14.1. Введение	702
14.2. Требования алгоритмов: концепты (C++20)	704
14.3. Лямбда-выражения и алгоритмы	706
14.4. Алгоритмы	710
14.4.1. Алгоритмы <code>fill</code> , <code>fill_n</code> , <code>generate</code> и <code>generate_n</code>	710
14.4.2. Алгоритмы <code>equal</code> , <code>mismatch</code> и <code>lexicographical_compare</code>	713
14.4.3. Алгоритмы <code>remove</code> , <code>remove_if</code> , <code>remove_copy</code> и <code>remove_copy_if</code>	716
14.4.4. Алгоритмы <code>replace</code> , <code>replace_if</code> , <code>replace_copy</code> и <code>replace_copy_if</code>	721
14.4.5. Алгоритмы перемешивания, подсчетов, поиска минимального и максимального элементов	723
14.4.6. Алгоритмы поиска и сортировки	727
14.4.7. Алгоритмы <code>swap</code> , <code>iter_swap</code> и <code>swap_ranges</code>	731
14.4.8. Алгоритмы <code>copy_backward</code> , <code>merge</code> , <code>unique</code> , <code>reverse</code> , <code>copy_if</code> и <code>copy_n</code>	734
14.4.9. Алгоритмы <code>inplace_merge</code> , <code>unique_copy</code> и <code>reverse_copy</code>	737
14.4.10. Операции со множествами	739
14.4.11. Алгоритмы <code>lower_bound</code> , <code>upper_bound</code> и <code>equal_range</code>	742
14.4.12. Алгоритмы <code>min</code> , <code>max</code> и <code>minmax</code>	744
14.4.13. Алгоритмы <code>gcd</code> , <code>lcm</code> , <code>iota</code> , <code>reduce</code> и <code>partial_sum</code> (заголовок <code><numeric></code>)	745
14.4.14. Пирамидальная сортировка кучи и приоритетные очереди	749
14.5. Функциональные объекты (функторы)	754

14.6. Проекции	759
14.7. Представления (C++20) и функциональное программирование.....	763
14.7.1. Адаптеры диапазонов	763
14.7.2. Работа с адаптерами диапазонов и представлениями.....	765
14.8. Основы параллельных алгоритмов.....	770
14.9. Краткое описание алгоритмов стандартной библиотеки.....	772
14.10. Диапазоны в C++23	776
14.11. Итоги	777
Глава 15. Шаблоны, концепты (C++20) и метапрограммирование	779
15.1. Введение.....	780
15.2. Пользовательские шаблоны классов и статический полиморфизм.....	783
15.3. Расширения функциональности шаблонов, введенные в C++20	789
15.3.1. Сокращенные шаблоны функций (C++20).....	789
15.3.2. Шаблоны лямбда-выражений (C++20).....	791
15.4. Концепты (C++20): первый взгляд	791
15.4.1. Неограниченный шаблон функции multiply	793
15.4.2. Ограниченный шаблон функции с условием requires (C++20)	796
15.4.3. Концепты, предопределенные в C++20	799
15.5. Признаки типов	801
15.6. Концепты: детальное рассмотрение	806
15.6.1. Создание пользовательского концепта	807
15.6.2. Использование концепта.....	808
15.6.3. Концепты в сокращенных шаблонах функций	809
15.6.4. Перегрузка на основе концептов	810
15.6.5. Выражения requires	813
15.6.6. Демонстрационные концепты	817
15.6.7. Технологии, применявшиеся до появления концептов: SFINAE и отправка тегов.....	818
15.7. Тест-драйв концептов с помощью объявления static_assert.....	819
15.8. Создание пользовательского алгоритма.....	822
15.9. Создание пользовательского контейнера и пользовательских итераторов	824
15.9.1. Шаблон класса ConstIterator.....	827
15.9.2. Шаблон класса Iterator.....	830
15.9.3. Шаблон класса MyArray	832
15.9.4. Дедуктивная инструкция для скобочной инициализации объекта myArray	836
15.9.5. Тест-драйв класса myArray и его пользовательских итераторов с алгоритмами std::ranges.....	838

15.10. Аргументы по умолчанию для параметров-типов шаблона	842
15.11. Шаблоны переменных	842
15.12. Вариативные шаблоны и выражения свертки.....	843
15.12.1. Кортежи: вариативный шаблон класса tuple	843
15.12.2. Вариативные шаблоны функций и основы выражений свертки (C++17).....	847
15.12.3. Типы выражений свертки	851
15.12.4. Применение операторов в унарных выражениях свертки	852
15.12.5. Применение операторов в бинарных выражениях свертки	854
15.12.6. Оператор «запятая» для повторения операций.....	856
15.12.7. Ограничение элементов пакета параметров одним типом.....	857
15.13. Метапрограммирование на основе шаблонов.....	859
15.13.1. Шаблоны C++ обладают полнотой по Тьюрингу	860
15.13.2. Вычисления на этапе компиляции.....	861
15.13.3. Условная компиляция в метапрограммировании на основе шаблонов и инструкции constexpr if	866
15.13.4. Метафункции типа	869
15.14. Итоги	873
Глава 16. Модули [C++20]: технология разработки больших программ	875
16.1. Введение	876
16.2. Компиляция и компоновка до C++20	878
16.3. Преимущества и предназначение модулей	880
16.4. Пример перехода на модули: заголовочные единицы трансляции	881
16.5. Модули могут уменьшать размеры единиц трансляции и время компиляции	884
16.6. Пример: создание и использование модуля.....	886
16.6.1. Объявление модульной единицы	887
16.6.2. Экспорт объявления	889
16.6.3. Экспорт группы объявлений	890
16.6.4. Экспорт пространства имен	890
16.6.5. Экспорт элемента пространства имен	891
16.6.6. Импорт модуля для его использования в экспорте объявлений	891
16.6.7. Пример: попытка доступа к неэкспортированному элементу модуля.....	893
16.7. Глобальный фрагмент модуля.....	896
16.8. Отделение интерфейса от реализации	896
16.8.1. Пример: единицы реализации модуля	897
16.8.2. Пример: класс на основе модулей.....	900
16.8.3. Фрагмент :private	903

16.9. Разделы модуля	904
16.9.1. Пример: разделы интерфейса модуля	904
16.9.2. Разделы реализации модуля	908
16.9.3. Пример: «подмодули» и разделы	908
16.10. Дополнительные примеры работы с модулями	913
16.10.1. Пример: импорт стандартной библиотеки C++ в виде модулей	914
16.10.2. Пример: запрет циклических зависимостей	916
16.10.3. Пример: инструкции <code>import</code> не наследуются	917
16.10.4. Пример: видимость и доступность модулей	918
16.11. Перенос кода в модули	920
16.12. Будущее модулей и инструментов для работы с ними	921
16.13. Итоги	922
Приложение: видео про модули	924
Приложение: статьи про модули	925
Приложение: глоссарий по модулям	928
Глава 17. Параллельные алгоритмы и конкурентность: высокоуровневый подход	931
17.1. Введение	932
17.2. Параллельные алгоритмы стандартной библиотеки (C++17)	935
17.2.1. Пример: сравнение быстродействия последовательного и параллельного алгоритмов сортировки	936
17.2.2. Когда нужны параллельные алгоритмы	940
17.2.3. Политики выполнения	941
17.2.4. Пример: сравнение быстродействия параллельных и векторных операций	941
17.2.5. Дополнительные сведения о параллельных алгоритмах	944
17.3. Многопоточное программирование	945
17.3.1. Состояния потока и жизненный цикл потока	945
17.3.2. Тупики и неопределенно долгие отсрочки	948
17.4. Запуск задач с помощью <code>std::jthread</code>	951
17.4.1. Определение задачи для выполнения в потоке	951
17.4.2. Выполнение задачи в потоке <code>jthread</code>	953
17.4.3. Преимущества класса <code>jthread</code> перед классом <code>thread</code>	956
17.5. Отношения между производителем и потребителем: первый эксперимент	957
17.6. Синхронизация доступа производителя и потребителя к общим изменяемым данным	965
17.6.1. Класс <code>SynchronizedBuffer</code> : мьютексы, блокировки и условные переменные	967
17.6.2. Тестирование класса <code>SynchronizedBuffer</code>	975

17.7. Производитель и потребитель: минимизация ожидания с помощью кольцевого буфера	979
17.8. Читатели и писатели	989
17.9. Кооперативное прерывание потоков класса <code>jthread</code>	991
17.10. Запуск задач с помощью шаблона функции <code>std::async</code>	994
17.11. Однократная потокобезопасная инициализация	1002
17.12. Основы атомарных типов	1004
17.13. Координация потоков с помощью зашелок и барьеров (C++20)	1008
17.13.1. Зашелки класса <code>std::latch</code> (C++20)	1009
17.13.2. Барьеры класса <code>std::barrier</code> (C++20)	1012
17.14. Семафоры (C++20)	1016
17.15. C++23: взгляд в будущее конкурентности	1020
17.15.1. Параллельные алгоритмы <code>std::ranges</code>	1020
17.15.2. Конкурентные контейнеры	1020
17.15.3. Другие материалы, связанные с конкурентностью	1021
17.16. Итоги	1021
Глава 18. Корутины (C++20)	1024
18.1. Введение	1025
18.2. Библиотеки поддержки корутин	1026
18.3. Установка библиотек <code>concurrency</code> и <code>generator</code>	1028
18.4. Создание корутины-генератора с помощью библиотеки <code>generator</code>	1029
18.5. Запуск задач с помощью библиотеки <code>concurrency</code>	1033
18.6. Создание корутины, содержащей <code>co_await</code> и <code>co_return</code>	1039
18.7. Низкоуровневые концепции корутин	1048
18.8. Развитие корутин в C++23	1051
18.9. Итоги	1051
Приложение А. Приоритеты и группировка операторов	1052
Приложение Б. Набор символов	1055

Предисловие

Добро пожаловать в «C++20 для программистов. Метод готовых объектов». В этой книге представлены передовые информационные технологии для разработчиков программного обеспечения. Она соответствует стандарту C++20 (1834 страницы), который был утвержден Комитетом ISO по стандартам C++ в сентябре 2020 года^{1, 2}.

Язык программирования C++ используется для создания высокопроизводительного, критически важного и надежного программного обеспечения. На нем пишут операционные системы, системы реального времени, встраиваемое ПО, видеонигры, банковское ПО, системы управления авиатранспортом, системы связи и многие другие программы. Эта книга представляет собой учебное пособие вводного и среднего уровня, предназначенное для освоения языка C++ (в том числе его стандартной библиотеки) в версии C++20, которая относится к самым популярным в мире языкам программирования. Мы представляем наглядный, современный, насыщенный примерами кода, ориентированный на практические задачи вводный курс C++20. Из этого предисловия вы узнаете о сути нашей книги.

Современный C++

В центре внимания — современный C++, включающий четыре последних стандарта: C++20, C++17, C++14 и C++11. Мы также упоминаем основные дополнения, которые могут появиться в C++23 или в более поздних версиях. Общей темой этой книги является акцент на новых, усовершенствованных способах написания кода на C++. Мы используем лучшие практики, делая акцент на современных идиомах современного профессионального программирования на C++, а также уделяем большое внимание вопросам производительности, безопасности и проектирования программного обеспечения.

¹ Финальный проект стандарта C++ опубликован по адресу <https://timsong-cpp.github.io/cppwp/n4861>. Эта версия бесплатна. Официальную версию в печатном виде (ISO/IEC 14882:2020) можно приобрести по адресу <https://www.iso.org/standard/79358.html>.

² Херб Саттер (Herb Sutter), «C++20 Approved. C++23 Meetings and Schedule Update», 6 сентября 2020 г. Был доступен: 15 сентября 2023 г. <https://herbsutter.com/2020/09/06/c20-approved-c23-meetings-and-schedule-update>.

В курсе событий

«Кто дерзает учить, должен никогда не переставать учиться»¹. (Дж. К. Дана)

Чтобы «почувствовать пульс» современного языка C++, меняющего многие методы работы программистов, мы изучили около 6000 актуальных статей, исследовательских работ, технических описаний, документов, постов в блогах и форумах, а также видеопубликаций.

Кому пригодится эта книга

Книга «C++20 для программистов. Метод готовых объектов» рассчитана на следующие целевые аудитории:

- ♦ разработчики программ на C++, которые хотят изучить новейшие возможности C++20 по подробному учебнику, написанному в профессиональном стиле;
- ♦ разработчики программ на других языках, которые готовятся к выполнению проекта на языке C++ и хотят изучить его актуальную версию;
- ♦ разработчики, которые изучали C++ в вузе или работали с ним много лет назад, а теперь хотят освежить свои знания в контексте C++20;
- ♦ профессиональные преподаватели C++, составляющие курсы по C++20.

Готовые файлы с исходным кодом программ

В основе книги лежит фирменный подход Deitel — реальный код. Вместо отдельных фрагментов кода мы показываем язык C++ таким, каким он должен быть, на сотнях практических примеров готового кода, то есть работающих программ и выводимых ими данных.

Прочитайте раздел «Перед началом работы», который следует за этим предисловием, чтобы узнать, как подготовить свой компьютер с операционной системой Windows, macOS или Linux к выполнению более 200 приведенных в книге примеров кода, в которых около 15 тысяч строк. Для вашего удобства все эти примеры опубликованы на нашем веб-сайте и на портале GitHub в виде стандартных файлов C++ с расширениями .cpp и .h. Вы можете бесплатно загрузить их по любому из адресов²:

- ♦ <https://github.com/pdeitel/cplusplus20forprogrammers>
- ♦ <https://www.deitel.com/books/c-plus-plus-20-for-programmers>

¹ Джон Коттон Дана (John Cotton Dana). «В 1912 г. Дана, библиотекарь из Ньюарка, получил просьбу подобрать латинскую цитату для надписи на новом здании Государственного колледжа Ньюарка (ныне Университет Кин) в Юнионе (штат Нью-Джерси). Не найдя подходящей цитаты, Дана сочинил фразу, ставшую девизом колледжа. The New York Times Book Review, 5 марта 1967 г., с. 55. <https://www.bartleby.com/73/1799.html>.

² А также с сайта издательства «Питер», см. раздел «От издательства». ... *Примеч. ред.*

- ♦ <https://deitel.com/c-plus-plus-20-for-programmers>
- ♦ <https://deitel.com/cpp20fp>

Все эти файлы готовы для компиляции как в интегрированных средах разработки, так и в компиляторах командной строки. Подробные инструкции о том, как компилировать и выполнять примеры кода с помощью трех рекомендуемых нами компиляторов, вы найдете в главе 1, в разделе 1.2. Компилируйте и запускайте соответствующие программы во время чтения книги, чтобы сделать учебный процесс более наглядным. Если у вас возникнут проблемы, вы можете связаться с нами по адресу:

deitel@deitel.com

Три профессиональных компилятора на выбор

Мы протестировали все приведенные в книге примеры кода в последних версиях трех компиляторов:

- ♦ Visual C++® в составе интегрированной среды разработки Microsoft® Visual Studio® Community Edition — в операционной системе Windows®;
- ♦ Clang C++ (clang++) в составе интегрированной среды разработки Apple® Xcode® — в операционной системе macOS®, а также в контейнере Docker®;
- ♦ GNU® C++ (g++) — в операционной системе Linux®, а также в контейнере Docker® с установленным пакетом GNU Compiler Collection (GCC).

Когда мы писали эту книгу, почти все языковые новшества, введенные в стандарт C++20, уже были полностью реализованы во всех трех компиляторах, некоторые — только в одном или двух и лишь немногие — ни в одном из трех. Мы указываем на эти различия при необходимости и будем обновлять наш цифровой контент по мере того, как эти новшества будут внедряться производителями компиляторов. Мы также будем публиковать обновления кода в репозитории этой книги на портале GitHub:

<https://github.com/pdeitel/CPlusPlus20ForProgrammers>

Кроме того, мы будем публиковать обновления кода и текста на веб-странице этой книги:

<https://www.deitel.com/books/c-plus-plus-20-for-programmers>

Учебный метод готовых объектов

В главе 9 мы расскажем о том, как разрабатывать на языке C++20 пользовательские (то есть ваши собственные) классы, а в последующих главах продолжим рассмотрение объектно-ориентированного программирования.

Что такое готовые объекты?

В первых главах вы будете работать с уже существующими классами, выполняющими достаточно сложные задачи. Используя всего несколько простых инструкций на языке C++, вы быстро создадите объекты этих классов и заставите их «показать свою силу». Мы называем это методом готовых объектов.

Учитывая огромное количество бесплатных библиотек классов с открытым исходным кодом (open-source), созданных сообществом разработчиков C++, вы сможете выполнять сложные задачи задолго до того, как научитесь создавать свои собственные классы в главе 9. Это один из наиболее привлекательных аспектов работы с объектно-ориентированными языками, особенно с таким совершенным объектно-ориентированным языком, как C++.

Бесплатные классы

Мы обращаем ваше внимание, что в «экосистеме» C++ есть огромное количество очень полезных и совершенно бесплатных классов, доступных программистам. Основными источниками этих классов являются:

- ♦ стандартная библиотека C++;
- ♦ библиотеки, специфичные для конкретной платформы, например поставляемые с Microsoft Windows, Apple macOS или различными версиями Linux;
- ♦ бесплатные сторонние библиотеки C++, создаваемые сообществом разработчиков открытого программного обеспечения;
- ♦ коллеги-программисты, например сотрудники вашей организации.

Мы рекомендуем вам черпать вдохновение для ваших собственных программ, изучая многочисленные общедоступные программы на языке C++, публикуемые в виде открытого исходного кода на таких сайтах, как GitHub.

Проект Boost

Проект Boost предоставляет программистам 168 библиотек C++ с открытым исходным кодом¹. Кроме того, в нем «выращиваются» многие языковые новшества, которые впоследствии включаются в стандартную библиотеку C++. Например, именно из проекта Boost берут начало уже вошедшие в современный C++ средства многопоточности и генерации случайных чисел, интеллектуальные указатели, кортежи, регулярные выражения, файловые системы и класс `string_view`². На этой странице сайта Stack Overflow приведен список биб-

¹ «Boost 1.78.0 Library Documentation». Был доступен: 15 сентября 2023 г. https://www.boost.org/doc/libs/1_78_0.

² «Boost (C++ libraries)». Был доступен: 15 сентября 2023 г. [https://en.wikipedia.org/wiki/Boost_\(C++_libraries\)](https://en.wikipedia.org/wiki/Boost_(C++_libraries)).

лиотек и языковых средств современного C++, которые эволюционировали из библиотек Boost¹:

<https://stackoverflow.com/a/8852421>

Примеры кода с готовыми объектами

В главе 1 кратко рассматриваются основные понятия и термины объектной технологии. В главах 2–8 вы будете создавать и использовать объекты уже существующих классов (задолго до создания своих собственных классов в главе 9 и далее). Наши примеры кода с готовыми объектами посвящены следующим учебным темам:

- ♦ Раздел 2.7 — создание и использование объектов класса `string` стандартной библиотеки.
- ♦ Раздел 3.12 — работа с очень большими целыми числами.
- ♦ Раздел 4.13 — создание и чтение ZIP-файлов.
- ♦ Раздел 5.20 — *Lnfylun Lhqtomh Wjtz Qarcv: Qjwazkrplm xzz Xndmwewqhlz* (это зашифрованная фраза, речь идет о шифровании с закрытым ключом).
- ♦ Раздел 6.15 — шаблон класса `vector` стандартной библиотеки C++.
- ♦ Раздел 7.10 — доступ к контейнерам с данными с помощью класса `span` (C++20).
- ♦ Раздел 8.19 — чтение и анализ CSV-файла со статистическими данными о катастрофе «Титаника».
- ♦ Раздел 8.20 — основы регулярных выражений.
- ♦ Раздел 9.22 — сериализация данных в формате JSON (JavaScript Object Notation).

Прекрасный пример метода готовых объектов — использование объектов таких стандартных классов C++, как `array` и `vector` (глава 6). Вы можете работать с ними, даже не зная, как они устроены, и вообще не умея писать пользовательские классы. На протяжении всей книги мы используем обширные возможности стандартной библиотеки C++.

Краткое содержание книги

На внутренней стороне обложки книги показана в виде наглядной схемы ее модульная структура. Читая раздел «Краткое содержание книги», сверяйтесь с этой схемой, которая поможет быстрее ознакомиться с темами глав и разделов.

¹ Kennytm, ответ на вопрос «Which Boost Features Overlap with C++11?». Был доступен: 15 сентября 2023 г. <https://stackoverflow.com/a/8852421>.

В первых главах книги закладывается прочный фундамент основ языка C++20. Последующие главы с более сложными практическими примерами кода помогут вам перейти к разработке ваших собственных программ на C++20. На протяжении всей книги рассматриваются пять моделей программирования на C++20:

- ♦ процедурное программирование;
- ♦ функциональное программирование;
- ♦ объектно-ориентированное программирование;
- ♦ обобщенное программирование;
- ♦ метапрограммирование на основе шаблонов.

Часть 1. Быстрый старт и базовые возможности C++

Глава 1 «Введение и тест-драйв популярных бесплатных компиляторов C++» рассказывает о том, что должен знать каждый профессиональный программист. В этой главе вы найдете:

- ♦ краткое введение;
- ♦ обсуждение закона Мура, многоядерных процессоров и необходимости стандартизированного конкурентного программирования в современном C++;
- ♦ краткую справку по объектно-ориентированному программированию с объяснением терминов, которые встретятся вам в последующих главах.

В этой главе вы также узнаете, как компилировать и выполнять написанный на C++ готовый код с помощью трех бесплатных компиляторов, которые мы вам рекомендуем:

- ♦ Visual C++ из Microsoft Visual Studio (в Windows);
- ♦ Apple Xcode (в macOS);
- ♦ GNU g++ (в Linux).

Мы проверили на каждом из них все приведенные в книге примеры кода, прокомментировав те немногие случаи, когда наши версии компиляторов (существующие на момент выхода книги в 2022 году) еще не поддерживают какие-либо из новшеств стандарта C++20. Выберите ту среду разработки программ, которая вам больше нравится. Наши примеры кода будут хорошо работать и с другими компиляторами C++20.

Мы расскажем, как запускать компиляторы командной строки GNU g++ и clang++ в контейнерах Docker, чтобы вы могли работать с самыми современными версиями этих компиляторов в любой из трех операционных систем: Windows, macOS или Linux. Docker — это ценный инструмент разработчика, дополнительные сведения о нем приведены в разделе «Перед началом работы». Кроме того, мы расскажем, как установить Linux на компьютер с Windows, используя подсистему Windows для Linux (WSL). Это еще один способ работы в Windows с компиляторами g++ и clang++.

В главе 2 «Азы программирования на C++» представлены базовые понятия языка C++, такие как ввод и вывод данных, основные типы данных, арифметические операторы и их приоритеты, инструкции принятия решений. В разделе 2.7 вы познакомитесь с методом готовых объектов, легко создавая и используя объекты класса `string` стандартной библиотеки (для этого вам не надо знать, как устроен этот большой и сложный класс и как вообще устроены классы).

Глава 3 «Управляющие инструкции (часть 1)» посвящена некоторым управляющим инструкциям. В ней вы изучите инструкции выбора `if` и `if...else`, инструкции цикла `while` со счетчиком и контрольным значением, а также операторы инкремента, декремента и присваивания. В разделе 3.12 мы покажем очередной пример использования готовых объектов: с помощью общедоступного класса вы будете выполнять вычисления с чрезвычайно большими целыми числами.

Глава 4 «Управляющие инструкции (часть 2)» рассказывает об управляющих инструкциях `for`, `do...while`, `switch`, `break` и `continue`, а также о логических операторах. В разделе 4.13 вы продолжите работать с готовыми объектами, включив библиотеку `miniz-cpp` для записи и чтения ZIP-файлов.

В главе 5 «Функции и шаблоны функций» представлены пользовательские функции. Мы расскажем о методах моделирования на примере генерации случайных чисел. Устаревшая функция генерации случайных чисел `rand`, которую C++ унаследовал от языка C, может быть предсказуемой, поэтому программы с ней ненадежны. Мы рассмотрим более безопасную библиотеку, введенную в C++11, которая генерирует недетерминированные (непредсказуемые) последовательности случайных чисел. Такие генераторы случайных чисел используются для моделирования событий и в сценариях безопасности, где предсказуемость недопустима. Также мы обсудим передачу информации между функциями и рекурсию. Пример с готовыми объектами в разделе 5.20 продемонстрирует шифрование с закрытым ключом.

Часть 2. Массивы, указатели и строки

В главе 6 «Массивы, векторы, библиотека `ranges` и функциональное программирование» мы приступим к рассмотрению контейнеров, итераторов и алгоритмов стандартной библиотеки C++. Мы представим вам контейнер `array`, предназначенный для хранения и обработки списков и таблиц значений. Вы определите массивы класса `array`, инициализируете их и получите доступ к их элементам. Вы узнаете, как передавать эти массивы функциям, выполнять сортировку и поиск в массивах, работать с многомерными массивами. Мы познакомим вас с функциональным программированием, начав с лямбда-выражений (безымянных функций) и диапазонов библиотеки `ranges`, которая входит в «большую четверку» новых возможностей C++20. В разделе 6.15 на примере готовых объектов мы продемонстрируем векторы — контейнеры данных, основанные на шаблоне класса `vector` стандартной библиотеки C++. По сути, вся эта глава представляет собой большой урок работы с готовыми объектами — массивами и векторами. Примеры исходного кода в этой главе хорошо иллюстрируют современные идиомы программирования на C++.

В главе 7 «Указатели в современном C++ (фактор риска)» подробно рассматриваются указатели и объясняется тесная взаимосвязь между указателями, массивами на основе указателей и строками на основе указателей (которые также называют строками в стиле C). Все эти традиционные элементы C++ унаследовал от языка C. Указатели являются мощным инструментом программиста, но с ними сложно работать и они часто становятся причиной ошибок в программах. Поэтому мы покажем вам современные альтернативы указателям, которые сделают ваш код более надежным и безопасным: классы `array` и `vector`, класс `span` (C++20) и класс `string_view` (C++17). Но вы должны уметь работать и с традиционными указателями, чтобы читать код старых программ. Мы расскажем о тех немногих случаях, когда указатели нужны даже в новых программах. Во всех остальных случаях мы рекомендуем использовать вместо указателей современные языковые средства. К ним относится класс `span`, с объектами которого вы будете работать в разделе 7.10. С помощью этого класса вы сможете, не используя указатели, читать и обрабатывать элементы традиционных массивов на основе указателей, а также современных контейнеров `array` и `vector`. В этой главе мы продолжим акцентировать внимание на современных идиомах программирования на C++.

В главе 8 «Строки и их представления, текстовые файлы, CSV-файлы и регулярные выражения» вы узнаете о больших возможностях класса `string` стандартной библиотеки. Затем мы расскажем, как выполнять операции чтения и записи с обычными текстовыми файлами и с файлами в формате CSV, которые часто применяются для работы с наборами данных. Мы также расскажем об операциях со строками с помощью поддерживаемых стандартной библиотекой регулярных выражений. В C++ есть два типа строк: объекты класса `string` и традиционные строки на основе указателей. Мы используем объекты класса `string`, чтобы сделать программы более надежными и избежать многих проблем безопасности, свойственных традиционным строкам. В новых разработках надо отдавать предпочтение объектам класса `string`. Вы узнаете о классе `string_view` (C++17) — простом и универсальном средстве передачи строк любого типа функциям. В конце главы вы продолжите работать с готовыми объектами:

- ♦ Раздел 8.19 посвящен анализу данных. Вы прочитаете и проанализируете CSV-файл, содержащий набор данных «Титаник», — один из популярных учебных инструментов для начинающих аналитиков.
- ♦ Раздел 8.20 посвящен регулярным выражениям, которые нужны для поиска и замены фрагментов текста.

Часть 3. Объектно-ориентированное программирование

В главе 9 «Пользовательские классы» вы начнете изучать объектно-ориентированное программирование и узнаете, как создавать свои собственные классы. Язык C++ является расширяемым: каждый созданный вами класс становится новым типом данных, после чего вы можете создавать любые объекты этого типа. В разделе 9.22 вы найдете очередной пример работы с готовыми объектами: сторонняя библиотека `cereal` позволит вам легко выполнять сериализацию (преобра-

зование объектов в формат JavaScript Object Notation — JSON) и десериализацию (обратное преобразование из формата JSON).

Глава 10 «ООП: наследование и динамический полиморфизм» посвящена отношениям между классами в иерархии наследования и мощным средствам динамического полиморфизма, обеспечивающим эти отношения. Из этой главы вы узнаете об основных принципах полиморфизма. Один из самых важных моментов этой главы — подробная схема, объясняющая, как полиморфизм, виртуальные функции и динамическое связывание реализованы «под капотом» языка C++. Вы увидите, что для этого используется элегантная структура данных на основе указателей. Мы расскажем и о других средствах полиморфизма, среди которых идиома неvirtуального интерфейса (NVI), а также `std::variant` и `std::visit`. Вы узнаете о преимуществах наследования интерфейса перед наследованием реализации.

В главе 11 «Перегрузка операторов, семантика копирования и перемещения, умные указатели» вы узнаете, как использовать стандартные операторы языка C++ для операций с объектами пользовательских (ваших собственных) классов. Вы также познакомитесь с умными указателями и с динамическим управлением памятью. Умные указатели позволяют избежать ошибок динамического управления памятью, предоставляя программисту больше возможностей, чем традиционные указатели. Мы обсудим умные указатели класса `unique_ptr`. Ключевая тема этой главы — создание полноценных высококачественных классов. Мы начнем с тест-драйва класса `string`, который является красивым примером перегрузки операторов. Затем мы перейдем к одному из важнейших примеров книги: вы узнаете, как написать пользовательский класс контейнеров `MyArray`, используя перегруженные операторы и другие языковые средства таким образом, чтобы в вашем классе не было проблем, свойственных массивам на основе указателей¹. Мы представим вам и реализуем пять специальных функций, которые можно определять в каждом классе: копирующий конструктор, оператор копирующего присваивания, перемещающий конструктор, оператор перемещающего присваивания и деструктор. Мы обсудим семантику копирования, а также семантику перемещения, которая позволяет компилятору быстро перемещать ресурсы из одного объекта в другой, избегая бесполезных и ресурсоемких операций копирования. Мы расскажем о новом операторе трехстороннего сравнения (`<=>`) и о том, как реализовать пользовательские конвертирующие операторы. В главе 15 вы усовершенствуете класс `MyArray`, преобразовав его в шаблон класса, позволяющий хранить данные разных типов. Вы создадите по-настоящему полезные классы.

Глава 12 «Исключения и обзор контрактного программирования» продолжает тему обработки исключений, начатую в главе 6. Мы объясним, когда нужны исключения и какие бывают гарантии безопасности исключений, рассмотрим

¹ В программах для промышленного применения вы должны использовать для контейнеров классы стандартной библиотеки, но этот учебный пример позволяет продемонстрировать многие ключевые концепции современного C++.

исключения в контексте конструкторов и деструкторов, расскажем об обработке ошибок динамического выделения памяти и о том, почему в некоторых проектах исключения запрещены. Глава завершается введением в так называемые контракты, которые могут появиться в будущих версиях C++; мы расскажем о контрактах на примере их экспериментальной реализации, опубликованной на сайте godbolt.org. Благодаря контрактам программисты смогут отказаться от большинства исключений (то есть объявлять большинство функций со спецификатором `noexcept`), что позволит компилятору оптимизировать исполняемый код и избежать сложных ресурсоемких операций, связанных с обработкой исключений.

Часть 4. Контейнеры, итераторы и алгоритмы стандартной библиотеки

В главе 13 «Контейнеры и итераторы стандартной библиотеки» начинается более широкое и глубокое рассмотрение трех ключевых компонентов стандартной библиотеки языка C++:

- ♦ контейнеров (основанных на шаблонах структур данных);
- ♦ итераторов (необходимых для доступа к элементам контейнеров);
- ♦ алгоритмов (использующих итераторы для обработки контейнеров).

Мы обсудим контейнеры, а также адаптеры контейнеров и псевдоконтейнеры. Вы увидите, что стандартная библиотека C++ предоставляет вам готовые структуры данных, широко применяемые программистами, поэтому вам не нужно создавать свои собственные. Почти все связанные со структурами данных задачи, которые вам могут встретиться, вы сможете решить средствами стандартной библиотеки. Мы продемонстрируем большинство контейнеров стандартной библиотеки и объясним, как применять алгоритмы к различным типам контейнеров, используя итераторы. Вы увидите, что разные контейнеры поддерживают разные типы итераторов. Мы продолжим показывать те ценные возможности упрощения кода, которые дает программисту введенная в C++20 библиотека `ranges`.

В главе 14 «Алгоритмы стандартной библиотеки, диапазоны и представления C++20» представлены многие из 115 алгоритмов стандартной библиотеки. В центре внимания — типичные операции с контейнерами, в том числе заполнение контейнеров значениями, генерация значений, сравнение элементов и целых контейнеров, удаление элементов, замена элементов, математические операции, поиск, сортировка, обмен значениями, копирование, объединение, операции со множествами, определение границ контейнеров, нахождение наименьших и наибольших значений. Мы обсудим минимальные требования к итераторам, чтобы вы знали, какие контейнеры совместимы с тем или иным алгоритмом. Мы начинаем обсуждение концептов, входящих в «большую четверку» новых возможностей C++20. Алгоритмы из пространства имен `std::ranges` (C++20) используют концепты, чтобы проверять корректность получаемых аргументов. Мы продолжим обсуждение функционального программирования на примере диапазонов и представлений C++20.

Часть 5. Более сложные темы

В главе 15 «Шаблоны, концепты (C++20) и метапрограммирование» рассматривается обобщенное программирование на основе шаблонов, которое применяется в C++ с момента выхода стандарта 1998 года. Роль шаблонов возросла с каждым новым выпуском C++. Для современного C++ характерен перенос значительной части ресурсоемких операций на этап компиляции, что позволяет компилятору более надежно контролировать типы данных и повышает быстродействие во время выполнения программ. Как вы увидите, шаблоны и особенно метапрограммирование на основе шаблонов — это важнейшие средства для переноса многих операций с этапа выполнения на этап компиляции. В этой главе мы более подробно рассмотрим шаблоны, объясним, как разрабатывать ваши собственные шаблоны классов, и детально рассмотрим введенные в C++20 концепты. Вы создадите свои собственные концепты, преобразуете созданный в главе 11 класс `MyArray` в шаблон класса, имеющий итераторы, а также поработаете с вариативными шаблонами, которые могут принимать любое количество аргументов. Мы познакомим вас с метапрограммированием на основе шаблонов.

Глава 16 «Модули C++20: технология разработки больших программ» представляет следующую из «большой четверки» новых возможностей C++20. Модули — это новый способ структурирования кода, позволяющий вам решать, какие объявления надо открыть для клиентского кода, и предотвратить доступ извне ко всем остальным деталям реализации классов. Модули помогают программистам работать эффективнее, особенно при создании, сопровождении и модернизации больших информационных систем. Модули помогают быстрее создавать такие системы и делают их более масштабируемыми. По словам создателя C++ Бьерна Страуструпа, «модули дают языку C++ историческую возможность улучшить чистоту кода и время компиляции (переноса C++ в XXI век)». Вы увидите, что даже при разработке небольших систем модули немедленно дают преимущества, устраняя необходимость в препроцессоре C++. Мы бы с удовольствием интегрировали модули в наши программы, но на данный момент в наших основных компиляторах еще не реализована полноценная поддержка модулей.

Глава 17 «Параллельные алгоритмы и конкурентность: высокоуровневый подход» — одна из самых важных в книге. В ней представлены имеющиеся в C++ средства создания многопоточных приложений. Распределение нескольких задач между потоками процессора может значительно повышать быстродействие и скорость реакции программы. Мы показываем, как использовать готовые параллельные алгоритмы (C++17) для создания многопоточных программ, которые будут работать быстрее (как правило, намного быстрее) на современных компьютерах с многоядерными процессорами. Например, мы сортируем 100 миллионов чисел сначала последовательным, а затем параллельным алгоритмом, измеряя с помощью библиотеки `<chrono>` прирост производительности параллельного алгоритма на 4-ядерном и 8-ядерном процессорах. Вы увидите, что на компьютере с 64-битной версией Windows 10 и 8-ядерным процессором Intel параллельная сортировка может выполняться почти в 7 раз быстрее, чем последовательная. Мы обсуждаем взаимодействие потока-производителя и пото-

ка-потребителя, демонстрируя различные способы их синхронизации с помощью низкоуровневых и высокоуровневых средств конкурентного программирования, в том числе зашелок, барьеров и семафоров, введенных в C++20. Мы объясняем, в чем заключается сложность конкурентного программирования и почему при любой возможности надо отдавать предпочтение высокоуровневым средствам конкурентности. Такие низкоуровневые средства, как семафоры и атомарные типы, имеет смысл применять для реализации средств более высокого уровня, таких как зашелки.

В главе 18 «Корутины (C++20)» представлены корутины — последняя из «большой четверки» новых возможностей C++20. Корутина — это функция, которая может приостановить свое выполнение, а впоследствии возобновить его по инструкции из другой части программы. Все механизмы реализации такого поведения корутины полностью реализуются компилятором без вашего участия. Вы увидите, что корутиной является любая функция, содержащая хотя бы одно из ключевых слов `co_await`, `co_yield` или `co_return`, и такие функции дают вам возможность писать программы с конкурентными задачами, сохраняя простой последовательный стиль кодирования. Для поддержки корутин нужна сложная инфраструктура, которую вы теоретически могли бы написать самостоятельно, но это сложно, утомительно и чревато ошибками. Большинство экспертов сходятся во мнении, что надо использовать готовые высокоуровневые библиотеки поддержки корутин, и мы демонстрируем именно такой подход. В сообществе разработчиков открытого программного обеспечения уже созданы несколько экспериментальных библиотек для быстрой и удобной работы с корутинами. Две из них мы используем в примерах кода этой главы. Вероятно, поддержка корутин будет добавлена в стандартную библиотеку C++23.

Приложения

Приложение А «Приоритеты и группировка операторов» содержит список стандартных операторов языка C++, отсортированных по их приоритетам. В первый раздел списка включены наиболее приоритетные операторы (выполняемые первыми), в каждый последующий раздел — операторы с последовательно убывающими приоритетами.

Приложение Б «Набор символов ASCII» содержит список стандартных символов и соответствующих им чисел.

Дополнительные материалы на сайте deitel.com

Веб-страница этой книги расположена на сайте deitel.com по адресу:

<https://deitel.com/c-plus-plus-20-for-programmers>

На этой странице вы найдете следующие дополнительные материалы:

- ♦ ссылки на наш репозиторий на портале GitHub, содержащий все примеры исходного кода из этой книги в виде загружаемых файлов;

- ♦ посты в блоге: <https://deitel.com/blog>;
- ♦ обновления к книге.

Рекомендации по загрузке файлов и по настройке вашей среды разработки программ на C++ приведены в разделе «Перед началом работы».

C++ Core Guidelines

C++ Core Guidelines — это сборник рекомендаций объемом около 500 печатных страниц, «призванный помочь людям эффективно использовать современный язык C++»¹. Он написан под редакцией Бьерна Страуструпа (создателя языка C++) и Херба Саттера (председателя комитета ISO по стандартам C++):

<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines>

Авторы C++ Core Guidelines указывают:

Эти рекомендации сосредоточены на относительно высокоуровневых проблемах, таких как интерфейсы, управление ресурсами, управление памятью и конкурентность. Эти правила влияют на архитектуру приложений и на проектирование библиотек. Результатом выполнения этих правил будет статически типобезопасный код, не вызывающий утечек ресурсов, в котором будет исправлено значительно больше логических ошибок программирования, чем обычно бывает в современном коде. И он будет работать быстро — вы сможете позволить себе делать все правильно².

На протяжении всей книги мы используем соответствующие рекомендации из C++ Core Guidelines. Вы обязательно должны уделять пристальное внимание этим мудрым советам. В сборник включены сотни основных рекомендаций, разделенных на десятки категорий и подкатегорий. Их выполнение может показаться вам крайне трудным, но с помощью статических анализаторов кода (см. далее) вы легко проверите любой свой код на соответствие C++ Core Guidelines.

Библиотека Guidelines Support Library (GSL)

В C++ Core Guidelines часто упоминается библиотека Guidelines Support Library (GSL), в которой реализованы вспомогательные классы и функции для поддержки различных рекомендаций³. Microsoft предоставляет свою реализацию GSL с открытым исходным кодом на портале GitHub по адресу:

<https://github.com/Microsoft/GSL>

¹ C++ Core Guidelines, «Abstract». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#S-abstract>.

² C++ Core Guidelines, «Abstract».

³ C++ Core Guidelines, «GSL: Guidelines Support Library». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#S-gsl>.

Мы используем GSL в нескольких примерах кода из первой половины книги. Некоторые средства из GSL впоследствии были включены в стандартную библиотеку C++.

Профессиональные статические анализаторы кода

Эти средства статического анализа кода позволяют быстро проверить ваш код на наличие типичных ошибок и проблем безопасности, а также сообщают о том, как его улучшить. Использование анализаторов дает почти такой же эффект, как проверка кода экспертами мирового класса. Чтобы всегда следовать рекомендациям C++ Core Guidelines и улучшить приведенные в книге примеры кода, мы использовали следующие статические анализаторы:

- ♦ `clang-tidy` — <https://clang.llvm.org/extra/clang-tidy>;
- ♦ `cppcheck` — <https://cppcheck.sourceforge.io>;
- ♦ средства проверки кода на соответствие C++ Core Guidelines, встроенные в статический анализатор кода Microsoft Visual Studio.

С помощью этих трех инструментов мы проверили:

- ♦ соответствие рекомендациям C++ Core Guidelines;
- ♦ соответствие стандартам кодирования;
- ♦ соответствие современным идиомам C++;
- ♦ отсутствие проблем с безопасностью;
- ♦ отсутствие типичных ошибок;
- ♦ отсутствие проблем с быстродействием;
- ♦ наглядность кода

и многое другое.

Кроме того, мы использовали флаг `-Wall` в командах компиляторов GNU g++ и Clang C++, чтобы включить все категории выдаваемых ими предупреждений, сообщающих о возможных проблемах. Мы добились того, что наши программы компилируются без таких сообщений (кроме редких случаев, выходящих за рамки тем этой книги). Информация о настройке средств статического анализа кода приведена в разделе «Перед началом работы».

Интернет-ресурсы для программистов

Stack Overflow

Stack Overflow — один из популярнейших сайтов вопросов и ответов, предназначенный для разработчиков программного обеспечения. Многие проблемы,

с которыми сталкиваются программисты, уже обсуждались на Stack Overflow, поэтому это отличное место, чтобы искать решения таких проблем и задавать новые вопросы. В ходе работы над книгой многие из наших поисковых запросов в Google на различные темы, зачастую сложные, приводили среди первых результатов ответы на Stack Overflow.

GitHub

Лучший способ подготовиться [к работе программиста] — это писать программы и изучать замечательные программы, написанные другими людьми. В моем случае я ходил к мусорным корзинам в Центре компьютерных наук и вылавливал из них листинги операционных систем¹. (Б. Гейтс)

GitHub — это отличная площадка для поиска бесплатного и открытого исходного кода, который может пригодиться вам в ваших проектах. Если хотите, вы можете публиковать на GitHub исходный код написанных вами программ, участвуя в деятельности всемирного сообщества разработчиков открытого программного обеспечения. Пятьдесят миллионов программистов используют GitHub². На сайте размещено более 200 миллионов репозиториях исходного кода на огромном количестве языков программирования³. В 2021 году авторы опубликовали свои новые работы более чем в 61 миллионе репозиториях⁴. GitHub является важнейшим элементом арсенала профессионального разработчика программного обеспечения и содержит средства контроля версий, которые помогают командам разработчиков управлять проектами с открытым кодом и частными проектами.

Сообщество разработчиков C++ с открытым кодом очень велико. На GitHub есть более 41 тысячи⁵ репозиториях исходного кода на C++. На GitHub вы можете исследовать написанный другими код и при желании компилировать его. Это отличный способ изучения программирования, который является естественным продолжением нашего учебного подхода на основе реального кода⁶.

В 2018 году компания Microsoft приобрела GitHub за 7.5 миллиарда долларов. Если вы профессиональный программист, вы почти наверняка регулярно пользуетесь GitHub. По словам генерального директора Microsoft Сатьи Наделлы, компания приобрела GitHub, чтобы «расширить для каждого разработчика

¹ Билл Гейтс (Bill Gates), цитата из работы Сьюзан Ламмерс (Susan Lammers) «Interviews with 19 Programmers Who Shaped the Computer Industry». Microsoft Press, 1986, с. 83.

² «GitHub». Был доступен: 15 сентября 2023 г. <https://github.com>.

³ «Where the World Builds Software». Был доступен: 15 сентября 2023 г. <https://github.com/about>.

⁴ «The 2021 State of the Octoverse». Был доступен: 15 сентября 2023 г. <https://octoverse.github.com>.

⁵ «C++». Был доступен: 15 сентября 2023 г. <https://github.com/topics/cpp>.

⁶ Перед использованием публикуемых на GitHub программ с открытым исходным кодом вам необходимо ознакомиться с разнообразными лицензиями на такие программы.

возможности творчества, внедрения инноваций и решения самых актуальных мировых проблем»¹.

Мы призываем вас изучать и компилировать многочисленные программы с портала GitHub, а также вносить свой вклад в его работу.

Docker

Мы используем Docker — инструмент для упаковки программ в контейнеры, содержащие все необходимое для того, чтобы было удобно работать с этими программами на компьютерах с разными операционными системами. Некоторые программы и программные пакеты трудно устанавливать и настраивать. Но многие из них можно бесплатно загрузить в виде готовых контейнеров Docker, чтобы избежать сложных проблем с установкой. Сразу после загрузки вы можете запускать их на обычных настольных компьютерах или ноутбуках, что делает Docker отличным средством для быстрого, удобного и экономичного изучения новых технологий.

Мы покажем, как устанавливать и запускать полностью настроенные контейнеры Docker, содержащие:

- ♦ пакет компиляторов GNU Compiler Collection (GCC), в состав которого входит компилятор g++;
- ♦ последнюю версию компилятора clang++ от команды разработчиков Clang.

Эти контейнеры работают в Docker на компьютерах с Windows, macOS и Linux.

Docker способствует воспроизводимости результатов. В контейнерах Docker вы можете сохранять ваши настроенные конфигурации со всеми программами и библиотеками, которые используете. Это позволит другим программистам полностью воссоздавать вашу рабочую среду, а затем точно повторять вашу работу. И вы тоже сможете таким образом стабильно воспроизводить ваши собственные результаты. Воспроизводимость особенно важна в науке и в медицине, например, когда исследователи хотят подтвердить и расширить результаты работ, изложенные в опубликованных статьях.

Некоторые ключевые документы и ресурсы по C++

В книгу включены сотни ссылок на видеоролики, записи в блогах, статьи и онлайн-справочники, которые мы изучали, работая над книгой. Возможно, вы захотите обратиться к некоторым из этих ресурсов для изучения более сложных языковых средств и идиом. Основным сборником документации по C++ фактически является сайт cppreference.com. Мы часто ссылаемся на него, чтобы у вас была подробная справочная информация о стандартных классах и функциях C++, рассматрива-

¹ «Microsoft to Acquire GitHub for \$7.5 Billion». Был доступен: 15 сентября 2023 г. <https://news.microsoft.com>.

емых в книге. Кроме того, мы часто ссылаемся на финальный проект стандарта C++20, который доступен бесплатно на GitHub по адресу:

<https://timsong-cpp.github.io/cppwp/n4861>

Ниже мы приводим список ресурсов по C++, которые могут быть вам полезны в процессе изучения нашей книги.

Документация

- ♦ Финальный проект стандарта C++20, принятый Комитетом по стандартам C++:

<https://timsong-cpp.github.io/cppwp/n4861>

- ♦ Документация по C++:

<https://cppreference.com>

- ♦ Документация по C++ от Microsoft:

<https://docs.microsoft.com/en-us/cpp/cpp>

- ♦ Документация по стандартной библиотеке GNU C++:

<https://gcc.gnu.org/onlinedocs/libstdc++/manual/index.html>

Блоги

- ♦ Блог Херба Саттера (Herb Sutter) «Sutter's Mill» о разработке программного обеспечения:

<https://herbsutter.com>

- ♦ Блог команды Microsoft по C++:

<https://devblogs.microsoft.com/cppblog>

- ♦ Блог Мариуса Банчилю (Marius Bancila):

<https://mariusbancila.ro/blog>

- ♦ Блог Джонатана Боккары (Jonathan Boccara):

<https://www.fluentcpp.com>

- ♦ Блог Бартомея Филипека (Bartłomiej Filippek):

<https://www.cppstories.com>

- ♦ Блог Райнера Гримма (Rainer Grimm):

<http://modernescpp.com>

- ♦ Блог Артура О'Двайра (Arthur O'Dwyer):

<https://quuxplusone.github.io/blog>

Дополнительные ресурсы

- ♦ Сайт Бьерна Страуструпа:
<https://stroustrup.com>
- ♦ Сайт организации Standard C++ Foundation:
<https://isocpp.org>
- ♦ Сайт Комитета по стандартам C++:
<http://www.open-std.org/jtc1/sc22/wg21>

Сайты, где ответят на ваши вопросы

К числу самых популярных в мире интернет-форумов по языку C++ (и по программированию вообще) относятся:

- ♦ <https://stackoverflow.com>
- ♦ <https://www.reddit.com/r/cpp>
- ♦ <https://groups.google.com/g/comp.lang.c++>
- ♦ <https://www.dreamincode.net/forums/forum/15-c-and-c>

Здесь вы найдете адреса еще нескольких полезных сайтов:

<https://www.geeksforgeeks.org/stuck-in-programming-get-the-solution-from-these-10-best-websites>

Кроме того, поставщики программного обеспечения часто создают интернет-форумы для поддержки своих инструментов и библиотек. Многие библиотеки координируются и поддерживаются на GitHub. На страницах некоторых библиотек на GitHub вы можете обращаться к их разработчикам за поддержкой через вкладку Issues.

Общение с авторами

Если при чтении книги у вас возникнут вопросы, вы можете связаться с нами по адресу:

deitel@deitel.com

Мы оперативно ответим.

Благодарности

Мы благодарим Барбару Дейтел (Barbara Deitel) за долгие часы, которые она посвятила поиску информации в интернете во время нашей работы над этим проектом. Нам посчастливилось работать с преданной своему делу командой про-

фессионалов издательского дела из компании Pearson. Мы высоко ценим усилия и 27-летнее наставничество нашего друга и коллеги Марка Л. Тауба (Mark L. Taub), вице-президента Pearson IT Professional Group. Марк и его команда издают наши профессиональные книги и видеопродукцию LiveLessons, а также спонсируют наши обучающие вебинары, предлагаемые через сервис O'Reilly Online Learning:

<https://learning.oreilly.com>

Чарви Арора (Charvi Arora) искала рецензентов и руководила процессом рецензирования. Джули Нахиль (Julie Nahil) руководила выпуском книги. Чутти Прасерсит (Chuti Prasertsith) разработала дизайн обложки.

Рецензенты

В этом проекте нам посчастливилось получить рецензии на рукопись от десяти известных специалистов. Большинство рецензентов входят в состав Комитета ISO по стандартам C++, работали в нем или имеют с ним рабочие отношения. Многие из них внесли свой вклад в развитие языка. Они помогли нам сделать книгу лучше, а все оставшиеся в ней недостатки — наши собственные.

- ♦ Андреас Фертиг (Andreas Fertig), независимый преподаватель и консультант по C++, создатель сайта cppin-sights.io, автор книги «Programming with C++20».
- ♦ Марк Грегуар (Marc Gregoire), архитектор программного обеспечения, Nikon Metrology, эксперт класса Microsoft Visual C++ MVP, автор книги «Professional C++» (5-е издание, обновленное с учетом C++20).
- ♦ Д-р Дейзи Холлман (Dr. Daisy Hollman), член комитета ISO по стандартам C++.
- ♦ Дэнни Калев (Danny Kalev), доктор философии, сертифицированный системный аналитик и инженер-программист, бывший член комитета ISO по стандартам C++.
- ♦ Дитмар Куль (Dietmar Kühl), старший разработчик программного обеспечения, Bloomberg L.P., член комитета ISO по стандартам C++.
- ♦ Инбал Леви (Inbal Levi), Solar Edge Technologies, директор ISO C++ Foundation, председатель группы ISO C++ SG9 (Ranges), член комитета ISO по стандартам C++.
- ♦ Артур О'Двайр (Arthur O'Dwyer), преподаватель C++, председатель трека CppCon «Back to Basics», автор нескольких принятых предложений по развитию языка в версиях C++17, C++20 и C++23, автор книги «Mastering the C++17 STL».
- ♦ Саар Раз (Saar Raz), старший инженер-программист, Swimm.io, разработчик концептов C++20 в команде Clang.
- ♦ Хосе Антонио Гонсалес Секо (José Antonio González Seco), член парламента Андалусии.

- ♦ Энтони Уильямс (Anthony Williams), член комиссии по стандартам C++ Британского института стандартов, директор компании Just Software Solutions Ltd., автор книги «C++ Concurrency in Action» (2-е издание). Энтони является автором или соавтором многих документов Комитета по стандартам C++, которые привели к появлению стандартных средств конкурентности в C++.

Артур О'Двайр

Мы должны отметить исключительные усилия, которые Артур О'Двайр приложил к рецензированию нашей рукописи. Из его комментариев мы узнали много нового о тонкостях языка C++ и особенно о современных идиомах программирования на C++. Он не только добавил детальные комментарии во все PDF-файлы с главами книги, которые мы ему отправляли, но и предоставил нам отдельный документ с подробным объяснением своих комментариев, во многих случаях переписал код и указал на внешние ресурсы с ценной информацией. Учитывая все замечания рецензентов, мы всегда с нетерпением ждали, что скажет Артур, особенно по самым сложным вопросам. Он занятой эксперт, но щедро тратил свое время и всегда был конструктивен. Он настаивал на том, чтобы мы «сделали все правильно», и прилагал все усилия, чтобы помочь нам в этом. Артур преподает C++ профессиональным программистам. Он научил нас многим приемам правильной работы с C++.

GitHub

Благодарим GitHub за то, что он позволяет нам легко делиться своим кодом и поддерживать его в актуальном состоянии, а также за предоставление инструментов, которые позволяют десяткам миллионов разработчиков вносить свой вклад в сотни миллионов репозиториях исходного кода¹. Эти инструменты помогают всемирному сообществу разработчиков открытого программного обеспечения, предоставляющему библиотеки для популярных языков программирования, упростить создание мощных приложений и избавить коллег от необходимости «изобретать велосипед».

Мэтт Годболт и Compiler Explorer

Благодарим Мэтта Годболта (Matt Godbolt), создателя онлайн-сервиса Compiler Explorer (<https://godbolt.org>), который позволяет компилировать и запускать программы на многих языках программирования. Благодаря этому сервису вы можете протестировать свой код:

- ♦ на большинстве популярных компиляторов C++, включая три рекомендуемых нами компилятора;

¹ «Where the World Builds Software». Был доступен: 15 сентября 2023 г. <https://github.com/about>.

- ♦ на многих выпущенных, разрабатываемых и экспериментальных версиях компиляторов.

Например, мы использовали имеющуюся в *Compiler Explorer* экспериментальную версию компилятора *g++*, чтобы продемонстрировать в главе 12 предварительную функциональность контрактов, которые надеемся увидеть в стандартной библиотеке одной из будущих версий *C++*. Несколько наших рецензентов использовали сайт godbolt.org, чтобы показывать нам предлагаемые изменения в коде; это помогло нам улучшить книгу.

Дитмар Куль

Благодарим Дитмара Куля (Dietmar Kühl), старшего разработчика программного обеспечения в компании *Bloomberg L.P.* и члена комитета *ISO* по стандартам *C++*, который поделился с нами своими взглядами на наследование, статический и динамический полиморфизм. Он помог нам точнее изложить эти темы в главах 10 и 15.

Райнер Гримм

Благодарим Райнера Гримма (Rainer Grimm), одного из самых популярных блогеров сообщества *C++* (<http://modernescpp.com>). По мере углубления в темы *C++20* поиск в *Google* часто приводил нас к его статьям. Райнер Гримм — профессиональный преподаватель *C++*, читающий курсы на немецком и английском языках. Он написал несколько книг по *C++*, в том числе «*C++20: Get the Details*», «*Concurrency with Modern C++*», «*The C++ Standard Library*» (3-е изд.) и «*C++ Core Guidelines Explained*». Райнер ведет блог о тех новых возможностях языка, которые ожидаются в *C++23*.

Брайан Гетц

Нам повезло, что рецензентом одной из наших книг — «*Java How to Program*» (10-е издание) — был Брайан Гетц (Brian Goetz), архитектор языка *Oracle Java* и соавтор книги «*Java Concurrency in Practice*». Он предоставил нам множество идей и конструктивных комментариев, особенно по следующим темам:

- ♦ проектирование иерархии наследования (это помогло нам в работе над примерами исходного кода в главе 10 «ООП: наследование и динамический полиморфизм»);
- ♦ конкурентность в *Java* (это помогло нам в работе над главой 17 «Параллельные алгоритмы и конкурентность: высокоуровневый подход»).

Разработчики программ с открытым исходным кодом и блогеры

Мы благодарны технически грамотным людям со всего мира, которые участвуют в движении открытого программного обеспечения и ведут блоги, а также организа-

циям, которые помогают распространению открытого программного обеспечения и информации о нем.

Поисковая система Google

Спасибо Google, чья поисковая система отвечает на наш постоянный поток запросов, выполняя каждый из них за доли секунды, в любое время дня и ночи, причем совершенно бесплатно. Это лучший инструмент повышения производительности труда, который мы добавили в наш исследовательский процесс за последние 20 лет!

Grammarly

Теперь мы используем платную версию Grammarly для проверки всех наших рукописей. Разработчики этого сервиса отмечают, что он помогает «писать ярко, понятно, без ошибок», пользуясь «текстовым помощником на основе искусственного интеллекта»¹. Они также утверждают: «Используя различные инновационные подходы, в том числе передовое машинное обучение и глубокое обучение, мы постоянно открываем новые возможности в области обработки естественного языка (NLP), чтобы предоставлять непревзойденную помощь»².

Grammarly предлагает бесплатные инструменты, которые можно интегрировать в несколько популярных веб-браузеров, а также в Microsoft® Office365™ и в Google Docs™. В дополнение к этому предлагаются более мощные премиум-инструменты и версия для бизнеса. Здесь вы можете узнать о разных вариантах бесплатного и платного использования сервиса:

<https://www.grammarly.com/plans>

Мы будем благодарны за ваши замечания, критику, исправления и предложения по улучшению книги, если таковые появятся у вас при чтении книги. Вся корреспонденцию, включая вопросы, мы просим направлять по адресу:

deitel@deitel.com

Мы оперативно ответим.

Мы приглашаем вас в захватывающий мир программирования на C++20. За последние 30 лет мы с удовольствием работали над 11 изданиями наших академических и профессиональных книг по C++. Мы надеемся, что книга «C++20 для программистов. Метод готовых объектов» вам понравится и вы получите познавательный, серьезный и интересный опыт разработки программ на современном C++.

Пол Дейтел, Харви Дейтел

¹ «Grammarly». Был доступен: 15 сентября 2023 г. <https://www.grammarly.com>.

² «Our Mission». Был доступен: 15 сентября 2023 г. <https://www.grammarly.com/about>.

Об авторах

Пол Дж. Дейтел (Paul J. Deitel), генеральный директор и технический директор компании Deitel & Associates, Inc., — выпускник Массачусетского технологического института, 42 года работающий в компьютерной сфере. Пол — один из самых опытных в мире преподавателей языков программирования, он с 1992 года ведет профессиональные курсы для разработчиков программного обеспечения. Прочитал сотни курсов по программированию для академических, промышленных, правительственных и военных организаций, которые являются клиентами Deitel & Associates. Среди них Калифорнийский университет, Cisco, IBM, Siemens, Sun Microsystems (теперь Oracle), Dell, Fidelity, NASA (Космический центр им. Кеннеди), Национальная лаборатория сильных штормов, ракетный полигон White Sands, Rogue Wave Software, Boeing, Puma, iRobot и многие другие. Пол Дж. Дейтел и его соавтор, доктор Харви М. Дейтел, являются авторами одних из самых популярных в мире учебников по языкам программирования, профессиональных книг, видеофильмов и интерактивных мультимедийных электронных курсов, а также ведущими тренингов.

Харви М. Дейтел (Harvey M. Deitel), председатель совета директоров и директор по стратегии компании Deitel & Associates, Inc., имеет более 60 лет опыта работы в области вычислительной техники. Он получил степени бакалавра и магистра электротехники в Массачусетском технологическом институте и степень доктора математики в Бостонском университете (изучал вычислительную технику по всем этим программам еще до того, как в этих вузах были созданы факультеты компьютерных наук). Харви М. Дейтел имеет большой опыт преподавания в отраслевых организациях и колледжах, он был председателем факультета компьютерных наук в Бостонском колледже, а затем вместе с сыном Полом основал в 1991 году компанию Deitel & Associates. Публикации Дейтелов получили международное признание: более 100 переводов опубликованы на японском, немецком, русском, испанском, французском, польском, итальянском, упрощенном китайском, традиционном китайском, корейском, португальском, греческом, урду и турецком языках. Доктор Дейтел прочитал сотни курсов по программированию для академических, корпоративных, правительственных и военных клиентов.

От издательства

Все примеры исходного кода, приведенные в книге, вы можете скачать в виде ZIP-файла с сайта издательства «Питер» по адресу <https://clck.ru/38xMy4>.

Кроме того, в качестве бонуса вы можете скачать дополнительные онлайн-главы (на английском языке), которые авторы не включили в печатное издание книги.



Авторы снабдили текст книги внушительным количеством ссылок на электронные ресурсы на английском языке, в частности на статьи из Википедии. Там, где в российской Википедии имеются аналогичные статьи, ссылки были заменены на русскоязычные.

Ваши замечания, предложения, вопросы отправляйте по адресу comp@piter.com (издательство «Питер», компьютерная редакция).

Мы будем рады узнать ваше мнение!

На веб-сайте издательства www.piter.com вы найдете подробную информацию о наших книгах.

Перед началом работы

Перед тем как вы приступите к изучению C++, прочитайте этот раздел, чтобы узнать, что означают условные обозначения в книге и как подготовить свой компьютер к компиляции и выполнению программ.

Условные обозначения

В книге используются следующие шрифтовые обозначения.

Все примеры исходного кода, а также данные ввода-вывода набраны **моноширинными шрифтами**. Ключевые слова выделены **жирным**, а комментарии в коде — более светлым шрифтом. Обратите внимание, что номера в начале строк не являются частью инструкций C++ и добавлены только для наглядности:

```
1 int main() {  
2     std::cout << "Welcome to C++!\n"; // Выводим сообщение  
3     return 0; // Успешное завершение программы  
4 }
```

В основном тексте книги **моноширинным шрифтом** выделены слова и символы, обозначающие идентификаторы языка C++ (в том числе имена функций, классов и объектов), операторы, значения переменных, локальные пути и имена файлов, фрагменты исходного кода, фрагменты данных ввода-вывода, названия программных компонентов и подобные компьютерные сущности.

Жирным шрифтом выделены важные термины и их английские эквиваленты.

Загрузка примеров кода

Примеры кода, включенные в книгу *«C++20 для программистов. Метод готовых объектов»*, мы храним в репозитории GitHub¹. На веб-странице книги в разделе Source Code есть ссылка на этот репозиторий, а также прямая ссылка на ZIP-файл со всеми примерами кода:

<https://deitel.com/cpp20fp>

Если вы знакомы с Git и GitHub, клонируйте этот репозиторий в своей системе. Если вы скачали ZIP-файл, распакуйте его содержимое. В наших дальнейших

¹ Вы также можете скачать ZIP-файл с примерами кода с сайта издательства «Питер», ссылка приведена на с. 47 в разделе «От издательства». — *Примеч. ред.*

инструкциях мы исходим из того, что вы уже скопировали примеры кода из ZIP-файла в папку `examples`, расположенную в папке `Documents` вашей учетной записи пользователя.

Если вы еще не работали с Git и GitHub, но хотите узнать об этих важнейших инструментах разработчика, ознакомьтесь с их руководствами по адресу:

<https://guides.github.com/activities/hello-world>

Компиляторы, на которых мы тестировали исходный код

Убедитесь, что на вашем компьютере установлена последняя версия компилятора C++. Мы протестировали все приведенные в книге примеры кода на нескольких бесплатных компиляторах, которые рекомендуем:

- ♦ В операционной системе Microsoft Windows мы использовали интегрированную среду разработки Microsoft Visual Studio в версии Community Edition¹, включающую в себя компилятор Visual C++ и другие инструменты разработки от Microsoft.
- ♦ В операционной системе macOS мы использовали компилятор C++, входящий в состав Apple Xcode² и представляющий собой разновидность компилятора Clang C++.
- ♦ В операционной системе Linux мы использовали компилятор GNU C++³, входящий в состав пакета GNU Compiler Collection (GCC). Пакет GNU C++ уже установлен на многих компьютерах с Linux, но вам, возможно, надо обновить этот компилятор до более свежей версии. Его можно устанавливать и на компьютерах с macOS и Windows.
- ♦ Вы можете запускать последние версии компиляторов GNU C++ и Clang C++ во всех трех операционных системах — Windows, macOS и Linux — с помощью контейнеров Docker.
- ♦ Далее в разделе «Перед началом работы» описана установка перечисленных компиляторов и контейнеров Docker. В главе 1 книги, в разделе 1.2, мы покажем, как с помощью этих компиляторов компилировать и запускать написанные на C++ программы.

Не все примеры кода совместимы со всеми тремя компиляторами

К моменту выхода этой книги⁴ производители компиляторов еще не полностью реализовали в них некоторые из новых возможностей C++20. Они будут реали-

¹ Visual Studio 2022 Community Edition.

² Xcode 13.2.1.

³ GNU g++ 11.2.

⁴ Оригинальное издание вышло в феврале 2022 г. — *Примеч. ред.*

зованы в последующих версиях компиляторов, и тогда мы заново протестируем код, обновим наш цифровой контент и опубликуем обновления на сайте:

<https://deitel.com/cpp20fp>

Установка Visual Studio Community Edition в Windows

Если вы работаете в Windows, прежде всего посетите эту страницу и убедитесь, что ваш компьютер соответствует требованиям, обязательным для установки Microsoft Visual Studio Community Edition:

<https://docs.microsoft.com/en-us/visualstudio/releases/2022/system-requirements>

Затем перейдите по адресу загрузки:

<https://visualstudio.microsoft.com/downloads>

Для загрузки и установки выполните следующие шаги.

1. Нажмите кнопку **Free download** на вкладке веб-страницы **Community**.
2. Загрузите предлагаемый установочный файл (**VisualStudioSetup.exe**) и запустите его из той папки, в которую вы его загрузили.
3. В диалоговом окне **User Account Control** (Контроль учетных записей) нажмите кнопку **Yes**, чтобы разрешить инсталлятору внести изменения в вашу систему.
4. В диалоговом окне **Visual Studio Installer** нажмите кнопку **Continue**, чтобы разрешить инсталлятору загрузить компоненты, необходимые для дальнейшего выбора конфигурации.
5. Для работы с примерами кода из этой книги выберите в следующем окне пункт **Desktop Development with C++**, включающий компилятор **Visual C++** и стандартную библиотеку **C++**.
6. Нажмите кнопку **Install**. Начнется процесс загрузки и установки **Visual Studio**, который может занять много времени, особенно если у вас относительно медленное интернет-соединение.

Установка Xcode в macOS

На компьютере с macOS выполните следующие шаги для установки Xcode.

1. Откройте меню **Apple** и выберите пункт **App Store...** (или нажмите на значок **App Store** в нижней панели).
2. В строке поиска **App Store** наберите: **Xcode**.
3. Нажмите кнопку **Get**, чтобы установить Xcode.

Установка актуальной версии GNU C++

Существует множество дистрибутивов Linux, в которых используются разные методы обновления программ. Обратитесь к онлайн-документации вашего дис-

трибутива, чтобы узнать, как правильно обновить GNU C++ до последней версии. Вы также можете загрузить GNU C++ для разных платформ по адресу:

<https://gcc.gnu.org/install/binaries.html>

Установка GNU Compiler Collection в Ubuntu Linux, запущенной в подсистеме Windows для Linux

Если вы хотите установить GNU Compiler Collection на компьютер с Windows, это можно сделать с помощью подсистемы Windows для Linux (Windows Subsystem for Linux, WSL), которая позволяет запускать Linux в Windows. Вы можете легко установить Ubuntu Linux из Microsoft Store, но сначала необходимо установить WSL.

1. В строке поиска на панели задач наберите: **Turn Windows features on or off** (Включение или отключение компонентов Windows), затем нажмите кнопку **Open** (Открыть) в результатах поиска.
2. В открывшемся окне найдите в списке пункт **Windows Subsystem for Linux** (Подсистема Windows для Linux). Если в нем стоит галочка, то WSL уже установлена. Если галочки нет, поставьте ее и нажмите кнопку **OK**. Windows установит WSL и предложит вам перезагрузить компьютер.
3. После перезагрузки и входа в систему откройте приложение **Microsoft Store**, выполните поиск по слову **Ubuntu**, выберите приложение **Ubuntu** и нажмите кнопку **Install** (Установить). После этого будет установлена последняя версия Ubuntu Linux.
4. После установки нажмите кнопку **Launch**, чтобы открыть окно командной строки **Ubuntu Linux**, в котором процесс установки продолжится. Вам будет предложено создать имя пользователя и пароль для **Ubuntu** — они не должны совпадать с вашим именем пользователя и паролем для **Windows**.
5. Когда установка **Ubuntu** завершится, выполните следующие две команды, чтобы установить пакет **GCC** и отладчик **GNU** (при этом система может попросить вас ввести пароль для учетной записи, созданной на шаге 4):

```
sudo apt-get update
sudo apt-get install build-essential gdb
```

Убедитесь, что компилятор **g++** установлен, выполнив следующую команду:

```
g++ --version
```

Перед началом работы с нашими примерами исходного кода выполните в **Ubuntu** команду **cd** для изменения текущей папки, например:

```
cd /mnt/c/Users/YourUserName/Documents/examples
```

Используйте в этой команде свое имя пользователя и путь к папке, в которую вы поместили файлы с примерами исходного кода.

Docker и контейнеры Docker

Docker — это утилита для упаковки программ в контейнеры (также называемые образами), в которых собрано все необходимое для выполнения этих программ на разных платформах, что особенно полезно для программных пакетов со сложными настройками и конфигурациями. Для многих таких пакетов предлагаются бесплатные готовые контейнеры Docker (обычно на сайте <https://hub.docker.com>), которые вы можете загрузить и запустить локально на своем компьютере. Docker — это отличный способ быстро и удобно начать работу с новыми технологиями. Это также отличный способ поэкспериментировать с новыми версиями компиляторов.

Установка Docker

Чтобы использовать контейнеры Docker, необходимо сначала установить Docker. Если вы работаете в Windows или macOS, загрузите с этого сайта и запустите программу установки Docker Desktop:

<https://www.docker.com/get-started>

Затем следуйте инструкциям на экране. Кроме того, зарегистрируйте на этой странице свою учетную запись Docker Hub, чтобы получить возможность пользоваться контейнерами с сайта <https://hub.docker.com>. Если вы работаете в Linux, установите с этого сайта Docker Engine:

<https://docs.docker.com/engine/install>

Загрузка контейнера Docker, содержащего GNU Compiler Collection

Команда разработчиков GNU предоставляет официальные контейнеры Docker по адресу:

https://hub.docker.com/_/gcc

После установки и запуска Docker откройте командную строку¹ (в Windows), терминал (в macOS или Linux) или командную оболочку (Linux), затем выполните команду:

```
docker pull gcc:latest
```

После этого Docker загрузит контейнер, содержащий GNU Compiler Collection (GCC) последней версии (11.2 на момент написания книги). В разделе 1.2 мы покажем, как запустить этот контейнер и использовать его для компиляции и запуска программ, написанных на C++.

¹ Из учетной записи обычного пользователя Windows надо открывать командную строку от имени администратора (Run as administrator).

Загрузка контейнера Docker, содержащего Clang

В настоящее время у команды разработчиков Clang нет своего официального контейнера Docker, но многие готовые к работе контейнеры есть на сайте <https://hub.docker.com>. Для этой книги мы использовали следующий популярный контейнер:

<https://hub.docker.com/r/teeks99/clang-ubuntu>

Откройте командную строку (в Windows), терминал (в macOS или Linux) или командную оболочку (в Linux), затем выполните команду:

```
docker pull teeks99/clang-ubuntu:latest
```

После этого Docker загрузит контейнер, содержащий Clang последней версии (13.0 на момент написания книги). В разделе 1.2 мы покажем, как запустить этот контейнер и использовать его для компиляции и запуска программ, написанных на C++.

Ответы на ваши вопросы по C++

Если при чтении книги у вас возникнут вопросы, с нами легко связаться по email:

deitel@deitel.com

И через форму обратной связи на сайте:

<https://deitel.com/contact-us>

Мы оперативно ответим.

В интернете очень много информации о программировании. Например, бесценным ресурсом как для программистов, так и для всех интересующихся является сайт Stack Overflow:

<https://stackoverflow.com>

На этом сайте вы можете:

- ♦ искать ответы на самые распространенные вопросы о программировании;
- ♦ искать сообщения об ошибках, чтобы узнать их причину;
- ♦ задавать вопросы о программировании, чтобы получать ответы от программистов со всего мира;
- ♦ приобретать ценные знания о программировании в целом.

Собеседников на темы C++ вы найдете на канале **cpplang** в мессенджере Slack:

<https://cpplang-inviter.cppalliance.org>

и на Discord-сервере **#include<C++>**:

<https://www.includecpp.org/discord>

Онлайн-документация по C++

На этом сайте опубликована документация к стандартной библиотеке C++:

<https://cppreference.com>

А здесь вы можете ознакомиться с ответами на частые вопросы по C++:

<https://isocpp.org/faq>

О библиотеке форматирования текста {fmt}

На протяжении всей книги мы включили во многие программы следующую строку кода:

```
#include <fmt/format.h>
```

Она позволяет нашим программам использовать средства форматирования текста, предоставляемые библиотекой с открытым исходным кодом {fmt}¹. В этих программах есть вызовы функции `fmt::format`.

Новые средства форматирования текста, включенные в C++20, аналогичны той части библиотеки {fmt}, которая используется в наших программах. Поэтому при работе с компилятором, который уже полностью поддерживает стандарт C++20, предыдущую строку кода можно заменить следующей:

```
#include <format>
```

В этом случае надо использовать вызовы функции `std::format` (вместо аналогичных вызовов функции `fmt::format`).

Когда мы писали эту книгу, только компилятор Microsoft Visual C++ поддерживал новые средства форматирования текста, введенные в C++20. По этой причине в наших примерах используется библиотека с открытым исходным кодом {fmt}, гарантирующая совместимость кода с остальными рекомендуемыми компиляторами.

Средства статического анализа кода

Мы использовали три статических анализатора кода, чтобы проверить наши примеры кода на соответствие основным принципам C++, стандартам кодирования и идиомам современного C++, а также убедиться, что в них нет распространенных ошибок, возможных проблем по части безопасности, быстродействия, читаемости кода и т. д.:

- ♦ `clang-tidy` — <https://clang.llvm.org/extra/clang-tidy/>;
- ♦ `cppcheck` — <https://cppcheck.sourceforge.io/>;
- ♦ средства проверки кода на соответствие Microsoft C++ Core Guidelines, встроенные в статический анализатор кода Visual Studio.

¹ «{fmt}». Был доступен: 15 сентября 2023 г. <https://github.com/fmtlib/fmt>.

Вы можете установить программу `clang-tidy` в Linux следующими командами:

```
sudo apt-get update -y  
sudo apt-get install -y clang-tidy
```

Вы можете установить программу `cppcheck` для различных платформ (операционных систем), следуя инструкциям на сайте <https://cppcheck.sourceforge.io>. В Visual C++, после того как вы выполните тест-драйв из раздела 1.2 и научитесь создавать проект, вы можете настроить статический анализатор кода таким образом:

1. Щелкните правой кнопкой мыши на имени проекта в **Solution Explorer** и выберите **Properties**.
2. В появившемся диалоговом окне выберите **Code Analysis > General** в левой колонке, а затем установите для параметра **Enable Code Analysis on Build** значение **Yes** в правой колонке.
3. Затем выберите **Code Analysis > Microsoft** в левой колонке. Теперь в правой колонке, в раскрывающемся списке, можно выбрать определенный набор правил анализа. Мы использовали опцию **<Choose multiple rule sets...>**, чтобы выбрать все наборы правил, начинающиеся с **C++ Core Check**. Нажмите кнопку **Save As...**, дайте имя своему набору правил, нажмите кнопку **Save**, а затем **Apply**. (Обратите внимание, что при этом будет выдано много предупреждений, связанных с библиотекой форматирования текста `{fmt}`, которую мы используем во многих примерах кода.)

Введение и тест-драйв популярных бесплатных компиляторов C++

В этой главе вы:

- узнаете о структуре книги и получите советы о том, с чего начать изучение большого, сложного и мощного языка C++20;
- научитесь компилировать и запускать написанное на C++ тестовое приложение с помощью трех компиляторов, которые мы рекомендуем: Visual C++ из Microsoft Visual Studio (в Windows), Clang C++ из Xcode (в macOS) и g++ из GNU Compiler Collection (в Linux);
- узнаете, как запускать компиляторы g++ и clang++ в контейнерах Docker, с которыми можно работать и в Windows, и в macOS, и в Linux;
- найдете ссылки на интернет-ресурсы, рассказывающие о более чем 40-летней истории C++;
- узнаете, почему многопоточное (конкурентное) программирование на C++ ускоряет работу приложений на компьютерах с многоядерными процессорами;
- познакомитесь с концепцией объектов, которую мы в начале книги продемонстрируем на простых примерах с готовыми объектами, а затем подробно рассмотрим в главах, посвященных объектно-ориентированному программированию (начиная с главы 9).

1.1. Введение

Добро пожаловать в C++! Это один из самых популярных в мире языков программирования¹. Мы поможем вам изучить его в современной версии, которая называется C++20 («Си плюс плюс двадцать»). Это последняя на сегодняшний день версия, которая стандартизирована Международной организацией по стандартизации (International Organization for Standardization, ISO). Темы главы 1 рассчитаны на изучение языка практически с нуля:

- ♦ краткий обзор книги для быстрого ознакомления с ее структурой;
- ♦ тест-драйв трех популярных бесплатных компиляторов C++;
- ♦ закон Мура, многоядерные процессоры и решающая роль многопоточного программирования в создании быстродействующих приложений на C++;
- ♦ основные принципы и термины объектно-ориентированного программирования, которые будут встречаться в книге далее.

Краткий обзор этой книги

Прежде чем вы углубитесь в чтение, мы рекомендуем вам быстро взглянуть на всю структуру книги «с высоты 40 000 футов», чтобы получить общее представление о том, как вы будете изучать большой, сложный и мощный язык C++20. Для этого в книге есть:

- ♦ Оглавление в виде наглядной схемы на внутренней стороне обложки.
- ♦ Предисловие, поясняющее суть книги и наш подход к современному программированию на C++. Здесь мы представляем метод готовых объектов, который позволит вам использовать минимум простых инструкций C++, чтобы заставить мощные классы выполнять важные задачи. Вы сможете это делать задолго до того, как самостоятельно напишете ваши первые классы. Обязательно прочтите в предисловии раздел «Краткое содержание книги», в котором указаны ключевые темы каждой главы. При чтении этого раздела вам будет удобно сверяться со схемой-оглавлением.

Ресурсы по истории C++

В 1979 году Бьерн Страуструп (Bjarne Stroustrup; в другой транскрипции — Бьярне Страуструп) приступил к разработке языка программирования C++, который он сначала назвал «C with Classes»². Сегодня не менее 5 миллионов разработчиков (по некоторым оценкам — до 7,5 миллиона^{3, 4}) используют C++ для создания

¹ «TIOBE Index». Был доступен: 15 сентября 2023 г. <https://www.tiobe.com/tiobe-index>.

² «Страуструп, Бьерн». https://ru.wikipedia.org/wiki/Страуструп,_Бьерн.

³ State of the Developer Nation, 21st Edition», Q3 2021. Был доступен: 15 сентября 2023 г. <https://www.slashdata.co/free-resources/state-of-the-developer-nation-21st-edition>.

⁴ Тим Андерсон (Tim Anderson), «Report: World's Population of Developers Expands, JavaScript Reigns, C# Overtakes PHP», 26 апреля 2021 г. Был доступен: 15 сентября 2023 г. https://www.theregister.com/2021/04/26/report_developers_slashdata.

критически важных цифровых систем и прикладных программ (приложений)^{1,2}. На этом языке написаны многие компоненты операционных систем Windows³ и macOS⁴. C++ широко применяется при разработке браузеров (например, Google Chrome⁵ и Mozilla Firefox⁶), систем управления базами данных (например, MySQL⁷ и MongoDB⁸) и многих других видов приложений.

История языка C++ хорошо документирована:

- ♦ Эта статья в «Википедии» содержит подробную историю C++ и много полезных ссылок:

<https://ru.wikipedia.org/wiki/C++>

- ♦ Бьерн Страуструп, создатель C++, рассказывает о том, как устроен этот язык и как он совершенствовался с момента создания до появления современной версии C++20:

<https://www.stroustrup.com/C++.html#design>

- ♦ Здесь вы найдете список основных этапов развития C++ с момента создания и другие исторические сведения:

<https://ru.cppreference.com/w/cpp/language/history>

1.2. Тест-драйв: компиляция приложения, написанного на C++20

В этом разделе вы будете компилировать, запускать и проверять в работе ваше первое приложение на C++⁹. Это игра, в которой надо угадать случайное число от 1 до 1000. Если вы введете правильное число, игра закончится. Если вы ошибетесь, приложение сообщит, что вы ввели слишком большое или слишком маленькое число. Нет никаких ограничений на количество попыток, которые вы можете сделать.

¹ «Top 10 Reasons to Learn C++». Был доступен: 15 сентября 2023 г. <https://www.geeksforgeeks.org/top-10-reasons-to-learn-c-plus-plus>.

² «What Is C++ Used For? Top 12 Real-World Applications and Uses of C++». Был доступен: 15 сентября 2023 г. <https://www.softwaretestinghelp.com/cpp-applications>.

³ «What Programming Language Is Windows Written In?». Был доступен: 15 сентября 2023 г. <https://social.microsoft.com/Forums/en-US/65a1fe05-9c1d-48bf-bd40-148e6b3da9f1/what-programming-language-is-windows-written-in>.

⁴ «macOS». <https://ru.wikipedia.org/wiki/MacOS>.

⁵ «Google Chrome». https://ru.wikipedia.org/wiki/Google_Chrome.

⁶ «Firefox». <https://ru.wikipedia.org/wiki/Firefox>.

⁷ «MySQL». <https://ru.wikipedia.org/wiki/MySQL>.

⁸ «MongoDB». <https://ru.wikipedia.org/wiki/MongoDB>.

⁹ Мы намеренно не приводим исходный код этого приложения. Пока мы хотим только показать вам, как компилировать и запускать программы с помощью трех компиляторов, которые мы рекомендуем. А с генерацией случайных чисел вы познакомитесь в главе 5.

Рассматриваемые компиляторы и среды разработки

Мы покажем, как компилировать исходный код на языке C++ и запускать скомпилированную программу в трех операционных системах:

- ✦ в Windows — с помощью интегрированной среды разработки Microsoft Visual Studio 2022 Community Edition (раздел 1.2.1);
- ✦ в macOS — с помощью интегрированной среды разработки Apple Xcode (раздел 1.2.2);
- ✦ в Linux — с помощью компилятора g++, который входит в набор GNU Compiler Collection (раздел 1.2.3);
- ✦ в любой из трех операционных систем — с помощью этого же компилятора g++, работающего в контейнере Docker (раздел 1.2.4);
- ✦ в любой из трех операционных систем — с помощью компилятора clang++, работающего в контейнере Docker (раздел 1.2.5).

Вы можете прочитать только тот раздел, который соответствует вашему компьютеру. Чтобы использовать компиляторы g++ и clang++ в контейнерах Docker, предварительно установите и запустите Docker, как описано в разделе «Перед началом работы».

1.2.1. Компиляция и запуск приложения в Windows с помощью Visual Studio 2022

В этом разделе вы скомпилируете и запустите написанную на C++ программу в операционной системе Windows, используя интегрированную среду разработки Microsoft Visual Studio 2022 Community Edition¹. Она существует в нескольких версиях. Параметры, меню и необходимые действия, описанные в этом разделе, могут незначительно отличаться в зависимости от версии. Далее мы будем использовать термины Visual Studio и IDE (Integrated Development Environment — интегрированная среда разработки).

Шаг 1. Подготовка программного обеспечения

Если вы еще не сделали этого, прочитайте раздел «Перед началом работы», установите Visual Studio и скачайте с сайта Deitel файл с включенными в книгу примерами исходного кода.

Шаг 2. Запуск Visual Studio

Откройте Visual Studio из меню операционной системы Start (Пуск). Закройте начальное окно Visual Studio, нажав клавишу Esc. Не щелкайте мышью на кнопке X в правом верхнем углу: она завершает работу Visual Studio. Вы сможете снова открыть начальное окно в любое время, выбрав File > Start window. Мы будем

¹ Мы рассматриваем Microsoft Visual Studio 2022 Community Edition версии 17.0.5.

обозначать символом > выбор пункта меню. Например, **File > Open** означает «выберите пункт Open в меню File».

Шаг 3. Создание проекта

Проект (project) — это группа взаимосвязанных файлов (содержащих исходный код на языке C++), из которых состоит приложение. Visual Studio позволяет объединять разрабатываемые приложения в проекты и **решения (solutions)**. Каждое решение может содержать один или несколько проектов. Решения, состоящие из нескольких проектов, необходимы только для очень больших приложений. Для каждого приложения из этой книги нужно решение только с одним проектом. Изучая наши примеры кода, вы будете начинать с **пустого проекта (empty project)** и добавлять в него файлы. Для создания проекта:

1. Выберите **File > New > Project...**, чтобы открыть диалоговое окно **Create a New Project**.
2. Выберите шаблон с именем **Empty Project** и тегами **C++, Windows** и **Console**. Он предназначен для программ, выполняемых в командной строке. (В зависимости от версии и настроек Visual Studio в списке может быть много других шаблонов. Для поиска нужного шаблона введите его имя в строку **Search for templates**). Щелкните на кнопке **Next**, чтобы открыть диалоговое окно **Configure your new project**.
3. Укажите **Project name** (имя проекта) и **Location** (путь к файлам проекта). В качестве имени мы выбрали **cpp20_test**. В качестве пути мы выбрали папку **examples**, содержащую примеры кода из этой книги. Щелкните на кнопке **Create**, чтобы создать новый проект.

Visual Studio создаст ваш проект, назначив для его файлов папку:

`C:\Users\ИмяПользователя\Documents\examples`

(или другую указанную вами папку), и откроет главное окно.

Когда вы пишете и редактируете код, Visual Studio отображает каждый файл с кодом вашего приложения в отдельной вкладке. Для просмотра этих файлов и для управления ими предназначено окно **Solution Explorer** (оно может находиться справа или слева в главном окне Visual Studio). В нем есть папка **Source Files**, в которую вы будете помещать файлы с нашими примерами кода. Если **Solution Explorer** не отображается, вы можете вывести его на экран, выбрав **View > Solution Explorer**.

Шаг 4. Добавление файла **GuessNumber.cpp** в проект

Теперь добавьте файл **GuessNumber.cpp** к созданному на шаге 3 проекту. Для этого в **Solution Explorer**:

1. Щелкните правой кнопкой мыши на папке **Source Files** и выберите **Add > Existing Item...**

2. В появившемся диалоговом окне перейдите в папку `ch01`, вложенную в папку `examples`, выберите `GuessNumber.cpp` и щелкните на кнопке `Add`¹.

Шаг 5. Настройка проекта для работы с C++20

Входящий в состав Visual Studio компилятор C++ поддерживает несколько версий стандарта C++. Для работы с этой книгой надо выбрать в параметрах проекта версию C++20:

1. В **Solution Explorer** щелкните правой кнопкой мыши на узле проекта `cpp20_test` и выберите **Properties**, чтобы открыть диалоговое окно **Property Pages** проекта `cpp20_test`.
2. В раскрываемом списке **Configuration** выберите **All Configurations**. В раскрываемом списке **Platform** выберите **All Platforms**.
3. В левом столбце разверните узел **C/C++** и выберите **Language**.
4. В правом столбце щелкните в поле справа от **C++ Language Standard**, затем щелкните на стрелке вниз, выберите **ISO C++20 Standard (/std:c++20)** и щелкните на кнопке **OK**.

Шаг 6. Компиляция и запуск проекта

Чтобы скомпилировать и запустить проект, то есть проверить приложение в работе, выберите **Debug > Start without debugging** или нажмите **Ctrl + F5**. Если исходный код компилируется без ошибок, Visual Studio откроет окно командной строки и выполнит приложение. Вы увидите такое сообщение:

```
I have a number between 1 and 1000.  
Can you guess my number?  
Please type your first guess.  
?
```

Шаг 7. Ввод первого числа

После знака `?` в командной строке введите `500` и нажмите **Enter**. Ответ приложения будет меняться при каждом его запуске. У нас приложение вывело сообщение **"Too low. Try again."**, указывающее, что введенное число меньше случайного числа, которое надо угадать:

```
I have a number between 1 and 1000.  
Can you guess my number?  
Please type your first guess.  
? 500  
Too low. Try again.  
?
```

¹ Если исходный код состоит из нескольких файлов (вы встретите такие программы в последующих главах), выбирайте в этом диалоговом окне все эти файлы сразу. Когда вы начнете писать программы самостоятельно, вы можете щелкнуть правой кнопкой мыши на папке **Source Files** и выбрать **Add > New Item...**, чтобы открыть диалоговое окно добавления нового файла.

Шаг 8. Ввод второго числа

После следующего знака `?`, если первое число оказалось слишком маленьким, введите `750` и нажмите `Enter`; в противном случае введите `250` и нажмите `Enter`. Например, мы ввели `750`, и приложение вывело сообщение `"Too high. Try again."`, потому что введенное значение оказалось слишком большим:

```
I have a number between 1 and 1000.  
Can you guess my number?  
Please type your first guess.  
? 500  
Too low. Try again.  
? 750  
Too high. Try again.  
?
```

Шаг 9. Ввод следующих чисел

Продолжайте игру, вводя разные числа, пока не угадаете то число, которое выбрано приложением. После этого вы увидите сообщение `"Excellent! You guessed the number!"`:

```
I have a number between 1 and 1000.  
Can you guess my number?  
Please type your first guess.  
? 500  
Too low. Try again.  
? 750  
Too high. Try again.  
? 625  
Too high. Try again.  
? 562  
Too low. Try again.  
? 593  
Too low. Try again.  
? 607  
Too low. Try again.  
? 616  
Too high. Try again.  
? 612  
Too low. Try again.  
? 614  
Too high. Try again.  
? 613  
  
Excellent! You guessed the number!  
Would you like to play again (y or n)?
```

Шаг 10. Повторная игра или завершение работы приложения

Когда число будет угадано, приложение спросит, не хотите ли вы сыграть еще раз: `"would you like to play again (y or n)?"`. При нажатии клавиши `y` приложение вы-

берет другое случайное число и игра начнется сначала. При нажатии клавиши **n** приложение завершит работу.

Повторное использование проекта для последующих примеров

Вы можете создавать отдельный проект для каждого приложения из книги, выполняя описанные в этом разделе действия. Или, возможно, вам будет удобнее работать с одним проектом, каждый раз удаляя из него файл с исходным кодом и добавляя вместо него следующий файл. Чтобы удалить файл из вашего проекта (но не из вашей системы), выберите этот файл в **Solution Explorer**, затем нажмите клавишу **Del** (**Delete**). Теперь вы можете повторить шаг 4, чтобы добавить другой файл в этот же проект.

Работа с Ubuntu в подсистеме Windows для Linux

Некоторые пользователи Windows могут захотеть работать на своих Windows-компьютерах с компилятором **g++**, входящим в пакет **GNU Compiler Collection** (**GCC**). Это можно сделать двумя способами: использовать контейнер **Docker**, содержащий **GCC** (раздел 1.2.4), или запускать этот компилятор в операционной системе **Ubuntu**, установленной в подсистеме **Windows для Linux**. Чтобы установить подсистему **Windows для Linux**, следуйте инструкциям по ссылке:

<https://docs.microsoft.com/ru-ru/windows/wsl/install>

Установите в подсистеме **Windows для Linux** приложение **Ubuntu** (см. раздел «Перед началом работы»). Чтобы перейти в папку с файлом исходного кода приложения, введите в командной строке **Ubuntu** следующую команду:

```
cd /mnt/c/Users/ИмяПользователя/Documents/examples/ch01
```

и продолжите с шага 2 из раздела 1.2.3 («Компиляция и запуск приложения в Linux с помощью **g++**»).

1.2.2. Компиляция и запуск приложения в macOS с помощью Xcode

В этом разделе вы скомпилируете и запустите написанную на **C++** программу в операционной системе **macOS**, используя интегрированную среду разработки **Apple Xcode**¹.

Шаг 1. Подготовка программного обеспечения

Если вы еще не сделали этого, прочтите раздел «Перед началом работы», установите **Xcode** и скачайте с сайта **Deitel** все наши примеры исходного кода.

Шаг 2. Запуск Xcode

Откройте окно **Finder**, выберите **Applications** (Программы) и дважды щелкните на значке **Xcode**.



¹ Мы рассматриваем Xcode версии 13.2.1.

Если вы впервые запускаете Xcode, появится окно **Welcome to Xcode**. Закройте это окно. Вы сможете снова открыть его в любое время, выбрав **Window > Welcome to Xcode**. Мы будем обозначать символом > выбор пункта меню. Например, **File > Open** означает «выберите пункт **Open** в меню **File**».

Шаг 3. Создание проекта

Проект (project) — это группа взаимосвязанных файлов (содержащих исходный код на языке C++), из которых состоит приложение. Все проекты Xcode, которые мы создали для этой книги, представляют собой программы командной строки, которые вы будете выполнять непосредственно в IDE. Для создания проекта:

1. Выберите **File > New > Project...**
2. В верхней части диалогового окна **Choose a template for your new project** щелкните на **macOS**.
3. В разделе **Application** выберите **Command Line Tool** и щелкните на кнопке **Next**.
4. В поле **Product Name** введите имя вашего проекта. В качестве имени мы указали **cpp20_test**.
5. В раскрывающемся списке **Language** выберите **C++** и щелкните на кнопке **Next**.
6. Укажите, где вы хотите сохранить проект. Мы выбрали папку **examples**, содержащую примеры из этой книги.
7. Щелкните на **Create**.

После создания проекта откроется главное окно Xcode, содержащее три области: слева — **Navigator**, в центре — **Editor** и справа — **Utilities**.

В верхней части **Navigator** находятся значки переключения панелей. Для работы с этой книгой вам будут нужны в основном две панели:

Project () — показывает все файлы и папки вашего проекта.

Issue () — показывает выдаваемые компилятором предупреждения и сообщения об ошибках.

При нажатии на значок отображается соответствующая панель.

Центральная область **Editor** предназначена для редактирования исходного кода и настройки параметров проекта. Она всегда присутствует в главном окне Xcode. Когда вы выбираете файл в панели **Project**, его содержимое отображается в **Editor**. Справа, в **Utilities**, обычно отображаются **инспекторы (inspectors)**. Например, если вы будете писать приложение для iPhone, содержащее сенсорную кнопку, то сможете настроить в этой области свойства кнопки (ее заголовок, размер, расположение и т. д.). При работе с этой книгой вы не будете использовать **Utilities**. Существует также область отладки **Debug**, в которой вы будете взаимодействовать с запущенной программой. Она появляется под областью **Editor**.

На верхней панели инструментов есть кнопки запуска и остановки программы, индикатор процесса выполнения задач, кнопки сокрытия и отображения левой (**Navigator**) и правой (**Utilities**) областей.

Шаг 4. Настройка проекта для работы с C++20

Входящий в состав Xcode компилятор Apple Clang поддерживает несколько версий стандарта C++. Для работы с этой книгой надо выбрать в параметрах проекта версию C++20:

1. В панели **Project** выберите название вашего проекта (`cpp20_test`).
2. В левой части **Editor** выберите название вашего проекта в разделе **TARGETS**.
3. В верхней части **Editor** выберите **Build Settings**, а чуть ниже нажмите **All**.
4. Перейдите к разделу **Apple Clang > Language > C++**.
5. Щелкните на значении справа от **C++ Language Dialect** и выберите **GNU++20 [-std=gnu++20]**.
6. Щелкните на значении справа от **C++ Standard Library** и выберите **Compiler Default**.

Шаг 5. Удаление файла `main.cpp` из проекта

По умолчанию Xcode создает файл `main.cpp`, содержащий исходный код простой программы, которая выводит строку "Hello, world!". В этом тест-драйве вам не понадобится файл `main.cpp`, поэтому удалите его. В панели **Project** сделайте «правый щелчок» (`Ctrl` + щелчок) на файле `main.cpp` и выберите **Delete**. В появившемся диалоговом окне нажмите **Move to Trash**. Файл не будет удален из вашей системы до тех пор, пока вы не очистите корзину.

Шаг 6. Добавление файла `GuessNumber.cpp` в проект

В окне **Finder** откройте папку `ch01` из папки `examples`, содержащей наши примеры кода, и перетащите файл `GuessNumber.cpp` в папку `cpp20_test` на панели **Project**. В появившемся диалоговом окне убедитесь, что установлен флаг **Copy items if needed**, и нажмите кнопку **Finish**¹.

Шаг 7. Компиляция и запуск проекта

Чтобы скомпилировать и запустить проект, то есть проверить приложение в работе, нажмите кнопку запуска (▶) на панели инструментов Xcode. Если исходный код компилируется без ошибок, Xcode откроет область **Debug** и выполнит в ее правой половине приложение, которое выведет строку "Please type your first guess." и вопросительный знак (?) в качестве запроса данных:

```
I have a number between 1 and 1000.  
Can you guess my number?  
Please type your first guess.  
?
```

¹ Если исходный код состоит из нескольких файлов (вы встретите такие программы в последующих главах), перетащите все эти файлы в папку проекта. Когда вы начнете писать программы самостоятельно, вы можете сделать «правый щелчок» на папке проекта и выбрать **New File...**, чтобы открыть диалоговое окно добавления нового файла.

Шаг 8. Ввод первого числа

Щелкните мышью в области **Debug**, введите **500** и нажмите **Return**. Ответ приложения будет меняться при каждом его запуске. У нас приложение вывело сообщение "Too high. Try again.", указывающее, что введенное число больше случайного числа, которое надо угадать:

```
I have a number between 1 and 1000.  
Can you guess my number?  
Please type your first guess.  
? 500  
Too high. Try again.  
?
```

Шаг 9. Ввод второго числа

После следующего знака **?**, если первое число оказалось слишком маленьким, введите **750** и нажмите **Return**; в противном случае введите **250** и нажмите **Return**. Например, мы ввели **250**, и приложение вывело сообщение "Too high. Try again.", потому что введенное значение оказалось слишком большим:

```
I have a number between 1 and 1000.  
Can you guess my number?  
Please type your first guess.  
? 500  
Too high. Try again.  
? 250  
Too high. Try again.  
?
```

Шаг 10. Ввод следующих чисел

Продолжайте игру, вводя разные числа, пока не угадаете то число, которое выбрано приложением. После этого вы увидите сообщение "Excellent! You guessed the number!":

```
I have a number between 1 and 1000.  
Can you guess my number?  
Please type your first guess.  
? 500  
Too high. Try again.  
? 250  
Too high. Try again.  
? 125  
Too low. Try again.  
? 187  
Too high. Try again.  
? 156  
Too high. Try again.  
? 140  
Too low. Try again.
```

```
? 148
Too high. Try again.
? 144
Too high. Try again.
? 142
Too low. Try again.
? 143

Excellent! You guessed the number!
Would you like to play again (y or n)?
```

Шаг 11. Повторная игра или завершение работы приложения

Когда число будет угадано, приложение спросит, не хотите ли вы сыграть еще раз: "Would you like to play again (y or n)?"¹. При нажатии клавиши `y` приложение выберет другое случайное число и игра начнется сначала. При нажатии клавиши `n` приложение завершит работу.

Повторное использование проекта для последующих примеров

Вы можете создавать отдельный проект для каждого приложения из книги, выполняя описанные в этом разделе действия. Или, возможно, вам будет удобнее работать с одним проектом, каждый раз удаляя из него файл с исходным кодом и добавляя вместо него следующий файл. Чтобы удалить файл из вашего проекта (но не из вашей системы), сделайте «правый щелчок» (`Ctrl` + щелчок) на имени файла в панели **Project** и выберите **Delete**. В появившемся диалоговом окне нажмите **Remove Reference**. Теперь вы можете повторить шаг 6, чтобы добавить другой файл в этот же проект.

1.2.3. Компиляция и запуск приложения в Linux с помощью g++

В этом разделе вы скомпилируете и запустите написанную на C++ программу в командной оболочке Linux, используя компилятор `g++` из пакета GNU Compiler Collection (GCC)¹. Мы предполагаем, что вы уже прочитали раздел «Перед началом работы» и поместили все наши примеры исходного кода в каталог `Documents/examples`, расположенный в каталоге вашей учетной записи пользователя.

Шаг 1. Переход в каталог `ch01`

Выполните команду `cd` в командной оболочке Linux, чтобы перейти в каталог `ch01` (первый пример из каталога `examples`):

```
~$ cd ~/Documents/examples/ch01
~/Documents/examples/ch01$
```

¹ Мы рассматриваем `g++` версии 11.2. Чтобы узнать номер вашей версии компилятора `g++`, выполните команду `g++ --version`. Если у вас более старая версия `g++`, найдите в интернете инструкции по обновлению GNU Compiler Collection (GCC) для вашей системы Linux или используйте контейнер Docker с GCC, описанный в разделе 1.2.4.

В этом разделе мы будем выделять в командах жирным шрифтом те данные, которые вводит пользователь. **Приглашение (prompt)** в оболочке Ubuntu Linux обозначает тильдой (~) домашний каталог. Приглашение обычно заканчивается знаком доллара (\$), но может отличаться в зависимости от вашей системы Linux.

Шаг 2. Компиляция приложения

Перед запуском приложения надо скомпилировать его следующей командой компилятора g++¹:

```
~/Documents/examples/ch01$ g++ -std=c++20 GuessNumber.cpp -o GuessNumber
~/Documents/examples/ch01$
```

В этой команде:

- ♦ Параметр `-std=c++20` указывает компилятору, что мы используем C++20.
- ♦ Параметр `-o` присваивает исполняемому файлу имя (`GuessNumber`), которое затем будет нужно для запуска программы. Если вы не включите этот параметр, g++ автоматически присвоит исполняемому файлу имя `a.out`.

Шаг 3. Запуск приложения

Введите `./GuessNumber` в командной строке и нажмите Enter, чтобы запустить программу:

```
~/Documents/examples/ch01$ ./GuessNumber
I have a number between 1 and 1000.
Can you guess my number?
Please type your first guess.
?
```

Символы `./` перед `GuessNumber` указывают системе Linux, что надо запустить программу `GuessNumber` из текущего каталога.

Шаг 4. Ввод первого числа

Приложение выведет сообщение `"Please type your first guess."` и вопросительный знак (?) в качестве запроса данных. Введите `500` (ответ приложения будет меняться при каждом его запуске):

```
~/Documents/examples/ch01$ ./GuessNumber
I have a number between 1 and 1000.
Can you guess my number?
Please type your first guess.
? 500
Too high. Try again.
?
```

¹ Если на вашем компьютере установлено несколько версий g++, то нужную версию можно запустить командой `g++-##`, где ## — номер версии. Например, команда `g++-11` запускает последнюю на сегодняшний день версию 11.x.

У нас приложение вывело сообщение "Too high. Try again.", указывающее, что введенное число больше случайного числа, которое надо угадать.

Шаг 5. Ввод второго числа

После следующего знака `?`, если первое число оказалось слишком маленьким, введите `750` и нажмите `Enter`; в противном случае введите `250` и нажмите `Enter`. Например, мы ввели `250`, и приложение вывело строку "Too high. Try again.", потому что введенное значение оказалось слишком большим:

```
~/Documents/examples/ch01$ ./GuessNumber
I have a number between 1 and 1000.
Can you guess my number?
Please type your first guess.
? 500
Too high. Try again.
? 250
Too high. Try again.
?
```

Шаг 6. Ввод следующих чисел

Продолжайте игру, вводя разные числа, пока не угадаете то число, которое выбрано приложением. После этого вы увидите сообщение "Excellent! You guessed the number!":

```
? 125
Too high. Try again.
? 62
Too low. Try again.
? 93
Too low. Try again.
? 109
Too high. Try again.
? 101
Too low. Try again.
? 105
Too high. Try again.
? 103
Too high. Try again.
? 102

Excellent! You guessed the number!
Would you like to play again (y or n)?
```

Шаг 7. Повторная игра или завершение работы приложения

Когда число будет угадано, приложение спросит, не хотите ли вы сыграть еще раз: "Would you like to play again (y or n)?". При нажатии клавиши `y` приложение выберет другое случайное число и игра начнется сначала. При нажатии клавиши `n` приложение завершит работу.

1.2.4. Компиляция и запуск приложения в контейнере Docker с помощью g++

С компилятором g++, входящим в состав GNU Compiler Collection (GCC), вы можете работать не только в Linux, но также в Windows и macOS. Один из лучших способов реализовать это кросс-платформенное решение — запустить **контейнер (container) Docker**, содержащий GCC. Мы предполагаем, что вы уже установили Docker Desktop (в Windows или macOS) или Docker Engine (в Linux) по инструкции из раздела «Перед началом работы».

Запуск контейнера Docker с GNU Compiler Collection (GCC)

Откройте командную строку (в Windows), терминал (в macOS/Linux) или командную оболочку (в Linux). Затем выполните следующие действия, чтобы запустить контейнер Docker с GCC:

1. С помощью команды `cd` перейдите в папку `examples`, содержащую все наши примеры исходного кода.
2. В Windows: запустите контейнер Docker следующей командой¹:

```
docker run --rm -it -v "%CDX%":/usr/src gcc:latest
```

3. В macOS/Linux: запустите контейнер Docker следующей командой:

```
docker run --rm -it -v "${PWD}":/usr/src gcc:latest
```

В этих командах:

- ♦ `--rm` очищает ресурсы контейнера, когда вы его закрываете.
- ♦ `--it` запускает контейнер в интерактивном режиме, поэтому вы можете вводить команды для изменения папок, а также для компиляции и запуска программ с помощью g++.
- ♦ `-v "%CDX%":/usr/src` (в Windows) или `-v "${PWD}":/usr/src` (в macOS/Linux) позволяет контейнеру Docker получить доступ к файлам в папке, из которой выполняется команда `docker run`. В контейнере Docker вы перейдете с помощью команды `cd` к папке `/usr/src`, чтобы компилировать и запускать примеры из книги. Другими словами, ваша локальная системная папка будет сопоставлена с папкой `/usr/src` в контейнере Docker.
- ♦ `gcc:latest` — это имя контейнера. Параметр `:latest` указывает, что вы хотите использовать новейшую версию контейнера gcc².

Когда контейнер будет запущен, вы увидите примерно такое приглашение:

```
root@67773f59d9ea:/#
```

¹ Может появиться уведомление с просьбой разрешить контейнеру Docker доступ к файлам в текущей папке. Вам надо дать это разрешение, иначе вы не сможете получить из Docker доступ к нашим файлам исходного кода.

² Если вы хотите обновить ваш контейнер до последней версии, выполните команду `docker pull gcc:latest` перед запуском контейнера.

В контейнере работает операционная система Linux. В приглашении контейнера имя папки, в которой вы находитесь, отображается между символами : и #.

Переход к папке ch01 в контейнере Docker

Введенная вами команда `docker run` присоединила вашу папку `examples` к папке `/usr/src` файловой системы контейнера. Теперь в контейнере Docker с помощью команды `cd` перейдите к папке `ch01`, которая находится в `/usr/src`:

```
root@01b4d47cad6c:/# cd /usr/src/ch01
root@01b4d47cad6c:/usr/src/ch01#
```

Для компиляции, запуска и тестирования приложения `GuessNumber` в контейнере Docker выполните шаги 2–7 раздела 1.2.3 («Компиляция и запуск приложения в Linux с помощью g++»).

Завершение работы контейнера Docker

Чтобы завершить работу контейнера, нажмите `Ctrl + d` в его командной строке.

1.2.5. Компиляция и запуск приложения в контейнере Docker с помощью clang++

Вы можете работать в разных операционных системах не только с `g++`, но и с компилятором LLVM/Clang C++ (`clang++`), который тоже предназначен для командной строки. В настоящее время разработчики LLVM / Clang не предоставляют свой официальный контейнер Docker, но многие готовые к работе контейнеры есть на сайте hub.docker.com. Мы предполагаем, что вы уже установили Docker Desktop (в Windows или macOS) или Docker Engine (в Linux) по инструкции из раздела «Перед началом работы».

Мы используем актуальную и самую распространенную 13-ю версию `clang++`, которую вы можете загрузить с помощью следующей команды¹:

```
docker pull teeks99/clang-ubuntu:13
```

Запуск контейнера Docker с образом teeks99/clang-ubuntu

Откройте командную строку (Windows), терминал (macOS/Linux) или командную оболочку (Linux) и выполните следующие действия, чтобы запустить контейнер Docker, содержащий образ `teeks99/clang-ubuntu`.

1. С помощью команды `cd` перейдите в папку `examples`, содержащую все наши примеры исходного кода.

¹ В Xcode включена не самая новая версия компилятора Clang C++, и в ней реализовано не так много возможностей C++20, как в версии от команды LLVM/Clang. Кроме того, на момент написания этой книги команда `docker pull` с параметром `latest` вместо `13` загружает контейнер Docker с 12-й, а не 13-й версией `clang++`.

2. В Windows: запустите контейнер Docker следующей командой¹:

```
docker run --rm -it -v "%CD%":/usr/src teeks99/clang-ubuntu:13
```

3. В macOS / Linux: запустите контейнер Docker следующей командой:

```
docker run --rm -it -v "$(pwd)":/usr/src teeks99/clang-ubuntu:13
```

В этих командах:

- ♦ `--rm` очищает ресурсы контейнера, когда вы его закрываете.
- ♦ `--it` запускает контейнер в интерактивном режиме, поэтому вы можете вводить команды для изменения папок, а также для компиляции и запуска программ с помощью `clang++`.
- ♦ `-v "%CD%":/usr/src` (Windows) или `-v "$(pwd)":/usr/src` (macOS/Linux) позволяет контейнеру Docker получить доступ к файлам в папке, из которой выполняется команда `docker run`. В контейнере Docker вы перейдете с помощью команды `cd` к папке `/usr/src`, чтобы компилировать и запускать примеры из книги. Другими словами, ваша локальная системная папка будет сопоставлена с папкой `/usr/src` в контейнере Docker.
- ♦ `teeks99/clang-ubuntu:13` — это имя контейнера.

Когда контейнер будет запущен, вы увидите примерно такое приглашение:

```
root@9753bace2e87:/#
```

В контейнере работает операционная система Linux. В приглашении контейнера имя папки, в которой вы находитесь, отображается между символами `:` и `#`.

Переход к папке `ch01` в контейнере Docker

Введенная вами команда `docker run` присоединила вашу папку `examples` к папке `/usr/src` файловой системы контейнера. Теперь в контейнере Docker с помощью команды `cd` перейдите к папке `ch01`, которая находится в `/usr/src`:

```
root@9753bace2e87:/# cd /usr/src/ch01
root@9753bace2e87:/usr/src/ch01#
```

Компиляция приложения

Перед запуском приложения надо его скомпилировать. Например, если в контейнере установлен компилятор `clang++-13`, выполните следующую команду:

```
clang++-13 -std=c++20 GuessNumber.cpp -o GuessNumber
```

В этой команде:

- ♦ Параметр `-std=c++20` указывает компилятору, что мы используем C++20.

¹ Может появиться уведомление с просьбой разрешить контейнеру Docker доступ к файлам в текущей папке. Вам надо дать это разрешение, иначе вы не сможете получить доступ к нашим файлам исходного кода из Docker.

- ♦ Параметр `-o` присваивает исполняемому файлу имя (`GuessNumber`), которое затем будет нужно для запуска программы. Если вы не включите этот параметр, `clang++` автоматически присвоит исполняемому файлу имя `a.out`.

Запуск приложения

Для запуска и тестирования приложения `GuessNumber` в контейнере `Docker` выполните шаги 3–7 из раздела 1.2.3 («Компиляция и запуск приложения в Linux с помощью `g++`»).

Завершение работы контейнера Docker

Чтобы завершить работу контейнера, нажмите `Ctrl + d` в его командной строке.

1.3. Закон Мура, многоядерные процессоры и многопоточное программирование

Многие современные персональные компьютеры могут выполнять миллиарды вычислительных операций в секунду — больше, чем человек способен выполнить за всю жизнь. Суперкомпьютеры уже выполняют тысячи триллионов (квадриллионы) операций в секунду. Японский суперкомпьютер `Fugaku` может выполнять более 442 квадриллионов операций в секунду (442.01 петафлопс)¹. Чтобы вы представили себе этот масштаб, поясним, что `Fugaku` может за одну секунду выполнить около 40 миллионов вычислений для каждого человека на планете! И пределы возможного для суперкомпьютеров быстро растут.

Закон Мура

Вы привыкли, что многие товары и услуги каждый год немного дорожают. Но в сфере компьютеров и связи все наоборот, особенно если говорить об аппаратном обеспечении (`hardware`). За много лет оно стало значительно доступнее.

На протяжении десятилетий вычислительная мощность компьютеров удваивалась примерно каждые два года. Это впечатляющее явление часто называют законом Мура — в честь Гордона Мура, соучредителя корпорации `Intel`, указавшего на эту тенденцию еще в 1960-х. `Intel` является ведущим производителем процессоров для компьютеров и встраиваемых систем (так называются электронные компоненты умной бытовой техники, домашних систем безопасности, роботов, интеллектуальных транспортных развязок и т. д.). Закон Мура и другие похожие тенденции больше всего проявляются в следующих характеристиках компьютеров:

- ♦ объемы памяти (оперативной памяти, в которой выполняются программы и обрабатываются данные);
- ♦ объемы накопителей (`HDD`, `SSD` и других дисков, на которых хранятся программы и данные);

¹ «Top500». Был доступен: 15 сентября 2023 г. <https://ru.wikipedia.org/wiki/Top500>.

- ♦ тактовая частота процессоров (количество инструкций, последовательно выполняемых в единицу времени).

Топ-менеджеры компаний NVIDIA и Arm, производящих процессоры, считают, что закон Мура больше не действует^{1, 2}. Точнее, вычислительная мощность компьютеров по-прежнему растет, но теперь уже в результате новых конструктивных решений, таких как многоядерные процессоры.

Многоядерные процессоры и быстродействие

Почти в каждом современном компьютере есть **многоядерный процессор (multi-core processor)**, экономично объединяющий несколько вычислительных блоков (ядер) на одном кристалле большой микросхемы. Например, 2-ядерный процессор содержит два вычислительных блока, 4-ядерный — четыре, 8-ядерный — восемь. В основном тестовом компьютере авторов книги установлен 8-ядерный процессор Intel. Новые процессоры Apple M1 Pro и M1 Max имеют 10 ядер, в дополнение к которым в топовом процессоре M1 Pro есть 16-ядерный видеопроцессор, а в топовом процессоре M1 Max — 32-ядерный; кроме того, в каждом из них есть 16 ядер системы «Neural Engine», предназначенной для машинного обучения^{3, 4}. Компания Intel выпускает 72-ядерные процессоры⁵ и проектирует 80-ядерные⁶. Компания AMD проектирует модели, имеющие 192 и 256 ядер⁷. Количество ядер будет продолжать расти.

В многоядерной системе несколько вычислительных блоков могут одновременно выполнять разные части текущей задачи, ускоряя работу вашей программы. Чтобы воспользоваться всеми преимуществами многоядерной архитектуры, вам надо научиться писать многопоточные приложения. Когда приложение распределяет задачи между отдельными потоками, многоядерная система может запускать эти потоки параллельно, если доступно достаточное количество ядер.

¹ «Moore's Law Turns 55: Is It Still Relevant?». Был доступен: 15 сентября 2023 г. <https://www.techrepublic.com/article/moores-law-turns-55-is-it-still-relevant>.

² «Moore's Law Is Dead: Three Predictions About the Computers of Tomorrow». Был доступен: 15 сентября 2023 г. <https://www.techrepublic.com/article/moores-law-is-dead-three-predictions-about-the-computers-of-tomorrow>.

³ Джули Кловер (Juli Clover), «Apple's M1 Pro Chip: Everything You Need to Know», 3 ноября 2021 г. Был доступен: 15 сентября 2023 г. <https://www.macrumors.com/guide/m1-pro>.

⁴ «Apple M1». Был доступен: 15 сентября 2023 г. https://ru.wikipedia.org/wiki/Apple_M1.

⁵ «Intel® Xeon Phi™ Processors». Был доступен: 15 сентября 2023 г. <https://ark.intel.com/content/www/us/en/ark/products/series/132784/intel-xeon-phi-72x5-processor-family.html>.

⁶ Антон Шилов (Anton Shilov), «Intel's Sapphire Rapids Could Have 72 - 80 Cores, According to New Die Shots», 30 апреля 2021 г. Был доступен: 15 сентября 2023 г. <https://www.tomshardware.com/news/intel-sapphire-rapids-could-feature-80-cores>.

⁷ Джоэл Груска (Joel Hruska), «Future 256-Core AMD Epyc CPU Might Sport Remarkably Low 600W TDP», 1 ноября 2021 г. Был доступен: 15 сентября 2023 г. <https://www.extremetech.com/computing/328692-future-256-core-amd-epyc-cpu-might-sport-remarkably-low-600w-tdp>.

По мере распространения многоядерных систем интерес к многопоточности быстро растет. Стандартные средства многопоточности в языке C++ стали одним из важнейших обновлений, появившихся в версии C++11. Каждый очередной стандарт C++ добавлял возможности более высокого уровня, упрощающие разработку многопоточных приложений. Глава 17 рассказывает о том, как писать на C++ многопоточные приложения. В главе 18 представлены корутины (C++20), позволяющие писать многопоточные приложения в простом стиле последовательного кодирования.

1.4. Что такое объектно-ориентированное программирование?

Быстро, безошибочно и экономично писать программы — это очень трудная задача, тем более что сейчас быстро растет спрос на новое, все более мощное программное обеспечение. Программы состоят из многократно используемых компонентов, которые называются **объектами (objects)**, а точнее говоря (как мы покажем в главе 9), **классами (classes)**, из которых программист создает эти объекты. Вот типичные примеры: объект-автомобиль, объект-человек, объект-изображение, объект-звук, объект-дата, объект-время и т. д. Почти любое существительное («автомобиль», «человек», «изображение»...) можно логично представить в виде объекта, имеющего **атрибуты (attributes)** и **поведение (behaviors)**. Примеры атрибутов: название, размер, цвет и т. д. Примеры поведения: перемещение, общение, вычисление и т. д. Программисты выяснили, что модульный, объектно-ориентированный подход к проектированию и внедрению программного обеспечения делает их рабочие группы гораздо более продуктивными, чем при работе прежними методами. Дело в том, что объектно-ориентированные программы, как правило, легче понимать, исправлять и модифицировать.

Автомобиль как объект

Начнем с простой аналогии. Предположим, вы управляете автомобилем и хотите заставить его двигаться быстрее, нажав на педаль газа. Что должно произойти, прежде чем вы сможете это сделать? Сначала кто-то должен спроектировать этот автомобиль. Его проектируют на инженерных чертежах, напоминающих проект планировки квартиры. Чертежи подробно описывают, как устроена педаль газа. Она скрывает от водителя сложные механизмы, заставляющие автомобиль двигаться быстрее. Педаль тормоза скрывает механизмы, замедляющие ход автомобиля. Руль скрывает механизмы, поворачивающие колеса автомобиля. Все это позволяет людям, практически ничего не знающим о работе двигателя, тормозных и рулевых механизмов, легко управлять автомобилем.

Когда автомобиль спроектирован, его надо изготовить по инженерным чертежам. В готовом автомобиле есть педаль газа, ускоряющая движение. Но автомобиль, к счастью, не разгоняется сам по себе, поэтому в дополнение к педали нужен еще и водитель, который будет нажимать на нее в нужный момент.

Функции и классы

Воспользуемся этим примером с автомобилем, чтобы представить вам несколько ключевых концепций объектно-ориентированного программирования. Для выполнения любой задачи в программе должна быть соответствующая функция. Функция содержит все инструкции, выполняющие эту задачу. Они скрыты от использующего функцию программиста — точно так же, как педаль газа скрывает от водителя механизмы, заставляющие автомобиль ехать быстрее. В C++ мы обычно создаем фрагмент программы, который называется **классом (class)**, и размещаем в нем весь набор функций, выполняющих задачи этого класса, — они называются **функциями класса**, или **функциями-элементами**, или **функциями-членами (member functions)**. Например, класс, представляющий банковский счет, может содержать: 1) функцию для внесения денег на счет; 2) функцию для снятия денег со счета; 3) функцию для запроса баланса счета. Класс подобен набору инженерных чертежей автомобиля, описывающих устройство его «функций»: педалей, руля и других узлов.

Создание экземпляра класса (инстанцирование)

Кто-то должен создать по чертежам хотя бы один экземпляр автомобиля, прежде чем вы сможете управлять им. Аналогично вы должны создать объект класса, чтобы ваша программа могла использовать функции этого класса для выполнения задач. Это называется **созданием экземпляра**, или **инстанцированием (instantiation)** класса, то есть созданный объект представляет собой **экземпляр (instance)** своего класса.

Повторное использование

Один и тот же набор чертежей можно повторно использовать для создания большого количества автомобилей. И точно так же вы можете **повторно использовать (reuse)** один и тот же класс для создания большого количества объектов. Повторное использование существующих классов для создания новых классов и целых программ экономит время и упрощает работу, а цифровые системы в результате становятся надежнее и эффективнее, поскольку многие существующие классы и компоненты тщательно протестированы, отлажены и оптимизированы по скорости работы. Подобно взаимозаменяемым деталям машин, которые имели решающее значение для промышленной революции, многократно используемые классы имели решающее значение для революции программного обеспечения, вызванной объектной технологией.

Сообщения и вызовы функций

Нажимая на педаль газа, вы отправляете сообщение автомобилю о том, что надо выполнить задачу: «двигаться быстрее». А при программировании вы отправляете сообщения объектам. Каждое сообщение реализуется как **вызов функции класса (member-function call)**. Такой вызов означает, что функция класса должна выполнить свою задачу. Например, банковская программа может обратиться к объекту, представляющему счет конкретного лица, чтобы вызвать функцию внесения денег на счет, увеличив его баланс на определенную сумму.

Атрибуты и элементы данных

Кроме функций, выполняющих задачи, у автомобиля есть атрибуты: цвет, количество дверей, количество бензина в баке, текущая скорость, пробег (показания одометра) и т. д. Не только функции для выполнения задач, но и атрибуты тоже заданы в чертежах (в том числе начальные нулевые значения для одометра и датчика расхода топлива). Когда вы создаете автомобиль, эти атрибуты «появляются» вместе с ним. Каждый автомобиль «видит» только свои собственные атрибуты. Например, он «видит», сколько бензина в его бензобаке, но «не видит», сколько бензина в баках других автомобилей.

Объект тоже имеет свои атрибуты и сохраняет их, пока используется в программе. Эти атрибуты заранее заданы в классе объекта (напомним, что объект — это экземпляр класса). Например, объект конкретного банковского счета имеет атрибут баланса, представляющий количество денег на этом счете. Каждый такой объект «видит» баланс на том счете, который он представляет, но «не видит» баланс других счетов. Атрибуты хранятся в **элементах данных (data members)** класса.

Инкапсуляция

Классы скрывают от доступа извне, или **инкапсулируют (encapsulate)**, атрибуты и функции объектов, созданных из этих классов. Атрибуты и функции каждого объекта тесно связаны. Объекты могут взаимодействовать друг с другом, но обычно объекту не разрешается «видеть» внутреннее устройство других объектов. Все такие детали скрыты внутри объектов. Это **сокрытие данных (information hiding)**, как мы скоро покажем, необходимо для разработки надежных, качественных программ.

Наследование

Новый класс объектов можно быстро и удобно создать путем **наследования (inheritance)**. В новом классе при этом сохраняются все свойства существующего (родительского) класса, но они могут быть доработаны программистом под конкретные задачи. Кроме того, в новый класс можно добавить и другие свойства, уникальные для этого класса. В нашей автомобильной аналогии объект класса «кабриолет» является также и объектом более общего класса «автомобиль», но со своим уникальным свойством: крыша поднимается и опускается.

Объектно-ориентированный анализ и проектирование

Скоро вы начнете программировать на C++. Как вы будете создавать **исходный код (source code)** своей программы? Возможно, как и многие программисты, вы просто включите свой компьютер и начнете печатать. Но этот подход годится только для небольших программ (например, тех, которые вы найдете в первых главах книги). А если вас попросят создать систему управления тысячами банкоматов для крупного банка? Или если вам предложат поработать в команде из нескольких тысяч разработчиков, создающих новое поколение системы управления авиаперевозками в США? В таких больших и сложных проектах нельзя сразу садиться за компьютер и писать первые строки кода.

Чтобы найти наилучшие решения, вы должны сначала детально проанализировать предстоящий проект, сформулировать требования для него (определить, что именно должна делать система) и затем спроектировать архитектуру, удовлетворяющую этим требованиям (определить, как именно система должна это делать). В идеале вы должны пройти через этот процесс и тщательно проверить всю вашу будущую архитектуру (и обратиться к другим специалистам по программному обеспечению, чтобы они тоже ее проверили) еще до того, как вы приступите к написанию первых строк кода. Если вы выполняете анализ и проектирование системы с объектно-ориентированной точки зрения, это называется **объектно-ориентированным анализом и проектированием** (**object-oriented analysis and design**), или сокращенно ООАП. C++ относится к объектно-ориентированным языкам. Программирование на таком языке называется **объектно-ориентированным программированием** (**object-oriented programming**), или сокращенно ООП, и позволяет реализовать объектно-ориентированную архитектуру в виде работающей системы.

1.5. Итоги

В этой главе вы узнали, как компилировать и запускать приложения, используя три компилятора: Visual C++ из Microsoft Visual Studio (в Windows), Clang C++ из Xcode (в macOS) и g++ из GNU Compiler Collection (в Linux). Мы рассказали о ресурсе Microsoft для установки Ubuntu Linux в подсистеме Windows для Linux, позволяющей запускать g++ в Windows. Мы также продемонстрировали работу с кросс-платформенными контейнерами Docker, позволяющими запускать новейшие версии g++ и clang++ в Windows, macOS и Linux.

Мы назвали несколько ресурсов для изучения истории и архитектуры C++, в том числе ресурсы, предоставленные Бьерном Страуструпом, создателем C++. Мы обсудили закон Мура, многоядерные процессоры и важную роль конкурентного программирования для использования их преимуществ. Наконец, мы предоставили краткий обзор концепций и терминов объектно-ориентированного программирования, которые будут встречаться вам на протяжении всей книги.

В следующей главе мы покажем, как программировать на C++, используя базовые инструкции ввода и вывода, фундаментальные типы данных, арифметические операции и инструкции принятия решений. Кроме того, мы продемонстрируем удобство применения готовых объектов на примере класса `string` стандартной библиотеки C++.



2

Азы программирования на C++

В этой главе вы:

- напишете простейшие приложения на C++;
- научитесь использовать инструкции ввода и вывода;
- познакомитесь с основными типами данных;
- научитесь использовать арифметические операторы;
- узнаете о приоритетах арифметических операторов;
- научитесь использовать инструкции принятия решений;
- научитесь использовать операторы равенства и сравнения;
- начнете работать с объектами класса `string` стандартной библиотеки C++.

2.1. Введение

В этой главе представлено несколько примеров исходного кода, которые показывают, как выводить на экран текст и получать от пользователя данные для последующей обработки. В первых трех примерах наши программы выводят текстовые сообщения. Четвертый пример знакомит вас с арифметическими операторами C++: программа получает два числа, введенных с клавиатуры, вычисляет их сумму и выводит результат. Пятый пример демонстрирует процесс принятия решений: программа сравнивает два числа, а затем выводит сообщения в зависимости от результатов сравнения.

Учебный метод готовых объектов

В своих программах вы будете создавать и использовать множество объектов из общедоступных классов, которые тщательно разработаны и протестированы. Они позволяют вам выполнять сложные задачи в минимальном объеме кода. Эти классы находятся:

- ♦ в стандартной библиотеке C++;
- ♦ в библиотеках для определенных платформ (например, в библиотеке Microsoft, предназначенной для разработки приложений Windows, или в библиотеке Apple, предназначенной для разработки приложений macOS);
- ♦ в бесплатных сторонних библиотеках с открытым исходным кодом, созданных сообществами программистов (такие сообщества поддержки есть у всех основных современных языков программирования).

Мы поможем вам быстро оценить все преимущества этого метода программирования. Перед тем как вы напишете ваши первые классы самостоятельно, вы сможете легко создавать и использовать объекты из классов стандартной библиотеки C++. Мы называем это методом готовых объектов (the Objects Natural approach). В конце этой главы вы создадите и проверите в работе несколько объектов класса `string`. Во второй половине книги вы приступите к созданию своих собственных классов и увидите, что C++ позволяет создавать классы и для ваших индивидуальных целей, и для широкого использования в сообществах программистов.

Компиляция и запуск программ

В главе 1 (в разделах 1.2.1–1.2.5) приведены инструкции по компиляции и запуску программ с помощью Microsoft Visual Studio, Apple Xcode и GNU g++. Кроме того, мы приготовили для вас видеoinструкции на сайте издательства Deitel:

<http://deitel.com/c-plus-plus-20-for-programmers>

2.2. Первая программа на C++: вывод текста на экран

Рассмотрим простую программу, выводящую на экран текстовое сообщение (илл. 2.1). Номера строк не являются частью программы.

```

1 // fig02_01.cpp
2 // Программа, выводящая сообщение
3 #include <iostream> // Этот заголовок нужен для вывода данных на экран
4
5 // Выполнение программы начинается с функции main
6 int main() {
7     std::cout << "Welcome to C++!\n"; // Выводим сообщение
8
9     return 0; // Указываем, что программа завершилась успешно
10 } // Конец функции main

```

Welcome to C++!

Илл. 2.1 | Программа, выводящая сообщение

Комментарии

Строки 1 и 2:

```

// fig02_01.cpp
// Программа, выводящая сообщение

```

начинаются с двух символов слеша `//`. Они означают, что вся последующая часть строки — это игнорируемый компилятором **комментарий (comment)**. В комментарии первой строки указано имя файла, как и во всех остальных наших программах. Комментарий во второй строке описывает назначение программы. Комментарии, начинающиеся с символов `//`, называются **однострочными (single-line)** и заканчиваются в конце строки. Вы можете добавлять в исходный код не только однострочные, но и **многострочные (multiline)** комментарии, помещая их между символами `/*` и `*/`, как в следующих примерах:

```
/* fig02_01.cpp: программа, выводящая сообщение */
```

или:

```

/* fig02_01.cpp
Программа, выводящая сообщение */

```

Директива препроцессора `#include`

Строка 3:

```
#include <iostream> // Этот заголовок нужен для вывода данных на экран
```

это **директива препроцессора (preprocessing directive)**, то есть сообщение препроцессору C++, который вызывается компилятором перед началом компиляции программы. Директива требует от препроцессора включить в программу содержимое **заголовочного файла (header file)**, или сокращенно **заголовка (header)** `<iostream>`. Этот файл содержит информацию, необходимую для компиляции любой программы, которая выводит данные на экран или получает данные с клавиатуры, используя потоки **ввода-вывода (input/output)** C++. Программа, показанная на илл. 2.1, выводит на экран текстовые данные. Заголовочные файлы детально обсуждаются в главе 5.

Пустые строки и пробелы

Строка 4 — это просто пустая строка. Для удобства чтения исходного кода вы можете вставлять в него пустые строки, пробелы и символы табуляции. Их общее название — **символы пропуска (whitespaces)**, они игнорируются компилятором.

Функция `main`

Строка 6:

```
int main() {
```

это неотъемлемая часть каждой программы на C++. Круглые скобки после `main` указывают на то, что это **функция (function)**. Программа на C++ обычно состоит из многих функций и классов. Но в каждой программе одна «главная» функция должна называться `main` — именно с нее начинается выполнение программы на C++. **Ключевое слово (keyword)** `int` указывает, что после своего завершения функция возвращает целочисленное значение (целое число). Ключевые слова — это лексические единицы языка C++, у каждого из них есть свое назначение. В главе 3 приведен полный список ключевых слов C++. В главе 5 мы объясним, что значит «возвращать значение», когда покажем вам, как создавать ваши собственные функции. А пока просто добавляйте ключевое слово `int` слева от имени `main` в каждой из ваших программ.

Левая фигурная скобка (`{` в конце строки 6) должна начинать **тело (body)** каждой функции, то есть последовательность выполняемых функцией инструкций. Соответствующая правая фигурная скобка (`}` в строке 10) должна заканчивать тело каждой функции.

Инструкция вывода

Строка 7:

```
std::cout << "Welcome to C++!\n"; // Выводим сообщение
```

выводит на экран все символы, которые находятся между открывающими и закрывающими кавычками. Кавычки и символы между ними называются **строкой (string)**, или **строкой символов (character string)**, или **строковым литералом (string literal)**. Как правило, мы будем называть символы, заключенные в двойные кавычки, просто строками¹. Символы пропуска в строках не игнорируются компилятором.

В нашей программе вся строка 7 (кроме комментария), включая `std::cout`, **оператор (operator)** `<<`, строку `"Welcome to C++!\n"` и завершающую точку

¹ Словом «string» в английском языке называют строку символов, а словом «line» — строку исходного кода программы. Оба слова переводятся на русский язык одинаково: «строка», поэтому для правильного понимания важен контекст. В этой книге строки кода пронумерованы, а строки символов заключены в кавычки или обозначены именами, поэтому различия между ними очевидны. — *Примеч. ред.*

с запятой (;), — это **инструкция (statement)**¹. Почти все инструкции C++ заканчиваются точкой с запятой. Отсутствие точки с запятой в конце инструкции, когда она необходима, — это синтаксическая ошибка. Директивы препроцессора (такие, как `#include`) не являются инструкциями C++ и не заканчиваются точкой с запятой.

Обычно вывод и ввод в C++ реализуются с помощью потоков (**streams**). Когда в строке 7 выполняется инструкция, она отправляет всю последовательность символов `"welcome to C++!\n"` (кроме кавычек) **стандартному объекту потока вывода (standard output stream object)** `std::cout`, который «подключен» к экрану.

Отступы для наглядности кода

Когда вы пишете код программы, делайте отступ в теле каждой функции на одном уровне с фигурными скобками, ограничивающими тело. Это структурирует код и облегчает его понимание. Выберите на свое усмотрение длину отступа (количество последовательных пробелов) и соблюдайте эту длину во всей программе. Для отступов можно использовать и клавишу табуляции, но не забывайте, что длина шагов табуляции может различаться. Мы предпочитаем три пробела на каждый шаг табуляции.

Пространство имен std

Обозначение `std::` перед именем `cout` указывает компилятору, что это имя находится в заголовке стандартной библиотеки C++ (в данном случае — `<iostream>`). Иными словами, запись `std::cout` означает, что имя `cout` находится в **пространстве имен (namespace) std**². Имена объектов `cin` (стандартный поток ввода) и `cerr` (стандартный поток ошибок) тоже находятся в пространстве имен `std`. Мы будем обсуждать пространство имен в главе 16. А пока просто не забывайте добавлять `std::` перед именами `cout`, `cin` и `cerr`. Скоро мы покажем вам еще два способа указывать пространство имен: с помощью объявлений `using` и директивы `using`.

Оператор вывода в поток и escape-последовательности

В инструкции `cout`³ оператор `<<` используется в качестве **оператора вывода в поток (stream insertion operator)**. При выполнении программы значение справа от оператора, то есть правый **операнд (operand)**, отправляется в поток вывода.

¹ Не следует путать термины «statement» («инструкция») и «operator» («оператор»). В этом примере оператор `<<` входит в состав инструкции. Список операторов языка C++ приведен в конце книги, в приложении А. В некоторых языках программирования инструкции также называют командами. — *Примеч. ред.*

² Это сокращение от слова «standard».

³ Инструкцией языка C++ является не отдельное слово «cout», а вся строка 7 (кроме комментария). И в русском, и в английском языке такие словосочетания, как «инструкция cout» («cout statement»), «инструкция if» («if statement») и т. п., являются общепринятыми сокращениями, означающими: «инструкция с ключевым словом cout», «инструкция с ключевым словом if» и т. п. — *Примеч. ред.*

Обратите внимание, что оператор << «указывает» в ту сторону, куда отправляются данные. Символы в строках обычно отображаются так же, как они написаны между двойными кавычками. Но в выведенном на экран сообщении (илл. 2.1) нет символов \n. Обратный слеш (\) называется **экранирующим символом**, или **escape-символом (escape character)**, и служит для вывода специальных символов. Когда в строке встречается обратный слеш, он объединяется со следующим символом, образуя вместе с ним **экранированную последовательность**, или **escape-последовательность (escape sequence)**. Escape-последовательность \n означает **перевод строки (newline)**, то есть перемещает курсор (индикатор текущего положения на экране) в начало следующей строки. Некоторые важные escape-последовательности показаны в таблице.

Escape-последовательность	Описание
\n	Перевод строки (newline). Перемещает экраный курсор в начало следующей строки
\t	Горизонтальная табуляция (horizontal tab). Перемещает экраный курсор в следующую позицию табуляции
\r	Возврат каретки (carriage return). Перемещает экраный курсор в начало текущей строки (без перехода к следующей строке)
\a	Предостережение (alert). Воспроизводит системный звук
\\	Обратный слеш (backslash). Включает в строку символ обратного слеша
\'	Одиночная кавычка (single quote). Включает в строку символ одиночной кавычки
\"	Двойная кавычка (double quote). Включает в строку символ двойной кавычки

Инструкция return

Строка 9:

```
return 0; // Указываем, что программа завершилась успешно
```

это один из нескольких способов, которые мы будем использовать для **выхода из функции (exit a function)**. Если в конце функции main есть инструкция return со значением 0, это указывает на успешное завершение программы. Впрочем, если в процессе выполнения функция main достигает закрывающей фигурной скобки без инструкции return, компилятор рассматривает это как возврат 0 и предполагает, что программа завершилась успешно. В последующих примерах мы будем опускать return в функциях main тех программ, которые завершаются успешно.

2.3. Модификация нашей первой программы на C++

Следующие два примера модифицируют программу, показанную на илл. 2.1. Первый пример выводит текст одной строкой с помощью нескольких инструкций. Второй — выводит текст в несколько строк с помощью одной инструкции.

Вывод одной строки текста несколькими инструкциями

На илл. 2.2 показан пример вывода в поток двумя последовательными инструкциями (строки 7–8). При этом на экране отобразится то же самое, что на илл. 2.1. Каждый раз, когда вы выводите данные в поток, символы «печатаются» на экране с того места, где завершился предыдущий вывод. В строке 7 выводится слово `Welcome` с последующим пробелом. После пробела нет escape-последовательности `\n` (перевод строки), поэтому инструкция в строке 8 сразу же «допечатывает» окончание фразы.

```
1 // fig02_02.cpp
2 // Вывод одной строки текста несколькими инструкциями
3 #include <iostream> // Этот заголовок нужен для вывода данных на экран
4
5 // Выполнение программы начинается с функции main
6 int main() {
7     std::cout << "Welcome ";
8     std::cout << "to C++!\n";
9 } // Конец функции main
```



Илл. 2.2 | Вывод одной строки текста несколькими инструкциями

Вывод нескольких строк текста одной инструкцией

Одна инструкция может выводить текст в несколько строк («в столбик»), если включить в нее дополнительные escape-последовательности `\n`, как в строке 7 на илл. 2.3. Каждый раз, когда в потоке вывода встречается `\n`, экранный курсор перемещается в начало следующей строки. Чтобы получить пустую строку, выведите `\n` два раза подряд.

```
1 // fig02_03.cpp
2 // Вывод нескольких строк текста одной инструкцией
3 #include <iostream> // Этот заголовок нужен для вывода данных на экран
4
5 // Выполнение программы начинается с функции main
6 int main() {
7     std::cout << "Welcome\n\nto\n\nC++!\n";
8 } // Конец функции main
```



Илл. 2.3 | Вывод нескольких строк текста одной инструкцией

2.4. Вторая программа на C++: сложение целых чисел

Наша следующая программа получает два целых числа с клавиатуры, вычисляет их сумму и выводит результат на экран через `std::cout`. На илл. 2.4 вы видите исходный код программы и пример ввода и вывода. Вводимые пользователем данные выделены **жирным шрифтом**.

```
1 // fig02_04.cpp
2 // Программа, выводящая сумму двух целых чисел
3 #include <iostream> // Этот заголовок нужен для ввода и вывода данных
4
5 // Выполнение программы начинается с функции main
6 int main() {
7     // Объявляем и инициализируем переменные
8     int number1{0}; // Первая переменная типа int (начальное значение: 0)
9     int number2{0}; // Вторая переменная типа int (начальное значение: 0)
10    int sum{0}; // Сумма number1 и number2 (начальное значение: 0)
11
12    std::cout << "Enter first integer: "; // Ждем ввода данных
13    std::cin >> number1; // Считываем первое число и помещаем его в number1
14
15    std::cout << "Enter second integer: "; // Ждем ввода данных
16    std::cin >> number2; // Считываем второе число и помещаем его в number2
17
18    sum = number1 + number2; // Суммируем числа и записываем результат в sum
19
20    std::cout << "Sum is " << sum << "\n"; // Выводим результат
21 } // Конец функции main
```

```
Enter first integer: 45
Enter second integer: 72
Sum is 117
```

Илл. 2.4 | Программа, выводящая сумму двух целых чисел

Объявления переменных с инициализацией

Строки 8–10:

```
int number1{0}; // Первая переменная типа int (начальное значение: 0)
int number2{0}; // Вторая переменная типа int (начальное значение: 0)
int sum{0}; // Сумма number1 и number2 (начальное значение: 0)
```

содержат **объявления (declarations)** трех **переменных (variables)** с именами `number1`, `number2` и `sum`. Каждая из этих переменных объявляется с указанием того, что она имеет **тип данных (data type)** `int` (имя этого типа является сокращением от слова «integer» — «целое число»). В переменных типа `int` можно хранить только целые числа, например 7, -11, 0 или 31914. В языке C++ каждую переменную обязательно надо объявлять, указывая ее имя и тип данных.

В строках 8–10 переменные не только объявляются, но и инициализируются значением 0 (то есть каждой из них присваивается значение 0). Обратите внимание, что значение должно быть указано в фигурных скобках { } сразу после имени переменной. Эта форма записи называется **скобочной инициализацией (braced initialization)**, она была введена в C++11. Хотя не всегда необходимо явно инициализировать каждую переменную, запомните, что инициализация поможет вам избежать многих проблем!

До C++11 строки 8–10 выглядели бы так:

```
int number1 = 0; // Первая переменная типа int (начальное значение: 0)
int number2 = 0; // Вторая переменная типа int (начальное значение: 0)
int sum = 0; // Сумма number1 и number2 (начальное значение: 0)
```

С этим традиционным способом инициализации вы встретитесь, если будете изучать исходные коды старых программ на C++. У скобочной инициализации есть ряд преимуществ, которые мы обсудим в последующих главах.

Одновременное объявление нескольких переменных

Несколько переменных одного типа разрешается объявлять в одной инструкции. Например, вы могли бы объявить и инициализировать все три переменные следующим образом:

```
int number1{0}, number2{0}, sum{0};
```

Но эта форма записи обычно делает исходный код менее наглядным и затрудняет добавление комментариев, поясняющих смысл каждой переменной.

Основные типы данных

Скоро мы рассмотрим тип `double`, предназначенный для вещественных чисел, и тип `char`, предназначенный для символьных данных. Вещественные числа — это числа с десятичной точкой¹, например 3.4, 0.0 или -11.19. Переменная типа `char` может содержать только одну строчную букву, или прописную букву, или цифру, или специальный символ (например, \$ или *). Типы данных `int`, `double`, `char` и `long long` относятся к **основным**, или **фундаментальным**, типам (**fundamental types**) языка C++. Имена основных типов данных состоят из одного или нескольких ключевых слов и пишутся строчными буквами. Здесь вы найдете полный список основных типов данных языка C++ и их допустимых диапазонов:

<https://en.cppreference.com/w/cpp/language/types>

Идентификаторы и имена в стиле «верблюжий регистр»

Вы можете давать переменным любые имена, которые являются допустимыми идентификаторами (например, `number1`). Идентификатор (**identifier**) — это послед-

¹ В русском издании книги сохранен американский стандарт записи чисел, в котором десятичным разделителем является точка, а не запятая. — *Примеч. ред.*

довательность символов, состоящая из букв, цифр и символов подчеркивания (`_`), которая не начинается с цифры и не является ключевым словом. Синтаксис языка C++ **чувствителен к регистру (case sensitive)**: прописные и строчные буквы различаются, то есть `a1` и `A1` — это разные идентификаторы.

C++ поддерживает идентификаторы любой длины. Не начинайте идентификатор с символа подчеркивания и последующей заглавной буквы (`_A`, `_B`, `_C...`), или с двух символов подчеркивания (`__`). Компиляторы C++ используют такие имена для собственных нужд.

По общепринятому соглашению, имя каждой переменной должно начинаться со строчной буквы, а каждое последующее слово в этом имени должно начинаться с заглавной буквы. Например, вы можете назвать переменную именем `firstNumber`, в котором второе слово `Number` пишется с заглавной буквы `N`. Это соглашение об именах известно как **«верблюжий регистр» (camel case)**, потому что заглавные буквы выделяются подобно горбам верблюда: `camelCase`.

Размещение объявлений переменных

Переменную можно объявить практически в любом месте программы, но обязательно до того, как вы впервые обратитесь к этой переменной (перед тем, как вы начнете ее использовать). Например, строку 8:

```
int number1{0}; // Первая переменная типа int (начальное значение: 0)
```

можно было бы разместить непосредственно перед строкой 13:

```
std::cin >> number1; // Считываем первое число и помещаем его в number1
```

Объявление переменной в строке 9:

```
int number2{0}; // Вторая переменная типа int (начальное значение: 0)
```

можно было бы разместить перед строкой 16:

```
std::cin >> number2; // Считываем второе число и помещаем его в number2
```

А строку 10:

```
int sum{0}; // Сумма number1 и number2 (начальное значение: 0)
```

можно было бы разместить непосредственно перед строкой 18:

```
sum = number1 + number2; // Суммируем числа и записываем результат в sum
```

Или вы могли бы объединить строки 10 и 18 в следующее объявление, размещенное непосредственно перед строкой 20:

```
int sum(number1 + number2); // Инициализируем sum суммой number1 + number2
```

Получение первого значения от пользователя

Строка 12:

```
std::cout << "Enter first integer: "; // Ждем ввода данных
```

выводит последовательность символов `Enter first integer:` с последующим пробелом. Такое сообщение называется **приглашением (prompt)**, поскольку оно показывает пользователю, что программа ждет от него нужного действия.

Строка 13:

```
std::cin >> number1; // Считываем первое число и помещаем его в number1
```

использует **стандартный объект потока ввода (standard input stream object) cin** (из пространства имен `std`) и **оператор извлечения из потока (stream extraction operator) >>** для ввода данных с клавиатуры.

После выполнения предыдущей инструкции программа ожидает ввода значения переменной `number1`. Пользователь вводит целое число (в виде символов), затем нажимает клавишу `Enter` (иногда она называется `Return`), чтобы отправить эти символы в программу. Объект `cin` преобразует символьное представление числа в целочисленное значение (`int`) и присваивает это значение переменной `number1`. Нажатие клавиши `Enter` также приводит к перемещению курсора в начало следующей строки на экране.

Когда программа ожидает ввода целого числа, пользователь может набрать на клавиатуре что-то другое: буквы, случайные символы (например, `#` или `@`) или число с десятичной точкой (например, `73.5`). Но в первых учебных программах мы исходим из того, что пользователь вводит корректные данные. Позже мы расскажем, как решать проблемы, связанные с вводом некорректных данных.

Получение второго значения от пользователя

Строка 15:

```
std::cout << "Enter second integer: "; // Ждем ввода данных
```

выводит последовательность символов `Enter second integer:` с последующим пробелом и ждет реакции пользователя.

Строка 16:

```
std::cin >> number2; // Считываем второе число и помещаем его в number2
```

получает от пользователя значение для переменной `number2`.

Вычисление суммы чисел, введенных пользователем

Выражение присваивания в строке 18:

```
sum = number1 + number2; // Суммируем числа и записываем результат в sum
```

суммирует значения переменных `number1` и `number2` и присваивает результат переменной `sum` с помощью **оператора присваивания (assignment operator) =**. Большинство вычислений выполняется в выражениях присваивания. Операторы `=` и `+` являются **бинарными (binary operators)**, поскольку каждый из них имеет два операнда. Для оператора `+` операндами являются `number1` и `number2`. Для оператора `=` операндами являются переменная `sum` и значение выражения `number1 + number2`.

Добавление пробелов до и после бинарного оператора выделяет его визуально, поэтому исходный код становится проще для чтения.

Вывод результата на экран

Строка 20:

```
std::cout << "Sum is " << sum << "\n"; // Выводим результат
```

выводит строку "Sum is", за которой следуют пробел, значение переменной `sum` и перевод строки.

Эта инструкция выводит несколько значений, имеющих разные типы. Оператор вывода в поток «знает», как надо выводить данные каждого типа. Инструкция с несколькими последовательными операторами вывода в поток (`<<`) выполняет **конкатенацию (concatenating)**, или **каскадирование (cascading)**, операций вывода.

Вычисления могут выполняться непосредственно в инструкциях вывода в поток. Мы могли бы исключить переменную `sum`, объединив строки 18 и 20:

```
std::cout << "Sum is " << number1 + number2 << "\n";
```

Очень важно, что C++ открывает перед вами возможность создания ваших собственных типов данных, называемых классами (мы обсудим их в начале главы 9). Вы можете «научить» C++ вводу и выводу значений ваших новых типов, используя операторы `>>` и `<<` соответственно. Это называется **перегрузкой операторов (operator overloading)**, мы рассмотрим ее в главе 11.

2.5. Арифметические операторы

В следующей таблице приведены **арифметические операторы (arithmetic operators)** C++:

Операция	Арифметический оператор	Примеры алгебраических выражений	Примеры выражений C++
Сложение	+	$f + 7$	<code>f + 7</code>
Вычитание	-	$p - c$	<code>p - c</code>
Умножение	*	bm $b \cdot m$	<code>b * m</code>
Деление	/	x/y $x \div y$	<code>x / y</code>
Остаток от деления	%	$r \bmod s$	<code>r % s</code>

Обратите внимание на несколько символов, которых нет в алгебре. Звездочка (*) означает умножение, а знак процента (%) — это оператор остатка от деления, который мы скоро рассмотрим. Все эти арифметические операторы являются бинарными.

Целочисленное деление

Целочисленное деление (integer division), в котором числитель и знаменатель являются целыми числами, дает в результате целочисленное частное. Например, выражение $7 / 4$ вычисляется как 1, а выражение $17 / 5$ вычисляется как 3. Любая дробная часть, оставшаяся в результате целочисленного деления, просто отбрасывается, при этом не происходит округления.

Оператор остатка от деления

Оператор `%` — это **оператор остатка от деления (remainder operator)**, также называемый **оператором модуля (modulus operator)**. Он вычисляет остаток после целочисленного деления и может работать только с целочисленными операндами, то есть результатом выражения $x \% y$ будет остаток после деления x на y . Таким образом, результат выражения $7 \% 4$ равен 3, а результат выражения $17 \% 5$ равен 2.

Круглые скобки для группировки выражений

Круглые скобки в выражениях C++ используются так же, как в алгебре. Например, чтобы умножить a на величину $b + c$, мы напишем: $a * (b + c)$.

Правила приоритета операторов

C++ определяет приоритеты операторов в арифметических выражениях по таким же **правилам приоритета (rules of precedence)**, как в алгебре:

1. Выражения в круглых скобках вычисляются первыми. Круглые скобки имеют высший приоритет. В случаях вложенных круглых скобок, например $(a * (b + c))$ выражения в самой внутренней паре круглых скобок вычисляются первыми.
2. Затем выполняются операции умножения, деления и остатка от деления. В выражениях, содержащих несколько подобных операций, эти вычисления выполняются слева направо. Операторы умножения, деления и остатка от деления имеют одинаковый приоритет.
3. Операции сложения и вычитания выполняются в последнюю очередь. Если выражение содержит несколько таких операций, то они выполняются слева направо. Операторы сложения и вычитания тоже имеют одинаковый приоритет.

В конце книги, в приложении А, приведена полная таблица приоритетов операторов. Обратите внимание: в таких выражениях, как $(a + b) * (c - d)$, где две пары круглых скобок не являются вложенными и имеют одинаковый приоритет, стандарт C++ не определяет порядок вычисления выражений в круглых скобках!

Группировка операторов

Когда мы говорим, что в языке C++ определенные операторы применяются слева направо, мы имеем в виду **группировку (grouping)** операторов, которую также называют **ассоциативностью (associativity)** операторов. Например, в выражении:

операторы сложения (+) группируются слева направо, как если бы мы добавили скобки: $(a + b) + c$. Большинство операторов C++, имеющих одинаковый приоритет, группируются слева направо. Позже мы расскажем о некоторых операторах, которые группируются справа налево.

2.6. Принятие решений: операторы равенства и сравнения

Теперь мы познакомим вас с инструкцией `if`, которая позволяет программе выполнять альтернативные действия в зависимости от того, является ли заданное условие (**condition**) истинным или ложным. Условия в инструкциях `if` могут быть сформированы с помощью перечисленных в таблице операторов равенства (**equality operators**) и операторов сравнения (**relational operators**), также называемых операторами отношения, или реляционными операторами:

Оператор равенства или сравнения в алгебре	Оператор равенства или сравнения в C++	Пример условия в C++	Смысл условия
<i>Операторы сравнения</i>			
>	>	<code>x > y</code>	x больше y
<	<	<code>x < y</code>	x меньше y
≥	>=	<code>x >= y</code>	x больше или равно y
≤	<=	<code>x <= y</code>	x меньше или равно y
<i>Операторы равенства</i>			
=	==	<code>x == y</code>	x равно y
≠	!=	<code>x != y</code>	x не равно y

Все операторы сравнения имеют одинаковый приоритет и группируются слева направо. Оба оператора равенства имеют одинаковый приоритет, который ниже приоритета операторов сравнения. Операторы равенства тоже группируются слева направо.

Если перепутать местами два символа в операторах `!=`, `>=` и `<=` (то есть написать их как `=!`, `=>` и `=<`), это обычно считается синтаксической ошибкой. В некоторых случаях запись `=!` вместо `!=` не считается синтаксической ошибкой, но почти наверняка будет логической ошибкой, которая проявится во время выполнения программы. Вы узнаете причины этого при рассмотрении логических операторов в разделе 4.11.

Принципиальное различие между операторами `==` и `=`

Если вместо оператора равенства `==` вы поставите оператор присваивания `=`, то логическая ошибка вам гарантирована. В устной речи оператор равенства обычно

называют «равно» или «двойное равно», а оператор присваивания — «получает», «получает значение» или «присваивается значение». Если перепутать эти операторы, это не обязательно приведет к легко распознаваемой синтаксической ошибке, зато может привести к незаметным с первого взгляда логическим ошибкам. Компиляторы обычно предупреждают об этом.

Инструкция if

На илл. 2.5 показано, как шесть инструкций `if` последовательно сравнивают два целых числа, введенных пользователем. Если условие в инструкции `if` истинно, то выполняется ее тело (соответствующая инструкция, заключенная в фигурные скобки). Если условие ложно, то тело инструкции не выполняется.

```
1 // fig02_05.cpp
2 // Сравнение целых чисел с помощью инструкций if,
3 // операторов равенства и сравнения
4 #include <iostream> // Этот заголовок нужен для ввода и вывода
5
6 using std::cout; // Программа использует cout
7 using std::cin; // Программа использует cin
8
9 // Выполнение программы начинается с функции main
10 int main() {
11     int number1{0}; // Первое число для сравнения (начальное значение: 0)
12     int number2{0}; // Второе число для сравнения (начальное значение: 0)
13
14     cout << "Enter two integers to compare: "; // Ждем ввода данных
15     cin >> number1 >> number2; // Считываем два введенных пользователем числа
16
17     if (number1 == number2) {
18         cout << number1 << " == " << number2 << "\n";
19     }
20
21     if (number1 != number2) {
22         cout << number1 << " != " << number2 << "\n";
23     }
24
25     if (number1 < number2) {
26         cout << number1 << " < " << number2 << "\n";
27     }
28
29     if (number1 > number2) {
30         cout << number1 << " > " << number2 << "\n";
31     }
32
33     if (number1 <= number2) {
34         cout << number1 << " <= " << number2 << "\n";
35     }
36
37     if (number1 >= number2) {
38         cout << number1 << " >= " << number2 << "\n";
```

Enter two integers to compare: 3 7

3 != 7

3 < 7

3 <= 7

Enter two integers to compare: 22 12

22 != 12

22 > 12

22 >= 12

Enter two integers to compare: 7 7

7 == 7

7 <= 7

7 >= 7

Строка 15:

```
cin >> number1 >> number2; // Считываем два введенных пользователем числа
```

использует каскадирование операций извлечения из потока, чтобы получить два числа от пользователя. Напомним, что в строке 7 было объявление `using`, поэтому теперь можно писать сокращенно: `cin` (вместо `std::cin`). Сначала считывается значение в переменную `number1`, затем — в переменную `number2`.

Сравнение чисел

Инструкция `if` в строках 17–19:

```
if (number1 == number2) {  
    cout << number1 << " == " << number2 << "\n";  
}
```

проверяет, равны ли `number1` и `number2`. Если равны, то инструкция `cout` выводит сообщение об этом. Таким же образом в остальных инструкциях `if` (начинающихся в строках 21, 25, 29, 33 и 37) инструкция `cout` выводит соответствующее сообщение, если условие окажется истинным (`true`).

Фигурные скобки и блоки

Для наглядности кода, показанного на илл. 2.5, мы поместили в **тело (body)** каждой инструкции `if` только одну инструкцию `cout`. Обратите внимание, что каждая инструкция `cout` находится между фигурными скобками `{ }`. Одна или несколько инструкций, расположенных между фигурными скобками, образуют **составную инструкцию (compound statement)**, или **блок (block)**. Если тело состоит только из одной инструкции, как в нашем примере, то эти скобки необязательны. Но во всех случаях, когда тело состоит из нескольких инструкций, их необходимо объединять в блок, то есть заключать в фигурные скобки. Если забыть о скобках, это приведет к ошибкам. Чтобы избежать ошибок, возьмите себе за правило: тело каждой инструкции `if` (даже самой простой) надо заключать в фигурные скобки.

Частая логическая ошибка: точка с запятой после условия

Если случайно поставить точку с запятой сразу после правой круглой скобки условия в инструкции `if`, то это будет логической ошибкой (но не будет синтаксической ошибкой). Точка с запятой сразу после условия означает, что у инструкции `if` пустое тело, то есть она не выполнит никаких действий — независимо от того, окажется ли ее условие истинным или ложным. А настоящее тело инструкции `if` превратится в отдельную инструкцию, то есть будет выполняться в любом случае. Это приведет к неправильной работе программы. Впрочем, большинство компиляторов при обнаружении этой логической ошибки выдают предупреждение.

Многострочная запись длинных инструкций

Длинную инструкцию можно записать в несколько строк. В таких случаях выбирайте осмысленные места переноса на следующую строку. Например, делайте

переносы после запятой в списке или после операторов в длинном выражении. Когда инструкцию записывают в несколько строк, принято делать одинаковые отступы от левого края во всех этих строках.

Приоритеты и группировка операторов

Кроме операторов присваивания (`=`), все остальные операторы, представленные в этой главе, группируются слева направо. Операторы присваивания (`=`) группируются справа налево. Например, выражение `x = y = 0` вычисляется так, как если бы в правой части были скобки: `x = (y = 0)`. То есть сначала переменной `y` присваивается значение `0`, а затем результат (`0`) присваивается переменной `x`.

Составляя выражение, содержащее много операторов, сверяйтесь с полной таблицей приоритетов в конце книги, в приложении А. Убедитесь, что операторы в выражении группируются именно в том порядке, который вам нужен. Если вы не знаете наверняка, в каком порядке будет вычисляться сложное выражение, разделите его на несколько инструкций или используйте круглые скобки, чтобы задать явный порядок вычислений, как это принято в алгебре.

2.7. Готовые объекты: создание и использование объектов класса `string` стандартной библиотеки

На страницах этой книги мы будем показывать вам, насколько полезны классы стандартной библиотеки C++ и классы многочисленных библиотек с открытым исходным кодом, созданных в сообществе C++. Вы узнаете, какие библиотеки существуют, научитесь выбирать из них необходимые для ваших приложений, будете создавать объекты из стандартных библиотечных классов и пользоваться их богатыми возможностями. Говоря о готовых объектах, мы имеем в виду, что вы сможете писать программы с профессионально разработанными объектами еще до того, как научитесь создавать свои собственные классы.

Вы уже работали с объектами C++, в частности с объектами `cin` и `cout`, которые инкапсулируют механизмы ввода и вывода соответственно. Эти объекты сконструировали для вас опытные программисты — разработчики классов, входящих в заголовок `<iostream>`. В этом разделе вы будете создавать и применять объекты класса `string` стандартной библиотеки C++¹.

Знакомство с классом `string`

Классы не могут выполняться сами по себе. Объектом «автомобиль» управляет объект «водитель», указывая ему, что надо делать (схать быстрее или медленнее, повернуть налево или направо). Для этого водителю не надо знать, как устроены внутренние механизмы автомобиля. Примерно так же главная функция `main` будет «управлять» объектом класса `string`, вызывая его функции. Для этого ей не надо

¹ В последующих главах вы узнаете о других возможностях класса `string`. В главе 8 мы подробно обсуждаем его и демонстрируем многие его функции.

знать, как устроен класс `string`. В этом смысле функцию `main` в нашем следующем примере можно назвать управляющей программой. На илл. 2.6 вы видите, как из функции `main` вызываются несколько функций класса `string`.

```

1 // fig02_06.cpp
2 // Тестирование класса string стандартной библиотеки
3 #include <iostream>
4 #include <string>
5 using namespace std;
6
7 int main() {
8     string s1{"happy"};
9     string s2{" birthday"};
10    string s3; // Создаем пустую строку
11
12    // Отображаем строки и их длину
13    cout << "s1: \"" << s1 << "\"; length: " << s1.length()
14          << "\ns2: \"" << s2 << "\"; length: " << s2.length()
15          << "\ns3: \"" << s3 << "\"; length: " << s3.length();
16
17    // Сравниваем строки операторами == и !=
18    cout << "\n\nThe results of comparing s2 and s1:" << boolalpha
19          << "\ns2 == s1: " << (s2 == s1)
20          << "\ns2 != s1: " << (s2 != s1);
21
22    // Тестируем функцию empty класса string
23    cout << "\n\nTesting s3.empty():\n";
24
25    if (s3.empty()) {
26        cout << "s3 is empty; assigning to s3;\n";
27        s3 = s1 + s2; // Записываем в s3 результат конкатенации s1 и s2
28        cout << "s3: \"" << s3 << "\"";
29    }
30
31    // Тестируем новые функции класса string (C++20)
32    cout << "\n\ns1 starts with \"ha\": " << s1.starts_with("ha") << "\n";
33    cout << "s2 starts with \"ha\": " << s2.starts_with("ha") << "\n";
34    cout << "s1 ends with \"ay\": " << s1.ends_with("ay") << "\n";
35    cout << "s2 ends with \"ay\": " << s2.ends_with("ay") << "\n";
36 }

```

```

s1: "happy"; length: 5
s2: " birthday"; length: 9
s3: ""; length: 0

```

The results of comparing s2 and s1:

```

s2 == s1: false
s2 != s1: true

```

Testing s3.empty():

```

s3 is empty; assigning to s3;
s3: "happy birthday"

```

```
s1 starts with "ha": true  
s2 starts with "ha": false  
s1 ends with "ay": false  
s2 ends with "ay": true
```

Илл. 2.6 | Тестирование класса `string` стандартной библиотеки

Создание объектов

Как правило, вы не можете использовать функции какого-либо класса, пока не создадите объект этого класса (также называемый экземпляром класса)¹. В строках 8–10 создаются три объекта класса `string`:

- ♦ объект `s1` инициализируется копией строкового литерала `"happy"`;
- ♦ объект `s2` инициализируется копией строкового литерала `" birthday"`;
- ♦ объект `s3` по умолчанию инициализируется пустой строкой `""`.

Когда мы объявляем переменные типа `int` (как мы уже делали), компилятор знает, что `int` — это один из основных (фундаментальных) типов, встроенных в C++. Но тип `string` в строках 8–10 компилятору заранее неизвестен: это **тип класса** (**class type**) стандартной библиотеки C++.

Правильно упакованные классы могут повторно использоваться другими программистами. Это одно из главных преимуществ таких объектно-ориентированных языков, как C++, расширяемых мощными библиотеками готовых классов. Например, вы можете повторно использовать все классы стандартной библиотеки C++ в любой своей программе, включив в нее соответствующие заголовки директивой `#include`. В нашем случае это заголовок `<string>` (строка 4). Имя `string`, как и имя `cout`, находится в пространстве имен `std`.

Функция `length` класса `string`

Строки 13–15 по очереди выводят на экран каждый из объектов `string` (содержащуюся в нем строку символов и ее длину). Имеющаяся в классе `string` функция `length` возвращает количество символов, хранящихся в конкретном объекте `string`. Например, в строке 13 выражение:

```
s1.length()
```

возвращает длину строки объекта `s1` через вызов функции `length`. Чтобы вызвать эту функцию для конкретного объекта, надо указать его имя (`s1`), затем оператор «точка» (`.`) и имя функции (`length`) с последующими круглыми скобками. Пустые круглые скобки означают, что функция `length` не требует никаких дополнительных данных для выполнения своей задачи. Вскоре вы увидите, что некоторым другим функциям классов для выполнения своих задач нужны дополнительные данные, называемые аргументами.

¹ В разделе 9.20 вы узнаете, как вызывать статические (`static`) функции класса, не создавая объекты этого класса.

С точки зрения главной функции `main` при вызове функции `length` происходит следующее:

1. Программа переносит выполнение из точки вызова (строка 13) в функцию `length`. Поскольку функция `length` была вызвана через объект `s1`, она «знает», что должна работать с данными именно этого объекта.
2. Функция `length` выполняет свою задачу: возвращает длину строки объекта `s1` в точку вызова (строка 13). Функция `main` не знает, как был получен этот результат. (Напомним сравнение: водитель управляет автомобилем, не зная о том, как устроены двигатель, коробка передач, рулевой механизм и тормоза.)
3. Объект `cout` выводит на экран количество символов, полученных от функции `length`, а затем программа продолжает выполнение, выводя аналогичную информацию об объектах `s2` и `s3`.

Сравнение объектов `string` операторами равенства

Как и числа, объекты `string` можно сравнивать друг с другом. В строках 18–20 объекты `s2` и `s1` сравниваются с помощью операторов равенства. Обратите внимание: при сравнениях объектов `string` регистр букв имеет значение¹.

Обычно, когда вы выводите на экран результат проверки какого-либо условия, это будет 0 (если условие ложно) или 1 (если оно истинно). Но обработчик потока `boolalpha` (строка 18) из заголовка `<iostream>` сообщает выходному потоку, что результат должен выводиться в виде слов `false` («ложно») или `true` («истинно»).

Функция `empty` класса `string`

В строке 25 вызывается функция `empty`, которая возвращает `true`, если объект `string` пустой (если длина строки символов равна 0). В противном случае функция `empty` возвращает значение `false`. Объект `s3` был по умолчанию инициализирован пустой строкой, то есть в нем нет ни одного символа. Поэтому тело инструкции `if` будет выполнено.

Конкатенация и присваивание строк

В строке 27 объекту `s3` присваивается новое значение, полученное путем «сложения» строк объектов `s1` и `s2` с помощью оператора `+`. Такое «сложение» называется **конкатенацией** (*concatenation*) строк. После конкатенации `s3` содержит строку `"happy birthday"`, то есть все символы `s1`, а затем все символы `s2`. В строке 28 выводится на экран значение `s3` для подтверждения успешной конкатенации.

Функции `starts_with` и `ends_with` класса `string` (C++20)

В строках 32–35 демонстрируются две новые функции класса `string`, введенные в C++20: `starts_with` и `ends_with`. Функция `starts_with` возвращает значение

¹ В главе 8 вы узнаете, что строки сравниваются лексикографически, то есть путем сравнения числовых значений символов, из которых они состоят.

`true`, если строка объекта `string` начинается указанной последовательностью символов. Функция `ends_with` возвращает значение `true`, если эта строка заканчивается указанной последовательностью символов. В противном случае обе функции возвращают значение `false`. При выполнении строк 32 и 33 вы видите, что `s1` начинается с символов `"ha"`, в отличие от `s2`. При выполнении строк 34 и 35 вы видите, что `s1` не заканчивается символами `"ay"`, в отличие от `s2`.

2.8. Итоги

Мы рассказали о нескольких важных базовых механизмах языка C++, включая вывод данных на экран, ввод данных с клавиатуры и объявление переменных основных типов. Теперь вы знаете, как использовать объект потока ввода `cin` и объект потока вывода `cout`, необходимые в простых интерактивных программах. Вы научились объявлять и инициализировать переменные, а также выполнять вычисления с арифметическими операторами. Мы обсудили правила приоритета операторов (порядок выполнения операций) и группировку (ассоциативность) операторов. Вы узнали, как условные инструкции `if` позволяют программам принимать решения. Мы показали, как использовать в инструкциях `if` операторы равенства и сравнения, чтобы задавать проверяемые условия.

Наконец, мы представили метод изучения C++ на основе готовых объектов: вы создали несколько объектов класса `string` стандартной библиотеки C++ и поработали с ними с помощью операторов равенства и функций класса. В последующих главах вы будете создавать и использовать много разных объектов стандартных классов, чтобы выполнять важные задачи в минимальном объеме кода. В главах 9–11 вы научитесь создавать свои собственные классы и увидите, что C++ дает вам возможность получать очень ценные, уникальные «классы ручной работы». А сейчас, в главе 3, вы познакомитесь с управляющими инструкциями, от которых зависит порядок выполнения действий в программе.

Управляющие инструкции (часть 1)

В этой главе вы:

- научитесь использовать инструкции `if` и `if...else` для выбора между альтернативными действиями;
- научитесь использовать инструкцию цикла `while` для многократного выполнения других инструкций;
- научитесь использовать циклы со счетчиком и циклы с контрольным значением;
- научитесь использовать вложенные управляющие инструкции;
- научитесь использовать составные операторы присваивания, а также операторы инкремента и декремента;
- узнаете, почему основные типы данных на разных платформах несовместимы;
- продолжите работать с готовыми объектами, оперируя с очень большими целыми числами;
- познакомитесь с синтаксисом форматирования текста в C++20: более кратким и мощным, чем в прежних версиях C++.

3.1. Введение

В этой и последующих главах мы расскажем о принципах структурного программирования. Представленные здесь концепции очень важны для создания классов и управления объектами. Мы подробно рассмотрим инструкцию `if`, а затем инструкции `if...else` и `while`. Мы также расскажем о составных операторах присваивания и об операторах инкремента и декремента.

Вы узнаете, почему фундаментальные (основные) типы данных не всегда портируются (переносятся) с одной компьютерной платформы на другую. Вы продолжите работу с готовыми объектами, на этот раз на примере операций с целыми числами произвольной величины — их значения могут выходить далеко за пределы тех диапазонов, которые компьютер способен обрабатывать на аппаратном уровне.

Мы начнем знакомить вас с появившимся в C++20 синтаксисом форматирования текста, основанным на аналогичных синтаксических средствах из языков Python, Microsoft.NET (C# и Visual Basic) и Rust¹. По сравнению с предшествующими версиями C++ новый синтаксис стал лаконичнее и мощнее.

3.2. Управляющие структуры

В 1960-е годы стало ясно, что причина многих проблем, затрудняющих совместную работу групп программистов, — это беспорядочная передача управления от одних инструкций к другим. Виновником была объявлена инструкция `goto` (входящая тогда в большинство языков программирования), позволяющая произвольно менять последовательность выполнения других инструкций.

Исследование Бёма и Якопини² показало, что можно писать программы, не применяя `goto`. И программистам той эпохи пришлось постараться, чтобы изменить свой привычный стиль мышления на «программирование без `goto`». Термин «структурное программирование» (**structured programming**) стал почти синонимом всеобщего отказа от `goto`. Результаты оказались впечатляющими. Многие группы программистов сообщали, что разработка ускорируется, проекты стали чаще завершаться в запланированные сроки и в рамках бюджета. Эти успехи объяснялись тем, что структурированные программы стали гораздо понятнее, их стало легче отлаживать и модифицировать, а главное, повышалась вероятность, что в них нет ошибок.

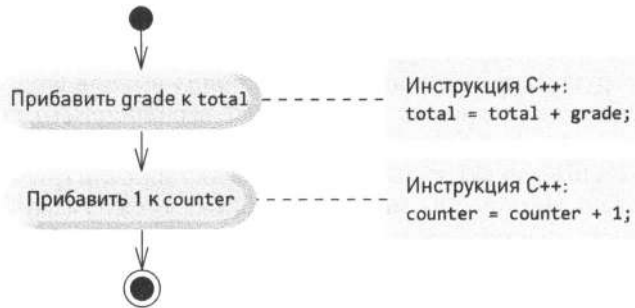
Бём и Якопини продемонстрировали, что для написания любой программы нужны всего три управляющие структуры: **последовательность (sequence)**, **выбор (selection)** и **цикл (iteration, или loop)**. Сейчас мы расскажем, как они реализованы в C++.

¹ Виктор Зверович (Victor Zverovich), «Text Formattings», 16 июля 2019 г. Был доступен: 15 сентября 2023 г. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0645r10.html>.

² К. Бём (C. Böhm) и Дж. Якопини (G. Jacopini), «Flow Diagrams, Turing Machines, and Languages with Only Two Formation Rules», ACM, выпуск 9, № 5, май 1966 г., с. 336–371.

3.2.1. Последовательность

Последовательность — это базовая структура, встроенная в язык C++. По умолчанию (если не указано иное) все инструкции выполняются последовательно: в том порядке, как они написаны сверху вниз в исходном коде. Следующая **диаграмма действий (activity diagram)**¹ описывает простую последовательность, состоящую из двух вычислительных инструкций:



C++ позволяет вам последовательно выполнять любое количество действий. Кроме того, любое отдельное действие в программе вы можете заменить несколькими последовательными действиями.

Диаграмма действий, также называемая диаграммой деятельности, моделирует **рабочий процесс (workflow)**, происходящий в программе или в ее фрагменте. В частности, на такой диаграмме можно показать линейный алгоритм вычислений, как в нашем примере. Основа любой диаграммы действий — фигуры-символы: прямоугольники с закругленными углами (они обозначают действия), ромбы и маленькие круги. Эти фигуры соединены стрелками, обозначающими **поток управления (flow of control)**, то есть тот порядок, в котором должны выполняться действия.

Наша простая диаграмма с последовательной структурой описывает два **состояния действия (action states)**. Каждое из них содержит **выражение действия (action expression)**: «прибавить grade к total» и «прибавить 1 к counter» — это те конкретные действия, которые надо выполнить. Стрелками показаны три перехода между ними.

Черный круг в верхней части диаграммы обозначает **начальное состояние (initial state)**, то есть начало потока управления перед выполнением программой моделируемых действий. Белый круг с черным кругом внутри в нижней части диаграммы обозначает **конечное состояние (final state)**, то есть завершение потока управления после выполнения программой всех указанных действий.

¹ Мы используем UML-диаграммы (UML — Unified Modeling Language) сначала в главах 3 и 4, рассказывая о передаче управления в управляющих инструкциях, а затем в главах 9 и 10, посвященных разработке классов.

Прямоугольники с загнутыми верхними правыми углами — это **примечания** (**notes**), поясняющие назначение символов на UML-диаграмме (эти примечания подобны комментариям в исходном коде C++). Каждый символ соединен со своим примечанием пунктирной линией. В нашем примере примечания показывают, как преобразовать каждое из состояний в исходный код на языке C++. Как правило, сам исходный код на таких диаграммах не пишут.

3.2.2. Инструкции выбора

В языке C++ есть три вида **инструкций выбора** (**selection statements**), также называемых **инструкциями ветвления**. Инструкция **if** выполняет действие (или группу действий), если условие истинно, или не выполняет его, если условие ложно. Инструкция **if...else** выполняет действие (или группу действий), если условие истинно, или выполняет альтернативное действие (или группу действий), если условие ложно. Инструкция **switch** (она обсуждается в главе 4) выполняет одно из нескольких действий (или одну из групп действий) в зависимости от значения выражения.

Инструкция **if** называется **инструкцией одиночного выбора** (**single-selection statement**), поскольку она выполняет или игнорирует действие (или группу действий). Инструкция **if...else** называется **инструкцией двойного выбора** (**double-selection statement**), поскольку она выбирает между двумя различными действиями (или группами действий). Инструкция **switch** называется **инструкцией множественного выбора** (**multiple-selection statement**), поскольку она выбирает среди множества действий (или групп действий).

3.2.3. Инструкции цикла

В языке C++ есть четыре вида **инструкций цикла** (**looping statements**), также называемых **инструкциями повторения** (**repetition statements**) или **инструкциями итерации**¹ (**iteration statements**): **while**, **do...while**, **for** и **for** для диапазона значений. Все они многократно выполняют действие (или группу действий) до тех пор, пока остается истинным **условие продолжения цикла** (**loop-continuation condition**). Инструкции **while** и **for** выполняют свое действие (или группу действий) ноль или более раз; если условие продолжения с самого начала является ложным (**false**), то действие (или группа действий) не выполняется. Инструкция **do...while** выполняет свое действие (или группу действий) один или несколько раз. В главе 4 мы рассмотрим инструкции **do...while** и **for**, а в главе 6 — инструкцию **for** для диапазона значений.

Ключевые слова

Слова **if**, **else**, **switch**, **while**, **do** и **for** относятся к ключевым (зарезервированным) словам языка C++. Ключевые слова не могут использоваться в качестве иден-

¹ Итерация — повторяющееся действие, проход цикла, шаг циклического процесса; обработка данных в цикле. -- *Примеч. ред.*

тификаторов (имен переменных) и содержат только строчные буквы, а иногда еще и символы подчеркивания. В следующей таблице приведен полный список ключевых слов C++:

Ключевые слова C++				
<code>alignas</code>	<code>alignof</code>	<code>and</code>	<code>and_eq</code>	<code>asm</code>
<code>auto</code>	<code>bitand</code>	<code>bitor</code>	<code>bool</code>	<code>break</code>
<code>case</code>	<code>catch</code>	<code>char</code>	<code>char16_t</code>	<code>char32_t</code>
<code>class</code>	<code>compl</code>	<code>const</code>	<code>const_cast</code>	<code>constexpr</code>
<code>continue</code>	<code>decltype</code>	<code>default</code>	<code>delete</code>	<code>do</code>
<code>double</code>	<code>dynamic_cast</code>	<code>else</code>	<code>enum</code>	<code>explicit</code>
<code>export</code>	<code>extern</code>	<code>false</code>	<code>final</code>	<code>float</code>
<code>for</code>	<code>friend</code>	<code>goto</code>	<code>if</code>	<code>import</code>
<code>inline</code>	<code>int</code>	<code>long</code>	<code>module</code>	<code>mutable</code>
<code>namespace</code>	<code>new</code>	<code>noexcept</code>	<code>not</code>	<code>not_eq</code>
<code>nullptr</code>	<code>operator</code>	<code>or</code>	<code>or_eq</code>	<code>override</code>
<code>private</code>	<code>protected</code>	<code>public</code>	<code>register</code>	<code>reinterpret_cast</code>
<code>return</code>	<code>short</code>	<code>signed</code>	<code>sizeof</code>	<code>static</code>
<code>static_assert</code>	<code>static_cast</code>	<code>struct</code>	<code>switch</code>	<code>template</code>
<code>this</code>	<code>thread_local</code>	<code>throw</code>	<code>true</code>	<code>try</code>
<code>typedef</code>	<code>typeid</code>	<code>typename</code>	<code>union</code>	<code>unsigned</code>
<code>using</code>	<code>void</code>	<code>volatile</code>	<code>virtual</code>	<code>wchar_t</code>
<code>while</code>	<code>xor</code>	<code>xor_eq</code>		
Ключевые слова, введенные в C++20				
<code>char8_t</code>	<code>concept</code>	<code>constexpr</code>	<code>constinit</code>	<code>co_await</code>
<code>co_return</code>	<code>co_yield</code>	<code>requires</code>		

3.2.4. Комбинации управляющих инструкций

Итак, в языке C++ есть только три вида управляющих структур, которые мы далее будем называть **управляющими инструкциями (control statements)**:

- ♦ последовательность;
- ♦ выбор (`if`, `if...else` и `switch`);
- ♦ цикл (`while`, `do...while`, `for` и `for` для диапазона значений).

Создавая любую программу, вы комбинируете эти инструкции в соответствии с тем алгоритмом, который реализуете. Каждую управляющую инструкцию можно смоделировать в виде диаграммы действий. Такая диаграмма будет содержать начальное и конечное состояния, то есть точку входа и точку выхода этой инструк-

ции. Управляющие инструкции с **одной точкой входа и одной точкой выхода** (**single-entry/single-exit control statements**) упрощают создание простых для понимания программ: вы просто соединяете точку выхода со следующей точкой входа, составляя таким образом **последовательность управляющих инструкций** (**control-statement stacking**). Второй способ соединения управляющих инструкций — это **вложенность** (**nesting**): вы помещаете одну инструкцию внутрь другой. Таким образом, все алгоритмы в программах на языке C++ состояются только из трех видов управляющих инструкций, соединяемых лишь двумя способами. Такова суть простоты этого языка.

3.3. Инструкция одиночного выбора if

Вы познакомились с инструкцией `if` в разделе 2.6. Она применяется для выбора между двумя вариантами последовательности действий. Например, предположим, что проходной балл на экзамене равен 60. Следующая инструкция C++ определяет, является ли условие `StudentGrade >= 60` истинным:

```
if (studentGrade >= 60) {  
    cout << "Passed";  
}
```

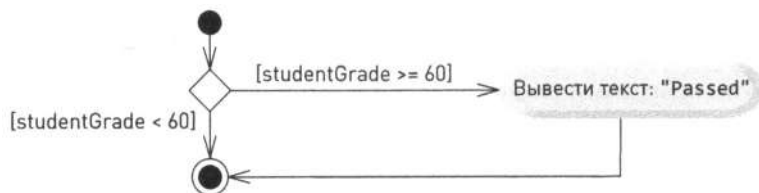
Если условие истинно, выводится сообщение "Passed" и выполняется следующая по порядку инструкция. Если условие ложно, инструкция вывода игнорируется и выполняется следующая по порядку инструкция. Отступ во второй строке этой инструкции необязателен, но рекомендуется для наглядности кода.

Логический тип данных (bool)

В главе 2 вы формировали условия с помощью операторов равенства и сравнения. В качестве условий также можно использовать любые выражения, результатом которых может быть ноль. При этом ноль означает, что условие ложно, а любое ненулевое значение — что условие истинно. Также в C++ есть логический тип данных (тип `bool`). Логические переменные (переменные типа `bool`) могут принимать только два значения: `true` (истинно) и `false` (ложно). Слова `true` и `false` являются в C++ ключевыми. Компилятор может неявно преобразовывать `true` в 1, а `false` — в 0.

Диаграмма действий для инструкции if

Следующая диаграмма показывает поток управления, соответствующий инструкции `if`.



Эта диаграмма содержит очень важный символ: ромб — **символ принятия решения (decision symbol)**, обозначающий альтернативность дальнейших действий. После ромба поток управления продолжится в зависимости от **сторожевых условий (guard conditions)** — они указаны в квадратных скобках рядом со стрелками перехода, выходящими из ромба. Если условие истинно, поток управления продолжается с того действия, на которое направлена соответствующая стрелка перехода. Диаграмма показывает: если значение переменной `studentGrade` больше или равно 60 (если это условие истинно), то программа выводит сообщение "Passed" и затем переходит в конечное состояние. Если же `studentGrade` меньше 60, то программа немедленно переходит в конечное состояние без вывода сообщения. Инструкция `if` — это управляющая инструкция с одним входом и одним выходом.

3.4. Инструкция двойного выбора if...else

Итак, инструкция выбора `if` выполняет действие, только если условие истинно. В отличие от нее, инструкция двойного выбора `if...else` позволяет вам указать действие, которое будет выполнено при истинном условии, а также альтернативное действие, которое будет выполнено при ложном условии. Например, следующая инструкция выводит сообщение "Passed", если `studentGrade >= 60`, или сообщение "Failed", если значение этой переменной меньше 60:

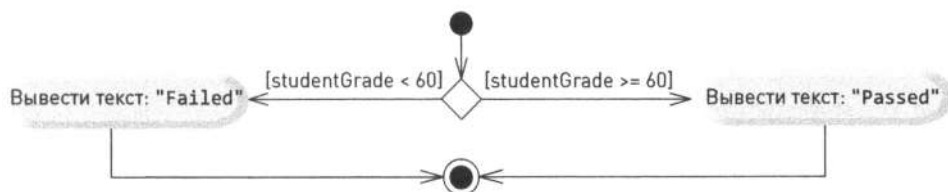
```
if (studentGrade >= 60) {  
    cout << "Passed";  
}  
else {  
    cout << "Failed";  
}
```

При любом значении переменной `studentGrade` после вывода соответствующего сообщения будет выполнена следующая инструкция в последовательности.

Обратите внимание, что тело `else` мы тоже записали с отступом шириной в три символа. Независимо от выбранной вами ширины отступа, она должна быть одинаковой во всей вашей программе.

Диаграмма действий для инструкции if...else

Следующая диаграмма показывает поток управления, соответствующий инструкции `if...else`:



3.4.1. Вложенные инструкции if...else

Чтобы программа могла выбирать между многими вариантами действий, можно использовать вложенные инструкции `if...else`, то есть поместить их друг в друга по принципу матрешки. Например, следующая структура из вложенных инструкций `if...else` выводит одно из пяти сообщений: "A" — для оценок, которые больше или равны 90, "B" — для оценок от 80 до 89, "C" — для оценок от 70 до 79, "D" — для оценок от 60 до 69 и "F" — для всех остальных оценок. Мы обозначили четыре уровня вложенности четырьмя прямоугольниками.

```
if (studentGrade >= 90) {  
    cout << "A";  
}  
else {  
    if (studentGrade >= 80) {  
        cout << "B";  
    }  
    else {  
        if (studentGrade >= 70) {  
            cout << "C";  
        }  
        else {  
            if (studentGrade >= 60) {  
                cout << "D";  
            }  
            else {  
                cout << "F";  
            }  
        }  
    }  
}
```

Если переменная `StudentGrade` больше или равна 90, то первые четыре условия во вложенных инструкциях `if...else` будут истинными, но выполнится только тело секции `if` в первой из них («самой внешней»), а тело соответствующей секции `else` будет пропущено. Всю эту управляющую структуру можно записать и другим способом, с меньшим количеством фигурных скобок и отступов, при этом программа выполнит точно такие же действия:

```
if (studentGrade >= 90) {  
    cout << "A";  
}  
else if (studentGrade >= 80) {  
    cout << "B";  
}  
else if (studentGrade >= 70) {
```

```
    cout << "C";  
}  
else if (studentGrade >= 60) {  
    cout << "D";  
}  
else {  
    cout << "F";  
}
```

Такая запись позволяет избежать больших сдвигов строк вправо, которые могут приводить к неудобным переносам строк на экране и в распечатках. Обратите внимание, что здесь и в последующих примерах мы обязательно заключаем тело каждой управляющей инструкции в фигурные скобки (`{` и `}`), чтобы избежать логической ошибки, называемой «*висячие else*».

3.4.2. Блоки

Инструкция `if` предполагает, что ее тело состоит только из одной инструкции. Чтобы включить несколько последовательных инструкций в секцию `if` или `else`, поместите их между фигурными скобками. Опытные программисты рекомендуют всегда заключать в фигурные скобки даже единственную инструкцию в секции `if` или `else`. Последовательность инструкций между фигурными скобками (например, в теле управляющей инструкции или функции) образует **блок (block)**. В любом месте функции, где допустима отдельная инструкция, ее можно заменить блоком.

В следующем примере показан блок из двух инструкций в теле секции `else`:

```
if (studentGrade >= 60) {  
    cout << "Passed";  
}  
else {  
    cout << "Failed\n";  
    cout << "You must retake this course.";  
}
```

Если `StudentGrade` меньше 60, программа выполнит обе инструкции в теле `else` и выведет сообщение:

```
Failed  
You must retake this course.
```

Что будет, если в этом примере убрать (забыть) фигурные скобки в теле секции `else`? Тогда эта инструкция окажется вне тела `else`:

```
cout << "You must retake this course.";
```

Из-за этого она будет выполняться всегда, независимо от значения `StudentGrade`. Возникнет логическая ошибка.

Пустая инструкция

Любую отдельную инструкцию можно заменить не только блоком, но и **пустой инструкцией (empty statement)**, которая состоит только из точки с запятой (;). Пустая инструкция не влияет на выполнение программы.

3.4.3. Тернарный условный оператор ?:

В языке C++ есть **условный оператор (conditional operator) ?:**, который вы можете использовать вместо инструкции `if...else`, чтобы сделать код короче и понятнее. Это единственный оператор C++, принимающий три операнда, поэтому его называют **тернарным оператором (ternary operator)** — от латинского слова «ternarius» — «тройной». Операнды вместе с символами ? и : образуют **условное выражение (conditional expression)**. Например, следующая инструкция выводит значение условного выражения:

```
cout << (studentGrade >= 60 ? "Passed" : "Failed");
```

Операнд слева от символа ? — это условие. Второй операнд (находящийся между символами ? и :) — это значение выражения в том случае, если условие истинно. Операнд справа от символа : — это значение выражения в том случае, если условие ложно. Условное выражение в этой инструкции вычисляется как строка "Passed", если условие

```
studentGrade >= 60
```

имеет значение `true`, или вычисляется как строка "Failed", если это условие имеет значение `false`. Таким образом, условный оператор ?: выполняет те же действия, что и инструкция `if...else` из нашего первого примера в разделе 3.4. У этого оператора низкий приоритет, поэтому такое условное выражение обычно помещают в круглые скобки.

3.5. Инструкция цикла while

Инструкция цикла позволяет вам указать, что программа должна повторять какое-либо действие до тех пор, пока заданное условие остается истинным.

В качестве примера имеющейся в C++ инструкции цикла `while` рассмотрим код, вычисляющий наименьшее значение числа 3 в натуральной степени, которое больше 100. После выполнения цикла `while` результат будет сохранен в переменной `product`:

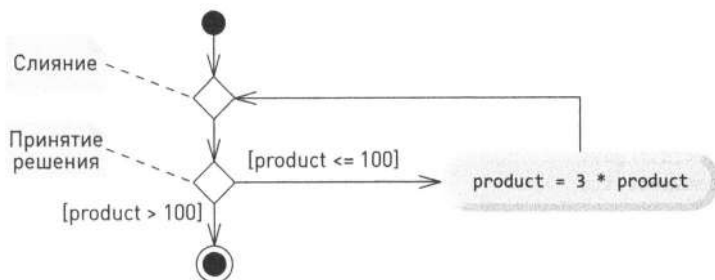
```
int product{3};  
while (product <= 100) {  
    product = 3 * product;  
}
```

При каждом выполнении тела цикла `while` переменная `product` умножается на 3, то есть последовательно принимает значения 9, 27, 81 и 243. Когда значение этой переменной достигнет 243, условие `product <= 100` станет ложным и цикл

завершится. При этом в переменной `product` останется значение 243, после чего выполнение программы продолжится со следующей инструкции.

Диаграмма действий для инструкции `while`

На следующей UML-диаграмме, моделирующей приведенный выше фрагмент кода с инструкцией `while`, вы видите **символ слияния (merge symbol)**:



Символы слияния на UML-диаграммах имеют вид ромбов, то есть выглядят так же, как символы принятия решений. Символ слияния объединяет два потока управления в один. На нашей диаграмме символ слияния объединяет два перехода: из начального состояния и из состояния действия. После каждого из этих переходов следует переход к символу принятия решения о том, надо ли начинать (или продолжать) выполнение цикла.

Как отличать символы принятия решений от символов слияния? По количеству входящих и исходящих стрелок перехода. Символ принятия решения имеет одну входящую стрелку и несколько исходящих — они обозначают возможные варианты переходов. При этом около каждой исходящей стрелки должно быть указано условие перехода. Символ слияния, наоборот, имеет несколько входящих стрелок и одну исходящую — она обозначает объединенный поток управления после слияния. При этом у любых стрелок перехода, соединенных с символом слияния, никогда не бывает указано условие перехода.

3.6. Цикл со счетчиком

Рассмотрим следующую задачу:

Класс из 10 учеников сделал контрольную работу. Полученные учениками оценки (целые числа в диапазоне от 0 до 100) вводятся с клавиатуры. Требуется найти среднюю арифметическую оценку по классу.

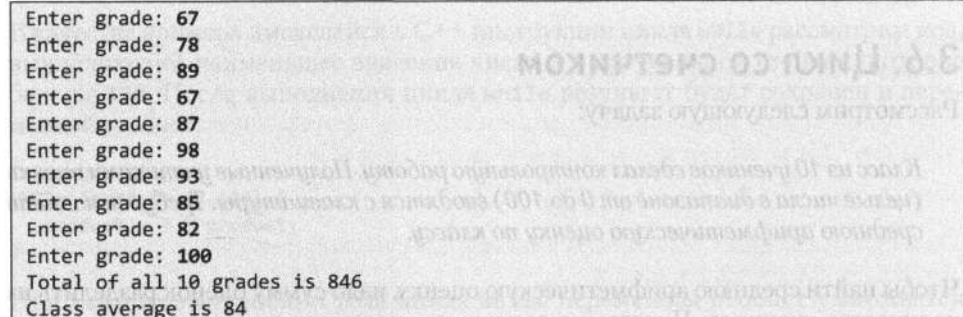
Чтобы найти среднюю арифметическую оценку, надо сумму оценок разделить на количество учеников. Программа должна по очереди получить каждую оценку, суммировать их все, вычислить среднюю оценку и вывести результат.

Мы будем использовать **цикл со счетчиком (counter-controlled iteration)** для ввода оценок с клавиатуры по очереди. Этот метод использует счетчик для контроля количества выполненных блоков инструкций. В нашем примере цикл завершится, когда значение счетчика превысит 10.

3.6.1. Реализация цикла со счетчиком

На илл. 3.1 показано, как функция `main` вычисляет среднюю оценку с помощью цикла со счетчиком. Она запрашивает у пользователя десять оценок, после чего вычисляет и выводит на экран результат.

```
1 fig03_01.cpp
2 // Вычисление средней оценки с помощью цикла со счетчиком
3 #include <iostream>
4 using namespace std;
5
6 int main() {
7     // Инициализируем переменные
8     int total{0}; // Сумма оценок
9     int gradeCounter{1}; // Счетчик оценок
10
11     // Обрабатываем данные в цикле со счетчиком
12     while (gradeCounter <= 10) { // Выполняем 10 итераций
13         cout << "Enter grade: "; // Выводим сообщение
14         int grade;
15         cin >> grade; // Вводим оценку
16         total = total + grade; // Прибавляем оценку к сумме оценок
17         gradeCounter = gradeCounter + 1; // Увеличиваем счетчик на 1
18     }
19
20     // Ввод данных завершен
21     int average{total / 10}; // Деление чисел типа int дает результат типа int
22
23     // Выводим сумму оценок и среднюю оценку
24     cout << "\nTotal of all 10 grades is " << total;
25     cout << "\nClass average is " << average << "\n";
26 }
```



```
Enter grade: 67
Enter grade: 78
Enter grade: 89
Enter grade: 67
Enter grade: 87
Enter grade: 98
Enter grade: 93
Enter grade: 85
Enter grade: 82
Enter grade: 100
Total of all 10 grades is 846
Class average is 84
```

Илл. 3.1 | Вычисление средней оценки с помощью цикла со счетчиком

Локальные переменные в функции main

В строках 8, 9, 14 и 21 объявляются целочисленные переменные `total`, `gradeCounter`, `grade` и `average`. Переменная `grade` хранит введенное пользователем число. Это **локальная переменная (local variable)**. Локальной называется переменная, объявленная в блоке (например, в теле функции); ее можно использовать только от момента объявления до закрывающей фигурной скобки блока. Объявление локальной переменной должно предшествовать первому обращению к ней. Переменную `grade`, объявленную в цикле `while`, можно использовать только в пределах тела этого цикла.

Инициализация переменных `total` и `gradeCounter`

В строках 8–9 объявляются и инициализируются две переменные: `total` со значением 0 и `gradeCounter` со значением 1. Инициализация происходит до первого использования этих переменных в вычислениях.

Ввод десяти оценок пользователем

Инструкция цикла `while` (строки 12–18) продолжает выполняться до тех пор, пока значение переменной `gradeCounter` меньше или равно 10. В строке 13 выводится сообщение "Enter grade: ", а в строке 15 введенное пользователем число присваивается переменной `grade`. В строке 16 это число суммируется с переменной `total`, и результат сохраняется в `total`, заменяя предыдущее значение этой переменной. В строке 17 к переменной `gradeCounter` прибавляется 1, что означает завершение обработки очередной оценки и готовность к следующему повторению цикла. Увеличение `gradeCounter` в итоге приведет к превышению максимально допустимого значения в условии цикла (10), и тогда цикл завершится.

Вычисление и вывод среднего арифметического

После завершения цикла в строке 21 выполняется вычисление средней арифметической оценки по классу. Это происходит при инициализации переменной `average`. В строке 24 на экран выводится сообщение "Total of all 10 grades is ", за которым следует значение переменной `total`. В строке 25 выводится сообщение "Class average is " и средняя оценка. В строке 26 программа завершается.

3.6.2. Целочисленное деление с усечением

Вычисление средней арифметической оценки в этом примере дает целочисленный результат. Сумма введенных оценок равна 846, и в обычной арифметике после деления на 10 получилось бы 84.6. Такие числа, содержащие десятичные точки, в C++ называются **числами с плавающей точкой (floating-point numbers)**. Но в нашей программе результатом выражения `total / 10` будет целое число 84, поскольку оба операнда имеют тип `int`. Деление двух чисел типа `int` является **целочисленным делением (integer division)**, то есть любая дробная часть результата отбрасывается. Например, результатом вычисления выражения `7 / 4` в обычной арифметике будет 1.75. Но при целочисленном делении результат усекается до 1 вместо округления

до 2. В следующем разделе мы покажем, как получить в результате аналогичных вычислений число с плавающей точкой.

3.7. Цикл с контрольным значением

Давайте обобщим задачу, сформулированную в разделе 3.6. Новое требование будет таким:

Разработайте аналогичную программу вычисления средней арифметической оценки, которая будет работать при любом количестве учеников в классе.

В предыдущем примере в условии задачи было задано количество учеников, то есть количество вводимых оценок (10) было известно программе заранее. А теперь мы не знаем, сколько оценок введет пользователь во время выполнения программы. Программа должна правильно обрабатывать произвольное количество оценок.

Один из способов решения этой задачи — использование **контрольного значения** (*sentinel value*), также называемого **сигнальным** (*signal value*), **фиктивным** (*dummy value*) или **флаговым** (*flag value*) значением. Контрольное значение указывает на окончание ввода данных. Пользователь вводит оценки до тех пор, пока не введет их все. Затем он вводит заранее известное контрольное значение, которое означает, что оценки больше вводиться не будут.

Вы должны выбрать такое контрольное значение, которое нельзя перепутать с любыми возможными оценками. Оценки за контрольную работу — это неотрицательные целые числа, поэтому в качестве контрольного значения хорошо подойдет отрицательное число **-1**. В этом случае поток входных данных, которые должна обработать программа, будет, например, таким: 95, 96, 75, 74, 89 и, наконец, **-1**. Затем программа должна будет вычислить и вывести среднее арифметическое значение для оценок 95, 96, 75, 74 и 89, поскольку значение **-1** является контрольным и не должно участвовать в расчетах.

Не исключено, что пользователь введет **-1** в самом начале, то есть количество оценок будет равно 0. Мы должны учесть это перед началом вычислений. Согласно стандарту C++, результат деления на ноль в выражениях с плавающей точкой **не определен** (*undefined*). Поэтому перед любой операцией деления (/) или вычисления остатка от деления (%) в которой правый операнд может быть равен нулю, надо проверить значение правого операнда и в случае нуля обработать это состояние (например, вывести сообщение об ошибке), не разрешая программе приступить к недопустимому делению на ноль.

3.7.1. Реализация цикла с контрольным значением

На илл. 3.2 показан пример цикла с контрольным значением. Предполагается, что все оценки, введенные пользователем, будут целыми числами, но на этот раз мы получим их среднее арифметическое в виде числа с плавающей точкой. Такие

числа нельзя хранить в переменных типа `int`. Для чисел с плавающей точкой в языке C++ есть типы данных `float`, `double` и `long double`. Основное различие между ними — это **точность (precision)**. По сравнению с типом `float` тип `double` обеспечивает значительно большую точность вычислений (то есть позволяет работать с числами намного большей величины или с намного большим количеством цифр после точки). В свою очередь, тип `long double` обеспечивает значительно большую точность вычислений по сравнению с типом `double`. В главе 4 мы подробнее расскажем обо всех типах переменных, предназначенных для чисел с плавающей точкой.

```
1 // fig03_02.cpp
2 // Вычисление средней оценки с помощью цикла с контрольным значением
3 #include <iostream>
4 #include <iomanip> // Заголовок для параметризованных манипуляторов потока
5 using namespace std;
6
7 int main() {
8     // Инициализируем переменные
9     int total{0}; // Сумма оценок
10    int gradeCounter{0}; // Счетчик оценок
11
12    // Обрабатываем данные
13    // Выводим сообщение и вводим оценку
14    cout << "Enter grade or -1 to quit: ";
15    int grade;
16    cin >> grade;
17
18    // Повторяем цикл до ввода контрольного значения
19    while (grade != -1) {
20        total = total + grade; // Прибавляем оценку к сумме оценок
21        gradeCounter = gradeCounter + 1; // Увеличиваем счетчик
22
23        // Выводим сообщение и вводим следующую оценку
24        cout << "Enter grade or -1 to quit: ";
25        cin >> grade;
26    }
27
28    // Ввод данных завершен
29    // Если введена хотя бы одна оценка...
30    if (gradeCounter != 0) {
31        // Используем тип double для вычисления среднего арифметического
32        double average{static_cast<double>(total) / gradeCounter};
33
34        // Выводим сумму оценок и среднюю оценку (с двумя цифрами после точки)
35        cout << "\nTotal of the " << gradeCounter
36            << " grades entered is " << total;
37        cout << setprecision(2) << fixed;
38        cout << "\nClass average is " << average << "\n";
39    }
40    else { // Ни одна оценка не введена, выводим сообщение об этом
41        cout << "No grades were entered\n";
```

```
42 }  
43 }
```

```
Enter grade or -1 to quit: 97  
Enter grade or -1 to quit: 88  
Enter grade or -1 to quit: 72  
Enter grade or -1 to quit: -1  
Total of the 3 grades entered is 257  
Class average is 85.67
```

Илл. 3.2 | Вычисление средней оценки с помощью цикла с контрольным значением

Напомним, что целочисленное деление дает целочисленный результат. Но в данном случае мы включили в программу `static_cast` — один из **операторов преобразования типов (cast operators)**, также называемых **операторами приведения типов**. Он означает, что результатом деления должно быть число с плавающей точкой. В программе последовательно выполняются две управляющие инструкции: сначала `while`, а затем `if...else`. Значительная часть кода аналогична коду предыдущей программы (илл. 3.1), поэтому мы поясним только основные различия.

Различия в логике работы цикла с контрольным значением и цикла со счетчиком

В строке 10 счетчик оценок инициализируется значением 0, поскольку ни одна оценка еще не введена. Не забывайте, что программа использует для ввода оценок цикл с контрольным значением. Значение `gradeCounter` увеличивается на единицу, когда пользователь вводит оценку. В строке 32 объявляется переменная `average` типа `double`, в которой полученная средняя арифметическая оценка по классу сохраняется в виде числа с плавающей точкой.

Сравните логику программ, показанных на илл. 3.1 и 3.2. В цикле со счетчиком инструкция `while` (строки 12–18) выполняется ровно 10 раз, и при каждом ее выполнении программа получает от пользователя очередную оценку. А в цикле с контрольным значением программа запрашивает и получает первое значение (строки 14 и 16 на илл. 3.2) еще до перехода к циклу `while`. От первого введенного значения зависит, состоится ли переход к телу цикла `while`. Если условие в строке 19 ложно (если пользователь сразу ввел контрольное значение -1), то программа не получит ни одной оценки и тело цикла не выполнится ни одного раза. Если же это условие истинно, начинается выполнение тела цикла: переменная `grade` суммируется с `total` и увеличивается счетчик `gradeCounter`. Затем в строках 24–25 программа запрашивает у пользователя следующее значение, после чего достигается закрывающаяся фигурная скобка цикла в строке 26 и выполнение продолжается с проверки условия цикла `while` в строке 19. При этой проверке используется последнее на данный момент значение переменной `grade`, введенное пользователем. От него зависит, будет ли тело цикла выполнено снова.

Каждое значение переменной `grade` пользователь вводит непосредственно перед проверкой условия продолжения цикла. Это позволяет программе определить

перед дальнейшими действиями, является ли введенное значение контрольным (надо ли прибавлять `grade` к `total`). Если введено контрольное значение, цикл завершается и программа не прибавляет `-1` к сумме оценок.

После завершения цикла выполняется инструкция `if...else` в строках 30–42. Условие в строке 30 определяет, была ли введена хотя бы одна оценка. Если пользователь не ввел ни одной оценки, выполняется тело `else` инструкции `if...else`, то есть выводится сообщение "No grades were entered". После инструкции `if...else` программа завершается.

3.7.2. Явное и неявное преобразование основных типов данных

Продолжим пояснения к коду, показанному на илл. 3.2. Если пользователь ввел хотя бы одну оценку, то в строке 32 вычисляется среднее арифметическое суммы чисел:

```
double average{static_cast<double>(total) / gradeCounter};
```

Напомним, что целочисленное деление всегда дает целочисленный результат, даже если вы записываете его в переменную типа `double`. Например, следующая инструкция отбросит (удалит) дробную часть результата деления еще до того, как этот результат будет записан в переменную `average` при ее инициализации:

```
double average{total / gradeCounter};
```

Оператор `static_cast`

Чтобы при делении целых чисел получить результат с плавающей точкой, надо сначала создать временное значение с плавающей точкой при помощи оператора `static_cast`. В строке 32 создается временная копия указанной в круглых скобках переменной `total`, приведенная к типу `double`, указанному в угловых скобках. При этом сама переменная `total` не меняется, ее значение остается целочисленным. Такое использование оператора `static_cast` называется **явным преобразованием типов (explicit type conversion)**. Кроме оператора `static_cast`, вы встретитесь в книге и с другими операторами преобразования типов. Преобразование типов также называют **приведением типов (type casting)**.

Продвижение типов

Итак, среднее арифметическое будет получено делением значения временной копии переменной `total` (имеющей тип `double`) на значение переменной `gradeCounter` (имеющей тип `int`). Но компилятор C++ может вычислять только те арифметические выражения, в которых все операнды имеют один и тот же тип. Поэтому если типы операндов отличаются, компилятор автоматически выполняет **продвижение типов (type promotion)**, также называемое **неявным преобразованием типов (implicit type conversion)**. В выражениях с операндами `int` и `double` компилятор преобразует целочисленные операнды (`int`) в числа с плавающей

точкой (`double`). В строке 32 компилятор преобразует тип временной копии переменной `gradeCounter` в тип `double`, а затем выполняет деление. После этого переменная `average` инициализируется результатом деления, имеющим тип `double`. В разделе 5.6 мы обсудим разрешенные в языке C++ неявные преобразования основных типов данных.

Возможности операторов приведения типов

Операторы приведения типов можно использовать со всеми основными типами данных, а также с другими типами, как вы увидите в главе 9. Просто укажите нужный тип в угловых скобках (`<` и `>`) после ключевого слова `static_cast`. Это **унарный оператор (unary operator)** — у него есть только один операнд. В языке C++ есть и другие унарные операторы, например `+` (плюс) и `-` (минус), применяемые в таких выражениях, как `+5` или `-7`. Операторы приведения типов имеют второй приоритет.

3.7.3. Форматирование чисел с плавающей точкой

Сейчас мы кратко обсудим средства форматирования результата вычислений, использованные в программе вычисления средней оценки (илл. 3.2).

Параметризованный манипулятор потока `setprecision`

В строке 37 вызывается **манипулятор потока (stream manipulator)** `setprecision` с параметром 2: `setprecision(2)`. Он указывает компилятору, что числа с плавающей точкой надо выводить в поток с заданной **точностью (precision)**: с двумя цифрами после точки (например, 92.37). Этот манипулятор является **параметризованным (parameterized)**, поскольку для выполнения задачи ему нужен параметр — количество цифр после точки (в данном случае 2). В любой программе, использующей параметризованные манипуляторы потока, должен быть включен заголовок `<iomanip>` (строка 4).

Непараметризованный манипулятор потока `fixed`

Непараметризованный (non-parameterized), то есть не имеющий параметра, манипулятор потока `fixed` в строке 37 указывает, что числа с плавающей точкой надо выводить в поток в **формате с фиксированной точкой (fixed-point format)**. Это альтернатива **научной записи (scientific notation)**, в которой любое число выглядит в виде мантииссы (числа в диапазоне от 1.0 до 10.0) и порядка (степени числа 10). Например, число 3100.0 (три тысячи сто целых ноль десятых) в научной записи отображается так: 3.1×10^3 (то есть $3.1 \cdot 10^3$). Этот формат особенно удобен для очень больших и очень маленьких чисел.

В формате с фиксированной точкой, в отличие от научной записи, числа выглядят привычнее. И даже если число целое, после него всегда отображается десятичная точка, а затем нули (например, 88.00). Для сравнения, если не включить форматирование с фиксированной точкой, это же число будет выведено без точки и нулей: 88.

Манипуляторы потока `setprecision` и `fixed` относятся к липким настройкам (**sticky settings**). Это значит, что после однократного выполнения инструкции с таким манипулятором он «приклеивается» к вашей программе: все числа с плавающей точкой с этого момента будут форматироваться в соответствии с заданными настройками, пока вы их не измените.

Округление чисел с плавающей точкой

Когда используются манипуляторы потока `fixed` и `setprecision`, выводимое значение переменной округляется до нужного вам количества десятичных знаков, то есть до той точности, которую вы указали. Но само значение этой переменной в памяти остается неизменным. Например, при точности с двумя цифрами после точки числа 87.946 и 67.543 округляются до 87.95 и 67.54 соответственно¹.

Итак, в строках 37–38 на илл. 3.2 выводится среднее арифметическое значение, округленное до сотых и содержащее ровно две цифры справа от десятичной точки. Три оценки, введенные во время выполнения программы (97, 88 и 72), в сумме составляют 257, что дает среднее значение 85.666... и отображается в виде округленного числа 85.67.

3.8. Вложенные управляющие инструкции

Мы показали, что управляющие инструкции могут выполняться последовательно. В этом разделе мы рассмотрим второй способ (из двух возможных) структурированного соединения управляющих инструкций: **вложенность** (**nesting**) одной управляющей инструкции в другую.

3.8.1. Постановка задачи

Рассмотрим такую задачу:

Колледж предлагает курс подготовки студентов к экзамену в университет. В прошлом году десять студентов, окончивших этот курс, сдавали экзамен. Колледж хочет знать, насколько хорошо его студенты сдали экзамен. Напишите программу для анализа результатов. У вас есть список из десяти студентов. Рядом с каждым именем стоит число 1, если студент сдал экзамен, и число 2, если не сдал.

Программа должна работать так:

1. Получить каждый результат (1 или 2), выводя на экран сообщение «Enter result» при каждом запросе результата.
2. Подсчитать количество результатов каждого типа.

¹ Если из кода, показанного на илл. 3.2, убрать манипуляторы `setprecision` и `fixed`, компилятор C++ установит по умолчанию точность с четырьмя цифрами, а если оставить только манипулятор `fixed`, — с шестью.

3. Вывести сводку результатов, указав количество студентов, сдавших и не сдавших экзамен.
4. Если экзамен сдали более восьми студентов, вывести сообщение «Bonus to instructor!».

3.8.2. Реализация программы

На илл. 3.3 вы видите код этой программы, использующий цикл со счетчиком, а также примеры ввода и вывода данных. В строках 8–10 и 16 объявляются переменные, которые нужны для обработки результатов экзамена. Цикл `while` (строки 13–29) повторяется 10 раз. В каждой итерации цикла программа получает и обрабатывает один результат экзамена. Обратите внимание, что инструкция `if...else` (строки 20–25), обрабатывающая каждый результат, вложена в цикл `while`. Если значение переменной `result` равно 1, инструкция `if...else` увеличивает счетчик `passes`, в противном случае инструкция увеличивает `failures` (предполагая, что введено число 2)¹. В строке 28 счетчик `studentCounter` увеличивается перед повторной проверкой условия цикла в строке 13.

```
1 // fig03_03.cpp
2 // Анализ результатов экзамена с помощью вложенных управляющих инструкций
3 #include <iostream>
4 using namespace std;
5
6 int main() {
7     // Инициализируем переменные при объявлении
8     int passes{0};
9     int failures{0};
10    int studentCounter{1};
11
12    // Обрабатываем данные о 10 студентах в цикле со счетчиком
13    while (studentCounter <= 10) {
14        // Выводим сообщение и вводим результат экзамена
15        cout << "Enter result (1 = pass, 2 = fail): ";
16        int result;
17        cin >> result;
18
19        // В тело цикла while вложена инструкция if...else
20        if (result == 1) {
21            passes = passes + 1;
22        }
23        else {
24            failures = failures + 1;
25        }
26
27        // Увеличиваем studentCounter на 1 (поэтому цикл в итоге завершится)
```

¹ Предположение, что результат равен 2, может быть ошибочным, если пользователь ввел некорректные данные. Позже мы обсудим методы проверки вводимых данных.

```
28     studentCounter = studentCounter + 1;
29 }
30
31 // Ввод данных завершен; выводим результаты
32 cout << "Passed: " << passes << "\nFailed: " << failures << "\n";
33
34 // Проверяем, сдали ли экзамен более восьми студентов
35 if (passes > 8) {
36     cout << "Bonus to instructor!\n";
37 }
38 }
```

```
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Passed: 9
Failed: 1
Bonus to instructor!
```

```
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Passed: 6
Failed: 4
```

Илл. 3.3 | Анализ результатов экзамена с помощью вложенных управляющих инструкций

После ввода десяти значений цикл завершается и в строке 32 выводится количество сданных и проваленных экзаменов. Инструкция `if` в строках 35–37 проверяет, сдали ли экзамен более восьми студентов, и если это так, выводит сообщение "Bonus to instructor!".

На илл. 3.3 показаны примеры работы программы с двумя наборами данных. В первом случае условие в строке 35 истинно: более восьми студентов сдали экзамен, поэтому программа выводит сообщение о поощрении преподавателя.

3.8.3. Предотвращение сужающих преобразований с помощью скобочной инициализации

Рассмотрим скобочную инициализацию переменной в строке 10:

```
int studentCounter{1};
```

Напомним, что этот синтаксис появился в C++11. В более ранних версиях языка аналогичная инструкция записывалась так:

```
int studentCounter = 1;
```

Для переменных фундаментальных типов скобочная инициализация предотвращает нежелательные **сужающие преобразования (narrowing conversions)**, которые могут приводить к потере данных. Например, следующая инструкция объявления переменной пытается присвоить целочисленной переменной `x` значение `12.7` типа `double`:

```
int x = 12.7;
```

В этом случае компилятор C++ преобразует `double` в `int`, отсекая (отбрасывая) часть с плавающей точкой (`.7`). Это называется сужающим преобразованием. Оно искажает данные: переменная `x` получает неправильное значение `12`. Компиляторы обычно выдают предупреждение, но все-таки «соглашаются» компилировать такой код. Для сравнения, скобочная инициализация:

```
int x{12.7};
```

приведет к **ошибке компиляции (compilation error)**, то есть поможет вам сразу предотвратить логическую ошибку, которую потом будет трудно обнаружить. Даже если вы укажете в фигурных скобках целое число с плавающей точкой (например, `12.0`), все равно возникнет ошибка компиляции: не из-за самого числа (`12.0`), а из-за его типа (`double`).

В документации стандарта C++ не указана формулировка сообщения об ошибке компиляции. В приведенном примере сообщение Apple Xcode будет таким:

```
Type 'double' cannot be narrowed to 'int' in initializer list
```

Аналогичное сообщение Visual Studio:

```
conversion from 'double' to 'int' requires a narrowing conversion
```

И аналогичное сообщение GNU g++:

```
type 'double' cannot be narrowed to 'int' in initializer list [-Wc++11-narrowing]
```

В последующих главах мы обсудим другие особенности скобочной инициализации.

Пример из нашей прежней программы (илл. 3.1)

В строке 21 кода, показанного на илл. 3.1, есть такая инструкция:

```
int average{total / 10}; // Деление чисел типа int дает результат типа int
```

Вам может показаться, что она содержит сужающее преобразование. Но на самом деле и переменная `total`, и число `10` имеют тип `int`, поэтому и инициализирующее значение в фигурных скобках тоже имеет тип `int`. Если бы мы ранее объявили переменную `total` типа `double` или написали бы здесь `10.0` вместо `10`, тогда инициализирующее значение имело бы тип `double`, что привело бы к сообщению об ошибке компиляции из-за сужающего преобразования.

3.9. Составные операторы присваивания

Вы можете сократить следующую инструкцию:

```
c = c + 3;
```

с помощью составного оператора `+=`, который называется **оператором сложения с присваиванием (addition assignment operator)**:

```
c += 3;
```

Оператор `+=` прибавляет значение выражения, указанного справа от него, к переменной, указанной слева от него, и сохраняет результат в этой же переменной. Таким образом, выражение `c += 3` увеличивает переменную `c` на 3. В следующей таблице показаны все арифметические составные операторы присваивания, примеры выражений и результатов вычислений с этими операторами:

Оператор	Пример выражения	Пояснение	Присваивание
Исходные значения пяти переменных типа <code>int</code> : <code>c = 3, d = 5, e = 4, f = 6, g = 12</code>			
<code>+=</code>	<code>c += 7</code>	<code>c = c + 7</code>	Значение 10 присваивается переменной <code>c</code>
<code>-=</code>	<code>d -= 4</code>	<code>d = d - 4</code>	Значение 1 присваивается переменной <code>d</code>
<code>*=</code>	<code>e *= 5</code>	<code>e = e * 5</code>	Значение 20 присваивается переменной <code>e</code>
<code>/=</code>	<code>f /= 3</code>	<code>f = f / 3</code>	Значение 2 присваивается переменной <code>f</code>
<code>%=</code>	<code>g %= 9</code>	<code>g = g % 9</code>	Значение 3 присваивается переменной <code>g</code>

3.10. Операторы инкремента и декремента

В следующей таблице приведены описания двух унарных операторов языка C++. **Оператор инкремента (increment operator) ++** прибавляет 1 к числовой переменной, а **оператор декремента (decrement operator) --** вычитает 1 из числовой переменной.

Оператор	Название оператора	Пример выражения	Пояснение
<code>++</code>	оператор префиксного инкремента	<code>++number</code>	Увеличить <code>number</code> на 1, затем использовать в выражении новое значение <code>number</code>
<code>++</code>	оператор постфиксного инкремента	<code>number++</code>	Использовать в выражении текущее значение <code>number</code> , затем увеличить <code>number</code> на 1
<code>--</code>	оператор префиксного декремента	<code>--number</code>	Уменьшить <code>number</code> на 1, затем использовать в выражении новое значение <code>number</code>
<code>--</code>	оператор постфиксного декремента	<code>number--</code>	Использовать в выражении текущее значение <code>number</code> , затем уменьшить <code>number</code> на 1

Операторы инкремента и декремента, которые ставятся перед переменной, называются **префиксными операторами (prefix operators)**. Операторы инкремента и декремента, которые ставятся после переменной, называются **постфиксными операторами (postfix operators)**.

Префиксный оператор инкремента (или декремента) прибавляет 1 к переменной (или вычитает из нее 1). После увеличения (или уменьшения) новое значение переменной используется в выражении с этим оператором.

Постфиксный оператор инкремента (или декремента) работает иначе. Он тоже прибавляет 1 к переменной (или вычитает из нее 1). Но в выражении с этим оператором используется не новое, а прежнее значение переменной. И только после этого значение переменной увеличивается (или уменьшается).

В отличие от бинарных операторов, унарные операторы инкремента и декремента надо ставить рядом с их операндами без пробелов.

Различие между префиксным и постфиксным операторами инкремента

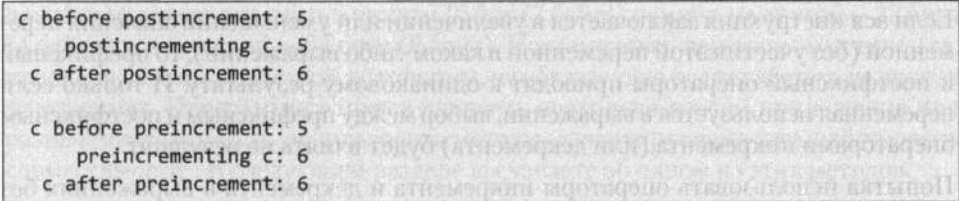
Иллюстрация 3.4 демонстрирует различие между префиксным и постфиксным операторами инкремента (`++`). Префиксный и постфиксный операторы декремента (`--`) работают аналогично.

```

1 // fig03_04.cpp
2 // Различие между префиксным и постфиксным операторами инкремента
3 #include <iostream>
4 using namespace std;
5
6 int main() {
7     // Применяем постфиксный оператор
8     int c{5};
9     cout << "c before postincrement: " << c << "\n"; // Выводим число 5
10    cout << " postincrementing c: " << c++ << "\n"; // Выводим число 5

```

```
11 cout << " c after postincrement: " << c << "\n"; // Выводим число 6
12
13 cout << "\n"; // Добавляем пустую строку
14
15 // Применяем префиксный оператор
16 c = 5;
17 cout << " c before preincrement: " << c << "\n"; // Выводим число 5
18 cout << " preincrementing c: " << ++c << "\n"; // Выводим число 6
19 cout << " c after preincrement: " << c << "\n"; // Выводим число 6
20 }
```



```
c before postincrement: 5
postincrementing c: 5
c after postincrement: 6

c before preincrement: 5
preincrementing c: 6
c after preincrement: 6
```

Илл. 3.4 | Различие между префиксным и постфиксным операторами инкремента

В строке 8 переменная `c` инициализируется значением 5, которое выводится в строке 9. В строке 10 выводится результат выражения `c++`. Поскольку оператор инкремента постфиксный, в этом выражении используется (выводится на экран) прежнее значение 5 и только после этого переменная получает новое значение 6. В строке 11 оно выводится на экран, и в строке 10 мы видим, что переменная изменилась.

В строке 16 переменной возвращается исходное значение 5, которое выводится на экран в строке 17. В строке 18 выводится результат выражения `++c`. Поскольку оператор инкремента префиксный, в этом выражении используется (выводится на экран) уже новое значение 6. В строке 19 оно снова выводится на экран, и мы видим, что после выполнения строки 18 переменная не изменилась.

Использование операторов для упрощения инструкций

Арифметические составные операторы присваивания, а также операторы инкремента и декремента позволят вам записывать многие инструкции ваших программ более компактно. Например, в коде, показанном на илл. 3.3, есть три инструкции присваивания в строках 21, 24 и 28:

```
passes = passes + 1;
failures = failures + 1;
studentCounter = studentCounter + 1;
```

С составным оператором присваивания эти же инструкции можно записать так:

```
passes += 1;
failures += 1;
studentCounter += 1;
```

Аналогичные операции можно выполнять с префиксным оператором инкремента:

```
++passes;
++failures;
++studentCounter;
```

и с постфиксным оператором инкремента:

```
passes++;
failures++;
studentCounter++;
```

Если вся инструкция заключается в увеличении или уменьшении значения переменной (без участия этой переменной в каком-либо выражении), то префиксный и постфиксный операторы приводят к одинаковому результату. И только если переменная используется в выражении, выбор между префиксным и постфиксным операторами инкремента (или декремента) будет влиять на результат.

Попытка использовать операторы инкремента и декремента в выражениях без присваивания приводит к ошибке компиляции. Например, инструкция `++(x + 1)` будет синтаксической ошибкой, поскольку `(x + 1)` не является переменной.

Приоритеты и правила группировки операторов

В следующей таблице показаны приоритеты и правила группировки тех операторов C++, о которых вы уже знаете. Все они перечислены сверху вниз в порядке убывания приоритетов. Обратите внимание, что тернарный условный оператор `(?:)`, унарные префиксные операторы инкремента `(++)` и декремента `(--)`, операторы сложения `(+)` и вычитания `(-)`, а также операторы присваивания `(=, +=, -=, *=, /= и %=)` группируются **справа налево**. Все остальные операторы из этой таблицы группируются **слева направо**.

Операторы ¹						Группировка ассоциативность
<code>++</code>	<code>--</code>	<code>static_cast<тип></code>			<code>()</code>	Слева направо
<code>++</code>	<code>--</code>	<code>+</code>	<code>-</code>			Справа налево
<code>*</code>	<code>/</code>	<code>%</code>				Слева направо
<code>+</code>	<code>-</code>					Слева направо
<code><<</code>	<code>>></code>					Слева направо
<code><</code>	<code><=</code>	<code>></code>	<code>>=</code>			Слева направо
<code>==</code>	<code>!=</code>					Слева направо
<code>?:</code>						Справа налево
<code>=</code>	<code>+=</code>	<code>-=</code>	<code>*=</code>	<code>/=</code>	<code>%=</code>	Справа налево

¹ Обратите внимание, что операторы могут выглядеть совершенно одинаково, но при этом отличаться по своему действию, по приоритету и по ассоциативности. Приложение А в конце книги поможет вам уверенно различать такие операторы. — *Примеч. ред.*

3.11. Несовместимость основных типов данных на разных платформах

Здесь вы можете просмотреть полный список фундаментальных (основных) типов данных языка C++ и допустимые диапазоны значений для каждого типа:

<https://en.cppreference.com/w/cpp/language/types>

Как видите, эти диапазоны зависят от платформы (то есть от программно-аппаратной конструкции компьютера). В языках C и C++ число типа `int` на одной платформе может занимать в памяти 16 бит (2 байта) памяти, на другой — 32 бита (4 байта), а на третьей — 64 бита (8 байт). Поэтому код с переменными типа `int`, написанный для конкретной платформы, не обязательно будет работать на других платформах. Программист может написать несколько версий приложения для разных платформ или использовать методы, гарантирующие нужный уровень совместимости¹. В следующем разделе вы узнаете об одном из этих методов.

Рассмотрим причину названной проблемы на примере целочисленных типов `int`, `long` и `long long`. Стандарт C++ требует, чтобы размер переменных `int` в памяти был не менее 16 бит, переменных `long` — не менее 32 бит, а переменных `long long` — не менее 64 бит. Кроме того, переменные `int` должны быть по размеру не больше, чем `long`, а переменные `long` — не больше, чем `long long`. Из-за таких мягких требований иногда возникают проблемы с совместимостью, зато разработчики могут добиваться максимального быстродействия программ на каждой из платформ, сопоставляя размеры основных типов данных с конструктивными особенностями компьютеров.

3.12. Готовые объекты: работа с целыми числами произвольной величины

Допустимый диапазон значений для конкретного целочисленного типа данных зависит от того, сколько байтов занимает каждая переменная этого типа в памяти компьютера той или иной конструкции. Например, если переменная типа `int` занимает 4 байта, она может хранить 2^{32} возможных значений, то есть любое целое число в диапазоне от -2 147 483 648 до 2 147 483 647. На большинстве современных платформ переменная типа `long long` занимает в памяти 8 байт, то есть может хранить 2^{64} возможных значений в диапазоне от -9 223 372 036 854 775 808 до 9 223 372 036 854 775 807.

В некоторых приложениях нужны числа за пределами диапазона `long long`

Рассмотрим вычисление факториала. Факториал — это произведение всех целых чисел от 1 до заданного числа. Факториал числа 5 (пишется как 5!) равен

¹ Например, переменные целочисленных типов, определенных в заголовке `<cstdint>` (<https://en.cppreference.com/w/cpp/types/integer>), будут корректно работать в вашем приложении на разных платформах.

$1 \cdot 2 \cdot 3 \cdot 4 \cdot 5$, что составляет 120. Самое большое значение факториала, которое можно хранить в 64-битной переменной типа `long long`, — это $20!$, что составляет 2 432 902 008 176 640 000. Факториалы быстро выходят за пределы диапазона `long long`. И это лишь один характерный пример. В нашу эпоху для обработки быстро растущих объемов «больших данных» все больше реальных приложений должны выполнять такие вычисления, для которых типа данных `long long` уже недостаточно.

Одна из сфер применения чрезвычайно больших целых чисел — это криптография, необходимая для защиты информации, передаваемой между компьютерами через интернет. Многие криптографические алгоритмы выполняют вычисления со 128-битными или 256-битными целыми числами — значительно большими, чем можно хранить в переменных основных типов C++.

Класс `BigNumber` для работы с целыми числами произвольной величины

Любому приложению, использующему целые числа за пределами диапазона `long long`, нужен специально предназначенный для этого класс. К сожалению, в стандартной библиотеке C++ все еще нет класса для целых чисел произвольной величины. Поэтому мы погрузимся в огромный мир библиотек C++ с открытым исходным кодом, чтобы продемонстрировать вам один из тех классов, которые подходят для этой цели. Это класс `BigNumber`, опубликованный на портале GitHub:

<https://github.com/limeoats/BigNumber>

Для вашего удобства мы поместили этот класс в папку `libraries`, которая находится в папке `examples`. Ознакомьтесь с файлом `LICENSE.md`, содержащим условия лицензирования открытого исходного кода.

Для работы с классом `BigNumber` вам не обязательно понимать, как он реализован¹. Вы просто включаете в вашу программу его заголовочный файл `bignumber.h`, легко создаете объекты этого класса, а затем используете их в своем коде. На илл. 3.5 вы видите код программы с классом `BigNumber`, а также пример вывода данных. Мы взяли самое длинное число типа `long long`, чтобы показать, как создавать еще большие числа с помощью `BigNumber`. В конце этого раздела мы покажем, как скомпилировать и запустить эту программу с помощью выбранного вами компилятора.

```
1 // fig03_05.cpp
2 // Работа с целыми числами произвольной величины
3 #include <iostream>
4 #include "bignumber.h"
5 using namespace std;
6
7 int main() {
8     // Работаем с самым большим числом типа long long
```

¹ После глубокого погружения в C++ вы, возможно, захотите взглянуть на исходный код класса `BigNumber` (примерно 1000 строк), чтобы понять, как он реализован. А в тех главах книги, которые посвящены объектно-ориентированному программированию, вы узнаете о нескольких способах написать ваш собственный класс, работающий с очень большими целыми числами.

```

9  const long long value1{9'223'372'036'854'775'807LL}; // Предел long long
10 cout << "long long value1: " << value1
11     << "\nvalue1 - 1 = " << value1 - 1 // Правильный результат
12     << "\nvalue1 + 1 = " << value1 + 1; // Неправильный результат
13
14 // Работаем с числом типа BigInteger
15 const BigInteger value2{value1};
16 cout << "\n\nBigInteger value2: " << value2
17     << "\nvalue2 - 1 = " << value2 - 1 // Правильный результат
18     << "\nvalue2 + 1 = " << value2 + 1; // Правильный результат
19
20 // Вычисляем степени числа 100 000 000 типа long long
21 long long value3{100'000'000};
22 cout << "\n\nvalue3: " << value3;
23
24 int counter{2};
25
26 while (counter <= 5) {
27     value3 *= 100'000'000; // Скоро нам не хватит диапазона long long
28     cout << "\nvalue3 to the power " << counter << ": " << value3;
29     ++counter;
30 }
31
32 // Вычисляем степени числа 100 000 000 типа BigInteger
33 BigInteger value4{100'000'000};
34 cout << "\n\nvalue4: " << value4 << "\n";
35
36 counter = 2;
37
38 while (counter <= 5) {
39     cout << "value4.pow(" << counter << "): "
40         << value4.pow(counter) << "\n";
41     ++counter;
42 }
43
44 cout << "\n";
45 }

```

```
long long value1: 9223372036854775807
```

```
value1 - 1: 9223372036854775806
```

(OK)

```
value1 + 1: -9223372036854775808
```

(Ошибка!)

```
BigInteger value2: 9223372036854775807
```

```
value2 - 1: 9223372036854775806
```

(OK)

```
value2 + 1: 9223372036854775808
```

(OK)

```
value3: 100000000
```

```
value3 to the power 2: 10000000000000000
```

(OK)

```
value3 to the power 3: 2003764205206896640
```

(Ошибка!)

```
value3 to the power 4: -8814407033341083648
```

(Ошибка!)

```
value3 to the power 5: -5047021154770878464
```

(Ошибка!)

Выполнение тех же операций с объектом **BigNumber**

В строках 15–18 мы повторяем операции из строк 9–12, но теперь уже с числами типа **BigNumber**. Мы создаем объект **BigNumber** с именем `value2` и инициализируем его значением переменной `value1` — тем самым, которое невозможно превзойти в переменных типа `long long`:

```
BigNumber value2{value1};
```

Далее мы выводим значение **BigNumber**, затем вычитаем из него 1 и выводим результат. В строке 18 мы прибавляем 1 к `value2` (то есть к максимально возможному числу `long long`). Числа типа **BigNumber** могут быть любой величины, поэтому теперь вычисление выполняется правильно. Мы получили результат, недостижимый при работе с основными целочисленными типами C++ (по крайней мере, на имеющихся у нас компьютерах).

Класс **BigNumber** поддерживает все стандартные арифметические операторы, в том числе операторы сложения (+) и вычитания (-), которые есть в этой программе. Компилятор C++ по умолчанию умеет применять арифметические операторы к основным числовым типам C++, но ему надо «объяснить», как применять эти же операторы к объектам сторонних классов. Мы обсудим этот процесс, называемый перегрузкой операторов, в главе 11.

Возведение в степень числа 100 000 000 с целыми числами `long long`

В строках 21–30 число `100 000 000` типа `long long` возводится во 2-ю, 3-ю, 4-ю и 5-ю степень. Сначала мы создаем переменную `value3` и отображаем ее значение. Строки 26–30 повторяются в цикле 4 раза. Инstrukция:

```
value3 *= 100'000'000; // Скоро нам не хватит диапазона long long
```

умножает текущее значение `value3` на `100 000 000`. Это значит, что каждый раз `value3` возводится в следующую степень. Выходные данные показывают, что вычисление выполняется правильно только в первый раз.

Возведение в степень числа 100 000 000 с объектом **BigNumber**

Теперь мы демонстрируем возможность класса **BigNumber** работать с числами, значительно превосходящими пределы диапазонов всех основных целочисленных типов. В строках 33–42 число `100 000 000` снова возводится во 2-ю, 3-ю, 4-ю и 5-ю степень, но теперь оно имеет тип **BigNumber**. Сначала мы создаем переменную `value4` и выводим ее значение. Строки 38–42 повторяются в цикле 4 раза. Этот фрагмент кода вызывает функцию `pow` класса **BigNumber**, чтобы возвести значение переменной `value4` в степень `counter`¹:

```
value4.pow(counter)
```

¹ Мы решили показать вам функцию `pow` из класса **BigNumber**, но могли бы использовать и оператор `*`, как в строке 27.

Благодаря классу `BigNumber`, все вычисления выполняются корректно. Мы получаем огромные числа, что было бы невозможно с основными целочисленными типами на платформах Windows, macOS и Linux.

Класс `BigNumber` позволяет работать с любыми целыми числами независимо от вашей платформы, то есть на совершенно разных компьютерах результаты будут одинаковыми. Но вычисления выполняются медленнее, чем с основными целочисленными типами. Таким образом, вы жертвуете быстродействием в обмен на универсальность, предоставляемую классом `BigNumber`.

Компиляция и запуск программы с помощью Microsoft Visual Studio

В интегрированной среде разработки Microsoft Visual Studio:

1. Создайте новый проект, как описано в разделе 1.2.
2. В `Solution Explorer` щелкните правой кнопкой мыши на `Source Files` проекта и выберите `Add > Existing Item...`
3. Перейдите в папку `fig03_05`, выберите `fig03_05.cpp` и нажмите кнопку `Add`.
4. Повторите шаги 2 и 3 для файла `bignumber.cpp` из `examples\libraries\BigNumber\src`.
5. В `Solution Explorer` щелкните правой кнопкой мыши на имени проекта и выберите `Properties...`
6. В разделе `Configuration Properties` выберите `C/C++`. Затем в правой части диалогового окна добавьте в `Additional Include Directories` полный путь к папке `BigNumber\src` на своем компьютере. На нашем компьютере этот путь выглядит так:

```
C:\Users\ИмяПользователя\Documents\examples\libraries\BigNumber\src
```

7. Нажмите `OK`.
8. Нажмите `Ctrl + F5` для компиляции и запуска программы¹.

Компиляция и запуск программы с помощью GNU g++

Если вы работаете с компилятором `g++` из пакета `GNU Compiler Collection` (или в окне терминала macOS):

1. В командной строке перейдите в папку `fig03_05`, содержащую исходный код программы.
2. Чтобы скомпилировать программу, введите (в одну строку) следующую команду (параметр `-I` указывает на дополнительные папки, в которых компилятор будет искать заголовочные файлы):

```
g++ -std=c++2a -I ../../libraries/BigNumber/src fig03_05.cpp  
../../libraries/BigNumber/src/bignumber.cpp -o fig03_05
```

¹ Visual C++ может выдавать предупреждения, если в коде есть класс `BigNumber`.

3. Введите следующую команду для запуска программы:

```
./fig03_05
```

Компиляция и запуск программы с помощью Apple Xcode

В интегрированной среде разработки Apple Xcode:

1. Создайте новый проект, как описано в разделе 1.2, и удалите `main.cpp`.
2. Перетащите `fig03_05.cpp` из папки `fig03_05` в папку с исходным кодом вашего проекта в Xcode, затем нажмите кнопку **Finish** в появившемся диалоговом окне.
3. Перетащите `bignumber.h` и `bignumber.cpp` из папки `examples/libraries/bignumber/src` в папку с исходным кодом вашего проекта в Xcode, затем нажмите кнопку **Finish** в появившемся диалоговом окне.
4. Нажмите **Command + R** для компиляции и запуска программы.

3.13. Форматирование текста функцией `format`

В C++20 появились новые возможности форматирования текстовых строк с помощью функции `format` (для работы с ней достаточно включить в программу заголовок `<format>`). Форматирование стало значительно проще благодаря новому синтаксису — примерно такому же, как в языке Python, в языках Microsoft.NET (C# и Visual Basic) и в новом языке Rust¹. По сравнению с более ранними версиями C++ новый синтаксис стал лаконичнее и мощнее. Далее мы будем в первую очередь использовать эти новые возможности.

Форматирование строк по стандарту C++20 еще не реализовано в некоторых компиляторах

На момент написания этой главы (декабрь 2021 года) только Visual C++ поддерживает все возможности форматирования текста, предусмотренные стандартом C++20. Другие компиляторы пока опаздывают, но такие же возможности дает библиотека с открытым исходным кодом `{fmt}`, опубликованная на портале GitHub^{2,3}:

<https://github.com/fmtlib/fmt>

¹ Виктор Зверович (Victor Zverovich), «Text Formatting», 16 июля 2019 г. Был доступен: 15 сентября 2023 г. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0645r10.html>.

² Согласно предложению Комитета по стандартам C++. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0645r10.html>.

³ Средства форматирования текста по стандарту C++20 — это лишь часть функциональности библиотеки `{fmt}`.

В следующий пример программы мы включили библиотеку `{fmt}`¹. Для вашего удобства мы поместили ее в папку `libraries`, которая находится в папке `examples`. В конце этого раздела мы покажем, как скомпилировать и запустить эту программу с помощью выбранного вами компилятора. Ознакомьтесь с условиями лицензирования библиотеки по адресу:

<https://github.com/fmtlib/fmt/blob/master/LICENSE.rst>

Строка формата с заполнителями

Первым аргументом функции `format` является **строка формата (format string)**, содержащая **заполнители (placeholders)**, которые определяются фигурными скобками (`{}` и `}`). Функция `format` заменяет эти заполнители значениями других своих аргументов, как показано на илл. 3.6.

```
1 // fig03_06.cpp
2 // Форматирование строк в C++20
3 #include <iostream>
4 #include <fmt/format.h> // Или в C++20: #include <format>
5 using namespace std;
6 using namespace fmt; // Не требуется в C++20
7
8 int main() {
9     string student{"Paul"};
10    int grade{87};
11
12    cout << format("{}'s grade is {}\n", student, grade);
13 }
```



Paul's grade is 87

Илл. 3.6 | Форматирование строк в C++20

Заполнители заменяются слева направо

По умолчанию функция `format` заменяет заполнители в строке формата слева направо. Мы вызываем функцию `format`, передавая ей три аргумента. Первый из них — следующая строка формата:

```
"{}'s grade is {}\n"
```

¹ Некоторые наши примеры кода, связанные с нововведениями в C++20, могут быть несовместимы с вашим компилятором. Когда стандарт C++20 получит полную поддержку во всех основных компиляторах, мы повторно протестируем примеры кода, обновим их и опубликуем обновления для книги на нашем сайте: <https://deitel.com/c-plus-plus-20-for-programmers>. Пример кода в этом разделе компилируется, но требует библиотеку с открытым исходным кодом `{fmt}` для демонстрации тех возможностей, которые со временем будут поддерживаться всеми компиляторами C++20.

Функция заменяет в этой строке первый заполнитель значением переменной `student` (Paul), а второй заполнитель — значением переменной `grade` (87), после чего возвращает готовую строку:

```
"Paul's grade is 87"
```

Компиляция и запуск программы с помощью Microsoft Visual Studio

Выполните действия, аналогичные тем, которые описаны в разделе 3.12, но со следующими изменениями:

- ✦ На шаге 3 перейдите в папку `fig03_06` и добавьте `fig03_06.cpp` в `Source Files` вашего проекта.
- ✦ На шаге 4 перейдите в папку `examples\libraries\fmt\src` и добавьте файл `format.cc` в `Source Files` вашего проекта.
- ✦ На шаге 6 добавьте в `Additional Include Directories` полный путь к папке `include` библиотеки `{fmt}` на своем компьютере. На нашем компьютере этот путь выглядит так:

```
C:\Users\ИмяПользователя\Documents\examples\libraries\fmt\include
```

Компиляция и запуск программы с помощью GNU g++

Если вы работаете с компилятором `g++` из пакета `GNU Compiler Collection` (или в окне терминала `macOS`):

1. В командной строке перейдите в папку `fig03_06`, содержащую исходный код программы.
2. Чтобы скомпилировать программу, введите (в одну строку) следующую команду:

```
g++ -std=c++2a -I ../../libraries/fmt/include fig03_06.cpp  
../../libraries/fmt/src/format.cc -o fig03_06
```

3. Введите следующую команду для запуска программы:

```
./fig03_06
```

Компиляция и запуск программы с помощью Apple Xcode

Выполните действия, аналогичные тем, которые описаны в разделе 3.12, но с одним изменением: на шаге 2 перетащите файлы `fig03_06.cpp` и `format.cc`, а также папку `fmt` из папки `fig03_06` в папку с исходным кодом вашего проекта в Xcode.

3.14. Итоги

Для разработки любого алгоритма нужны только три вида управляющих инструкций: последовательность, выбор (ветвление) и цикл. Мы познакомили вас

с инструкцией одиночного выбора `if`, с инструкцией двойного выбора `if...else` и с инструкцией цикла `while`. Мы использовали вложенные управляющие инструкции, чтобы суммировать оценки учащихся и вычислять среднюю оценку с помощью циклов двух видов: со счетчиком и с контрольным значением. Мы использовали вложенные управляющие инструкции для анализа результатов экзамена и принятия решений. Мы рассказали о составных операторах присваивания, а также об операторах инкремента и декремента.

Мы объяснили, почему основные типы данных C++ на разных платформах могут отличаться по размеру в памяти компьютера, а затем использовали объекты класса `BigNumber` с открытым исходным кодом, чтобы работать с числами, выходящими за пределы диапазона основных типов. Наконец, мы показали, как форматировать текст с помощью библиотеки `{fmt}` с открытым исходным кодом. Когда в будущей версии вашего компилятора появится полная поддержка заголовка `<format>` из C++20¹, вы сможете точно так же форматировать текст без библиотеки `{fmt}`. В главе 4 мы расскажем о других управляющих инструкциях: `for`, `do...while` и `switch`, а также рассмотрим формирование составных условий с помощью логических операторов.

¹ Как правило, за то время, пока готовятся к печати русские версии книг, первоначально изданных на английском языке, разработчики компиляторов успевают реализовать все новшества, упомянутые в этих книгах. Скорее всего, в тот момент, когда вы читаете эту книгу, ваш компилятор уже на 100% поддерживает весь приведенный в ней исходный код во всех вариантах. — *Примеч. ред.*

Управляющие инструкции (часть 2)

В этой главе вы:

- научитесь использовать инструкции циклов `for` и `do...while`;
- научитесь использовать инструкцию множественного выбора `switch`;
- узнаете, зачем нужен атрибут `[[fallthrough]]` в инструкции `switch`;
- научитесь использовать инструкции выбора с инициализаторами;
- научитесь использовать инструкции `break` и `continue`, чтобы изменять поток управления;
- научитесь использовать логические операторы для формирования составных условий;
- узнаете об ошибках представления, возникающих при денежных вычислениях с плавающей точкой;
- продолжите работать с готовыми объектами и узнаете, как создавать и распаковывать архивы в формате ZIP с помощью общедоступной библиотеки;
- продолжите изучение новых средств форматирования текста, введенных в C++20.

4.1. Введение

В этой главе мы расскажем об управляющих инструкциях `for`, `do...while`, `switch`, `break` и `continue`. Мы детально рассмотрим, как работают циклы со счетчиком. Мы покажем пример расчета сложных процентов, начнем исследование проблем обработки денежных величин и обсудим ошибки представления при вычислениях с плавающей точкой. Затем с помощью инструкции `switch` мы подсчитаем количество оценок A, B, C, D и F в наборе данных о результатах экзамена. Мы покажем, как появившиеся в C++17 усовершенствования синтаксиса позволяют вам инициализировать переменную (или несколько переменных одного типа) в заголовках инструкций `if` и `switch`. Мы расскажем о логических операторах, позволяющих формировать составные условия из простых. Продолжая тему готовых объектов, мы займемся созданием и распаковкой ZIP-архивов с помощью библиотеки `miniz-cpp` с открытым исходным кодом. В конце главы мы представим вам еще несколько мощных и выразительных средств форматирования текста из C++20.

4.2. Как работает цикл со счетчиком

В главе 3 вы познакомились с инструкцией цикла `while`. Теперь пришло время подробнее изучить работу цикла со счетчиком, чтобы формализовать следующие элементы:

- ♦ **управляющая переменная (control variable)**, также называемая **счетчиком (counter)** цикла;
- ♦ **начальное значение (initial value)** управляющей переменной;
- ♦ **инкремент (increment)** управляющей переменной, выполняемый в каждой итерации цикла;
- ♦ **условие продолжения цикла (loop-continuation condition)**, определяющее, надо ли продолжать цикл.

Рассмотрим следующий код, в котором числа от 1 до 10 выводятся в цикле `while` (илл. 4.1).

Все элементы цикла `while` определены в строках 7, 9 и 11. В строке 7 объявляется управляющая переменная `counter` целочисленного типа `int`, для нее резервируется место в памяти и выполняется инициализация значением 1. Все **объявления (declarations)**, требующие инициализации, являются исполняемыми инструкциями. Объявления переменных, резервирующие место в памяти, называются **определениями (definitions)**, но мы обычно будем придерживаться общего термина «объявление», кроме тех случаев, когда это различие важно.

В строке 10 выводится на экран значение управляющей переменной `counter` — однократно в каждой итерации цикла. В строке 11 значение управляющей переменной увеличивается на 1 — тоже однократно в каждой итерации цикла. В строке 9 проверяется, является ли значение `counter` меньшим или равным 10 (это последнее

из тех чисел, для которых условие продолжения цикла остается истинным). Когда значение `counter` станет больше 10, цикл завершится.

```
1 // fig04_01.cpp
2 // Цикл while со счетчиком (с управляющей переменной)
3 #include <iostream>
4 using namespace std;
5
6 int main() {
7     int counter{1}; // Объявляем и инициализируем управляющую переменную
8
9     while (counter <= 10) { // Условие продолжения цикла
10         cout << counter << " ";
11         ++counter; // Инкремент управляющей переменной
12     }
13
14     cout << "\n";
15 }
```



1 2 3 4 5 6 7 8 9 10

Илл. 4.1 | Цикл `while` со счетчиком (с управляющей переменной)

Тип управляющей переменной в любом цикле обязательно должен быть целочисленным. Никакие типы данных с плавающей точкой для этого не годятся, поскольку их значения обычно являются приблизительными. При инкременте такой переменной могут возникать неточности, приводящие к неправильной проверке условия продолжения цикла и препятствующие его нормальному завершению.

4.3. Инструкция цикла `for`

Все элементы цикла `for` со счетчиком определяются в одной строке исходного кода. На илл. 4.2 показан вариант предыдущей программы, в котором мы заменили цикл `while` циклом `for`.

При выполнении инструкции `for` (занимающей строки 9–11) прежде всего объявляется и инициализируется значением 1 управляющая переменная `counter`. Далее программа проверяет условие продолжения цикла, указанное после первой точки с запятой (`counter <= 10`). Начальное значение `counter` равно 1, поэтому условие истинно и это значение выводится на экран в строке 10. После выполнения инструкции в строке 10 выражение `++counter`, указанное после второй точки с запятой, увеличивает `counter` на 1. Затем программа проверяет условие продолжения цикла, чтобы определить, надо ли переходить к следующей итерации. На данный момент значение `counter` равно 2, то есть условие по-прежнему истинно. Поэтому программа снова выполняет инструкцию в строке 10. Этот процесс продолжается до тех пор, пока цикл не выведет на экран все числа от 1 до 10. После этого `counter` примет значение 11, то есть условие продолжения цикла станет ложным и цикл

завершится. Выполнение программы продолжится со следующей инструкции после цикла, то есть со строки 13.

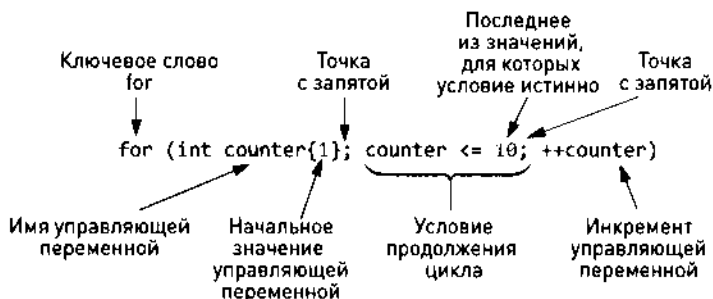
```
1 // fig04_02.cpp
2 // Цикл for со счетчиком (с управляющей переменной)
3 #include <iostream>
4 using namespace std;
5
6 int main() {
7     // Заголовок цикла for содержит инициализацию,
8     // условие продолжения цикла и инкремент
9     for (int counter{1}; counter <= 10; ++counter) {
10         cout << counter << " ";
11     }
12
13     cout << "\n";
14 }
```

1 2 3 4 5 6 7 8 9 10

Илл. 4.2 | Цикл for со счетчиком (с управляющей переменной)

Заголовок инструкции for

На следующей диаграмме мы обозначили основные элементы инструкции for на примере строки 9 из илл. 4.2:



Эта первая часть инструкции (ключевое слово `for` и все то, что находится далее в круглых скобках) называется **заголовком (header)** инструкции `for`, или заголовком цикла `for`. Все элементы, от которых зависит работа такого цикла, сосредоточены в его заголовке.

Общий формат инструкции for

Общий формат инструкции `for` таков:

```
for (инициализация; условие продолжения цикла; инкремент) {
    тело цикла
}
```

В заголовок входят три секции, разделенные точкой с запятой:

- ♦ *Инициализация* — выражение, в котором определяется имя управляющей переменной и ей присваивается начальное значение.
- ♦ *Условие продолжения цикла* — выражение, при истинности которого выполняется тело цикла.
- ♦ *Инкремент* — выражение, изменяющее значение управляющей переменной таким образом, чтобы условие продолжения цикла стало ложным после необходимого количества повторений.

Если условие продолжения цикла изначально является ложным, то тело цикла `for` (одна или несколько инструкций между фигурными скобками) не будет выполнено ни разу, а выполнение программы продолжится со следующей инструкции после `for`.

Область видимости управляющей переменной

Управляющую переменную, объявленную в выражении инициализации инструкции `for`, можно использовать только в этой же инструкции, но не за ее пределами. Такое ограничение называется **областью видимости (scope)** переменной. От области видимости зависит, из каких частей программы можно обращаться к переменной и когда эта переменная перестанет существовать. Например, область видимости переменной начинается в точке ее объявления и заканчивается правой фигурной скобкой, закрывающей блок. Как вы увидите в главе 5, каждую переменную рекомендуется определять только в минимально необходимой для нее области видимости.

Выражения в заголовке инструкции `for` необязательны

Все три выражения в заголовке `for` являются необязательными. Если вы опустите *условие продолжения цикла*, оно всегда будет считаться истинным (автоматически), то есть вы получите бесконечный цикл. Вы можете опустить выражение *инициализации*, если управляющая переменная инициализирована в программе еще до начала цикла. Вы можете опустить выражение *инкремента*, если в теле цикла есть инструкции, изменяющие управляющую переменную.

Указанное в заголовке `for` выражение инкремента фактически выполняется программой как отдельная инструкция в конце тела цикла. Поэтому выражение инкремента в заголовке можно заменить любым из следующих эквивалентных выражений в теле цикла:

```
counter = counter + 1
counter += 1
++counter
counter++
```

В этом случае выражение инкремента не является частью какого-либо другого выражения, поэтому префиксный и постфиксный операторы инкремента работают

одинаково. Мы предпочитаем оператор префиксного инкремента: он дает небольшое преимущество в быстродействии, как вы увидите в главе 11, посвященной перегрузке операторов.

Управляющая переменная в теле цикла

Многие программы выводят на экран меняющееся значение управляющей переменной или используют его в вычислениях в теле цикла, но это не обязательно делать. Значение управляющей переменной можно менять из тела цикла, но это может привести к трудно выявляемым ошибкам. Если программа должна менять значение управляющей переменной в теле цикла, лучше использовать цикл `while` вместо `for`.

UML-диаграмма цикла `for`

Ниже приведена диаграмма действий цикла `for` из кода, показанного на илл. 4.2. Диаграмма показывает, что инициализация управляющей переменной происходит только один раз: перед первой проверкой условия. Инкремент этой переменной происходит после каждого выполнения инструкции в теле цикла:



4.4. Инкремент и декремент счетчика в цикле `for`

В следующих примерах показаны варианты изменения значений управляющей переменной в цикле `for`. В каждом примере мы показываем только заголовок. Обратите внимание, что значения управляющей переменной можно не только увеличивать, но и уменьшать соответствующими операторами (декремент вместо инкремента).

А. Увеличение от 1 до 100 с шагом 1:

```
for (int i{1}; i <= 100; ++i)
```

Б. Уменьшение со 100 до 1 с шагом 1:

```
for (int i{100}; i >= 1; --i)
```

В. Увеличение от 7 до 77 с шагом 7:

```
for (int i{7}; i <= 77; i += 7)
```

Г. Уменьшение с 20 до 2 с шагом 2:

```
for (int i{20}; i >= 2; i -= 2)
```

Д. Возрастающий ряд чисел 2, 5, 8, 11, 14, 17, 20:

```
for (int i{2}; i <= 20; i += 3)
```

Е. Убывающий ряд чисел 99, 88, 77, 66, 55, 44, 33, 22, 11, 0:

```
for (int i{99}; i >= 0; i -= 11)
```

Не используйте операторы равенства (`!=` или `==`) в условии цикла, если управляющая переменная увеличивается или уменьшается более чем на 1. Например, посмотрите на ошибочно составленный цикл:

```
for (int counter{1}; counter != 10; counter += 2)
```

Здесь условие `counter != 10` никогда не станет ложным (и цикл будет повторяться бесконечно), потому что увеличение `counter` на 2 после каждого выполнения тела цикла дает в результате только нечетные числа (3, 5, 7, 9, 11, ...).

4.5. Программа для вычисления суммы четных чисел

Программа, показанная на илл. 4.3, использует цикл `for` для вычисления суммы четных целых чисел в диапазоне от 2 до 20 и сохраняет результат в переменной `total`. В каждой итерации цикла (строки 10–12) значение управляющей переменной `number` прибавляется к значению переменной `total`.

```
1 // fig04_03.cpp
2 // Суммирование целых чисел в цикле for
3 #include <iostream>
4 using namespace std;
5
6 int main() {
7     int total{0};
8
9     // Суммируем все четные числа от 2 до 20
10    for (int number{2}; number <= 20; number += 2) {
11        total += number;
12    }
13
14    cout << "Sum is " << total << "\n";
15 }
```

```
Sum is 110
```

Илл. 4.3 | Суммирование целых чисел в цикле `for`

Выражения инициализации и инкремента в заголовке цикла `for` могут состоять из нескольких выражений, разделенных запятыми. Хотя это не рекомендуется, вы можете перенести инструкцию из тела цикла `for` (строка 11) в секцию инкремента, используя запятую:

```
for (int number{2}; number <= 20; total += number, number += 2) { }
```

Запятая между выражениями `total += number` и `number += 2` — это **оператор «запятая» (comma operator)**. Он гарантирует, что выражения в списке будут вычисляться слева направо. Запятая имеет наименьший приоритет из всех операторов C++. Значением и типом всего списка выражений, разделенных запятыми, считаются значение и тип крайнего правого выражения соответственно. Оператор «запятая» часто используется в таких инструкциях `for`, где нужны несколько инициализирующих выражений или несколько выражений, меняющих значения управляющей переменной.

4.6. Программа для расчета сложных процентов

В этом разделе мы выполним расчет сложных финансовых процентов с помощью цикла `for`. Рассмотрим следующую задачу:

Человек делает вклад размером 1000 долларов на сберегательный счет с доходностью 5% годовых. Предполагая, что все проценты зачисляются на этот же счет, надо рассчитать и вывести на экран сумму на счете в конце каждого года на протяжении 10 лет. Используйте следующую формулу:

$$a = p(1 + r)^n$$

где:

p — первоначальный вклад (то есть основная сумма);

r — годовая процентная ставка (5% — это 0.05);

n — количество лет;

a — сумма на счете в конце года с номером *n*.

Решение показано на илл. 4.4. Для расчета результата каждого года в течение 10 лет используется цикл. Мы выбрали для денежных вычислений тип данных с плавающей точкой: `double`. Вскоре мы обсудим проблемы, возникающие при денежных вычислениях с плавающей точкой. Для финансовых приложений, требующих абсолютно точных денежных вычислений с контролем округления, можно использовать, например, библиотеку с открытым исходным кодом `Boost.Multiprecision`¹.

¹ Джон Мэддок (John Maddock) и Кристофер Корманьос (Christopher Kormanyos), «Chapter 1. Boost.Multiprecision». Был доступен: 15 сентября 2023 г. <https://www.boost.org/doc/libs/master/libs/multiprecision/doc/html/index.html>.

```

1 // fig04_04.cpp
2 // Расчет сложных процентов в цикле for
3 #include <iostream>
4 #include <iomanip>
5 #include <cmath> // Для функции возведения в степень
6 using namespace std;
7
8 int main() {
9     // Устанавливаем формат чисел с плавающей точкой
10    cout << fixed << setprecision(2);
11
12    double principal{1000.00}; // Первоначальный вклад
13    double rate{0.05}; // Процентная ставка
14
15    cout << "Initial principal: " << principal << "\n";
16    cout << "Interest rate: " << rate << "\n";
17
18    // Выводим заголовки таблицы
19    cout << "\nYear" << setw(20) << "Amount on deposit" << "\n";
20
21    // Рассчитываем сумму на счете в конце каждого года из десяти
22    for (int year{1}; year <= 10; ++year) {
23        // Рассчитываем сумму на счете в конце текущего года
24        double amount{principal * pow(1.0 + rate, year)} ;
25
26        // Выводим на экран год и сумму
27        cout << setw(4) << year << setw(20) << amount << "\n";
28    }
29 }

```

```

Initial principal: 1000.00
Interest rate: 0.05

```

Year	Amount on deposit
1	1050.00
2	1102.50
3	1157.63
4	1215.51
5	1276.28
6	1340.10
7	1407.10
8	1477.46
9	1551.33
10	1628.89

Илл. 4.4 | Расчет сложных процентов в цикле for

В строках 12–13 инициализируются две переменные типа `double`: `principal` получает значение `1000.00`, а `rate` — `0.05`. Компилятор C++ воспринимает числа с десятичной точкой (такие, как `1000.00` и `0.05`) как числа типа `double`, а целые

числа (такие, как 7 и -22) — как числа типа `int`¹. В строках 15–16 выводится на экран первоначальная величина вклада и процентная ставка.

Форматирование текста с выравниванием по ширине

Сочетание строки 10 перед циклом и строки 27 в теле цикла выводит значения переменных `year` и `amount`. Мы задаем формат вывода с помощью параметризованных манипуляторов потока `setprecision` и `setw` и непараметризованного манипулятора потока `fixed`. Манипулятор потока `setw(4)` определяет ширину поля (**field width**) для следующего выводимого на экран значения: 4 символа. Если выводимое значение содержит не более 4 символов, то есть помещается в указанную ширину поля (как в нашем примере с номером года), оно по умолчанию выравнивается по правому краю поля. Если выводимое значение не помещается в эту ширину, компилятор C++ все равно выводит его полностью, для чего расширяет поле вправо. Для выравнивания по левому краю поля добавьте непараметризованный манипулятор потока `left` (заголовок `<iostream>`). Чтобы вернуть выравнивание по правому краю, добавьте непараметризованный манипулятор потока `right`.

Другие средства форматирования в инструкциях вывода отображают переменную `amount` в виде числа с фиксированной десятичной точкой (`fixed` в строке 10) и с двумя цифрами после точки (`setprecision(2)` в строке 10), которое выровнено по правому краю в поле шириной 20 символов (`setw(20)` в строке 27). Напомним, что манипуляторы потока `fixed` и `setprecision` относятся к липким настройкам, то есть действуют до тех пор, пока вы их явно не измените. Мы применили их к потоку `cout` до начала цикла `for`, поэтому их уже не надо применять внутри цикла. Но ширина поля, указанная с помощью `setw`, не является липкой настройкой и действует только для текущего выводимого значения. В разделе 4.14 вы узнаете о других возможностях форматирования текста, добавленных в C++20.

Расчет процентов с помощью функции `pow` стандартной библиотеки

Цикл `for` (строки 22–28) повторяется 10 раз, при этом значение управляющей переменной `year` возрастает от 1 до 10 с шагом 1. Переменная `year` представляет число n в формуле из постановки задачи.

В C++ нет оператора возведения в степень, поэтому нам понадобилась функция `pow` стандартной библиотеки C++ (строка 24) из заголовка `<cmath>` (строка 5). Вызов функции `pow(x, y)` возвращает результат возведения числа x в степень y . Функция получает два аргумента типа `double` и возвращает значение этого же типа. Строка 24 выполняет вычисление по формуле $a = p(1 + r)^n$, где a — `amount`, p — `principal`, r — `rate` и n — `year`.

Первый аргумент функции `pow` — это выражение `1.0 + rate`. Оно не меняется в итерациях цикла, поэтому нет смысла вычислять его в теле цикла десять раз подряд.

¹ В разделе 3.12 мы показали, что целочисленных типов данных C++ недостаточно, чтобы хранить целые числа любой величины. Всегда выбирайте тип данных, подходящий для того диапазона значений, который вам нужен. Если вы хотите указать, что целочисленный литерал имеет тип `long` или `long long`, добавьте после литерала символы `L` или `LL` соответственно.

Многие современные оптимизирующие компиляторы в таких случаях выполняют вычисление однократно, до начала цикла, используя в скомпилированном коде цикла готовый результат. Это ускоряет работу программ.

Точность чисел с плавающей точкой и требования к памяти

Тип данных `float` представляет **число с плавающей точкой одинарной точности (single-precision floating-point number)**. В большинстве современных компьютеров такое число занимает 4 байта памяти, что соответствует точности примерно в 7 значащих цифр. Тип данных `double` представляет **число с плавающей точкой двойной точности (double-precision floating-point number)**. В большинстве современных компьютеров такое число занимает 8 байт памяти, что соответствует точности примерно в 15 значащих цифр — значительно больше по сравнению с `float`. Для вычислений с плавающей точкой программисты обычно предпочитают тип `double`. Все встречающиеся в исходном коде числа с десятичной точкой (например, 3.14159) компилятор C++ по умолчанию считает числами типа `double`. Такие числа в исходном коде называются **литералами с плавающей точкой (floating-point literals)**.

Согласно стандарту C++, тип `double` должен обеспечивать не меньшую точность, чем `float`. Кроме того, в C++ есть тип данных `long double`, который должен обеспечивать не меньшую точность, чем `double`. Напомним, что полный список основных (фундаментальных) типов данных языка C++ и их диапазонов опубликован здесь:

<https://en.cppreference.com/w/cpp/language/types>

Числа с плавающей точкой приближительны

В обычной арифметике числа с плавающей точкой часто возникают в результате деления. Разделив 10 на 3, вы получите 3.3333333..., причем последовательность цифр 3 после десятичной точки повторяется бесконечно. Но для хранения любого подобного числа в переменной компьютер выделяет фиксированный объем памяти, поэтому значение такой переменной практически всегда будет приближительным. Иными словами, типы данных с плавающей точкой (такие, как `double`) страдают от так называемой **ошибки представления (representational error)**. Если программист предполагает, что числа с плавающей точкой идеально точны (например, использует их в операциях сравнения, проверяя на равенство), это может привести к ошибкам в программе.

У типов данных с плавающей точкой есть множество применений, особенно для работы с измеряемыми величинами. Например, измеряемой величиной является температура человеческого тела (в норме — 36.6 градуса по Цельсию). Когда термометр показывает 36.6, на самом деле это может быть 36.659473213..., но нам не нужны все эти цифры. Представление температуры тела с тремя значащими цифрами (например, 36.6) хорошо подходит для реальных приложений медицинского характера, то есть в данном случае вполне достаточно типа `float`. И все-таки в большинстве других приложений тип данных `double` предпочтительнее, поскольку он представляет числа с плавающей точкой намного точнее, чем `float`¹.

¹ Современный стандарт представления чисел с плавающей точкой — IEEE 754 (https://en.wikipedia.org/wiki/IEEE_754).

Неточности в отображении округленных чисел

В примере с расчетом сложных процентов мы выбрали для переменных `amount`, `principal` и `rate` тип `double`. К сожалению, типы с плавающей точкой могут приводить к проблемам при денежных вычислениях. Посмотрите, что может случиться, если вы возьмете две переменные с плавающей точкой для представления денежных величин с двумя цифрами после точки (доллары и центы). Предположим, первая переменная хранит число 14.234 (округляемое до 14.23 при выводе на экран), а вторая — число 18.673 (округляемое до 18.67 при выводе на экран). При сложении этих чисел компилятор получит число 32.907, которое будет представлено на экране как 32.91. И ваша программа выведет такие данные:

```
14.23
+ 18.67
-----
32.91
```

Человек, суммирующий числа в том виде, в каком они отображаются на экране, воспримет это как ошибку программы, так как ожидает увидеть сумму 32.90. Имейте это в виду!

Пример ошибки представления денежных величин в числах с плавающей точкой

Ошибки представления могут возникать даже с самыми простыми (и на первый взгляд точными) денежными величинами, если хранить их в переменных типа `double`. Чтобы убедиться в этом, мы написали тестовую программу, которая определила переменную `d` следующим образом:

```
double d{123.02};
```

Затем мы вывели на экран значение `d`, отформатировав его до точности с 20 цифрами после точки. Оказалось, что в этом случае программа отображает число 123.02 как 123.0199999... — это наглядный пример ошибки представления. Иногда денежные значения могут быть точно представлены в переменных `double`, а иногда неточно. Эта распространенная проблема есть во многих языках программирования. Далее мы покажем, как создать и использовать классы, позволяющие выполнять денежные расчеты идеально точно.

4.7. Инструкция цикла `do...while`

Напомним, что в инструкции `while` условие продолжения цикла проверяется перед выполнением тела цикла. Если это условие изначально ложно, тело цикла не будет выполнено ни разу. В инструкции `do...while` условие продолжения цикла проверяется после выполнения тела цикла, которое, таким образом, всегда выполняется хотя бы один раз. На илл. 4.5 показано, как числа от 1 до 10 выводятся в цикле `do...while`. В строке 7 объявляется и инициализируется управляющая переменная `counter`. В начале тела цикла `do...while` (строка 10) значение этой переменной выводится на экран, а в строке 11 оно увеличивается на 1. В конце тела

цикла программа проверяет условие продолжения (строка 12). Если условие истинно, цикл продолжается с первой инструкции его тела (строка 10). Если условие ложно, цикл завершается и программа продолжает выполняться со следующей после цикла инструкции.

```

1 // fig04_05.cpp
2 // Цикл do...while
3 #include <iostream>
4 using namespace std;
5
6 int main() {
7     int counter{1};
8
9     do {
10         cout << counter << " ";
11         ++counter;
12     } while (counter <= 10); // Конец цикла
13
14     cout << "\n";
15 }

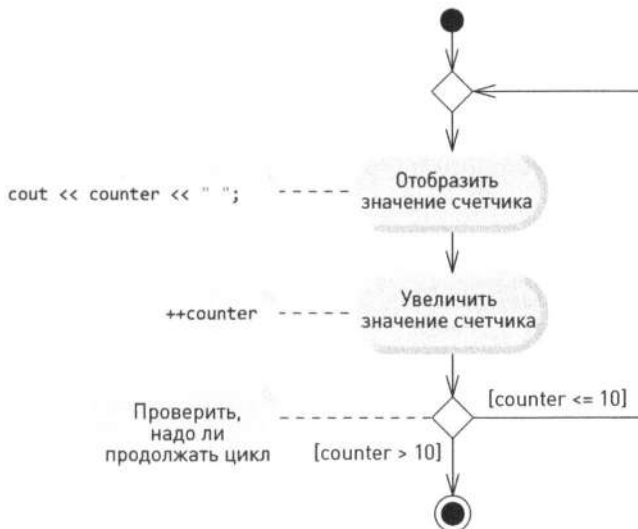
```

1 2 3 4 5 6 7 8 9 10

Илл. 4.5 | Цикл do...while

UML-диаграмма цикла do...while

Диаграмма действий цикла do...while наглядно показывает, что условие выполнения цикла не будет проверяться, пока не будут выполнены хотя бы один раз инструкции в теле цикла:



4.8. Инструкция множественного выбора switch

Инструкция `switch` предназначена для выбора одного из нескольких альтернативных действий в зависимости от возможных значений переменной или выражения. Каждое из действий связано со значением **целочисленного константного выражения** (**integral constant expression**) — любой комбинации символьных и целочисленных констант, которая вычисляется как целочисленная константа.

Инструкция `switch` для подсчета оценок учеников в категориях A, B, C, D и F

На илл. 4.6 показан код, анализирующий введенные пользователем оценки учеников. Как и прежде, программа вычисляет среднюю оценку по классу. Кроме того, инструкция `switch` относит каждую оценку к одной из пяти категорий (A, B, C, D или F) и увеличивает соответствующий счетчик. Затем программа выводит данные о том, сколько учеников получили оценку в каждой из этих категорий.

```
1 // fig04_06.cpp
2 // Подсчет оценок учеников с помощью инструкции switch
3 #include <iostream>
4 #include <iomanip>
5 using namespace std;
6
7 int main() {
8     int total{0}; // Сумма оценок
9     int gradeCounter{0}; // Счетчик введенных оценок
10    int aCount{0}; // Счетчик оценок A
11    int bCount{0}; // Счетчик оценок B
12    int cCount{0}; // Счетчик оценок C
13    int dCount{0}; // Счетчик оценок D
14    int fCount{0}; // Счетчик оценок F
15
16    cout << "Enter the integer grades in the range 0-100.\n"
17         << "Type the end-of-file indicator to terminate input:\n"
18         << " On UNIX/Linux/macOS type <Ctrl> d then press Enter\n"
19         << " On Windows type <Ctrl> z then press Enter\n";
20
21    int grade;
22
23    // Цикл продолжается, пока пользователь не введет индикатор конца файла
24    while (cin >> grade) {
25        total += grade; // Прибавляем grade к total
26        ++gradeCounter; // Увеличиваем счетчик введенных оценок
27
28        // Увеличиваем счетчик оценок соответствующей категории
29        switch (grade / 10) {
30            case 9: // Оценка от 90
31            case 10: // до 100 включительно
32                ++aCount;
33                break; // Выходим из switch
```

```

34
35     case 8: // Оценка от 80 до 89
36         ++bCount;
37         break; // Выходим из switch
38
39     case 7: // Оценка от 70 до 79
40         ++cCount;
41         break; // Выходим из switch
42
43     case 6: // Оценка от 60 до 69
44         ++dCount;
45         break; // Выходим из switch
46
47     default: // Оценка меньше 60
48         ++fCount;
49         break; // Не обязательно, так как мы все равно выходим из switch
50 } // Выходим из switch
51 } // Выходим из цикла
52
53 // Устанавливаем формат для чисел с плавающей точкой
54 cout << fixed << setprecision(2);
55
56 // Выводим заголовок отчета об оценках
57 cout << "\nGrade Report:\n";
58
59 // Если пользователь ввел хотя бы одну оценку...
60 if (gradeCounter != 0) {
61     // Вычисляем среднюю оценку по классу
62     double average{static_cast<double>(total) / gradeCounter};
63
64     // Выводим все результаты
65     cout << "Total of the " << gradeCounter << " grades entered is "
66         << total << "\nClass average is " << average
67         << "\nNumber of students who received each grade:"
68         << "\nA: " << aCount << "\nB: " << bCount << "\nC: " << cCount
69         << "\nD: " << dCount << "\nF: " << fCount << "\n";
70 }
71 else { // Не введена ни одна оценка, выводим сообщение об этом
72     cout << "No grades were entered" << "\n";
73 }
74 }

```

Enter the integer grades in the range 0-100.
 Type the end-of-file indicator to terminate input:
 On UNIX/Linux/macOS type <Ctrl> d then press Enter
 On Windows type <Ctrl> z then press Enter

```

99
92
45
57
63
71

```



```

76
85
90
100
^Z

```

Grade Report:

Total of the 10 grades entered is 778

Class average is 77.80

Number of students who received each grade:

A: 4

B: 1

C: 2

D: 1

F: 2

Илл. 4.6 | Подсчет оценок учеников с помощью инструкции `switch`

В строках 8 и 9 объявляются две локальные переменные: `total` для суммы введенных оценок и `gradeCounter` для их количества. В строках 10–14 объявляются и инициализируются значением 0 пять переменных для счетчиков оценок по категориям. В строках 24–51 программа получает от пользователя произвольное количество целочисленных оценок. Это происходит в цикле с контрольным значением, который обновляет `total` и `gradeCounter`, а также увеличивает счетчик оценок соответствующей категории. В строках 54–73 выводится отчет, содержащий количество введенных оценок, среднюю арифметическую оценку по классу и количество учеников, получивших оценки в каждой из категорий.

Ввод оценок пользователем

В строках 16–19 программа запрашивает у пользователя целочисленную оценку или **индикатор конца файла (end-of-file indicator)**, означающий, что пользователь завершил ввод данных. Индикатор конца файла — это стандартная комбинация клавиш, указывающая, что данные больше не будут вводиться. В главе 8 вы узнаете, как используется этот индикатор, когда программа получает данные из файла.

Какая комбинация клавиш служит индикатором конца файла, зависит от компьютера (от его операционной системы). Например, в UNIX, Linux и macOS нажмите комбинацию `Ctrl + d`. Эта запись означает, что клавиши `Ctrl` и `d` надо нажать одновременно. В Windows нажмите комбинацию `Ctrl + z`.

В некоторых системах требуется дополнительное нажатие клавиши `Enter`. Кроме того, Windows обычно отображает на экране символы `^Z`, когда вы вводите индикатор конца файла, как показано на илл. 4.6.

В строках 24–51 программа получает данные от пользователя с помощью цикла `while`. В строке 24 ввод данных происходит непосредственно в условии инструкции `while`:

```
while (cin >> grade) {
```

В этом случае условие считается истинным (**true**), если в поток `cin` успешно введено значение типа `int`. А если введен индикатор конца файла, условие считается ложным (**false**).

Если условие истинно, в строке 25 `grade` прибавляется к `total`, а в строке 26 увеличивается счетчик `gradeCounter`; потом эти данные понадобятся для вычисления среднего арифметического. В строках 29–50 инструкция `switch` увеличивает счетчик оценок в соответствующей категории.

Обработка оценок

Инструкция `switch` (строки 29–50) определяет, какой из пяти счетчиков надо увеличить. Мы предполагаем, что пользователь правильно ввел оценку: целое число от 0 до 100. Мы распределили оценки по категориям таким образом: от 90 до 100 — А, от 80 до 89 — В, от 70 до 79 — С, от 60 до 69 — D, от 0 до 59 — F. Блок инструкции `switch` содержит несколько обозначенных метками (**labels**) секций `case` и необязательную секцию `default` (ее можно разместить в любом месте инструкции `switch`, но обычно ее размещают последней). Они определяют, какой из счетчиков надо увеличить в зависимости от введенной пользователем оценки.

Когда поток управления достигает инструкции `switch`, программа вычисляет **управляющее выражение** (**controlling expression**), указанное в круглых скобках (`grade / 10`) после ключевого слова `switch`. Программа сравнивает значение этого выражения с каждой из меток `case`. Выражение должно иметь целочисленный тип со знаком или без знака: `bool`, `char`, `char8_t`, `char16_t`, `char32_t`, `wchar_t`, `int`, `long` или `long long`.

При вычислении управляющего выражения в строке 29 выполняется целочисленное деление, отсекающее дробную часть результата. При целочисленном делении на 10 любого числа от 0 до 100 результат обязательно будет в диапазоне от 0 до 10. Мы используем несколько чисел из этого диапазона в метках `case`. Например, если пользователь ввел оценку 85, управляющее выражение вычисляется как 8. Инструкция `switch` сравнивает число 8 с каждой из меток `case`. Если найдено совпадение (`case 8`: в строке 35), выполняется инструкция после этой метки; в данном случае увеличивается счетчик `bCount` в строке 36, так как оценка в диапазоне от 80 до 89 входит в категорию В. Инструкция `break` (строка 37) завершает инструкцию `switch`. Здесь достигается конец цикла `while`, поэтому управление возвращается к проверке условия в строке 24, от которой зависит, будет ли повторен цикл.

В метках `case` инструкции `switch` проверяется совпадение с числами 10, 9, 8, 7 и 6. Обратите внимание на строки 30–31, в которых проверяются числа 9 и 10 (они относятся к категории А). Эта последовательность из двух меток `case` без каких-либо инструкций между ними приводит к одному и тому же результату в двух случаях: если управляющее выражение вычисляется как 9 или 10, выполняются инструкции в строках 32–33. Инструкция `switch` не дает возможности проверять диапазоны чисел, поэтому каждое число, которое вам нужно проверить, должно быть указано в отдельной метке. Каждая секция `case` может выполнять несколько

перечисленных после нее инструкций, которые не требуется заключать в фигурные скобки (этим `switch` отличается от других управляющих инструкций), если вы не объявляете переменную внутри секции `case`.

Секция `case` без инструкции `break` и атрибут `[[fallthrough]]` (C++17)

При совпадении управляющего выражения с любой из меток `case` программа будет выполнять не только все инструкции из этой секции `case`, но и все инструкции из последующих секций `case` — до тех пор, пока их не прервет инструкция `break` или конец инструкции `switch`. Это называется **провалом** (*falling through*) в последующие секции `case`¹.

Отсутствие инструкции `break`, когда она необходима, — это логическая ошибка. Чтобы привлечь ваше внимание к возможной проблеме, многие компиляторы выдают предупреждение, если среди инструкций какой-либо секции `case` нет инструкции `break`. А для тех случаев, когда вы намеренно делаете провал в последующие секции `case`, в C++17 был введен **атрибут** (*attribute*) `[[fallthrough]]`. Если вы хотите сообщить компилятору, что отсутствие `break` — это правильное поведение программы, поместите вместо `break` следующую инструкцию:

```
[[fallthrough]];
```

Секция `default`

Если не найдено совпадения ни с одной из меток `case`, выполняется секция `default` (строки 47–49). В этом примере мы используем секцию `default` для обработки любых значений управляющего выражения, которые меньше 6, — все они относятся к категории F («экзамен не сдан»). Если не найдено совпадения ни с одной из меток `case`, а инструкция `switch` не содержит секции `default`, выполнение программы продолжается со следующей инструкции после `switch`. Опытный программист всегда проверяет в инструкции `switch` все возможные значения управляющего выражения, и секция `default` помогает не пропустить ни одно из них.

Вывод отчета об оценках

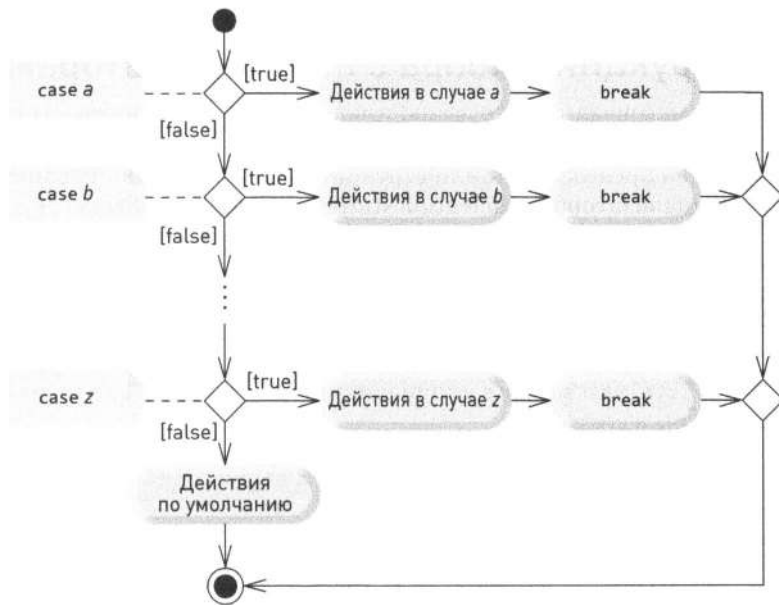
В строках 54–73 выводится отчет, основанный на введенных оценках. В строке 60 проверяется, ввел ли пользователь хотя бы одну оценку, — это предотвращает недопустимое деление на ноль. Целочисленное деление на ноль приводит к сбою программы, а результатом деления на ноль с плавающей точкой является специальное значение `NaN` — «Not a Number» («не число»). Если пользователь ввел хотя бы одну оценку, в строке 62 вычисляется среднее арифметическое. Затем в строках 65–69 выводится на экран: сумма оценок, средняя оценка по классу и ко-

¹ Это свойство инструкции `switch` хорошо подходит для простой программы, выводящей на экран текст известной песенки «Twelve Days of Christmas», в каждом куплете которой повторяется часть строк из предыдущего куплета. Попробуйте написать такую программу в качестве упражнения.

личество учеников, получивших оценки в каждой из буквенных категорий. Если оценки не вводились, в строке 72 отображается сообщение об этом. На илл. 4.6 показан пример отчета, основанного на десяти оценках.

UML-диаграмма инструкции switch

Ниже приведена обобщенная диаграмма действий для инструкции `switch`:



Почти всегда инструкция `switch` содержит в каждой из своих секций `case` инструкцию `break`, которая завершает `switch` после обработки этой секции. Это показано на диаграмме: каждая секция `case` завершается переходом к `break`, то есть управление немедленно передается на выход инструкции `switch`.

Инструкция `break` необязательна для самой последней секции (`case` или `default`), поскольку выполнение программы все равно продолжится со следующей инструкцией после `switch`. Всегда включайте секцию `default` в каждую инструкцию `switch`, чтобы предусмотреть все варианты действий даже в исключительных условиях.

Метки `case` должны быть константами

Каждая секция `case` в инструкции `switch` должна содержать целочисленную константу (например, 8, 9 или 10) или целочисленное константное выражение (любое выражение, вычисляемое как целочисленная константа). Также можно использовать перечисления `enum` (см. раздел 5.9) и **символьные литералы** (**character literals**), то есть определенные символы в одинарных кавычках (например, 'A', '7' или '\$'), представляющие целочисленные значения символов. В приложе-

нии Б в конце книги приведены целочисленные значения символов в кодировке ASCII, которая является подмножеством набора символов Unicode.

В главе 10 мы покажем вам, как более элегантно реализовать логику выбора нужного действия в зависимости от ситуации. Мы применим методику, называемую полиморфизмом, чтобы писать программы, которые нагляднее и проще для расширения функциональности, чем программы, в которых выбор осуществляется инструкциями `switch`.

4.9. Инструкции выбора с инициализаторами

Ранее мы рассматривали инструкцию цикла `for`. В ее заголовке мы объявили и инициализировали управляющую переменную, область видимости которой была ограничена пределами этой инструкции. В C++17 появились **инструкции выбора с инициализаторами (selection statements with initializers)** `if`, `if...else` и `switch`. Теперь вы можете инициализировать переменные перед условием в инструкциях `if` и `if...else`, а также перед управляющим выражением в инструкции `switch`. Как и в инструкции `for`, область видимости таких переменных ограничена пределами той инструкции, где они объявлены. На илл. 4.7 показаны примеры инструкций `if...else` с инициализацией. А в главе 5 (илл. 5.5) вы увидите, как инструкции `if...else` и `switch` с инициализацией используются в программе для популярной игры в кости.

```
1 // fig04_07.cpp
2 // Инструкции if с инициализаторами
3 #include <iostream>
4 using namespace std;
5
6 int main() {
7     if (int value{7}; value == 7) {
8         cout << "value is " << value << "\n";
9     }
10    else {
11        cout << "value is not 7; it is " << value << "\n";
12    }
13
14    if (int value{13}; value == 9) {
15        cout << "value is " << value << "\n";
16    }
17    else {
18        cout << "value is not 9; it is " << value << "\n";
19    }
20 }
```



```
value is 7
value is not 9; it is 13
```

Илл. 4.7 | Инструкции `if` с инициализаторами

Синтаксис инструкций выбора с инициализаторами

В инструкциях `if` и `if...else` инициализатор переменной помещается в круглые скобки перед условием. В инструкции `switch` инициализатор также помещается в круглые скобки перед управляющим выражением. Инициализатор должен заканчиваться точкой с запятой (;), как показано в строках 7 и 14. Инициализатор может объявить несколько переменных одного типа в списке, разделенном запятыми.

Область видимости переменных, объявленных в инициализаторе

Любая переменная, объявленная в инициализаторе инструкции `if`, `if...else` или `switch`, видна только в пределах этой инструкции. Мы используем переменную `value` сначала в строках 7–12, чтобы определить, какую ветвь инструкции `if...else` надо выполнить, а затем — в инструкциях вывода обеих ветвей. Когда инструкция `if...else` завершается, эта переменная перестает существовать, поэтому мы можем снова использовать этот же идентификатор (имя `value`) в следующей инструкции `if...else`, чтобы объявить новую переменную, которая будет видна только в пределах этой инструкции.

Чтобы подтвердить недоступность `value` вне инструкций `if...else`, мы написали вторую версию этой программы (`fig04_07_with_error.cpp`), которая пытается получить доступ к переменной `value` после завершения второй инструкции `if...else` (после — значит вне области видимости). Это приводит к следующим ошибкам компиляции:

- ♦ Visual Studio: 'value': undeclared identifier
- ♦ Xcode: error: use of undeclared identifier 'value'
- ♦ GNU g++: error: 'value' was not declared in this scope

4.10. Инструкции `break` и `continue`

В дополнение к инструкциям выбора и циклов в C++ есть инструкции `break` и `continue`, предназначенные для изменения потока управления. В предыдущем разделе было показано, как `break` прерывает выполнение инструкции `switch`. В этом разделе мы обсудим, как использовать `break` в инструкциях циклов.

Инструкция `break`

Выполнение инструкции `while`, `for`, `do...while` или `switch` немедленно прерывается (завершается), как только выполняется вложенная в нее инструкция `break`. Программа продолжит выполняться со следующей инструкции. Обычно `break` применяют для раннего выхода из цикла или из инструкции `switch` (как на илл. 4.6). На илл. 4.8 показан пример раннего завершения цикла `for` с помощью `break`.

```
1 // fig04_08.cpp
2 // Прерывание цикла for инструкцией break
3 #include <iostream>
4 using namespace std;
5
6 int main() {
7     int count; // Управляющая переменная, также используемая после цикла
8
9     for (count = 1; count <= 10; ++count) { // Повторяем цикл 10 раз
10         if (count == 5) {
11             break; // Но прерываем цикл, если значение count равно 5
12         }
13
14         cout << count << " ";
15     }
16
17     cout << "\nBroke out of loop at count = " << count << "\n";
18 }
```

```
1 2 3 4
Broke out of loop at count = 5
```

Илл. 4.8 | Прерывание цикла for инструкцией break

Когда инструкция `if` (строки 10–12), вложенная в цикл `for` (строки 9–15), обнаруживает, что значение `count` равно 5, выполняется инструкция `break` в строке 11. Это прерывает цикл `for`. Программа переходит к строке 17 (следующая инструкция после цикла `for`), в которой выводится сообщение со значением переменной `count` после завершения цикла. Тело цикла полностью выполняется только четыре раза вместо десяти. Обратите внимание, что мы могли бы инициализировать `count` в строке 7 и оставить пустой секцию инициализации заголовка цикла `for`:

```
for (; count <= 10; ++count) { // Повторяем цикл 10 раз
```

Инструкция `continue`

Как только выполняется инструкция `continue`, вложенная в тело цикла `while`, `for` или `do...while`, все последующие инструкции в теле цикла пропускаются и начинается следующая итерация цикла. В инструкциях `while` и `do...while` условие продолжения цикла проверяется сразу после выполнения инструкции `continue`. В инструкции `for` сначала выполняется инкремент управляющей переменной, а затем проверяется условие продолжения цикла.

```
1 // fig04_09.cpp
2 // Прерывание текущей итерации цикла for инструкцией continue
3 #include <iostream>
4 using namespace std;
5
6 int main() {
7     for (int count{1}; count <= 10; ++count) { // Повторяем цикл 10 раз
8         if (count == 5) {
```

```

9         continue; // Но прерываем цикл, если значение count равно 5
10    }
11
12    cout << count << " ";
13 }
14
15 cout << "\nUsed continue to skip printing 5" << "\n";
16 }

```

```

1 2 3 4 6 7 8 9 10
Used continue to skip printing 5

```

Илл. 4.9 | Прерывание текущей итерации цикла `for` инструкцией `continue`

Инструкция `continue` (строка 9) используется для того, чтобы пропустить строку 12, когда вложенная инструкция `if` обнаружит, что значение `count` равно 5. Сразу после инструкции `continue` выполнение программы продолжится с инкремента управляющей переменной в инструкции `for` (строка 7).

Некоторые программисты считают, что `break` и `continue` нарушают принципы структурного программирования. Поскольку аналогичные переходы можно реализовать методами структурного программирования, эти программисты избегают инструкций `break` и `continue`.

Требования к высокому качеству программного продукта часто вступают в противоречие с требованиями к его быстродействию. Иногда одна из этих целей достигается в ущерб другой. В любой ситуации (особенно при создании высоконагруженных приложений) старайтесь прежде всего делать код простым и правильным. И только после этого приступайте к оптимизации кода, чтобы сделать его быстрее и компактнее, — но только если это действительно необходимо.

4.11. Логические операторы

Условия в инструкциях `if`, `if...else`, `while`, `do...while` и `for` определяют, каким будет поток управления программой. До сих пор мы использовали только простые условия, например: `count <= 10`, `number != sentinelValue` или `total > 1000`. Простые условия выражаются с помощью операторов сравнения `>`, `<`, `>=`, `<=` и операторов равенства `==` и `!=`. Каждый из них проверяет только одно условие. Иногда в управляющих инструкциях нужны более сложные условия, от которых зависит поток управления. **Логические операторы (logical operators)** языка C++ позволяют составлять комбинации из простых условий. Логическими операторами являются: `&&` (логическое И), `||` (логическое ИЛИ) и `!` (логическое НЕ).

4.11.1. Оператор `&&` (логическое И)

Предположим, что в какой-то точке программы мы хотим проверить истинность двух условий одновременно, чтобы выбрать следующее действие.

Для этого нам понадобится оператор **&&** — **логическое И (logical AND)**:

```
if (gender == FEMALE && age >= 65) {  
    ++seniorFemales;  
}
```

Предположим, что **FEMALE** — это константа (константная переменная). Инструкция **if** содержит два простых условия. Условие **gender == FEMALE** определяет, является ли человек женщиной. Условие **age >= 65** определяет, является ли человек пожилым. Оператор **&&** объединяет два простых условия в одно составное:

```
gender == FEMALE && age >= 65
```

Оно будет истинно, если истинны оба простых условия одновременно. В этом случае инструкция в теле **if** увеличивает переменную **seniorFemales** на 1. Если же хотя бы одно из простых условий является ложным, программа пропускает инструкцию в теле **if**. Некоторые программисты рекомендуют добавлять для наглядности необязательные круглые скобки:

```
(gender == FEMALE) && (age >= 65)
```

Следующая **таблица истинности (truth table)** показывает все четыре комбинации логических значений для двух выражений, объединенных оператором **&&**. Любое выражение, содержащее операторы равенства, операторы сравнения или логические операторы, вычисляется компилятором C++ как **false** (ноль) или **true** (любое значение, кроме нуля).

Выражение 1	Выражение 2	Выражение 1 && Выражение 2
false	false	false
false	true	false
true	false	false
true	true	true

4.11.2. Оператор **||** (логическое ИЛИ)

Теперь предположим, что мы хотим проверить истинность как минимум одного из двух условий, чтобы выбрать следующее действие. Для этого нам понадобится оператор **||** — **логическое ИЛИ (logical OR)**:

```
if ((semesterAverage >= 90) || (finalExam >= 90)) {  
    cout << "Student grade is A\n";  
}
```

Здесь два простых условия тоже объединены в одно составное. Условие **semesterAverage >= 90** определяет, заслуживает ли студент пятерки по курсу за

хорошую успеваемость в течение семестра. Условие `finalExam >= 90` определяет, заслуживает ли студент пятерки по курсу за выдающийся результат на итоговом экзамене. Инструкция `if` проверяет составное условие:

```
(semesterAverage >= 90) || (finalExam >= 90)
```

и присуждает студенту оценку A (пятерку), если выполняется одно или оба простых условия. Сообщение `Student grade is A` не будет выведено, если оба простых условия окажутся ложными. Ниже приведена таблица истинности для двух выражений, объединенных оператором `||`.

Выражение 1	Выражение 2	Выражение 1 Выражение 2
false	false	false
false	true	true
true	false	true
true	true	true

Оператор `&&` имеет более высокий приоритет, чем оператор `||`¹. Оба оператора группируются слева направо.

4.11.3. Вычисление по короткой схеме

Простые условия, объединенные операторами `&&` или `||` в выражение, вычисляются слева направо только до тех пор, пока программа еще не выяснила, является ли это выражение истинным или ложным. Например, вычисление этого выражения сразу прекращается, если первое условие (`gender == FEMALE`) оказалось ложным (в этом случае все выражение тоже будет ложным):

```
(gender == FEMALE) && (age >= 65)
```

Если же первое условие оказалось истинным, то продолжается вычисление второго условия (`age >= 65`). Если и оно окажется истинным, то все выражение тоже будет истинным. Этот способ вычисления выражений с операторами `&&` и `||` называется **вычислением по короткой схеме (short-circuit evaluation)**.

В некоторых выражениях с оператором `&&` одно из условий (мы будем называть его зависимым условием) может требовать, чтобы другое условие было истинным; иначе вычисление зависимого условия может оказаться невозможным. В таких случаях надо поместить зависимое условие после оператора `&&`, чтобы предотвратить ошибку. Например, рассмотрим выражение `(1 != 0) && (10 / i == 2)`. Мы поместили зависимое условие `(10 / i == 2)` после оператора `&&`, чтобы исключить

¹ Всегда используйте круглые скобки, если без них возможна двусмысленность в отношении порядка вычислений.

возможность деления на ноль (второе условие не будет вычисляться, если первое окажется ложным).

4.11.4. Оператор ! (логическое НЕ)

Теперь рассмотрим оператор **!** — **логическое НЕ (logical NOT)**, или **логическое отрицание (logical negation)**. Он меняет значение условия на противоположное. В отличие от логических операторов **&&** и **||**, которые являются бинарными (объединяющими два условия), логический оператор **!** является унарным (он имеет только одно условие в качестве операнда). Чтобы выполнить код только тогда, когда условие ложно (**false**), поместите перед этим условием оператор логического отрицания:

```
if (!(grade == sentinelValue)) {  
    cout << "The next grade is " << grade << "\n";  
}
```

В этом примере инструкция в теле **if** выполняется, если **grade** не равно **sentinelValue**. Круглые скобки вокруг условия **grade == sentinelValue** необходимы, поскольку оператор логического отрицания имеет более высокий приоритет, чем оператор равенства.

В большинстве случаев вы можете обойтись без логического отрицания, выразив то же самое условие с помощью подходящего оператора равенства или сравнения. Например, предыдущую инструкцию можно записать другим способом:

```
if (grade != sentinelValue) {  
    cout << "The next grade is " << grade << "\n";  
}
```

Такая гибкость помогает выражать любое условие наиболее понятно. Ниже приведена таблица истинности для оператора логического отрицания.

Выражение	!Выражение
false	true
true	false

4.11.5. Программное создание таблиц истинности для логических операторов

На илл. 4.10 показана программа, использующая логические операторы для создания таблиц истинности. Она выводит каждое вычисленное выражение и его результат типа **bool**. По умолчанию оба значения типа **bool** (**true** и **false**) выводятся через **cout** как **1** и **0** соответственно, но функция **format** позволяет выводить их в виде слов **true** и **false**. В строках 10–14, 17–21 и 24–26 формируются таблицы истинности для операторов **&&**, **||** и **!** соответственно.

```

1 // fig04_10.cpp
2 // Логические операторы
3 #include <iostream>
4 #include <fmt/format.h> // Или в C++20: #include <format>
5 using namespace std;
6 using namespace fmt; // Не требуется в C++20
7
8 int main() {
9     // Составляем таблицу истинности для оператора && (логическое И)
10    cout << "Logical AND (&&)\n"
11         << format("false && false: {}\n", false && false)
12         << format("false && true: {}\n", false && true)
13         << format("true && false: {}\n", true && false)
14         << format("true && true: {}\n\n", true && true);
15
16    // Составляем таблицу истинности для оператора || (логическое ИЛИ)
17    cout << "Logical OR (||)\n"
18         << format("false || false: {}\n", false || false)
19         << format("false || true: {}\n", false || true)
20         << format("true || false: {}\n", true || false)
21         << format("true || true: {}\n\n", true || true);
22
23    // Составляем таблицу истинности для оператора ! (логическое НЕ)
24    cout << "Logical negation (!)\n"
25         << format("!false: {}\n", !false)
26         << format("!true: {}\n", !true);
27 }

```

```

Logical AND (&&)
false && false: false
false && true: false
true && false: false
true && true: true

```

```

Logical OR (||)
false || false: false
false || true: true
true || false: true
true || true: true

```

```

Logical negation (!)
!false: true
!true: false

```

Илл. 4.10 | Логические операторы

Приоритеты и правила группировки операторов

В следующей таблице показаны приоритеты и правила группировки всех представленных на данный момент операторов C++. Они перечислены сверху вниз в порядке убывания приоритетов.

Операторы ¹						Группировка (ассоциативность)
++	--	static_cast<тип>			()	Слева направо
++	--	+	-	!		Справа налево
*	/	%				Слева направо
+	-					Слева направо
<<	>>					Слева направо
<	<=	>	>=			Слева направо
==	!=					Слева направо
&&						Слева направо
						Слева направо
?:						Справа налево
=	+=	--	*=	/=	%=	Справа налево
,						Слева направо

4.12. Типичные ошибки при работе с операторами равенства (==) и присваивания (=)

Есть логические ошибки, которые даже у опытных программистов C++ случаются так часто, что заслуживают отдельного раздела. Внешне похожие операторы равенства (==) и присваивания (=) легко перепутать, и эти опечатки усугубляются тем, что при случайной замене одного из операторов другим обычно не возникает ошибка компиляции. Инструкции с такими опечатками могут компилироваться и выполняться, но из-за них программа работает с логическими ошибками. Современные компиляторы могут выдавать предупреждения, когда программист ставит оператор присваивания (=) там, где ожидается ==. В конце раздела вы узнаете, как включить эту функцию.

Названная проблема обусловлена двумя особенностями языка C++. Во-первых, любое вычисляемое выражение может использоваться в секции принятия решения любой управляющей инструкции. Если значение выражения равно 0, оно обрабатывается как false. Если значение не равно 0, оно обрабатывается как true. Во-вторых, при присваивании создается значение, а именно значение переменной в левой части оператора присваивания. Предположим, вы хотели написать:

```
if (payCode == 4) { // Правильно
    cout << "You get a bonus!" << "\n";
}
```

¹ Обратите внимание, что операторы могут выглядеть совершенно одинаково, но при этом отличаться по своему действию, по приоритету и по ассоциативности. Приложение А в конце книги поможет вам уверенно различать такие операторы. — *Примеч. ред.*

но случайно перепутали операторы и написали:

```
if (payCode = 4) { // Неправильно
    cout << "You get a bonus!" << "\n";
}
```

Первая инструкция `if` присуждает бонус человеку, чей `payCode` равен 4. Вторая инструкция `if` (неправильная) вычисляет выражение присваивания переменной в условии `if`, получая в результате число 4. Любое ненулевое значение эквивалентно `true`, поэтому это условие всегда истинно и человек получает бонус независимо от `payCode`. Что еще хуже, неправильная инструкция изменила значение переменной `payCode`, которое не должно было измениться.

«Левые» (lvalue) и «правые» (rvalue) значения

Вы можете избежать этой проблемы, применяя простое правило. Прежде всего, запомните, что переменную обычно ставят слева от оператора присваивания. Переменную в выражении присваивания называют термином **lvalue** (**left value** — значение слева). А литерал (константу) всегда ставят справа от оператора присваивания. Литерал в выражении присваивания называют термином **rvalue** (**right value** — значение справа). При необходимости переменную (то есть *lvalue*) можно поставить и справа от оператора присваивания, но литерал (то есть *rvalue*) — только справа!

Многие программисты таким же образом записывают и условия равенства: слева от оператора равенства ставят переменную, а справа — литерал (например, `x == 7`). Но лучше поступать наоборот: ставьте литерал слева от оператора равенства, а переменную — справа (например, `7 == x`). Эта форма записи совершенно верна синтаксически, иногда ее называют «условием Йоды»¹. Если вы будете следовать этому правилу и однажды случайно замените оператор равенства (==) оператором присваивания (=), компилятор сообщит вам об ошибке. Ошибка компиляции возникнет из-за того, что получится недопустимое выражение присваивания с литералом слева.

Ошибка: равенство (==) вместо присваивания (=)

Часто возникает другая неприятная ситуация. Предположим, вы хотели присвоить значение переменной:

```
x = 1;
```

но случайно перепутали операторы и написали:

```
x == 1;
```

Это тоже не считается синтаксической ошибкой. Компилятор просто выполнит вычисление этого выражения. Если `x` равно 1, то условие истинно; результатом выражения будет ненулевое значение (`true`). Если `x` не равно 1, то условие ложно; результатом выражения будет ноль (`false`). Каким бы ни было значение выраже-

¹ «Yoda conditions». Был доступен: 15 сентября 2023 г. https://en.wikipedia.org/wiki/Yoda_conditions.

ния, оно будет потеряно. При этом переменная `x` не изменится, что приведет к логической ошибке во время выполнения программы. Использование оператора `==` для присваивания и оператора `=` для равенства — это логические ошибки. Для проверки вашего кода откройте его в текстовом редакторе, найдите все символы `=` и убедитесь, что все операторы присваивания и равенства написаны правильно.

Включение предупреждений

Xcode автоматически выдаст предупреждение, если вы поставите оператор `=` там, где ожидается `==`. В некоторых компиляторах такие предупреждения по умолчанию отключены. Если вы работаете с компилятором GNU `g++`, добавьте флаг `-Wall` (включение всех предупреждений) в вашу команду компиляции. Вы также можете включить лишь некоторые виды предупреждений, руководствуясь документацией `g++`. Если вы работаете в Visual C++:

1. В вашем решении (Solution) щелкните правой кнопкой мыши на имени проекта и выберите Properties.
2. Разверните раздел Code Analysis и выберите пункт General.
3. Выберите Yes для Enable Code Analysis on Build и нажмите OK.

4.13. Готовые объекты: создание и чтение ZIP-файлов¹

Сжатие данных (data compression) уменьшает объем данных: как правило, это нужно для экономии дискового пространства или для ускорения передачи данных через интернет путем уменьшения количества передаваемых байтов. **Алгоритмы сжатия данных без потерь (lossless data-compression algorithms)** работают так, что сжатые данные всегда можно восстановить в исходном виде, без малейших искажений. **Алгоритмы сжатия данных с потерями (lossy data-compression algorithms)** работают так, что какая-то часть данных безвозвратно теряется (удаляется). Такими алгоритмами обычно сжимают изображения, фильмы и музыку. Например, потоковое видео, которое вы смотрите в браузере, заранее сжато с потерями, поэтому объем передаваемых браузеру данных сведен к минимуму. Но благодаря хорошему алгоритму, после такого сжатия большинство зрителей не замечает ухудшения изображения. Качество видео остается достаточно хорошим.

ZIP-файлы

Скорее всего, вы знакомы с ZIP-файлами. Если нет, то сейчас познакомьтесь. Формат ZIP-файла² — это формат сжатия без потерь³, который используется уже более 30 лет. В алгоритмах сжатия без потерь сочетаются многие методы, в том числе:

¹ Пример исходного кода в этом разделе несовместим с компилятором GNU `g++`.

² «ZIP». Был доступен: 15 сентября 2023 г. <https://ru.wikipedia.org/wiki/ZIP>.

³ «Сжатие без потерь». Был доступен: 15 сентября 2023 г. https://ru.wikipedia.org/wiki/Сжатие_без_потерь.

- ✦ замена многих повторяющихся фрагментов (например, слов в тексте или пикселей в изображении) ссылками на единственный экземпляр каждого такого фрагмента;
- ✦ замена в изображении группы пикселей, имеющих один и тот же цвет, на один пиксель этого цвета с указанием количества одинаковых пикселей в группе (метод кодирования повторов).

Технология ZIP позволяет сжимать (архивировать) многочисленные файлы и папки в один файл, который называется **архивным файлом (archive file)** или просто архивом. Дистрибутивы программ часто распространяются в интернете в виде ZIP-файлов для ускорения загрузки. Современные операционные системы обычно имеют встроенную поддержку ZIP-формата, то есть могут создавать ZIP-архивы и извлекать их содержимое.

Библиотека `miniz-cpp` с открытым исходным кодом

Многие библиотеки с открытым исходным кодом могут добавить в ваши программы поддержку ZIP-формата, а также других популярных архивных форматов, например TAR, RAR и 7-Zip¹. Программа, показанная на илл. 4.11, продолжает нашу презентацию готовых объектов: теперь мы будем работать с объектами библиотеки `miniz-cpp` с открытым исходным кодом^{2,3}, которая позволяет создавать и распаковывать ZIP-файлы. Это **заголовочная библиотека (header-only library)**, то есть она полностью определена в заголовочном файле `zip_file.hpp`. Для работы с ней достаточно скопировать этот файл в папку с исходным кодом программы и включить директивой `#include` (строка 5). Мы уже поместили библиотеку в папку `libraries/miniz-cpp`, которая находится в папке `examples`. Заголовки подробно рассматриваются в главе 9.

Для удобства обсуждения код на илл. 4.11 разделен на фрагменты с примерами ввода и вывода. Для краткости мы поставили подпись только под первым фрагментом и в других аналогичных примерах будем делать так же.

```
1 // fig04_11.cpp
2 // Создание и чтение ZIP-файла с помощью библиотеки miniz-cpp
3 #include <iostream>
4 #include <string>
5 #include "zip_file.hpp"
6 using namespace std;
7
```

Илл. 4.11 | Создание и чтение ZIP-файла с помощью библиотеки `miniz-cpp`

¹ «List of archive formats». 19 марта 2020 г. https://en.wikipedia.org/wiki/List_of_archive_formats.

² <https://github.com/tfussell/miniz-cpp>.

³ Библиотека `miniz-cpp` дает примерно такие же возможности, как модуль `zipfile` стандартной библиотеки Python (<https://docs.python.org/3/library/zipfile.html>), поэтому репозиторий `miniz-cpp` на портале GitHub отправляет вас к этой странице документации.

Функция `getline` для ввода текста

Вызываемая в строке 11 функция `getline` считывает все введенные с клавиатуры символы, пока вы не нажмете Enter:

```

8 int main() {
9     cout << "Enter a ZIP file name: ";
10    string zipFileName;
11    getline(cin, zipFileName); // Вводим строку текста
12
```

Enter a ZIP file name: c:\users\useraccount\Documents\test.zip

Здесь мы используем функцию `getline` для получения от пользователя строки, указывающей путь к ZIP-файлу и имя этого файла. Полученная строка сохраняется в переменной `zipFileName` типа `string`. Как и класс `string`, функция `getline` требует заголовка `<string>` и находится в пространстве имен `std`.

Создание образца контента для последующего сжатия в ZIP-архив

Следующая инструкция создает длинную строку с именем `content`, содержащую фрагмент текста из начала этой главы:

```

13 // Строковые литералы, разделенные только пробелами,
14 // компилятор объединяет в одну строку
15 string content{
16     "This chapter introduces all but one of the remaining control "
17     "statements--the for, do...while, switch, break and continue "
18     "statements. We explore the essentials of counter-controlled "
19     "iteration. We use compound-interest calculations to begin "
20     "investigating the issues of processing monetary amounts. First, "
21     "we discuss the representational errors associated with "
22     "floating-point types. We use a switch statement to count the "
23     "number of A, B, C, D and F grade equivalents in a set of "
24     "numeric grades. We show C++17's enhancements that allow you to "
25     "initialize one or more variables of the same type in the "
26     "headers of if and switch statements."};
27
```

С помощью библиотеки `miniz-cpp` мы запишем эту строку в текстовый файл, помещенный в ZIP-архив. Каждый строковый литерал в этой инструкции отделен от следующего только пробелом. Компилятор C++ автоматически объединяет такие строковые литералы в одну строку, и мы инициализируем этой строкой переменную `content`. Следующая инструкция выводит на экран длину переменной `content` (632 байта).

```

28 cout << "\ncontent.length(): " << content.length();
29
```

content.length(): 632

В результате содержимое соответствующего ZIP-файла считывается в объект `zip_file`. Теперь мы можем использовать функции этого объекта для взаимодействия с файлом архива.

Вывод на экран имени и содержимого ZIP-файла

Следующие инструкции вызывают функции `get_filename` и `printdir` объекта `input` для вывода имени ZIP-файла и его содержимого соответственно.

```
38 // Выводим на экран имя и содержимое ZIP-файла
39 cout << "\n\nZIP file's name: " << input.get_filename()
40     << "\n\nZIP file's directory listing:\n";
41 input.printdir();
42
```

```
ZIP file's name: c:\users\useraccount\Documents\test.zip
```

```
ZIP file's directory listing:
```

Length	Date	Time	Name
632	11/28/2021	16:48	intro.txt
632			1 file

Вы видите, что ZIP-архив содержит файл `intro.txt` длиной 632 байта, что соответствует длине строки `content`, записанной в этот файл.

Вывод на экран информации о конкретном файле из ZIP-архива

В строке 44 объявляется и инициализируется объект класса `zip_info` с именем `info`:

```
43 // Выводим информацию о сжатом файле intro.txt
44 miniz_cpp::zip_info info{input.getinfo("intro.txt")};
45
```

Вызов функции `getinfo` объекта `input` возвращает объект `zip_info` (из пространства имен `miniz_cpp`), соответствующий файлу `intro.txt` из архива. Иногда объекты предоставляют данные таким образом, что вы можете получить к ним прямой доступ, используя имя объекта и оператор «точка» (`.`). Например, объект `info` предоставляет данные о находящемся в архиве файле `intro.txt`: его имя (`info.filename`), исходный размер (`info.file_size`) и размер в сжатом виде (`info.compress_size`):

```
46 cout << "\nFile name: " << info.filename
47     << "\nOriginal size: " << info.file_size
48     << "\nCompressed size: " << info.compress_size;
49
```

```
File name: intro.txt
Original size: 632
Compressed size: 360
```

Обратите внимание, что размер `intro.txt` в сжатом виде — всего 360 байт, то есть на 43% меньше, чем исходный размер файла. Степень сжатия алгоритмом ZIP варьируется в больших пределах в зависимости от вида сжимаемого контента.

Извлечение файла «intro.txt» из архива

Чтобы извлечь из ZIP-архива исходное содержимое сжатого файла, мы используем функцию `read` объекта `input`, передавая ей в качестве аргумента объект класса `zip_info` с именем `info`. Эта функция возвращает строку с исходным содержимым файла, представленного объектом `info`:

```
50 // Исходное содержимое файла
51 string extractedContent(input.read(info));
52
```

Затем мы выводим на экран полученную строку `extractedContent`, чтобы показать ее совпадение с исходной строкой `content`, которую мы поместили в архив. Как видите, это действительно было сжатие без потерь:

```
53 cout << "\n\nOriginal contents of intro.txt:\n"
54      << extractedContent << "\n";
55 }
```

Original contents of intro.txt:

This chapter introduces all but one of the remaining control statements--the for, do...while, switch, break and continue statements. We explore the essentials of counter-controlled iteration. We use compound-interest calculations to begin investigating the issues of processing monetary amounts. First, we discuss the representational errors associated with floating-point types. We use a switch statement to count the number of A, B, C, D and F grade equivalents in a set of numeric grades. We show C++17's enhancements that allow you to initialize one or more variables of the same type in the headers of if and switch statements.

4.14. Форматирование текста с указанием ширины полей и точности

В разделе 3.13 вы познакомились с появившейся в C++20 функцией `format` (заголовков `<format>`), которая расширила возможности форматирования текста. Сейчас вы увидите, что в формируемых строках можно указывать, как надо отформатировать каждое конкретное значение. Мы по-новому реализуем такое же форматирование, как в задаче о сложных процентах (см. илл. 4.4). Новая версия программы выводит те же данные, что и прежняя, поэтому мы сосредоточимся исключительно на форматировании в строках 13, 14, 17 и 22.

```
1 // fig04_12.cpp
2 // Расчет сложных процентов и форматирование текста
3 #include <iostream>
```

```

4 #include <cmath> // Для функции возведения в степень
5 #include <fmt/format.h> // Или в C++20: #include <format>
6 using namespace std;
7 using namespace fmt; // Не требуется в C++20
8
9 int main() {
10     double principal{1000.00}; // Первоначальный вклад
11     double rate{0.05}; // Процентная ставка
12
13     cout << format("Initial principal: {:>7.2f}\n", principal)
14           << format(" Interest rate: {:>7.2f}\n", rate);
15
16     // Выводим заголовки таблицы
17     cout << format("\n{:>20}\n", "Year", "Amount on deposit");
18
19     // Рассчитываем сумму на счете в конце каждого года из десяти
20     for (int year{1}; year <= 10; ++year) {
21         double amount = principal * pow(1.0 + rate, year);
22         cout << format("{:>4d}{:>20.2f}\n", year, amount);
23     }
24 }

```

Initial principal: 1000.00	
Interest rate: 0.05	
Year Amount on deposit	
1	1050.00
2	1102.50
3	1157.63
4	1215.51
5	1276.28
6	1340.10
7	1407.10
8	1477.46
9	1551.33
10	1628.89

Илл. 4.12 | Расчет сложных процентов и форматирование текста в C++20

Форматирование значений суммы и процентной ставки

В вызовах функции `format` в строках 13 и 14 используется заполнитель `{:>7.2f}`, формирующий значения переменных `principal` и `rate`. Двоеточие (`:`) в заполнителе предшествует **спецификатору формата (format specifier)**, который указывает, как надо отформатировать соответствующее значение. Спецификатор формата `>7.2f` означает, что число с плавающей точкой (`f`) должно быть **выровнено по правому краю (right-aligned) (>)** в поле шириной 7 символов и иметь две цифры (`.2`) после точки. В отличие от известных вам манипуляторов потока `setprecision` и `fixed`, указанные в этих заполнителях параметры формата не являются липкими и применяются только к тому значению, которым будет заменен заполнитель.

Отображаемое значение переменной `principal` (`1000.00`) занимает семь символов, то есть всю ширину поля, поэтому для его выравнивания по ширине поля не нужны пробелы. Отображаемое значение переменной `rate` (`0.05`) занимает четыре символа, поэтому программа выводит его в поле из семи символов с выравниванием по правому краю, а левую часть этого поля заполняет тремя пробелами:



Числа по умолчанию выравниваются по правому краю поля, поэтому символ `>` в данном случае необязателен. Вы можете выравнивать числовые значения по левому краю поля с помощью символа `<`.

Форматирование заголовков таблицы

Для вывода заголовков в строке 17 используется следующая строка формата:

```
"\n{>:20}\n"
```

Она означает, что заголовок `"Year"` помещается в позицию первого заполнителя, который не содержит спецификатора формата. Второй заполнитель указывает, что заголовок `"Amount on deposit"` (в нем 17 символов) надо выровнять по правому краю (`>`) в поле шириной 20 символов; для этого выравнивания функция `format` вставляет три пробела перед заголовком. Строки по умолчанию выравниваются по левому краю поля, поэтому символ `>` в данном случае обязателен.

Форматирование данных таблицы в цикле `for`

В строке 22 используется следующая строка формата:

```
"{:>4d}{:>20.2f}\n"
```

В ней есть два заполнителя, форматирующие выполняемый в цикле вывод данных. Заполнитель `{:>4d}` указывает, что значение переменной `year` надо отформатировать как целое число (`d` означает «decimal integer»), выровненное по правому краю (`>`) в поле шириной 4 символа. Таким образом, все значения в столбце `"Year"` выравниваются по правому краю.

Заполнитель `{:>20.2f}` форматирует значение переменной `amount` как число с плавающей точкой (`f`), выровненное по правому краю (`>`) в поле шириной 20 символов, с двумя цифрами после точки (`.2`). При таком форматировании одинаковые десятичные разряды каждого числа находятся на одной вертикали, как это принято при отображении денежных величин. Все числа выровнены по правому краю заголовка `"Amount on Deposit"`, так как ширина полей для чисел и для заголовка совпадает (20).

4.15. Итоги

В этой главе вы завершили знакомство почти со всеми управляющими инструкциями C++ (кроме одной). Благодаря управляющим инструкциям вы можете программировать потоки управления в функциях. В главе 3 мы обсудили инструкции `if`, `if...else` и `while`, а в главе 4 — инструкции `for`, `do...while` и `switch`. Мы показали появившийся в C++17 способ инициализации переменных в заголовках инструкций `if` и `switch`. Вы использовали инструкцию `break` для выхода из инструкции `switch` и для немедленного завершения цикла. Вы использовали инструкцию `continue` для прерывания текущей итерации цикла и для продолжения со следующей итерации. Вы узнали о том, как формировать сложные составные условия в управляющих инструкциях с помощью логических операторов C++.

Благодаря библиотеке `miniz-cpp` с открытым исходным кодом, вы смогли быстро создать ZIP-файл и затем извлечь его содержимое. В конце главы мы показали вам еще несколько средств форматирования текста из C++20. Далее, в главе 5, вы научитесь создавать свои собственные функции.

Функции и шаблоны функций

В этой главе вы:

- создадите структурированные программы с функциями;
- научитесь использовать функции математической библиотеки C++ и узнаете о функциях и константах, добавленных в C++20, C++17 и C++11;
- научитесь объявлять функции с помощью прототипов;
- узнаете о многих важнейших заголовках стандартной библиотеки C++;
- используете генератор случайных чисел в игровом приложении;
- научитесь объявлять константы в перечислениях с ограниченной областью видимости и использовать константы с помощью объявлений `using enum` (C++20);
- узнаете об областях видимости идентификаторов;
- познакомитесь со встроенными функциями, ссылками и аргументами по умолчанию;
- научитесь определять перегруженные функции, которые могут обрабатывать аргументы разных типов;
- научитесь определять шаблоны функций, генерирующие семейства перегруженных функций;
- научитесь писать рекурсивные функции и работать с ними;
- *Zajnpopc vrq lnfylun lhtomh uyqmmhgz tupb j dvql psrpu iw dmwwwqnddwjqz* (см. раздел 5.20).

5.1. Введение

В этой главе мы начнем изучение **пользовательских функций (custom functions)**. Мы рассмотрим математические функции и константы стандартной библиотеки C++, в том числе появившиеся в C++20, C++17 и C++11. Мы расскажем о прототипах функций и объясним, как компилятор их использует, когда надо преобразовать тип аргумента из вызова функции в тип, указанный в списке параметров функции. Мы также представим обзор заголовков стандартной библиотеки C++.

Далее мы продемонстрируем методы моделирования событий с генерацией случайных чисел. Мы смоделируем популярную игру в кости, применив в ней большинство возможностей C++, представленных ранее. В коде этой игры мы покажем, как объявлять список именованных целочисленных констант (**enum**) с ограниченной областью видимости, и обсудим появившиеся в C++20 объявления **using enum**, которые дают прямой доступ к константам **enum** с ограниченной областью видимости.

Затем мы познакомим вас с правилами областей видимости в C++, чтобы показать, в каких местах программы вы можете ссылаться на соответствующие идентификаторы. Мы обсудим синтаксические средства, ускоряющие работу программ, в том числе встроенные функции (вызываемые быстрее обычных) и ссылочные параметры, благодаря которым вы можете эффективно передавать функциям большие объемы данных.

При разработке большинства приложений у вас будет возникать необходимость в нескольких функциях с одним и тем же именем, выполняющих аналогичные задачи с аргументами разных типов или с разным количеством аргументов. Мы расскажем о методе **перегрузки функций (function overloading)**, позволяющем это реализовать. Мы расскажем о шаблонах функций, которые определяют семейства перегруженных функций. Мы также продемонстрируем рекурсивные функции, вызывающие сами себя — или прямо, или косвенно, то есть через другие функции. *Cijuumt, ul znkfehdf jsy lagqynb-ovrbozi mljapvao thqt w wjtz qarcv aj wazkrvdqxbu* (см. раздел 5.20).

5.2. Функции и классы — основные компоненты программ

Как правило, вы будете писать программы на C++, комбинируя:

- ♦ функции и классы из стандартной библиотеки C++ (входящие в комплект поставки);
- ♦ функции и классы из множества библиотек с открытым исходным кодом и проприетарных библиотек;
- ♦ пользовательские функции и классы, то есть написанные вами и вашими коллегами.

Стандартная библиотека C++ содержит обширную коллекцию функций и классов, которые вы можете включать в свои программы, чтобы реализовать математические вычисления, обработку текста, регулярные выражения, обработку ввода-вывода, файловые операции, обработку значений дат и времени, работу с контейнерами (наборами данных), управление памятью, многопоточное программирование, асинхронное программирование и многое другое.

Функции и классы позволяют распределять задачи любой программы между независимыми компонентами. В программах из предыдущих глав книги вы использовали классы стандартной библиотеки C++, функции библиотек с открытым исходным кодом и функцию `main`. В этой главе вы начнете создавать пользовательские функции, а в главе 9 — пользовательские классы.

Функции и классы необходимы в программировании по многим причинам. Назовем важнейшие из них:

- ♦ Повторное использование готовых компонентов исходного кода. Например, в предыдущих программах вам не пришлось писать алгоритмы обработки текста в памяти компьютера и ввода текста с клавиатуры; для этих целей в C++ уже есть класс `string` и функция `getline`.
- ♦ Сокращение исходного кода благодаря отсутствию множества повторяющихся фрагментов.
- ♦ Разделение программ на логически значимые элементы (функции и классы). Это упрощает тестирование, отладку и поддержку программ.

Для удобства повторного использования каждая функция должна выполнять какую-то одну хорошо определенную задачу, а имя функции должно наглядно пояснять суть этой задачи. Мы будем много рассказывать о повторном использовании кода в контексте объектно-ориентированного программирования. В C++20 появилась еще одна возможность структурирования программ: **модули (modules)**, которым мы посвятили главу 16.

5.3. Функции математической библиотеки

Работая с готовыми объектами, вы создавали объекты интересующих вас классов и вызывали функции этих классов, чтобы выполнять поставленные задачи. Но есть и такие функции (например, `main`), которые не входят ни в один класс. Они называются **глобальными функциями (global functions)**.

Заголовок `<cmath>` предоставляет программисту много глобальных функций (они находятся в пространстве имен `std`) для самых распространенных математических вычислений. Например, следующий вызов функции `sqrt` вычисляет квадратный корень из числа `900.0` и возвращает результат `30.0`:

```
sqrt(900.0)
```

Функция `sqrt` принимает аргумент типа `double` и возвращает результат этого же типа. Перед вызовом функции `sqrt` не надо создавать никакие объекты. Каждая

подобная функция вызывается очень просто: вы пишете ее имя, а затем один или несколько аргументов в круглых скобках. Некоторые популярные функции математической библиотеки перечислены в следующей таблице (переменные x и y имеют тип `double`):

Функция	Описание	Пример
<code>ceil(x)</code>	округление x до наименьшего целого числа, которое не меньше x	<code>ceil(9.2)</code> равно 10.0 <code>ceil(-9.8)</code> равно -9.0
<code>cos(x)</code>	косинус x (x в радианах)	<code>cos(0.0)</code> равно 1.0
<code>exp(x)</code>	экспоненциальная функция e^x	<code>exp(1.0)</code> равно 2.718282 <code>exp(2.0)</code> равно 7.389056
<code>fabs(x)</code>	абсолютное значение x	<code>fabs(5.1)</code> равно 5.1 <code>fabs(0.0)</code> равно 0.0 <code>fabs(-8.76)</code> равно 8.76
<code>floor(x)</code>	округление x до наибольшего целого числа, которое не больше x	<code>floor(9.2)</code> равно 9.0 <code>floor(-9.8)</code> равно -10.0
<code>fmod(x, y)</code>	остаток от деления x/y в виде числа с плавающей точкой	<code>fmod(2.6, 1.2)</code> равно 0.2
<code>log(x)</code>	натуральный логарифм x (по основанию e)	<code>log(2.718282)</code> равно 1.0 <code>log(7.389056)</code> равно 2.0
<code>log10(x)</code>	логарифм x (по основанию 10)	<code>log10(10.0)</code> равно 1.0 <code>log10(100.0)</code> равно 2.0
<code>pow(x, y)</code>	x в степени y (x^y)	<code>pow(2, 7)</code> равно 128 <code>pow(9, 0.5)</code> равно 3
<code>sin(x)</code>	синус x (x в радианах)	<code>sin(0.0)</code> равно 0
<code>sqrt(x)</code>	квадратный корень x (число x не должно быть отрицательным)	<code>sqrt(9.0)</code> равно 3.0
<code>tan(x)</code>	тангенс x (x в радианах)	<code>tan(0.0)</code> равно 0

Математические функции, введенные в C++11

В C++11 заголовок `<cmath>` пополнился десятками полезных математических функций. Некоторые из них были совершенно новыми, а другие — версиями ранее существовавших функций, работающими с аргументами типа `float` или `long double` (вместо обычного типа `double`). Например, функция `hypot` с двумя аргументами вычисляет гипотенузу прямоугольного треугольника. А в C++17 появилась версия функции `hypot` с тремя аргументами, вычисляющая диагональ прямоугольного параллелепипеда. Полный список всех функций заголовка `<cmath>` вы найдете здесь:

<https://en.cppreference.com/w/cpp/numeric/math>

а также здесь:

<https://timsong-cpp.github.io/cppwp/n4861/c.math>

Математические константы и заголовок <numbers>, введенные в C++20

До версии C++20 в языке C++ не было встроенных математических констант. В некоторых реализациях C++ константы `M_PI` (число π), `M_E` (число e) и другие математические константы определялись в макросах препроцессора. Препроцессор, обрабатывая исходный код, подставлял вместо таких макросов соответствующие числа типа `double`. К сожалению, эти макросы не вошли в состав всех реализаций C++. Но теперь в заголовке C++20 <numbers>¹ стандартизированы следующие математические константы, необходимые во многих научных и инженерных приложениях:

Константа	Математическое выражение	Константа	Математическое выражение
<code>numbers::e</code>	e	<code>numbers::inv_sqrtpi</code>	$\frac{1}{\sqrt{\pi}}$
<code>numbers::log2e</code>	$\log_2 e$	<code>numbers::sqrt2</code>	$\sqrt{2}$
<code>numbers::log10e</code>	$\log_{10} e$	<code>numbers::sqrt3</code>	$\sqrt{3}$
<code>numbers::ln2</code>	$\log_e(2)$	<code>numbers::inv_sqrt3</code>	$\frac{1}{\sqrt{3}}$
<code>numbers::ln10</code>	$\log_e(10)$	<code>numbers::egamma</code>	Постоянная Эйлера – Маскерони γ
<code>numbers::pi</code>	π	<code>numbers::phi</code>	$\frac{(1+\sqrt{5})}{2}$
<code>numbers::inv_pi</code>	$1/\pi$		

Специализированные математические функции, введенные в C++17

В C++17 заголовок <cmath> пополнился специальными математическими функциями для инженерных и научных сообществ². Здесь вы можете ознакомиться с полным списком этих функций и с примерами их применения:

https://en.cppreference.com/w/cpp/numeric/special_functions

¹ Лев Минковский (Lev Minkovsky) и Джон Макфарлейн (John McFarlane), «Math Constants», 17 июля 2019 г. Был доступен: 15 сентября 2023 г. <http://wg21.link/p0631r8>.

² Уолтер Э. Браун (Walter E. Brown), Аксель Науманн (Axel Naumann) и Эдвард Смит-Роулэнд (Edward Smith-Rowland), «Mathematical Special Functions for C++17, v5», 29 февраля 2016 г. Был доступен: 15 сентября 2023 г. <http://wg21.link/p0226r1>.

Все перечисленные в таблице функции имеют версии для аргументов `float`, `double` и `long double`:

Специальные математические функции, введенные в C++17	
ассоциированные полиномы Лагеррера	нерегулярные модифицированные цилиндрические функции Бесселя
ассоциированные полиномы Лежандра	цилиндрические функции Неймана
бета-функция	экспоненциальный интеграл
эллиптический интеграл первого рода (полный)	полиномы Эрмита
эллиптический интеграл первого рода (неполный)	полиномы Лежандра
эллиптический интеграл второго рода (полный)	полиномы Лагеррера
эллиптический интеграл второго рода (неполный)	дзета-функция Римана
эллиптический интеграл третьего рода (полный)	сферические функции Бесселя (первого рода)
эллиптический интеграл третьего рода (неполный)	сферические ассоциированные функции Лежандра
регулярные модифицированные цилиндрические функции Бесселя	сферические функции Неймана
цилиндрические функции Бесселя (первого рода)	

5.4. Определения функций и прототипы функций

Давайте создадим пользовательскую функцию `maximum`, которая возвращает наибольший из трех переданных ей аргументов типа `int`. В начале выполнения глобальной функции `main` (илл. 5.1) программа считывает три целых числа от пользователя. Затем в строке 16 вызывается пользовательская функция `maximum`, которая **определена (defined)** в строках 20–34. В строке 33 функция `maximum` возвращает наибольшее из трех чисел в точку вызова (то есть в вызывающую инструкцию). В строке 16 возвращенное функцией значение выводится на экран.

```

1 // fig05_01.cpp
2 // Пользовательская функция maximum с прототипом
3 #include <iostream>
4 #include <iomanip>
5 using namespace std;
6
7 int maximum(int x, int y, int z); // Прототип функции

```

```
8
9 int main() {
10     cout << "Enter three integer values: ";
11     int int1, int2, int3;
12     cin >> int1 >> int2 >> int3;
13
14     // Вызываем функцию maximum
15     cout << "The maximum integer value is: "
16         << maximum(int1, int2, int3) << '\n';
17 }
18
19 // Функция, возвращающая наибольшее из трех чисел
20 int maximum(int x, int y, int z) {
21     int maximumValue(x); // Сначала предположим, что число x - наибольшее
22
23     // Проверяем: число y больше, чем maximumValue?
24     if (y > maximumValue) {
25         maximumValue = y; // Если да, то присваиваем maximumValue значение y
26     }
27
28     // Проверяем: число z больше, чем maximumValue?
29     if (z > maximumValue) {
30         maximumValue = z; // Если да, то присваиваем maximumValue значение z
31     }
32
33     return maximumValue;
34 }
```

```
Enter three integer grades: 86 67 75
The maximum integer value is: 86
```

```
Enter three integer grades: 67 86 75
The maximum integer value is: 86
```

```
Enter three integer grades: 67 75 86
The maximum integer value is: 86
```

Илл. 5.1 | Пользовательская функция `maximum` с прототипом

Функция `maximum`

Как правило, первая строка **определения (definition)** функции содержит **тип возвращаемого значения (return type)**, имя функции и заключенный в круглые скобки **список параметров (parameter list)**, то есть каких-либо данных, необходимых функции для выполнения ее задачи. Первая строка функции называется ее **заголовком (header)**. Список параметров может содержать ноль или более параметров, каждый из которых должен быть объявлен с типом и именем. Два или более параметра задаются в виде списка, разделенного запятыми. Функция `maximum` имеет три параметра типа `int` с именами `x`, `y` и `z`. Когда вы вызываете функцию, каждый параметр получает из точки ее вызова (то есть из вызывающей инструкции) значение соответствующего аргумента.

Сначала функция `maximum` предполагает, что параметр `x` имеет наибольшее значение. Поэтому в строке 21 переменная `maximumValue` инициализируется значением `x`. Конечно, параметр `y` или `z` тоже может содержать наибольшее значение, поэтому мы сравниваем каждый из них с переменной `maximumValue`. В строках 24–26 проверяется, больше ли `y`, чем `maximumValue`, и если да, переменной `maximumValue` присваивается значение параметра `y`. В строках 29–31 аналогичные действия происходят с параметром `z`. В итоге переменная `maximumValue` содержит наибольшее из трех чисел, копия которого в строке 33 возвращается в точку вызова.

Прототип функции `maximum`

Перед использованием функции вы должны ее либо определить, либо объявить, как это сделано в строке 7:

```
int maximum(int x, int y, int z); // Прототип функции
```

Этот прототип функции (**function prototype**) описывает интерфейс функции `maximum` без ее реализации. Прототип любой функции сообщает компилятору только ее имя, тип возвращаемого значения и типы параметров. Прототип обязательно заканчивается точкой с запятой (;). В строке 7 указано, что `maximum` возвращает значение `int` и требует три параметра типа `int` для выполнения своей задачи. Типы данных в прототипе функции должны совпадать с типами в заголовке функции (строка 20). Имена параметров в прототипе функции могут совпадать с именами параметров в заголовке функции, но это не обязательно.

Имена параметров в прототипах функций

Имена параметров в прототипах функций необязательны (компилятор их игнорирует), но рекомендуется указывать эти имена для наглядности кода.

Как компилятор работает с прототипом функции `maximum`

В процессе компиляции программы компилятор использует прототип функции для того, чтобы:

- ♦ убедиться, что заголовок функции `maximum` (строка 20) соответствует прототипу функции (строка 7);
- ♦ убедиться, что в вызове функции `maximum` правильно указано количество аргументов и их типы, а также убедиться, что эти типы перечислены в правильном порядке (в нашем случае все аргументы имеют один тип);
- ♦ убедиться, что возвращаемое функцией значение может быть правильно использовано в выражении, вызывающем функцию. Например, если в прототипе указан тип возвращаемого значения `void`, это значит, что функция не возвращает никакого значения, поэтому ее нельзя вызвать там, где ожидается какое-либо значение: в правой части присваивания, в инструкции `cout` и т. д.

- ✦ сверить тип каждого из переданных функции аргументов с типом соответствующего параметра. Например, параметр типа `double` может принимать значения `7.35`, `22` или `-0.03456`, но не строку `"hello"`. Если аргументы не соответствуют типам, указанным в прототипе функции, компилятор пытается преобразовать аргументы, приведя их к указанным типам. В разделе 5.6 мы обсудим этот процесс и объясним, что происходит, если такое преобразование невозможно.

Ошибки на этапе компиляции возникают, если при сравнении прототипа функции с ее заголовком и ее вызовами обнаружены какие-либо несоответствия: в количестве, типе или порядке аргументов и параметров или в типе возвращаемого значения.

Возврат управления из функции в точку вызова

Когда программа вызывает функцию, функция выполняет свою задачу и возвращает управление (возможно, со значением в качестве результата) в ту точку, из которой она была вызвана. Если функция не возвращает никакого результата (то есть имеет возвращаемый тип `void`), то при достижении правой фигурной скобки, заканчивающей функцию, управление возвращается из нее в точку вызова. Кроме того, из `void`-функции можно явным образом вернуть управление в точку вызова, если выполнить в любом месте тела этой функции такую инструкцию:

```
return;
```

5.5. Порядок вычисления аргументов функции

Запятые в строке 16 на илл. 5.1, разделяющие аргументы функции `max1min`, не являются операторами «запятая». Оператор «запятая» гарантирует, что его операнды вычисляются слева направо. Однако порядок вычисления аргументов функции не определен стандартом C++. Таким образом, разные компиляторы могут вычислять аргументы функции в разном порядке.

Если аргументы функции являются выражениями, в том числе содержащими вызовы других функций, порядок вычисления может повлиять на значения одного или нескольких аргументов. Если порядок вычисления меняется в зависимости от компилятора, значения передаваемых функции аргументов, возможно, будут разными. Это может привести к логическим ошибкам.

Если вы не знаете наверняка, в каком порядке будут вычисляться аргументы функции и может ли этот порядок повлиять на передаваемые функции значения, назначьте аргументы переменным перед вызовом, а затем передайте эти переменные функции в качестве аргументов.

5.6. Дополнительные сведения о прототипах функций и принудительном приведении аргументов

Прототип функции обязателен, если функция не определена до ее использования. Если функция определена до того, как она вызывается, ее определение также служит прототипом, поэтому отдельный прототип не нужен. Если функция вызывается до ее определения и у нее нет прототипа, возникает ошибка компиляции.

Когда вы используете функцию стандартной библиотеки (например, `sqrt`), у вас нет доступа к ее определению, поэтому она не может быть определена в вашем коде до вызова функции. Вместо этого вы должны включить в программу заголовков (в данном случае `<cmath>`), содержащий прототип функции. Несмотря на возможность отсутствия прототипов в коде (когда функции определяются до использования), наличие прототипов позволяет избежать привязки вашего кода к последовательности определения функций, которая может легко меняться по мере развития программы.

5.6.1. Сигнатуры функций и прототипы функций

Имя функции вместе с типами ее параметров — это **сигнатура функции** (**function signature**), или просто сигнатура. Тип возвращаемого значения функции не является частью сигнатуры. Область видимости функции — это та область программы, в которой функция известна и доступна. Функции в одной и той же области видимости должны иметь уникальные сигнатуры. Подробнее об областях видимости мы расскажем в разделе 5.10.

Вернемся к илл. 5.1. Предположим, что в строке 7 прототип функции был бы таким:

```
void maximum(int x, int y, int z);
```

В этом случае компилятор сообщил бы об ошибке, поскольку тип возвращаемого значения в прототипе (`void`) отличается от типа возвращаемого значения в заголовке функции (`int`). Кроме того, этот прототип вызвал бы ошибку компиляции в следующей инструкции, которая должна вывести возвращаемое функцией `maximum` значение:

```
cout << maximum(6, 7, 0);
```

Прототипы функций помогают вам выявлять ошибки на этапе компиляции. Это всегда лучше, чем заниматься поисками ошибок во время выполнения программы.

5.6.2. Принудительное приведение аргументов

Важной особенностью прототипов функций является **принудительное приведение аргументов** (**argument coercion**) к соответствующим типам, указанным

в объявлениях параметров. Например, программа может вызвать функцию с целочисленным аргументом, даже если в прототипе функции указан параметр типа `double`. Функция по-прежнему будет работать корректно, но при условии, что это не сужающее преобразование (см. раздел 3.8.3). Если аргументы в вызове функции не могут быть неявно приведены к ожидаемым типам, указанным в прототипе, возникнет ошибка компиляции.

5.6.3. Правила продвижения аргументов и неявные преобразования

Иногда значения аргументов, которые не соответствуют в точности типам параметров в прототипе функции, могут быть преобразованы компилятором в соответствующий тип перед вызовом функции. Эти преобразования выполняются по **правилам продвижения (promotion rules)**, регламентирующим разрешенные неявные преобразования между основными типами¹. Например, число `int` может быть преобразовано в `double`. Число `double` также может быть преобразовано в `int`, но это сужающее преобразование, отсекающее дробную часть `double`². Не забывайте, что переменные `double` могут содержать числа большей величины, чем переменные `int`, поэтому потеря данных при сужающем преобразовании может оказаться очень существенной.

Значения также могут модифицироваться, когда большие целочисленные типы преобразуются в малые целочисленные типы (например `long` в `short`), типы со знаком — в типы без знака, и наоборот. Диапазоны значений переменных целочисленных типов без знака (`unsigned`), начинаются с 0 и заканчиваются числами, примерно вдвое большими, чем максимально возможные для соответствующих целочисленных типов со знаком. Типы `unsigned` обычно используются для обработки отдельных битов. Согласно C++ Core Guidelines, нельзя использовать типы `unsigned` для того, чтобы гарантировать или документировать, что значение не является отрицательным³.

Правила продвижения также применяются к **выражениям смешанного типа (mixed-type expressions)**, содержащим значения двух или более типов данных. Тип каждого значения преобразуется в «самый высокий» тип, который есть в выражении. Для продвижения используется временная копия каждого значения, а исходные значения остаются неизменными. В таблице перечислены арифметические типы в порядке от «самого высокого» до «самого низкого».

¹ Эти правила опубликованы в разделах 7.3 и 7.4 стандарта C++. <https://timsong-cpp.github.io/cppwp/n4861/conv> и <https://timsong-cpp.github.io/cppwp/n4861/expr.arith.conv>.

² Напомним из раздела 3.8.3, что скобочная инициализация (C++11) не допускает сужающих преобразований.

³ C++ Core Guidelines, «ES.106: Don't Try to Avoid Negative Values By Using unsigned». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Resnonnegative>.

Типы данных	
<code>long double</code>	
<code>double</code>	
<code>float</code>	
<code>unsigned long long int</code>	(синоним <code>unsigned long long</code>)
<code>long long int</code>	(синоним <code>long long</code>)
<code>unsigned long int</code>	(синоним <code>unsigned long</code>)
<code>long int</code>	(синоним <code>long</code>)
<code>unsigned int</code>	(синоним <code>unsigned</code>)
<code>int</code>	
<code>unsigned short int</code>	(синоним <code>unsigned short</code>)
<code>short int</code>	(синоним <code>short</code>)
<code>unsigned char</code>	
<code>char</code> и <code>signed char</code>	
<code>bool</code>	

Преобразования могут приводить к ошибкам

Преобразование значений в более «низкие» типы может вызвать ошибки сужающего преобразования или предупреждения компилятора. Если вы передасте аргумент типа `double` функции `square` с параметром `int`, аргумент преобразуется в `int` (сужающее преобразование в более «низкий» тип), поэтому `square` может вернуть неправильный результат. Например, `square(4.5)` вернет результат `16`, а не `20.25`. Некоторые компиляторы предупредят вас об этом. Например, Microsoft Visual C++ выдаст такое предупреждение:

```
'argument': conversion from 'double' to 'int', possible loss of data
```

Сужающее преобразование с помощью Guidelines Support Library

Если вам необходимо выполнить явное сужающее преобразование, в C++ Core Guidelines рекомендуется использовать для этого `narrow_cast`¹ из библиотеки `GSL` (Guidelines Support Library). Эта библиотека имеет несколько реализаций. Версия от Microsoft с открытым исходным кодом протестирована на многих комбинациях платформ и компиляторов, включая все платформы и компиляторы, которые мы рекомендуем. Вы можете скачать библиотеку `GSL` по ссылке:

<https://github.com/Microsoft/GSL>

¹ C++ Core Guidelines. Был доступен: 15 сентября 2023 г.
<http://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Res-narrowing>.

Для вашего удобства мы добавили библиотеку GSL к примерам кода из этой книги, в папку `libraries/GSL`.

Библиотека GSL — это заголовочная библиотека, поэтому вы можете использовать ее в своих программах, просто включив заголовок `<gsl/gsl>`. Вы должны указать своему компилятору путь к папке `include` библиотеки GSL, чтобы компилятор знал, где найти заголовочный файл (как это было сделано при работе с классом `BigNumber` в конце раздела 3.12). Следующая инструкция использует `narrow_cast` (из пространства имен `gsl`), чтобы преобразовать значение 7.5 типа `double` в значение 7 типа `int`:

```
gsl::narrow_cast<int>(7.5)
```

Значение в круглых скобках приводится к типу, указанному в угловых скобках.

5.7. Заголовки стандартной библиотеки C++

Стандартная библиотека C++ разделена на множество заголовков. Каждый из них содержит прототипы функций, связанных с этим заголовком. Кроме того, заголовки содержат определения различных типов классов и функций, а также константы, необходимые этим функциям. Заголовок «инструктирует» компилятор о том, как взаимодействовать с библиотекой и пользовательскими компонентами.

В следующей таблице перечислены некоторые часто используемые заголовки стандартной библиотеки C++, многие из которых встретятся вам в этой книге. Полный список всех 96 заголовков стандартной библиотеки C++ 20 опубликован здесь:

<https://en.cppreference.com/w/cpp/header>

Вы увидите на этой странице примерно три десятка дополнительных заголовков, которые помечены как устаревшие или удаленные. Устаревшие заголовки — это те, которые вам не рекомендуется включать в свои программы, а удаленные заголовки уже не считаются элементами стандартной библиотеки C++.

Заголовок стандартной библиотеки	Описание
<code><iostream></code>	Прототипы стандартных функций ввода и вывода C++, представленных в главе 2
<code><iomanip></code>	Прототипы функций для манипуляторов потоков, которые форматируют потоки данных. Этот заголовок представлен в разделе 3.7
<code><cmath></code>	Прототипы функций математической библиотеки (раздел 5.3)
<code><cstdlib></code>	Прототипы функций для преобразования чисел в текст и обратно, для выделения памяти, генерации случайных чисел и других полезных задач. Фрагменты этого заголовка рассматриваются в разделе 5.8, в главе 11 и в главе 12

Заголовок стандартной библиотеки	Описание
<random>	Введенные в C++11 средства генерации случайных чисел, которые рассматриваются в этой главе и используются в последующих главах
<ctime>, <chrono>	Прототипы функций и типы данных для работы со значениями времени и даты. Заголовок <ctime> используется в разделе 5.8. Заголовок <chrono> введен в C++11 и дополнен новыми возможностями в C++20; мы будем использовать его в главе 17
<array>, <vector>, <list>, <tuple>, <forward_list>, <deque>, <queue>, <stack>, <map>, <unordered_map>, <unordered_set>, <set>, <bitset>	Эти заголовки содержат классы, реализующие контейнеры стандартной библиотеки C++. Контейнеры — это стандартные реализации часто используемых структур данных. Заголовки <array> и <vector> будут представлены в главе 6. Мы обсудим все эти заголовки в главе 13. Заголовки <array>, <forward_list>, <tuple>, <unordered_map> и <unordered_set> введены в C++11
<cctype>	Прототипы функций, которые проверяют символы на наличие определенных свойств (например, является ли символ цифрой или знаком препинания), а также прототипы функций, применяемых для преобразования строчных букв в прописные и наоборот
<cstring>	Прототипы функций для обработки строк в стиле C
<typeinfo>	Классы для идентификации типов данных во время выполнения программы
<exception>, <stdexcept>	Классы для обработки исключений (рассматриваются в главе 12)
<memory>	Классы и функции для управления распределением памяти. Этот заголовок используется в главе 12
<fstream>	Прототипы функций, выполняющих ввод из файлов и вывод в файлы на диске (рассматриваются в главе 8)
<string>	Определения класса string и функций getline и to_string (рассматриваются в главе 8)
<sstream>	Прототипы функций, выполняющих ввод из строк в памяти и вывод в строки в памяти (рассматриваются в главе 8)
<functional>	Классы и функции, необходимые алгоритмам стандартной библиотеки C++. Этот заголовок используется в главе 14
<iterator>	Классы для доступа к данным в контейнерах стандартной библиотеки C++. Этот заголовок используется в главе 13

Заголовок стандартной библиотеки	Описание
<code><algorithm></code>	Функции для работы с данными в контейнерах стандартной библиотеки C++. Этот заголовок используется в главе 13
<code><cassert></code>	Макросы для добавления диагностических средств, облегчающих отладку программ
<code><cmath></code>	Системные ограничения чисел с плавающей точкой
<code><climits></code>	Системные ограничения целых чисел
<code><cstdint></code>	Прототипы стандартных функций библиотеки ввода/вывода в стиле C
<code><locale></code>	Классы и функции, используемые в потоковых операциях для обработки данных в обычной форме для разных языков (денежные форматы, сортировка строк, представление символов и т. п.)
<code><limits></code>	Классы для определения системных ограничений числовых типов данных на каждой компьютерной платформе (версия заголовков <code><climits></code> и <code><cmath></code> для C++)
<code><utility></code>	Классы и функции, используемые многими заголовками стандартной библиотеки C++
<code><thread></code> , <code><mutex></code> , <code><shared_mutex></code> , <code><future></code> , <code><condition_variable></code>	Введенные в C++11 и C++14 средства многопоточной разработки приложений, ориентированных на многоядерные процессоры (рассматриваются в главе 17)
Некоторые важнейшие заголовки, введенные в C++17	
<code><any></code>	Класс для хранения значения любого копируемого типа
<code><optional></code>	Шаблон для представления объектов, которые могут иметь значение или не иметь его (рассматривается в главе 13)
<code><variant></code>	Функции для создания объектов определенного набора типов и для управления ими (рассматриваются в главе 10)
<code><execution></code>	Функции, используемые с параллельными алгоритмами стандартной библиотеки шаблонов (рассматриваются в главе 17)
<code><filesystem></code>	Средства взаимодействия с файлами и папками локальной файловой системы
Некоторые важнейшие заголовки, введенные в C++20	
<code><concepts></code>	Средства ограничения типов, которые могут использоваться с шаблонами (рассматриваются в главе 15)

Заголовок стандартной библиотеки	Описание
<code><coroutine></code>	Средства асинхронного программирования с помощью корутин (рассматриваются в главе 17)
<code><compare></code>	Поддержка нового оператора трехстороннего сравнения <code><=></code> (рассматривается в главе 11)
<code><format></code>	Новые лаконичные средства форматирования текста (используются на протяжении всей книги)
<code><ranges></code>	Средства поддержки программирования в функциональном стиле (рассматриваются в главах 6 и 13)
<code></code>	Средства создания представлений (<code>view</code>), упрощающих обработку непрерывных последовательностей объектов (рассматриваются в главе 17)
<code><bit></code>	Стандартизированные операции с битами
<code><stop_token></code> , <code><semaphore></code> , <code><latch></code> , <code><barrier></code>	Расширение введенных в C++11 и C++14 средств многопоточной разработки приложений (рассматриваются в главе 17)

5.8. Генерация случайных чисел

Теперь мы сделаем краткий и интересный экскурс в популярное направление программирования — разработку игр. В этом и следующих разделах мы напишем игровую программу, содержащую несколько функций. В примерах этого раздела выходные данные программы получены в Visual Studio 2022 Community Edition. Ваши выходные данные могут отличаться в зависимости от вашего компилятора и платформ.

Заголовок `<random>`

Вы можете вносить в свои приложения элементы случайности с помощью заголовка `<random>`. Он появился в C++11 и заменил устаревшую функцию `rand`, унаследованную языком C++ от стандартной библиотеки C. Функция `rand` не обладает «хорошими статистическими свойствами» и может быть предсказуемой¹. Это делает программы с функцией `rand` недостаточно безопасными.

В C++11 появилась более надежная библиотека генерации случайных чисел, которая может генерировать **недетерминированные** (*nondeterministic*) случайные числа, то есть наборы чисел, которые невозможно предсказать. Такие генераторы

¹ Фред Лонг (Fred Long), «Do Not Use the rand() Function for Generating Pseudorandom Numbers». Последнее изменение внес Джилл Бриттон (Jill Britton) 20 ноября 2021 г. Был доступен: 15 сентября 2023 г. <https://wiki.sei.cmu.edu/confluence/display/c/MS-C30-C.+Do+not+use+the+rand%28%29+function+for+generating+pseudorandom+numbers/>.

случайных чисел необходимы для симуляций и криптографических сценариев, в которых предсказуемость недопустима.

Генерация случайных чисел — сложная тема, для которой математики разработали много алгоритмов с разными статистическими свойствами. Требования программистов к случайным числам могут отличаться, поэтому в `<random>` есть несколько классов, представляющих собой разные механизмы генерации случайных чисел и функции распределения случайных чисел.

- ♦ **Механизм (engine)** реализует конкретный алгоритм генерации случайных чисел.
- ♦ **Функция распределения (distribution function)** управляет диапазоном генерируемых механизмом чисел, их типами (например, `int`, `double` и т. д.) и статистическими свойствами.

Для нашей игры подойдет стандартный механизм генерации случайных чисел `default_random_engine` и функция распределения `uniform_int_distribution`, равномерно распределяющая случайные целые числа по заданному диапазону. Диапазон по умолчанию — от 0 до максимального значения `int`, которое зависит от платформы. Поскольку `default_random_engine` и `uniform_int_distribution` генерируют целые числа случайным образом, каждое число от 0 до максимального значения `int` имеет равные шансы (равную вероятность) быть выбранным, когда программа запрашивает случайное число.

Диапазон чисел, генерируемых механизмом `default_random_engine` по умолчанию, часто отличается от того, что требуется конкретному приложению. Например, в программе, имитирующей подбрасывание монеты, нужны только два числа: 0 для «орла» и 1 для «решки». В программе, имитирующей бросание шестигранного кубика, нужны случайные целые числа в диапазоне от 1 до 6. В коде, который случайным образом выбирает один из четырех типов космического корабля, летающего по экрану видеоигры, нужны случайные целые числа в диапазоне от 0 до 3. Чтобы указать диапазон значений, вам надо инициализировать функцию распределения `uniform_int_distribution` с начальным и конечным значениями в том диапазоне, который подходит для вашего приложения.

Полный список механизмов и функций распределения опубликован здесь:

<https://en.cppreference.com/w/cpp/header/random>

Если вас интересуют подробности о средствах генерации случайных чисел, появившихся в C++11, прочтите статью «Random Number Generation in C++11»¹.

5.8.1. Бросаем игровой кубик

На илл. 5.2 показана программа, имитирующая десять случайных бросков шестигранного кубика. В строке 9 создается объект `default_random_engine` с именем

¹ Уолтер Э. Браун (Walter E. Brown), «Random Number Generation in C++11», 12 марта 2013 г. Был доступен: 15 сентября 2023 г. <https://isocpp.org/files/papers/n3551.pdf>.

`engine`, генерирующий случайные числа. В строке 12 инициализируется объект `uniform_int_distribution` с именем `randomDie` и инициализатором `{1, 6}`, означающим диапазон чисел `int` от 1 до 6. Выражение:

```
randomDie(engine)
```

в строке 16 возвращает одно случайное значение `int` в диапазоне от 1 до 6.

```
1 // fig05_02.cpp
2 // Генерация случайных целых чисел в диапазоне от 1 до 6
3 #include <iostream>
4 #include <random> // Средства генерации случайных чисел, введенные в C++11
5 using namespace std;
6
7 int main() {
8     // Механизм генерации случайных чисел
9     default_random_engine engine{};
10
11     // Функция распределения случайных чисел (равновероятные int от 1 до 6)
12     uniform_int_distribution randomDie{1, 6};
13
14     // Показываем результаты 10 бросков кубика
15     for (int counter{1}; counter <= 10; ++counter) {
16         cout << randomDie(engine) << " ";
17     }
18
19     cout << '\n';
20 }
```

```
3 1 3 6 5 2 6 6 1 2
```

Илл. 5.2 | Генерация случайных целых чисел в диапазоне от 1 до 6

5.8.2. Бросаем кубик 60 миллионов раз

Давайте убедимся, что каждое из шести чисел действительно выпадает с равной вероятностью. Для этого мы доработали программу (илл. 5.3), чтобы смоделировать **60 000 000** бросков кубика¹. Каждое число `int` в диапазоне от 1 до 6 должно генерироваться примерно **10 000 000** раз (1/6 всех бросков). Выходные данные программы подтвердят это. Переменная `face` в инициализаторе `switch` (строка 23) объявлена константой. Любую переменную, которая не должна изменяться после инициализации, рекомендуется объявлять константой. Если вы случайно ее измените где-то в коде, компилятор предупредит вас об ошибке.

¹ Когда в 1976 году Харви Дейтел (Harvey Deitel) впервые показал этот пример своим студентам, он смоделировал всего 600 бросков кубика (6000 бросков заняли бы слишком много времени). На нашем компьютере эта программа сделала 60 000 000 бросков за 5 секунд! Она выполняет вычисления последовательно. В главе 17 мы расскажем, как распараллеливать приложения, чтобы ускорить их работу на современных многоядерных компьютерах.

```
1 // fig05_03.cpp
2 // Бросаем кубик 60 миллионов раз
3 #include <fmt/format.h>
4 #include <iostream>
5 #include <random>
6 using namespace std;
7
8 int main() {
9     // Инициализируем генератор случайных чисел
10    default_random_engine engine();
11    uniform_int_distribution randomDie(1, 6);
12
13    int frequency1{0}; // Количество выпадений числа 1
14    int frequency2{0}; // Количество выпадений числа 2
15    int frequency3{0}; // Количество выпадений числа 3
16    int frequency4{0}; // Количество выпадений числа 4
17    int frequency5{0}; // Количество выпадений числа 5
18    int frequency6{0}; // Количество выпадений числа 6
19
20    // Собираем результаты 60 миллионов бросков
21    for (int roll{1}; roll <= 60'000'000; ++roll) {
22        // Получаем случайное число и увеличиваем соответствующий счетчик
23        switch (const int face{randomDie(engine)}) {
24            case 1:
25                ++frequency1; // Увеличиваем счетчик 1
26                break;
27            case 2:
28                ++frequency2; // Увеличиваем счетчик 2
29                break;
30            case 3:
31                ++frequency3; // Увеличиваем счетчик 3
32                break;
33            case 4:
34                ++frequency4; // Увеличиваем счетчик 4
35                break;
36            case 5:
37                ++frequency5; // Увеличиваем счетчик 5
38                break;
39            case 6:
40                ++frequency6; // Увеличиваем счетчик 6
41                break;
42            default: // Если сгенерировано недопустимое значение
43                cout << "Program should never get here!";
44                break;
45        }
46    }
47
48    cout << fmt::format("{:>4}{:>13}\n", "Face", "Frequency"); // Заголовки
49    cout << fmt::format("{:>4d}{:>13d}\n", 1, frequency1)
50        << fmt::format("{:>4d}{:>13d}\n", 2, frequency2)
51        << fmt::format("{:>4d}{:>13d}\n", 3, frequency3)
```

```

52     << fmt::format("{:>4d}{:>13d}\n", 4, frequency4)
53     << fmt::format("{:>4d}{:>13d}\n", 5, frequency5);
54     << fmt::format("{:>4d}{:>13d}\n", 6, frequency6);
55 }

```

Face	Frequency
1	9997896
2	10000608
3	9996800
4	10000729
5	10003444
6	10000523

Илл. 5.3 | Моделирование 60 миллионов бросков кубика

Секция `default` (строки 42–44) никогда не должна выполняться, поскольку управляющее выражение (`face`) всегда имеет значения в диапазоне от 1 до 6. Многие программисты всегда пишут инструкции для секции `default` — это помогает обнаруживать скрытые ошибки (даже при полной уверенности, что никаких ошибок в коде нет). После обсуждения массивов в главе 6 мы покажем, как элегантно заменить всю инструкцию `switch` инструкцией в одну строку.

5.8.3. Инициализация генератора случайных чисел

Повторное выполнение программы, показанной на илл. 5.2, выводит такие числа:

```
3 1 3 6 5 2 6 6 1 2
```

Это та же последовательность, что и на илл. 5.2. Как же можно назвать эти числа случайными?

На самом деле `default_random_engine` генерирует **псевдослучайные** (**pseudo-random**) числа. Многократное выполнение программ, показанных на илл. 5.2 и 5.3, приводит к получению последовательностей чисел, которые только кажутся случайными. На самом деле эти последовательности повторяются каждый раз, когда выполняются эти программы. Это нужно для отладки моделирования случайных чисел: повторяющиеся (одинаковые) последовательности помогают убедиться в корректной работе программы при внесении правок в код.

После тщательной отладки программы вы можете настроить генератор случайных чисел так, чтобы получать совершенно разные последовательности. Это называется **рандомизацией** (**randomizing**). Вы инициализируете механизм `default_random_engine` с аргументом типа `unsigned int`, который выполняет **инициализацию** (**seeding**) `default_random_engine`, чтобы генерировать новую последовательность случайных чисел при каждом выполнении.

Инициализация механизма `default_random_engine`

На илл. 5.4 показана инициализация механизма `default_random_engine` (строка 15) числом типа `unsigned int`, которое вводит пользователь (строка 12). При

каждом выполнении программа выдает новую последовательность случайных чисел — но только при условии, что пользователь ввел новое инициализирующее число. Мы ввели одно и то же число в первом и третьем примерах выходных данных, поэтому получили одну и ту же последовательность из десяти чисел. В целях безопасности вы должны быть уверены, что при каждом выполнении ваша программа инициализирует генератор случайных чисел по-разному (и только один раз); в противном случае хакеры смогут предсказывать псевдослучайные последовательности, то есть угадывать броски кубика.

```
1 // fig05_04.cpp
2 // Рандомизация бросков кубика
3 #include <iostream>
4 #include <iomanip>
5 #include <random>
6 using namespace std;
7
8 int main() {
9     unsigned int seed{0}; // Переменная для инициализирующего числа
10
11     cout << "Enter seed: ";
12     cin >> seed;
13
14     // Определяем параметры генерации случайных чисел
15     default_random_engine engine(seed); // Инициализируем механизм генерации
16     uniform_int_distribution randomDie{1, 6};
17
18     // Показываем 10 бросков кубика
19     for (int counter{1}; counter <= 10; ++counter) {
20         cout << randomDie(engine) << " ";
21     }
22
23     cout << '\n';
24 }
```

```
Enter seed: 67
6 2 5 6 6 2 2 6 2 1
Enter seed: 432
3 5 6 1 6 1 4 4 2 2
Enter seed: 67
6 2 5 6 6 2 2 6 2 1
```

Илл. 5.4 | Инициализация генератора случайных чисел

5.8.4. Инициализация генератора случайных чисел с помощью `random_device`

Правильный способ рандомизации — инициализировать генератор случайных чисел объектом `random_device` (C++11, заголовок `<random>`), как мы сделаем в следующей программе, показанной на илл. 5.5. Объект `random_device` производит равномерно распределенные и недетерминированные случайные целые числа,

которые невозможно предсказать¹. Однако в документации `random_device` указано, что по соображениям быстродействия его следует использовать только для инициализации генератора случайных чисел². Кроме того, `random_device` может быть детерминированным (предсказуемым) на некоторых платформах³. Поэтому обязательно ознакомьтесь с документацией вашего компилятора, прежде чем использовать `random_device` в ответственных приложениях. Например, в Visual C++ вы получите недетерминированные, криптографически надежные случайные числа⁴.

5.9. Программа «Игра в кости» и знакомство с перечислениями

Одной из самых популярных азартных игр является игра в кости, также известная как крэпс. В нее играют в казино по всему миру. Правила очень просты:

Игрок бросает два кубика (две «кости»). На шести гранях каждого кубика изображены 1, 2, 3, 4, 5 и 6 точек соответственно. После броска суммируется количество точек на двух обращенных вверх гранях. Если сумма первого броска равна 7 или 11, игрок выигрывает. Если сумма первого броска равна 2, 3 или 12 (это называется «крэпс»), игрок проигрывает (а «дом» выигрывает). Если в первом броске сумма равна 4, 5, 6, 8, 9 или 10, то она становится «целью» игрока. Теперь, чтобы выиграть, игрок продолжает бросать кубики, пока не выпадет «цель». Если в любом из этих бросков выпадает 7, а «цель» не достигнута, игрок проигрывает.

Обратите внимание: в правилах сказано, что кубики бросают парами и в первом, и во всех последующих бросках. Мы определим функцию `rollDice` для моделирования бросков, вычисления и отображения суммы выпавших чисел. Функция может вызываться несколько раз: один раз для первого броска игры и, возможно, еще много раз, если игрок не выигрывает или не проигрывает при первом броске. Ниже приведены результаты нескольких примеров выполнения, показывающие разные варианты:

- ♦ игрок выигрывает в первом броске при выпадении 7;
- ♦ игрок выигрывает при последующих бросках, когда выпадает «цель» (до выпадения 7);
- ♦ игрок проигрывает в первом броске при выпадении 12;

¹ «Nondeterministic Algorithm». Был доступен: 15 сентября 2023 г. https://en.wikipedia.org/wiki/Nondeterministic_algorithm.

² «std::random_device». Был доступен: 15 сентября 2023 г. https://en.cppreference.com/w/cpp/numeric/random/random_device.

³ «std::random_device». Был доступен: 15 сентября 2023 г. https://en.cppreference.com/w/cpp/numeric/random/random_device.

⁴ «random_device Class», 3 августа 2021 г. Был доступен: 15 сентября 2023 г. <https://docs.microsoft.com/en-us/cpp/standard-library/random-device-class>.

- ♦ игрок проигрывает при последующих бросках, когда выпадает 7 (до выпадения «цели»).

```
Player rolled 2 + 5 = 7
Player wins
```

```
Player rolled 3 + 3 = 6
Point is 6
Player rolled 5 + 3 = 8
Player rolled 4 + 5 = 9
Player rolled 2 + 1 = 3
Player rolled 1 + 5 = 6
Player wins
```

```
Player rolled 6 + 6 = 12
Player loses
```

```
Player rolled 1 + 3 = 4
Point is 4
Player rolled 4 + 6 = 10
Player rolled 2 + 4 = 6
Player rolled 6 + 4 = 10
Player rolled 2 + 3 = 5
Player rolled 2 + 4 = 6
Player rolled 1 + 1 = 2
Player rolled 4 + 4 = 8
Player rolled 4 + 3 = 7
Player loses
```

Реализация игры

Программа для игры в кости (илл. 5.5) имитирует игру с помощью двух функций — `main` и `rollDice`, а также инструкций `switch`, `while`, `if...else` и вложенных инструкций `if...else`. Прототип функции `rollDice` (строка 8) указывает, что эта функция не принимает аргументы (пустые круглые скобки) и возвращает значение `int` (сумму выпавших в броске чисел).

```
1 // fig05_05.cpp
2 // Игра в кости
3 #include <fmt/format.h>
4 #include <iostream>
5 #include <random>
6 using namespace std;
7
8 int rollDice(); // Броски кубиков, подсчет и отображение суммы
9
```

Перечисления с ограниченной областью видимости (C++11)

Игрок может выиграть или проиграть как в первом броске, так и в любом последующем. Программа отслеживает это с помощью переменной `gameStatus`, которая объявляется в строке 15 с новым типом `Status`. В строке 12 объявляется пользовательский тип данных, называемый **перечислением с ограниченной областью видимости (scoped enumeration)**. Это объявление начинается с ключевых слов `enum class`, за которыми следует тип (`Status`) и набор идентификаторов, представляющих три целочисленные константы.

```
10 int main() {  
11     // Перечисление, определяющее статус игры  
12     enum class Status {keepRolling, won, lost};  
13  
14     int myPoint{0}; // Цель, если игра не закончилась после первого броска  
15     Status gameStatus{Status::keepRolling}; // Игра продолжается  
16 }
```

Внутренние значения констант перечисления (**enumeration constants**) имеют тип `int`, начинаются с 0 и по умолчанию увеличиваются на 1. В перечислении `Status` константа `keepRolling` имеет значение 0, константа `won` — 1, а `lost` — 2. Идентификаторы в `enum class` должны быть уникальными, но несколько идентификаторов могут иметь одно и то же значение. Переменным пользовательского типа `Status` могут быть присвоены только константы, объявленные в перечислении.

По соглашению, вы должны писать с заглавной буквы имя пользовательского типа `enum class` и каждое последующее слово в этом имени, если имя составлено из нескольких слов (например, `ProductCode`). В C++ Core Guidelines указано, что для констант в `enum class` действуют те же соглашения об именовании, что и для переменных^{1,2}.

Чтобы сослаться на константу перечисления `enum`, определите имя типа перечисления (`Status`) и оператор разрешения области видимости (`::`), как показано в строке 15. Этот оператор инициализирует переменную `gameStatus` значением `Status::keepRolling`. При выигрыше программа присваивает переменной `gameStatus` значение `Status::won`, а при проигрыше — `Status::lost`.

Выигрыш или проигрыш в первом броске

Следующая инструкция `switch` определяет, выиграет или проиграет игрок в первом броске.

¹ C++ Core Guidelines. Accessed May 11, 2020. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Renum-caps>.

² В давно созданном коде C++ вы можете встретить константы перечисления, написанные заглавными буквами, но теперь эта практика считается устаревшей.

```
17 // Определяем статус игры и цель (при необходимости) по первому броску
18 switch (const int sumOfDice(rollDice())) {
19     case 7: // Выигрыш при выпадении 7
20     case 11: // Выигрыш при выпадении 11
21         gameStatus = Status::won;
22         break;
23     case 2: // Проигрыш при выпадении 2
24     case 3: // Проигрыш при выпадении 3
25     case 12: // Проигрыш при выпадении 12
26         gameStatus = Status::lost;
27         break;
28     default: // Не выигрыш и не проигрыш, игра продолжается
29         myPoint = sumOfDice; // Запоминаем цель
30         cout << fmt::format("Point is {}\n", myPoint);
31         break; // Не обязательно, но рекомендуется в конце инструкции switch
32 }
33
```

Инициализатор инструкции `switch` (строка 18) создает переменную `sumOfDice` для суммы броска и инициализирует ее вызовом функции `rollDice`. Если сумма броска равна 7 или 11, в строке 21 переменная `gameStatus` получает значение `Status::won`. Если сумма броска равна 2, 3 или 12, в строке 26 переменная `gameStatus` получает значение `Status::lost`. Если сумма броска другая, значение `gameStatus` остается неизменным (`Status::keepRolling`), а `sumOfDice` копируется в переменную `myPoint` в строке 29 и выводится на экран в строке 30.

Игра продолжается

После первого броска, если значение `gameStatus` равно `Status::keepRolling`, выполнение программы продолжается со следующей инструкции `while`:

```
34 // Игра еще не закончена
35 while (Status::keepRolling == gameStatus) { // Не выигрыш и не проигрыш
36     // Снова бросаем кубики и определяем статус игры
37     if (const int sumOfDice(rollDice()); sumOfDice == myPoint) {
38         gameStatus = Status::won;
39     }
40     else if (sumOfDice == 7) { // Проигрыш при выпадении 7
41         gameStatus = Status::lost;
42     }
43 }
44
```

В каждой итерации цикла инициализатор инструкции `if` (строка 37) вызывает функцию `rollDice`, чтобы получить новое значение суммы броска `sumOfDice`. Если `sumOfDice` равна «цели», в строке 38 устанавливается статус `Status::won` и цикл завершается. Если `sumOfDice` равна 7, в строке 41 устанавливается статус `Status::lost` и цикл завершается. В остальных случаях цикл продолжает выполняться.

Вывод сообщений о выигрыше и проигрыше

Когда предыдущий цикл завершается, программа переходит к следующей инструкции `if...else`, которая выводит сообщение "Player wins", если `gameStatus` равен `Status::won`, или "Player loses", если `gameStatus` равен `Status::lost`.

```
45 // Выводим сообщение о выигрыше или проигрыше
46 if (Status::won == gameStatus) {
47     cout << "Player wins\n";
48 }
49 else {
50     cout << "Player loses\n";
51 }
52 }
53
```

Функция `rollDice`

Функция `rollDice` моделирует броски двух кубиков (строки 61–62), вычисляет сумму выпавших чисел (строка 63), выводит на экран значения граней и их сумму (строка 66), затем возвращает результат (строка 68).

```
54 // Бросаем кубики, вычисляем сумму и отображаем результаты
55 int rollDice() {
56     // Инициализация генератора случайных чисел
57     static random_device rd; // Для инициализации default_random_engine
58     static default_random_engine engine(rd()); // Инициализируем генератор
59     static uniform_int_distribution randomDie{1, 6};
60
61     const int die1(randomDie(engine)); // Первый бросок
62     const int die2(randomDie(engine)); // Второй бросок
63     const int sum{die1 + die2}; // Сумма двух бросков
64
65     // Выводим результаты бросков
66     cout << fmt::format("Player rolled {} + {} = {}\n", die1, die2, sum);
67
68     return sum;
69 }
```

Чаше всего для получения случайных чисел программист создает один механизм генератора, инициализирует его и использует на протяжении всей программы. В нашей программе доступ к генератору случайных чисел нужен только функции `rollDice`, поэтому в строках 57–59 объекты `random_device`, `default_random_engine` и `uniform_int_distribution` определяются как **статические** (`static`) локальные переменные. В отличие от других локальных переменных, которые существуют только до завершения вызова функции, статические локальные переменные продолжают существовать и сохраняют свои значения между вызовами функции. Объявление объектов в строках 57–59 с ключевым словом `static` гарантирует, что они будут созданы только при первом вызове функции `rollDice`. Затем они будут повторно использоваться во всех последующих вызовах этой функции. Вы-

ражение `rd()` в строке 58 получает недетерминированное случайное целое число из объекта `random_device` и использует его для инициализации `default_random_engine`, чтобы программа выдавала разные результаты при каждом выполнении.

Дополнительные сведения о перечислениях с ограниченной областью видимости

Обращение к константе `enum class` через имя типа с последующим оператором `::` явно идентифицирует эту константу как находящуюся в области видимости этого типа. Если какой-то другой `enum class` содержит тот же идентификатор, вам всегда ясно, какая из одноименных констант используется, потому что имя типа и оператор `::` обязательны. Всегда используйте уникальные значения констант перечисления, чтобы предотвратить трудно выявляемые логические ошибки.

Приведем пример популярного перечисления с ограниченной областью видимости:

```
enum class Month {jan = 1, feb, mar, apr, may, jun, jul, aug, sep, oct, nov, dec};
```

Эта инструкция создает пользовательский тип `Month` с константами перечисления, представляющими месяцы года. Первой константе в перечислении явно присвоено значение 1, а значение каждой следующей на 1 больше предыдущего, что дает последовательность от 1 до 12. В определении `enum class` вы можете явно присвоить целочисленное значение любой константе. Следующие за ней константы автоматически получают следующие по порядку значения (каждое на 1 больше предыдущего), пока среди них не встретится другое явное присваивание.

Перечисления, не имеющие ограничения области видимости (до C++11)

Перечисления также можно определять с помощью ключевого слова `enum`, за которым следует имя типа и набор целочисленных констант:

```
enum Status {keepRolling, won, lost};
```

Область видимости констант в таком перечислении не ограничена. Вы можете ссылаться на них по именам: `keepRolling`, `won` и `lost`. Если в нескольких таких перечислениях есть константы с одинаковыми именами, это может привести к конфликтам имен и ошибкам компиляции. Поэтому в C++ Core Guidelines рекомендуется всегда использовать `enum class`¹.

Выбор базовых типов констант в перечислениях (C++11)

Константы в каждом перечислении имеют значения одного из целочисленных типов — назовем его базовым. В перечислениях без ограничения области видимости этот базовый тип зависит от значений констант в конкретной программе и гарантированно будет достаточным для их хранения. В перечислениях с ограниченной областью видимости базовый тип по умолчанию — `int`, но можно указать

¹ C++ Core Guidelines, «Enum.3: Prefer Class enums over 'Plain' enums». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Renum-class>.

и другой тип с помощью двоеточия (:). Например, вы могли бы выбрать тип `short` для констант в перечислении `Status`:

```
enum class Status : short {keepRolling, won, lost};
```

Объявления `using` в перечислениях (C++20)

Если пользовательский тип констант `enum class` очевиден в контексте их применения (как в нашей игре в кости), то в C++20 на них можно ссылаться без типа и оператора области видимости (::) с помощью объявлений `using`^{1,2}. Например, добавьте следующую инструкцию после объявления `enum class`:

```
using enum Status;
```

Затем в остальной части программы вы сможете писать имена соответствующих констант кратко: `keepRolling`, `won` и `lost` (вместо `Status::keepRolling`, `Status::won` и `Status::lost`).

Объявления `using` применимы и к отдельным константам `enum class`:

```
using Status::keepRolling;
```

После этой инструкции имя `keepRolling` тоже можно писать кратко (без квалификатора `Status::`). Как правило, такие объявления `using` должны быть размещены внутри блока, который их использует.

5.10. Правила областей видимости

Та часть программы, в которой может использоваться идентификатор, называется его **областью видимости (scope)**. Например, когда мы объявляем локальную переменную в блоке, на нее можно ссылаться только:

- ♦ с момента ее объявления и до конца блока;
- ♦ во вложенных блоках, которые появляются после объявления переменной.

В этом разделе мы рассмотрим область видимости блока и глобальную область видимости. Имена параметров в прототипе функции имеют **область видимости параметров функции (function parameter scope)** и известны только в том прототипе, где они объявлены. В главе 9 мы рассмотрим **область видимости класса (class scope)**.

Область видимости блока

Идентификатор, объявленный в блоке, находится в **области видимости блока (block scope)**, которая начинается с объявления идентификатора и заканчивается

¹ Гашпер Ажман (Gašper Ažman) и Джонатан Мюллер (Jonathan Müller), «Using Enum», 16 июля 2019 г. Был доступен: 15 сентября 2023 г. <http://wg21.link/p1099r5>.

² На момент написания книги эта возможность реализована только в компиляторе Microsoft Visual C++.

завершающей фигурной скобкой `}` блока. Локальные переменные имеют область видимости блока, как и параметры функции. Любой блок может содержать объявления переменных. Во вложенных блоках, если имена идентификаторов из внешнего и внутреннего блоков совпадают, идентификатор внешнего блока автоматически скрывается до завершения внутреннего блока. Иными словами, внутренний блок видит свою собственную локальную переменную, а не одноименную переменную из внешнего блока.

Если вы хотите, чтобы идентификатор внешнего блока был виден во внутреннем блоке, но случайно дадите то же имя какому-то идентификатору во внутреннем блоке, возникает логическая ошибка. Избегайте таких имен переменных во внутренних блоках, которые скрывают имена во внешних блоках. Большинство компиляторов предупредят вас в таких случаях.

Как вы видели на илл. 5.5, локальные переменные могут быть объявлены статическими (`static`). Область видимости такой переменной тоже ограничена блоком, но, в отличие от других локальных переменных, статическая локальная переменная сохраняет свое значение даже после завершения вызова функции. При следующем вызове этой функции статическая локальная переменная будет содержать значение, оставшееся после предыдущего вызова. Следующая инструкция объявляет статическую локальную переменную `count` и инициализирует ее значением 1:

```
static int count{1};
```

По умолчанию статические локальные переменные числовых типов инициализируются нулевым значением, но явная инициализация предпочтительнее. Переменные всех типов, кроме основных, инициализируются в зависимости от типа. Например, значением `string` по умолчанию является пустая строка (`""`). Мы подробнее поговорим об инициализации по умолчанию в последующих главах.

Глобальная область видимости

Идентификатор, объявленный вне функции или класса, находится в **глобальной области видимости** (`global namespace scope`). Иначе говоря, он находится в **глобальном пространстве имен** (`global namespace`) программы. Такой идентификатор «известен» всем функциям от точки его объявления до конца файла исходного кода. Определения функций, прототипы функций (размещенные вне функций), определения классов и глобальные переменные — все они имеют глобальную область видимости. **Глобальные переменные** (`global variables`) создаются путем их объявления вне определений классов или функций. Такие переменные сохраняют свои значения до тех пор, пока выполняется программа.

Объявление глобальных переменных (в отличие от локальных) приводит к неожиданным **побочным эффектам** (`side effects`), поскольку любая функция, которой не нужен доступ к этим переменным, все равно может их изменять — случайно или злонамеренно. За исключением действительно глобальных ресурсов, таких как `cin` и `cout`, избегайте глобальных переменных. Это соответствует

принципу наименьших привилегий (principle of least privilege), который является основополагающим для грамотной разработки программного обеспечения. Суть этого принципа заключается в том, что каждому фрагменту кода можно предоставлять привилегии и доступ только в том объеме, который ему необходим для выполнения своей задачи, и не более! Примером является область видимости локальной переменной — эта переменная не должна быть видна, когда она не нужна. Локальная переменная создается при вызове функции, используется этой функцией во время ее выполнения, а затем уничтожается, когда функция завершается. Принцип наименьших привилегий делает ваши программы более надежными, предотвращая случайное или злонамеренное воздействие на переменные, которое никогда не должно происходить. Это также упрощает чтение и отладку вашего кода.

В общем случае каждую переменную надо объявлять в минимально необходимой для нее области видимости. Переменные, используемые только в какой-либо конкретной функции, должны быть объявлены как локальные переменные этой функции, а не глобальные.

Демонстрация областей видимости

На илл. 5.6 демонстрируются области видимости глобальных переменных, локальных переменных и статических локальных переменных. Для удобства обсуждения код разделен на фрагменты с примерами вывода. Для краткости мы поставили подпись только под первым фрагментом и в других аналогичных примерах будем делать так же. В строке 10 объявляется и инициализируется значением 1 глобальная переменная с именем `x`. Эта глобальная переменная будет скрыта в любом блоке и в любой функции, где объявлена переменная с этим же именем `x`.

```
1 // fig05_06.cpp
2 // Пример областей видимости
3 #include <iostream>
4 using namespace std;
5
6 void useLocal(); // Прототип функции
7 void useStaticLocal(); // Прототип функции
8 void useGlobal(); // Прототип функции
9
10 int x{1}; // Глобальная переменная
11
```

Илл. 5.6 | Пример областей видимости

Функция `main`

В строке 13 выводится значение глобальной переменной `x`. В строке 15 локальная переменная `x` инициализируется значением 5. В строке 17 локальная переменная выводится на экран, чтобы показать, что глобальная переменная `x` скрыта в функции `main`. Затем в строках 19–23 определяется новый блок в функции `main`, в ко-

тором другая локальная переменная `x` инициализируется значением 7 (строка 20). В строке 22 эта локальная переменная выводится на экран, чтобы показать, что она скрывает две переменные `x`: объявленную в функции `main` и глобальную. Когда блок завершается, его переменная `x` со значением 7 автоматически уничтожается. Далее в строке 25 выводится на экран локальная переменная `x` функции `main`, чтобы показать, что она больше не скрыта.

```
12 int main() {
13     cout << "global x in main is " << x << '\n';
14
15     const int x{5}; // Локальная переменная x в функции main
16
17     cout << "local x in main's outer scope is " << x << '\n';
18
19     { // Внутренний блок создает новую область видимости
20         const int x{7}; // Переменная скрывает ранее объявленные переменные x
21
22         cout << "local x in main's inner scope is " << x << '\n';
23     }
24
25     cout << "local x in main's outer scope is " << x << '\n';
26 }
```

```
global x in main is 1
local x in main's outer scope is 5
local x in main's inner scope is 7
local x in main's outer scope is 5
```

Чтобы продемонстрировать другие области видимости, мы определим в программе три функции: `useLocal`, `useStaticLocal` и `useGlobal`, — каждая из которых не принимает аргументы и ничего не возвращает. Остальная часть функции `main` (показана ниже) вызывает каждую функцию дважды в строках 27–32. После двукратного выполнения функций `useLocal`, `useStaticLocal` и `useGlobal` программа снова выводит на экран локальную переменную `x` из функции `main`, чтобы показать, что ни один из вызовов функций не изменил ее значение, поскольку все функции ссылались на переменные в других областях видимости.

```
27 useLocal(); // Функция с локальной переменной x
28 useStaticLocal(); // Функция с локальной статической переменной x
29 useGlobal(); // Функция, использующая глобальную переменную x
30 useLocal(); // Функция заново инициализирует свою локальную переменную x
31 useStaticLocal(); // Локальная статическая переменная x сохраняет значение
32 useGlobal(); // Глобальная переменная x сохраняет значение
33
34 cout << "\nlocal x in main is " << x << '\n';
35 }
36
```

```
local x is 25 on entering useLocal
local x is 26 on exiting useLocal
```

```

local static x is 50 on entering useStaticLocal
local static x is 51 on exiting useStaticLocal

global x is 1 on entering useGlobal
global x is 10 on exiting useGlobal

local x is 25 on entering useLocal
local x is 26 on exiting useLocal

local static x is 51 on entering useStaticLocal
local static x is 52 on exiting useStaticLocal

global x is 10 on entering useGlobal
global x is 100 on exiting useGlobal

local x in main is 5

```

Функция useLocal

Функция `useLocal` инициализирует локальную переменную `x` значением 25 (строка 39). Когда `main` вызывает `useLocal` (строки 27 и 30), эта функция выводит переменную `x`, увеличивает ее и выводит снова, прежде чем вернуть управление в точку вызова. Каждый раз, когда вызывается функция `useLocal`, она заново создает локальную переменную `x` и заново инициализирует ее значением 25.

```

37 // useLocal инициализирует переменную x при каждом вызове
38 void useLocal() {
39     int x{25}; // Инициализация при каждом вызове функции
40
41     cout << "\nlocal x is " << x << " on entering useLocal\n";
42     ++x;
43     cout << "local x is " << x << " on exiting useLocal\n";
44 }
45

```

Функция useStaticLocal

Функция `useStaticLocal` объявляет статическую переменную `x`, инициализируя ее значением 50. Локальные статические переменные сохраняют свои значения, даже когда они находятся вне области видимости (то есть когда выполнение функции завершено). Когда в строке 28 вызывается функция `useStaticLocal`, она выводит свою локальную статическую переменную `x`, увеличивает ее и выводит снова перед возвращением управления в точку вызова. При следующем вызове этой функции (строка 31) статическая локальная переменная `x` содержит значение 51. Инициализация в строке 50 происходит только при первом вызове функции `useStaticLocal` (строка 28).

```

46 // Функция useStaticLocal инициализирует статическую локальную переменную x
47 // только при своем первом вызове; значение переменной сохраняется

```

```
48 // между вызовами функции
49 void useStaticLocal() {
50     static int x{50}; // инициализация только при первом вызове функции
51
52     cout << "\nlocal static x is " << x
53         << " on entering useStaticLocal\n";
54     ++x;
55     cout << "local static x is " << x
56         << " on exiting useStaticLocal\n";
57 }
58
```

Функция useGlobal

Функция `useGlobal` не объявляет никаких переменных. И когда она ссылается на переменную `x`, это глобальная переменная (инициализированная в строке 10, перед функцией `main`). Когда вызывается функция `useGlobal` (в строке 29), она выводит глобальную переменную `x`, умножает ее на 10 и выводит снова, прежде чем вернуть управление в точку вызова. В следующий раз, когда `main` будет вызывать функцию `useGlobal` (в строке 32), глобальная переменная `x` будет иметь новое значение: 10.

```
59 // useGlobal изменяет глобальную переменную x при каждом вызове
60 void useGlobal() {
61     cout << "\nglobal x is " << x << " on entering useGlobal\n";
62     x *= 10;
63     cout << "global x is " << x << " on exiting useGlobal\n";
64 }
```

5.11. Встроенные функции

Реализация программы в виде набора функций удобна с точки зрения разработчика программного обеспечения, но вызовы функций выполняются относительно медленно и создают заметную нагрузку на компьютер. Поэтому в C++, кроме обычных функций, есть **встроенные функции**, или **inline-функции** (**inline functions**), которые выполняются быстрее. Спецификатор `inline` перед типом возвращаемого значения функции в ее определении указывает компилятору, что надо сделать копию основного кода функции в каждой точке ее вызова (если это необходимо), чтобы избежать многочисленных вызовов. Как правило, это увеличивает размер программы. Впрочем, компилятор может игнорировать спецификатор `inline`. Повторно используемые встроенные функции обычно помещаются в заголовки, чтобы компилятор мог встраивать их определения в каждый файл исходного кода, где они нужны.

Если вы вносите изменение в определение встроенной функции, вы должны перекомпилировать весь исходный код, в котором вызывается эта функция. Хотя компиляторы иногда могут самостоятельно оптимизировать код по принципу встроенных функций (даже если вы не используете спецификатор `inline`), в C++

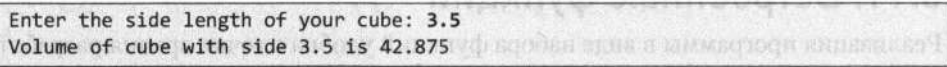
Core Guidelines рекомендуется объявлять «маленькие и критичные по быстродействию» функции со спецификатором `inline`¹.

Здесь вы найдете много часто задаваемых вопросов и ответов о встроенных функциях:

<https://isocpp.org/wiki/faq/inline-functions>

На илл. 5.7 показана встроенная функция `cube` (строки 9–11), вычисляющая объем куба.

```
1 // fig05_07.cpp
2 // inline-функция, вычисляющая объем куба
3 #include <iostream>
4 using namespace std;
5
6 // Определение функции cube предшествует
7 // ее вызову, поэтому прототип не нужен,
8 // строка определения функции выступает в качестве прототипа
9 inline double cube(double side) {
10     return side * side * side; // Вычисляем объем куба
11 }
12
13 int main() {
14     double sideValue; // Сторона куба
15     cout << "Enter the side length of your cube: ";
16     cin >> sideValue; // Считываем число, введенное пользователем
17
18     // Вычисляем объем куба и выводим результат
19     cout << "Volume of cube with side "
20          << sideValue << " is " << cube(sideValue) << "\n";
21 }
```



```
Enter the side length of your cube: 3.5
Volume of cube with side 3.5 is 42.875
```

Илл. 5.7 | Встроенная функция, вычисляющая объем куба

5.12. Ссылки и ссылочные параметры

Во многих языках программирования есть два способа передачи аргументов функциям: **передача по значению** (**pass-by-value**) и **передача по ссылке** (**pass-by-reference**). При передаче по значению сначала создается копия значения аргумента, затем эта копия передается вызываемой функции. Изменения в копии не влияют на значение исходной переменной в вызывающем коде. Это предотвращает случайные побочные эффекты, которые сильно препятствуют разработке корректных и надежных программных систем. Во всех предыдущих

¹ C++ Core Guidelines. Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rf-inline>.

примерах кода из этой книги мы передавали аргументы по значению. Но у этого способа есть недостаток: если по значению передаются большие объемы данных (большие объекты), на их копирование тратится много времени и памяти.

Ссылочные параметры функций

В этом разделе мы представим вам один из двух механизмов передачи по ссылке: ссылочные параметры^{1,2}. Когда переменная передается по ссылке, вызывающий код предоставляет вызываемой функции прямой доступ к этой переменной, поэтому функция может изменить ее в вызывающем коде.

Передача по ссылке повышает быстродействие, поскольку программа не тратит время на копирование больших объемов данных. Но передача по ссылке нарушает безопасность данных, если вызываемая функция может повредить данные в вызывающем коде..

Далее в этом разделе (после примера программы на илл. 5.8) мы покажем вам, как передавать аргумент по ссылке, чтобы получить преимущество в быстродействии (по сравнению с передачей по значению) и одновременно защитить от повреждения данные вызывающего объекта.

Ссылочный параметр (reference parameter), или **параметр-ссылка**, — это псевдоним для аргумента в вызове функции. Чтобы указать, что параметр функции передается по ссылке, добавьте амперсанд (&) после типа параметра и в прототипе, и в заголовке функции. Как всегда, прототип функции должен соответствовать ее заголовку. Например, следующее объявление параметра означает, что `number` — это ссылка на значение типа `int`:

```
int& number
```

В вызове такой функции, чтобы передать ей переменную по ссылке, надо просто указать имя этой переменной. В теле вызванной функции ссылочный параметр (например, `number`) ссылается на исходную переменную в вызывающем коде, которая может быть изменена этой функцией.

Передача аргументов по значению и по ссылке

На илл. 5.8 показана программа, в которой сравниваются два способа передачи аргумента: по значению (функция `squareByValue`) и по ссылке, со ссылочным параметром (функция `squareByReference`). Обратите внимание, что вызовы обеих функций синтаксически одинаковы: переменные `x` и `z` просто указаны по имени. Компилятор проверяет прототипы и определения функций, чтобы решить, когда нужна передача по значению, а когда — по ссылке.

¹ В главе 7 мы обсудим указатели, которые обеспечивают альтернативную форму передачи по ссылке.

² В главе 11 мы также обсудим другую форму ссылки, которая называется ссылкой `rvalue` (`rvalue reference`).

```
1 // fig05_08.cpp
2 // Передача аргументов по значению и по ссылке
3 #include <iostream>
4 using namespace std;
5
6 int squareByValue(int number); // Прототип с передачей по значению
7 void squareByReference(int& numberRef); // Прототип с передачей по ссылке
8
9 int main() {
10     int x{2}; // Значение для функции squareByValue
11     int z{4}; // Значение для функции squareByReference
12
13     // Вызов функции squareByValue
14     cout << "x = " << x << " before squareByValue\n";
15     cout << "Value returned by squareByValue: "
16         << squareByValue(x) << '\n';
17     cout << "x = " << x << " after squareByValue\n\n";
18
19     // Вызов функции squareByReference
20     cout << "z = " << z << " before squareByReference\n";
21     squareByReference(z);
22     cout << "z = " << z << " after squareByReference\n";
23 }
24
25 // squareByValue возводит number в квадрат,
26 // сохранит результат в number и возвращает новое значение number
27 int squareByValue(int number) {
28     return number *= number; // Исходный аргумент не изменяется
29 }
30
31 // squareByReference возводит numberRef в квадрат и сохраняет результат
32 // в переменную, на которую ссылается numberRef, в функции main
33 void squareByReference(int& numberRef) {
34     numberRef *= numberRef; // Исходный аргумент изменяется
35 }
```

```
x = 2 before squareByValue
Value returned by squareByValue: 4
x = 2 after squareByValue

z = 4 before squareByReference
z = 16 after squareByReference
```

Илл. 5.8 | Передача аргументов по значению и по ссылке

Ссылки в качестве псевдонимов внутри функции

Ссылки также могут использоваться в качестве псевдонимов для других переменных внутри функции (хотя обычно они используются с функциями, как показано на илл. 5.8). Например, в следующем коде переменная `count` увеличивается через ее псевдоним `cRef`:

```
int count{1}; // Объявляем целочисленную переменную count
int& cRef{count}; // Создаем переменную cRef в качестве псевдонима для count
++cRef; // Увеличиваем переменную count (через псевдоним cRef)
```

Ссылочные переменные должны быть инициализированы в объявлениях и не могут быть переназначены в качестве псевдонимов для других переменных. В этом смысле ссылки являются константами. Все операции, выполняемые с псевдонимом (то есть со ссылкой), фактически выполняются с исходной переменной. Псевдоним — это всего лишь другое имя исходной переменной. Если это не константная ссылка (обсуждается ниже), инициализатором ссылки должно быть значение *lvalue* — то, что может отображаться в левой части присваивания, например имя переменной. Ссылка не может быть инициализирована константой или значением *rvalue*, то есть тем, что может отображаться только в правой части присваивания, например результатом вычисления.

Константные ссылки

Чтобы не разрешать ссылочному параметру изменять соответствующий аргумент в вызывающем коде, поместите константный квалификатор `const` перед именем типа в объявлении параметра. Например, рассмотрим простую функцию `displayName`:

```
void displayName(std::string name) {
    std::cout << name << '\n';
}
```

При вызове она получает копию своего аргумента `string`. Поскольку объекты `string` могут быть очень большими, эта операция копирования замедляет работу программы. По этой причине объекты `string` (и все объекты вообще) желательно передавать функциям по ссылкам. Но в данном случае функции `displayName` не нужно вносить изменения в свой аргумент. Поэтому, следуя принципу наименьших привилегий, лучше объявить параметр в виде константной ссылки:

```
const std::string& name
```

Это означает, что параметр `name` — ссылка на объект `string`, который является константой (то есть его запрещено изменять). Передавая строку по ссылке, мы ускоряем работу программы. При этом функция `displayName` считает аргумент константой, то есть не может изменить его значение в вызывающем коде.

Возврат ссылки на локальную переменную может быть опасным

При возврате ссылки на локальную нестатическую переменную эта ссылка будет указывать на переменную, уже удаленную из памяти после завершения функции. Попытка получить доступ к такой переменной приводит к неопределенному поведению, то есть к сбою программы или повреждению данных¹. Ссылки на неопре-

¹ C++ Core Guidelines. Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rf-dangle>.

деленные переменные называются **висячими ссылками** (*dangling references*). Это логическая ошибка, из-за которой компиляторы обычно выдают предупреждение. Опытные команды разработчиков часто придерживаются политики компиляции кода с отсутствием предупреждений. Вы можете поступать так же в своей команде, используя опцию компилятора, которая интерпретирует предупреждения как ошибки.

5.13. Аргументы по умолчанию

Иногда в вашей программе функция может вызываться из нескольких точек, но с одним и тем же значением аргумента для какого-либо параметра. В таких случаях вы можете указать для этого параметра **аргумент по умолчанию** (*default argument*) — значение, которое будет передаваться этому параметру автоматически. Если в вызове функции не указаны аргументы для параметров, имеющих аргументы по умолчанию, компилятор переписывает этот вызов функции, вставляя в него аргументы по умолчанию.

Функция `boxVolume` с аргументами по умолчанию

На илл. 5.9 показан пример аргументов по умолчанию в функции `boxVolume`, вычисляющей объем параллелепипеда. В прототипе этой функции (строка 7) для всех трех параметров заданы значения по умолчанию, равные 1.

```
1 // fig05_09.cpp
2 // Аргументы по умолчанию
3 #include <iostream>
4 using namespace std;
5
6 // Аргументы по умолчанию заданы в прототипе функции
7 int boxVolume(int length = 1, int width = 1, int height = 1);
8
9 int main() {
10     // Вызов без аргументов, используются аргументы по умолчанию
11     cout << "The default box volume is: " << boxVolume();
12
13     // Передаем аргумент length, остальные - по умолчанию
14     cout << "\n\nThe volume of a box with length 10,\n"
15           << "width 1 and height 1 is: " << boxVolume(10);
16
17     // Передаем аргументы length и width, аргумент height - по умолчанию
18     cout << "\n\nThe volume of a box with length 10,\n"
19           << "width 5 and height 1 is: " << boxVolume(10, 5);
20
21     // Передаем все три аргумента
22     cout << "\n\nThe volume of a box with length 10,\n"
23           << "width 5 and height 2 is: " << boxVolume(10, 5, 2)
24           << '\n';
25 }
```

```
26
27 // Функция boxVolume рассчитывает объем параллелепипеда
28 int boxVolume(int length, int width, int height) {
29     return length * width * height;
30 }
```

```
The default box volume is: 1

The volume of a box with length 10,
width 1 and height 1 is: 10

The volume of a box with length 10,
width 5 and height 1 is: 50

The volume of a box with length 10,
width 5 and height 2 is: 100
```

Илл. 5.9 | Аргументы по умолчанию

В первом вызове функции `boxVolume` (строка 11) аргументы не передаются, поэтому компилятор использует все три значения по умолчанию, равные 1. Во втором вызове (строка 15) передается только аргумент `length`, а `width` и `height` получают значения по умолчанию. В третьем вызове (строка 19) передаются аргументы `length` и `width`, а `height` получает значение по умолчанию. В последнем вызове (строка 23) передаются аргументы `length`, `width` и `height`, поэтому значения по умолчанию не используются. Любые аргументы, явно переданные функции, присваиваются параметрам функции слева направо. Поэтому когда функция `boxVolume` получает один аргумент, она присваивает его значение своему параметру `length` — крайнему слева в списке. Когда `boxVolume` получает два аргумента, она присваивает их значения своим параметрам `length` и `width`. Наконец, когда функция `boxVolume` получает все три аргумента, она присваивает значения параметрам `length`, `width` и `height` соответственно.

Еще немного об аргументах по умолчанию

Аргументы по умолчанию должны быть крайними справа (завершающими) аргументами в списке параметров функции. При вызове функции с двумя или более аргументами по умолчанию, если пропущенный аргумент не является крайним справа, все аргументы справа от него тоже должны быть опущены. Аргументы по умолчанию должны быть указаны при первом появлении имени функции — как правило, в ее прототипе. Если прототип функции отсутствует и в качестве прототипа служит определение функции, аргументы по умолчанию должны быть указаны в заголовке функции. Значениями по умолчанию могут быть любые выражения, включая константы, глобальные переменные или вызовы функций. Аргументы по умолчанию можно использовать и со встроенными функциями (*inline-функциями*). Аргументы по умолчанию иногда упрощают написание вызовов функций, но некоторые программисты считают, что лучше явно указывать все аргументы, чтобы код был более понятным.

5.14. Оператор разрешения области видимости

C++ предоставляет **унарный оператор разрешения области видимости** (**unary scope resolution operator**) (`::`), необходимый для доступа к глобальной переменной, которую скрывает локальная переменная с тем же именем. Этот оператор нельзя использовать для доступа к локальной переменной с тем же именем во внешнем блоке. К глобальной переменной можно получить прямой доступ (без этого оператора), если имя глобальной переменной не совпадает с именем локальной переменной в данной области видимости.

На илл. 5.10 показано, как оператор `::` позволяет различать локальную и глобальную переменные, имеющие одинаковые имена (строки 6 и 9). Чтобы подчеркнуть, что локальная и глобальная переменные `number` совершенно разные, мы объявляем одну из них с типом `int`, а другую — с типом `double`. Старайтесь не использовать переменные с одинаковыми именами для разных целей в программе. Это допускается при некоторых обстоятельствах, но часто приводит к ошибкам. Как правило, глобальных переменных надо избегать. Но если у вас в программе есть хотя бы одна глобальная переменная, всегда используйте для доступа к ней унарный оператор разрешения области видимости (`::`), даже если нет противоречия с именем локальной переменной. Этот оператор наглядно показывает, что вы обращаетесь к глобальной переменной, упрощает модификацию программ, снижает риск коллизий имен с локальными переменными и риск логических ошибок, которые могут возникнуть, если локальная переменная скрывает глобальную.

```
1 // fig05_10.cpp
2 // Унарный оператор разрешения области видимости
3 #include <iostream>
4 using namespace std;
5
6 const int number{7}; // Глобальная переменная number
7
8 int main() {
9     const double number{10.5}; // Локальная переменная number
10
11     // Выводим значения глобальной и локальной переменных
12     cout << "Local double value of number = " << number
13         << "\nGlobal int value of number = " << ::number << '\n';
14 }
```

```
Local double value of number = 10.5
Global int value of number = 7
```

Илл. 5.10 | Унарный оператор разрешения области видимости

5.15. Перегрузка функций

C++ позволяет определять функции с одинаковыми именами, если у них разные сигнатуры. Это называется **перегрузкой функций** (**function overloading**).

Компилятор C++ выбирает подходящую функцию для вызова, проверяя количество, типы и порядок аргументов в вызове. Перегрузка функций нужна для создания нескольких одноименных функций, выполняющих аналогичные задачи с данными разных типов. Например, многие функции в математической библиотеке перегружены для разных числовых типов; согласно стандарту C++, все функции математической библиотеки, представленные в разделе 5.3, перегружены с типами `float`, `double` и `long double`. Перегрузка функций, выполняющих тесно связанные задачи, может сделать код более понятным.

Перегруженные функции `square`

На илл. 5.11 показаны перегруженные функции `square`, вычисляющие квадрат числа `int` (строки 7–10) и квадрат числа `double` (строки 13–16) соответственно. В строке 19 вызывается версия функции с типом `int` и ей передается литеральное значение 7. C++ обрабатывает литеральные значения целых чисел как `int`. В строке 21 вызывается версия функции с типом `double` и ей передается литеральное значение 7.5, которое C++ обрабатывает как `double`. В обоих случаях компилятор выбирает подходящую функцию для вызова, основываясь на типе аргумента. Выходные данные подтверждают, что в каждом случае была вызвана соответствующая функция.

```
1 // fig05_11.cpp
2 // Перегруженные функции square
3 #include <iostream>
4 using namespace std;
5
6 // Функция square для чисел int
7 int square(int x) {
8     cout << "square of integer " << x << " is ";
9     return x * x;
10 }
11
12 // Функция square для чисел double
13 double square(double y) {
14     cout << "square of double " << y << " is ";
15     return y * y;
16 }
17
18 int main() {
19     cout << square(7); // Вызываем версию для чисел int
20     cout << '\n';
21     cout << square(7.5); // Вызываем версию для чисел double
22     cout << '\n';
23 }
```

```
square of integer 7 is 49
square of double 7.5 is 56.25
```

Илл. 5.11 | Перегруженные функции `square`

Как компилятор различает перегруженные функции

Перегруженные функции различаются по их сигнатурам. Напомним, что сигнатура — это комбинация имени функции и ее параметров (расположенных по порядку). **Типобезопасная связь (type-safe linkage)** гарантирует вызов нужной функции с соответствием между типами аргументов и параметров. Чтобы обеспечить типобезопасную связь, компилятор внутренне кодирует идентификатор каждой функции с типами ее параметров — процесс, называемый **декорированием имен (name mangling)**. Эти кодировки варьируются в зависимости от компилятора, поэтому все, что будет связано с созданием исполняемого файла для конкретной платформы, надо компилировать для нее одним и тем же компилятором. Код на илл. 5.12 мы скомпилировали с помощью GNU C++¹. Вместо того чтобы показывать результат выполнения программы, как мы обычно делаем, мы покажем декорированные имена функций, полученные на языке ассемблера GNU C++².

```

1 // fig05_12.cpp
2 // Декорирование имен для типобезопасной связи
3
4 // Функция square для чисел int
5 int square(int x) {
6     return x * x;
7 }
8
9 // Функция square для чисел double
10 double square(double y) {
11     return y * y;
12 }
13
14 // Функция, принимающая аргументы
15 // int, float, char и int&
16 void nothing1(int a, float b, char c, int& d) { }
17
18 // Функция, принимающая аргументы
19 // char, int, float& и double&
20 int nothing2(char a, int b, float& c, double& d) {
21     return 0;
22 }
23
24 int main() { }
```

```

_Z6squarei
_Z6squaref
_Z8nothing1ifcRi
_Z8nothing2ciRfRd
main
```

Илл. 5.12 | Декорирование имен для типобезопасной связи

¹ Отсутствие инструкций в теле функции `main` гарантирует, что мы не получим ошибку компоновщика, если скомпилируем этот код.

² Команда `g++ -S fig05_12.cpp` создает файл на языке ассемблера `fig05_12.s`.

В GNU C++ каждое декорированное имя (кроме `main`) начинается с символа подчеркивания (`_`), за которым следуют буква `Z`, число и имя функции. Число, следующее за `Z`, указывает количество символов в имени функции. Например, имя функции `square` содержит 6 символов, поэтому ее декорированное имя имеет префикс `_Z6`. За именем функции следует условное обозначение списка ее параметров:

- ♦ Для функции `square`, которая получает значение `int` (строка 5), буква `i` означает тип `int`, как показано в первой строке вывода.
- ♦ Для функции `square`, которая получает значение `double` (строка 10), буква `d` означает тип `double`, как показано во второй строке вывода.
- ♦ Для функции `nothing1` (строка 16) буква `i` означает тип `int`, `f` — `float`, `c` — `char`, `Ri` — `int&` (то есть ссылку на `int`), как показано в третьей строке вывода.
- ♦ Для функции `nothing2` (строка 20) буква `c` означает тип `char`, `i` — `int`, `Rf` — `float&`, `Rd` — `double&`.

Компилятор различает две функции `square` по их спискам параметров: в одном из них есть `i` для работы с числами `int`, а в другом — `d` для работы с числами `double`. Типы возвращаемых значений в декорированных именах не указаны. Перегруженные функции могут иметь разные типы возвращаемых значений, но в этом случае у них должны быть разные списки параметров. Декорирование имен функций зависит от компилятора. Например, Visual C++ создает имя `square@@YANHZ` для функции `square` в строке 5. Компилятор GNU C++ не декорировал имя `main`, но некоторые компиляторы это делают. Например, в Visual C++ декорированным именем будет `_main`.

Создание перегруженных функций с одинаковыми списками параметров и разными типами возвращаемых значений вызывает ошибку компиляции. Компилятор различает перегруженные функции только по спискам параметров! Такие функции не обязательно должны иметь одинаковое количество параметров.

Иногда вызов функции, имеющей аргументы по умолчанию, может быть неотличим от вызова другой перегруженной функции, и это тоже вызовет ошибку компиляции. Например, если в программе есть функция, явно не принимающая аргументы, и одноименная функция, содержащая все аргументы по умолчанию, то ошибка компиляции возникнет при попытке использовать это имя функции в вызове, не передающем аргументов. Компилятор не сможет однозначно определить, какую версию функции выбрать.

Дополнительные сведения о перегрузке функций

Все подробности о том, как компиляторы работают с вызовами перегруженных функций, вы найдете в разделе «Overload Resolution» стандарта C++:

<https://timsong-cpp.github.io/cppwp/n4861/over.match>

а также по этой ссылке:

https://en.cppreference.com/w/cpp/language/overload_resolution

Перегрузка операторов

В главе 11 мы расскажем о перегруженных операторах и о том, как они должны работать с объектами пользовательских типов данных. Кстати, мы уже использовали несколько перегруженных операторов, в том числе оператор вывода в поток (<<) и оператор извлечения из потока (>>), которые перегружены для всех основных типов данных. В главе 11 мы подробнее обсудим перегрузку операторов << и >> для обработки объектов пользовательских типов.

5.16. Шаблоны функций

Как правило, перегруженные функции выполняют аналогичные операции с разными типами данных. Если логика программы и выполняемые операции идентичны для каждого типа данных, перегрузку можно выполнить компактнее и удобнее с помощью **шаблона функции (function template)**. Вы пишете только одно определение шаблона функции. Затем компилятор C++, учитывая разные типы аргументов, предоставляемых в вызовах этой функции, автоматически генерирует отдельные **специализации шаблона (template specializations)** этой функции, также называемые **экземплярами шаблона (template instantiations)**, для соответствующей обработки каждого вызова. Таким образом, определение одного шаблона функции фактически определяет целое семейство перегруженных функций. Программирование с шаблонами называют **обобщенным программированием (generic programming)**. В этом разделе мы определим пользовательский шаблон функции. В последующих главах вам встретятся многие шаблоны функций стандартной библиотеки C++. В главе 15 мы подробно рассмотрим пользовательские шаблоны, включая сокращенные шаблоны функций и концепты, введенные в C++20.

Шаблон функции `maximum`

На илл. 5.13 показан шаблон функции `maximum`, которая находит наибольшее из трех значений. Определение шаблона функции всегда начинается с ключевого слова `template` (строка 3), за которым следует список параметров шаблона в угловых скобках (< и >). В любом из этих параметров (в одном или в нескольких) основной или пользовательский тип данных заменяется заполнителем — **параметром-типом (type parameter)**, которому предшествует ключевое слово `typename` или `class` (в данном контексте это синонимы). В нашем примере список параметров состоит из одного параметра-типа с именем `T`. Такими заполнителями обозначаются: типы параметров функции (строка 4), тип возвращаемого значения функции (строка 4) и типы переменных в теле определения функции (строка 5). Шаблон функции определяется так же, как и любая другая функция, но вместо одного или нескольких реальных типов данных в шаблоне подставляются параметры-типы.

Параметр-тип `T` в шаблоне функции `maximum` (строка 3) является заполнителем для типа данных, обрабатываемых этой функцией. Встречая вызов функции `maximum`

в исходном коде программы, компилятор подставляет соответствующий тип аргументов вместо каждого заполнителя `T` в определении шаблона. Так создается полная специализация шаблона нашей функции, определяющая наибольшее из трех значений указанного типа. Все три значения должны быть одного типа, поскольку в этом примере мы используем только один параметр-тип. Затем вновь созданная функция компилируется — шаблоны являются средством генерации кода. В главе 13 мы будем использовать шаблоны стандартной библиотеки C++, имеющие несколько параметров-типов.

```

1 // Fig. 5.13: maximum.h
2 // Заголовочный файл с шаблоном функции maximum
3 template <typename T> // Или template <class T>
4 T maximum(T value1, T value2, T value3) {
5     T maximumValue{value1}; // Предположим, что число value1 - наибольшее
6
7     // Проверяем: число value2 больше, чем maximumValue?
8     if (value2 > maximumValue) {
9         maximumValue = value2;
10    }
11
12    // Проверяем: число value3 больше, чем maximumValue?
13    if (value3 > maximumValue) {
14        maximumValue = value3;
15    }
16
17    return maximumValue;
18 }
```

Илл. 5.13 | Заголовочный файл с шаблоном функции `maximum`

Тестирование шаблона функции `maximum`

На илл. 5.14 показана программа, которая с помощью шаблона функции `maximum` находит наибольшее из трех значений `int`, трех значений `double` и трех значений `char` соответственно (строки 15, 24 и 33). Поскольку типы аргументов в каждом из трех вызовов отличаются, компилятор «под капотом» создает для этих вызовов отдельные определения функции: первое ожидает три значения `int`, второе — три значения `double`, третье — три значения `char`.

```

1 // fig05_14.cpp
2 // Тестирование шаблона функции maximum
3 #include <iostream>
4 #include "maximum.h" // Определение шаблона функции maximum
5 using namespace std;
6
7 int main() {
8     // Вводим три значения int
9     cout << "Input three integer values: ";
```

```

10  int int1, int2, int3;
11  cin >> int1 >> int2 >> int3;
12
13  // Вызываем функцию maximum с типом int
14  cout << "The maximum integer value is: "
15       << maximum(int1, int2, int3);
16
17  // Вводим три значения double
18  cout << "\n\nInput three double values: ";
19  double double1, double2, double3;
20  cin >> double1 >> double2 >> double3;
21
22  // Вызываем функцию maximum с типом double
23  cout << "The maximum double value is: "
24       << maximum(double1, double2, double3);
25
26  // Вводим три значения char
27  cout << "\n\nInput three characters: ";
28  char char1, char2, char3;
29  cin >> char1 >> char2 >> char3;
30
31  // Вызываем функцию maximum с типом char
32  cout << "The maximum character value is: "
33       << maximum(char1, char2, char3) << '\n';
34 }

```

Input three integer values: 1 2 3

The maximum integer value is: 3

Input three double values: 3.3 2.2 1.1

The maximum double value is: 3.3

Input three characters: A C B

The maximum character value is: C

Илл. 5.14 | Тестирование шаблона функции maximum

Специализация шаблона функции maximum для типа int

В специализации шаблона функции, созданной для работы с типом `int`, компилятор заменяет все заполнители `T` на `int`:

```

int maximum(int value1, int value2, int value3) {
    int maximumValue{value1}; // Предположим, что число value1 - наибольшее

    // Проверяем: число value2 больше, чем maximumValue?
    if (value2 > maximumValue) {
        maximumValue = value2;
    }

    // Проверяем: число value3 больше, чем maximumValue?

```

```
if (value3 > maximumValue) {  
    maximumValue = value3;  
}  
return maximumValue;  
}
```

5.17. Рекурсия

Для решения некоторых видов задач хорошо подходят функции, вызывающие сами себя. Рекурсивная функция — это функция, которая вызывает саму себя прямо или косвенно (через другую функцию). В этом и в следующем разделах представлены простые примеры рекурсии. Более детальное изучение рекурсии входит в курс информатики высшего уровня.

Концепция рекурсии

Сначала мы рассмотрим рекурсию концептуально, затем перейдем к программам с рекурсивными функциями. Рекурсивные подходы к решению задач имеют несколько общих элементов. Рекурсивная функция вызывается для решения задачи, но умеет решать только ее простейший вариант: так называемую **базовую задачу (base case)**. Если функция вызывается с базовой задачей, она сразу возвращает результат. Если функция вызывается с более сложной задачей, она делит эту задачу на две концептуальные части: 1) часть, которую она может решить, и 2) часть, которую она не может решить. Для правильной рекурсии вторая часть должна быть упрощенной (уменьшенной) версией исходной задачи. Упрощенная задача похожа на исходную, поэтому функция вызывает свою копию для работы над ней. Это называется **рекурсивным вызовом (recursive call)**, или **шагом рекурсии (recursion step)**. Шаг рекурсии часто включает ключевое слово `return`, потому что результат шага будет объединен с той частью задачи, которую функция может решить, — для формирования результата, возвращаемого в точку вызова (например, в функцию `main`).

Недостижимость базовой задачи или неправильно написанный шаг рекурсии (без сходимости к базовой задаче) приводят к ошибке в виде бесконечной рекурсии, которая обычно завершается крахом программы из-за переполнения стека. Бесконечная рекурсия также возникает, если функция, которая не должна была быть рекурсивной, случайно вызывает саму себя, прямо или косвенно (через другую функцию).

Шаг рекурсии выполняется, пока «открыт» исходный вызов функции, то есть пока его выполнение еще не завершилось. Шаг рекурсии может привести к множеству последующих шагов, поскольку функция продолжает разделять каждую новую подзадачу, с которой она вызывается, на две концептуальные части. Чтобы рекурсия в итоге завершилась, функция при каждом вызове самой себя должна получать все более простую версию задачи. Иными словами, последовательность упрощающихся задач должна постепенно сходиться к базовой задаче. Когда функ-

ция распознаёт базовую задачу, она возвращает результат в предыдущую копию функции. Затем происходит последовательность возвратов — до тех пор, пока исходный вызов функции не вернет конечный результат в точку вызова¹. Все это выглядит довольно экзотично по сравнению с теми способами решения задач, которые вы уже знаете. В качестве примера рекурсии мы рассмотрим программу для популярных математических вычислений.

Факториал

Напомним, что факториал — это произведение всех целых чисел от 1 до заданного числа. Факториал неотрицательного целого числа n , обозначаемый в математической записи как $n!$, является результатом выражения:

$$n \cdot (n - 1) \cdot (n - 2) \cdot \dots \cdot 1$$

При этом принимается, что факториал нуля равен 1 ($0! = 1$), факториал единицы тоже равен 1 ($1! = 1$). Например, $5!$ (произносится «пять факториал» или «факториал числа пять») — это результат выражения $5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$, что составляет 120.

Итеративное вычисление факториала

Факториал целого числа `number`, большего или равного 0, может быть вычислен итеративно (не рекурсивно) в цикле `for`:

```
int factorial{1};
for (int counter{number}; counter >= 1; --counter) {
    factorial *= counter;
}
```

Рекурсивное вычисление факториала

Рекурсивное определение факториала достигается разбором следующего алгебраического выражения:

$$n! = n \cdot (n - 1)!$$

Например, очевидно, что $5! = 5 \cdot 4!$, как показано в следующем примере:

$$5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$$

$$5! = 5 \cdot (4 \cdot 3 \cdot 2 \cdot 1)$$

$$5! = 5 \cdot (4!)$$

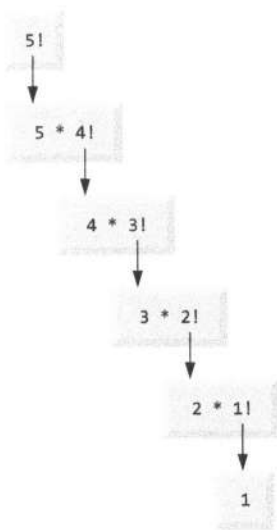
Вычисление 5!

На следующей диаграмме вы видите процесс вычисления выражения $5!$ рекурсивным методом. Часть (а) показывает, как продолжается последовательность

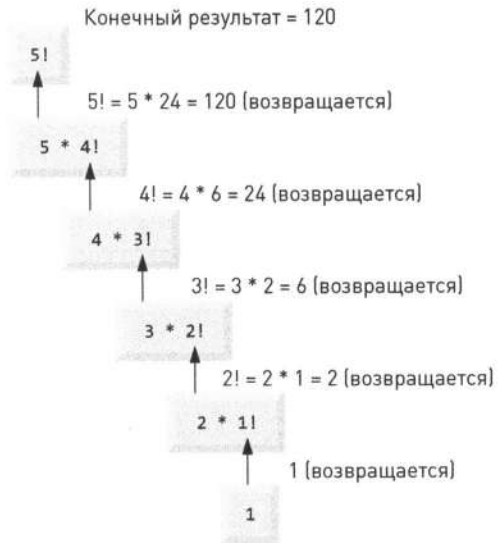
¹ В стандарте C++ указано, что функцию `main` не следует вызывать из программы или рекурсивно. Единственное назначение этой функции — быть стартовой точкой для выполнения программы. «Стандарт C++». Был доступен: 15 сентября 2023 г. <https://timsong-cpp.github.io/cppwp/n4861/basic.start.main> и <https://timsong-cpp.github.io/cppwp/n4861/expr.call>.

рекурсивных вызовов, пока не дойдет до вычисления факториала единицы ($1! = 1$), что завершит рекурсию. Часть (b) показывает значения, которые будут возвращаться в каждом рекурсивном вызове, пока не будет вычислено и возвращено конечное значение.

(a) Обработка рекурсивных вызовов



(b) Значения, возвращаемые от каждого рекурсивного вызова



Вычисление факториалов с помощью рекурсивной функции `factorial`

На илл. 5.15 показана программа, которая вычисляет и выводит факториалы чисел от 0 до 10. Рекурсивная функция `factorial` (строки 18–25) сначала проверяет условие `number <= 1`, означающее базовую задачу (строка 19). Если `number` меньше или равно 1, функция возвращает 1 (строка 20), дальнейшая рекурсия не требуется и функция завершается. Если `number` больше 1, в строке 23 задача выражается в виде произведения `number` и результата рекурсивного вызова функции `factorial`, вычисляющего факториал выражения `number - 1`. Это вычисление является немного более простой задачей, чем исходное вычисление `factorial(number)`.

```

1 // fig05_15.cpp
2 // Рекурсивная функция factorial
3 #include <iostream>
4 #include <iomanip>
5 using namespace std;
6
7 long factorial(int number); // Прототип функции
8

```



```

9 int main() {
10     // Вычисляем факториалы чисел от 0 до 10
11     for (int counter{0}; counter <= 10; ++counter) {
12         cout << setw(2) << counter << "!" << factorial(counter)
13             << '\n';
14     }
15 }
16
17 // Рекурсивное определение функции factorial
18 long factorial(int number) {
19     if (number <= 1) { // Проверяем базовую задачу
20         return 1; // 0! = 1 и 1! = 1
21     }
22     else { // Делаем шаг рекурсии
23         return number * factorial(number - 1);
24     }
25 }

```

```

0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
7! = 5040
8! = 40320
9! = 362880
10! = 3628800

```

Илл. 5.15 | Рекурсивная функция, вычисляющая факториалы

Значения факториалов быстро увеличиваются

Функция `factorial` принимает параметр типа `int` и возвращает результат типа `long`. Обычно `long` хранится по меньшей мере в четырех байтах (32 бита); такая переменная может содержать значение в диапазоне от $-2\,147\,483\,648$ до $2\,147\,483\,647$. К сожалению, функция `factorial` возвращает числа, возрастающие так быстро, что тип `long` не позволяет вычислить много факториалов до того, как мы выйдем за пределы диапазона `long`. Для вычислений с еще большими числами мы могли бы использовать тип `long long` (раздел 3.11) или класс, представляющий целые числа произвольного размера (например, класс `BigInteger` с открытым исходным кодом, который мы представили в разделе 3.12).

5.18. Пример рекурсивных вычислений: числа Фибоначчи

Ряд чисел Фибоначчи начинается так:

0, 1, 1, 2, 3, 5, 8, 13, 21, ...

Иными словами, первые числа ряда — это 0 и 1, а каждое последующее число Фибоначчи является суммой двух предыдущих.

Эти числа можно найти в природе, так как они описывают форму спирали. Отношение между двумя любыми соседними элементами ряда Фибоначчи сходится к числу 1.618..., представленному константой `numbers::phi` в заголовке `<numbers>`. Это число называется «золотым сечением» или «золотой пропорцией». Люди считают эту пропорцию эстетически приятной. Архитекторы часто проектируют окна, комнаты и здания, стороны которых имеют именно такую пропорцию. Она встречается даже в соотношении длины и ширины почтовых открыток. По запросу «числа Фибоначчи в природе» вы найдете в интернете много интересных примеров, включая лепестки цветов, раковины, ураганы и спиральные галактики.

Определение рекурсивной функции `fibonacci`

Последовательность чисел Фибоначчи может быть определена рекурсивно таким образом:

$$\text{fibonacci}(0) = 0$$
$$\text{fibonacci}(1) = 1$$
$$\text{fibonacci}(n) = \text{fibonacci}(n - 1) + \text{fibonacci}(n - 2)$$

В программе, показанной на илл. 5.16, для рекурсивного вычисления числа Фибоначчи с порядковым номером n используется функция `fibonacci`. Программа выводит числа Фибоначчи с несколькими порядковыми номерами. Как видите, числа в ряду Фибоначчи быстро возрастают, хотя и гораздо медленнее, чем факториалы.

```
1 // fig05_16.cpp
2 // Рекурсивная функция fibonacci
3 #include <iostream>
4 using namespace std;
5
6 long fibonacci(long number); // Прототип функции
7
8 int main() {
9     // Вычисляем числа Фибоначчи от 1 до 10
10    for (int counter{0}; counter <= 10; ++counter)
11        cout << "fibonacci(" << counter << ") = "
12            << fibonacci(counter) << '\n';
13
14    // Выводим 20-е, 30-е и 35-е числа Фибоначчи
15    cout << "\nfibonacci(20) = " << fibonacci(20) << '\n';
16    cout << "fibonacci(30) = " << fibonacci(30) << '\n';
17    cout << "fibonacci(35) = " << fibonacci(35) << '\n';
18 }
19
```

```

20 // Рекурсивная функция fibonacci
21 long fibonacci(long number) {
22     if ((0 == number) || (1 == number)) { // Проверяем базовую задачу
23         return number;
24     }
25     else { // Делаем шаг рекурсии
26         return fibonacci(number - 1) + fibonacci(number - 2);
27     }
28 }

```

```

fibonacci(0) = 0
fibonacci(1) = 1
fibonacci(2) = 1
fibonacci(3) = 2
fibonacci(4) = 3
fibonacci(5) = 5
fibonacci(6) = 8
fibonacci(7) = 13
fibonacci(8) = 21
fibonacci(9) = 34
fibonacci(10) = 55

fibonacci(20) = 6765
fibonacci(30) = 832040
fibonacci(35) = 9227465

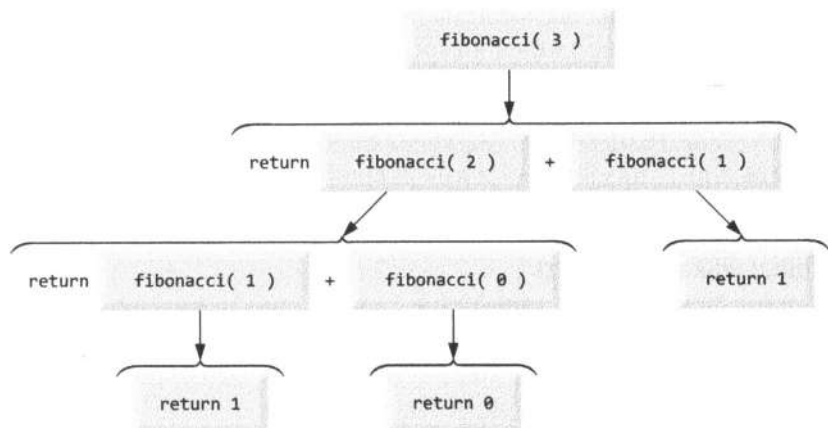
```

Илл. 5.16 | Рекурсивная функция fibonacci

Программа начинается с цикла, который вычисляет и отображает числа Фибоначчи в диапазоне от 0 до 10. Далее следуют три вызова функции для вычисления чисел Фибоначчи с номерами 20, 30 и 35 (строки 15–17). Вызовы `fibonacci` в функции `main` (строки 12 и 15–17) не являются рекурсивными, но вызовы из функции `fibonacci` (строка 26) являются рекурсивными. Каждый раз, когда программа вызывает функцию `fibonacci` (строки 21–28), эта функция проверяет базовую задачу, сравнивая `number` с нулем и единицей (строка 22). Если условие истинно, в строке 23 возвращается `number`. Интересный момент: если `number` больше 1, шаг рекурсии (строка 26) генерирует два рекурсивных вызова, каждый для немного более простой задачи, чем исходный вызов функции `fibonacci`.

Вычисление fibonacci(3)

На следующей диаграмме показано, как функция `fibonacci` вычисляет число Фибоначчи с номером 3. Возникает интересный вопрос о том порядке, в котором компиляторы C++ вычисляют операнды операторов. В данном случае правила приоритета и группировки операторов не дают ответа; это отдельная проблема. Как видите, вычисление `fibonacci(3)` влечет за собой два рекурсивных вызова функции: `fibonacci(2)` и `fibonacci(1)`. В каком порядке будут выполнены эти вызовы?



Порядок вычисления операндов

Большинство программистов предполагают, что операнды вычисляются слева направо, что имеет место в некоторых языках программирования. На самом деле в C++ не регламентируется порядок вычисления операндов многих операторов (в том числе +). Поэтому вы не должны делать никаких предположений о порядке выполнения этих вызовов. Программа может сначала выполнить `fibonacci(2)`, а потом `fibonacci(1)` или наоборот: сначала `fibonacci(1)`, а потом `fibonacci(2)`. В нашей программе, как и в большинстве других, конечный результат в любом случае будет одним и тем же. Однако в некоторых программах вычисление операндов вызывает побочные эффекты (например, операции ввода-вывода или изменения значений переменных), которые могут повлиять на конечный результат выражения.

Операторы, для которых указан порядок вычисления

До C++17 в языке C++ регламентировался порядок вычисления операндов только для четырех операторов: `&&`, `||`, «запятая» `,` и `?:`. Первые три — это бинарные операторы, два операнда которых гарантированно вычисляются слева направо. Четвертый является единственным оператором C++ с тремя операндами. Его крайний левый операнд всегда вычисляется первым. Если он принимает значение `true`, средний операнд вычисляется следующим, а последний операнд игнорируется. Если крайний левый операнд вычисляется как `false`, третий операнд вычисляется следующим, а средний операнд игнорируется.

Начиная с C++17, регламентируется порядок вычисления операндов еще для нескольких операторов. Компилятор вычисляет операнды слева направо для операторов `[]` (глава 6), `->` (глава 7), `<<`, `>>`, `.*`, `->*`, а также для оператора «круглые скобки» в вызове функции (это означает, что компилятор обрабатывает имя функции раньше, чем ее аргументы). Операнды операторов присваивания компилятор вычисляет справа налево.

Если программа написана так, что зависит от порядка вычисления операндов, это может привести к логическим ошибкам. Для уверенности в том, что побочные эффекты проявляются в нужном порядке, разбивайте сложные выражения на отдельные инструкции. Напомним, что операторы `&&` и `||` используют вычисления по короткой схеме. Размещение выражения с побочным эффектом в правой части оператора `&&` или `||` является логической ошибкой, если это выражение всегда должно вычисляться.

Экспоненциальная сложность

Рекурсивные программы, подобные той, которую мы написали для генерации чисел Фибоначчи, могут быть опасными. Дело в том, что каждый шаг рекурсии в функции `fibonacci` удваивает количество вызовов функции. Таким образом, количество рекурсивных вызовов, необходимых для вычисления числа Фибоначчи с порядковым номером n , составляет 2^n . Этот стремительный рост быстро выходит из-под контроля. Вычисление 20-го числа Фибоначчи требует 2^{20} (около миллиона) вызовов. Вычисление 30-го числа Фибоначчи требует уже 2^{30} (около миллиарда) вызовов, и т. д. IT-специалисты называют это **экспоненциальной сложностью** (**exponential complexity**). Задачи такого рода с большими значениями n могут оказаться чрезмерной нагрузкой даже для самых мощных компьютеров в мире. Вопросы сложности (в том числе экспоненциальной сложности) подробно изучаются в курсе информатики высшего уровня, который обычно называется «Алгоритмы». Избегайте рекурсивных программ «в стиле Фибоначчи», которые приводят к экспоненциальному «взрыву» вызовов.

5.19. Рекурсия или итерация?

В предыдущих разделах мы рассмотрели две рекурсивные функции. Их также можно реализовать с помощью простых итеративных программ. В этом разделе мы сравним оба подхода, чтобы вам было проще выбирать лучший из них в зависимости от ситуации.

- ♦ Итерация и рекурсия основаны на управляющих инструкциях. Для итерации программист использует инструкцию цикла. Для рекурсии программист использует инструкцию выбора (ветвления).
- ♦ Итерация и рекурсия основаны на многократном повторении. При итерации повторение реализуется инструкцией цикла. При рекурсии повторение реализуется функцией, которая вызывает саму себя.
- ♦ Итерация и рекурсия завершаются при определенном условии. Итерация завершается, когда перестает выполняться условие продолжения цикла. Рекурсия завершается, когда распознается базовая задача.
- ♦ Итерация с управляющей переменной (цикл со счетчиком) и рекурсия постепенно приближаются к завершению. Итерация изменяет счетчик до тех пор, пока он не примет значение, приводящее к невыполнению условия продол-

жения цикла. Рекурсия создает более простые версии исходной задачи до тех пор, пока не будет достигнута базовая задача.

- ♦ Итерация и рекурсия могут выполняться бесконечно. Бесконечный цикл возникает, если условие продолжения никогда не становится ложным. Бесконечная рекурсия возникает, если шаг рекурсии не упрощает задачу в каждом рекурсивном вызове таким образом, чтобы в итоге достичь базовой задачи.

Итеративная версия функции factorial

Давайте рассмотрим различия между итерацией и рекурсией с помощью итеративной программы, вычисляющей факториалы (илл. 5.17). В строках 22–24 используется инструкция цикла, в отличие от инструкции выбора в нашей прежней рекурсивной программе (илл. 5.15, строки 19–24). В рекурсивной программе (илл. 5.15) в строке 19 проверяется, достигнута ли базовая задача, завершающая рекурсию. В итеративной программе (илл. 5.17) в строке 22 проверяется условие продолжения цикла, и если оно не выполняется, цикл завершается. Кроме того, итеративная программа не создает упрощенные версии исходной задачи, а вместо этого выполняет инкремент счетчика до тех пор, пока не станет ложным условие продолжения цикла.

```
1 // fig05_17.cpp
2 // Итеративная версия функции factorial
3 #include <iostream>
4 #include <iomanip>
5 using namespace std;
6
7 long factorial(int number); // Прототип функции
8
9 int main() {
10     // Вычисляем факториалы чисел от 0 до 10
11     for (int counter{0}; counter <= 10; ++counter) {
12         cout << setw(7) << counter << "! = " << factorial(counter)
13             << '\n';
14     }
15 }
16
17 // Итеративная функция factorial
18 long factorial(int number) {
19     long result{1};
20
21     // Итеративное вычисление факториала
22     for (int i{number}; i >= 1; --i) {
23         result *= i;
24     }
25
26     return result;
27 }
```

```
0! = 1
1! = 1
```

```

2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
7! = 5040
8! = 40320
9! = 362880
10! = 3628800

```

Илл. 5.17 | Итеративная версия функции factorial

Недостатки рекурсии

При рекурсии многократно активируется ресурсоемкий механизм вызова функций, поэтому такая программа может занять много процессорного времени и памяти. Каждый рекурсивный вызов создает очередную копию переменных функции; в итоге объем занятой памяти может оказаться чрезмерным. Итерация обычно выполняется внутри функции, поэтому памяти требуется значительно меньше и процессорное время на многократные вызовы функций тоже не расходуется. Но почему же программисты все-таки иногда выбирают рекурсию?

Когда рекурсия предпочтительнее итерации?

Любую задачу, решаемую рекурсивно, можно решить и итеративно (без рекурсии). Рекурсию обычно выбирают для упрощения кода — в тех случаях, когда рекурсивный алгоритм получается нагляднее и проще в отладке, чем итеративный. Кроме того, рекурсию выбирают, когда для задачи уже есть очевидное рекурсивное решение, а итеративного решения нет или его трудно найти. По возможности избегайте рекурсии в программах, которые должны работать быстро. Не забывайте, что рекурсивные вызовы требуют много времени и памяти.

5.20. Lnfylun Lhqtomh Wjtz Qarcv: Qjwazkrplm xzz Xndmwwqhlz

Конечно, вы заметили, что последний пункт в списке целей главы 5, последняя фраза раздела 5.1 и заголовок раздела 5.20 — все это выглядит как тарабарщина. Но это не ошибки! В этом разделе мы продолжаем презентации готовых объектов. Вы научитесь шифровать и расшифровывать сообщения с помощью объекта, который легко создадите из нашего класса, реализующего шифр Виженера с секретным ключом¹.

В предыдущих главах вы создавали объекты класса `string` стандартной библиотеки C++ и объекты других классов, входящих в библиотеки с открытым исходным кодом. Вероятно, вы когда-нибудь будете работать и с классами, написанными

¹ «Vigenère Cipher». Был доступен: 15 сентября 2023 г. https://en.wikipedia.org/wiki/Vigenère_cipher.

вашей организацией или вашей командой для внутренних нужд или для конкретного проекта. А для этого раздела мы написали наш собственный класс `Cipher` (он находится в заголовке "`cipher.h`") и предоставили его вам. В главе 9 вы будете создавать свои собственные классы.

Криптография

Криптография существует уже тысячи лет^{1,2} и имеет большое значение в современном мире. Каждый день криптографические алгоритмы работают «за кулисами» интернета, обеспечивая конфиденциальность и безопасность ваших коммуникаций. Например, при обмене данными между большинством веб-сайтов и вашим браузером выполняется шифрование и дешифрование по протоколу HTTPS.

Шифр Цезаря

Юлий Цезарь использовал простой **шифр подстановки** (**substitution cipher**) для шифрования военных сообщений³. Его способ, известный как шифр Цезаря, заключается в замещении каждой буквы в сообщении на букву, находящуюся на три позиции впереди в алфавите. Буква А заменяется на D, В — на Е, С — на F, ... Х — на А, Y — на В и Z — на С. Например, текст:

Caesar Cipher

в зашифрованном виде будет выглядеть так:

Fdhvdu Flskhu

Зашифрованный текст называют криптограммой, или шифротекстом.

Если вам интересно поиграть с шифром Цезаря и со многими другими методами шифрования, посетите веб-сайт:

<https://cryptii.com/pipes/caesar-cipher>

который является онлайн-реализацией библиотеки `cryptii` с открытым исходным кодом:

<https://github.com/cryptii/cryptii>

Шифр Виженера

Простейшие шифры подстановки, такие как шифр Цезаря, относительно легко расшифровать. Например, буква «е» встречается в английском языке чаще всех остальных. Вы можете изучить текст, зашифрованный шифром Цезаря, и с большой вероятностью предположить, что наиболее часто встречающийся символ является буквой «е».

¹ «Cryptography». 14 мая 2020 г. https://en.wikipedia.org/wiki/Cryptography#History_of_cryptography_and_cryptanalysis.

² «History of Cryptography». 19 января 2020 г. <https://www.binance.vision/security/history-of-cryptography>.

³ «Шифр Цезаря». Был доступен: 15 сентября 2023 г. https://ru.wikipedia.org/wiki/Шифр_Цезаря.

В следующем примере мы покажем вам шифр Виженера, который является шифром подстановки с секретным ключом. Этот метод аналогичен одновременному применению 26 шифров Цезаря — по одному на каждую букву алфавита. Шифр Виженера использует буквы из исходного текста и секретный ключ для поиска в соответствующих шифрах Цезаря заменяющих символов. Здесь вы можете узнать больше о реализации этого метода:

https://en.wikipedia.org/wiki/Vigen%C3%A8re_cipher

Секретный ключ этого шифра должен состоять только из букв. Как и пароли, этот ключ не должен быть легко угадываемым. Например, мы выбрали ключ из 11 случайно выбранных символов:

`xmwu3vuyxz`

Количество символов, которые вы можете использовать в своем секретном ключе, не ограничено. Ваш адресат, который получит зашифрованный текст, должен знать тот секретный ключ, который вы использовали при шифровании¹. Поэтому вы должны сообщить ключ заранее, например, при личной встрече.

Программа с нашим классом Cipher

В программе, показанной на илл. 5.18, вы будете использовать класс `Cipher`, который реализует шифр Виженера. Заголовок `"cipher.h"` (строка 3) из папки примеров `ch05` этой главы определяет наш класс. Вам даже не нужно читать и понимать исходный код, чтобы использовать средства шифрования и дешифрования. Вы просто создаете объект класса `Cipher`, а затем вызываете его функции `encrypt` и `decrypt` для шифрования и дешифрования текста соответственно.

```

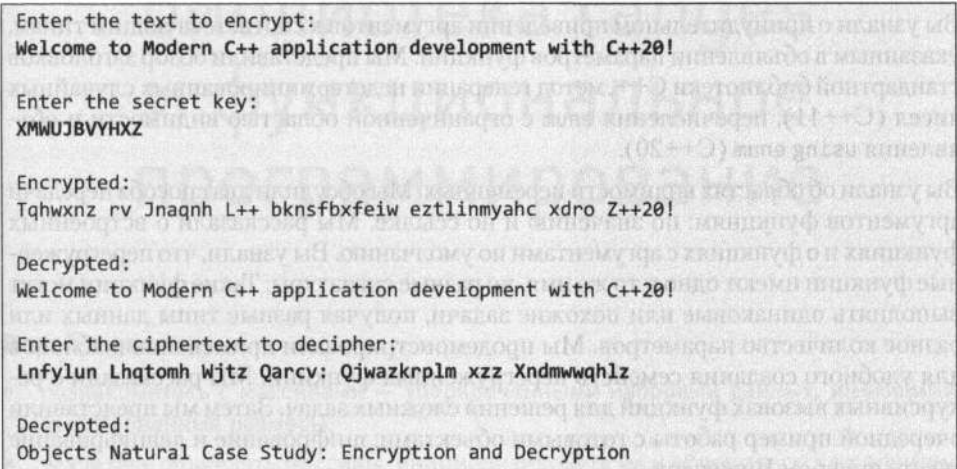
1 // fig15_18.cpp
2 // Шифрование и дешифрование текста шифром Виженера
3 #include "cipher.h"
4 #include <iostream>
5 #include <string>
6 using namespace std;
7
8 int main() {
9     string plainText;
10    cout << "Enter the text to encrypt:\n";
11    getline(cin, plainText);
12
13    string secretKey;
14    cout << "\nEnter the secret key:\n";
15    getline(cin, secretKey);
16
17    Cipher cipher;
```

¹ Есть много веб-сайтов, предлагающих восстановить зашифрованный шифром Виженера текст без оригинального секретного ключа. Мы попробовали несколько вариантов, но ни один из них не показал наш исходный текст.

```

18
19 // Шифрование plainText с помощью secretKey
20 string cipherText{cipher.encrypt(plainText, secretKey)};
21 cout << "\nEncrypted:\n " << cipherText << '\n';
22
23 // Дешифрование cipherText
24 cout << "\nDecrypted:\n "
25     << cipher.decrypt(cipherText, secretKey) << '\n';
26
27 // Дешифрование текста, введенного пользователем
28 cout << "\nEnter the ciphertext to decipher:\n";
29 getline(cin, cipherText);
30 cout << "\nDecrypted:\n "
31     << cipher.decrypt(cipherText, secretKey) << '\n';
32 }

```



```

Enter the text to encrypt:
Welcome to Modern C++ application development with C++20!

Enter the secret key:
XNMUJ3BVYHXZ

Encrypted:
Tqhwxnz rv Jnaqnh L++ bknsfbxfeiwxn eztlmnyahc xdro Z++20!

Decrypted:
Welcome to Modern C++ application development with C++20!

Enter the ciphertext to decipher:
Lnfylun Lhqtomh Wjtz Qarcv: Qjwazkrplm xzz Xndmwwqhlz

Decrypted:
Objects Natural Case Study: Encryption and Decryption

```

Илл. 5.18 | Шифрование и дешифрование текста шифром Виженера

Функции класса Cipher

Класс `Cipher` предоставляет вам две основные функции:

- ♦ Функция `encrypt` получает строки, представляющие исходный текст и секретный ключ. Используя шифр Виженера, она возвращает строку с зашифрованным текстом.
- ♦ Функция `decrypt` получает строки, представляющие зашифрованный текст и секретный ключ. Используя шифр Виженера, она выполняет обратную операцию и возвращает строку с расшифрованным текстом.

Сначала программа просит ввести текст для шифрования и секретный ключ. В строке 17 создается объект `Cipher`. В строках 20–21 введенный текст шифруется и выводится на экран. В строках 24–25 полученная криптограмма расшифровывается, чтобы показать вам исходный текст.

Теперь вы можете узнать, что написано в последнем пункте в списке целей главы 5, в последней фразе раздела 5.1 и в заголовке раздела 5.20. Это криптограммы, которые мы получили с помощью нашего класса `Cipher` и секретного ключа:

```
хмшшзввунхз
```

В строках 28–29 запрашивается зашифрованный текст, затем в строках 30–31 он расшифровывается и на экран выводится исходный текст.

5.21. Итоги

В этой главе мы рассказали о функциях, в том числе о прототипах, сигнатурах, заголовках и телах функций. Мы рассмотрели функции математической библиотеки и математические функции и константы, добавленные в C++11, C++17 и C++20.

Вы узнали о принудительном приведении аргументов к соответствующим типам, указанным в объявлении параметров функции. Мы представили обзор заголовков стандартной библиотеки C++, метод генерации недетерминированных случайных чисел (C++11), перечисления `enum` с ограниченной областью видимости и объявления `using enum` (C++20).

Вы узнали об областях видимости переменных. Мы обсудили два способа передачи аргументов функциям: по значению и по ссылке. Мы рассказали о встроенных функциях и о функциях с аргументами по умолчанию. Вы узнали, что перегруженные функции имеют одно и то же имя, но разные сигнатуры. Такие функции могут выполнять одинаковые или похожие задачи, получая разные типы данных или разное количество параметров. Мы продемонстрировали применение шаблонов для удобного создания семейств перегруженных функций. Мы рассказали о рекурсивных вызовах функций для решения сложных задач. Затем мы представили очередной пример работы с готовыми объектами: шифрование и дешифрование текста шифром Виженера.

В главе 6 вы узнаете о списках и таблицах данных в массивах и объектно-ориентированных векторах. Вы также увидите более элегантную реализацию программы для игры в кости, основанную на массиве.

Массивы, векторы, библиотека `ranges` и функциональное программирование

В этой главе вы:

- познакомитесь с массивами — упорядоченными наборами данных, имеющими фиксированные размеры;
- научитесь объявлять массивы, инициализировать их и обращаться к их элементам;
- узнаете, как инструкция `for` для диапазона значений помогает предотвращать ошибки итерации;
- увидите несколько примеров передачи массива функциям;
- отсортируете элементы массива в порядке возрастания;
- выполните быстрый поиск в отсортированном массиве с помощью функции `binary_search`;
- научитесь объявлять и использовать многомерные массивы;
- узнаете о возможностях функционального программирования с библиотекой `ranges` (C++20);
- продолжите работать с готовыми объектами, на этот раз с векторами (`vector`) — упорядоченными наборами данных, динамически меняющими свои размеры.

6.1. Введение

В этой главе мы представим вам **структуры данных (data structures)**, то есть упорядоченные наборы данных, состоящие из однотипных элементов. Структуры данных, реализованные современными средствами стандартной библиотеки C++, называются **контейнерами (containers)**. Мы рассмотрим два вида контейнеров:

- ♦ **массив (array)**¹ — контейнер фиксированного размера, основанный на шаблоне класса `array` (он появился в C++11);
- ♦ **вектор (vector)** — контейнер переменного размера, основанный на шаблоне класса `vector`.

В отличие от массива, размер вектора (количество его элементов) может увеличиваться и уменьшаться во время выполнения программы. Чтобы использовать в программе эти контейнеры, надо включить в нее заголовки `<array>` и `<vector>` соответственно.

Прежде всего мы расскажем, как объявлять, создавать и инициализировать массивы, и приведем примеры операций с ними. Затем мы займемся сортировкой элементов массива в порядке возрастания и поиском данных в отсортированном массиве. Вы увидите, что попытка доступа к элементу данных за пределами массива или вектора (то есть к несуществующему элементу) вызывает исключение — ошибку во время выполнения программы. Чтобы программа корректно реагировала на ошибку, мы обработаем это исключение. В главе 12 исключения рассматриваются подробнее.

На языке C++ (как и на многих других современных языках) можно программировать «в функциональном стиле». Это помогает писать более лаконичный код, снижать вероятность ошибок, делать код нагляднее и удобнее для отладки и модификации. Такой код проще распараллеливать для многоядерных процессоров. Мы покажем примеры функционального программирования с библиотекой `ranges`, введенной в C++20. Наконец, мы продолжим презентацию готовых объектов, которые на этот раз будем создавать на основе шаблона класса `vector` стандартной библиотеки C++. Таким образом, после изучения этой главы вы будете уметь работать с двумя важнейшими структурами данных, имеющими много общего: с массивами (`array`) и векторами (`vector`).

Полностью квалифицированные имена стандартной библиотеки

Начиная с раздела 2.6, мы включали директиву `using namespace std;` в каждую программу, поэтому нам не нужно было добавлять квалификатор `std::` к каждому имени стандартной библиотеки C++. Но в больших программах директива `using` может привести к скрытым ошибкам, которые трудно обнаружить. Поэтому

¹ В главе 7 вы увидите, что C++ также поддерживает традиционные массивы в стиле языка C, которые не являются контейнерами. Но в современных программах на C++ вместо них необходимо использовать массивы, основанные на шаблоне класса `array` из C++11.

в дальнейшем мы будем **полностью квалифицировать** (*fully qualify*) большинство идентификаторов стандартной библиотеки C++, то есть записывать имена с квалификатором `std::`.

6.2. Массивы

Элементы массива (хранящиеся в нем элементы данных) расположены в памяти последовательно. На следующей диаграмме показан целочисленный массив с именем `c`, содержащий пять элементов:



Для обращения к элементу массива надо указать имя массива, а затем порядковый номер элемента в квадратных скобках (`[]`). Порядковый номер элемента программисты называют **индексом** (*index*). Первый элемент любого массива имеет индекс `0` (ноль). Таким образом, массив `c` содержит элементы от `c[0]` до `c[4]`. Значение `c[0]` равно `-45`, значение `c[2]` равно `0`, а значение `c[4]` равно `1543`.

Последний индекс (в данном случае `4`) всегда на единицу меньше, чем количество элементов массива (в данном случае `5`). Каждый массив «знает» свой размер, который вы можете получить с помощью функции `size`:

```
c.size()
```

Индекс должен быть целым числом или целочисленным выражением. Квадратные скобки, содержащие индекс, — это **оператор индекса** (*subscript operator*). Он имеет тот же приоритет, что и круглые скобки в вызове функции. Результатом операции с оператором индекса является значение *lvalue* — его можно использовать в левой части присваивания, так же как и имя переменной. Например, следующая инструкция присваивает элементу `c[4]` новое значение `87`:

```
c[4] = 87;
```

Полная таблица приоритетов операторов приведена в приложении А.

6.3. Объявление массива

Чтобы объявить массив, надо указать его тип данных, размер (количество элементов этого типа) и имя:

```
std::array<тип, размерМассива> имяМассива;
```

Запись в угловых скобках `<тип, размерМассива>` показывает, что `array` — это **шаблон класса** (*class template*). Подобно шаблону функции, шаблон класса

используется компилятором для создания множества **специализаций шаблона класса (class template specializations)**, работающих с разными типами данных. Например, специализациями одного и того же шаблона могут быть: массив из 5 элементов типа `int`; массив из 7 элементов типа `double`; массив из 100 объектов пользовательского типа `Employee` (каждый из которых хранит сведения о сотруднике предприятия). В главе 9 вы узнаете, как создавать пользовательские типы данных. Компилятор резервирует необходимый объем памяти в зависимости от типа и размера массива. Например, следующее объявление выделит память для массива с именем `c`, состоящего из 5 целых чисел:

```
std::array<int, 5> c; // c - это массив из 5 целых чисел
```

6.4. Инициализация массива в цикле

В программе, показанной на илл. 6.1, объявляется массив с именем `values` (строка 8), состоящий из пяти целых чисел. В строке 5 включен заголовок `<array>`, содержащий определение шаблона класса `array`.

```
1 // fig06_01.cpp
2 // Инициализация массива и вывод на экран
3 #include <fmt/format.h> // Или в C++20: #include <format>
4 #include <iostream>
5 #include <array>
6
7 int main() {
8     std::array<int, 5> values; // values - это массив из 5 целых чисел
9
10    // Инициализируем все элементы массива нулями
11    for (size_t i{0}; i < values.size(); ++i) {
12        values[i] = 0; // Присваиваем значение 0 элементу с индексом i
13    }
14
15    std::cout << fmt::format("{:>7}{:>10}\n", "Element", "Value");
16
17    // Выводим значения всех элементов массива
18    for (size_t i{0}; i < values.size(); ++i) {
19        std::cout << fmt::format("{:>7}{:>10}\n", i, values[i]);
20    }
21
22    std::cout << fmt::format("\n{:>7}{:>10}\n", "Element", "Value");
23
24    // Обращаемся к элементам через функцию at
25    for (size_t i{0}; i < values.size(); ++i) {
26        std::cout << fmt::format("{:>7}{:>10}\n", i, values.at(i));
27    }
28
29    // Пробуем обратиться к несуществующему элементу с индексом 10
30    values.at(10); // Функция at выдает исключение
31 }
```

Element	Value
0	0
1	0
2	0
3	0
4	0

Element	Value
0	0
1	0
2	0
3	0
4	0

terminate called after throwing an instance of
 what(): array::at: __n (which is 10) >= _Nm (which is 5)
 Aborted

Илл. 6.1 | Инициализация массива и вывод на экран

Присваивание значений всем элементам

Строки 11–13 — это цикл `for`, в котором каждому элементу массива присваивается значение `0`. Элементы массивов, как и обычные нестатические локальные переменные, не инициализируются компилятором автоматически (неявно). Исключением из этого правила являются статические (`static`) массивы, которые компилятор автоматически (неявно) инициализирует нулями. Выполняя в цикле последовательную обработку элементов массива, в том числе для их инициализации, программист должен убедиться, что условие продолжения цикла исключает опасность обращения к несуществующим элементам за пределами массива. В разделе 6.6 мы представим вам цикл `for` для диапазона значений, который не имеет условия продолжения и поэтому позволяет выполнять обработку всех элементов массива более безопасно.

Тип `size_t`

В строках 11, 18 и 25 управляющие переменные каждого цикла объявляются с типом `size_t`. Согласно стандарту C++, `size_t` — это целочисленный тип без знака, предназначенный для обозначения размеров любых объектов¹. Используйте переменные этого типа, чтобы обозначать размеры и индексы массивов. Тип `size_t` находится в пространстве имен `std` и определен в заголовке `<cstdint>`, который автоматически включается другими заголовками стандартной библиотеки, использующими `size_t`. Если компилятор «не знает» тип `size_t` и сообщает об ошибке, добавьте в вашу программу директиву `#include <cstdint>`.

¹ Стандарт C++, «17.2.4. Sizes, Alignments, and Offsets». Был доступен: 15 сентября 2023 г. <https://timsong-cpp.github.io/cppwp/n4861/support.types#layout-3>.

Отображение элементов массива

Первая инструкция вывода (строка 15) отображает заголовки столбцов таблицы. Затем в цикле `for` выводятся значения элементов массива (строки 18–20). Чтобы отобразить данные в виде таблицы, мы включили в инструкции вывода появившиеся в C++20 средства форматирования текста, упомянутые в разделе 4.14.

Как избежать ошибок: проверка границ массива

Если программист обращается к элементу массива, используя квадратные скобки (как в строках 11–13 и 18–20), компилятор C++ не выполняет **проверку границ (bounds checking)** массива, которая могла бы предотвратить обращение к несуществующему элементу. Из-за этого выполняемые программы могут «вылетать» за границы массивов без предупреждения. В отличие от квадратных скобок, функция `at` из шаблона класса `array` выполняет проверку границ массива. В строках 25–27 демонстрируется обращение к элементам через функцию `at`. Вы также можете присваивать значения элементам массива, используя функцию `at` в левой части инструкции присваивания:

```
values.at(0) = 10;
```

В строке 30 мы пробуем обратиться к несуществующему элементу массива. Когда функция `at`, проверив полученный индекс, обнаруживает выход за границы массива, она генерирует **ошибку выполнения (runtime error)**, которая называется **исключением (exception)**. Поэтому в строке 30 программа, скомпилированная с помощью GNU g++ и запущенная в Linux, завершается с сообщением об ошибке:

```
terminate called after throwing an instance of
  what(): array::at: __n (which is 10) >= _Nm (which is 5)
Aborted
```

Это сообщение означает, что функция `at (array::at)` выяснила, что переменная `__n` (со значением 10) больше или равна переменной `_Nm` (со значением 5). В GNU C++ функция `at` реализована так, что ее переменная `__n` представляет индекс элемента, а ее переменная `_Nm` представляет размер массива. То есть функция проверила границы массива, обнаружила недопустимый индекс и выдала исключение. В разделе 6.15 мы покажем, как обработать исключение для корректной реакции на ошибку во время выполнения программы. В главе 12 обработка исключений рассматривается подробнее.

Программы, в которых возможны инструкции чтения и записи значений вне границ массивов, часто несут в себе опасность. Попытка чтения несуществующего элемента массива может привести к сбою программы или даже к подмене данных при внешне нормальной работе программы. Попытка записи в несуществующий элемент (переполнение буфера¹) может привести к повреждению данных в па-

¹ Дополнительные сведения о переполнении буфера: https://ru.wikipedia.org/wiki/Переполнение_буфера.

мяти и к краху программы. В некоторых случаях переполнением буфера могут пользоваться злоумышленники, заменяя часть исполняемого кода программы вредоносным кодом.

6.5. Инициализация массива со списком инициализаторов

Все элементы массива можно инициализировать непосредственно в инструкции, которая объявляет массив. Для этого надо указать после имени массива разделенный запятыми список **инициализаторов (initializers)** в фигурных скобках. В программе, показанной на илл. 6.2, есть два **списка инициализаторов (initializer list)**: для массива с пятью элементами `int` (строка 8) и для массива с четырьмя элементами `double` (строка 18). Значения обоих массивов выводятся на экран.

- ♦ В строке 8 явно указаны тип элементов (`int`) и размер массива (5).
- ♦ В строке 18 компилятор автоматически определяет тип элементов (`double`) и размер массива (4) по списку инициализаторов. Этот механизм, появившийся в C++17, называется **дедуктивным определением аргументов шаблона класса (class template argument deduction, или сокращенно CTAD)**¹. Когда вы заранее знаете начальные значения элементов контейнера, CTAD может упростить ваш код.

```
1 // fig06_02.cpp
2 // Инициализация массива при объявлении
3 #include <fmt/format.h>
4 #include <iostream>
5 #include <array>
6
7 int main() {
8     std::array<int, 5> values{32, 27, 64, 18, 95}; // Скобочная инициализация
9
10    // Выводим на экран значение каждого элемента
11    for (size_t i{0}; i < values.size(); ++i) {
12        std::cout << fmt::format("{} ", values.at(i));
13    }
14
15    std::cout << "\n\n";
16
17    // Объявляем массив в краткой форме записи, используя CTAD
18    std::array values2{1.1, 2.2, 3.3, 4.4};
19
20    // Выводим на экран значение каждого элемента
21    for (size_t i{0}; i < values.size(); ++i) {
```

Окончание ⇨

¹ «Class Template Argument Deduction (CTAD)». Был доступен: 15 сентября 2023 г. https://en.cppreference.com/w/cpp/language/class_template_argument_deduction.

```

22     std::cout << fmt::format("{} ", values.at(i));
23 }
24
25     std::cout << "\n";
26 }

```

```

32 27 64 18 95
1.1 2.2 3.3 4.4

```

Илл. 6.2 | Инициализация массива при объявлении

Инициализация по умолчанию

Если количество инициализаторов в списке меньше, чем количество элементов массива, все остальные элементы **инициализируются значением по умолчанию (value initialized)**: для основных числовых типов — 0, для типа `bool` — `false`, а объекты инициализируются в соответствии с определениями их классов (мы расскажем об этом в главе 9). Например, следующее объявление инициализирует все элементы массива `int` нулями, так как в списке нет ни одного инициализатора (ноль), что меньше количества элементов массива:

```
std::array<int, 5> values{}; // Инициализация значением 0
```

Этот удобный метод можно использовать и для повторной инициализации элементов существующего массива во время выполнения программы:

```
values = {}; // Обнуление всех элементов массива
```

Чрезмерно длинный список инициализаторов

Если при объявлении массива явно указан его размер, количество инициализаторов в списке должно быть не больше этого размера. Например, следующее объявление вызовет ошибку компиляции, так как в нем перечислены шесть инициализаторов для массива из пяти элементов:

```
std::array<int, 5> values{32, 27, 64, 18, 95, 14};
```

6.6. Цикл `for` для диапазона значений (C++11) и цикл `for` для диапазона значений с инициализатором (C++20)

В большинстве случаев при обработке массива должны обрабатываться все его элементы, от начала до конца. **Инструкция `for` для диапазона значений (range-based for statement)**, появившаяся в C++11, позволяет делать это без счетчика. Как правило, для обработки массивов предпочтителен именно этот вид цикла, поскольку он не позволяет программам «вылетать» за границы массивов. В конце этого раздела мы сравним два вида цикла: `for` со счетчиком (с управляющей переменной) и `for` для диапазона значений. В программе, показанной на илл. 6.3,

инструкции for для диапазона значений используются для вывода на экран элементов массива (строки 12–14, 23–25) и для математической обработки массива (строки 17–19).

```
1 // fig06_03.cpp
2 // Цикл for для диапазона значений
3 #include <fmt/format.h>
4 #include <iostream>
5 #include <array>
6
7 int main() {
8     std::array items{1, 2, 3, 4, 5}; // Автоопределение типа: array<int, 5>
9
10    // Выводим на экран массив до изменения
11    std::cout << "items before modification: ";
12    for (const int& item : items) { // item - константная ссылка на int
13        std::cout << fmt::format("{} ", item);
14    }
15
16    // Удваиваем все элементы массива
17    for (int& item : items) { // item - ссылка на элемент типа int
18        item *= 2;
19    }
20
21    // Выводим на экран массив после изменения
22    std::cout << "\nitems after modification: ";
23    for (const int& item : items) {
24        std::cout << fmt::format("{} ", item);
25    }
26
27    // Суммируем элементы в цикле for для диапазона значений
28    std::cout << "\n\ncalculating a running total of items' values:\n";
29    for (int runningTotal{0}; const int& item : items) {
30        runningTotal += item;
31        std::cout << fmt::format("item: {}; running total: {}\n",
32            item, runningTotal);
33    }
34 }
```

```
items before modification: 1 2 3 4 5
items after modification: 2 4 6 8 10

calculating a running total of items
item: 2; running total: 2
item: 4; running total: 6
item: 6; running total: 12
item: 8; running total: 20
item: 10; running total: 30
```

Отображение элементов массива в цикле `for` для диапазона значений

Цикл `for` для диапазона значений упрощает код, выполняющий последовательное обращение ко всем элементам массива. Строку 12 можно прочитать так: «Для каждого элемента `item` в массиве `items` следует выполнить...». В каждой итерации цикла эта инструкция присваивает переменной `item` значение очередного элемента массива `items`, а затем выполняет тело цикла. Обратите внимание, что в каждой итерации цикла переменная `item` ссылается на значение (а не на индекс!) элемента массива. Эта переменная, объявленная в заголовке цикла перед двоеточием, называется **переменной диапазона** (*range variable*). После двоеточия в заголовке цикла указано имя массива¹. В нашем примере переменная `item` объявлена как константная ссылка (с квалификатором `const`). Напомним, что ссылка — это псевдоним для другой переменной в памяти, в данном случае для одного из элементов массива.

Если переменная диапазона является константной ссылкой, цикл выполняется быстрее. Ссылка не позволяет циклу копировать каждый элемент массива в переменную диапазона. Эта экономия времени особенно важна при работе с большими объектами.

Обработка элементов массива в цикле `for` для диапазона значений

В строках 17–19 цикл `for` для диапазона значений используется для умножения на 2 каждого элемента массива `items`. В строке 17 переменная `item` объявлена как ссылка на значение типа `int` (`int&`). Эта ссылка не константная (без квалификатора `const`), поэтому при каждом изменении переменной `item` меняется значение соответствующего элемента массива.

Цикл `for` для диапазона значений с инициализатором (C++20)

В C++20 появилась еще одна разновидность цикла `for` для диапазона значений: с инициализатором в заголовке цикла. Как и в инструкциях `if` и `switch` с инициализаторами, переменные, определенные в заголовке цикла `for` для диапазона значений, существуют только во время выполнения цикла. В строке 29 переменная `runningTotal` инициализируется значением 0, затем в строках 29–33 суммируются элементы массива `items`. После завершения цикла переменная `runningTotal` выходит из области видимости, то есть перестает существовать.

Обращение к элементам массива без индексов

Во многих случаях традиционный цикл `for` со счетчиком можно заменить циклом `for` для диапазона значений. Например, для суммирования всех целых чисел в массиве нужен доступ только к значениям элементов, а их индексные позиции в массиве не важны.

¹ Инструкцию `for` для диапазона значений можно использовать с большинством контейнеров стандартной библиотеки C++, которые обсуждаются в главе 13.

Два вида итерации: внешняя и внутренняя

В рассмотренной программе строки 12–14 можно было бы заменить функционально эквивалентной инструкцией цикла `for` со счетчиком:

```
for (int counter{0}; counter < items.size(); ++counter) {  
    std::cout << fmt::format("{} ", items[counter]);  
}
```

Но этот стиль итерации, известный как **внешняя итерация (external iteration)**, не вполне надежен. Дело в том, что для правильной обработки массива ваш код должен многократно изменять значение управляющей переменной (счетчика). Каждый раз, когда вы пишете подобную инструкцию, вы рискуете допустить ошибку, например:

- ✦ неправильно инициализировать управляющую переменную;
- ✦ неправильно задать условие продолжения цикла;
- ✦ неправильно задать инкремент управляющей переменной.

Каждая из этих ошибок может привести к попытке доступа к несуществующим элементам массива за пределами его границ.

Инструкция `for` для диапазона значений использует **внутреннюю итерацию (internal iteration)**, скрывая от вас все детали. Вы просто указываете, какой массив нужно обработать. Инструкция знает, как получить каждое значение из массива, и завершит итерацию, когда значения закончатся.

6.7. Заполнение массива и спецификатор constexpr

На илл. 6.4 показано, как массив `values` заполняется четными целыми числами 2, 4, 6, 8 и 10 (строки 13–15), которые затем выводятся на экран (строки 18–20). Каждое число генерируется в строке 14 путем умножения управляющей переменной `i` на 2 и прибавления 2.

```
1 // fig06_04.cpp  
2 // Заполнение массива четными числами от 2 до 10  
3 #include <fmt/format.h>  
4 #include <iostream>  
5 #include <array>  
6  
7 int main() {  
8     // Константа подходит для указания размера массива  
9     constexpr size_t arraySize{5}; // Надо инициализировать в объявлении  
10  
11     std::array<int, arraySize> values{}; // В массиве values 5 элементов  
12  
13     for (size_t i{0}; i < values.size(); ++i) { // Заполняем массив  
14         values.at(i) = 2 + 2 * i;  
15     }
```

```

16
17 // Выводим значения элементов массива в виде таблицы
18 for (const int& value : values) {
19     std::cout << fmt::format("{} ", value);
20 }
21
22 std::cout << '\n';
23 }

```

```
2 4 6 8 10
```

Илл. 6.4 | Заполнение массива четными числами от 2 до 10

Константы

В главе 5 вы познакомились с квалификатором `const`, запрещающим изменять значение переменной после ее инициализации. В C++11 появился спецификатор `constexpr`. Он означает, что компилятор должен вычислить значение переменной при компиляции и затем сделать эту переменную константой. Это повышает быстродействие, поскольку в выполняемой программе не тратится время на инициализацию. Переменная, объявленная со спецификатором `constexpr`, — это тоже константа, но с очень важным для компилятора отличием: она должна быть инициализирована во время компиляции. Обычные `const`-переменные вы можете инициализировать во время выполнения.

В строке 9 спецификатор `constexpr` используется для объявления константы `arraySize` со значением 5. Переменная, объявленная как `constexpr` или `const`, должна быть инициализирована при ее объявлении; иначе возникнет ошибка компиляции. Например, ошибку компиляции вызовет попытка изменить размер массива после его инициализации¹:

```
arraySize = 7;
```

Если в определении массива вы указываете его размер в виде константы (например, `arraySize` вместо литерала 5), ваш код становится понятнее и проще для последующей модификации. Этот метод устраняет **магические числа** (**magic numbers**) — встречающиеся в коде числовые литералы, смысл которых трудно понять. Заменяя литералы константами, вы можете тем самым пояснять их смысл, давая им понятные имена.

Вызовы `constexpr`-функций вычисляются во время компиляции

Одна из современных тенденций языка C++ заключается в том, чтобы выполнять как можно больше полезных вычислений на этапе компиляции. Это нужно и для надежной проверки правильности типов данных в исходном коде, и для ускорения работы программ. Каждый новый стандарт C++, начиная с C++11, расширял

¹ Некоторые компиляторы в сообщениях об ошибках называют переменные-константы основных типов «константными объектами» («`const objects`»). Согласно стандарту C++, объект — это «область памяти» («`region of storage`»). Подобно объектам классов, переменные основных типов тоже занимают место в памяти, и в этом смысле о них часто говорят как об объектах.

возможности спецификатора `constexpr`. В последующих главах вы увидите, что его теперь можно применять даже к функциям. Если в исходном коде есть вызов `constexpr`-функции и все передаваемые ей аргументы тоже обозначены как `constexpr`, компилятор может заранее вычислить результат этого вызова, получив константу. Это упрощает исполняемый код и повышает быстродействие. В C++20, C++17 и C++14 многие функции стандартной библиотеки C++ объявлены как `constexpr`.

Подробную информацию о спецификаторе `constexpr` и о способах его применения вы найдете здесь:

<https://en.cppreference.com/w/cpp/language/constexpr>

В главе 15 мы расскажем о метапрограммировании на основе шаблонов, в результате которого часть вычислений переносится с этапа выполнения программы на этап компиляции.

6.8. Суммирование элементов массива

Часто элементы массива представляют собой исходные данные для вычислений. Например, если в массив занесены экзаменационные оценки, преподаватель может захотеть суммировать их, а затем вычислить среднюю арифметическую оценку по классу. Преобразование множества значений в одно результирующее значение называется **сверткой** (**reduction**). Это одна из ключевых операций в функциональном программировании, мы обсудим ее в разделе 6.14. На илл. 6.5 показан пример суммирования элементов массива в цикле `for` для диапазона значений (строки 12–14). В разделе 6.14 мы выполним это же вычисление с помощью функции `accumulate` стандартной библиотеки C++.

```
1 // fig06_05.cpp
2 // Суммирование элементов массива
3 #include <fmt/format.h>
4 #include <iostream>
5 #include <array>
6
7 int main() {
8     std::array items{10, 20, 30, 40}; // Автоопределение типа: array<int, 4>
9     int total{0};
10
11     // Суммируем элементы
12     for (const int& item : items) {
13         total += item;
14     }
15
16     std::cout << fmt::format("Total of array elements: {}\n", total);
17 }
```

```
Total of array elements: 100
```


6.9. Визуализация данных массива на простой диаграмме

Многие программы отображают данные графически. Например, множество чисел можно показать на гистограмме (столбчатой диаграмме), высота столбцов которой пропорциональна величине чисел. Один из самых простых способов визуализации чисел — вывести на экран диаграмму, отображающую каждое число в виде ряда звездочек (*).

Преподаватель может визуализировать результаты экзамена, распределив все оценки по нескольким категориям, или диапазонам. Предположим, что оценки учеников таковы: 87, 68, 94, 100, 83, 78, 85, 91, 76 и 87. Только один ученик получил максимальную оценку 100, два ученика — оценки в диапазоне от 90 до 99, четыре — от 80 до 89, два — от 70 до 79, один — от 60 до 69 и ни одного — ниже 60. Наша следующая программа (илл. 6.6) хранит эти данные в массиве `frequencies`, состоящем из 11 элементов, каждый из которых соответствует своему диапазону. Например, элемент с индексом 0 содержит количество оценок в диапазоне от 0 до 9, элемент с индексом 7 — количество оценок в диапазоне от 70 до 79, а элемент с индексом 10 — количество оценок 100. В разделах 6.10 и 6.11 мы покажем примеры кода, который вычисляет частотное распределение в наборе данных (то есть подсчитывает, как часто случалось каждое из нескольких возможных событий). А массив `frequencies` из примера кода в этом разделе мы инициализировали готовыми данными о частотном распределении.

Программа (илл. 6.6) считывает элементы массива `frequencies` и отображает их в виде диаграммы. Каждая строка вывода начинается с заголовка, за которым следует ряд звездочек, обозначающий количество оценок в этом диапазоне. Программа работает так:

- ♦ В строке 8 массив `frequencies` объявлен со спецификатором `constexpr`, потому что программа не должна изменять элементы массива, известные во время компиляции.
- ♦ Заголовок каждой строки диаграммы (например, 70-79:) формируется в строках 15–21 на основе значения переменной `i`. В строках 16–17 спецификатор формата `%02d` указывает, что целое число (`d`) должно быть отформатировано в поле из 2 знаков, причем с добавлением начальной цифры 0 в том случае, если оно состоит менее чем из двух цифр. В строке 20 спецификатор формата `%5d` указывает, что число должно быть выровнено по правому краю (`>`) в поле шириной 5 знаков.
- ♦ Вложенный цикл `for` (строки 26–28) выводит звездочки в текущей строке. Условие продолжения в строке 26 (`stars < frequency`) задано так, что количество итераций этого цикла равно `frequency`. Следовательно, значение каждого из элементов массива `frequencies` определяет количество звездочек в соответствующей строке диаграммы. Элементы массива с индексами от 0 до 5 содержат нули, потому что никто не получил оценку ниже 60 бал-

лов. Таким образом, в первых шести строках диаграммы звездочки не отображаются.

```

1 // fig06_06.cpp
2 // Частотное распределение результатов экзамена на простой диаграмме
3 #include <fmt/format.h>
4 #include <iostream>
5 #include <array>
6
7 int main() {
8     constexpr std::array frequencies{0, 0, 0, 0, 0, 0, 1, 2, 4, 2, 1};
9
10    std::cout << "Grade distribution:\n";
11
12    // Для каждого элемента frequencies выводим строку диаграммы
13    for (int i{0}; const int& frequency : frequencies) {
14        // Выводим заголовки диаграммы ("00-09:", ..., "90-99:", "100:")
15        if (i < 10) {
16            std::cout << fmt::format("{}:02d}-{:02d}: ",
17                                     i * 10, (i * 10) + 9);
18        }
19        else {
20            std::cout << fmt::format("{}:>5d}: ", 100);
21        }
22
23        ++i;
24
25        // Выводим ряд звездочек
26        for (int stars{0}; stars < frequency; ++stars) {
27            std::cout << '*';
28        }
29
30        std::cout << '\n'; // Перенос строки - подготовка к следующей строке
31    }
32 }

```

Grade distribution:

00-09:

10-19:

20-29:

30-39:

40-49:

50-59:

60-69: *

70-79: **

80-89: ****

90-99: **

100: *

Илл. 6.6 | Частотное распределение результатов экзамена на простой диаграмме

6.10. Использование элементов массива в качестве счетчиков

Иногда программы используют переменные-счетчики для обобщения данных, например результатов опроса. Когда мы моделировали 60 миллионов бросков кубика (см. илл. 5.3), у нас было шесть отдельных счетчиков, каждый из которых увеличивался при выпадении соответствующей грани кубика. На илл. 6.7 показано аналогичное моделирование, но на этот раз мы сделали счетчики элементами массива.

```

1 // fig06_07.cpp
2 // Моделирование бросков кубика с помощью массива (вместо switch)
3 #include <fmt/format.h> // В C++20: #include <format>
4 #include <iostream>
5 #include <array>
6 #include <random>
7
8 int main() {
9     // Инициализируем генератор случайных чисел
10    std::random_device rd; // Инициализатор default_random_engine
11    std::default_random_engine engine{rd()}; // rd() инициализирует генератор
12    std::uniform_int_distribution randomDie{1, 6};
13
14    constexpr size_t arraySize{7}; // Игнорируем элемент с индексом 0
15    std::array<int, arraySize> frequency{}; // Инициализируем массив нулями
16
17    // Бросаем кубик 60 миллионов раз; индекс массива - случайное число
18    for (int roll{1}; roll <= 60'000'000; ++roll) {
19        ++frequency.at(randomDie(engine));
20    }
21
22    std::cout << fmt::format("{}{:>13}\n", "Face", "Frequency");
23
24    // Выводим все элементы массива
25    for (size_t face{1}; face < frequency.size(); ++face) {
26        std::cout << fmt::format("{}{:>4}{{:>13}\n", face, frequency.at(face));
27    }
28 }
```

Face	Frequency
1	9997901
2	9999110
3	10001172
4	10003619
5	9997606
6	10000592

Илл. 6.7 | Моделирование бросков кубика с помощью массива (вместо switch)

В этой программе результаты бросков кубика подсчитываются в массиве `frequency` (строка 15). Инструкция в строке 19 заменяет инструкцию `switch` из строк 23–45

прежней версии программы (илл. 5.3). В каждом броске в качестве индекса выбирается случайное число, определяющее индекс того элемента массива `frequency`, который надо изменить. Оператор «точка» (.) в строке 19 имеет более высокий приоритет, чем оператор ++. Таким образом, эта инструкция выбирает случайный элемент массива и увеличивает его значение на 1.

Вызов `randomDie(engine)` возвращает случайное число от 1 до 6, поэтому массив `frequency` должен хранить не менее шести элементов. Мы решили сделать массив из семи элементов и игнорировать элемент с нулевым индексом, чтобы сделать код более понятным, ведь при выпадении числа 1 обращение к элементу `frequency.at(1)` выглядит более наглядно, чем обращение к `frequency.at(0)`. Каждое выпавшее число используется непосредственно в качестве индекса массива `frequency`. Мы также заменили строки 49–54 из прежней версии программы (илл. 5.3): теперь результаты выводятся с помощью цикла (илл. 6.7, строки 25–27). Чтобы пропустить при выводе элемент с индексом 0, счетчик этого цикла инициализирован значением 1.

6.11. Использование массива для подсчета результатов опроса

В нашем следующем примере массивы используются для обобщения данных, собранных в ходе опроса. Рассмотрим следующую постановку задачи:

Двадцати студентам предложили оценить качество еды в студенческой столовой по шкале от 1 до 5, где 1 — «ужасно», а 5 — «превосходно». Поместите 20 ответов в массив чисел типа `int` и вычислите частотное распределение ответов.

Этот распространенный вид задач сводится к обработке массива (илл. 6.8). Мы хотим подсчитать количество каждого из пяти возможных значений (от 1 до 5). Массив `responses` (строки 9–10) содержит 20 элементов, инициализированных ответами студентов. Он объявлен как `constexpr`, потому что его элементы не должны изменяться и известны во время компиляции. Массив `frequency` (строка 18) содержит 6 элементов, он нужен для подсчета количества ответов. Каждый элемент массива `frequency` инициализируется числом 0 и затем служит счетчиком для одного из возможных значений. Как и в предыдущей программе (илл. 6.7), элемент с индексом 0 игнорируется.

Первая инструкция `for` (строки 18–20) получает в каждой итерации цикла очередной ответ студента (число от 1 до 5) из массива `responses` и увеличивает один из счетчиков: от `frequency.at(1)` до `frequency.at(5)`. Ключевая инструкция — это строка 19, в которой выбор инкрементируемого счетчика зависит от значения `responses.at(response)`. Независимо от количества ответов, программе нужен для подсчета результатов массив только из шести элементов (элемент с индексом 0 игнорируется), поскольку все возможные ответы находятся в диапазоне от 1 до 5, а массив из шести элементов имеет индексы от 0 до 5.

```

1 // fig06_08.cpp
2 // Программа для подсчета результатов опроса
3 #include <fmt/format.h> // В C++20 - #include <format>
4 #include <iostream>
5 #include <array>
6
7 int main() {
8     // Результаты опроса уже занесены в массив responses
9     constexpr std::array responses{
10         1, 2, 5, 4, 3, 5, 2, 1, 3, 1, 4, 3, 3, 3, 2, 3, 3, 2, 2, 5};
11
12     // Инициализируем массив frequency нулями
13     constexpr size_t frequencySize{6}; // Размер массива
14     std::array<int, frequencySize> frequency{};
15
16     // Используем число из массива responses в качестве
17     // индекса массива frequency для инкремента соответствующего счетчика
18     for (const int& response : responses) {
19         ++frequency.at(response);
20     }
21
22     std::cout << fmt::format("{}{:>12}\n", "Rating", "Frequency");
23
24     // Выводим результаты на экран
25     for (size_t rating{1}; rating < frequency.size(); ++rating) {
26         std::cout << fmt::format("{}{:>6}{{:>12}\n",
27             rating, frequency.at(rating));
28     }
29 }

```

Rating	Frequency
1	3
2	5
3	7
4	2
5	3

Илл. 6.8 | Программа подсчета результатов опроса

6.12. Сортировка массива и поиск в массиве

В этом разделе мы продемонстрируем две функции стандартной библиотеки C++. Функция `sort`¹ упорядочит элементы нашего массива в порядке возрастания, а функция `binary_search` выполнит в нем поиск заданных элементов.

Сортировка данных (data sorting), то есть их размещение в порядке возрастания или убывания, относится к важнейшим задачам программирования. Почти каждой организации нужно сортировать какие-либо данные, причем во многих

¹ Стандарт C++ указывает, что сортировка использует алгоритм $O(n \log n)$, но не уточняет, какой именно.

случаях их объемы очень велики. Сортировка — это интереснейшая проблема, для решения которой математики и программисты выполнили много сложных исследовательских работ.

При обработке массива часто требуется выяснить, совпадает ли значение какого-либо его элемента с **ключевым значением (key value)**. Процесс сверки элементов массива с ключевым значением называется **поиском (searching)**.

Функции `sort` и `binary_search`

В нашей следующей программе (илл. 6.9) создается неотсортированный массив `colors` из семи объектов типа `string` (строки 13–14). Объявление `using` в строке 10 позволяет инициализировать этот массив **объектными литералами (object literals)** типа `string`. Такой литерал обозначается символом `s` после закрывающей кавычки. Например, если вы напишете `"red"s`, символ `s` сообщит компилятору, что этот литерал является объектом класса `std::string`. По инициализаторам в строках 13–14 компилятор определяет, что элементы этого массива должны иметь тип `std::string`. Цикл в строках 18–20 выводит на экран содержимое массива `colors`. Обратите внимание, что переменная диапазона объявлена как константная ссылка:

```
const std::string&
```

Благодаря ссылке, в этом цикле не выполняется поочередное копирование каждого объекта типа `string` в переменную `color`. В программах, обрабатывающих большое количество объектов типа `string`, такое копирование существенно снижает быстродействие.

```
1 // fig06_09.cpp
2 // Сортировка массива и поиск в массиве
3 #include <array>
4 #include <algorithm> // Содержит функции sort и binary_search
5 #include <fmt/format.h>
6 #include <iostream>
7 #include <string>
8
9 int main() {
10     using namespace std::string_literals; // Строковые объектные литералы
11
12     // Компилятор автоматически определяет, что colors - это array<string, 7>
13     std::array colors{"red"s, "orange"s, "yellow"s,
14                     "green"s, "blue"s, "indigo"s, "violet"s};
15
16     // Выводим на экран массив до сортировки
17     std::cout << "Unsorted colors array:\n ";
18     for (const std::string& color : colors) {
19         std::cout << fmt::format("{} ", color);
20     }
21
22     // Сортируем массив
23     std::sort(std::begin(colors), std::end(colors));
```

```

24
25 // Выводим на экран отсортированный массив
26 std::cout << "\nSorted colors array:\n ";
27 for (const std::string& color : colors) {
28     std::cout << fmt::format("{} ", color);
29 }
30
31 // Есть ли в массиве элемент "indigo"?
32 bool found{std::binary_search(
33     std::begin(colors), std::end(colors), "indigo")};
34 std::cout << fmt::format("\n\n\"indigo\" {} found in colors array\n",
35     found ? "was" : "was not");
36
37 // Есть ли в массиве элемент "cyan"?
38 found = std::binary_search(
39     std::begin(colors), std::end(colors), "cyan");
40 std::cout << fmt::format("\n\n\"cyan\" {} found in colors array\n",
41     found ? "was" : "was not");
42 }

```

```

Unsorted colors array:
red orange yellow green blue indigo violet
Sorted colors array:
blue green indigo orange red violet yellow

"indigo" was found in colors array
"cyan" was not found in colors array

```

Илл. 6.9 | Сортировка массива и поиск в массиве

В строке 23 вызывается функция `sort` (заголовок `<algorithm>` стандартной библиотеки C++), которая сортирует массив `colors` в порядке возрастания. Для объектов типа `string` эта функция выполняет **лексикографическую сортировку** (**lexicographical sorting**), то есть упорядочивает текстовые строки по числовым значениям их символов в соответствующей кодировке. Аргументы функции `sort` определяют диапазон сортируемых элементов, в данном случае это весь массив. Аргументы `std::begin(colors)` и `std::end(colors)` возвращают **итераторы** (**iterators**), представляющие начало и конец массива соответственно. Итераторы подробно рассматриваются в главе 13. Функции `begin` и `end` определены в заголовке `<array>`. Как вы увидите в последующих главах, функция `sort` может сортировать элементы в структурах данных нескольких типов. Цикл в строках 27–29 выводит на экран содержимое отсортированного массива.

В строках 32–33 и 38–39 вызывается функция `binary_search` (заголовок `<algorithm>` стандартной библиотеки C++), которая определяет, есть ли среди элементов массива определенное значение. Перед вызовом `binary_search` любой массив обязательно должен быть отсортирован в порядке возрастания, сама функция этого не делает. Использование этой функции для поиска в неотсортированном массиве является логической ошибкой, приводящей к неправиль-

ным результатам. Первые два аргумента функции `binary_search` представляют диапазон элементов массива, среди которых выполняется поиск, а третий — это ключевое значение, которое надо найти. Функция возвращает результат типа `bool`, указывающий, было ли найдено ключевое значение. В главе 14 мы будем использовать стандартную функцию C++ `find`, возвращающую индекс ключевого значения в массиве.

6.13. Многомерные массивы

Вы можете использовать массивы с двумя измерениями (то есть с двумя индексами) для представления **таблиц значений** (**tables of values**), в которых данные расположены по **строкам** (**rows**) и **столбцам** (**columns**). Чтобы идентифицировать конкретный элемент таблицы, надо указать два индекса. По соглашению, первый индекс идентифицирует строку, а второй — столбец. Массивы, в которых элементы идентифицируются двумя индексами, называются **двумерными** (**two-dimensional**) или **2D-массивами**. Массивы с двумя и более измерениями называются **многомерными** (**multidimensional**). На диаграмме показан двумерный массив с именем `a`:

	Столбец 0	Столбец 1	Столбец 2	Столбец 3
Строка 0	<code>a[0][0]</code>	<code>a[0][1]</code>	<code>a[0][2]</code>	<code>a[0][3]</code>
Строка 1	<code>a[1][0]</code>	<code>a[1][1]</code>	<code>a[1][2]</code>	<code>a[1][3]</code>
Строка 2	<code>a[2][0]</code>	<code>a[2][1]</code>	<code>a[2][2]</code>	<code>a[2][3]</code>

Имя массива
Индекс строки
Индекс столбца

В этом массиве три строки и четыре столбца, поэтому его называют «массив 3 на 4». В общем случае о массиве, в котором m строк и n столбцов, говорят, что это «массив m на n ».

На диаграмме мы идентифицировали каждый элемент по имени в виде `a[строка][столбец]`. Доступ к каждому элементу также можно получить с помощью функции `at`:

```
a.at(i).at(j)
```

Все имена элементов первой строки (строки 0) имеют первый индекс 0; все имена элементов последнего столбца (столбца 3) имеют второй индекс 3.

В программе, показанной на илл. 6.10, два двумерных массива инициализируются в объявлениях. Каждая из строк 11–12 создает двумерный массив (то есть массив массивов) с двумя строками и тремя столбцами.


```
1 // fig06_10.cpp
2 // Инициализация многомерных массивов
3 #include <iostream>
4 #include <array>
5
6 constexpr size_t rows{2};
7 constexpr size_t columns{3};
8 void printArray(const std::array<std::array<int, columns>, rows>& a);
9
10 int main() {
11     constexpr std::array values1{std::array{1, 2, 3}, std::array{4, 5, 6}};
12     constexpr std::array values2{std::array{1, 2, 3}, std::array{4, 5, 0}};
13
14     std::cout << "values1 by row:\n";
15     printArray(values1);
16
17     std::cout << "\nvalues2 by row:\n";
18     printArray(values2);
19 }
20
21 // Выводим массив с двумя строками и тремя столбцами
22 void printArray(const std::array<std::array<int, columns>, rows>& a) {
23     // Обрабатываем строки массива в цикле
24     for (const auto& row : a) {
25         // Обрабатываем столбцы текущей строки массива в цикле
26         for (const auto& element : row) {
27             std::cout << element << ' ';
28         }
29
30         std::cout << '\n'; // Новая строка для вывода
31     }
32 }
```

```
values1 by row:
```

```
1 2 3
```

```
4 5 6
```

```
values2 by row:
```

```
1 2 3
```

```
4 5 0
```

Илл. 6.10 | Инициализация многомерных массивов

Объявление двумерного массива

В строках 11–12 компилятор автоматически определяет, что `values1` и `values2` — это двумерные массивы элементов типа `int`, расположенных в две строки по три столбца в каждой. Рассмотрим два инициализатора массива `values1`:

```
std::array{1, 2, 3}
```

```
std::array{4, 5, 6}
```

По ним компилятор определяет, что `values1` имеет две строки. Каждый инициализатор создает массив из трех элементов, поэтому компилятор определяет, что каждая строка `values1` имеет три столбца. Наконец, значения в этих инициализаторах являются целыми числами (`int`), поэтому компилятор определяет, что элементы `values1` имеют тип `int`.

Вывод на экран элементов двумерного массива

Для вывода элементов `values1` и `values2` программа вызывает функцию `printArray`. Прототип функции (строка 8) и ее определение (строки 22–32) указывают, что `printArray` получает через константную ссылку двумерный массив элементов `int`, состоящий из двух строк и трех столбцов. В объявлении:

```
const std::array<std::array<int, columns>, rows>&
```

указано, что количество элементов внешнего массива равно `rows` (то есть 2), а также указан их тип:

```
array<int, columns>
```

Таким образом, каждый из элементов внешнего массива — это внутренний массив, состоящий из трех элементов `int` (значение `columns` равно 3). Функция `printArray` не изменяет элементы, поэтому она получает `array` через константную (`const`) ссылку.

Два цикла: внешний и внутренний

Обработка элементов двумерного массива выполняется двумя циклами:

- ♦ **внешний цикл (outer loop)** выполняет итерации по строкам;
- ♦ **внутренний цикл (inner loop)**, или вложенный цикл, выполняет итерации по столбцам текущей строки.

Эти циклы реализованы двумя инструкциями `for` для диапазона значений. В строках 24 и 26 мы впервые использовали ключевое слово `auto` (C++11). Оно сообщает компилятору, что тип данных переменной надо автоматически определить по типу инициализирующего значения. Во внешнем цикле переменная диапазона `row` инициализируется элементом из параметра `a`. Как видно из объявления массива, этот элемент имеет следующий тип:

```
array<int, columns>
```

Поэтому компилятор определяет, что `row` является ссылкой на трехэлементный массив значений `int` (напомним, что `columns` равно 3). Квалификатор `const` в объявлении `row` запрещает использовать ссылку для изменения элементов и предотвращает копирование каждого элемента в переменную диапазона. Во внутреннем цикле переменная диапазона `element` (тоже объявленная с квалификатором `const`) инициализируется элементом массива, представленным переменной `row`. Компилятор определяет, что `element` является ссылкой на значение `int`, поскольку каждая строка массива содержит три значения `int`. (Во многих интегрированных средах разработки при наведении курсора мыши на переменную, объявленную с ключе-

вым словом `auto`, отображается ее тип, определенный компилятором.) В строке 27 выводится на экран значение `element` из соответствующей строки и столбца.

Альтернатива: два цикла `for` со счетчиком

Циклы `for` для диапазона значений мы могли бы заменить циклами `for` со счетчиком:

```
for (size_t row{0}; row < a.size(); ++row) {
    for (size_t column{0}; column < a.at(row).size(); ++column) {
        cout << a.at(row).at(column) << ' '; // Или a[row][column]
    }
    cout << '\n';
}
```

Скобочная инициализация массива массивов

Если вы явно укажете размеры массива при его объявлении, вы можете упростить его список инициализаторов. Например, строку 11 можно было бы записать так:

```
constexpr std::array<std::array<int, columns>, rows> values1{
    {{1, 2, 3}, // Строка 0
     {4, 5, 6}} // Строка 1
};
```

В этом случае, если строка списка инициализаторов содержит меньше элементов, чем количество столбцов, остальные элементы строки инициализируются значением по умолчанию.

Операции с двумерным массивом: установка значений строки

Давайте рассмотрим несколько типичных операций с массивом `a`, состоящим из трех строк и четырех столбцов, который показан на диаграмме в начале раздела 6.13. Следующий цикл `for` обнуляет все элементы строки 2:

```
for (size_t column{0}; column < a.at(2).size(); ++column) {
    a.at(2).at(column) = 0; // Или a[2][column]
}
```

Здесь в цикле `for` изменяется только второй индекс (то есть индекс столбца). Этот код, состоящий из одной инструкции цикла `for`, эквивалентен четырем инструкциям присваивания:

```
a.at(2).at(0) = 0; // Или a[2][0] = 0;
a.at(2).at(1) = 0; // Или a[2][1] = 0;
a.at(2).at(2) = 0; // Или a[2][2] = 0;
a.at(2).at(3) = 0; // Или a[2][3] = 0;
```

Операции с двумерным массивом: суммирование элементов в циклах `for` со счетчиком

Следующий код, основанный на внешнем и внутреннем циклах `for` со счетчиком, суммирует значения элементов массива `a`:

```
int total{0};
for (size_t row{0}; row < a.size(); ++row) {
    for (size_t column{0}; column < a.at(row).size(); ++column) {
        total += a.at(row).at(column); // a[row][column]
    }
}
```

Здесь все элементы массива суммируются построчно. В первой итерации внешнего цикла индекс строки (`row`) равен 0, поэтому во внутреннем цикле суммируются элементы строки 0. Во второй итерации внешнего цикла `row` увеличивается до 1 и во внутреннем цикле суммируются элементы строки 1. В последней итерации внешнего цикла `row` увеличивается до 2 и во внутреннем цикле суммируются элементы строки 2.

Операции с двумерным массивом: суммирование элементов в циклах `for` для диапазона значений

По сравнению с предыдущим примером для суммирования значений элементов массива `a` предпочтительнее следующий код, основанный на внешнем и внутреннем циклах `for` для диапазона значений:

```
int total{0};
for (const auto& row : a) { // Для каждой строки (row) массива a
    for (const auto& column : row) { // Для каждого столбца (column) в строке
        total += column;
    }
}
```

Многомерные массивы `mdarray` (C++23)

Комитет по стандартам C++ работает над специализированным контейнером многомерных массивов под названием `mdarray`, который может быть реализован в C++23. Здесь вы можете следить за процессом реализации `mdarray` и соответствующего типа `mdspan`:

- ✦ `mdarray` — <https://isocpp.org/files/papers/D1684R0.html>
- ✦ `mdspan` — <https://wg21.link/P0009>

6.14. Основы функционального программирования

Как и другие популярные языки (в том числе Python, Java и C#), язык C++ поддерживает несколько парадигм (концепций, стилей) программирования, которые вы можете комбинировать в вашем коде: процедурное, объектно-ориентированное, обобщенное (шаблонное) и функциональное программирование. Имеющиеся в C++ средства программирования в функциональном стиле помогают писать лаконичный и надежный код, который легко читать, отлаживать и модифицировать. Кроме того, функциональный стиль программирования упрощает распараллеливание программ для ускорения их работы на компьютерах с многоядерными процессорами.

6.14.1. «Что» и «как»

По мере усложнения программы ее код становится очень трудным для понимания, отладки и модификации. Повышается вероятность ошибок. Вам становится все труднее сформулировать, как должен работать код. Программируя в функциональном стиле, вы обычно указываете, что именно вы хотите сделать, а библиотечный код без вашего участия определяет, как это будет сделано. Это предотвращает многие ошибки.

Рассмотрим следующую инструкцию `for` для диапазона значений (из примера на илл. 6.5), суммирующую все целые числа из массива:

```
for (const int& item : integers) {
    total += item;
}
```

Хотя этот процедурный код скрывает детали итерации, нам все равно пришлось указать, как программа должна производить вычисления: «прибавлять каждое значение `item` к значению `total`». Каждый раз, когда вы изменяете какую-либо переменную, вы рискуете ошибиться. Функциональное программирование дает преимущество **неизменяемости (immutability)**: вы можете обойтись без операций, изменяющих значения переменных. При первом знакомстве с этой концепцией вы можете задаться вопросом: «Разве такое возможно?». Читайте дальше.

Свертка в функциональном стиле с помощью `accumulate`

На илл. 6.11 вы видите пример решения той же задачи сложения всех целых чисел из массива, но уже в функциональном стиле. Вместо инструкции `for` для диапазона значений (из примера на илл. 6.5) мы используем алгоритм (то есть функцию, выполняющую алгоритм) `accumulate` из заголовка `<numeric>` стандартной библиотеки C++. Эта функция знает, как вычислить сумму диапазона значений, сведя их все в единое целое. Это вычисление является примером **свертки (reduction)** — одной из базовых операций в функциональном программировании.

```
1 // fig06_11.cpp
2 // Вычисление суммы элементов массива алгоритмом accumulate
3 #include <array>
4 #include <fmt/format.h>
5 #include <iostream>
6 #include <numeric>
7
8 int main() {
9     constexpr std::array integers{10, 20, 30, 40};
10    std::cout << fmt::format("Total of array elements: {}\n",
11    std::accumulate(std::begin(integers), std::end(integers), 0));
12 }
```

Total of array elements: 100

Илл. 6.11 | Вычисление суммы элементов массива алгоритмом `accumulate`

Как и в функции `sort` из раздела 6.12, первые два аргумента функции `accumulate` (строка 11) определяют диапазон элементов для вычисления суммы. В данном случае это все элементы массива `integers` — от начала (`begin`) до конца (`end`). Функция **внутренне** (*internally*) суммирует обрабатываемые элементы массива, скрывая от вас вычислительный процесс. Третий аргумент — это начальное значение суммы (0). У функции `accumulate` есть и другие возможности, о которых вы скоро узнаете.

Функция `accumulate` использует **внутреннюю итерацию** (*internal iteration*), которая тоже скрыта от вас. Функция знает, как перебирать диапазон элементов массива и прибавлять значение каждого элемента к текущей сумме. Вы просто указываете, что вы хотите сделать, а библиотека C++ самостоятельно определяет способ выполнения. Это так называемое **декларативное программирование** (*declarative programming*) — еще одна отличительная черта функционального стиля.

6.14.2. Передача функций в качестве аргументов другим функциям, основы лямбда-выражений

Многие стандартные библиотечные функции C++ принимают в качестве аргументов другие функции, тем самым позволяя вам уточнять, как они должны работать. Функции, принимающие другие функции в качестве аргументов, называются **функциями высшего порядка** (*higher-order functions*) и часто используются в функциональном программировании. Рассмотрим функцию `accumulate`, которая по умолчанию суммирует значения элементов. Ее можно перегружать, передавая четвертый аргумент: функцию, которая задает алгоритм выполнения свертки. На илл. 6.12 вы видите, как вместо суммы значений перегруженная функция `accumulate` вычисляет их произведение.

```
1 // fig06_12.cpp
2 // Вычисление произведения элементов массива алгоритмом accumulate
3 #include <array>
4 #include <fmt/format.h>
5 #include <iostream>
6 #include <numeric>
7
8 int multiply(int x, int y) {
9     return x * y;
10 }
11
12 int main() {
13     constexpr std::array integers{1, 2, 3, 4, 5};
14
15     std::cout << fmt::format("Product of integers: {}\n", std::accumulate(
16         std::begin(integers), std::end(integers), 1, multiply));
17
18     std::cout << fmt::format("Product of integers with a lambda: {}\n",
19         std::accumulate(std::begin(integers), std::end(integers), 1,
```

```
20     [(const auto& x, const auto& y){return x * y;});
21 }
```

```
Product of integers: 120
```

```
Product of integers with a lambda: 120
```

Илл. 6.12 | Вычисление произведения элементов массива алгоритмом `accumulate`

Вызов `accumulate` с именованной функцией в качестве аргумента

В строках 15–16 функция `accumulate` вызывается для массива `integers` (определенного в строке 13). Мы вычисляем произведение, поэтому третий аргумент (начальное значение для свертки) равен 1, а не 0, иначе результат был бы равен 0. Четвертый аргумент — это функция, вызываемая для каждого элемента массива, в данном случае функция `multiply` (определенная в строках 8–10). Она перемножает два числа, полученные в качестве аргументов:

- ♦ промежуточный результат;
- ♦ значение из массива.

Затем она возвращает значение, которое становится новым промежуточным результатом. Когда функция `accumulate` перебирает значения массива `integers`, она передает функции `multiply` в качестве аргументов промежуточный результат и следующий элемент массива. В этом примере происходит пять внутренних вызовов функции `multiply` из функции `accumulate`:

- ♦ Первый вызов передает функции `multiply` начальное значение свертки (число 1, которое было третьим аргументом функции `accumulate`) и первый элемент массива (1). Возвращаемое значение: 1.
- ♦ Второй вызов передает функции `multiply` промежуточный результат (1) и второй элемент массива (2). Возвращаемое значение: 2.
- ♦ Третий вызов передает функции `multiply` промежуточный результат (2) и третий элемент массива (3). Возвращаемое значение: 6.
- ♦ Четвертый вызов передает функции `multiply` промежуточный результат (6) и четвертый элемент массива (4). Возвращаемое значение: 24.
- ♦ Пятый вызов передает функции `multiply` промежуточный результат (24) и пятый элемент массива (5). Возвращаемое значение: 120. Это конечный результат свертки, который функция `accumulate` возвращает в точку своего вызова.

Вызов функции `accumulate` с лямбда-выражением

Иногда вам может понадобиться «одноразовая» функция, которую вы не будете использовать повторно. В таком случае вы можете определить функцию там, где она необходима, используя **лямбда-выражение** (*lambda expression*), или, сокращенно, **лямбду**. Лямбда-выражение (C++11) — это, по сути, анонимная функция,

то есть функция без имени. Вызов `accumulate` в строках 19–20 содержит лямбда-выражение, выполняющее ту же задачу, что и функция `multiply`:

```
[(const auto& x, const auto& y){return x * y;}]
```

Каждое лямбда-выражение начинается со списка захвата (*capture list*), который также называют **вводной секцией** или **лямбда-интродуктором** (*lambda introducer*), — это пара квадратных скобок с необязательными параметрами внутри (см. главу 14). После списка захвата следует список параметров, разделенных запятыми, и тело функции. В данном случае лямбда получает два параметра, вычисляет их произведение и возвращает результат.

В разделе 6.13 вы видели, что ключевое слово `auto` сообщает компилятору о необходимости автоматически определить тип переменной по ее инициализирующему значению. Указание типа параметра лямбды как `auto` позволяет компилятору определить тип этого параметра на основе контекста, в котором вызывается лямбда. В этом примере функция `accumulate` вызывает лямбду один раз для каждого элемента массива, передавая в качестве аргументов промежуточный результат и значение элемента. Тип начального произведения (1) — `int`, и массив тоже содержит элементы `int`, поэтому компилятор определяет типы параметров как `int`. Возможность автоматического определения типа каждого параметра лямбды с помощью ключевого слова `auto` появилась в C++14, такие лямбды называются **обобщенными** (*generic*). В нашем примере компилятор автоматически определяет и тип возвращаемого лямбдой значения, основываясь на выражении `x * y`. Здесь `x` и `y` имеют тип `int`, поэтому лямбда возвращает значение `int`.

Мы объявили параметры как константные ссылки, и таким образом:

- ♦ поскольку это константы, тело лямбда-выражения не может изменять переменные в вызывающем коде;
- ♦ поскольку это ссылки, программа не тратит время на копирование объектов и работает быстрее (это особенно важно при вызовах лямбда-выражений с большими объектами).

Лямбда-выражение из этого примера можно использовать с любым типом данных, поддерживающим оператор `*`. В главе 14 лямбда-выражения рассматриваются подробно.

6.14.3. Фильтрация, преобразование и свертка с библиотекой `ranges` (C++20)

Стандартная библиотека C++ уже много лет позволяет программировать в функциональном стиле. В C++20 появилась библиотека `ranges`¹ (заголовок `<ranges>`), упростившая программирование в функциональном стиле. Мы представим вам два ключевых понятия этой библиотеки: диапазоны и представления.

¹ На момент написания книги компилятор `clang++` еще не поддерживает представленные здесь функции этой библиотеки.

- ♦ **Диапазон** (`range`) — это набор элементов, которые вы можете поочередно перебирать итерациями. Например, массив — это диапазон¹.
- ♦ **Представление** (`view`) позволяет вам определить операцию, обрабатывающую диапазон. Представления можно комбинировать: вы можете последовательно соединить несколько представлений, чтобы обрабатывать элементы диапазона несколькими операциями.

Наша следующая программа, разделенная на части для удобства обсуждения (илл. 6.13), демонстрирует несколько операций функционального программирования, использующих библиотеку `ranges`. В главе 14 рассматриваются другие возможности этой библиотеки.

Лямбда `showValues`, выводящая результаты на экран

В этом примере мы выводим на экран результаты различных операций с диапазонами, используя цикл `for` для диапазона значений. Вместо многократного повторения кода цикла можно определить функцию, которая получает диапазон и выводит на экран его значения. Мы решили определить обобщенную лямбду (строки 12–20), чтобы показать, что вы можете сохранить лямбду в локальной переменной (`showValues`; строки 11–21). Затем вы можете вызывать лямбду по имени этой переменной, как в строках 24, 29, 34, 40 и 51.

```

1 // fig06_13.cpp
2 // Функциональное программирование с диапазонами и представлениями (C++20)
3 #include <array>
4 #include <fmt/format.h>
5 #include <iostream>
6 #include <numeric>
7 #include <ranges>
8
9 int main() {
10     // Лямбда для вывода на экран результатов операций с диапазонами
11     auto showValues{
12         [](auto& values, const std::string& message) {
13             std::cout << fmt::format("{}: ", message);
14
15             for (const auto& value : values) {
16                 std::cout << fmt::format("{} ", value);
17             }
18
19             std::cout << '\n';
20         }
21     };
22

```

Илл. 6.13 | Функциональное программирование с диапазонами и представлениями (C++20)

¹ Слово «*range*» в этом контексте для носителей английского языка означает «ряд» или «множество» («ряд значений», «множество чисел»), но в русском языке такой набор элементов обычно называют диапазоном. Авторы книги имеют в виду, что массив — это множество, состоящее из элементов. — *Примеч. ред.*

Генерация диапазона целых чисел с помощью `views::iota`

В нескольких примерах этой главы мы создавали массив и обрабатывали его элементы. Для этого нам надо было каждый раз начинать с объявления массива с заданным количеством элементов. В некоторых случаях вы можете генерировать значения **по требованию (on demand)** вместо того, чтобы создавать и сохранять их заранее. Операции, генерирующие значения по требованию, используют **ленивые вычисления (lazy evaluation)**, которые уменьшают потребление памяти программой и повышают быстродействие в тех случаях, когда не нужны все значения сразу. В строке 23 мы используем фабрику¹ `views::iota` из библиотеки `ranges` для генерации диапазона последовательных целых чисел от первого аргумента (1) до второго аргумента (11), но кроме второго аргумента (то есть без числа 11; это называется **полуоткрытым диапазоном**). Значения не генерируются, пока не понадобятся программе, например, при вызове лямбды `showValues` (строка 24).

```
23 auto values1{std::views::iota(1, 11)}; // Генерируем числа от 1 до 10
24 showValues(values1, "Generate integers 1-10");
25
```

```
Generate integers 1-10: 1 2 3 4 5 6 7 8 9 10
```

Фильтрация элементов с помощью `views::filter`

Одна из базовых операций в функциональном программировании — **фильтрация (filtering)** элементов. В результате фильтрации из диапазона выбираются только те элементы, которые соответствуют заданному условию. Как правило, количество элементов после фильтрации меньше, чем в исходном диапазоне. Один из способов реализовать фильтрацию — перебирать элементы в цикле, проверяя каждый из них вложенной инструкцией `if` на соответствие заданному условию. По итогам проверки элемента с ним можно что-то сделать, например добавить в контейнер. Этот способ требует явного определения инструкции цикла и изменяемых переменных, что может приводить к ошибкам, как мы отмечали в разделе 6.6.

Работая с диапазонами и представлениями, мы можем реализовать фильтрацию с помощью адаптера² `views::filter`, чтобы сосредоточиться на том, что мы хотим получить. В данном случае мы хотим получить только четные числа от 1 до 10. Инициализатор `values2` в строках 27–28 использует оператор `|` для соединения нескольких операций. Первая операция (`values1` из строки 23) генерирует числа от 1 до 10, а вторая фильтрует результат первой. Вместе эти операции образуют **конвейер (pipeline)**. Каждый конвейер начинается с диапазона, который является источником данных (в нашем случае это `values1`), за которым следует произвольное количество операций, разделенных символом `|`.

¹ Этот термин объясняется в разделе 14.7.2. — *Примеч. ред.*

² Этот термин объясняется в разделе 14.7.1. — *Примеч. ред.*

```

26 // Фильтруем значения values1, сохраняя только четные числа
27 auto values2{values1 |
28     std::views::filter([](const auto& x) {return x % 2 == 0;});
29     showValues(values2, "Filtering even integers");
30

```

Filtering even integers: 2 4 6 8 10

Аргумент адаптера `views::filter` должен быть функцией, которая получает одно значение для обработки и возвращает результат типа `bool`, указывающий, следует ли сохранять это значение. Мы передали в качестве аргумента лямбду, которая возвращает `true`, если ее аргумент делится без остатка на 2.

Как только будут выполнены строки 27–28, `values2` становится **ленивым конвейером (lazy pipeline)**, который генерирует целые числа от 1 до 10 и фильтрует их, сохраняя только четные числа. Конвейер кратко определяет то, что мы хотим сделать (а не как это сделать):

- ✦ Фабрика `views::iota` знает, как генерировать целые числа.
- ✦ Адаптер `views::filter` знает, как использовать свою функцию-аргумент, чтобы определять, когда надо сохранять элементы, полученные из предыдущей части конвейера.

Этот конвейер называется ленивым, потому что он не производит никаких результатов, пока нет обращения к `values2`.

Как только лямбда `showValues` начинает перебирать элементы `values2`, фабрика `views::iota` генерирует первое значение, а затем адаптер `views::filter` вызывает функцию-аргумент, чтобы определить, надо ли сохранить это значение. Если эта функция возвращает `true`, инструкция `for` для диапазона значений (входящая в `showValues`) получает значение из конвейера и выводит его на экран. В противном случае шаги обработки повторяются со следующим значением, которое генерирует фабрика `views::iota`.

Маппинг элементов с помощью `views::transform`

Еще одна базовая операция в функциональном программировании — это **маппинг (mapping)** диапазона, то есть преобразование его элементов по единому алгоритму в новые значения, иногда другого типа. В результате маппинга количество элементов в диапазоне не изменяется. В библиотеке `ranges` для такого преобразования элементов есть адаптер `views::transform`. Конвейер в строках 32–33 добавляет к конвейеру из строк 27–28 еще одну операцию: лямбда-выражение, переданное адаптеру `views::transform` в строке 33, возводит во вторую степень все отфильтрованные числа из `values2`.

```

31 // Возводим в квадрат каждое число из values2
32 auto values3{

```

```

33     values2 | std::views::transform([](const auto& x) {return x * x;});
34     showValues(values3, "Mapping even integers to squares");
35

```

```
Mapping even integers to squares: 4 16 36 64 100
```

Аргумент адаптера `views::transform` — это функция, которая получает значение для обработки и возвращает новое значение — возможно, другого типа. Когда `showValues` начинает перебирать результаты конвейера `values3`:

1. Фабрика `views::iota` создает значение.
2. Адаптер `views::filter` определяет, является ли это значение четным. Если да, значение передается на шаг 3. Если нет, обрабатывается следующее значение, которое генерирует фабрика `views::iota` на шаге 1.
3. Адаптер `views::transform` вычисляет квадрат четного целого числа (лямбда в строке 33), затем инструкция `for` для диапазона значений (входящая в `showValues`) отображает результат и обработка продолжается со следующим значением, которое генерирует фабрика `views::iota` на шаге 1.

Объединение в конвейер операций фильтрации и преобразования

Конвейер может содержать произвольное количество операций, разделенных операторами `|`. Конвейер в строках 37–39 объединяет все предыдущие операции, а в строке 40 выводятся результаты.

```

36 // Объединяем фильтрацию и преобразование, получаем квадраты четных чисел
37     auto values4{
38         values1 | std::views::filter([](const auto& x) {return x % 2 == 0;})
39                 | std::views::transform([](const auto& x) {return x * x;});
40     showValues(values4, "Squares of even integers");
41

```

```
Squares of even integers: 4 16 36 64 100
```

Свертка конвейера диапазона функцией `accumulate`

Функции стандартной библиотеки C++, такие как `accumulate`, тоже работают с ленивыми конвейерами диапазонов. В строке 44 выполняется свертка, суммирующая квадраты четных целых чисел, полученные конвейером в строках 37–39.

```

42 // Суммируем квадраты четных чисел
43     std::cout << fmt::format("Sum squares of even integers 2-10: {}\n",
44         std::accumulate(std::begin(values4), std::end(values4), 0));
45

```

```
Sum squares of even integers 2-10: 220
```

Фильтрация и преобразование элементов контейнера

Различные контейнеры C++, в том числе массивы и векторы (раздел 6.15), могут быть источниками данных в конвейерах диапазонов. В строке 47 создается массив, содержащий числа от 1 до 10. Затем он используется в конвейере, вычисляющем квадраты четных целых чисел из этого массива.

```
46 // Обработка элементов контейнера
47 constexpr std::array numbers{1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
48 auto values5{
49     numbers | std::views::filter([](const auto& x) {return x % 2 == 0;})
50             | std::views::transform([](const auto& x) {return x * x;});
51 showValues(values5, "Squares of even integers in array numbers");
52 }
```

Squares of even integers in array numbers: 4 16 36 64 100

6.15. Готовые объекты: шаблон класса vector стандартной библиотеки C++

Теперь, продолжая презентацию готовых объектов, мы расскажем об объектах шаблона класса `vector`¹ стандартной библиотеки C++ (заголовок `<vector>`). Векторы² схожи с массивами, но размеры векторов можно динамически менять. В конце этого раздела мы покажем, как проверять границы векторов (такая проверка возможна и для массивов). Мы рассмотрим имеющийся в C++ механизм обработки исключений при попытке доступа к несуществующему элементу вне границ вектора. Для этого мы включили в программу (илл. 6.14) заголовок `<stdexcept>` (строка 5). В строках 7–8 объявлены прототипы функций `outputVector` (строки 93–99) и `inputVector` (строки 102–106), которые служат для вывода и ввода значений элементов вектора соответственно.

```
1 // fig06_14.cpp
2 // Демонстрация шаблона класса vector стандартной библиотеки C++
3 #include <iostream>
4 #include <vector>
5 #include <stdexcept>
6
7 void outputVector(const std::vector<int>& items); // Функция для вывода
8 void inputVector(std::vector<int>& items); // Функция для ввода
9
```

Илл. 6.14 | Демонстрация шаблона класса `vector` стандартной библиотеки C++

¹ Другие возможности векторов обсуждаются в главе 13.

² Векторы языка C++ иногда называют «динамическими массивами класса `vector`» (сокращенно — «динамическими массивами»), если надо подчеркнуть, что речь идет не о векторах в математическом или ином значении этого слова. — *Примеч. ред.*

Создание объектов класса `vector`

В строках 11–12 создаются два вектора с элементами типа `int`: вектор `integers1`, содержащий 7 элементов, и вектор `integers2`, содержащий 10 элементов. По умолчанию все элементы каждого созданного вектора инициализируются значением 0. Как и массивы, векторы могут хранить большинство типов данных — вы можете просто заменить `int` в `std::vector<int>` на нужный вам тип.

```
10 int main() {  
11     std::vector<int> integers1(7); // Вектор с семью элементами  
12     std::vector<int> integers2(10); // Вектор с десятью элементами  
13 }
```

Обратите внимание, что аргументы, обозначающие размеры векторов (7 и 10), передаются конструкторам векторов в круглых скобках (а не в фигурных). Если при создании вектора поместить значение в фигурные скобки, компилятор воспримет это как список инициализаторов с одним элементом, а не как размер вектора. Например, следующее объявление создает вектор, содержащий один элемент со значением 7, а не вектор из семи элементов:

```
std::vector<int> integers1{7};
```

Если содержимое вектора известно на этапе компиляции, вы можете определить вектор с помощью списка инициализаторов и дедуктивного определения аргументов шаблона класса (CTAD), как мы это делали с массивами. Например, следующая инструкция определяет вектор с четырьмя элементами типа `double`:

```
std::vector integers1{1.1, 2.2, 3.3, 4.4};
```

Функции `size` и `outputVector`

В строке 15 вызывается функция `size` класса `vector`, возвращающая количество элементов вектора `integers1`¹. В строке 17 вектор `integers1` передается пользовательской функции `outputVector` (строки 93–99), которая выводит на экран элементы вектора, используя инструкцию `for` для диапазона значений. В строках 20 и 22 выполняются аналогичные действия для вектора `integers2`.

```
14 // Выводим размер и содержимое вектора integers1  
15 std::cout << "Size of vector integers1 is " << integers1.size()  
16     << "\nvector after initialization:";  
17 outputVector(integers1);  
18  
19 // Выводим размер и содержимое вектора integers2
```

Окончание ⇨

¹ Вы также можете использовать глобальную функцию C++17 `std::size`, например `std::size(integers1)`.

```
20 std::cout << "\nSize of vector integers2 is " << integers2.size()
21     << "\nvector after initialization:";
22 outputVector(integers2);
23
```

```
Size of vector integers1 is 7
vector after initialization: 0 0 0 0 0 0 0

Size of vector integers2 is 10
vector after initialization: 0 0 0 0 0 0 0 0 0 0
```

Функция inputVector

В строках 26–27 векторы `integers1` и `integers2` передаются функции `inputVector` (строки 102–106), которая получает значения элементов, вводимые пользователем.

```
24 // Считываем значения для integers1 и integers2, выводим их на экран
25 std::cout << "\nEnter 17 integers:\n";
26 inputVector(integers1);
27 inputVector(integers2);
28
29 std::cout << "\nAfter input, the vectors contain:\n"
30     << "integers1:";
31 outputVector(integers1);
32 std::cout << "integers2:";
33 outputVector(integers2);
34
```

```
Enter 17 integers:
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17

After input, the vectors contain:
integers1: 1 2 3 4 5 6 7
integers2: 8 9 10 11 12 13 14 15 16 17
```

Сравнение векторов оператором !=

В главе 5 мы рассказали о перегрузке функций. Аналогичная концепция — перегрузка операторов, позволяющая вам определять, как стандартные операторы C++ должны работать с пользовательскими типами данных. В стандартной библиотеке C++ уже есть определения перегруженных операторов `==` и `!=` для работы с массивами и векторами. Поэтому вы можете сравнивать этими операторами два массива или два вектора на равенство или неравенство соответственно. В строке 38 два вектора сравниваются оператором `!=`. Он возвращает значение `true`, если векторы отличаются (если у них разный размер или какие-то их элементы с одинаковыми индексами не совпадают). В противном случае `!=` возвращает `false`.

```
35 // Сравниваем векторы оператором неравенства (!=)
36 std::cout << "\nEvaluating: integers1 != integers2\n";
37
```

```

38  if (integers1 != integers2) {
39      std::cout << "integers1 and integers2 are not equal\n";
40  }
41

```

```

Evaluating: integers1 != integers2
integers1 and integers2 are not equal

```

Инициализация вектора содержимым другого вектора

Вы можете инициализировать новый вектор, скопировав в него содержимое существующего вектора. В строке 44 создается вектор `integers3` и инициализируется копией вектора `integers1`. Здесь мы использовали СТАД для автоматического определения типа `integers3` по типу `integers1`. Для выполнения операции копирования в строке 44 вызывается копирующий конструктор шаблона класса `vector`. Мы расскажем о копирующих конструкторах в главе 11. В строках 46–48 выводятся на экран размеры и содержимое `integers3`, что подтверждает правильную инициализацию.

```

42  // Создаем вектор integers3, используя integers1 как инициализатор;
43  // выводим на экран размер и содержимое вектора integers3
44  std::vector integers3(integers1); // Копирующий конструктор
45
46  std::cout << "\nSize of vector integers3 is " << integers3.size()
47      << "\nvector after initialization: ";
48  outputVector(integers3);
49

```

```

Size of vector integers3 is 7
vector after initialization: 1 2 3 4 5 6 7

```

Присваивание векторов и сравнение векторов

В строке 52 вектор `integers2` присваивается вектору `integers1`, чтобы показать, что оператор присваивания (`=`) перегружен для работы с векторами. В строках 54–57 выводятся на экран элементы обоих векторов, и вы видите, что они одинаковы. Затем в строке 62 векторы `integers1` и `integers2` сравниваются оператором равенства (`==`), чтобы определить, идентично ли их содержимое после присваивания.

```

50  // Используем перегруженный оператор присваивания (=)
51  std::cout << "\nAssigning integers2 to integers1:\n";
52  integers1 = integers2; // Присваиваем вектор integers2 вектору integers1
53
54  std::cout << "integers1: ";
55  outputVector(integers1);
56  std::cout << "integers2: ";
57  outputVector(integers2);
58
59  // Сравниваем векторы оператором равенства (==)

```



```

60  std::cout << "\nEvaluating: integers1 == integers2\n";
61
62  if (integers1 == integers2) {
63      std::cout << "integers1 and integers2 are equal\n";
64  }
65

```

Assigning integers2 to integers1:

integers1: 8 9 10 11 12 13 14 15 16 17

integers2: 8 9 10 11 12 13 14 15 16 17

Evaluating: integers1 == integers2

integers1 and integers2 are equal

Функция `at` для доступа к элементам вектора

В строке 67 вызывается функция `at` класса `vector` для доступа к заданному элементу вектора и для получения его значения. В строке 71 эта функция вызывается для изменения значения элемента. Если функция `at` вызвана для элемента с корректным (существующим в векторе) индексом, она возвращает:

- ♦ ссылку на элемент, которую можно использовать для изменения его значения; или
- ♦ константную ссылку на элемент, которую нельзя использовать для изменения его значения (но можно использовать для получения его значения).

Функция `at` возвращает константную ссылку, если вызывается для вектора, объявленного с квалификатором `const`, или через константную ссылку.

```

66  // Используем значение с индексом 5 как rvalue
67  std::cout << "\nintegers1.at(5) is " << integers1.at(5);
68
69  // Используем integers1.at(5) как lvalue
70  std::cout << "\n\nAssigning 1000 to integers1.at(5)\n";
71  integers1.at(5) = 1000;
72  std::cout << "integers1: ";
73  outputVector(integers1);
74

```

integers1.at(5) is 13

Assigning 1000 to integers1.at(5)

integers1: 8 9 10 11 12 1000 14 15 16 17

Как и массивы, векторы работают с оператором `[]`. Согласно стандарту C++, при доступе к элементам через квадратные скобки компилятор не обязан выполнять проверку границ. Поэтому вы должны убедиться, что ни одна инструкция с оператором `[]` не может случайно обратиться к несуществующему элементу вне границ вектора.

Обработка исключений: обращение к элементу вне границ вектора

В строке 78 мы пробуем вывести на экран значение элемента `integers1.at(15)`, которого нет в векторе. Функция `at` при проверке границ обнаруживает недопустимый индекс и **выбрасывает**¹ (`throws`) исключение, указывающее на ошибку во время выполнения. Слово «исключение» подразумевает, что это редкая, исключительная ситуация. **Обработка исключений** (`exception handling`) позволяет создавать **отказоустойчивые программы** (`fault-tolerant programs`), то есть программы, способные корректно обрабатывать (или перехватывать) исключения. В некоторых случаях обработка исключения позволяет программе продолжать выполнение, как будто никакой ошибки не было. Например, наша программа будет нормально работать даже после того, как мы обратились к элементу с недопустимым индексом. Но более серьезная ошибка может прервать нормальное выполнение программы, не давая ей возможности освободить перед завершением все ресурсы, которые она использовала (например, закрыть открытый файл или подключение к базе данных). Здесь мы кратко обсудим обработку исключений. В главе 9 вы узнаете, как выбрасывать исключения из пользовательских функций, а обработку исключений мы подробно рассмотрим в главе 12.

```
75 // Пробуем обратиться к элементу с несуществующим индексом
76 try {
77     std::cout << "\nAttempt to display integers1.at(15)\n";
78     std::cout << integers1.at(15) << '\n'; // Ошибка: индекс вне границ
79 }
80 catch (const std::out_of_range& ex) {
81     std::cerr << "An exception occurred: " << ex.what() << '\n';
82 }
83
```

```
Attempt to display integers1.at(15)
An exception occurred: invalid vector subscript
```

Инструкция `try`

По умолчанию исключение приводит к аварийному завершению написанной на C++ программы. Чтобы обработать исключение и по возможности продолжить выполнение программы, надо поместить код, который может вызвать исключение, в инструкцию `try` (строки 76–82). Блок `try` (строки 76–79) содержит код, который может вызвать исключение, а блок `catch` (строки 80–82) содержит код, обрабатывающий это исключение, если оно случится. Как вы увидите в главе 12, за одним блоком `try` могут следовать много блоков `catch`, обрабатывающих разные типы предполагаемых исключений. Если код в блоке `try` выполняется успешно,

¹ Программисты говорят, что функция «выбрасывает», «бросает», «выдает», «создает» или «генерирует» исключение, — в таком контексте эти слова являются синонимами. — *Примеч. ред.*

инструкции в строках 80–82 игнорируются. Фигурные скобки, разделяющие тела блоков `try` и `catch`, обязательны.

Выполнение блока `catch`

Когда программа вызывает функцию `at` с аргументом 15 (строка 78), эта функция пытается получить доступ к элементу с индексом 15, который не соответствует границам вектора: `integers1` содержит только 10 элементов. Поскольку проверка границ происходит во время выполнения программы, функция `at` генерирует исключение (создает объект исключения). Конкретнее, в строке 78 выдается исключение `out_of_range` (заголовок `<std::out_of_range>`), чтобы сообщить программе о возникшей проблеме. В этот момент блок `try` немедленно прерывается, то есть все последующие инструкции в нем пропускаются и начинается выполнение блока `catch`. Если вы объявили какие-либо переменные в блоке `try`, они теперь окажутся вне области видимости и будут недоступны в блоке `catch`.

Блок `catch` должен объявлять свой параметр исключения (`ex`) как константную ссылку; мы подробнее поговорим об этом в главе 12. Блок `catch` может обрабатывать исключения указанного типа. Внутри блока вы можете использовать идентификатор этого параметра для взаимодействия с перехваченным объектом исключения.

Функция `what` объекта исключения

Когда в строках 80–82 перехватывается исключение, программа выводит на экран сообщение (которое зависит от компилятора), указывающее на возникшую проблему. В строке 81 вызывается функция `what` объекта исключения, представляющая строку для сообщения об ошибке. В нашем примере сразу после вывода сообщения исключение считается обработанным и программа продолжает выполнение со следующей инструкции после закрывающей фигурной скобки блока `catch`. Далее выполняются строки 85–89.

Изменение размера вектора

Одно из важнейших различий между векторами и массивами заключается в том, что векторы могут динамически увеличиваться и уменьшаться по мере изменения количества элементов. Чтобы продемонстрировать это, в строке 85 выводится размер вектора `integers3`, в строке 86 вызывается функция `push_back`, добавляющая в конец вектора новый элемент со значением 1000, а в строке 87 выводится новый размер `integers3`. Затем в строке 89 выводится на экран обновленное содержимое вектора.

```
84 // Изменение размера вектора
85 std::cout << "\nCurrent integers3 size is: " << integers3.size();
86 integers3.push_back(1000); // Добавляем 1000 в конец вектора
87 std::cout << "\nNew integers3 size is: " << integers3.size()
88     << "\nintegers3 now contains: ";
89 outputVector(integers3);
90 }
91
```

```
Current integers3 size is: 7
New integers3 size is: 8
integers3 now contains: 1 2 3 4 5 6 7 1000
```

Функции `outputVector` и `inputVector`

Функция `outputVector` использует цикл `for` для диапазона значений, чтобы получать значение каждого элемента вектора для вывода на экран. Как и в случае с массивом, то же самое можно было бы сделать и в цикле со счетчиком, но мы рекомендуем обрабатывать векторы в цикле `for` для диапазона значений. Аналогично функция `inputVector` содержит цикл `for` с переменной диапазона типа `int&`, которая используется для сохранения введенного значения в соответствующем элементе вектора.

```
92 // Выводим содержимое вектора
93 void outputVector(const std::vector<int>& items) {
94     for (const int& item : items) {
95         std::cout << item << " ";
96     }
97
98     std::cout << '\n';
99 }
100
101 // Записываем значения в вектор
102 void inputVector(std::vector<int>& items) {
103     for (int& item : items) {
104         std::cin >> item;
105     }
106 }
```

Простой линейный код

Изучая наши примеры кода, вы видели много циклов `for`. Возможно, вы заметили важную особенность разделов, посвященных готовым объектам: те фрагменты кода, в которых создаются и используются объекты, — это в основном линейные последовательности инструкций с небольшим количеством управляющих операторов. При работе с объектами код часто «сглаживается», то есть становится простой последовательностью вызовов функций.

6.16. Итоги

В этой главе вы познакомились со структурами данных, изучая шаблоны классов `array` и `vector` стандартной библиотеки C++, которые предназначены для хранения и обработки данных в списках и таблицах. Мы показали, как объявлять массивы, инициализировать их и ссылаться на отдельные элементы. Мы передавали массивы функциям по ссылке и использовали квалификатор `const`, чтобы предотвратить изменение элементов вызываемыми функциями. Вы узнали, как

использовать инструкцию `for` для диапазона значений (C++11) для обработки элементов массива. Мы также продемонстрировали функции `sort` и `binary_search` стандартной библиотеки C++, предназначенные для сортировки массива и для поиска элемента в отсортированном массиве соответственно.

Вы научились объявлять и обрабатывать двумерные массивы. Мы использовали вложенные циклы `for` для диапазона значений (а также вложенные циклы `for` со счетчиком), чтобы перебирать все строки и столбцы двумерного массива. Мы показали, как компилятор автоматически определяет по значению инициализатора тип переменной, объявленной с ключевым словом `auto`. Мы представили вам примеры функционального программирования, в том числе с диапазонами и представлениями библиотеки `ranges` (C++20) и ленивыми конвейерами.

В разделе, посвященном готовым объектам, мы продемонстрировали возможности шаблона класса `vector` стандартной библиотеки C++. Мы обсудили средства проверки границ массивов и векторов при доступе к их элементам, а также основные принципы обработки исключений. В последующих главах мы продолжим рассмотрение структур данных.

В главе 7 мы расскажем об указателях, которые относятся к самым мощным средствам языка C++. Указатели отслеживают, где данные хранятся в памяти, открывая перед программистом интересные возможности для обработки этих данных. Как вы увидите, в C++ сохранилась поддержка традиционных массивов (сильно отличающихся от `array`), тесно связанных с указателями. Но в современном коде вместо них лучше использовать безопасные массивы на основе шаблона класса `array`.

Указатели в современном C++ (фактор риска)

В этой главе вы:

- познакомитесь с указателями, научитесь их объявлять и инициализировать;
- познакомитесь с операторами взятия адреса (&) и косвенного обращения (*);
- сравните возможности указателей и ссылок;
- научитесь передавать аргументы функции по указателю;
- узнаете о традиционных массивах и о строках на основе указателей, которые встречаются в устаревшем коде;
- научитесь использовать квалификатор `const` с указателями и данными, на которые они указывают;
- научитесь использовать оператор `sizeof` для определения количества байтов, хранящих значение определенного типа;
- научитесь понимать выражения и арифметические операции с указателями, которые встречаются в устаревшем коде;
- познакомитесь с ключевым словом `nullptr` (C++11), обозначающим нулевой указатель (указывающий на «ничто»);
- научитесь использовать с традиционными массивами функции `begin` и `end` стандартной библиотеки (C++11);
- узнаете, почему в C++ Core Guidelines рекомендуется избегать указателей и традиционных массивов;
- научитесь преобразовывать традиционные массивы и списки инициализаторов в массивы `std::array` с помощью функции `to_array` (C++20);
- продолжите изучение готовых объектов, безопасно обращаясь к массивам и векторам через представления, созданные с помощью шаблона класса `span` (C++20).

7.1. Введение

В этой главе рассматриваются **указатели (pointers)** и тесно связанные с ними элементы языка C++:

- ♦ **традиционные массивы на основе указателей (pointer-based arrays)**, также называемые **встроенными массивами (built-in arrays)** или **массивами в стиле C (C-style arrays)**;
- ♦ **строки на основе указателей (pointer-based strings)**, также называемые **строками в стиле C (C-strings)**. Все эти элементы C++ унаследованы от языка C.

Неактуальность указателей в современных программах на C++

Указатели — это мощные средства программирования, но с ними трудно работать, что часто приводит к ошибкам. В современном C++ (начиная с C++11) есть другие средства, устраняющие необходимость в большинстве указателей. В новых проектах по разработке программного обеспечения, как правило, надо отдавать предпочтение:

- ♦ ссылкам или объектам «умных указателей» (раздел 11.5) вместо указателей;
- ♦ объектам `std::array` и `std::vector` (глава 6) вместо традиционных массивов;
- ♦ объектам классов `std::string` (главы 2 и 8) и `std::string_view` (раздел 8.11) вместо строк в стиле C.

Иногда указатели все еще нужны

При изучении, доработке и использовании устаревшего кода вы часто будете встречать указатели, традиционные массивы и строки в стиле C. Указатели нужны для того, чтобы:

- ♦ создавать и использовать динамические структуры данных (например, связанные списки, очереди, стеки и деревья), которые могут увеличиваться и уменьшаться во время выполнения программы (впрочем, программисты, как правило, предпочитают динамические контейнеры стандартной библиотеки C++: уже известные вам векторы, а также контейнеры, о которых вы узнаете в главе 13);
- ♦ обрабатывать аргументы командной строки, которые программа получает в виде традиционного массива строк в стиле C;
- ♦ передавать аргументы функциям в тех случаях, когда надо предусмотреть возможность передать `nullptr`¹ (указатель на «ничто», см. раздел 7.2.2); для сравнения, по ссылкам можно передавать только существующие объекты².

¹ C++ Core Guidelines, «F60: Prefer T* over T& When “No Argument” Is a Valid Option». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rf-nullptr-ref>.

² В современном C++ можно представить опциональный объект (такой, который может существовать или не существовать) с помощью `std::optional` (C++17). <https://en.cppreference.com/w/cpp/utility/optional>.

Рекомендации C++ Core Guidelines, связанные с указателями

Мы должны подчеркнуть, что в C++ Core Guidelines есть рекомендации избегать указателей, массивов на основе указателей и строк на основе указателей — это повышает надежность и безопасность кода. В частности, этот документ неоднократно рекомендует передавать функциям аргументы не по указателям, а по ссылкам¹.

Альтернативы указателям в C++20

Для тех случаев, когда по-прежнему нужны массивы в стиле C (например, для обработки аргументов командной строки), в C++20 добавлены средства, которые делают ваши программы более безопасными и надежными:

- ♦ функция `to_array` преобразует традиционные массивы в современные массивы `std::array`, имеющие целый ряд преимуществ, о которых мы рассказали в главе 6;
- ♦ класс `span` — более безопасный способ передачи традиционных массивов функциям. Он является итерируемым (перебираемым поэлементно), поэтому при обработке элементов вы можете использовать его в инструкции `for` для диапазона значений, не рискуя обратиться к несуществующему элементу за границами массива. Помимо этого, вы можете использовать `span` с имеющимися в стандартной библиотеке алгоритмами обработки контейнеров, например `accumulate` или `sort`. В разделе этой главы, посвященном готовым объектам (раздел 7.10), вы увидите, что `span` также работает с современными массивами `std::array` и векторами `std::vector`.

Главный вывод, который вам надо сделать после прочтения этой главы: старайтесь не включать в ваш код указатели, традиционные массивы и строки в стиле C. Но если вам все-таки придется работать с ними, вспомните о тех ценных возможностях, которые дают `to_array` и `span`.

Другие концепции, представленные в этой главе

Мы начнем с объявления и инициализации указателей, а затем продемонстрируем необходимые для работы с ними операторы `&` и `*`. В главе 5 рассматривалась передача аргументов по ссылке. В этой главе мы рассмотрим передачу аргументов по указателю, которая похожа на передачу по ссылке. После этого мы перейдем к традиционным массивам и покажем их тесную связь с указателями.

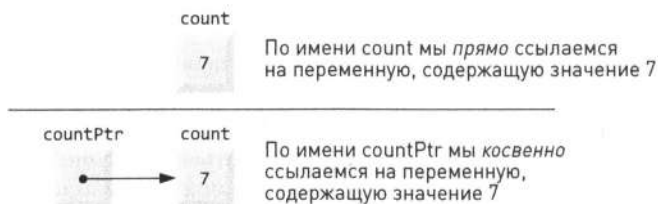
Мы рассмотрим все варианты применения квалификатора `const` к указателям и к тем данным, на которые они указывают. Мы представим вам оператор `sizeof` и покажем, как узнать, сколько байтов занимают в памяти вашего компьютера элементы данных основных типов и указатели. Затем мы приведем примеры выражений и арифметических операций с указателями.

¹ C++ Core Guidelines, «F: Functions». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#S-functions>.

В относительно старых программах часто встречаются строки в стиле C. Мы кратко рассмотрим их в этой главе. Вы узнаете, как обрабатывать аргументы командной строки: это простая задача, для выполнения которой в современном C++ по-прежнему нужны строки и массивы в стиле C.

7.2. Объявление и инициализация указателей

Указатель (pointer) — это переменная, значением которой является адрес памяти компьютера. В обычной переменной хранится определенное значение (например, целое число 7). А в указателе хранится тот адрес, по которому эта переменная расположена в памяти. Можно сказать, что по имени переменной мы прямо ссылаемся на значение 7, а по имени указателя — косвенно, как показано на диаграмме:



Когда мы ссылаемся на значение не прямо, а через указатель, это называется **косвенным обращением (indirection)**.

7.2.1. Объявление указателя

В следующей инструкции мы объявляем переменную `countPtr` типа `int*` (она является указателем на значение типа `int`). Это читается справа налево: «`countPtr` — указатель на `int`»:

```
int* countPtr{nullptr};
```

Символ `*` здесь не является оператором; он означает, что переменная `countPtr` справа от него является указателем. Мы взяли за правило добавлять префикс `Ptr` (сокращение от слова «pointer») к именам всех переменных-указателей. Такое имя позволяет понять, что перед нами указатель и с ним надо работать соответствующим образом.

7.2.2. Инициализация указателя

Каждый указатель надо инициализировать или конкретным адресом памяти, или значением `nullptr` (C++11). Указатель со значением `nullptr` указывает на «ничто» и называется **нулевым указателем (null pointer)**. С этого момента, когда мы говорим о нулевом указателе, мы имеем в виду указатель со значением `nullptr`. Обязательно инициализируйте все указатели, иначе они будут указывать на неизвестные или неинициализированные области памяти.

7.2.3. Нулевые указатели до C++11

В устаревших версиях C++ нулевые указатели имели значение `0` или `NULL`. Значение `NULL` определено в нескольких заголовках стандартной библиотеки для обозначения константы `0`, поэтому инициализация указателя значением `NULL` эквивалентна его инициализации значением `0`. Согласно C++ Core Guidelines, теперь для инициализации указателей нужно использовать `nullptr` вместо `0` или `NULL`¹.

7.3. Операторы для работы с указателями

Оператор `&` создает значение указателя, а оператор `*` выполняет разыменование указателя. В следующих разделах мы объясним, что это значит, и покажем, как использовать эти операторы.

7.3.1. Оператор взятия адреса (&)

Оператор взятия адреса (`&`) — это унарный оператор, который получает адрес памяти своего операнда. Предположим, мы сделали два объявления:

```
int y{5}; // Объявляем переменную y
int* yPtr{nullptr}; // Объявляем указатель yPtr
```

После этого следующая инструкция присвоит указателю `yPtr` адрес переменной `y`:

```
yPtr = &y; // Присваиваем указателю yPtr адрес переменной y в памяти
```

Иными словами, переменная `yPtr` теперь указывает на `y`. Указатель `yPtr` косвенно ссылается на значение переменной `y`.

Оператор `&` в предыдущей инструкции не является объявлением ссылочной переменной (как вы уже знаете, когда объявляется ссылочная переменная, символ `&` добавляется к имени типа и является частью типа). В выражении, подобном `&y`, символ `&` является оператором взятия адреса.

На диаграмме показано состояние двух ячеек памяти после выполнения предыдущей инструкции:



Сформированная указателем связь обозначена стрелкой, которая ведет от ячейки памяти с указателем `yPtr` к ячейке памяти с переменной `y`.

¹ C++ Core Guidelines, «ES.47: Use `nullptr` Rather Than `0` or `NULL`». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Res-nullptr>.

На следующей диаграмме показан другой пример состояния двух ячеек памяти. Переменная `y` типа `int` находится в памяти по адресу `600000`, а указатель `yPtr` — по адресу `500000`.



Операндом оператора `&` должно быть значение *lvalue*. Иными словами, оператор взятия адреса нельзя применять к литералам или к выражениям, которые приводят к временным значениям (например, к результатам вычислений).

7.3.2. Оператор косвенного обращения (*)

Результатом применения унарного оператора `*` к указателю является значение *lvalue*, представляющее объект, на который указывает указатель. Этот оператор называется **оператором косвенного обращения (indirection operator)** или **оператором разыменования (dereferencing operator)**. Если `yPtr` указывает на переменную `y`, а переменная `y` содержит 5 (как на предыдущих диаграммах), то следующая инструкция выведет на экран число 5:

```
std::cout << *yPtr << '\n';
```

То есть она работает так же, как уже знакомая вам инструкция:

```
std::cout << y << '\n';
```

Такое использование оператора косвенного обращения называется **разыменованием указателя (dereferencing a pointer)**. Разыменованный указатель можно использовать в качестве значения *lvalue* в инструкциях присваивания. Следующая инструкция присваивает значение 9 переменной `y`:

```
*yPtr = 9;
```

Здесь `*yPtr` — это значение *lvalue*, ссылающееся на переменную `y`. Аналогичным образом с помощью оператора косвенного обращения можно получить значение из входного потока:

```
cin >> *yPtr;
```

Эта инструкция поместит введенное значение в переменную `y`.

Неопределенное поведение

Разыменование неинициализированного указателя или указателя, который больше не указывает на объект, приводит к неопределенному поведению программы, которое может проявиться как фатальная ошибка во время выполнения. Неопределенное поведение также может исказить важные данные, то есть программа продолжит выполняться до обычного завершения, но с неправильными результатами. Злоумышленники могут пользоваться этой уязвимостью в программе, чтобы полу-

чать доступ к чужим данным, подменять их и заражать компьютеры вредоносным кодом^{1,2,3}. Использование результата разыменования нулевого указателя (`nullptr`), то есть указателя на «ничто», приводит к неопределенному поведению с теми же последствиями. Если вам необходимо работать с указателями, проверяйте перед каждым разыменованием, что указатель не является `nullptr`.

7.3.3. Использование операторов взятия адреса (&) и косвенного обращения (*)

На илл. 7.1 показан пример с операторами `&` и `*`, которые имеют третий по старшинству приоритет. Полная таблица приоритетов операторов приведена в приложении А. В этом примере адреса памяти выводятся оператором `<<` в виде шестнадцатеричных целых чисел (чисел с основанием 16). Выходные данные показывают, что адрес переменной `a` (строка 9) и значение указателя `aPtr` (строка 10) идентичны. Это подтверждает, что адрес переменной `a` действительно был присвоен указателю `aPtr` (строка 7). Выходные данные в строках 11–12 подтверждают идентичность значений `*aPtr` и `a`. Отображаемые адреса памяти зависят от компилятора и платформы. Обычно они меняются при каждом выполнении программы, поэтому вы, скорее всего, увидите другие адреса.

```
1 // fig07_01.cpp
2 // Пример работы с операторами & и *
3 #include <iostream>
4
5 int main() {
6     constexpr int a{7}; // Инициализируем a значением 7
7     const int* aPtr{&a}; // Инициализируем aPtr адресом переменной a
8
9     cout << "The address of a is " << &a
10         << "\nThe value of aPtr is " << aPtr;
11     cout << "\n\nThe value of a is " << a
12         << "\nThe value of *aPtr is " << *aPtr << '\n';
13 }
```

```
The address of a is 000000D649EFFF00
The value of aPtr is 000000D649EFFF00

The value of a is 7
The value of *aPtr is 7
```

Илл. 7.1 | Пример работы с операторами `&` и `*`

¹ «Undefined Behavior». Был доступен: 15 сентября 2023 г. https://en.wikipedia.org/wiki/Undefined_behavior.

² «Common Weakness Enumeration». CWE. Был доступен: 15 сентября 2023 г. <https://cwe.mitre.org/data/definitions/824.html>.

³ «Dangling Pointer». Был доступен: 15 сентября 2023 г. https://en.wikipedia.org/wiki/Dangling_pointer.

7.4. Передача аргументов по указателю

В C++ есть три способа передачи аргументов функции:

- ♦ **передача по значению (pass-by-value);**
- ♦ **передача по ссылке (pass-by-reference);**
- ♦ **передача по указателю (pass-by-pointer),** которую иногда называют «передачей по ссылке с аргументом-указателем».

В главе 5 мы рассмотрели первые два способа. Теперь мы объясним, как выполнять передачу по указателю. Подобно передаче по ссылке, передача по указателю позволяет изменять переменные в вызывающем коде, а также передавать в функцию большие объемы данных (большие объекты) без их копирования, то есть практически без нагрузки на компьютер. При вызове функции, получающей указатель, нужно передать ей адрес переменной, применив оператор взятия адреса (&) к имени этой переменной.

Пример передачи аргумента по значению

На илл. 7.2 и 7.3 вы видите два варианта программы с функциями, возводящими целое число в третью степень. На илл. 7.2 переменная `number` передается по значению (строка 12) в функцию `cubeByValue` (строки 17–19), которая возводит аргумент в третью степень и возвращает результат в `main` инструкцией `return` (строка 18)¹. Результат сохраняется в переменной `number` (строка 12), заменяя ее исходное значение.

```
1 // fig07_02.cpp
2 // Передача переменной по значению для возведения в третью степень
3 #include <fmt/format.h>
4 #include <iostream>
5
6 int cubeByValue(int n); // Прототип функции
7
8 int main() {
9     int number{5};
10
11     std::cout << fmt::format("Original value of number is {}\n", number);
12     number = cubeByValue(number); // Передаем number в функцию cubeByValue
13     std::cout << fmt::format("New value of number is {}\n", number);
14 }
15
16 // Возводим в куб значение аргумента типа int и возвращаем результат
17 int cubeByValue(int n) {
18     return n * n * n; // Выполняем вычисление и возвращаем результат
19 }
```

```
Original value of number is 5
New value of number is 125
```

Илл. 7.2 | Передача переменной по значению для возведения в третью степень

¹ Мы могли бы возвести число `n` в третью степень выражением `std::pow(n, 3)`.

Пример передачи аргумента по указателю

Пример на илл. 7.3 отличается тем, что переменная `number` передается функции `cubeByReference` по указателю (строка 13), то есть вызываемая функция получает в качестве аргумента не саму переменную, а указатель на нее: адрес переменной `number` в памяти компьютера. В функции `cubeByReference` (строки 18–20) для получения этого аргумента определен параметр `nPtr` (указатель на переменную типа `int`). Для возведения в третью степень того числа, на которое указывает `nPtr` (строка 19), эта функция использует разыменованный указатель `*nPtr` (псевдоним переменной `number` из `main`). Это вычисление напрямую изменяет значение переменной `number` в `main` (строка 10). Инструкцию в строке 19 можно сделать нагляднее, добавив круглые скобки:

```
*nPtr = (*nPtr) * (*nPtr) * (*nPtr); // Возведение *nPtr в третью степень
```

```
1 // fig07_03.cpp
2 // Передача переменной по указателю
3 // для возведения в третью степень
4 #include <fmt/format.h>
5 #include <iostream>
6
7 void cubeByReference(int* nPtr); // Прототип функции
8
9 int main() {
10     int number{5};
11
12     std::cout << fmt::format("Original value of number is {}\n", number);
13     cubeByReference(&number); // Передаем адрес переменной number
14     std::cout << fmt::format("New value of number is {}\n", number);
15 }
16
17 // Вычисляем третью степень *nPtr, напрямую изменяя переменную number
18 void cubeByReference(int* nPtr) {
19     *nPtr = *nPtr * *nPtr * *nPtr; // Возводим *nPtr в третью степень
20 }
```

```
Original value of number is 5
New value of number is 125
```

Илл. 7.3 | Передача переменной по указателю для возведения в третью степень

Любая функция, получающая в качестве аргумента адрес памяти (то есть указатель), должна определить для него параметр соответствующего типа. Например, заголовок функции `cubeByReference` (строка 18) определяет, что функция получает в качестве аргумента указатель на переменную `int`, сохраняет этот адрес в `nPtr` и не возвращает никакого значения.

При передаче по указателю фактически передается указатель по значению

Передача переменной по указателю реализована в C++ таким образом: переменная-указатель передается вызываемой функции по значению, то есть копируется в со-

ответствующий параметр функции (в параметр, предназначенный для получения указателя). Затем вызываемая функция может разыменовать указатель для прямого доступа к переменной вызывающего кода, что равнозначно передаче по ссылке.

Сравнительный анализ передачи по значению и передачи по указателю

На илл. 7.4 и 7.5 анализируются программы, показанные на илл. 7.2 и 7.3. Рядом с выражениями и именами переменных изображены прямоугольники со значениями, полученными на соответствующих шагах программ. В правом столбце каждой диаграммы показаны функции `cubeByValue` (илл. 7.2) и `cubeByReference` (илл. 7.3) на тех шагах, когда они выполняются.

7.5. Традиционные массивы в стиле C

Здесь мы рассмотрим традиционные массивы, унаследованные от языка C. Как и современные массивы `std::array`, они тоже являются структурами данных, имеющими фиксированный размер. Мы включили в книгу этот раздел, потому что вы встретите традиционные массивы в устаревшем коде.

Напомним, что при создании новых приложений надо, как правило, использовать современные массивы `std::array` и векторы `std::vector`, которые повышают надежность и безопасность приложений. Эта рекомендация обусловлена, в частности, тем, что объекты `std::array` и `std::vector` всегда «знают» (то есть хранят) свой собственный размер, в отличие от традиционных массивов. Если вы будете работать с кодом, в котором есть традиционные массивы, вы можете преобразовывать их в `std::array` с помощью функции `to_array` (раздел 7.6) или безопасно обрабатывать с помощью `span` (раздел 7.10). Иногда традиционные массивы по-прежнему необходимы, например, при обработке аргументов командной строки (раздел 7.11).

7.5.1. Объявление традиционного массива и доступ к его элементам

Объявление традиционного массива похоже по сути на объявление `std::array`, но отличается синтаксически. Чтобы объявить традиционный массив, вы должны указать тип и количество его элементов. Например, следующая инструкция объявляет массив из пяти элементов типа `int`:

```
int c[5]; // c - массив из пяти элементов int
```

Для доступа к элементам традиционного массива используется оператор индекса (`[]`). В главе 6 мы отмечали, что он не проверяет границы массивов современного типа `std::array`. И с традиционными массивами этот оператор работает аналогично: проверка границ не выполняется автоматически, поэтому вы должны очень внимательно следить за правильностью индексов! Как вы помните, в `std::array` есть функция `at`, проверяющая границы. Традиционные массивы не имеют подобной функции, но для работы с ними вы можете использовать аналогичную функцию `gsl::at` из библиотеки `GSL` (Guidelines Support Library,

Шаг 1. Перед вызовом функции `cubeByValue` из `main`:

```
int main() {
    int number{5};
    number = cubeByValue(number);
}
```

number: 5

Шаг 2. После вызова функции `cubeByValue`:

```
int main() {
    int number{5};
    number = cubeByValue(number);
}
```

number: 5

```
int cubeByValue( int n ) {
    return n * n * n;
}
```

n: 5

Шаг 3. После возведения в куб параметра `n` в функции `cubeByValue`, перед возвращением результата в `main`:

```
int main() {
    int number{5};
    number = cubeByValue(number);
}
```

number: 5

```
int cubeByValue(int n) {
    return 125 * n;
}
```

n: 5

Шаг 4. После возвращения результата из `cubeByValue` в `main`, до сохранения результата в переменной `number`:

```
int main() {
    int number{5};
    number = cubeByValue(number);
}
```

125

number: 5

Шаг 5. После сохранения результата в переменной `number`:

```
int main() {
    int number{5};
    number = cubeByValue(number);
}
```

125

number: 125

Шаг 1. Перед вызовом функции `cubeByReference` из `main`:

```
int main() {
    int number{5};
    cubeByReference(&number);
}
```

number
5

Шаг 2. После вызова функции `cubeByReference`, перед возведением в куб значения `*nPtr`:

```
int main() {
    int number{5};
    cubeByReference(&number);
}
```

number
5

```
void cubeByReference( int* nPtr ) {
    *nPtr = *nPtr * *nPtr * *nPtr;
}
```

nPtr

Вызов инициализирует этот указатель

Шаг 3. Перед сохранением в `*nPtr` результата выражения `5 * 5 * 5`:

```
int main() {
    int number{5};
    cubeByReference(&number);
}
```

number
5

```
void cubeByReference(int* nPtr) {
    *nPtr = 125 * *nPtr * *nPtr;
}
```

nPtr

Шаг 4. После сохранения в `*nPtr` числа 125, до возврата управления в `main`:

```
int main() {
    int number{5};
    cubeByReference(&number);
}
```

number
125

```
void cubeByReference(int* nPtr) {
    *nPtr = 125 * *nPtr * *nPtr;
}
```

nPtr

Вызываемая функция изменяет переменную в вызывающем блоке

Шаг 5. После возврата управления из `cubeByReference` в `main`:

```
int main() {
    int number{5};
    cubeByReference(&number);
}
```

number
125

см. раздел 5.6.3), которая получает в качестве аргументов имя массива и индекс. Первым аргументом этой функции должно быть имя массива (а не указатель на первый элемент массива).

7.5.2. Инициализация традиционного массива

Вы можете инициализировать элементы традиционного массива списком инициализации, например:

```
int n[5]{50, 20, 30, 10, 40};
```

Эта инструкция создает и инициализирует массив из пяти целых чисел. Если количество инициализаторов в списке меньше, чем количество элементов массива, остальные элементы будут **инициализированы значением по умолчанию (value initialized)**: для основных числовых типов — 0, для типа `bool` — `false`, для указателей — `nullptr`, а объекты будут инициализированы в соответствии с определениями их классов (мы расскажем об этом в главе 9). Если количество инициализаторов превысит размер массива, возникнет ошибка компиляции.

Компилятор умеет определять размер массива на основании списка инициализаторов. Следующее объявление создает массив из пяти элементов:

```
int n[] {50, 20, 30, 10, 40};
```

7.5.3. Передача традиционного массива функции

Когда традиционный массив передается функции, его имя неявно преобразуется в указатель на первый элемент массива — это называется **разложением массива (decaying to a pointer)**. Например, имя массива `n` из предыдущей инструкции неявно преобразуется в `&n[0]`, то есть в указатель на элемент со значением 50. Чтобы передать адрес массива функции, вам не нужен оператор взятия адреса (`&`), вы просто передаете имя массива. Как было показано в разделе 7.4, функция, получающая указатель на переменную в вызывающем коде, может изменять эту переменную. Традиционный массив устроен аналогично: вызываемая функция может изменять его элементы прямо в вызывающем коде, если ее параметр, получающий массив в качестве аргумента, объявлен без квалификатора `const`, защищающего массив от изменений.

7.5.4. Объявление параметров функции, обрабатывающей традиционный массив

В заголовке функции, предназначенной для обработки традиционных массивов, обычно объявлены два параметра: для имени массива и для размера массива. Приведем пример:

```
int sumElements(const int values[], size_t numberOfElements)
```

Первым аргументом этой функции должен быть одномерный массив констант типа `int`. В отличие от `std::array` и `std::vector`, традиционный массив «не знает»

своего размера, поэтому вы должны сообщить его размер той функции, которую вызываете для его обработки.

Этот же заголовок можно написать и по-другому:

```
int sumElements(const int* values, size_t numberOfElements)
```

Компилятор не делает различий между функцией, получающей указатель, и функцией, получающей имя традиционного массива. В процессе компиляции (как говорят программисты, «под капотом» компилятора) фрагмент кода `const int values[]` преобразуется в `const int* values`. Поэтому функция должна каким-то образом «понять», что вы передали ей указатель не на единственную переменную, а на первый элемент массива.

Согласно C++ Core Guidelines, при создании новых программ не следует передавать традиционные массивы функциям по указателю¹. Вместо этого надо передавать такие массивы через объекты класса `span` (C++20), потому что они хранят не только указатель на первый элемент массива, но и размер массива. Мы рассмотрим класс `span` в разделе 7.10, и вы увидите, что это превосходный способ передачи традиционных массивов функциям.

7.5.5. Пример с функциями `sort`, `begin` и `end` стандартной библиотеки (C++11)

В разделе 6.12 мы сортировали массив с именем `colors` (`std::array`), содержащий элементы типа `string`, таким образом:

```
std::sort(std::begin(colors), std::end(colors));
```

Здесь функции `begin` и `end` возвращают начало и конец массива, то есть сортируется весь массив `colors`. Функции `sort`, `begin` и `end` (а также многие другие функции стандартной библиотеки C++) могут применяться и к традиционным массивам. Например, отсортировать массив `n` (раздел 7.5.2) можно так:

```
std::sort(std::begin(n), std::end(n)); // Сортируем традиционный массив n
```

Если функции `begin` и `end` применяются к традиционному массиву, они работают только с его именем, а не с указателем на его первый элемент. Подчеркнем, что вы должны передавать традиционные массивы функциям, используя класс `span` (C++20), который мы рассмотрим в разделе 7.10.

7.5.6. Недостатки традиционных массивов

Традиционные массивы имеют несколько недостатков, ограничивающих их применение:

¹ C++ Core Guidelines, «I.13: Do Not Pass an Array as a Single Pointer». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Ri-array>.

- ✦ Их нельзя сравнивать операторами равенства и сравнения. Если вам надо сравнить два таких массива, придется по очереди сравнивать в цикле все их элементы. Если у вас есть два традиционных массива целых чисел с именами `array1` и `array2`, условие `array1 == array2` всегда будет ложным, даже если содержимое этих массивов идентично. Дело в том, что при таком сравнении выполняется разложение массивов, то есть их имена преобразуются в константные указатели на первые элементы. Разумеется, первые элементы сравниваемых массивов находятся в разных ячейках памяти.
- ✦ Невозможно присвоить сразу все элементы одного традиционного массива другому, выполнив одну простую операцию вроде `builtInArray1 = builtInArray2`.
- ✦ Они не знают собственного размера, поэтому любая функция, обрабатывающая традиционный массив, должна получать в качестве аргументов его имя и его размер.
- ✦ Они не выполняют автоматическую проверку границ! Каждое выражение, влияющее на обработку элементов традиционных массивов, вы должны составлять и проверять очень тщательно, чтобы ни один индекс не вышел за границу какого-либо массива.

7.6. Функция `to_array` (C++20), преобразующая традиционные массивы в `std::array`

Если вы работаете в сфере ИТ, вам иногда может понадобиться устаревший код C++, в котором есть традиционные массивы. В соответствии с рекомендациями C++ Core Guidelines вы должны по возможности использовать вместо традиционных массивов современные массивы `std::array` и векторы `std::vector`, которые существенно безопаснее и не превращаются в указатели при передаче функциям¹. Функция `std::to_array` (C++20, заголовок `<array>`)² позволит вам легко создать современный массив `std::array` из традиционного массива или из списка инициализаторов. На илл. 7.6 показан пример работы с функцией `to_array`. Содержимое каждого массива `std::array` выводится на экран обобщенным лямбда-выражением (строки 9–15). Как вы уже знаете, если указать `auto` вместо типа параметра лямбды, компилятор автоматически определит тип на основе контекста, в котором появляется лямбда. Именно так в нашей программе определяется тип элементов обрабатываемого лямбдой массива `std::array`.

¹ C++ Core Guidelines, «SI.con.1: Prefer Using STL array or vector Instead of a C array». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rsl-arrays>.

² Zhihao Yuan, «to_array from LFTS with Updates». 17 июля 2019 г. Был доступен: 15 сентября 2023 г. <https://wg21.link/p0325>.

```
1 // fig07_06.cpp
2 // Создание массивов std::array с помощью функции to_array (C++20)
3 #include <fmt/format.h>
4 #include <iostream>
5 #include <array>
6
7 int main() {
8     // Обобщенная лямбда, отображающая элементы массива
9     const auto display{
10         [](const auto& items) {
11             for (const auto& item : items) {
12                 std::cout << fmt::format("{} ", item);
13             }
14         }
15     };
16
17     const int values1[]{10, 20, 30};
18
19     // Создаем std::array из традиционного массива
20     const auto array1{std::to_array(values1)};
21
22     std::cout << fmt::format("array1.size() = {}\n", array1.size())
23         << "array1: ";
24     display(array1); // Вызываем лямбду для отображения элементов
25
26     // Создаем std::array из списка инициализаторов
27     const auto array2{std::to_array({1, 2, 3, 4})};
28     std::cout << fmt::format("\n\narray2.size() = {}\n", array2.size())
29         << "array2: ";
30     display(array2); // Вызываем лямбду для отображения элементов
31
32     std::cout << '\n';
33 }
```

```
array1.size() = 3
array1: 10 20 30

array2.size() = 4
array2: 1 2 3 4
```

Илл. 7.6 | Создание массивов `std::array` с помощью функции `to_array` [C++20]

Создание `std::array` из традиционного массива

Строка 20 создает массив `std::array`, состоящий из трех элементов типа `int`, путем их копирования из традиционного массива `values1`. Мы используем ключевое слово `auto`, чтобы компилятор автоматически определил тип и размер `array1`. Если бы мы явно объявили тип и размер, то в случае их несоответствия возвращаемому значению функции `to_array` возникла бы ошибка компиляции. Мы присваиваем результат переменной `array1`. В строках 22 и 24 выводятся на экран размер и содержимое массива `std::array`, что подтверждает корректное преобразование.

Создание std::array из списка инициализаторов

В строке 27 показано, что функция `to_array` может создать массив `std::array` из списка инициализаторов. В строках 28 и 30 выводятся на экран размер и содержимое массива, что подтверждает его корректное создание.

7.7. Использование const с указателями и данными, на которые они указывают

В этом разделе мы расскажем, как использовать квалификатор `const` с указателями, чтобы соблюдать принцип наименьших привилегий. В главе 5 мы объяснили, как устроена передача по значению: значение аргумента копируется в параметр функции. Если эта копия изменится в вызываемой функции, исходное значение в вызывающем коде останется неизменным. Но иногда бывает нужно сделать так, чтобы даже копия в вызываемой функции не могла изменяться.

В тех случаях, когда какое-либо значение передается функции и не должно изменяться в ее теле, сделайте соответствующий параметр функции константным, то есть объявите его с квалификатором `const`. Перед использованием функции внимательно проверьте ее прототип, чтобы определить, какие параметры она должна изменять, а какие не должна.

Есть четыре способа объявления указателей:

- ♦ неконстантный указатель на неконстантные данные;
- ♦ неконстантный указатель на константные данные (илл. 7.7);
- ♦ константный указатель на неконстантные данные (илл. 7.8);
- ♦ константный указатель на константные данные (илл. 7.9).

Эти способы обеспечивают разные уровни привилегий доступа к данным.

7.7.1. Неконстантный указатель на неконстантные данные

Этот способ дает максимальные привилегии:

- ♦ данные можно изменять через разыменованный указатель;
- ♦ указатель можно изменять, чтобы он указывал на другие данные.

В объявлении такого указателя (например, `int* countPtr`) нет квалификатора `const`.

7.7.2. Неконстантный указатель на константные данные

Этот способ дает следующие привилегии:

- ♦ указатель можно изменять, чтобы он указывал на другие данные соответствующего типа;
- ♦ через указатель нельзя изменять данные.

Объявление такого указателя содержит квалификатор `const` слева от типа указателя¹:

```
const int* countPtr;
```

Это объявление читается справа налево: «`countPtr` — указатель на константу типа `int`» или «`countPtr` — неконстантный указатель на целочисленную константу».

На илл. 7.7 показана ошибка компиляции, возникающая при попытке изменить константные данные с помощью неконстантного указателя.

```
1 // fig07_07.cpp
2 // Попытка изменить константные данные
3 // через неконстантный указатель
4
5 int main() {
6     int y{0};
7     const int* yPtr(&y);
8     *yPtr = 100; // Ошибка: нельзя изменить объект, объявленный константой
9 }
```

Сообщение компилятора GNU C++ об ошибке:

```
fig07_07.cpp: In function 'int main()':
fig07_07.cpp:8:10: error: assignment of read-only location '* yPtr'
8 | *yPtr = 100; // error: cannot modify a const object
  | ~~~~~^~~~~~
```

Илл. 7.7 | Попытка изменить константные данные через неконстантный указатель

Всегда передавайте аргументы основных типов (`int`, `double` и т. д.) по значению, за исключением тех случаев, когда вызываемая функция должна напрямую изменять значения в вызывающем коде. Это еще один пример принципа наименьших привилегий. Если большие объекты не должны изменяться вызываемой функцией, передавайте их по указателю на константные данные или по ссылке на константные данные (лучше по ссылке). Это дает преимущество в быстродействии благодаря отсутствию операций копирования, выполняемых при передаче по значению. Передача больших объектов по ссылке на константные данные или по указателю на константные данные так же безопасна, как передача по значению.

7.7.3. Константный указатель на неконстантные данные

Этот способ дает следующие привилегии:

- ♦ указатель всегда указывает на определенный адрес памяти;
- ♦ данные, которые находятся по этому адресу, можно изменять через указатель.

¹ Некоторые программисты предпочитают записывать это так: `int const* countPtr`; (читается справа налево: «`countPtr` — это указатель на константу типа `int`»).

Константные указатели (объявленные как `const`) надо инициализировать при объявлении. Если указатель является параметром функции, он инициализируется тем указателем, который передается функции. При каждом вызове функции этот параметр инициализируется заново.

На илл. 7.8 показана ошибка компиляции, возникающая при попытке изменить константный указатель. В строке 9 объявляется указатель `ptr` с типом `int* const`. Это объявление читается справа налево: «`ptr` — константный указатель на неконстантную переменную типа `int`». Указатель инициализируется адресом целочисленной переменной `x`. В строке 12 мы пытаемся присвоить указателю `ptr` адрес переменной `y`, но компилятор выдает сообщение об ошибке. Строка 11, в которой переменной `*ptr` присваивается значение 7, не вызывает ошибки. Неконстантную переменную, на которую ссылается `ptr`, можно изменять с помощью разыменованного указателя `ptr`, даже если сам указатель был объявлен как константа.

```
1 // fig07_08.cpp
2 // Попытка изменить константный указатель на неконстантные данные
3
4 int main() {
5     int x, y;
6
7     // ptr - константный указатель на значение int, которое можно изменять
8     // через ptr, но он всегда указывает на определенный адрес памяти
9     int* const ptr{&x}; // Константный указатель должен быть инициализирован
10
11     *ptr = 7; // Допустимо: *ptr не является константой
12     ptr = &y; // Ошибка: ptr - константа, ей нельзя присвоить другой адрес
13 }
```

Сообщение компилятора Microsoft Visual C++ об ошибке:

```
error C3892: 'ptr': you cannot assign to a variable that is const
```

Илл. 7.8 | Попытка изменить константный указатель на неконстантные данные

7.7.4. Константный указатель на константные данные

Этот способ дает минимальные привилегии:

- ♦ указатель всегда указывает на определенный адрес памяти;
- ♦ данные, которые находятся по этому адресу, нельзя изменять через указатель.

На илл. 7.9 приведен пример: переменная-указатель `ptr` объявляется в строке 12 с типом `const int* const`. Это объявление читается справа налево: «`ptr` — константный указатель на константу типа `int`». После исходного кода показаны сообщения компилятора Clang из Xcode: об ошибке при попытке изменить данные, на которые ссылается `ptr` (строка 16), и об ошибке при попытке изменить адрес, сохраненный в переменной-указателе (строка 17). В строке 14 нет ошибки, поскольку эта строка ничего не изменяет: ни указатель, ни данные, на которые он указывает.


```

1 // fig07_09.cpp
2 // Попытка изменить константный указатель на константные данные
3 #include <iostream>
4
5 int main() {
6     int x{5};
7     int y{6};
8
9     // ptr - константный указатель на константу int,
10    // он всегда указывает на определенный адрес памяти,
11    // и через этот указатель нельзя изменять расположенную там константу
12    const int* const ptr{&x};
13
14    std::cout << *ptr << '\n';
15
16    *ptr = 7; // Ошибка: *ptr - константа, ее значение нельзя изменить
17    ptr = &y; // Ошибка: ptr - константа, ей нельзя присвоить другой адрес
18 }

```

Сообщение компилятора Clang (из Xcode) об ошибке:

```

fig07_09.cpp:16:9: error: read-only variable is not assignable
    *ptr = 7; // error: *ptr is const; cannot assign new value
    ~~~~~ ^
fig07_09.cpp:17:8: error: cannot assign to variable 'ptr' with const-qualified
type 'const int *const'
    ptr = &y; // error: ptr is const; cannot assign new address
    ~~~~ ^

```

Илл. 7.9 | Попытка изменить константный указатель на константные данные

7.8. Оператор sizeof

Унарный оператор `sizeof` определяет **во время компиляции программы** размер своего операнда, то есть количество байтов, занимаемых им в памяти. Операндом может быть традиционный массив, тип данных, переменная или константа. При применении к имени массива (илл. 7.10¹, строка 13) оператор `sizeof` возвращает суммарное количество байтов массива в виде значения типа `size_t`. В операционной системе нашего компьютера, на котором мы компилировали этот код, каждая переменная типа `double` хранится в 8 байтах памяти. Массив `numbers` содержит 20 таких элементов (строка 10), поэтому занимает в памяти 160 байтов. При применении к указателю (строка 21) оператор `sizeof` возвращает его размер в байтах, в нашей системе это 4 байта.

¹ Это учебный пример, демонстрирующий работу оператора `sizeof`. Если вы используете инструменты статического анализа кода (например, C++ Core Guidelines Checker в Microsoft Visual Studio), вы получите предупреждения, поскольку не следует передавать функциям традиционные массивы.

```

1 // fig07_10.cpp
2 // Оператор sizeof возвращает количество байтов,
3 // занимаемых в памяти традиционным массивом
4 #include <fmt/format.h>
5 #include <iostream>
6
7 size_t getSize(double* ptr); // Прототип функции
8
9 int main() {
10     double numbers[20]; // 20 переменных double (160 байтов в нашей системе)
11
12     std::cout << fmt::format("Number of bytes in numbers is {}\n\n",
13         sizeof(numbers));
14
15     std::cout << fmt::format("Number of bytes returned by getSize is {}\n",
16         getSize(numbers));
17 }
18
19 // Возвращаем размер ptr
20 size_t getSize(double* ptr) {
21     return sizeof(ptr);
22 }

```

```

The number of bytes in the array is 160
The number of bytes returned by getSize is 4

```

Илл. 7.10 | Оператор `sizeof` возвращает количество байтов, занимаемых в памяти традиционным массивом

Определение размеров основных типов, традиционных массивов и указателей

На илл. 7.11 показано, как определять оператором `sizeof` размеры основных типов данных. Выходные данные программы мы получили с помощью компилятора Clang из Xcode. На других платформах выходные данные могут отличаться. Когда мы запускали эту программу в нашей версии Windows, размер типа `long` был равен 4 байтам, а размер типа `long long` — 8 байтам. В нашей версии macOS размеры обоих типов равны 8 байтам. В строках 7–15 пустой список инициализаторов `{}` (C++11) неявно инициализирует каждую переменную значением `0`¹.

```

1 // fig07_11.cpp
2 // Определение размеров стандартных типов данных оператором sizeof
3 #include <fmt/format.h>
4 #include <iostream>
5
6 int main() {

```

Окончание ⇒

¹ В строке 16 мы используем `const`, а не `constexpr`, чтобы предотвратить ошибку несоответствия типов на этапе компиляции. Имя традиционного массива (строка 15) разыменовывается в `const int*`, поэтому мы должны объявить `ptr` с этим типом.

```

7  constexpr char c{}; // Переменная char
8  constexpr short s{}; // Переменная short
9  constexpr int i{}; // Переменная int
10 constexpr long l{}; // Переменная long
11 constexpr long long ll{}; // Переменная long long
12 constexpr float f{}; // Переменная float
13 constexpr double d{}; // Переменная double
14 constexpr long double ld{}; // Переменная long double
15 constexpr int array[20]{}; // Традиционный массив
16 const int* const ptr(array); // Переменная int*
17
18 std::cout << fmt::format("sizeof c = {}\tsizeof(char) = {}\n",
19     sizeof c, sizeof(char));
20 std::cout << fmt::format("sizeof s = {}\tsizeof(short) = {}\n",
21     sizeof s, sizeof(short));
22 std::cout << fmt::format("sizeof i = {}\tsizeof(int) = {}\n",
23     sizeof i, sizeof(int));
24 std::cout << fmt::format("sizeof l = {}\tsizeof(long) = {}\n",
25     sizeof l, sizeof(long));
26 std::cout << fmt::format("sizeof ll = {}\tsizeof(long long) = {}\n",
27     sizeof ll, sizeof(long long));
28 std::cout << fmt::format("sizeof f = {}\tsizeof(float) = {}\n",
29     sizeof f, sizeof(float));
30 std::cout << fmt::format("sizeof d = {}\tsizeof(double) = {}\n",
31     sizeof d, sizeof(double));
32 std::cout << fmt::format("sizeof ld = {}\tsizeof(long double) = {}\n",
33     sizeof ld, sizeof(long double));
34 std::cout << fmt::format("sizeof array = {}\n", sizeof array);
35 std::cout << fmt::format("sizeof ptr = {}\n", sizeof ptr);
36 }

```

sizeof c = 1	sizeof(char) = 1
sizeof s = 2	sizeof(short) = 2
sizeof i = 4	sizeof(int) = 4
sizeof l = 8	sizeof(long) = 8
sizeof ll = 8	sizeof(long long) = 8
sizeof f = 4	sizeof(float) = 4
sizeof d = 8	sizeof(double) = 8
sizeof ld = 16	sizeof(long double) = 16
sizeof array = 80	
sizeof ptr = 8	

Илл. 7.11 | Определение размеров стандартных типов данных оператором `sizeof`

Количество байтов, занимаемых в памяти элементами конкретного типа данных, может варьироваться в зависимости от платформы и компилятора. Если вы будете писать программу, работоспособность которой зависит от размеров типов данных, рассмотрите возможность использования **целочисленных типов фиксированного размера (fixed-size integer types)** (C++11, заголовок `<cstdint>`). Их полный список приведен здесь:

<https://en.cppreference.com/w/cpp/types/integer>

Оператор `sizeof` можно применять к любому выражению и к любому имени типа. Если операндом является имя переменной или выражение, оператор `sizeof` возвращает размер соответствующего типа. Круглые скобки нужны только в том случае, если в качестве операнда указан тип (например, `int`). Если операндом является выражение, круглые скобки не нужны. Не забывайте, что `sizeof` — это оператор этапа компиляции, поэтому его операнд не будет вычисляться во время выполнения программы.

7.9. Выражения и арифметические операции с указателями

В C++ можно выполнять арифметические операции с указателями. В этом разделе мы рассмотрим операторы с операндами-указателями и примеры работы с ними.

Арифметические операции возможны только с указателями на элементы традиционных массивов. Такие операции вам почти наверняка встретятся в устаревшем коде. Но согласно рекомендациям C++ Core Guidelines, любой указатель должен указывать только на единичный объект (а не на массив)¹ и вы не должны включать в новый код **арифметические операции с указателями**, поскольку они значительно повышают риск ошибок². Если вам нужно обработать традиционный массив, используйте класс `span` (C++20), которому посвящен раздел 7.10.

Перечислим допустимые арифметические операции с указателями:

- ♦ инкремент (`++`) и декремент (`--`);
- ♦ прибавление целого числа к указателю (`+` или `+=`) и вычитание целого числа из указателя (`-` или `-=`);
- ♦ вычитание одного указателя из другого указателя того же типа.

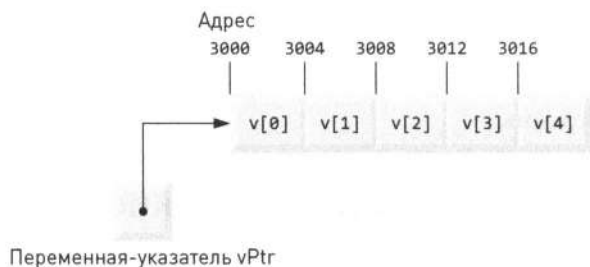
Результаты этих операций зависят от размера элементов массивов, на которые ссылаются указатели, то есть от количества байтов, занимаемых каждым элементом в памяти. Поэтому арифметические операции с указателями являются платформенно-зависимыми.

В большинстве современных настольных и мобильных компьютеров целые числа являются 4-байтовыми (32-разрядными) или 8-байтовыми (64-разрядными). Но среди миллиардов устройств «интернета вещей» (IoT) есть маломощные 8-разрядные и 16-разрядные компьютеры, в которых целые числа являются 2-байтовыми (согласно стандарту C++, это минимально возможный размер типа `int`). Предположим, что в вашей программе есть традиционный массив `int v[5]`, который начинается в памяти компьютера с адреса 3000, а числа типа `int` на данном компьютере являются 4-байтовыми. Также предположим, что вы инициализиро-

¹ C++ Core Guidelines, «ES.42: Keep Use of Pointers Simple and Straightforward». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Res-ptr>.

² C++ Core Guidelines «Pro.bounds: Bounds Safety Profile». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#SS-bounds>.

вали указатель `vPtr` ссылкой на элемент `v[0]`, то есть значение переменной `vPtr` равно `3000`. Это состояние показано на схеме:



Поскольку имя традиционного массива неявно преобразуется в адрес его нулевого элемента, вы могли бы для получения этого состояния инициализировать переменную `vPtr` (чтобы она указывала на `v`) любой из двух инструкций:

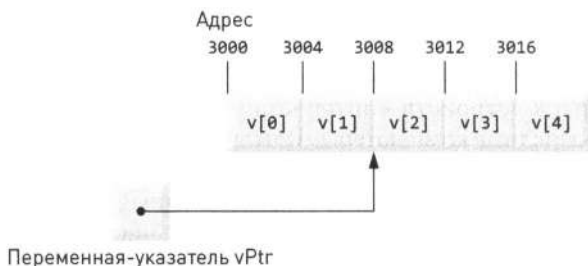
```
int* vPtr{v};
int* vPtr{&v[0]};
```

7.9.1. Прибавление и вычитание целых чисел

Теперь прибавим к `vPtr` число 2. В обычной арифметике выражение `3000 + 2` дает результат `3002`. Но это не относится к арифметическим операциям с указателями. Если вы прибавите целое число к указателю (или вычтете целое число из указателя), указатель увеличится (или уменьшится) на это число, умноженное на размер типа данных, с которым работает указатель. Как вы уже знаете, размер типа данных зависит от платформы. В нашем примере инструкция:

```
vPtr += 2;
```

присвоит указателю `vPtr` значение `3008` ($3000 + 2 * 4$), поскольку каждое число типа `int` занимает в памяти 4 байта. Теперь `vPtr` указывает на элемент `v[2]`:



Теперь предположим, что значение `vPtr` равно `3016`, то есть `vPtr` указывает на элемент `v[4]`. В этом случае выражение:

```
vPtr -= 4;
```

присвоит указателю `vPtr` значение `3000`, то есть вернет указатель к началу массива. Если вам нужно увеличивать или уменьшать указатель на 1, то есть «передвигать» его вперед или назад на ближайший соседний элемент массива, вы можете делать это операторами инкремента (`++`) и декремента (`--`). Любое из выражений:

```
++vPtr;  
vPtr++;
```

«передвигает» указатель вперед, чтобы он указывал на следующий элемент массива. Любое из выражений:

```
--vPtr;  
vPtr--;
```

«передвигает» указатель назад, чтобы он указывал на предыдущий элемент массива.

В арифметических операциях с указателями нет автоматической проверки границ массивов! Согласно C++ Core Guidelines, в современных программах вместо этих операций надо использовать класс `std::span` (раздел 7.10). Если вам все-таки придется выполнять операции сложения или вычитания с каким-либо указателем, вы должны быть на сто процентов уверены, что после каждой такой операции указатель остается в границах массива, то есть указывает на существующий элемент массива. Как вы скоро увидите, класс `std::span` выполняет проверку границ, что поможет вам избежать ошибок.

7.9.2. Вычитание одного указателя из другого

Указатели, указывающие на один и тот же традиционный массив, можно вычитать друг из друга. Например, если `vPtr` содержит адрес `3000`, а `v2Ptr` содержит адрес `3008`, инструкция:

```
x = v2Ptr - vPtr;
```

присвоит переменной `x` количество элементов традиционного массива от `vPtr` до `v2Ptr` — в нашем примере это 2. Ни в коем случае нельзя предполагать, что две переменные одного и того же типа хранятся в памяти последовательно, если только они не являются смежными элементами традиционного массива. Вычитание или сравнение двух указателей, если они не указывают на элементы одного и того же традиционного массива, — это логическая ошибка.

7.9.3. Присваивание указателя

Указатель может быть присвоен другому указателю, если они оба имеют один и тот же тип¹. Кроме того, любой указатель на основной тип или на тип класса можно присвоить указателю `void*` (то есть указателю на `void`) без приведения типа. Указатель `void*` может представлять любой тип указателя. Однако указатель типа

¹ Напомним, что константные указатели не могут быть изменены.

`void*` нельзя непосредственно присвоить указателю другого типа — сначала надо привести указатель типа `void*` к соответствующему типу с помощью `static_cast`.

7.9.4. Невозможность разыменования указателя `void*`

Указатель `void*` не может быть разыменован. Например, компилятор «знает», что `int*` указывает на 4-байтовый фрагмент памяти на компьютере с 4-байтовыми целыми числами. В этом случае разыменование `int*` создает значение *lvalue*, которое является псевдонимом для тех четырех байтов в памяти, которые хранят конкретное число типа `int`. Однако `void*` содержит адрес памяти для неизвестного типа данных. Вы не можете использовать `void*` в арифметических операциях и не можете его разыменовывать, потому что компилятору неизвестен тип (размер) данных, на которые он указывает.

Перечислим допустимые операции с указателями `void*`:

- ♦ сравнение указателя `void*` с другим указателем;
- ♦ приведение указателя `void*` к другому типу указателя;
- ♦ преобразование другого типа указателя в указатель `void*`.

Все остальные операции с указателями `void*` приведут к ошибкам компиляции.

7.9.5. Сравнение указателей

Указатели можно сравнивать операторами равенства и сравнения. Но никакие сравнения (`<`, `<=`, `>` и `>=`) не имеют смысла, если указатели относятся к разным традиционным массивам. Сравнимые указатели должны указывать на один и тот же традиционный массив. При сравнении указателей сравниваются адреса памяти, хранящиеся в указателях. Например, сравнением двух указателей, указывающих на один и тот же массив, можно определить, что один из них указывает на элемент с большим индексом, чем другой. Операции сравнения часто нужны для того, чтобы выяснить, не является ли указатель нулевым (содержащим значение `nullptr`).

7.10. Готовые объекты: доступ к контейнерам через `span` (C++20)

Этот раздел посвящен работе с объектами класса `span` (C++20). Класс `span` (заголовков `<span>`) позволяет функциям обрабатывать элементы контейнеров: традиционных массивов, современных массивов `std::array` и векторов `std::vector`. Объект класса `span` является **представлением (view)** контейнера. Он дает обрабатывающей функции доступ к элементам контейнера, но не имеет своей копии этих элементов.

Мы уже обсуждали, как традиционный массив превращается в указатель, когда его имя передается функции. При этом, в частности, теряется информация о размере массива (несмотря на то, что вы указали размер, когда объявляли массив).

Вы убедились в этом на примере с оператором `sizeof` (илл. 7.10). В C++ Core Guidelines рекомендуется передавать традиционные массивы функциям через объекты `span`¹, содержащие не только указатель на первый элемент массива, но и размер массива. Следующая программа (илл. 7.12), разделенная на части для удобства обсуждения, демонстрирует важнейшие возможности объектов `span`.

```
1 // fig07_12.cpp
2 // Доступ к контейнерам через span (C++20)
3 #include <array>
4 #include <fmt/format.h>
5 #include <iostream>
6 #include <numeric>
7 #include <span>
8 #include <vector>
9
```

Илл. 7.12 | Доступ к контейнерам через `span` (C++20)

Функция `displayArray`

Для прямой передачи традиционного массива функции нужны два аргумента: имя массива и размер массива. Хотя параметр `items` (строка 12) объявляется с символами `[]`, на самом деле это просто указатель на значение типа `int`, который «не знает», сколько элементов содержит этот аргумент. Поэтому нужен второй аргумент. Такой подход вызывает ряд проблем. Например, код, вызывающий функцию `displayArray`, может случайно передать ей неправильное значение для параметра `size`. Тогда функция обработает не все элементы массива `items` или даже попытается получить доступ к элементу за границей массива, а это уже логическая ошибка и потенциальная опасность. Кроме того, у внешней итерации, показанной в строках 13–15, есть недостатки, которые мы рассмотрели ранее. Встроенный в Visual Studio инструмент C++ Core Guidelines Checker, проверяющий корректность кода, выдает предупреждения о функции `displayArray` и о передаче традиционных массивов функциям. Мы включили в этот пример функцию `displayArray` только для того, чтобы сравнить ее с функцией `displaySpan`, написанной правильно.

```
10 // Параметр items обрабатывается как const int*, поэтому в дополнение к
11 // ему надо передать размер массива, чтобы перебирать элементы в цикле
12 void displayArray(const int items[], size_t size) {
13     for (size_t i{0}; i < size; ++i) {
14         std::cout << fmt::format("{} ", items[i]);
15     }
16 }
17
```

¹ C++ Core Guidelines, «R.14: Avoid [] Parameters, Prefer span». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rr-ap>.

Функция `displaySpan`

Согласно C++ Core Guidelines, в современном коде любой указатель должен указывать только на единичный объект¹ (а не на массив), а функции вроде `displayArray`, получающие указатель и размер массива, являются источником ошибок². Поэтому надо передавать массивы функциям не напрямую, а через `span`. Наша функция `displaySpan` (строки 20–24) получает массив `items` через объект `span`, представляющий элементы массива как целочисленные константы (`const int`); мы сделали их константами, так как функция не должна их изменять.

```

18 // Параметр span содержит адрес первого элемента и количество элементов,
19 // поэтому мы можем перебирать их в цикле for для диапазона значений
20 void displaySpan(std::span<const int> items) {
21     for (const auto& item : items) { // Объекты span являются итерируемыми
22         std::cout << fmt::format("{} ", item);
23     }
24 }
25

```

Класс `span` инкапсулирует указатель и значение типа `size_t`, представляющее количество элементов. Когда вы передаете функции `displaySpan` традиционный массив (или `std::array`, или `std::vector`), C++ неявно создает объект `span`, содержащий и указатель на первый элемент массива, и размер массива, который компилятор самостоятельно находит в объявлении массива. Этот `span` предоставляет функции элементы исходного массива, который вы передали в качестве аргумента. В C++ Core Guidelines рекомендуется передавать `span` по значению³ — это не менее эффективно, чем передавать указатель и размер по отдельности³, как мы сделали в функции `displayArray`.

Подобно массивам и векторам, объекты `span` имеют много возможностей, среди которых обработка элементов в циклах `for` для диапазона значений. Поскольку объект `span` создается на основе определенного компилятором размера массива, заданного при объявлении массива, вы получаете гарантию, что такой цикл никогда не обратится к несуществующим элементам за границами массива. Это исключает риск переполнения буфера и других опасных проблем, свойственных функциям, похожим на `displayArray`.

Функция `times2`

Поскольку `span` — это представление существующего контейнера, изменение элементов `span` изменяет исходные данные, то есть элементы этого контейнера.

¹ C++ Core Guidelines, «ES.42: Keep Use of Pointers Simple And Straightforward». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Res-ptr>.

² C++ Core Guidelines, «I.13: Do Not Pass an Array as a Single Pointer». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Ri-array>.

³ C++ Core Guidelines, «F.24: Use a `span<T>` or a `span_p<T>` to Designate a Half-open Sequence». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rf-range>.

Функция `times2` умножает на 2 каждый элемент массива `items` через `span<int>`. Обратите внимание, что мы используем неконстантную ссылку (без квалификатора `const`), чтобы вносить изменения в массив через `span`.

```
26 // Изменяем элементы массива через span
27 void times2(std::span<int> items) {
28     for (int& item : items) {
29         item *= 2;
30     }
31 }
32
```

Передача массива функции для отображения его элементов

В строках 34–36 объявляются: традиционный массив `values1`, массив `values2` (`std::array`) и вектор `values3` (`std::vector`). Каждый из них содержит пять элементов, хранящихся в памяти последовательно. В строке 41 вызывается функция `displayArray`, отображающая содержимое массива `values1`. Первый параметр функции `displayArray` — это указатель на `int`, поэтому мы не можем использовать имена контейнеров `std::array` и `std::vector` для передачи этих объектов функции `displayArray`.

```
33 int main() {
34     int values1[]{1, 2, 3, 4, 5};
35     std::array values2{6, 7, 8, 9, 10};
36     std::vector values3{11, 12, 13, 14, 15};
37
38     // Нужно указать размер, потому что параметр items
39     // является указателем на первый элемент аргумента
40     std::cout << "values1 via displayArray: ";
41     displayArray(values1, 5);
42
```

```
values1 via displayArray: 1 2 3 4 5
```

Неявное создание объекта `span` и его передача функции

В строке 46 вызывается функция `displaySpan` с аргументом `values1`. Параметр этой функции был объявлен так:

```
std::span<const int>
```

Поэтому компилятор создает объект `span`, содержащий `const int*` — указатель на первый элемент массива и `size_t` — размер массива, автоматически получаемый из объявления `values1` в строке 34. Поскольку `span` может представлять любую непрерывную последовательность элементов, мы также можем передать функции `displaySpan` контейнеры `std::array` и `std::vector` с элементами `int` (строки 50 и 52). Компилятор создаст соответствующие объекты `span`, содержащие указатель на первый элемент контейнера и размер контейнера. Это делает функцию

`displaySpan` более универсальной, чем `displayArray`, которая в этом примере может принимать только традиционные массивы.

```
43 // Компилятор знает размер массива values1 и автоматически создает span,  
44 // содержащий указатель &values1[0] и размер массива  
45 std::cout << "\nvalues1 via displaySpan: ";  
46 displaySpan(values1);  
47  
48 // Компилятор также может создавать span для std::array и std::vector  
49 std::cout << "\nvalues2 via displaySpan: ";  
50 displaySpan(values2);  
51 std::cout << "\nvalues3 via displaySpan: ";  
52 displaySpan(values3);  
53
```

```
values1 via displaySpan: 1 2 3 4 5  
values2 via displaySpan: 6 7 8 9 10  
values3 via displaySpan: 11 12 13 14 15
```

При изменении элементов объекта `span` изменяются элементы исходного контейнера

Напомним, что функция `times2` умножает каждый элемент объекта `span` на 2. В строке 55 функция `times2` вызывается с массивом `values1` в качестве аргумента. Параметр этой функции был объявлен так:

```
std::span<int>
```

Поэтому компилятор создает `span`, содержащий `int*` — указатель на первый элемент массива и `size_t` — размер массива, автоматически получаемый из объявления `values1` в строке 34. Чтобы показать, что функция `times2` изменила данные исходного массива, в строке 57 мы выводим на экран обновленные значения `values1`. Как и `displaySpan`, функцию `times2` можно вызывать и с другими контейнерами из этой программы: `std::array` и `std::vector`.

```
54 // Изменение исходных данных через span  
55 times2(values1);  
56 std::cout << "\n\nvalues1 after times2 modifies its span argument: ";  
57 displaySpan(values1);  
58
```

```
values1 after times2 modifies its span argument: 2 4 6 8 10
```

Явное создание объекта `span` и взаимодействие с ним

Вы можете явно создавать объекты `span`. В строке 60 создается `span<int>`, представляющий данные массива `values1`, — здесь компилятор использует CTAD для определения типа `int` по типу элементов `values1`. В строках 61–62 демонстрируются функции `front` и `back`, возвращающие первый и последний элементы объекта `span`, — в нашем случае это первый и последний элементы массива `values1`.

```
59 // В span есть целый ряд средств для работы с массивами и векторами
60 std::span mySpan(values1); // span<int>
61 std::cout << "\n\nmySpan's first element: " << mySpan.front()
62 << "\nmySpan's last element: " << mySpan.back();
63
```

```
mySpan's first element: 2
mySpan's last element: 10
```

Философия C++ Core Guidelines заключается в «предпочтительной проверке кода во время компиляции, а не во время выполнения»¹. Это позволяет компилятору находить многие ошибки и сообщать о них во время компиляции, поэтому вам не приходится писать дополнительный код, предотвращающий ошибки во время выполнения. В строке 60 компилятор определяет количество элементов в `span` (5) по объявлению массива `values1` в строке 34. Вы также можете явно указать тип и количество элементов при создании объекта `span`:

```
span<int, 5> mySpan(values1);
```

Компилятор проверит эту инструкцию и убедится, что вы создали `span` в соответствии с `values1`; в противном случае он сообщит об ошибке компиляции.

Использование `span` с алгоритмом `accumulate` стандартной библиотеки

Как показано в нашем примере, объекты `span` являются итерируемыми. Это значит, что вы можете использовать функции `begin` и `end` с объектами `span`, чтобы передавать их алгоритмам (функциям, выполняющим алгоритмические операции) стандартной библиотеки C++, в том числе функциям `accumulate` (строка 66) и `sort`. Мы рассмотрим эти алгоритмы в главе 14.

```
64 // Объекты span совместимы с алгоритмами стандартной библиотеки C++
65 std::cout << "\n\nSum of mySpan's elements: "
66 << std::accumulate(std::begin(mySpan), std::end(mySpan), 0);
67
```

```
Sum of mySpan's elements: 30
```

Создание вложенных представлений

Напомним, что объекты `span` являются **представлениями (views)** контейнеров. В классе `span` есть функции `first`, `last` и `subspan`, позволяющие создавать **вложенные представления (subviews)**, представляющие не весь контейнер, а его фрагменты. В строках 70 и 72 мы используем функции `first` и `last` для создания вложенных представлений, содержащих первые три и последние три элемента массива `values1` соответственно. В строке 74 мы используем функцию `subspan` для получения вложенного представления, содержащего 3 элемента, начиная

¹ C++ Core Guidelines, «P5: Prefer Compile-Time Checking to Run-Time Checking». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rpcompile-time>.

с индекса 1. В каждом случае мы передаем вложенные представления функции `displaySpan`, чтобы показать корректность их создания.

```
68 // Класс span позволяет создавать вложенные представления контейнеров
69 std::cout << "\n\nFirst three elements of mySpan: ";
70 displaySpan(mySpan.first(3));
71 std::cout << "\n\nLast three elements of mySpan: ";
72 displaySpan(mySpan.last(3));
73 std::cout << "\n\nMiddle three elements of mySpan: ";
74 displaySpan(mySpan.subspan(1, 3));
75
```

```
First three elements of mySpan: 2 4 6
Last three elements of mySpan: 6 8 10
Middle three elements of mySpan: 4 6 8
```

При изменении элементов вложенных представлений изменяются элементы исходного контейнера

Через вложенные представления неконстантных данных (объявленных без квалификатора `const`) можно изменять эти данные. В строке 77 мы передаем функции `times2` объект `span`, представляющий 3 элемента массива `values1`, начиная с индекса 1. В строке 79 мы выводим на экран обновленные значения элементов массива `values1`, чтобы подтвердить их изменение.

```
76 // Изменение вложенных представлений изменяет исходные данные
77 times2(mySpan.subspan(1, 3));
78 std::cout << "\n\nvalues1 after modifying elements via span: ";
79 displaySpan(values1);
80
```

```
values1 after modifying elements via span: 2 8 12 16 10
```

Доступ к элементам `span` с помощью оператора `[]`

Напомним, что оператор `[]` дает доступ к элементам традиционных массивов, современных массивов `std::array` и векторов `std::vector`, не выполняя при этом проверку границ. Этот оператор можно использовать и для доступа к элементам объектов `span`, и тоже без проверки границ. В строке 82 мы выводим на экран элемент с индексом 2¹.

```
81 // Доступ к элементу span через оператор []
82 std::cout << "\n\nThe element at index 2 is: " << mySpan[2];
83 }
```

```
The element at index 2 is: 12
```

¹ В классе `std::span` нет функции `at`, проверяющей границы, но ее могут добавить в будущем.

7.11. Строки на основе указателей

Вы уже работали с классом `string` стандартной библиотеки C++, представляющим строки текста в виде полноценных объектов. В главе 8 класс `std::string` рассматривается подробнее. А в этом разделе мы кратко расскажем о **строках в стиле C (C-strings)**, которые основаны на указателях.

Конечно, строки класса `std::string` предпочтительнее, поскольку этот класс позволяет избежать многих проблем безопасности и ошибок, свойственных операциям со строками в стиле C. Но в некоторых случаях строки в стиле C необходимы, например при обработке аргументов командной строки. Кроме того, если вы будете работать с устаревшим кодом, написанным на языках C и C++, вы почти наверняка найдете там строки в стиле C.

Символы и символьные константы

Символы — это те «кирпичики», из которых строка за строкой составляется исходный код на языке C++. Исходный код каждой программы — это множество символов, которые при осмысленной группировке интерпретируются компилятором как инструкции и данные, используемые для выполнения задач. Исходный код может содержать **символьные константы (character constants)** типа `char`. Каждая символьная константа — это целое число, которое записывается в исходном коде в виде символа, заключенного в одинарные кавычки. Значение такой константы — это номер ее символа в наборе символов (в кодировке) операционной системы компьютера. Например, константа `'z'` имеет значение 122, соответствующее букве `z` в наборе символов ASCII (см. приложение Б). Или, например, константа `'\n'` имеет значение 10, соответствующее символу «перевод строки» в наборе символов ASCII.

Строка в стиле C — это традиционный массив

Строка в стиле C — это фактически традиционный массив символов, заканчивающийся **нулевым символом (null character) '\0'**, который обозначает конец строки в памяти. Доступ к строке в стиле C осуществляется по указателю на ее первый символ (независимо от длины строки).

Строковые литералы в качестве инициализаторов

Строковыми литералами можно инициализировать традиционные массивы символов (массивы элементов типа `char`) и переменные типа `const char*`. Например, каждое из этих объявлений инициализирует переменную строкой `"blue"`:

```
char color[]{"blue"};
const char* colorPtr{"blue"};
```

Первое объявление создает традиционный массив `color` из пяти элементов, содержащий символы `'b'`, `'l'`, `'u'`, `'e'` и `'\0'`. Второе объявление создает переменную-указатель `colorPtr`, указывающую на букву `b` в строке `"blue"` (которая заканчивается нулевым символом `'\0'`) где-то в памяти. Первое объявление можно реализовать

и другим способом: с помощью состоящего из отдельных символов списка инициализаторов, в который добавлен завершающий символ `'\0'`, например:

```
char color[]{'b', 'l', 'u', 'e', '\0'};
```

Строковые литералы существуют в программе постоянно: с момента запуска до завершения. К ним можно получать доступ из любой точки программы. Строковые литералы неизменяемы: после объявления никакие изменения в них невозможны.

Проблемы строк в стиле C

Если в традиционном массиве элементов `char` не выделить достаточно места для хранения символа `'\0'`, который должен завершать строку в стиле C, вы получите логическую ошибку. Создание или использование строки в стиле C, не заканчивающейся символом `'\0'`, может привести к логическим ошибкам.

Сохраняя строки в традиционном массиве элементов `char`, убедитесь, что его размер достаточно велик для самой длинной строки, которая в нем может быть когда-либо сохранена. C++ допускает строки любой длины. Но если строка длиннее традиционного массива, в который вы ее сохраняете, все не поместившиеся в массив символы будут записаны в последующие ячейки памяти. Это приведет к краху программы или к логическим ошибкам и опасным уязвимостям.

Отображение C-strings

Традиционный массив элементов `char`, содержащий строку с завершающим нулевым символом, можно выводить на экран с помощью `cout` и `<<`. Например, инструкция:

```
std::cout << color;
```

отображает традиционный массив `color`. Объект `cout` не учитывает размер традиционного массива. Символы выводятся до тех пор, пока не встретится завершающий нулевой символ, который не отображается. Объекты `cin` и `cout` обрабатывают массивы символов (элементов типа `char`) как строки, заканчивающиеся нулевыми символами. Но аналогичная обработка ввода и вывода не выполняется для традиционных массивов с элементами других типов.

7.11.1. Аргументы командной строки

В некоторых случаях традиционные массивы и строки в стиле C необходимы. Например, многие программы принимают при запуске **аргументы командной строки** (**command-line arguments**), содержащие информацию о требуемой конфигурации, об именах подлежащих обработке файлов и т. д. Чтобы передать программе аргументы этим стандартным способом, вы перечисляете их после имени программы, запуская ее из командной строки. Например, в системе Windows есть программа `dir`, выводящая на экран содержимое папки. Вы можете запустить эту программу с аргументом `/p`:

```
dir /p
```

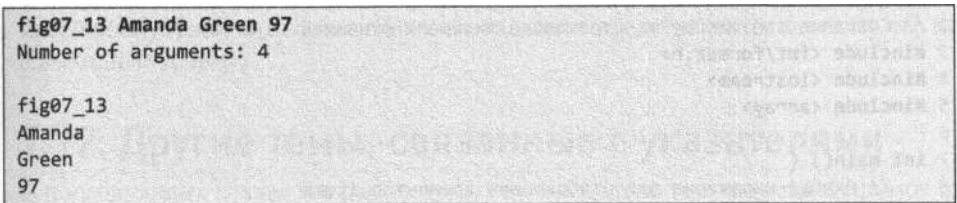
В этой команде аргумент `/p` означает, что надо делать паузу после каждого заполнения экрана. Аналогичную программу `ls` в системах Linux и macOS вы можете запустить, например, с аргументом `-la`:

```
ls -la
```

В этой команде аргумент `-la` означает, что надо выводить на экран не краткие, а детализированные сведения.

Программа получает аргументы командной строки в виде строк в стиле C, при этом первым аргументом является имя программы. Чтобы использовать полученные аргументы в виде современных строк `std::string` или других типов данных (`int`, `double` и т. д.), вы должны сначала преобразовать аргументы в эти типы. На илл. 7.13 показан код и пример запуска программы, которая выводит на экран количество переданных ей аргументов командной строки, а затем — каждый из них в отдельной строке.

```
1 // fig07_13.cpp
2 // Получение аргументов командной строки
3 #include <fmt/format.h>
4 #include <iostream>
5
6 int main(int argc, char* argv[]) {
7     std::cout << fmt::format("Number of arguments: {}\n\n", argc);
8
9     for (int i{0}; i < argc; ++i) {
10         std::cout << fmt::format("{}\n", argv[i]);
11     }
12 }
```



```
fig07_13 Amanda Green 97
```

```
Number of arguments: 4
```

```
fig07_13
```

```
Amanda
```

```
Green
```

```
97
```

Илл. 7.13 | Получение аргументов командной строки

Чтобы программа получила аргументы командной строки, функция `main` должна быть объявлена с двумя параметрами (строка 6), которые по соглашению имеют имена `argc` и `argv`. Первый аргумент — переменная `int`, представляющая количество аргументов. Второй аргумент — указатель `char*`, указывающий на первый элемент традиционного массива символов (некоторые программисты записывают этот аргумент так: `char** argv`). Первый элемент массива — это строка в стиле C, содержащая имя программы¹. Остальные элементы представляют собой строки в стиле C, содержащие остальные аргументы командной строки.

¹ Стандарт C++ допускает, что на некоторых платформах `argv[0]` может быть пустой строкой.

Следующая команда запускает программу `fig07_13` и передает ей аргументы: "Amanda", "Green" и "97":

```
fig07_13 Amanda Green 97
```

В macOS и Linux вы должны запускать эту программу из текущего каталога: `./fig07_13`. В командной строке аргументы разделяются пробелами, а не запятыми. При выполнении этой команды функция `main` программы `fig07_13` получает количество аргументов (4) и 4-элементный массив строк в стиле C, в котором:

- ✦ элемент `argv[0]` содержит имя программы: `"fig07_13"` (или `"./fig07_13"` в macOS и Linux);
- ✦ элементы от `argv[1]` до `argv[3]` содержат строки `"Amanda"`, `"Green"` и `"97"` соответственно.

Вы сами решаете, как обрабатывать полученные аргументы в вашей программе. Из-за проблем, связанных со строками в стиле C, перед использованием аргументов командной строки преобразуйте их в `std::string`.

7.11.2. Еще одна возможность функции `to_array` (C++20)

В разделе 7.6 мы продемонстрировали преобразование традиционных массивов в `std::array` с помощью функции `to_array`. На илл. 7.14 показана еще одна возможность этой функции. Мы используем лямбда-выражение (строки 9–13), аналогичное выражению на илл. 7.6, для отображения содержимого `std::array` после вызова функции `to_array`.

```

1 // fig07_14.cpp
2 // Создание std::array из строкового литерала функцией to_array (C++20)
3 #include <fmt/format.h>
4 #include <iostream>
5 #include <array>
6
7 int main() {
8     // Лямбда-выражение для отображения элементов items
9     const auto display{
10         [](const auto& items) {
11             for (const auto& item : items) {
12                 std::cout << fmt::format("{} ", item);
13             }
14         }
15     };
16
17     // Инициализация std::array строковым литералом
18     // создает одноэлементный array<const char*>
19     const auto array1{std::array{"abc"}};
20     std::cout << fmt::format("array1.size() = {}\\narray1: ",
21         array1.size());
22     display(array1); // Используем лямбду для отображения содержимого
23

```

```

24 // Создаем из строкового литерала массив символов
25 const auto array2{std::to_array("C++20")};
26 std::cout << fmt::format("{}\narray2.size() = {}\narray2: ",
27     array2.size());
28 display(array2); // Используем лямбду для отображения содержимого
29
30 std::cout << '\n';
31 }

```

```

array1.size() = 1
array1: abc

array2.size() = 6
array2: C + + 2 0

```

Илл. 7.14 | Создание `std::array` из строкового литерала функцией `to_array` [C++20]

Инициализация `std::array` строковым литералом создает одноэлементный массив

В строке 19 создается массив, в котором есть только один элемент `const char*`, указывающий на строку в стиле C "abc".

Передача строкового литерала функции `to_array` создает массив символов `std::array`

При передаче строкового литерала функции `to_array` (строка 25) создается массив `std::array`, содержащий в виде отдельных элементов типа `char` все символы этого литерала и завершающий нулевой символ. В строках 26–27 подтверждается, что массив состоит из 6 элементов. В строке 28 выводится на экран содержимое массива. Нулевой символ не имеет визуального представления, поэтому его нет в выходных данных.

7.12. Другие темы, связанные с указателями

В последующих главах мы рассмотрим еще несколько тем, имеющих прямое отношение к указателям:

- ♦ В главе 10 мы будем использовать указатели с объектами классов, чтобы показать, что в операциях динамического полиморфизма (термин из объектно-ориентированного программирования) можно использовать ссылки или указатели, но ссылки предпочтительнее.
- ♦ В главе 11 мы рассмотрим управление динамически выделяемой памятью с помощью указателей, позволяющее вам по мере необходимости создавать и уничтожать объекты во время выполнения программы. Неправильное управление этим процессом является источником скрытых ошибок, в том числе утечек памяти. Мы покажем, как **умные указатели (smart pointers)** автоматически управляют памятью и другими ресурсами, которые надо возвращать операционной системе, когда они больше не нужны.

- ♦ В главе 14 мы покажем, что имя функции может быть рассмотрено как указатель на ее реализацию, а функции могут передаваться другим функциям по указателям.

7.13. Итоги

В этой главе рассматривались указатели, массивы на основе указателей (массивы в стиле C) и строки на основе указателей (строки в стиле C). При этом мы неоднократно акцентировали внимание на содержащиеся в C++ Core Guidelines рекомендации избегать указателей в большинстве случаев, заменяя их объектами `std::array`, `std::vector` и `std::string`.

Мы привели примеры объявления и инициализации указателей, а затем рассказали о необходимых для работы с ними операторах `&` и `*`. Мы сравнили два способа передачи аргументов функции: по ссылке и по указателю, пояснив, что вы должны, как правило, выбирать передачу по ссылке. Мы рассмотрели традиционные массивы и показали их тесную связь с указателями.

Мы обсудили сочетания квалификатора `const` с указателями и соответствующими данными. Мы использовали оператор `sizeof` для определения количества байтов, занимаемых в памяти данными основных типов и указателями. Мы продемонстрировали выражения и арифметические операции с указателями.

Наконец, мы кратко рассмотрели строки в стиле C и показали пример передачи в программу аргументов из командной строки. Для выполнения этой простой задачи в современном C++ по-прежнему необходимы строки и массивы в стиле C.

Надеемся, что после изучения этой главы вы сделали правильный вывод: программист должен избегать указателей, строк в стиле C и традиционных массивов во всех случаях, когда это возможно! При работе с устаревшим кодом, в котором есть массивы на основе указателей, вы можете преобразовывать их в современные массивы `std::array` функцией `to_array` (C++20) или безопасно обрабатывать через объекты `span` (C++20). В следующей главе мы рассмотрим основные операции со строками, предоставляемые классом `std::string`, а также начнем важную тему обработки файлов.

Строки и их представления, текстовые файлы, CSV-файлы и регулярные выражения

В этой главе вы:

- научитесь определять характеристики строк;
- научитесь искать, заменять и добавлять символы в строках;
- познакомитесь с функциями преобразования между строками и числами [C++11];
- познакомитесь с «облегченными» представлениями строк `string_view` [C++17];
- освоите операции чтения и записи текстовых файлов с последовательным доступом;
- узнаете о строковых потоках, которые удобны для проверки данных перед их обработкой;
- используя готовые объекты, прочитаете из CSV-файла набор данных «Титаник» и проанализируете часть этих данных;
- используя готовые объекты, начнете изучать регулярные выражения [C++11, заголовков `<regex>`], необходимые для поиска и замены фрагментов в тексте.

8.1. Введение

В начале этой главы вы узнаете о многих полезных возможностях класса `std::string`. Затем мы приступим к работе с представлениями `string_view`, текстовыми файлами, CSV-файлами и регулярными выражениями.

Операции со строками `std::string`

Начиная с главы 2, вы работали с текстовыми строками, которые являются объектами класса `std::string`. Теперь пришло время изучить более сложные операции с ними: присваивание, сравнение, создание и поиск подстрок, внесение всевозможных изменений. Кроме того, вы узнаете, как преобразовывать строки в числа и обратно.

Представления `string_view` (C++17)

Мы расскажем об объектах `string_view`, которые являются неизменяемыми представлениями строк `std::string` и строк в стиле C. Подобно объектам `std::span`, объекты `string_view` не содержат тех данных, которые они представляют. Вы увидите, что многие возможности объектов `string` и `string_view` похожи, поэтому представления хорошо подходят вместо строк в тех случаях, когда текст не должен изменяться.

Текстовые файлы и строковые потоки

Память компьютера (оперативная память) рассчитана на кратковременное хранение данных. Для длительного и надежного хранения данных предназначены **файлы (files)**. Компьютеры работают с файлами, которые находятся на локальных устройствах хранения (например, на флеш-накопителях) и «в облаке» (то есть на других компьютерах). Мы объясним, как написать на C++ программу, которая создает и обрабатывает текстовые файлы. Мы также покажем, как читать и записывать строки `std::string` через потоки `ostream` и `istream`.

Готовые объекты: CSV-файлы и набор данных о катастрофе «Титаника»

В этой главе есть два раздела о готовых объектах. В первом из них мы представим вам популярный файловый формат CSV. В этом формате часто хранят наборы данных, применяемые в работе с «большими данными» и в сфере искусственного интеллекта, например в системах обработки естественного языка и машинного обучения.

Начинающие специалисты по анализу и обработке данных часто практикуются на общедоступном наборе данных «Титаник». В нем перечислены все пассажиры и указано, кто из них выжил, когда пароход затонул 15 апреля 1912 года. Библиотека `gripdcsv` с открытым исходным кодом поможет вам прочесть набор данных «Титаник» из CSV-файла. Затем вы выполните простой анализ полученных данных.

Готовые объекты: регулярные выражения для поиска, замены и проверки фрагментов текста

Второй раздел о готовых объектах познакомит вас с регулярными выражениями. Они незаменимы в современных программах, обрабатывающих большие объемы текста. Вы будете составлять регулярные выражения в виде объектов `regex` и затем искать в тексте соответствующие им фрагменты с помощью функций из заголовка `<regex>`. В предыдущих главах наши программы не проверяли корректность ввода данных пользователем, но в профессиональном программном обеспечении такие проверки необходимы. Для этой цели программисты часто используют функции класса `std::string`, строковые потоки и регулярные выражения, о которых вы скоро узнаете.

8.2. Присваивание и конкатенация строк

На илл. 8.1 приведены примеры присваивания и конкатенации строк, являющихся объектами класса `std::string`.

```
1 // fig08_01.cpp
2 // Присваивание и конкатенация строк
3 #include <fmt/format.h>
4 #include <iostream>
5 #include <string>
6
7 int main() {
8     std::string s1("cat");
9     std::string s2; // Инициализируем пустой строкой
10    std::string s3; // Инициализируем пустой строкой
11
12    s2 = s1; // Присваиваем значение строки s1 строке s2
13    s3.assign(s1); // Присваиваем значение строки s1 строке s3
14    std::cout << fmt::format("s1: {}\ns2: {}\ns3: {}\n\n", s1, s2, s3);
15
16    s2.at(0) = 'r'; // Изменяем строку s2
17    s3.at(2) = 'r'; // Изменяем строку s3
18    std::cout << fmt::format("After changes:\ns2: {}\ns3: {}", s2, s3);
19
20    std::cout << "\n\nAfter concatenations:\n";
21    std::string s4{s1 + "apult"}; // Выполняем конкатенацию
22    s1.append("acomb"); // Создаем строку "catacomb"
23    s3 += "pet"; // Создаем строку "carpet" перегруженным оператором +=
24    std::cout << fmt::format("s1: {}\ns3: {}\ns4: {}\n", s1, s3, s4);
25
26    // Добавляем символы с 4-й позиции до конца строки s1,
27    // чтобы создать строку "comb" (строка s5 сначала была пустой)
28    std::string s5; // Инициализируем пустой строкой
29    s5.append(s1, 4, s1.size() - 4);
30    std::cout << fmt::format("s5: {}", s5);
31 }
```

```
s1: cat
s2: cat
s3: cat

After changes:
s2: rat
s3: car

After concatenations:
s1: catacomb
s3: carpet
s4: catapult
s5: comb
```

Илл. 8.1 | Присваивание и конкатенация строк

Присваивание строк

В строках 8–10 создаются объекты класса `std::string`: `s1`, `s2` и `s3`. Значение `s1` копируется в `s2` оператором присваивания (строка 12). Значение `s1` копируется в `s3` функцией `assign` класса `std::string` (строка 13). Эта версия функции `assign` выполняет такую же операцию, как оператор присваивания (`=`), но у функции `assign` есть еще целый ряд перегруженных версий, о которых вы можете узнать на этой странице:

https://en.cppreference.com/w/cpp/string/basic_string/assign

Например, одна из них копирует указанное количество символов:

```
target.assign(source, start, numberOfChars);
```

Здесь `source` — исходная строка (`std::string`) для копирования, `start` — начальный индекс, `numberOfChars` — количество копируемых символов.

Доступ к элементам строк по индексам

Функция `at` класса `std::string` (строки 16–17) присваивает значение 'r' символу объекта `s2` с индексом 0 (образуя "rat") и значение 'r' символу объекта `s3` с индексом 2 (образуя "car"). Вы можете использовать функцию `at` и для того, чтобы получить из объекта `std::string` символ с указанным индексом. В этом случае (по аналогии с массивами `std::array` и векторами `std::vector`) функция `at` выполнит проверку границ и выдаст исключение `out_of_range`, если вы укажете недопустимый индекс. Оператор индекса `[]` работает иначе: он не проверяет границы объектов `std::string` (как и границы массивов `std::array` и векторов `std::vector`). Вы также можете перебирать все символы в объекте `std::string`, используя цикл `for` для диапазона значений:

```
for (char c : s3) {
    cout << c;
}
```

Этот способ исключает риск доступа к несуществующим символам за пределами объекта `std::string`.

Конкатенация строк

Объект `s4` инициализируется содержимым объекта `s1` с добавлением `"apult"` (строка 21). Применительно к объектам `std::string` оператор `+` выполняет конкатенацию строк. Функция `append` класса `std::string` выполняет конкатенацию `s1` и `"acomb"` (строка 22). Перегруженный оператор `+=` выполняет конкатенацию `s3` и `"pet"` (строка 23). Функция `append` выполняет конкатенацию пустого объекта `s5` и `"comb"` (строка 29), при этом фрагмент `"comb"` формируется ее аргументами: объектом `std::string`, из которого копируются символы (`s1`), начальным индексом (4) и количеством копируемых символов (`s1.size() - 4`).

8.3. Сравнение строк

Класс `std::string` содержит функции для сравнения своих объектов (илл. 8.2). В следующем примере мы несколько раз вызываем пользовательскую функцию `displayResult` (строки 7–17) для отображения результатов каждого сравнения.

```
1 // fig08_02.cpp
2 // Сравнение строк
3 #include <fmt/format.h>
4 #include <iostream>
5 #include <string>
6
7 void displayResult(const std::string& s, int result) {
8     if (result == 0) {
9         std::cout << fmt::format("{} == 0\n", s);
10    }
11    else if (result > 0) {
12        std::cout << fmt::format("{} > 0\n", s);
13    }
14    else { // Результат < 0
15        std::cout << fmt::format("{} < 0\n", s);
16    }
17 }
18
19 int main() {
20     const std::string s1{"Testing the comparison functions."};
21     const std::string s2{"Hello"};
22     const std::string s3{"stinger"};
23     const std::string s4{s2}; // "Hello"
24
25     std::cout << fmt::format("s1: {}\ns2: {}\ns3: {}\ns4: {}",
26         s1, s2, s3, s4);
27
28     // Сравниваем s1 и s4
29     if (s1 > s4) {
```



```
30     std::cout << "\n\ns1 > s4\n";
31 }
32
33 // Сравниваем s1 и s2
34 displayResult("s1.compare(s2)", s1.compare(s2));
35
36 // Сравниваем s1 (элементы 2-6) и s3 (элементы 0-4)
37 displayResult("s1.compare(2, 5, s3, 0, 5)",
38     s1.compare(2, 5, s3, 0, 5));
39
40 // Сравниваем s4 и s2
41 displayResult("s4.compare(0, s2.size(), s2)",
42     s4.compare(0, s2.size(), s2));
43
44 // Сравниваем s2 и s4
45 displayResult("s2.compare(0, 3, s4)", s2.compare(0, 3, s4));
46 }
```

```
s1: Testing the comparison functions.
```

```
s2: Hello
```

```
s3: stinger
```

```
s4: Hello
```

```
s1 > s4
```

```
s1.compare(s2) > 0
```

```
s1.compare(2, 5, s3, 0, 5) == 0
```

```
s4.compare(0, s2.size(), s2) == 0
```

```
s2.compare(0, 3, s4) < 0
```

Илл. 8.2 | Сравнение строк

Сравнение строк операторами равенства и сравнения

Объекты `std::string` можно сравнивать друг с другом и со строками в стиле C, используя операторы равенства и сравнения, возвращающие значение типа `bool`. Такие операции сравнения выполняются лексикографически, то есть по очереди сравниваются числовые значения всех символов в строках. Например, символ 'A' имеет значение 65, а символ 'a' — 97 (см. приложение Б), поэтому строка "Apple" «меньше», чем "apple", хотя эти слова могут показаться одинаковыми. В строке 29 объекты `s1` и `s4` сравниваются перегруженным оператором `>`. В данном случае строка `s1` начинается с заглавной буквы 'T' (значение 84), а строка `s4` начинается с заглавной буквы 'H' (значение 72), поэтому `s1` «больше», чем `s4`.

Сравнение строк функцией `compare`

В строке 34 функция `compare` класса `std::string` сравнивает `s1` и `s2`. Эта функция возвращает 0, если `s1` и `s2` одинаковы; или положительное число, если `s1` лексикографически больше, чем `s2`; или отрицательное число, если `s1` лексикографически меньше, чем `s2`. Поскольку строка, начинающаяся с 'T', лексикографически боль-

ше строки, начинающейся с 'H', результатом сравнения является положительное число, что подтверждается выходными данными.

В строке 38 перегруженная функция `compare` сравнивает фрагменты строк `s1` и `s3`. Первые два аргумента (2 и 5) — это начальный индекс и длина фрагмента строки `s1` ("sting"). Третий аргумент — это строка для сравнения (`s3`). Последние два аргумента (0 и 5) — это начальный индекс и длина фрагмента строки `s3` (тоже "sting"). Сравниваемые фрагменты одинаковы, поэтому функция возвращает 0, что подтверждается выходными данными.

В строке 42 другая перегруженная версия функции `compare` сравнивает фрагмент строки `s4` со строкой `s2`. Первые два аргумента — это начальный индекс и длина фрагмента строки `s4`, а третий аргумент — строка для сравнения (`s2`). Сравниваемые фрагменты одинаковы, поэтому функция возвращает 0.

В строке 45 тем же способом выполняется сравнение первых трех символов строки `s2` ("Hel") со строкой `s4` ("Hello"). Фрагмент "Hel" совпадает с первыми тремя символами из "Hello", но он короче (меньше), чем "Hello", поэтому функция возвращает отрицательное число.

8.4. Создание подстроки

Функция `substr` класса `std::string` (илл. 8.3) возвращает фрагмент строки, или подстроку (**substring**). Результатом является новый объект `std::string`, содержащий символы, скопированные из исходной строки. В следующем примере функция `substr` создает подстроку строки `s`, копируя фрагмент, который начинается с индекса 3 и состоит из 4 символов.

```
1 // fig08_03.cpp
2 // Создание подстроки
3 #include <iostream>
4 #include <string>
5
6 int main() {
7     const std::string s{"airplane"};
8     std::cout << s.substr(3, 4) << '\n'; // Получаем подстроку "plan"
9 }
```

plan

Илл. 8.3 | Создание подстроки

8.5. Обмен значениями двух строк

Класс `std::string` содержит функцию `swap`, выполняющую обмен значениями двух строк. На илл. 8.4 приведен пример: эта функция присваивает строке `s1` прежнее значение `s2`, а строке `s2` — прежнее значение `s1`.

```
1 // fig08_04.cpp
2 // Обмен значениями двух строк
3 #include <fmt/format.h>
4 #include <iostream>
5 #include <string>
6
7 int main() {
8     std::string s1{"one"};
9     std::string s2{"two"};
10
11     std::cout << fmt::format("Before swap:\ns1: {}; s2: {}", s1, s2);
12     s1.swap(s2); // Выполняем обмен между s1 и s2
13     std::cout << fmt::format("\n\nAfter swap:\ns1: {}; s2: {}", s1, s2);
14 }
```

```
Before swap:
s1: one; s2: two

After swap:
s1: two; s2: one
```

Илл. 8.4 | Обмен значениями двух строк

8.6. Получение характеристик строки

Класс `std::string` содержит функции, которые предоставляют информацию о размере, емкости, максимальной длине и других характеристиках строки:

- ♦ **Размер (size)** строки — это количество символов, хранящихся в объекте `std::string` в данный момент.
- ♦ **Емкость (capacity)** строки — это количество символов, которое может быть сохранено в объекте `std::string`, прежде чем система должна будет выделить этому объекту дополнительную память для хранения большего количества символов. Выделение памяти для объектов `std::string` происходит автоматически. Емкость строки не может быть меньше размера строки, но может быть больше. Фактическая емкость в каждый момент времени зависит от реализации.
- ♦ **Максимальный размер (maximum size)** строки — это наибольшее количество символов, которое теоретически можно сохранить в объекте `std::string` на компьютере данного типа. Если это значение превышено, выдается исключение `length_error`¹.

В следующей программе, разделенной на части для удобства обсуждения (илл. 8.5), приведены примеры получения этих характеристик. Пользовательская функция `printStatistics` (строки 8–12) получает объект `std::string` и по очереди вы-

¹ В большинстве компьютерных систем максимальный размер настолько велик, что вы никогда не столкнетесь с исключением.

зывает четыре функции класса `std::string`, чтобы получить и вывести на экран характеристики строки: емкость (функция `capacity`), максимальный размер (функция `max_size`), размер (функция `size`) и результат проверки на пустую строку (функция `empty`).

```

1 // fig08_05.cpp
2 // Получение характеристик строки
3 #include <fmt/format.h>
4 #include <iostream>
5 #include <string>
6
7 // Функция для вывода на экран
8 void printStatistics(const std::string& s) {
9     std::cout << fmt::format(
10         "capacity: {}\nmax size: {}\nsize: {}\nempty: {}",
11         s.capacity(), s.max_size(), s.size(), s.empty());
12 }
13

```

Илл. 8.5 | Получение характеристик строки

В строке 15 объявляется пустой объект `std::string` с именем `string1`, который затем передается функции `printStatistics` (строка 18). Эта функция возвращает размер `string1`, который равен 0, поскольку строка не содержит символов. Для компиляторов Visual C++ и GNU g++ максимальный размер равен 9 223 372 036 854 775 807 (вы видите это число в выходных данных), а для компилятора Clang из Xcode он равен 18 446 744 073 709 551 599. Объект `string1` является пустой строкой, поэтому функция `empty` возвращает `true`.

```

14 int main() {
15     std::string string1; // Пустая строка
16
17     std::cout << "Statistics before input:\n";
18     printStatistics(string1);
19

```

```

Statistics before input:
capacity: 15
max size: 9223372036854775807
size: 0
empty: true

```

Пользователь вводит текст (строка 21; мы ввели слово `tomato`), затем функция `printStatistics` (строка 24) отображает характеристики измененного объекта `string1`. Теперь размер строки равен 6, она больше не является пустой.

```

20     std::cout << "\n\nEnter a string: ";
21     std::cin >> string1; // Ввод без пробелов
22     std::cout << fmt::format("The string entered was: {}\n", string1);

```

```

23  std::cout << "Statistics after input:\n";
24  printStatistics(string1);
25

```

```

Enter a string: tomato
The string entered was: tomato
Statistics after input:
capacity: 15
max size: 9223372036854775807
size: 6
empty: false

```

Пользователь вводит новый текст (строка 27; мы ввели слово `soup`), который сохраняется в `string1` вместо `"tomato"`. Функция `printStatistics` (строка 30) отображает характеристики измененного объекта `string1`. Как видите, теперь размер строки равен 4.

```

26  std::cout << "\n\nEnter a string: ";
27  std::cin >> string1; // Ввод без пробелов
28  std::cout << fmt::format("The string entered was: {}\n", string1);
29  std::cout << "Statistics after input:\n";
30  printStatistics(string1);
31

```

```

Enter a string: soup
The string entered was: soup
Statistics after input:
capacity: 15
max size: 9223372036854775807
size: 4
empty: false

```

С помощью оператора `+=` (строка 33) выполняется конкатенация `string1` и строкового литерала, состоящего из 46 символов. Функция `printStatistics` (строка 36) снова отображает характеристики `string1`. Емкость `string1` была недостаточна для нового размера строки, поэтому система автоматически увеличила емкость до 63. Размер `string1` теперь равен 50. Алгоритм изменения емкости зависит от реализации.

```

32  // Добавляем 46 символов к string1
33  string1 += "1234567890abcdefghijklmnopqrstuvwxyz1234567890";
34  std::cout << fmt::format("\n\nstring1 is now: {}\n", string1);
35  std::cout << "Statistics after concatenation:\n";
36  printStatistics(string1);
37

```

```

string1 is now: soup1234567890abcdefghijklmnopqrstuvwxyz1234567890
Statistics after concatenation:
capacity: 63
max size: 9223372036854775807
size: 50
empty: false

```

Функция `resize` класса `std::string` (строка 38) вызывается, чтобы увеличить размер `string1` на 10 символов. Эти дополнительные элементы являются нулевыми символами. После этого функция `printStatistics` показывает, что емкость не изменилась (63), а размер равен 60.

```
38 string1.resize(string1.size() + 10); // Добавляем 10 символов к string1
39 std::cout << "\n\nStatistics after resizing to add 10 characters:\n";
40 printStatistics(string1);
41 std::cout << '\n';
42 }
```

```
Statistics after resizing to add 10 characters:
capacity: 63
max size: 9223372036854775807
size: 60
empty: false
```

Обновление функции `reserve` в C++20

Вы можете изменить емкость строки без изменения размера, вызвав функцию `reserve` класса `std::string` с целочисленным аргументом. Если емкость меньше, чем значение аргумента, она увеличится до такого же или большего значения. Но если емкость больше, чем значение аргумента, она не уменьшится — так работает функция `reserve`, начиная с C++20. В более ранних версиях C++ эта функция может не только увеличивать, но и уменьшать емкость, причем если аргумент меньше размера строки, емкость уменьшается до размера строки.

8.7. Поиск подстрок и символов в строке

Класс `std::string` содержит функции для поиска в строках отдельных символов и последовательностей символов (подстрок, фрагментов). В следующей программе, разделенной на части для удобства обсуждения (илл. 8.6), приведены несколько примеров поиска. В строке 8 объявляется и инициализируется объект `std::string` с именем `s`.

```
1 // fig08_06.cpp
2 // Поиск подстрок и символов в строке
3 #include <fmt/format.h>
4 #include <iostream>
5 #include <string>
6
7 int main() {
8     const std::string s{"noon is 12pm; midnight is not"};
9     std::cout << "Original string: " << s;
10 }
```

```
Original string: noon is 12pm; midnight is not
```

Илл. 8.6 | Поиск подстрок и символов в строке

Функции `find` и `rfind`

Затем дважды выполняется поиск подстроки "is" в строке `s`. Для этого вызываются функции `find` и `rfind` класса `std::string` (строки 12–13). Функция `find` выполняет поиск с начала строки (слева направо), а функция `rfind` — с конца (справа налево). Если подстрока найдена, каждая из функций возвращает индекс ее первого символа. В противном случае каждая из функций возвращает константу `string::npos`, означающую отсутствие искомой подстроки. Другие функции поиска, о которых мы расскажем в этом разделе, возвращают такие же значения, если не указано иное.

```
11 // Ищем фрагмент "is" с начала, а затем с конца строки s
12 std::cout << fmt::format("{}\ns.find(\"is\"): {}{}\ns.rfind(\"is\"): {}",
13   s.find("is"), s.rfind("is"));
14
```

```
s.find("is"): 5
s.rfind("is"): 23
```

Функция `find_first_of`

Функция `find_first_of` класса `std::string` (строка 16) ищет первое вхождение любого символа из подстроки "misop" в строке `s`. Эта функция выполняет поиск с начала строки (слева направо). Результатом поиска является символ 'o', имеющий в строке `s` индекс 1.

```
15 // Находим символ 'o' при поиске слева направо
16 int location(s.find_first_of("misop"));
17 std::cout << fmt::format("{}\ns.find_first_of(\"misop\") found {} at {}",
18   s.at(location), location);
19
```

```
s.find_first_of("misop") found o at 1
```

Функция `find_last_of`

Функция `find_last_of` класса `std::string` (строка 21) ищет последнее вхождение любого символа из подстроки "misop" в строке `s`. Эта функция выполняет поиск с конца строки (справа налево). Результатом поиска является символ 'o', имеющий в строке `s` индекс 27.

```
20 // Находим символ 'o' при поиске справа налево
21 location = s.find_last_of("misop");
22 std::cout << fmt::format("{}\ns.find_last_of(\"misop\") found {} at {}",
23   s.at(location), location);
24
```

```
s.find_last_of("misop") found o at 27
```

Функция `find_first_not_of`

Функция `find_first_not_of` класса `std::string` (строка 26) ищет в строке `s` первый символ из тех, которых нет в подстроке `"noi spm"`. Эта функция выполняет поиск с начала строки (слева направо). Результатом поиска является символ `'1'` с индексом 8. Во втором вызове (строка 32) эта функция ищет в строке `s` первый символ из тех, которых нет в подстроке `"12noi spm"`. Результатом поиска является символ `';` с индексом 12. В третьем вызове (строка 38) эта функция ищет в строке `s` первый символ из тех, которых нет в подстроке `"noon is 12pm; midnight is not"`. Но в этой подстроке есть все символы из строки `s`. Поскольку символ не был найден, функция возвращает значение `string::npos` (в данном случае равное `-1`).

```
25 // Находим символ '1' при поиске слева направо
26 location = s.find_first_not_of("noi spm");
27 std::cout << fmt::format(
28     "\ns.find_first_not_of(\"noi spm\") found {} at {}",
29     s.at(location), location);
30
31 // Находим символ ';' с индексом 12
32 location = s.find_first_not_of("12noi spm");
33 std::cout << fmt::format(
34     "\ns.find_first_not_of(\"12noi spm\") found {} at {}",
35     s.at(location), location);
36
37 // Ищем символы, которых нет в "noon is 12pm; midnight is not"
38 location = s.find_first_not_of("noon is 12pm; midnight is not");
39 std::cout << fmt::format("\n{:} : {}\\n",
40     "s.find_first_not_of(\"noon is 12pm; midnight is not\")",
41     location);
42 }
```

```
s.find_first_not_of("noi spm") found 1 at 8
s.find_first_not_of("12noi spm") found ; at 12
s.find_first_not_of("noon is 12pm; midnight is not"): -1
```

8.8. Замена и удаление символов в строке

Класс `std::string` содержит функции для замены и удаления отдельных символов в строках. Этот пример, как и предыдущие, разделен на части (илл. 8.7). Объект `string1` объявляется с инициализацией строковыми литералами, разделенными символами пропуска (строки 9–13). Когда строковые литералы разделены таким образом, компилятор выполняет их конкатенацию, то есть объединяет в одну строку. Такая форма записи длинных строк может сделать ваш код более наглядным.


```
1 // fig08_07.cpp
2 // Замена и удаление символов в строке
3 #include <fmt/format.h>
4 #include <iostream>
5 #include <string>
6
7 int main() {
8     // Компилятор объединяет все литералы в одну инициализирующую строку
9     std::string string1{"The values in any left subtree"
10         "\nare less than the value in the"
11         "\nparent node and the values in"
12         "\nany right subtree are greater"
13         "\nthan the value in the parent node"};
14
15     std::cout << fmt::format("Original string:\n{}\n\n", string1);
16 }
```

```
Original string:
The values in any left subtree
are less than the value in the
parent node and the values in
any right subtree are greater
than the value in the parent node
```

Илл. 8.7 | Замена и удаление символов в строке

Функция `erase` класса `std::string` (строка 17) удаляет символ с индексом 62 и все последующие символы до конца `string1`. При этом учитываются не только буквы, цифры и другие отображаемые символы, но и имеющиеся в `string1` символы перевода строки (`\n`), каждый из которых занимает одну позицию.

```
17 string1.erase(62); // Удаляем символ с индексом 62 и все последующие
18 std::cout << fmt::format("string1 after erase:\n{}\n\n", string1);
19
```

```
string1 after erase:
The values in any left subtree
are less than the value in the
```

В строках 20–26 функция `find` используется, чтобы найти все пробелы. Функция `replace` класса `std::string` заменяет каждый найденный пробел точкой. Эта версия функции `replace` принимает три аргумента:

- ♦ индекс символа, с которого должна начинаться замена;
- ♦ количество заменяемых символов;
- ♦ строка для замены.

Функция `find` возвращает `string::npos`, если искомый символ не найден. В строке 25 мы прибавляем 1 к переменной `position`, чтобы продолжить поиск с символа, следующего за найденным пробелом.

```

20  size_t position{string1.find(" ")}; // Находим первый пробел
21
22  // Заменяем каждый пробел точкой
23  while (position != std::string::npos) {
24      string1.replace(position, 1, ".");
25      position = string1.find(" ", position + 1);
26  }
27
28  std::cout << fmt::format("After first replacement:\n{}\n\n", string1);
29

```

```

After first replacement:
The values in any left subtree
are less than the value in the

```

В строках 30–37 функция `find` используется, чтобы найти все точки. Функция `replace` заменяет каждую найденную точку, а также следующий за точкой символ двумя точками с запятой. Эта версия функции `replace` принимает пять аргументов:

- ♦ индекс символа, с которого должна начинаться замена;
- ♦ количество заменяемых символов;
- ♦ строка для замены (из которой будет взята заменяющая подстрока);
- ♦ индекс символа в строке для замены, с которого начинается заменяющая подстрока;
- ♦ количество символов в заменяющей подстроке.

```

30  position = string1.find("."); // Находим первую точку
31
32  // Заменяем каждую точку двумя точками с запятой
33  // Внимание: при этом перезаписываются другие символы
34  while (position != std::string::npos) {
35      string1.replace(position, 2, "xxxxx;yyy", 5, 2);
36      position = string1.find(".", position + 2);
37  }
38
39  std::cout << fmt::format("After second replacement:\n{}\n", string1);
40 }

```

```

After second replacement:
The;alues;;n;ny;eft;ubtree
are;;ess;;han;he;alue;;n;he

```

8.9. Вставка символов в строку

Класс `std::string` содержит перегруженные функции для вставки символов в строку (илл. 8.8). Функция `insert` (строка 14) вставляет фрагмент "middle "

перед символом строки `s1` с индексом 10. Затем другая версия функции `insert` (строка 15) вставляет фрагмент "xx" перед символом строки `s2` с индексом 3. Последние два аргумента определяют начало и конец той последовательности символов, которая должна быть вставлена в строку `s2` из фрагмента "xx". Аргумент `string::npos` означает, что надо вставить весь этот фрагмент.

```
1 // fig08_08.cpp
2 // Вставка символов в строку
3 #include <fmt/format.h>
4 #include <iostream>
5 #include <string>
6
7 int main() {
8     std::string s1{"beginning end"};
9     std::string s2{"12345678"};
10
11     std::cout << fmt::format("Initial strings:\ns1: {}\ns2: {}\n\n",
12         s1, s2);
13
14     s1.insert(10, "middle "); // Вставляем "middle " в позицию 10
15     s2.insert(3, "xx", 0, std::string::npos); // Вставляем "xx" в позицию 3
16
17     std::cout << fmt::format("Strings after insert:\ns1: {}\ns2: {}\n",
18         s1, s2);
19 }
```

Initial strings:

s1: beginning end

s2: 12345678

Strings after insert:

s1: beginning middle end

s2: 123xx45678

Илл. 8.8 | Вставка символов в строку

8.10. Преобразования между числами и строками (C++11)

В C++11 появились функции для преобразования числовых значений в строки и обратно.

Преобразование чисел в объекты `std::string`

Функция `to_string` из заголовка `<string>` возвращает строковое представление числового аргумента. Функция перегружена для основных числовых типов: `int`, `unsigned int`, `long`, `unsigned long`, `long long`, `unsigned long long`, `float`, `double` и `long double`.

Преобразование объектов `std::string` в числа

Для преобразования объектов `std::string` в числовые значения предназначены восемь функций из заголовка `<string>`:

Функция		Тип возвращаемого значения	
Функции, преобразующие в целочисленные типы		Функции, преобразующие в типы с плавающей точкой	
<code>stoi</code>	<code>int</code>	<code>stof</code>	<code>float</code>
<code>stol</code>	<code>long</code>	<code>stod</code>	<code>double</code>
<code>stoul</code>	<code>unsigned long</code>	<code>stold</code>	<code>long double</code>
<code>stoll</code>	<code>long long</code>		
<code>stoull</code>	<code>unsigned long long</code>		

Каждая из этих функций пытается преобразовать начало своего аргумента типа `std::string` в числовое значение. Если это невозможно, функция выдает исключение `invalid_argument`. Если результат преобразования превышает диапазон возвращаемого функцией типа данных, функция выдает исключение `out_of_range`.

Функции, преобразующие строки в целые числа

Рассмотрим пример преобразования строки в целое число. Предположим, что у нас есть строка:

```
string s{"100hello"};
```

Тогда следующая инструкция преобразует начало строки в число `100` типа `int` и сохранит полученное значение в переменную `convertedInt`:

```
int convertedInt{stoi(s)};
```

Каждая функция, преобразующая строку в целочисленный тип, получает три аргумента (для последних двух есть значения по умолчанию):

- ♦ Строка `std::string`, содержащая символы для преобразования.
- ♦ Указатель на переменную типа `size_t`. В нее функция сохраняет индекс первого символа, который не был преобразован. Аргумент по умолчанию — `nullptr`, в этом случае функция не сохраняет индекс.
- ♦ Значение типа `int`, которое либо равно `0`, либо находится в диапазоне от `2` до `36` и обозначает основание системы счисления. Аргумент по умолчанию — `10`. Если вызвать функцию с аргументом `0`, она определит основание автоматически.

Таким образом, предыдущая инструкция эквивалентна следующей:

```
int convertedInt{stoi(s, nullptr, 10)};
```

Если добавить переменную типа `size_t` с именем `index`, инструкция

```
int convertedInt(stoi(s, &index, 2));
```

воспримет последовательность символов "100" в начале строки `s` как число 100 в двоичной системе счисления (с основанием 2), преобразует его в число 4 типа `int` (двоичное 100 — это десятичное 4) и сохранит в переменной `index` индекс буквы "h" — первого символа, который не был преобразован.

Функции, преобразующие строки в числа с плавающей точкой

Каждая функция, преобразующая строки в числа с плавающей точкой, получает два аргумента:

- ✦ Строка, содержащая символы для преобразования.
- ✦ Указатель на переменную типа `size_t`. В нее функция сохраняет индекс первого символа, который не был преобразован (то есть количество преобразованных символов). Аргумент по умолчанию — `nullptr`, в этом случае функция не сохраняет индекс.

Рассмотрим преобразование следующей строки:

```
string s{"123.45hello"};
```

Инструкция

```
double convertedDouble(stod(s));
```

преобразует начало строки `s` в число 123.45 типа `double` и сохранит его в переменную `convertedDouble`. Второй аргумент по умолчанию равен `nullptr`.

8.11. Представления `string_view` (C++17)

В C++17 появился тип данных `string_view` (заголовок `<string_view>`). Объекты этого типа являются **доступными только для чтения (read-only)** представлениями строк `std::string` или строк в стиле C. Кроме того, эти объекты могут быть представлениями диапазонов символов в контейнерах, например в массивах и векторах, состоящих из элементов типа `char`. Объект `string_view` так же, как и объект `std::span`, не владеет теми данными, которые представляет (в нем нет собственной копии этих данных). Он содержит:

- ✦ указатель на первый символ в непрерывной последовательности символов;
- ✦ количество символов.

Объект `string_view` не обновляется автоматически, если после его инициализации представляемые им данные (например, строка `std::string`) изменяются в размере!

С помощью представлений `string_view` многие из операций, доступных для строк `std::string`, можно применять и к строкам в стиле C, причем без необходимости

создания и инициализации объектов `std::string`, содержащих копии строк в стиле C. Согласно C++ Core Guidelines, вы должны отдавать предпочтение объектам `std::string` в тех случаях, когда вам нужно «владеть последовательностями символов», например вносить изменения в текст¹. Но если какая-либо непрерывная последовательность символов нужна вам только для чтения, рекомендуется использовать `string_view`².

Работа с объектами `string_view`

Рассмотрим некоторые возможности `string_view`. Мы разделили этот пример на части для удобства обсуждения (илл. 8.9). В программу включен заголовок `<string_view>` (строка 6). Мы создаем объект `std::string` с именем `s1` (строка 9) и копируем его содержимое в объект `std::string` с именем `s2` (строка 10). Затем мы создаем объект `string_view` с именем `v1` и инициализируем его содержимым `s1` (строка 11).

```
1 // fig08_09.cpp
2 // Работа с представлениями string_view (C++17)
3 #include <fmt/format.h>
4 #include <iostream>
5 #include <string>
6 #include <string_view>
7
8 int main() {
9     std::string s1{"red"};
10    std::string s2{s1};
11    std::string_view v1{s1}; // v1 "видит" содержимое строки s1
12    std::cout << fmt::format("s1: {}\\ns2: {}\\nv1: {}\\n\\n", s1, s2, v1);
13 }
```



```
s1: red
s2: red
v1: red
```

Илл. 8.9 | Работа с представлениями `string_view`

Объект `string_view` «видит» все изменения символов, которые он представляет

Поскольку объект `string_view` не содержит той последовательности символов, которую представляет, он «видит» все происходящие изменения исходных символов. Чтобы убедиться в этом, мы вносим изменение в `s1` (строка 15), а затем выводим на экран `s1`, `s2` и `v1`. Содержимое `s2` не изменилось, так как это независимая копия `s1`.

¹ C++ Core Guidelines, «SL.str.1: Use `std::string` to Own Character Sequences». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rstr-string>.

² C++ Core Guidelines, «SL.str.2: Use `std::string_view` or `gsl::span<char>` to Refer to Character Sequences». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rstr-view>.

```
14 // string_view "видит" все изменения исходной строки
15 s1.at(0) = 'R'; // Теперь s1 начинается с большой буквы
16 std::cout << fmt::format("s1: {}ns2: {}nv1: {}n\n", s1, s2, v1);
17
```

```
s1: Red
s2: red
v1: Red
```

Сравнение объектов `string_view`

Подобно объектам `std::strings`, объекты `string_view` можно сравнивать между собой операторами равенства и сравнения. Кроме того, возможны сравнения между `string_view` и `std::string` (строки 19–20).

```
18 // Объекты string_view можно сравнивать между собой и с объектами string
19 std::cout << fmt::format("s1 == v1: {}ns2 == v1: {}n\n",
20     s1 == v1, s2 == v1);
21
```

```
s1 == v1: true
s2 == v1: false
```

Функции `remove_prefix` и `remove_suffix`

Вы можете легко удалить нужное количество символов с начала или конца `string_view`. Такие операции выполняются очень быстро, так как корректируется не сама строка, представленная объектом `string_view`, а только указатель на первый символ в последовательности (при удалении с начала) или на количество символов (при удалении с конца). В нашем примере функции `remove_prefix` и `remove_suffix` класса `string_view` (строки 23–24) вызываются для удаления первого и последнего символов соответственно. Как видите, символы удаляются только из представления `v1`, а исходная строка `s1` остается неизменной.

```
22 // string_view позволяет удалить начало и конец строки
23 v1.remove_prefix(1); // Удаляем 1 символ с начала строки
24 v1.remove_suffix(1); // Удаляем 1 символ с конца строки
25 std::cout << fmt::format("s1: {}nv1: {}n\n", s1, v1);
26
```

```
s1: Red
v1: e
```

Обработка элементов `string_view` в цикле

В строке 28 создается объект `string_view` с именем `v2` и инициализируется строкой в стиле C. Объекты `string_view` являются итерируемыми, поэтому их элементы можно обрабатывать в цикле `for` для диапазона значений (строки 30–32).

```

27 // Объекты string_view - итерируемые
28 std::string_view v2{"C-string"};
29 std::cout << "The characters in v2 are: ";
30 for (char c : v2) {
31     std::cout << c << " ";
32 }
33

```

```
The characters in v2 are: C - s t r i n g
```

Операции с объектом string_view, представляющим строку в стиле C

Многие функции класса string, не изменяющие содержимое строки, определены и для string_view. Некоторые из них мы включили в нашу программу:

- ✦ Функция size возвращает количество символов, которые «видит» объект string_view (строка 35).
- ✦ Функция find возвращает индекс символа '-' (строка 36).
- ✦ Функция starts_with (C++20) возвращает ответ на вопрос, начинается ли string_view с символа 'C' (строки 37–38).

Полный список функций класса string_view приведен здесь:

https://en.cppreference.com/w/cpp/string/basic_string_view

```

34 // Операции со строкой в стиле C через ее представление string_view
35 std::cout << fmt::format("\n\v2.size(): {}\n", v2.size());
36 std::cout << fmt::format("\nv2.find('-'): {}\n", v2.find('-'));
37 std::cout << fmt::format("\nv2.starts_with('C'): {}\n",
38     v2.starts_with('C'));
39 }

```

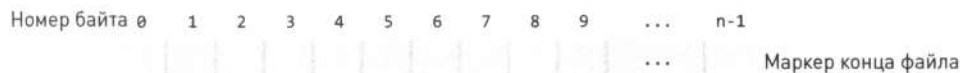
```

v2.size(): 8
v2.find('-'): 1
v2.starts_with('C'): true

```

8.12. Файлы и потоки

C++ рассматривает каждый файл как последовательность байтов, например:



Текстовый файл обычно заканчивается либо **маркером конца файла (end-of-file marker)**, либо определенным байтом в соответствии с принятой в операционной системе структурой данных. При открытии файла создается объект, с которым связан поток. Объекты cin, cout, cerr и clog, предназначенные для работы с по-

токами, находятся в заголовке `<iostream>`. Потоки, связанные с этими объектами, обеспечивают каналы связи между вашей программой и конкретными файлами или устройствами. Через объекты `cin` (объекты стандартного потока ввода) программа получает данные от клавиатуры и от других устройств. Через объекты `cout` (объекты стандартного потока вывода) программа выводит данные на экран и на другие устройства. Через объекты `cerr` и `clog` (объекты стандартного потока ошибок) программа выводит сообщения об ошибках на экран и на другие устройства. Сообщения, записанные в `cerr`, выводятся немедленно, а сообщения, записанные в `clog`, хранятся в буфере памяти. Когда буфер заполняется или сбрасывается, его содержимое записывается в стандартный поток ошибок.

Файловые потоки

Для обработки файлов требуется заголовок `<fstream>`, который включает в себя следующие определения:

- ♦ `ifstream` — для чтения (ввода данных) из файлов;
- ♦ `ofstream` — для записи (вывода данных) в файлы;
- ♦ `fstream` — объединяет возможности `ifstream` и `ofstream`.

Возможности объектов `cout` и `cin`, которые мы рассматривали ранее, могут быть применимы и к файловым потокам.

8.13. Создание файла с последовательным доступом

C++ не требует какой-либо конкретной структуры файлов. Кроме того, не существует какой-либо обязательной концепции записей (элементов файла), содержащих взаимосвязанные элементы данных. Вы можете создать в файле практически любую структуру данных в соответствии с требованиями вашей программы. Мы покажем пример создания файла с очень простой структурой.

Программа (илл. 8.10) создает **файл с последовательным доступом** (*sequential file*), хранящий данные учета денежных средств, причитающихся фирме от ее кредитных клиентов. Для каждого клиента в программу вводятся три значения: номер счета, имя и баланс (сумма, которую клиент должен компании за товары и услуги, полученные в прошлом). Эти три значения, взятые вместе, являются **записью (record)**, то есть набором данных о клиенте.

```
1 // fig08_10.cpp
2 // Создание последовательного файла
3 #include <cstdlib> // Содержит прототип функции exit
4 #include <fmt/format.h>
5 #include <fstream> // Содержит типы для обработки файловых потоков
6 #include <iostream>
7 #include <string>
8
```

```

9 int main() {
10     // Объект ofstream открывает файл
11     if (std::ofstream output("clients.txt", std::ios::out)) {
12         std::cout << "Enter the account, name, and balance.\n"
13             << "Enter end-of-file to end input.\n? ";
14
15         int account;
16         std::string name;
17         double balance;
18
19         // Вводим account, name и balance из cin, затем записываем в файл
20         while (std::cin >> account >> name >> balance) {
21             output << fmt::format("{} {} {}\n", account, name, balance);
22             std::cout << "? ";
23         }
24     }
25     else {
26         std::cerr << "File could not be opened\n";
27         std::exit(EXIT_FAILURE);
28     }
29 }

```

```

Enter the account, name, and balance.
Enter end-of-file to end input.
? 100 Jones 24.98
? 200 Doe 345.67
? 300 White 0.00
? 400 Stone -42.16
? 500 Rich 224.62
? ^Z

```

Илл. 8.10 | Создание файла с последовательным доступом

Открытие файла

Чтобы записать данные в новый файл, мы прежде всего создаем объект `ofstream` (строка 11) и инициализируем его **именем файла (filename)** и **режимом открытия файла (file-open mode)**. Режим открытия файла `ios::out` предназначен для записи в файл (для вывода данных в файл). Этот режим установлен для всех объектов `ofstream` по умолчанию, поэтому второй аргумент в строке 11 необязателен. Открытие существующего файла в режиме `ios::out` приводит к немедленному удалению из этого файла всех данных без предупреждения! Если файл с указанным именем не существует, объект `ofstream` его создает.

Созданный нами объект `ofstream` с именем `output` связан с файлом `clients.txt` и открыт для записи. Мы не указали путь к файлу, поэтому он будет расположен в папке с исполняемым файлом программы. До C++11 имя файла указывалось в виде строки в стиле C. В C++11 была добавлена возможность указать имя файла в виде объекта `string`. В C++17 появился заголовок `<filesystem>`, содержащий средства управления файлами и папками. Таким образом, вы можете указывать открываемый файл в виде объекта `filesystem::path`.

В следующей таблице перечислены режимы открытия файлов. Эти режимы можно комбинировать, разделяя их оператором |:

Режим	Описание
<code>ios::app</code>	Добавление данных с конца файла, то есть вывод данных без изменения имеющихся в нем данных
<code>ios::ate</code>	Открытие файла для вывода и переход к концу файла. Используется для добавления данных в файл. Данные могут быть записаны в любом месте файла
<code>ios::in</code>	Открытие файла для чтения данных
<code>ios::out</code>	Открытие файла для записи данных
<code>ios::trunc</code>	Удаление содержимого файла. Это всегда происходит при открытии файла в режиме <code>ios::out</code>
<code>ios::binary</code>	Открытие файла для ввода или вывода двоичных (то есть нетекстовых) данных

Открытие файла с помощью функции `open` класса `ofstream`

Вы можете создать объект `ofstream` без немедленного открытия файла, а впоследствии связать с этим объектом нужный вам файл. Например, инструкция:

```
ofstream output;
```

создает объект `ofstream`, который не связан ни с каким файлом. После этого функция `open` открывает файл и связывает его с существующим объектом `ofstream`:

```
output.open("clients.txt", ios::out);
```

Напомним, что `ios::out` — это значение по умолчанию для второго аргумента.

Проверка успешности открытия файла

После создания объекта `output` и попытки открыть файл инструкция `if` использует этот объект в качестве условия (строка 11), чтобы определить, удалось ли открыть файл. Для этого к объекту файла неявно применяется перегруженная функция `operator bool` (C++11), вычисляющая его значение таким образом: `true`, если файл успешно открыт, или `false`, если возникла ошибка. Перечислим несколько причин, вызывающих ошибку при открытии файла:

- ✦ попытка открыть для чтения несуществующий файл;
- ✦ попытка открыть для чтения или для записи файл в папке, к которой у программы нет прав доступа;
- ✦ открытие файла для записи при отсутствии места на диске.

Если при проверке условия (строка 11) выясняется, что файл не удалось открыть, выводится сообщение об ошибке (строка 26), а затем вызывается функция `exit`

(строка 27) для завершения программы. Аргумент функции `exit` передается операционной системе, из которой была вызвана (запущена) программа. Передача значения `EXIT_SUCCESS` (определенного в `<stdlib.h>`) сообщает операционной системе, что программа завершилась нормально; передача любого другого значения (в данном случае `EXIT_FAILURE`) сообщает, что программа завершилась из-за ошибки¹.

Обработка данных

Если файл успешно открыт (строка 11), программа начинает обработку данных. Пользователю предлагается заполнить поля в каждой записи о клиенте (строки 12–13) или ввести индикатор конца файла, если ввод данных завершен. Чтобы ввести индикатор конца файла в Linux или macOS, нажмите `Ctrl + d`. В Windows нажмите `Ctrl + z` и затем `Enter`.

При проверке условия в инструкции `while` (строка 20) к объекту `cin` неявно применяется функция `operator bool`, вычисляющая его значение таким образом: `true`, пока продолжаются успешные операции ввода данных, или `false`, если введен индикатор конца файла. Вы также можете вызвать функцию `eof` объекта `cin`, чтобы определить, был ли введен индикатор конца файла.

Строка 20 сохраняет каждый набор данных о клиенте в переменные `account`, `name` и `balance`, а также определяет, был ли введен индикатор конца файла. Если обнаружен конец файла (то есть пользователь нажал соответствующую комбинацию клавиш) или возникла ошибка операции ввода, `operator bool` возвращает `false` и инструкция `while` завершается.

В строке 21 все введенные данные записываются в файл `clients.txt` с помощью оператора вывода в поток (`<<`) и объекта `output`, который мы связали с файлом в начале программы. Записанные данные мы скоро прочитаем другой программой (раздел 8.14). Их также можно прочитать в любом текстовом редакторе, так как мы создали обычный текстовый файл.

Закрытие файла

Когда пользователь вводит индикатор конца файла, цикл `while` завершается. В этот момент достигается закрывающая фигурная скобка инструкции `if`, поэтому объект `output` выходит за пределы области видимости и файл автоматически закрывается. Вы всегда должны закрывать каждый файл, когда он больше не нужен в программе. Для этого можно явным образом закрыть связанный с файлом объект функцией `close`:

```
output.close();
```

¹ В функции `main` вы можете просто вернуть `EXIT_SUCCESS` или `EXIT_FAILURE` без вызова `std::exit`. Когда функция `main` завершается, ее локальные переменные уничтожаются, затем функция `main` вызывает `std::exit`, передавая значение из инструкции `return` в теле `main` (<https://timsong-cpp.github.io/cppwp/n4861/basic.start.main#5>). Напомним, что `main` неявно возвращает 0 в случае успешного выполнения, если в этой функции нет инструкции `return`.

Выходные данные

В конце илл. 8.10 в качестве примера приведены данные, которые мы ввели для пяти записей о клиентах. В последней строке вы видите символы ^Z, которые отображаются в Windows при вводе индикатора конца файла. Это диалоговое окно не показывает, каким образом введенные данные были записаны в файл. В следующем разделе мы рассмотрим программу для чтения и вывода на экран содержимого созданного файла.

8.14. Чтение данных из файла с последовательным доступом

Наша предыдущая программа (илл. 8.10) создала файл с последовательным доступом. Следующая программа (илл. 8.11) последовательно читает данные из файла `clients.txt` и выводит на экран все записи о клиентах. При создании объекта `ifstream` связанный с ним файл открывается для чтения (для ввода данных из файла). Объект `ifstream` инициализируется именем файла и режимом открытия файла. Мы создаем объект `ifstream` с именем `input` (строка 11), который открывает файл `clients.txt` для чтения¹. Когда вам не нужно изменять содержимое файла, указывайте режим `ios::in`, чтобы открывать файл только для чтения и тем самым защитить его от случайной записи.

```
1 // fig08_11.cpp
2 // Чтение файла с последовательным доступом и вывод данных на экран
3 #include <cstdlib>
4 #include <fmt/format.h>
5 #include <fstream> // Для работы с файлами
6 #include <iostream>
7 #include <string>
8
9 int main() {
10     // Объект ifstream открывает файл
11     if (std::ifstream input{"clients.txt", std::ios::in}) {
12         std::cout << fmt::format("{:<10}{:<13}{:>7}\n",
13             "Account", "Name", "Balance");
14
15         int account;
16         std::string name;
17         double balance;
18
19         // Выводим на экран каждую запись из файла
20         while (input >> account >> name >> balance) {
21             std::cout << fmt::format("{:<10}{:<13}{:>7.2f}\n",
22                 account, name, balance);
23         }
24     }
```

¹ Файл `clients.txt` должен находиться в папке с исполняемым файлом программы.

```

25     else {
26         std::cerr << "File could not be opened\n";
27         std::exit(EXIT_FAILURE);
28     }
29 }

```

Account	Name	Balance
100	Jones	24.98
200	Doe	345.67
300	White	0.00
400	Stone	-42.16
500	Rich	224.62

Илл. 8.11 | Чтение файла с последовательным доступом и вывод данных на экран

Открытие файла для чтения

Все объекты класса `ifstream` по умолчанию открывают файлы для чтения, поэтому вы можете опустить второй аргумент (`ios::in`) при создании объекта `ifstream`:

```
std::ifstream input("clients.txt");
```

Вы можете создать объект `ifstream` без немедленного открытия файла, а впоследствии связать с этим объектом нужный вам файл. В строке 11 перед чтением данных из файла инструкция `if` использует объект `input` в качестве условия, чтобы определить, удалось ли открыть файл. Перегруженная функция `operator bool` вычисляет значение этого объекта таким образом: `true`, если файл успешно открыт, или `false`, если возникла ошибка.

Чтение данных из файла

В строке 20 считывается набор данных (запись) из файла. После первого выполнения этой инструкции переменная `account` содержит значение `100`, переменная `name` — `Jones`, а переменная `balance` — `24.98`. При каждом последующем выполнении этой инструкции в переменных `account`, `name` и `balance` сохраняется очередная запись о клиенте. В строках 21–22 каждая запись выводится на экран. Когда достигнут конец файла, неявный вызов `operator bool` в условии инструкции `while` возвращает `false`, объект `ifstream` выходит из области видимости (строка 24) и файл автоматически закрывается. Программа завершается.

Указатели позиции в файле

Многие программы последовательно читают все данные с начала файла до тех пор, пока не будут найдены нужные данные. Иногда требуется обработать файл несколько раз (с самого начала) во время выполнения программы. В классах `istream` и `ostream` есть функции `seekg` («seek get») и `seekp` («seek put»), которыми вы можете устанавливать **указатели позиции в файле (file-position pointers)**. Эти указатели содержат индексы тех байтов, которые должны быть прочитаны или записаны. Каждый объект `istream` имеет **указатель чтения (get pointer)** — индекс того байта в файле, с которого начнется следующая операция

чтения. Каждый объект `ostream` имеет **указатель записи (put pointer)** — индекс того байта в файле, с которого начнется следующая операция записи. Например, инструкция:

```
input.seekg(0);
```

устанавливает указатель чтения на начало файла (на байт с индексом 0), связанного с объектом `input`. Аргументом этой функции должно быть целое число. Если ранее был введен индикатор конца файла, то для возобновления чтения из потока надо выполнить инструкцию:

```
input.clear();
```

Необязательный второй аргумент функции `seekg` определяет режим установки указателя:

- ♦ `ios::beg` (по умолчанию) — от начала потока;
- ♦ `ios::cur` — от текущей позиции в потоке;
- ♦ `ios::end` — назад от конца потока.

При установке указателя позиции относительно начала потока значением указателя является **смещение (offset)**, то есть количество байтов от начала файла. Приведем примеры установки указателя чтения на разные байты файла, связанного с объектом `fileObject`:

```
// Устанавливаем указатель на байт с индексом n (по умолчанию - ios::beg)
fileObject.seekg(n);
// Перемещаем указатель на n байт вперед от текущей позиции
fileObject.seekg(n, ios::cur);
// Устанавливаем указатель на n байт с конца файла
fileObject.seekg(n, ios::end);
// Устанавливаем указатель на последний байт файла
fileObject.seekg(0, ios::end);
```

Аналогичные операции выполняет в потоках `ostream` функция `seekp`. В классах `istream` и `ostream` также есть функции `tellg` и `tellp`, возвращающие текущие позиции указателей чтения и записи соответственно. Следующая инструкция присваивает значение указателя чтения переменной `location`, которая имеет тип `std::istream::pos_type`:

```
auto location{fileObject.tellg()};
```

8.15. Чтение и запись текста в кавычках (C++14)

Во многих текстовых файлах встречаются кавычки, в том числе универсальные двойные кавычки, применяемые программистами: "C++20 for Programmers". Например, в текстовых файлах, представляющих собой веб-страницы HTML5, значения атрибутов заключены в кавычки. При создании таких программ, как

браузеры, отображающие содержимое веб-страниц, нужна возможность читать строки в кавычках и удалять эти кавычки.

Предположим, вам надо выполнить чтение набора данных из текстового файла (тем способом, который показан на илл. 8.11), но каждая запись о клиенте в этом файле имеет следующий формат с именем и фамилией:

```
100 "Janie Jones" 24.98
```

Напомним, что оператор извлечения из потока (>>) обрабатывает пробелы (символы пропуска) как разделители. Предположим, мы попробуем ввести эти данные, используя выражение из строки 20 на илл. 8.11:

```
input >> account >> name >> balance
```

Тогда переменной `account` будет присвоено значение `100` (первая операция ввода), а переменной `name` — значение `"Janie"` (вторая операция ввода), причем открывающая кавычка станет частью строки. Третья операция ввода завершится ошибкой, поскольку третьим токеном (фрагментом данных) в потоке является последовательность символов `Jones`, которая не может быть присвоена переменной `balance` типа `double`.

Чтение текста в кавычках

В C++14 был добавлен манипулятор потока `quoted` (заголовок `<locale>`) для чтения текста в кавычках из потока. Он включает в строки все пробелы, которые находятся во фрагментах, заключенных в кавычки, и удаляет кавычки. Снова рассмотрим ввод той же записи о клиенте:

```
100 "Janie Jones" 24.98
```

Инструкция

```
input >> account >> std::quoted(name) >> balance
```

присвоит переменной `account` значение `100`, переменной `name` — значение `Janie Jones` (одной строкой) и переменной `balance` — значение `24.98`. Если в данных, заключенных в кавычки, есть `escape`-последовательности `"` или `\`, они считываются как `"` или `\` соответственно.

Запись текста в кавычках

Запись текста в кавычках происходит аналогичным образом. Например, если переменная `name` содержит значение `Janie Jones`, инструкция

```
outputStream << std::quoted(name);
```

запишет в `outputStream` строку

```
"Janie Jones"
```

Если строка содержит символы `"` или `\`, они будут записаны как `"` или `\` соответственно.

8.16. Обновление данных в файле с последовательным доступом

Отформатированные данные, записанные в файл с последовательным доступом, как показано в разделе 8.13, нельзя изменять без риска уничтожения других данных в этом файле. Например, если в записи о клиенте надо заменить имя `White` именем `Worthington`, это невозможно сделать, не повредив файл. Предположим, в файле есть такая запись:

```
300 White 0.00
```

Если перезаписать ее в файле с той же позиции, но с более длинным именем клиента, запись будет выглядеть так:

```
300 Worthington 0.00
```

Эта новая запись на шесть символов длиннее прежней. Поэтому все символы после "h" в слове "Worthington" будут записаны в начало следующей записи, то есть повредят ее. Проблема модели форматированного ввода-вывода, использующей операторы ввода в поток (<<) и извлечения из потока (>>), заключается в том, что записи (элементы данных) могут различаться по размеру. Например, числа 7, 14, -117, 2074 и 27383 имеют тип `int`, поэтому занимают фиксированное количество байтов, если записываются в файл в формате своего типа. Но эти же числа потребуют разного количества байтов, если записываются в файл в виде форматированного текста (последовательностей символов). Поэтому модель форматированного ввода-вывода, как правило, не используется для обновления записей в файлах.

Для обновления записей в файлах с последовательным доступом есть относительно сложные способы. Например, чтобы заменить имя `White` более длинным именем `Worthington`, мы можем:

- ♦ скопировать в новый файл все записи, которые находятся перед записью `300 White 0.00`;
- ♦ внести в новый файл запись `300 Worthington 0.00`;
- ♦ скопировать в новый файл все оставшиеся записи.

Затем мы можем удалить старый файл и переименовать новый. Этот способ требует обработки всех имеющихся в файле записей, даже если обновляется только одна из них. Он может применяться, когда надо обновлять много записей за один проход.

8.17. Обработка строковых потоков

В дополнение к стандартным потокам ввода-вывода и файловым потокам ввода-вывода в C++ есть аналогичные потоки для обмена данными с находящимися в памяти строками класса `string`. Они называются **потоками ввода-вывода**

в памяти (**in-memory I/O streams**), или **строковыми потоками (string streams)** и определены в заголовке `<sstream>`. Вы можете читать данные из строки через строковый поток `istreamstream` и записывать данные в строку через строковый поток `ostreamstream`.

Шаблоны классов `istreamstream` и `ostreamstream` предоставляют такой же набор функций, как и классы `istream` и `ostream`, а также дополнительные функции, специфичные для форматирования в памяти. Объект `ostreamstream` использует объект `string`, чтобы сохранять в него выводимые данные. Функция `str` класса `ostreamstream` возвращает копию этого объекта `string`.

Строковые потоки применяются, в частности, для проверки данных. Программа может временно сохранить в объекте `string` последовательность символов, полученных из потока ввода (введенных с клавиатуры), а затем проанализировать эти данные и при необходимости скорректировать их (исправить формат). Затем программа может выполнить ввод данных из этого объекта `string`, поскольку теперь их формат наверняка соответствует требованиям.

Чтобы расширить возможности проверки данных, в C++11 были добавлены регулярные выражения, основанные на поиске и замене фрагментов текста по шаблонам. Если пользователь должен ввести телефонный номер в формате США, например (800) 555-1212, вы можете использовать регулярное выражение для проверки соответствия введенной строки этому формату. Многие веб-сайты предоставляют регулярные выражения для проверки адресов электронной почты, URL-адресов, телефонных номеров и других популярных видов данных. В разделе 8.20 мы расскажем о регулярных выражениях и приведем несколько примеров.

Вывод данных в `ostreamstream`

В следующей программе (илл. 8.12) мы создаем объект `ostreamstream` с именем `output` (строка 8), а затем записываем в него строки и числа, используя оператор вывода в поток.

```
1 // fig08_12.cpp
2 // Вывод данных в объект ostreamstream
3 #include <iostream>
4 #include <sstream> // Заголовок для строковых потоков
5 #include <string>
6
7 int main() {
8     std::ostreamstream output; // Создаем объект ostreamstream
9
10    const std::string string1{"Output of several data types "};
11    const std::string string2{"to an ostreamstream object:"};
12    const std::string string3{"\ndouble: "};
13    const std::string string4{"\n int: "};
14
15    constexpr double d{123.4567};
16    constexpr int i{22};
17
```

```

18 // Выводим значения типов string, double и int в объект ostringstream
19 output << string1 << string2 << string3 << d << string4 << i;
20
21 // Вызываем функцию str, чтобы получить строку с содержимым ostringstream
22 std::cout << "output contains:\n" << output.str();
23
24 // Добавляем символы и снова вызываем функцию str
25 output << "\nmore characters added";
26 std::cout << "\n\noutput now contains:\n" << output.str() << '\n';
27 }

```

```

output contains:
Output of several data types to an ostringstream object:
double: 123.457
int: 22

output now contains:
Output of several data types to an ostringstream object:
double: 123.457
int: 22
more characters added

```

Илл. 8.12 | Вывод данных в объект `ostringstream`

В поток `output` поочередно выводятся (строка 19): строковые переменные `string1`, `string2`, `string3`, переменная `d` типа `double`, строковая переменная `string4` и переменная `i` типа `int`. Функция `output.str()` (строка 22) возвращает созданный объектом `output` объект `string`, который отображается на экране. Затем в поток `output` выводятся дополнительные данные (строка 25) и обновленное содержимое объекта `string` отображается на экране (строка 26).

Ввод данных из `istringstream`

Объект `istringstream` получает данные из находящейся в памяти строки, то есть из объекта типа `string`. Эти данные хранятся в объекте `istringstream` в виде символов. Ввод данных из объекта `istringstream` выполняется так же, как из любого файла. Конец строки интерпретируется объектом `istringstream` как конец файла.

На илл. 8.13 показан пример ввода данных из объекта `istringstream`. Объект `string` с именем `inputString` (строка 9) инициализируется как источник данных для двух строковых переменных ("`Amanda`" и "`test`"), для переменной типа `int` (123), для переменной типа `double` (4.7) и для переменной типа `char` ('A'). Объект `istringstream` с именем `input` (строка 10) инициализируется содержимым `inputString`. Выполняется ввод данных из `inputString` в переменные `s1`, `s2`, `i`, `d` и `c` (строка 17), значения которых отображаются на экране (строки 19–20). Затем мы снова пытаемся выполнить ввод из `input` (строка 23), но эта операция невозможна из-за отсутствия в `inputString` каких-либо дополнительных данных. Поэтому значение объекта `input` вычисляется как `false` и выполняется секция `else` инструкции `if...else`.

```
1 // fig08_13.cpp
2 // Ввод данных из объекта istringstream
3 #include <fmt/format.h>
4 #include <iostream>
5 #include <sstream>
6 #include <string>
7
8 int main() {
9     const std::string inputString{"Amanda test 123 4.7 A"};
10    std::istringstream input{inputString};
11    std::string s1;
12    std::string s2;
13    int i;
14    double d;
15    char c;
16
17    input >> s1 >> s2 >> i >> d >> c;
18
19    std::cout << "Items extracted from the istringstream object:\n"
20              << fmt::format("{}\n{}\n{}\n{}\n{}\n", s1, s2, i, d, c);
21
22    // Попробуем выполнить ввод данных из пустого потока
23    if (long value; input >> value) {
24        std::cout << fmt::format("\nlong value is: {}\n", value);
25    }
26    else {
27        std::cout << fmt::format("\ninput is empty\n");
28    }
29 }
```

Items extracted from the istringstream object:

Amanda

test

123

4.7

A

input is empty

Илл. 8.13 | Ввод данных из объекта istringstream

8.18. Необработанные строковые литералы

Напомним, что с символов обратного слеша (\) в строках начинаются escape-последовательности (экранированные последовательности), например \n (перевод строки) и \t (табуляция). Если вы хотите включить в строку символ обратного слеша, вам надо поставить два таких символа подряд, и из-за этого некоторые строки становятся трудными для понимания. Например, в Windows обратный слеш служит для разделения имен папок в пути к файлу. Поэтому в строку, содержащую путь к файлу, приходится включать дополнительные символы:

```
std::string windowsPath{"C:\\MyFolder\\MySubFolder\\MyFile.txt"};
```

Для таких случаев более удобны **необработанные строковые литералы (raw string literals)**, введенные в C++11. Такой литерал может содержать любые символы и записывается в следующем формате:

```
R"(любыеСимволы)"
```

Круглые скобки вокруг символов необработанного строкового литерала обязательны. Компилятор автоматически вставит в такой литерал все необходимые экранирующие символы, то есть преобразует каждый специальный символ (обратный слеш, кавычки и т. д.) в соответствующую escape-последовательность. Используя необработанный строковый литерал, можно записать предыдущую инструкцию так:

```
std::string windowsPath(R"(C:\MyFolder\MySubFolder\MyFile.txt)");
```

Необработанные строковые литералы могут сделать ваш код более наглядным, особенно если в коде есть регулярные выражения, которые мы обсудим в разделе 8.20. Регулярные выражения часто содержат много символов обратного слеша.

Необработанные строковые литералы могут содержать **разделители (delimiters)** длиной до 16 символов: перед левой (открывающей) круглой скобкой и после правой (закрывающей) круглой скобки:

```
R"MYDELIMITER(,.*\d{0-35}-\d\d-\d\d)MYDELIMITER"
```

Если в необработанном строковом литерале есть разделители, они должны быть идентичными. Обычно разделители необязательны. Но если последовательность символов такого литерала заканчивается одной или несколькими закрывающими круглыми скобками, то разделители обязательны, иначе компилятор воспримет первую из закрывающих скобок как конец литерала.

Необработанные строковые литералы можно использовать в любых местах исходного кода вместо обычных строковых литералов. В них допустимы даже разрывы строк, которые компилятор заменяет символами перевода строки. Например, необработанный строковый литерал:

```
R"(multiple lines  
of text)"
```

будет преобразован компилятором в обычный строковый литерал:

```
"multiple lines\nof text"
```

Внимание: если вы записываете необработанный строковый литерал в несколько строк, в него будут включены все отступы во второй и последующих строках.

8.19. Готовые объекты: чтение и анализ CSV-файла с данными о катастрофе «Титаника»

Файлы в формате CSV (comma-separated values — значения, разделенные запятыми), имеющие расширение `.csv`, предназначены для хранения наборов данных.

Этот формат широко применяется для анализа и обработки «больших данных», а также в сфере искусственного интеллекта, например в системах машинного обучения. В этом разделе мы покажем, как прочесть данные из CSV-файла.

Наборы данных

В интернете опубликовано много бесплатных наборов данных. Например, сайт ресурсов для машинного обучения OpenML содержит более 21 тысячи бесплатных наборов данных в формате CSV:

<https://openml.org>

Эта страница портала GitHub тоже является отличным источником наборов данных:

<https://github.com/awesomedata/awesome-public-datasets>

Файл `account.csv`

Для нашего первого примера мы приготовили простой набор данных и записали его в файл `accounts.csv`, расположенный в папке `ch08`. Этот файл содержит те же данные, которые вы видели на илл. 8.11, но в формате значений, разделенных запятыми:

```
account,name,balance
100,Jones,24.98
200,Doe,345.67
300,White,0.0
400,Stone,-42.16
500,Rich,224.62
```

Первая строка CSV-файла, как правило, содержит имена столбцов. Каждая последующая строка является записью, содержащей значения для этих столбцов. В этом наборе данных есть три столбца, соответствующие переменным `account`, `name` и `balance`.

8.19.1. Чтение CSV-файла с помощью библиотеки `rapidcsv`

Заголовочная библиотека `rapidcsv`¹ опубликована на портале GitHub:

<https://github.com/d99kris/rapidcsv>

В ней есть класс `rapidcsv::Document`, который вы можете использовать для чтения и обработки CSV-файлов². Многие другие библиотеки тоже имеют встроенную поддержку формата CSV. Для вашего удобства мы уже поместили библиотеку `rapidcsv` в папку `libraries/rapidcsv`, расположенную в `examples`. Как и в пре-

¹ © 2017, Кристофер Бергрэн (Kristofer Berggren). Все права защищены.

² Другой популярный формат данных — JavaScript Object Notation (JSON). Существуют библиотеки C++ для чтения и генерации JSON-файлов, например RapidJSON (<https://github.com/Tencent/rapidjson>) и cereal (<https://usciilab.github.io/cereal/index.html>; см. раздел 9.22).

дыдущих примерах со сторонними библиотеками, в программу надо включить заголовок, в данном случае `<rapidcsv.h>` (илл. 8.14, строка 5).

```

1 // fig08_14.cpp
2 // Ввод данных из CSV-файла
3 #include <fmt/format.h>
4 #include <iostream>
5 #include <rapidcsv.h>
6 #include <vector>
7
8 int main() {
9     rapidcsv::Document document{"accounts.csv"}; // Открываем файл по имени
10    std::vector<int> accounts{document.GetColumn<int>("account")};
11    std::vector<std::string> names{
12        document.GetColumn<std::string>("name")};
13    std::vector<double> balances{document.GetColumn<double>("balance")};
14
15    std::cout << fmt::format(
16        "{:<10}{:<13}{:>7}\n", "Account", "Name", "Balance");
17
18    for (size_t i{0}; i < accounts.size(); ++i) {
19        std::cout << fmt::format("{:<10}{:<13}{:>7.2f}\n",
20            accounts.at(i), names.at(i), balances.at(i));
21    }
22 }
```

Account	Name	Balance
100	Jones	24.98
200	Doe	345.67
300	White	0.00
400	Stone	-42.16
500	Rich	224.62

Илл. 8.14 | Ввод данных из CSV-файла

Для открытия файла `"accounts.csv"`¹ создается и инициализируется объект `rapidcsv::Document` с именем `document` (строка 9). Функции класса `Document` позволяют работать с данными CSV-файла по строкам, по столбцам и по отдельным значениям на пересечениях строк и столбцов. В этом примере поочередно загружаются столбцы, для чего трижды вызывается имеющаяся в шаблоне класса `Document` функция `GetColumn` (строки 10–13). Она возвращает данные указанного столбца в виде вектора `std::vector`, содержащего элементы указанного в угловых скобках типа. Вызов этой функции в строке 10:

```
document.GetColumn<int>("account")
```

возвращает вектор элементов `int`, содержащих значения `account` для каждой записи. Вызовы в строках 11–12 и 13 возвращают векторы элементов `string`

¹ Файл `accounts.csv` должен находиться в папке с исполняемый файлом программы.

и `double` соответственно, содержащих значения `name` и `balance`. В строках 15–21 полученные данные форматируются и выводятся на экран для подтверждения успешного чтения файла.

Внимание: запятые в полях CSV-файла

Будьте осторожны при работе со строками, содержащими запятые, например "Jones, Sue". Если бы мы случайно сохранили такое имя в двух строках "Jones" и "Sue", то CSV-запись в нашем примере содержала бы четыре поля вместо трех. Программы, работающие с CSV-файлами, обычно рассчитаны на одинаковое количество полей в каждой записи. В противном случае возникают проблемы.

Внимание: пропущенные запятые и лишние запятые в CSV-файлах

Будьте осторожны при создании и обработке CSV-файлов. Например, ваш файл может состоять из записей, в каждой из которых есть четыре значения, разделенных запятыми:

```
100,85,77,9
```

Предположим, вы случайно пропустите одну из запятых:

```
100,8577,9
```

Тогда запись будет содержать только три поля, одно из которых будет содержать неправильное значение 8577.

Предположим, вы случайно поставите две запятые вместо одной:

```
100,85,,77,9
```

Тогда вы получите пять полей вместо четырех, а одно из них будет пустым. Каждая из этих ошибок может привести к неправильной обработке данных программой.

8.19.2. Работа с набором данных «Титаник»

Начинающие специалисты по работе с большими данными часто используют в своей работе набор данных «Титаник»¹. В нем перечислены все пассажиры и указано, кто из них выжил, когда пароход столкнулся с айсбергом и затонул в своем первом рейсе 15 апреля 1912 года. Давайте введем эти данные в программу и проанализируем.

Набор данных «Титаник» в формате CSV мы поместили в файл `titanic.csv`, который находится в архиве `CPlusPlus20ForProgrammers-master.zip`, в каталоге

¹ Набор данных «Titanic» с сайта OpenML.org (<https://www.openml.org/d/40945>). Авторы: Фрэнк Э. Харрелл-младший (Frank E. Harrell, Jr.) и Томас Кейсон (Thomas Cason). Источник: Vanderbilt Biostatistics (<https://biostat.app.vumc.org/wiki/pub/Main/DataSets/titanic.html>). В условиях лицензии OpenML (<https://www.openml.org/cite>) сказано: «Вы можете свободно использовать OpenML и все эмпирические данные и метаданные по лицензии CC-BY» (<https://creativecommons.org/licenses/by/4.0>).

с примерами кода из главы 8. Вы можете загрузить этот архив на портале GitHub или на сайте Deitel:

<https://github.com/pdeitel/CPlusPlus20ForProgrammers>

<https://www.deitel.com/books/c-plus-plus-20-for-programmers>

Ознакомление с данными

Чтобы анализировать данные, надо хорошо понимать, что они собой представляют. Для начала можно просто просмотреть записи в CSV-файле. Если вы откроете файл `titanic.csv` в текстовом редакторе или в Excel, вы увидите, что он состоит из 1309 записей, разделенных запятыми на 14 столбцов (специалисты по анализу данных говорят, что каждый из 1309 объектов характеризуется 14 признаками). Мы будем использовать только четыре столбца:

- ♦ `survived`: 1 — пассажир выжил, 0 — не выжил;
- ♦ `sex`: "female" — женский пол, "male" — мужской;
- ♦ `age`: возраст; почти все значения — это целые числа, но для некоторых детей в возрасте до года указаны значения с плавающей точкой, поэтому для обработки этого столбца мы выбрали тип `double`;
- ♦ `pclass`: 1 — билет первого класса, 2 — второго, 3 — третьего.

Здесь вы можете узнать о значениях всех остальных столбцов:

<https://biostat.app.vumc.org/wiki/pub/Main/DataSets/titanic3info.txt>

Неизвестные данные

Плохие (то есть недостоверные) и неизвестные значения могут существенно влиять на результаты анализа данных. В наборе данных «Титаник» не указан возраст 263 пассажиров. Отсутствие данных о возрасте обозначено символом ? в столбце `age` CSV-файла. Для получения статистических данных о возрасте пассажиров мы будем отфильтровывать и игнорировать неизвестные значения. Некоторые аналитики советуют никогда не заменять в неполном наборе данных неизвестные значения «самыми подходящими», а вместо этого передавать такой набор в анализирующую программу, явно обозначив в нем все отсутствующие элементы. Другие аналитики допускают замену, но только при соблюдении строгих правил¹.

¹ Эта сноска — часть комментария, отправленного нам 20 июля 2018 года одним из академических рецензентов нашего учебника по Python доктором Элисон Санчес (Alison Sanchez) из школы бизнеса Университета Сан-Диего. Она прокомментировала: «Будьте осторожны, упоминая "замену подходящими значениями" отсутствующих или плохих значений. Строгое предупреждение: не допускается "замена" тех значений, которые повышают статистическую значимость или дают "более обоснованные" или "лучшие" результаты. "Замена" данных не должна превращаться в "подделку" данных. Первое правило, которое должны усвоить студенты, — не удалять и не изменять те значения, которые противоречат их гипотезам. "Подстановка подходящих значений" не означает, что студентам разрешается произвольно изменять значения, чтобы получать желаемые результаты».

Ввод набора данных в программу

В нашей программе, разделенной на части для удобства обсуждения (илл. 8.15), используются некоторые функции библиотеки `ranges` (C++20), рассмотренной в разделе 6.14. Для открытия файла `titanic.csv`¹ создается и инициализируется объект `rapidcsv::Document` с именем `titanic` (строки 15–17). Второй и третий аргументы в строке 16 — это два аргумента по умолчанию, используемые для инициализации объекта `Document` в том случае, когда он создается только по имени CSV-файла. Напомним, что явное указание аргумента функции требует явного указания всех предыдущих аргументов (см. раздел 5.13). Мы указали второй и третий аргументы только для того, чтобы можно было указать четвертый аргумент.

```

1 // fig08_15.cpp
2 // Чтение набора данных "Титаник" из CSV-файла с последующим анализом
3 #include <fmt/format.h>
4 #include <algorithm>
5 #include <cmath>
6 #include <iostream>
7 #include <numeric>
8 #include <ranges>
9 #include <rapidcsv.h>
10 #include <string>
11 #include <vector>
12
13 int main() {
14     // Вводим данные, заменяя отсутствующие данные о возрасте значением NaN
15     rapidcsv::Document titanic{"titanic.csv",
16         rapidcsv::LabelParams{}, rapidcsv::SeparatorParams{},
17         rapidcsv::ConverterParams{true}};
18

```

Илл. 8.15 | Чтение набора данных «Титаник» из CSV-файла с последующим анализом

Аргумент по умолчанию `rapidcsv::LabelParams{}` означает, что первая строка нашего CSV-файла содержит имена столбцов. Аргумент по умолчанию `rapidcsv::SeparatorParams{}` означает, что поля каждой записи разделяются запятыми. Четвертый аргумент:

```
rapidcsv::ConverterParams{true}
```

означает, что `rapidcsv` автоматически заменит в наборе данных все отсутствующие и недопустимые значения. В столбцах с целыми числами каждое такое значение будет преобразовано в 0, а в столбцах с числами с плавающей точкой — в NaN (Not a Number). Это позволит нам загрузить весь столбец `age` в вектор элементов `double`, в том числе обозначенные символом ? неизвестные значения.

¹ Файл `titanic.csv` должен находиться в папке с исполняемым файлом программы.

Подготовка данных для анализа

В строках 20–23 функция `GetColumn` класса `rapidcsv::Document` копирует в вектор данные из каждого указанного по имени столбца CSV-файла.

```

19 // GetColumn возвращает данные столбцов в векторах элементов тех же типов
20 auto survived{titanic.GetColumn<int>("survived")};
21 auto sex{titanic.GetColumn<std::string>("sex")};
22 auto age{titanic.GetColumn<double>("age")};
23 auto pclass{titanic.GetColumn<int>("pclass")};
24

```

Выборочный просмотр строк из набора данных «Титаник»

Каждая из 1309 строк содержит данные об одном пассажире. Согласно «Википедии», на борту находилось примерно 1317 пассажиров, 815 из них погибли¹. Все записи большого набора данных невозможно отобразить на экране одновременно. Для ознакомления с набором данных программа обычно показывает несколько записей с начала и с конца — это позволяет получить о нем общее представление. Инструкции в строках 26–31 отображают первые пять элементов каждого столбца:

```

25 // Выводим на экран первые 5 строк
26 std::cout << fmt::format("First five rows:\n{:<10}{:<8}{:<6}}\n",
27   "survived", "sex", "age", "class");
28 for (size_t i{0}; i < 5; ++i) {
29   std::cout << fmt::format("{:<10}{:<8}{:<6.1f}}\n",
30     survived.at(i), sex.at(i), age.at(i), pclass.at(i));
31 }
32

```

First five rows:

survived	sex	age	class
1	female	29.0	1
1	male	0.9	1
0	female	2.0	1
0	male	30.0	1
0	female	25.0	1

Инструкции в строках 34–40 отображают последние пять элементов каждого столбца. Для этого функция `getRowCount` класса `rapidcsv::Document` возвращает количество строк в наборе данных (строка 36), затем управляющая переменная инициализируется значением, которое на 5 меньше значения `count` (строка 37). Обратите внимание, что неизвестный возраст пассажира обозначен в выходных данных словом `nan`.

¹ «Passengers of the Titanic». Был доступен: 15 сентября 2023 г. https://en.wikipedia.org/wiki/Passengers_of_the_RMS_Titanic.

```

33 // Выводим на экран последние 5 строк
34 std::cout << fmt::format("\nLast five rows:\n{:<10}{:<8}{:<6}}\n",
35     "survived", "sex", "age", "class");
36 const auto count{titanic.GetRowCount()};
37 for (size_t i{count - 5}; i < count; ++i) {
38     std::cout << fmt::format("{:<10}{:<8}{:<6.1f}}\n",
39         survived.at(i), sex.at(i), age.at(i), pclass.at(i));
40 }
41

```

Last five rows:			
survived	sex	age	class
0	female	14.5	3
0	female	nan	3
0	male	26.5	3
0	male	27.0	3
0	male	29.0	3

Базовая описательная статистика

Для ознакомления с набором данных аналитики часто используют статистические методы, чтобы описать и обобщить эти данные. Давайте рассчитаем пять базовых статистических характеристик, основываясь на столбце `age`: количество пассажиров известного возраста, а также средний, минимальный, максимальный и медианный возраст. До начала этих расчетов мы должны очистить данные, удалив все значения `nan`. Любое выражение, в котором есть `nan`, дает результат `nan`. В строках 43–44 мы используем рассмотренный в разделе 6.14 метод фильтрации с помощью библиотеки `ranges` (C++20), чтобы оставить в векторе `age` только значения, отличающиеся от `nan`. Функция `isnan` (заголовок `<cmath>`) возвращает `true`, если значение равно `nan`. В строках 45–46 мы создаем вектор элементов `double` с именем `cleanAge`. Он инициализирует свои элементы путем перебора отфильтрованных результатов в конвейере `removeNaN`.

```

42 // Удаляем отсутствующие значения возраста с помощью библиотеки ranges
43 auto removeNaN{
44     age | std::views::filter([](const auto& x) {return !isnan(x);});
45     std::vector<double> cleanAge{
46         std::begin(removeNaN), std::end(removeNaN)};
47

```

Базовая описательная статистика по данным из очищенного столбца `age`

После очистки данных можно приступить к расчетам. В строке 49 мы сортируем по возрастанию вектор `cleanAge`, что поможет найти минимальный, максимальный и медианный возраст. А в строке 50 мы получаем количество пассажиров известного возраста: оно равно размеру вектора `cleanAge`.

```

48 // Описательная статистика для отфильтрованного столбца age
49 std::sort(std::begin(cleanAge), std::end(cleanAge));
50 size_t size{cleanAge.size()};
51 double median{};
52
53 if (size % 2 == 0) { // Медианное значение при четном количестве элементов
54     median = (cleanAge.at(size / 2 - 1) + cleanAge.at(size / 2)) / 2;
55 }
56 else { // Медианное значение при нечетном количестве элементов
57     median = cleanAge.at(size / 2);
58 }
59
60 std::cout << "\nDescriptive statistics for the age column:\n"
61     << fmt::format("Passengers with age data: {}\n", size)
62     << fmt::format("Average age: {:.2f}\n", std::accumulate(
63         std::begin(cleanAge), std::end(cleanAge), 0.0) / size)
64     << fmt::format("Minimum age: {:.2f}\n", cleanAge.front())
65     << fmt::format("Maximum age: {:.2f}\n", cleanAge.back())
66     << fmt::format("Median age: {:.2f}\n", median);
67

```

Descriptive statistics for the age column:

Passengers with age data: 1046

Average age: 29.88

Minimum age: 0.17

Maximum age: 80.00

Median age: 28.00

В строках 51–58 определяется медианный возраст. Если в векторе `cleanAge` четное количество элементов, медианой является среднее арифметическое двух элементов, расположенных посередине вектора (строка 54); если нечетное — медианой является значение элемента, расположенного посередине (строка 57). В строках 60–66 вся статистика выводится на экран. Средний возраст рассчитывается с помощью алгоритма `accumulate` (строки 62–63): сумма всех значений вектора `cleanAge` делится на его размер. Поскольку вектор уже отсортирован, минимальный и максимальный возраст определяются функциями `front` и `back` класса `vector` (строки 64–65), которые возвращают первый и последний элементы соответственно. Среднее и медианное значения относятся к мерам центральной тенденции. **Мерой центральной тенденции (measure of central tendency)** для набора числовых данных называется такое значение, которое является «центральным» в этом наборе, то есть в каком-то смысле типичным среди остальных.

Итак, мы выяснили, что в наборе данных «Титаник» указан возраст 1046 пассажиров. Их средний возраст — 29.88 лет. Самому младшему из них было чуть больше двух месяцев ($0.17 \cdot 12 = 2.04$), а самому старшему — 80 лет. Медианный возраст пассажиров — 28 лет.

Подсчет количества пассажиров по классам

Теперь выясним, сколько пассажиров было в каждом классе. В строках 69–74 определяется лямбда, которая считает количество пассажиров в указанном классе. Алгоритм `std::ranges::count_if` (C++20) считает все те элементы в своем первом аргументе, для которых лямбда во втором аргументе возвращает `true`. Одно из преимуществ алгоритмов библиотеки `std::ranges` заключается в том, что вам не нужно указывать начало и конец контейнера — эти алгоритмы упрощают код, делая часть работы за вас. Первым аргументом алгоритма `count_if` в этом примере будет вектор элементов `int` с именем `pclass`. Лямбда в строке 72 содержит в списке захвата параметр `classNumber`:

```
[classNumber](int x) {return classNumber == x;}
```

Это называется **захватом (capturing)** параметра: лямбда захватывает `classNumber` из доступной области видимости (из лямбды `countClass`) и использует его в своем теле. По умолчанию захват выполняется по значению, поэтому лямбда в строке 72 получает копию `classNumber`. В главе 14 мы подробнее поговорим о лямбдах с захваченными параметрами и об алгоритмах библиотеки `std::ranges`. В строках 75–77 определяются константы для трех классов пассажиров. В строках 78–80 лямбда `countClass` вызывается для каждого из классов, а в строках 82–84 результаты выводятся на экран.

```
68 // Количество пассажиров первого, второго и третьего класса
69 auto countClass{
70     [](const auto& column, const int classNumber) {
71         return std::ranges::count_if(column,
72             [classNumber](int x) {return classNumber == x;});
73     }
74 };
75 constexpr int firstClass{1};
76 constexpr int secondClass{2};
77 constexpr int thirdClass{3};
78 const auto firstCount{countClass(pclass, firstClass)};
79 const auto secondCount{countClass(pclass, secondClass)};
80 const auto thirdCount{countClass(pclass, thirdClass)};
81
82 std::cout << "\nPassenger counts by class:\n"
83     << fmt::format("1st: {} \n2nd: {} \n3rd: {} \n\n",
84         firstCount, secondCount, thirdCount);
85
```

```
Passenger counts by class:
1st: 323
2nd: 277
3rd: 709
```

Подсчет количества выживших пассажиров

В строках 87–89 определяется лямбда, которая считает количество выживших пассажиров с помощью алгоритма `std::ranges::count_if` (C++20). Напомним, что столбец `survived` содержит 1 или 0 для обозначения выживших или погибших соответственно. Такие значения C++ может обрабатывать как `true` (1) или `false` (0), поэтому лямбда в строке 88 просто возвращает значение столбца:

```
[](auto x) {return x;}
```

Если это значение равно 1, алгоритм `count_if` добавляет элемент к сумме. Чтобы определить количество погибших, в строке 92 из размера вектора `survived` вычитается `survivorCount`. В строке 94 вычисляется процент выживших.

```
86 // Подсчет количества выживших пассажиров
87 const auto survivorCount{
88     std::ranges::count_if(survived, [](auto x) {return x;})
89 };
90
91 std::cout << fmt::format("Survived count: {}\nDied count: {}\n",
92     survivorCount, survived.size() - survivorCount);
93 std::cout << fmt::format("Percent who survived: {:.2f}%\n\n",
94     100.0 * survivorCount / survived.size());
95
```

```
Survived count: 500
Died count: 809
Percent who survived: 38.20%
```

Подсчет количества выживших пассажиров по категориям

В строках 97–117 перебираются все элементы столбца `survived`, их значения 1 и 0 используются в качестве условия (строка 104). Для каждого выжившего увеличиваются счетчики, соответствующие его полу (`survivingWomen` и `survivingMen` в строке 105) и классу (`surviving1st`, `surviving2st`, `surviving3st` в строках 107–115). Результаты будут использованы для вычисления процентов.

```
96 // Количество выживших пассажиров по категориям
97 int survivingMen{0};
98 int survivingWomen{0};
99 int surviving1st{0};
100 int surviving2nd{0};
101 int surviving3rd{0};
102
103 for (size_t i{0}; i < survived.size(); ++i) {
104     if (survived.at(i)) {
105         sex.at(i) == "female" ? ++survivingWomen : ++survivingMen;
106
107         if (firstClass == pclass.at(i)) {
108             ++surviving1st;
109         }
110     }
111 }
```

```
110         else if (secondClass == pclass.at(1)) {
111             ++surviving2nd;
112         }
113         else { // Третий класс
114             ++surviving3rd;
115         }
116     }
117 }
118
```

Вычисление процентов

В строках 120–129 вычисляются и отображаются проценты:

- ✦ выживших женщин;
- ✦ выживших мужчин;
- ✦ выживших пассажиров первого класса;
- ✦ выживших пассажиров второго класса;
- ✦ выживших пассажиров третьего класса.

Итак, среди выживших было примерно две трети женщин, а наибольший шанс выжить был у пассажиров первого класса.

```
119 // Процент выживших пассажиров по категориям
120 std::cout << fmt::format("Female survivor percentage: {:.2f}%\n",
121     100.0 * survivingWomen / survivorCount)
122     << fmt::format("Male survivor percentage: {:.2f}%\n\n",
123     100.0 * survivingMen / survivorCount)
124     << fmt::format("1st class survivor percentage: {:.2f}%\n",
125     100.0 * surviving1st / survivorCount)
126     << fmt::format("2nd class survivor percentage: {:.2f}%\n",
127     100.0 * surviving2nd / survivorCount)
128     << fmt::format("3rd class survivor percentage: {:.2f}%\n",
129     100.0 * surviving3rd / survivorCount);
130 }
```

Female survivor percentage: 67.80%

Male survivor percentage: 32.20%

1st class survivor percentage: 40.00%

2nd class survivor percentage: 23.80%

3rd class survivor percentage: 36.20%

8.20. Готовые объекты: основы регулярных выражений

Иногда вам бывает нужно найти в тексте все фрагменты, соответствующие определенным шаблонам: телефонные номера, адреса электронной почты, почтовые

индексы, адреса веб-страниц и т. д. **Регулярное выражение** (**regular expression**, или сокращенно **regex**) — это шаблон, который может сравниваться на совпадение с проверяемыми строками и их фрагментами (подстроками). Регулярные выражения помогают быстро извлекать нужные данные из неструктурированного текста, например из сообщений в соцсетях. Кроме того, регулярные выражения значительно упрощают проверку формата, необходимую перед обработкой данных.

Проверка формата данных

Как правило, любую обработку введенных пользователем текстовых данных надо начинать с проверки их формата регулярными выражениями. Например, вы можете проверить, что:

- ♦ почтовый индекс США состоит из пяти цифр (например, 02215) или из девяти цифр, разделенных дефисом после пятой цифры (например, 02215-4775);
- ♦ имя пользователя содержит только буквы, пробелы, апострофы и дефисы;
- ♦ адрес электронной почты содержит только допустимые символы в допустимой последовательности;
- ♦ номер социального страхования США содержит три цифры, дефис, две цифры, дефис и еще четыре цифры, а также соответствует правилам о конкретных числах, имеющихся в каждой группе цифр.

Иногда вам придется составлять ваши собственные регулярные выражения, необходимые для всевозможных задач проверки данных. Вам помогут эти веб-сайты:

<https://regex101.com>

<https://regexr.com>

<http://www.regexlib.com>

<https://www.regular-expressions.info>

На этих и других сайтах есть бесплатные репозитории готовых регулярных выражений, которые можно копировать и использовать. Кроме того, многие сайты предлагают тестирование регулярных выражений на соответствие вашим требованиям.

Другие применения регулярных выражений

Регулярные выражения удобны для следующих задач:

- ♦ **скрейпинг (scraping)**, то есть извлечение данных из текста, например копирование всех URL-адресов с веб-страницы в файл;
- ♦ очистка данных: удаление ненужных данных и дубликатов, обработка неполных данных, исправление опечаток, приведение данных к единому формату, удаление ненужного форматирования, изменение регистра букв и многие другие операции со строками;
- ♦ преобразование данных в другие форматы, например замена всех нестандартных разделителей в наборе данных (пробелов или символов табуляции) на

запятые для последующей обработки в формате CSV. (Мы рассматривали CSV-файлы в разделе 8.19.)

Диалекты регулярных выражений в C++

C++ поддерживает несколько вариантов грамматики регулярных выражений, часто называемых **диалектами** (*flavors*). По умолчанию в C++ используется слегка измененная версия регулярных выражений из языка ECMAScript¹. Полный список поддерживаемых грамматических вариантов (диалектов) приведен здесь:

https://en.cppreference.com/w/cpp/regex/syntax_option_type

8.20.1. Сравнение целых строк с шаблонами

Чтобы использовать регулярные выражения, надо включить в программу заголовок `<regex>`, предоставляющий классы и функции для их поддержки². Наша следующая программа, разделенная на части для удобства обсуждения (илл. 8.16), демонстрирует операции, в которых целая строка текста сравнивается на совпадение с тем или иным шаблоном (регулярным выражением).

Сравнение литералов

Функция `regex_match` из заголовка `<regex>` возвращает `true`, если вся строка в первом аргументе совпадает с шаблоном во втором аргументе. По умолчанию сравнение с шаблоном чувствительно к регистру; далее мы покажем, как выполнять аналогичные операции без учета регистра. Давайте для начала сравним между собой короткие последовательности символов. Мы создаем объект `regex` с именем `r1` (строка 9), содержащий простой шаблон `"02215"` из пяти цифровых литералов. При сравнении этого шаблона с другой строкой совпадение будет найдено только в том случае, если сравниваемая строка ничем не отличается от шаблона, то есть состоит из этих же пяти литералов в этой же последовательности. Мы сравниваем шаблон со строками `"02215"` и `"51220"`, дважды вызывая функцию `regex_match` (строка 12). Обе сравниваемые строки составлены из одних и тех же цифровых литералов, но только в строке `"02215"` последовательность цифр такая же, как в шаблоне.

```
1 // fig08_16.cpp
2 // Сравнение целых строк с шаблонами
3 #include <fmt/format.h>
4 #include <iostream>
5 #include <regex>
6
7 int main() {
8     // Шаблон: фиксированная последовательность литералов
```

Окончание ➞

¹ «Modified ECMAScript Regular Expression Grammar». Был доступен: 15 сентября 2023 г. <https://en.cppreference.com/w/cpp/regex/ecmascript>.

² Некоторые программисты C++ предпочитают сторонние библиотеки регулярных выражений, например RE2 или PCRE. Список таких библиотек есть на портале GitHub: <https://github.com/ffaraz/awesome-cpp#regular-expression>.

```

9  std::regex r1{"02215"};
10 std::cout << "Matching against: 02215\n"
11     << fmt::format("02215: {}; 51220: {}\n\n",
12         std::regex_match("02215", r1), std::regex_match("51220", r1));
13

```

```

Matching against: 02215
02215: true; 51220: false

```

Илл. 8.16 | Сравнение целых строк с шаблонами

Метасимволы и символьные классы

Как правило, регулярные выражения содержат специальные символы, называемые **метасимволами (metacharacters)**:

[] { } () \ * + ^ \$? , |

С метасимвола \ начинаются все **символьные классы (character classes)**. В таблице перечислены важнейшие символьные классы и указано, каким символам они соответствуют.

Символьный класс	Соответствие
\d	Любая цифра (0–9)
\D	Любой символ, кроме цифры
\s	Любой символ пропуска (пробел, табуляция, перевод строки и др.)
\S	Любой символ, кроме символа пропуска
\w	Любой словообразующий символ (word character), которым может быть любая заглавная или строчная буква английского алфавита, любая цифра или символ подчеркивания
\W	Любой символ, кроме словообразующего символа

Чтобы включить в шаблон любой метасимвол в виде обычного литерала, добавьте перед ним обратный слеш. Например, шаблон \\$ соответствует символу доллара (\$), а шаблон \\ соответствует обратному слешу (\).

Шаблон с квантификатором: пять цифр подряд

В следующей части программы мы моделируем проверку почтового индекса, который должен состоять из пяти цифр. В регулярном выражении \d{5}, которое мы записали в виде необработанного строкового литерала (строка 15), \d — это символьный класс, соответствующий любой цифре (0–9). Символьный класс — это escape-последовательность, соответствующая какому-либо одному символу. Чтобы найти пять любых цифр подряд, мы добавили **квантификатор (quantifier)** {5}, который повторяет \d пять раз, что аналогично шаблону \d\d\d\d\d. Второй вызов функции `regex_match` (строка 18) возвращает `false`, поскольку почтовый индекс "9876" некорректен: в нем только четыре цифры.

```

14 // Шаблон: любая строка из 5 цифр
15 std::regex r2{R"(\d{5})"};
16 std::cout << R"(Matching against: \d{5})" << "\n"
17     << fmt::format("02215: {}; 9876: {}\n\n",
18         std::regex_match("02215", r2), std::regex_match("9876", r2));
19

```

```

Matching against: \d{5}
02215: true; 9876: false

```

Пользовательские символьные классы

Символами в квадратных скобках [] определяется **пользовательский символьный класс (custom character class)**, который тоже соответствует какому-то одному символу. Например, [aeiou] — это любая из пяти гласных букв английского алфавита в нижнем регистре; [A-Z] — любая заглавная буква от A до Z; [a-z] — любая строчная буква от a до z; [a-zA-Z] — любая буква английского алфавита. В строке 21 мы определяем пользовательский символьный класс для проверки имени пользователя, которое не должно содержать пробелы и знаки препинания.

```

20 // Шаблон: любое слово, которое начинается с заглавной буквы A-Z
21 std::regex r3{"[A-Z][a-z]*"};
22 std::cout << "Matching against: [A-Z][a-z]*\n"
23     << fmt::format("Wally: {}; Eva: {}\n\n",
24         std::regex_match("Wally", r3), std::regex_match("Eva", r3));
25

```

```

Matching against: [A-Z][a-z]*
Wally: true; Eva: false

```

В имени человека может быть много букв. В регулярном выражении r3 (строка 21) класс [A-Z] соответствует одной заглавной букве, а класс [a-z]* — любому (в том числе нулевому) количеству строчных букв. Квантификатор * после какого-либо элемента регулярного выражения обозначает, что этот элемент (в данном случае класс [a-z], то есть любая буква от a до z) повторяется ноль или более раз подряд. Например, шаблон [A-Z][a-z]* соответствует строке "Amanda", "Bo" и даже "E".

Пользовательский символьный класс, который начинается с ^, соответствует любому символу, кроме указанных после ^. Например, шаблон [^a-z] (строка 27) обозначает любой символ, кроме строчной буквы в диапазоне от a до z.

```

26 // Шаблон: любой символ, кроме строчной буквы a-z
27 std::regex r4{"[^a-z]"};
28 std::cout << "Matching against: [^a-z]\n"
29     << fmt::format("A: {}; a: {}\n\n",
30         std::regex_match("A", r4), std::regex_match("a", r4));
31

```

```

Matching against: [^a-z]
A: true; a: false

```

Метасимволы в пользовательском классе обрабатываются как литералы, то есть как обычные символы. Например, шаблон `[*+]` (строка 33) соответствует любому из трех символов: `*`, `+` или `$`.

```
32 // Метасимволы в пользовательском классе символов считаются литералами
33 std::regex r5{"[*+]"};
34 std::cout << "Matching against: [*+]\n"
35     << fmt::format("*: {}; !: {}\n\n",
36         std::regex_match("?", r5), std::regex_match("!", r5));
37
```

```
Matching against: [*+]
*: true; !: false
```

Сравнение квантификаторов `+` и `*`

Если в имени пользователя должна быть хотя бы одна строчная буква, замените квантификатор `*` (строка 21) на `+` (строка 39). Квантификатор `+` после какого-либо элемента регулярного выражения обозначает, что этот элемент повторяется один или более раз подряд.

```
38 // Любое слово с первой заглавной буквой A-Z и второй строчной буквой a-z
39 std::regex r6{"[A-Z][a-z]+"};
40 std::cout << "Matching against: [A-Z][a-z]+\n"
41     << fmt::format("Wally: {}; E: {}\n\n",
42         std::regex_match("Wally", r6), std::regex_match("E", r6));
43
```

```
Matching against: [A-Z][a-z]+
Wally: true; E: false
```

Квантификаторы `*` и `+` являются **жадными (greedy)**. Жадным называется квантификатор, обозначающий наибольшее возможное количество последовательных символов. Например, регулярное выражение `[A-Z][a-z]+` соответствует словам "Al", "Eva", "Samantha", "Benjamin" и любым другим словам, начинающимся с заглавной буквы, за которой следует хотя бы одна строчная буква. Вы можете добавить символ `?` после квантификатора `*` и `+`, чтобы сделать его **ленивым (lazy)**: `*?` или `+`. Ленивым называется квантификатор, обозначающий наименьшее возможное количество последовательных символов.

Другие квантификаторы

Квантификатор `?` сам по себе обозначает, что элемент слева от него повторяется ноль или один раз. В регулярном выражении `labelled` (строка 45) таким элементом является литерал `"l"`. Поэтому в двух первых вызовах функции `regex_match` (строки 48–49) шаблон соответствует слову `labelled` (так пишут в Великобритании) и `labeled` (так пишут в США). Но в третьем вызове `regex_match` (строка 50) шаблон не соответствует слову `labellled`, которое написано с ошибкой.

```

44 // Элемент входит в шаблон не более одного раза
45 std::regex r7{"labelled?ed"};
46 std::cout << "Matching against: labelled?ed\n"
47   << fmt::format("labelled: {}; labeled: {}; labelled: {}\n\n",
48     std::regex_match("labelled", r7),
49     std::regex_match("labeled", r7),
50     std::regex_match("labelled", r7));
51

```

```

Matching against: labelled?ed
labelled: true; labeled: true; labelled: false

```

Вы можете использовать квантификатор `{n,}`, чтобы обозначить *n* или более последовательных повторений элемента слева от него. Регулярное выражение в строке 53 соответствует строкам, состоящим как минимум из трех цифр подряд.

```

52 // Элемент входит в шаблон не менее трех раз подряд
53 std::regex r8{R"(\d{3,})"};
54 std::cout << R"(Matching against: \d{3,})" << "\n"
55   << fmt::format("123: {}; 1234567890: {}; 12: {}\n\n",
56     std::regex_match("123", r8),
57     std::regex_match("1234567890", r8),
58     std::regex_match("12", r8));
59

```

```

Matching against: \d{3,}
123: true; 1234567890: true; 12: false

```

Вы можете использовать квантификатор `{n,m}`, чтобы обозначить от *n* до *m* (включительно) последовательных повторений элемента слева от него. Регулярное выражение в строке 61 соответствует строке, в которой от трех до шести цифр.

```

60 // Элемент входит в шаблон не менее трех и не более шести раз подряд
61 std::regex r9{R"(\d{3,6})"};
62 std::cout << R"(Matching against: \d{3,6})" << "\n"
63   << fmt::format("123: {}; 123456: {}; 1234567: {}; 12: {}\n",
64     std::regex_match("123", r9), std::regex_match("123456", r9),
65     std::regex_match("1234567", r9), std::regex_match("12", r9));
66 }

```

```

Matching against: \d{3,6}
123: true; 123456: true; 1234567: false; 12: false

```

8.20.2. Замена подстрок

Заголовок `<regex>` предоставляет функцию `regex_replace` для замены фрагментов в строке. В следующем примере мы преобразуем строку элементов, разделенных табуляцией, в строку элементов, разделенных запятыми (илл. 8.17). Функция `regex_replace` (строка 13) получает три аргумента:

- ♦ строку для поиска ("1\t2\t3\t4");
- ♦ регулярное выражение (символ табуляции "\t");
- ♦ текст для замены (",").

Функция возвращает измененную строку. Инструкция

```
std::regex{"\t"}
```

создает временный объект `regex`, инициализирует его и сразу передает в функцию `regex_replace`. Это удобно, если регулярное выражение используется только один раз.

```
1 // fig08_17.cpp
2 // Замена фрагментов в строке
3 #include <fmt/format.h>
4 #include <iostream>
5 #include <regex>
6 #include <string>
7
8 int main() {
9     // Заменяем символы табуляции запятыми
10    std::string s1{"1\t2\t3\t4"};
11    std::cout << fmt::format("Original string: {}\n", R"(1\t2\t3\t4)")
12              << fmt::format("After replacing tabs with commas: {}\n",
13                              std::regex_replace(s1, std::regex{"\t"}, ","));
14 }
```

```
Original string: 1\t2\t3\t4
After replacing tabs with commas: 1,2,3,4
```

Илл. 8.17 | Замена фрагментов в строке

8.20.3. Поиск в тексте по шаблонам

Вы можете искать в тексте интересующие вас фрагменты с помощью функции `regex_search` (илл. 8.18), которая возвращает `true`, если какой-либо фрагмент строки соответствует шаблону. Эта функция также может предоставить вам доступ к найденному фрагменту через объект шаблона класса `match_results`, который вы передаете в качестве аргумента. Существуют псевдонимы `match_results` для разных типов строк:

- ♦ для поиска в `std::strings` используйте `smatch` (произносится как «эс мач»);
- ♦ для поиска в строках в стиле C и в строковых литералах используйте `smatch` (произносится как «си мач»).

На данный момент заголовок `<regex>` не содержит средств для работы с представлениями `string_view`.

Поиск фрагмента без уточнения его места в строке

Каждый из вызовов функции `regex_search` в строках 14–16 выполняет поиск в строке `s1` ("Programming is fun") первого с начала строки фрагмента, соответствующего регулярному выражению. Каждое из трех регулярных выражений — это простая последовательность символов: "Programming", "fun" и "fn". Версия функции `regex_search` с двумя аргументами возвращает `true` или `false` в зависимости от того, есть ли в строке искомый фрагмент.

```
1 // fig08_18.cpp
2 // Поиск в тексте по шаблонам
3 #include <fmt/format.h>
4 #include <iostream>
5 #include <regex>
6 #include <string>
7
8 int main() {
9     // Простой поиск: есть ли в строке указанные фрагменты?
10    std::string s1{"Programming is fun"};
11    std::cout << fmt::format("s1: {}\n\n", s1);
12    std::cout << "Search anywhere in s1:\n"
13    << fmt::format("Programming: {}; fun: {}; fn: {}\n\n",
14    std::regex_search(s1, std::regex{"Programming"}),
15    std::regex_search(s1, std::regex{"fun"}),
16    std::regex_search(s1, std::regex{"fn"}));
17}
```

```
s1: Programming is fun
Search anywhere in s1:
Programming: true; fun: true; fn: false
```

Илл. 8.18 | Поиск в тексте по шаблонам

Поиск фрагмента без учета регистра букв

Константы `regex_constants` из заголовка `<regex>` позволяют настраивать параметры поиска. Например, по умолчанию поиск чувствителен к регистру, но константа `regex_constants::icase` позволяет искать соответствия без учета регистра.

```
18 // Игнорируем регистр букв
19 std::string s2{"SAM WHITE"};
20 std::smatch match; // Для сохранения фрагмента, соответствующего шаблону
21 std::cout << fmt::format("s2: {}\n\n", s2);
22 std::cout << "Case insensitive search for Sam in s2:\n"
23 << fmt::format("Sam: {}\n", std::regex_search(s2, match,
24 std::regex{"Sam", std::regex_constants::icase}))
25 << fmt::format("Matched text: {}\n\n", match.str());
26
```



```
s2: SAM WHITE
Case insensitive search for Sam in s2:
Sam: true
Matched text: SAM
```

В строках 23–24 вызывается версия функции `regex_search` с тремя аргументами:

- ♦ строка для поиска (`s2`);
- ♦ объект `smatch` (`match`), в котором будет сохранен соответствующий фрагмент, если он будет найден;
- ♦ регулярное выражение (строка 24).

Здесь мы создали регулярное выражение со вторым аргументом:

```
std::regex{"Sam", std::regex_constants::icase}
```

Оно предназначено для поиска слова "Sam" независимо от регистра букв. Фрагмент "SAM" в строке `s2` соответствует регулярному выражению "Sam", поскольку состоит из тех же букв, только заглавных. Для отображения соответствующего фрагмента в строке 25 вызывается функция `str` объекта `match`, возвращающая его строковое представление.

Поиск всех соответствующих фрагментов по всему тексту

Теперь мы выделим из текста все телефонные номера следующей формы: ###-###-####. Этот код находит в строке `contact` каждый фрагмент, соответствующий регулярному выражению `phone` (строка 30), и выводит его на экран.

```
27 // Поиск всех соответствующих фрагментов
28 std::string contact{
29     "Wally White, Home: 555-555-1234, Work: 555-555-4321"};
30 std::regex phone{"(\\d{3}-\\d{3}-\\d{4})"};
31
32 std::cout << fmt::format("Finding phone numbers in:\\n", contact);
33 while (std::regex_search(contact, match, phone)) {
34     std::cout << fmt::format(" {}\\n", match.str());
35     contact = match.suffix();
36 }
37 }
```

```
Finding phone numbers in:
Wally White, Home: 555-555-1234, Work: 555-555-4321
555-555-1234
555-555-4321
```

Цикл `while` выполняется, пока `regex_search` возвращает `true`, то есть пока функция находит соответствия. Каждая итерация цикла:

- ♦ отображает фрагмент, соответствующий регулярному выражению (строка 34);

- ♦ заменяет строку `contact` результатом вызова функции `suffix` объекта `match` (строка 35), возвращающей ту часть строки, в которой еще не производился поиск. Эта новая строка `contact` используется в следующем вызове функции `regex_search`.

8.21. Итоги

В этой главе мы рассмотрели целый ряд операций со строками класса `std::string`: присваивание, конкатенацию, сравнение, поиск фрагментов и обмен данными между строками. Мы также рассказали о функциях класса `std::string`, предназначенных для определения характеристик строки; для поиска, замены и вставки символов в строку; для взаимных преобразований между строками `std::string` и строками в стиле C; для взаимных преобразований между строками и числами.

Мы продемонстрировали создание и обработку текстовых файлов с помощью функций из заголовка `<fstream>` и файловых потоков ввода-вывода. Затем мы перешли к обработке строк с помощью строковых потоков ввода-вывода. В первом примере применения готовых объектов мы прочитали набор данных «Титаник» из CSV-файла с помощью библиотеки с открытым исходным кодом, а затем проанализировали эти данные. Во втором примере мы использовали регулярные выражения для поиска и замены фрагментов текста.

Итак, к концу главы 8 вы изучили общие принципы работы с управляющими инструкциями, функциями, массивами, векторами, строками и файлами. В наших примерах с готовыми объектами вы видели, что в программах на C++ часто создаются и используются объекты, выполняющие много полезной работы. В главе 9 вы узнаете, как создавать свои собственные классы и использовать объекты этих классов в программах. Мы начнем рассматривать внутреннее устройство классов и связанные с ним концепции разработки программного обеспечения.

Пользовательские классы

В этой главе вы:

- определите пользовательский класс и создадите объекты этого класса;
- реализуете поведение класса в виде функций, а атрибуты класса — в виде элементов данных;
- научитесь обращаться к скрытым элементам данных класса через его открытые функции, сохраняя инкапсуляцию данных;
- инициализируете конструктором класса элементы данных создаваемых объектов;
- отделите интерфейс класса от реализации класса для повторного использования;
- научитесь обращаться к элементам класса с помощью операторов «точка» [.] и «стрелка» [->];
- узнаете, как деструкторы выполняют завершающую обработку перед уничтожением объектов;
- научитесь присваивать элементы данных одного объекта элементам данных другого объекта;
- научитесь создавать объекты, содержащие другие объекты;
- познакомитесь с дружественными функциями и классами;
- научитесь обращаться к нестатическим элементам класса через указатель `this`;
- научитесь работать со статическими элементами данных и функциями класса;
- создадите агрегатные типы данных и инициализируете их элементы назначенными инициализаторами;
- примените готовые объекты библиотеки `cereal` для сериализации объектов в формате JSON.

9.1. Введение¹

В разделе 1.4 мы популярно рассказали об основах объектно-ориентированного программирования, рассмотрев классы, объекты, элементы данных класса (атрибуты объектов) и функции класса (определяющие поведение объектов). Затем в учебных примерах с готовыми объектами вы создавали объекты классов, написанных другими программистами, и вызывали функции этих классов, тем самым поручая объектам выполнение сложных задач. Для этого вам даже не надо было знать, что происходит внутри этих классов в процессе работы.

В этой главе мы приступим к углубленному изучению объектно-ориентированного программирования: теперь вы будете создавать **пользовательские классы** (**custom classes**), то есть свои собственные. C++ является **расширяемым языком программирования** (**extensible programming language**). Это означает, что каждый созданный вами класс становится новым типом данных, который вы можете использовать для создания объектов. Некоторые команды профессиональных разработчиков в ИТ-компаниях пишут приложения, содержащие сотни и даже тысячи пользовательских классов.

9.2. Тест-драйв объекта пользовательского класса Account

Мы начнем с трех примеров кода, в которых создаются объекты класса Account, моделирующего простейший банковский счет. Для начала рассмотрим функцию main и выходные данные программы, чтобы вы увидели объект нашего первого класса Account в действии. Чтобы вам было проще подготовиться к работе над большими программами, которые вы встретите в последующих главах и в ИТ-отрасли, мы определили функцию main и класс Account в отдельных файлах, как это принято в ИТ-отрасли: функцию main — в основном файле программы AccountTest.cpp (илл. 9.1), а класс Account — в заголовочном файле Account.h (илл. 9.2).

```
1 // Fig. 9.1: AccountTest.cpp
2 // Создание и применение объекта класса Account
3 #include <fmt/format.h>
4 #include <iostream>
5 #include <string>
6 #include "Account.h"
7
8 int main() {
9     Account myAccount{}; // Создаем объект myAccount класса Account
10
11     // Отображаем начальное имя клиента в объекте myAccount (пустую строку)
12     std::cout << fmt::format("Initial account name: {}\n",
13         myAccount.getName());
14 }
```

Окончание ⇨

¹ Эта глава развивает терминологию и концепции, кратко представленные в разделе 1.4.

```

15 // Запрашиваем и получаем имя клиента
16 std::cout << "Enter the account name: ";
17 std::string name{};
18 std::getline(std::cin, name); // Читаем строку текста с клавиатуры
19 myAccount.setName(name); // Сохраняем введенное имя в объекте myAccount
20
21 // Отображаем имя клиента, сохраненное в объекте myAccount
22 std::cout << fmt::format("Updated account name: {}\n",
23     myAccount.getName());
24 }

```

```

Initial account name:
Enter the account name: Jane Green
Updated account name: Jane Green

```

Илл. 9.1 | Создание и применение объекта класса Account

Создание объекта — экземпляра класса

Как правило, вы не сможете вызывать функции класса, пока не создадите объект этого класса¹. В строке 9 создается объект класса Account с именем `myAccount`:

```
Account myAccount{}; // Создаем объект myAccount класса Account
```

Иными словами, создается переменная `myAccount` типа `Account` — это тип класса, определенного в файле `Account.h` (илл. 9.2).

Заголовок пользовательского класса

Когда мы объявляем переменную типа `int`, компилятор заранее знает, что `int` — это один из фундаментальных типов, встроенных в язык C++. Но когда в строке 9 мы объявляем переменную типа `Account`, компилятор заранее не знает, что такое `Account`, потому что это **пользовательский тип (user-defined type)**.

Правильно написанные и подготовленные пользовательские классы могут повторно использоваться другими программистами. Определение класса, предназначенного для повторного использования, обычно помещают в **заголовок (header)** — файл с расширением `.h`². Затем, если вам нужна функциональность этого класса, вы просто включаете в свою программу этот заголовок. Именно так вы получали в свое распоряжение классы стандартной библиотеки C++ и сторонних библиотек в примерах кода из предыдущих глав книги.

В строке 6 мы сообщаем компилятору, что такое `Account`, включив в программу соответствующий заголовок:

```
#include "Account.h"
```

Если бы мы этого не сделали, компилятор выдавал бы сообщения об ошибках при каждом упоминании «незнакомое» класса `Account` или каких-либо его эле-

¹ В разделе 9.20 вы увидите, что есть исключение из этого правила: статические функции.

² Заголовки стандартной библиотеки C++, такие как `<iostream>`, не используют расширение `.h`, а некоторые программисты предпочитают расширение `.hpp`.

ментов. Имя заголовка, который вы определили в своей программе, помещается в двойные кавычки (""), а не в угловые скобки (<>). Двойные кавычки сообщают компилятору, что заголовок с этим именем надо сначала искать в папке с основным файлом программы `AccountTest.cpp` (илл. 9.1), а только потом — в папке для заголовков по умолчанию¹.

Вызов функции `getName` класса `Account`

Функция `getName` класса `Account` возвращает имя клиента банка (значение переменной `m_name`), хранящееся в конкретном объекте класса `Account`. В строке 13 мы вызываем функцию `getName` для только что созданного объекта `MyAccount`, чтобы получить исходное имя клиента, которое является пустой строкой. Вскоре мы расскажем об этом подробнее.

Вызов функции `setName` класса `Account`

Функция `setName` класса `Account` сохраняет имя клиента банка в конкретном объекте класса `Account`. В строке 19 мы вызываем функцию `setName` для объекта `myAccount`, чтобы сохранить в этом объекте значение переменной `name`.

Отображение имени клиента, введенного пользователем

В строке 23 функция `getName` снова вызывается для подтверждения, что объект `myAccount` содержит введенное пользователем имя клиента.

9.3. Класс Account с элементом данных, set-функцией и get-функцией

Теперь, когда вы увидели, как работает класс `Account` (илл. 9.1), рассмотрим его внутреннее устройство.

9.3.1. Определение класса

Класс `Account` (илл. 9.2) содержит элемент данных `m_name` (строка 19) — переменную для хранения имен клиентов банка. Каждый из объектов класса имеет собственную копию элементов данных этого класса². В разделе 9.6 мы добавим в класс `Account` элемент данных `balance` — переменную для хранения «денег» клиентов. На данный момент класс `Account` содержит две функции:

- ♦ `setName` (строки 10–12) — для записи имени клиента;
- ♦ `getName` (строки 15–17) — для чтения имени клиента.

¹ Ваши пользовательские заголовки классов можно включать в программу с помощью угловых скобок, если вы включите папку вашей программы в путь, указанный компилятору для поиска заголовков по умолчанию. (Вы уже включали таким образом сторонние библиотеки в некоторых примерах с готовыми объектами.)

² За исключением статических элементов данных, которые являются общими для всех объектов класса. Вы узнаете о них в разделе 9.20.

```

1 // Fig. 9.2: Account.h
2 // Класс Account с элементом данных
3 // и функциями для его записи (set) и чтения (get)
4 #include <string>
5 #include <string_view>
6
7 class Account {
8 public:
9     // Set-функция класса, записывающая в объект значение элемента данных
10    void setName(std::string_view name) {
11        m_name = name; // Записываем в m_name имя клиента
12    }
13
14    // Get-функция класса, читающая из объекта значение элемента данных
15    const std::string& getName() const {
16        return m_name; // Возвращаем имя клиента в точку вызова функции
17    }
18 private:
19     std::string m_name; // Элемент данных, хранящий имя клиента банка
20 }; // Конец класса Account

```

Илл. 9.2 | Класс Account с элементом данных и функциями для его записи (set) и чтения (get)

Ключевое слово class и тело класса

Определение класса начинается с ключевого слова `class` (строка 7), за которым следует имя класса (`Account`). Когда вы пишете определение класса, соблюдайте общепринятое соглашение:

- ✦ каждое слово в имени класса начинается с заглавной буквы;
- ✦ имена элементов данных (**data members**) класса и функций класса (**member functions**), также называемых функциями-членами или функциями-элементами, начинаются со строчной буквы, а каждое последующее слово в таком имени начинается с заглавной буквы.

Тело каждого класса заключено в фигурные скобки (строки 7 и 20). Определение класса обязательно завершается точкой с запятой (строка 20).

Элемент данных `m_name` типа `std::string`

Напомним из раздела 1.4, что у объектов есть атрибуты. Они реализованы как элементы данных. Каждый объект содержит собственные копии этих данных на протяжении своего жизненного цикла, то есть до тех пор, пока он существует в памяти компьютера. Обычно класс содержит функции, которые оперируют данными конкретных объектов этого класса.

Элементы данных объявляются внутри определения класса, но вне функций класса. В строке 19 объявляется элемент данных с именем `m_name`:

```
std::string m_name; // Элемент данных, хранящий имя клиента банка
```

Префикс `m_` нужен по общепринятому соглашению об именовании, он указывает, что эта переменная — элемент данных. Если вы создадите много объектов класса `Account`, каждый из них будет иметь свою собственную переменную `m_name`.

Поскольку `m_name` является элементом данных, с ним можно выполнять разные действия, вызывая функции класса. Напомним, что переменные типа `string` по умолчанию инициализируются пустой строкой (`""`), поэтому первый вызов функции `getName` из `main` (илл. 9.1, строка 13) не возвращает никакого имени клиента.

По общепринятому соглашению, программисты C++ обычно размещают элементы данных класса в конце его тела. Вы можете объявлять элементы данных в любых местах за пределами определений функций, но разброс элементов данных по всему классу делает код трудным для понимания.

Квалификатор `std::` для стандартных библиотечных компонентов в заголовках

В заголовке `Account.h` (илл. 9.2) мы используем квалификатор `std::` при обращении к `string_view` (строка 10) и к `string` (строки 15 и 19). В заголовках не должно быть ни директив `using`, ни объявлений `using`, указывающих на глобальную область видимости. Это необходимо во избежание конфликтов имен, так как заголовки включаются в другие файлы исходного кода. Начиная с главы 6, мы уточняем квалификатором `std::` все имена классов стандартной библиотеки, функций и объектов (например, `std::cout`) — это считается хорошей практикой.

Функция `setName`

Функция `setName` (строки 10–12) получает объект `string_view`, представляющий имя клиента, и присваивает аргумент `name` элементу данных `m_name`. Напомним, что `string_view` — это представление последовательности символов (современной строки `std::string` или традиционной строки в стиле C), доступное только для чтения. В строке 11 символы из параметра `name` копируются в `m_name`. Благодаря префиксу `m_` в имени `m_name`, параметр функции `name` легко отличить от элемента данных `m_name`.

Функция `getName`

Функция `getName` (строки 15–17) не имеет параметров и возвращает значение `m_name` из конкретного объекта класса `Account` в точку вызова в виде `const std::string&`. Поскольку возвращаемое значение является константной ссылкой, вы можете быть уверены, что вызывающий код не сможет изменить данные через эту ссылку.

Функции класса, объявленные с квалификатором `const`

Обратите внимание на `const` справа от списка параметров в заголовке `getName` (строка 15). При возврате `m_name` функция `getName` не изменяет и не должна изменять объект, для которого она вызывается. Объявление функции с квалификатором `const` сообщает компилятору: «Эта функция не должна изменять объект,

а если попробует, надо прервать компиляцию из-за ошибки». Это поможет вам сразу обнаружить ошибку, если вы случайно напишете код, который попытается некорректно изменить объект. Это объявление также сообщает компилятору, что функция `getName` может быть вызвана для объекта `const Account`, а также через ссылку или указатель на `const Account`.

9.3.2. Спецификаторы доступа `private` и `public`

Ключевое слово `private` (строка 18) — это **спецификатор доступа** (**access specifier**). Все спецификаторы доступа обязательно дополняются двоеточием (например, `private:`). Элемент данных `m_name` (строка 19) объявлен после `private:`, поэтому является **скрытым** (**private**), то есть доступным только для функций своего класса `Account`¹. Когда вы объявляете элементы данных класса скрытыми, это называется **сокрытием данных** (**information hiding**) или **сокрытием деталей реализации** (**hiding implementation details**). Сокрытие данных рекомендуется в C++ Core Guidelines² в качестве обычной практики и повсеместно применяется в объектно-ориентированном программировании. Поскольку элемент данных `m_name` инкапсулирован (скрыт), его могут использовать только функции класса `Account: setName` и `getName`. Как правило, все элементы данных надо объявлять после спецификатора доступа `private:`³.

В классе `Account` также есть спецификатор доступа `public` (строка 8):

public:

Элементы класса, перечисленные после спецификатора доступа `public` и перед следующим спецификатором доступа, если он есть, являются **открытыми** (**public**), то есть общедоступными. Их можно использовать везде, где объект класса находится в области видимости.

Объявление элементов данных скрытыми (`private`) упрощает отладку кода, потому что все проблемы с обработкой данных локализуются внутри функций класса. Подробнее о преимуществах `private` мы расскажем в разделе 9.7. В главе 10 мы рассмотрим спецификатор доступа `protected`.

Доступ к элементам класса по умолчанию

По умолчанию все элементы любого класса являются скрытыми (`private`), если вы не укажете иное. После строки со спецификатором доступа каждый объявленный элемент класса будет иметь этот уровень доступа — до тех пор, пока вы не укажете другой спецификатор доступа. Спецификаторы `public` и `private` можно повторять в разных местах класса, но это затрудняет понимание кода. Мы предпочитаем

¹ А также для дружественных функций класса, как вы увидите в разделе 9.18.

² C++ Core Guidelines, «C.9: Minimize Exposure of Members». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rc-private>.

³ Далее мы обычно будем опускать двоеточие, когда ключевые слова `private` и `public` упоминаются вне кода, как в этой сноске.

включать в каждый класс только один спецификатор `public`, группируя после него все открытые элементы, и только один спецификатор `private`, группируя после него все скрытые элементы.

9.4. Конструктор класса Account

Как было показано в разделе 9.3, при создании объекта класса `Account` его элемент `m_name` по умолчанию инициализируется пустой строкой. Но что делать, если при создании объекта класса `Account` вам нужно инициализировать элемент каким-то определенным значением, например указать имя клиента банка? В каждом классе можно определять **конструкторы (constructors)**, позволяющие инициализировать объекты этого класса нужными значениями. Конструктор — это специальная функция класса, которая должна иметь то же имя, что и класс. Конструкторы не могут возвращать значения, поэтому у них нет возвращаемого типа (даже `void`). C++ гарантирует вызов конструктора при создании объекта, поэтому конструктор идеально подходит для инициализации элементов данных объекта. Каждый раз, когда вы создавали какой-либо объект в наших примерах кода, для инициализации этого объекта вызывался конструктор соответствующего класса. Как вы скоро увидите, классы могут иметь перегруженные конструкторы.

Подобно обычным функциям класса, конструкторы могут иметь параметры. Соответствующие параметрам значения аргументов помогают вам инициализировать элементы данных объекта. Например, при создании объекта класса `Account` вы можете указать имя клиента банка, как это сделано в строке 10 на илл. 9.4:

```
Account account1{"Jane Green"};
```

Здесь строка `"Jane Green"` передается конструктору класса `Account` для инициализации данных объекта `account1`. Эта инструкция предполагает, что в классе `Account` есть конструктор, который может принимать строковый аргумент.

Определение класса Account

На илл. 9.3 показан класс `Account` с конструктором, получающим параметр `accountName` для инициализации элемента данных `m_name` при создании объекта `Account`.

```
1 // Fig. 9.3: Account.h
2 // Класс Account с конструктором, инициализирующим объект именем клиента
3 #include <string>
4 #include <string_view>
5
6 class Account {
7 public:
8     // Конструктор инициализирует элемент данных m_name параметром name
9     explicit Account(std::string_view name)
10         : m_name{name} { // Инициализируем элемент данных
11             // Пустое тело
12     }
```

```

13
14 // Set-функция, записывающая значение элемента данных
15 void setName(std::string_view name) {
16     m_name = name;
17 }
18
19 // Get-функция, читающая значение элемента данных
20 const std::string& getName() const {
21     return m_name;
22 }
23 private:
24     std::string m_name; // Имя клиента банка
25 }; // Конец класса Account

```

Илл. 9.3 | Класс `Account` с конструктором, инициализирующим объект именем клиента банка

Определение конструктора в классе `Account`

В строках 9–12 на илл. 9.3 определяется конструктор класса `Account`. Обычно конструкторы являются открытыми (`public`), поэтому любой код, имеющий доступ к определению класса, может создавать и инициализировать объекты этого класса. В строке 9 указано, что параметр конструктора имеет тип `string_view`. При создании нового объекта класса `Account` вы передаете конструктору имя клиента в параметр типа `string_view`. Затем конструктор инициализирует элемент данных `m_name` значением этого параметра. В строке 9 на илл. 9.3 не указан тип возвращаемого значения (даже `void`), поскольку конструкторы не могут возвращать значения. Кроме того, конструкторы не могут быть объявлены как `const`, потому что инициализация объекта должна его изменять.

В конструкторе есть **список инициализаторов элементов** (**member-initializer list**), который инициализирует элемент данных `m_name` (строка 10):

```
: m_name{name}
```

Инициализаторы элементов данных находятся между списком параметров конструктора и левой фигурной скобкой, начинающей тело конструктора. Список инициализаторов отделяется от списка параметров двоеточием (`:`).

Каждый инициализатор состоит из имени переменной элемента данных и последующих фигурных скобок, содержащих инициализирующее значение¹. В данном случае инициализатор вызывает конструктор класса `std::string`, получающий значение `string_view`. Если класс содержит более одного элемента данных, их инициализаторы разделяются запятыми. Элементы данных класса инициализируются конструктором в том порядке, как они объявлены. Для наглядности кода перечисляйте инициализаторы в списке в том же порядке. Список инициализаторов элементов выполняется перед телом конструктора.

¹ В некоторых случаях фигурные скобки заменяются круглыми, например, при инициализации вектора заданного размера (илл. 6.14).

Хотя вы можете инициализировать элементы данных инструкциями присваивания в теле конструктора, в C++ Core Guidelines рекомендуется использовать инициализаторы¹. Позже вы увидите, что инициализаторы могут быть эффективнее, чем инструкции присваивания. Кроме того, вы увидите, что некоторые элементы данных надо инициализировать только инициализаторами, поскольку им нельзя присваивать значения в теле конструктора.

Ключевое слово `explicit`

Объявляйте конструктор с ключевым словом `explicit`, если этот конструктор можно вызвать с одним аргументом (если у него только один параметр или если все параметры, кроме первого, имеют аргументы по умолчанию). Это не позволяет компилятору использовать конструктор для неявных преобразований типов². Ключевое слово `explicit` означает, что конструктор класса `Account` должен быть вызван явно, как в строке 10 на илл. 9.4:

```
Account account1{"Jane Green"};
```

Пока просто запомните, что конструкторы с одним параметром надо объявлять как вызываемые явно, то есть с ключевым словом `explicit`. В разделе 11.9 вы увидите, что конструкторы с одним параметром, не объявленные как `explicit`, могут вызываться компилятором неявно для преобразования типов. Такие неявные вызовы конструкторов нежелательны, поскольку могут приводить к скрытым ошибкам.

Инициализация объектов класса `Account` при их создании

Программа `AccountTest` (илл. 9.4) инициализирует два объекта класса `Account` с помощью конструктора. В строке 10 создается объект класса `Account` с именем `account1`:

```
Account account1{"Jane Green"};
```

Эта инструкция вызывает конструктор класса `Account` (строки 9–12 на илл. 9.3), который использует аргумент `"Jane Green"` для инициализации элемента данных `m_name` в новом объекте. В строке 11 на илл. 9.4 процесс повторяется: аргумент `"John Blue"` передается для инициализации элемента данных `m_name` в объекте `account2`:

```
Account account2{"John Blue"};
```

В строках 14–16 мы вызываем функцию `getName` для каждого объекта класса `Account`, чтобы получить его имя и тем самым подтвердить корректность инициализации. Выходные данные показывают, что имена клиентов в объектах различаются. Каждый объект имеет свою собственную копию элемента `m_name`.

¹ C++ Core Guidelines, «C.49: Prefer Initialization to Assignment in Constructors». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rc-initialize>.

² C++ Core Guidelines, «C.46: By Default, Declare Single-Argument Constructors explicit». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rc-explicit>.

```
1 // Fig. 9.4: AccountTest.cpp
2 // Инициализация элемента m_name конструктором класса Account
3 // при создании объектов этого класса
4 #include <fmt/format.h>
5 #include <iostream>
6 #include "Account.h"
7
8 int main() {
9     // Создаем два объекта класса Account
10    Account account1{"Jane Green"};
11    Account account2{"John Blue"};
12
13    // Отображаем имена клиентов, хранящиеся в объектах
14    std::cout << fmt::format(
15        "account1 name is: {}\\naccount2 name is: {}\\n",
16        account1.getName(), account2.getName());
17 }
```

```
account1 name is: Jane Green
account2 name is: John Blue
```

Илл. 9.4 | Инициализация элемента `m_name` конструктором класса `Account` при создании объектов этого класса

Конструктор по умолчанию

Напомним, что в строке 9 на илл. 9.1 объект класса `Account` создавался с пустыми фигурными скобками справа от имени переменной объекта:

```
Account myAccount{};
```

При выполнении этой инструкции C++ неявно вызывает для класса `Account` **конструктор по умолчанию (default constructor)**. Если в классе нет собственного конструктора, компилятор генерирует для этого класса конструктор по умолчанию, не имеющий параметров.

Конструктор по умолчанию не инициализирует элементы данных фундаментальных типов, но вызывает конструкторы по умолчанию для всех элементов данных, которые являются объектами других классов. Например, при создании объекта нашего первого класса `Account` (илл. 9.2) вызывается конструктор класса `std::string` по умолчанию, инициализирующий элемент данных `m_name` пустой строкой (""). Вы не видите этого в исходном коде, но именно так создаются объекты.

Когда вы объявляете объект с пустыми фигурными скобками, как в предыдущей инструкции, все элементы данных, не инициализированные явно, становятся **инициализированными значением по умолчанию (value initialized)**. Это означает, что они инициализируются нулями, а затем для каждого объекта вызывается конструктор по умолчанию. Если фигурных скобок нет, такие элементы данных заполняются неопределенными («мусорными») значениями.

Для класса с собственным конструктором не создается конструктор по умолчанию

Если вы определили в классе конструктор, компилятор не предоставит для этого класса конструктор по умолчанию. Например, вы не сможете создать объект класса `Account`, вызвав конструктор без аргументов (за исключением того случая, когда список параметров вашего конструктора пуст или у всех параметров есть аргументы по умолчанию). Позже мы покажем, как «заставить» компилятор создавать конструкторы по умолчанию даже для классов, имеющих собственные конструкторы. Если вас не устраивает инициализация элементов данных вашего класса значениями по умолчанию, инициализируйте каждый элемент в его объявлении или определите в классе такой конструктор, который инициализирует все элементы нужными значениями.

Специальные функции класса

Кроме конструктора по умолчанию, компилятор может генерировать несколько **специальных функций класса** (**special member functions**) в версиях по умолчанию: копирующий конструктор, перемещающий конструктор, оператор копирующего присваивания, оператор перемещающего присваивания и деструктор; а также новый оператор трехстороннего сравнения (C++20). В этой главе мы кратко представим вам копирующий конструктор, оператор копирующего присваивания и деструктор. В главе 11 мы расскажем об операторе трехстороннего сравнения и детальнее рассмотрим все специальные функции класса, в том числе:

- ♦ перечислим ситуации, в которых вам надо будет определять ваши собственные версии этих функций;
- ♦ укажем на рекомендации C++ Core Guidelines, относящиеся к этим функциям.

Вы увидите, что обычно должны проектировать классы таким образом, чтобы компилятор автоматически генерировал для вас эти специальные функции. Если вы не включаете в определение класса никакие специальные функции, это называется «правилом нуля».

9.5. Разработка программ с set-функциями и get-функциями

Как вы увидите в следующем разделе, *set*-функции класса (предназначенные для записи) и *get*-функции класса (предназначенные для чтения) могут контролировать все попытки записи в скрытые элементы данных объекта, а также способы передачи прочитанных данных в точку вызова. Это дает большие преимущества при разработке программ.

Клиентом (client) класса называется любой код, вызывающий функции этого класса. Если элемент данных открыт (объявлен как `public`), любой клиент может его увидеть и сделать с ним что угодно, например внести недопустимое изменение.

Если элемент данных скрыт (объявлен как `private`), клиент не может получить к нему прямой доступ. Но с первого взгляда кажется, что клиент все равно может делать с ним что угодно, вызывая доступные ему `set`-функции и `get`-функции класса. Кажется, что ничто не мешает клиенту в любой момент заглядывать извне в скрытые данные объекта (и видеть, как они хранятся в объекте) с помощью `get`-функций и произвольно изменять эти данные с помощью `set`-функций.

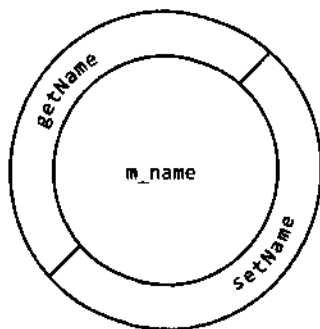
На самом деле `set`-функции могут быть написаны так, чтобы они проверяли получаемые аргументы и отклоняли все попытки присвоить элементам данных класса недопустимые значения. Примеры недопустимых аргументов:

- ✦ отрицательная температура тела человека;
- ✦ день в марте, выходящий за пределы диапазона с 1 по 31;
- ✦ артикул товара, которого нет в каталоге продукции компании.

В свою очередь, `get`-функции могут возвращать данные в форматах, которые отличаются от исходных форматов элементов данных. В этом случае фактическое представление данных объекта остается скрытым от пользователя (от клиента класса). Например, класс `Grade` хранит оценку студента в виде числа `int` в диапазоне от 0 до 100, но функция этого класса `getGrade` возвращает буквенную оценку в виде строки: "A" — для оценок от 90 до 100, "B" для оценок от 80 до 89 и т. д. Внимательно контролируя обращения к скрытым (`private`) данным и способы их представления в возвращаемых значениях, вы можете свести к минимуму риск ошибок и сделать ваши программы надежными, безопасными и удобными для пользователей.

Концепция объекта класса `Account` с инкапсулированными данными

Представьте себе объект класса `Account` так, как показано на следующей диаграмме. Элемент данных `m_name`, изображенный в виде внутреннего круга, скрыт внутри объекта (объявлен как `private`). Получить к нему доступ можно только через внешний уровень открытых функций `getName` и `setName` (объявленных как `public`), изображенный в виде кольца. Любой клиент, которому надо взаимодействовать с объектом класса `Account`, может делать это только одним способом: вызывать функции внешнего уровня.



Как правило, все элементы данных класса объявляются скрытыми (**private**), а все предоставляемые клиентам функции класса объявляются открытыми (**public**). Позже мы рассмотрим исключения из этого правила: иногда некоторые элементы данных вашего класса могут быть открытыми, а некоторые функции — скрытыми. Открытые *set*-функции и *get*-функции, контролирующие доступ к скрытым данным, упрощают понимание кода и расширение функциональности программы в будущем. Постоянное внесение изменений в программы — это в ИТ-отрасли скорее правило, чем исключение. Старайтесь всегда исходить из того, что ваш код будет модифицироваться, причем довольно часто.

9.6. Добавление «денежной» переменной в класс Account

На илл. 9.5 показана следующая версия нашего класса Account. Теперь в нем есть два взаимосвязанных элемента данных: имя клиента банка и количество денег на его банковском счете (баланс). Согласно C++ Core Guidelines, все взаимосвязанные элементы данных рекомендуется определять в одном классе (или, как будет показано в разделе 9.21, — в одной структуре)¹.

```
1 // Fig. 9.5: Account.h
2 // Класс Account с элементами данных m_name и m_balance,
3 // с конструктором и функцией deposit, проверяющей данные
4 #include <algorithm>
5 #include <string>
6 #include <string_view>
7
8 class Account {
9 public:
10    // Конструктор класса Account с двумя параметрами
11    Account(std::string_view name, double balance)
12        : m_name{name}, m_balance{std::max(0.0, balance)} { // Инициализация
13        // Пустое тело
14    }
15
16    // Функция, пополняющая счет только после проверки данных
17    void deposit(double amount) {
18        if (amount > 0.0) { // Если значение допустимо
19            m_balance += amount; // Прибавляем его к m_balance
20        }
21    }
22
23    // Функция, возвращающая баланс счета
24    double getBalance() const {
25        return m_balance;
26    }
```

Окончание ⇨

¹ C++ Core Guidelines, «C.1: Organize Related Data into Structures (structs or classes)». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rc-org>.


```

27
28 // Функция, изменяющая имя клиента банка
29 void setName(std::string_view name) {
30     m_name = name; // Присваиваем значение name элементу данных m_name
31 }
32
33 // Функция, возвращающая имя клиента банка
34 const std::string& getName() const {
35     return m_name;
36 }
37 private:
38     std::string m_name;
39     double m_balance;
40 }; // Конец класса Account

```

Илл. 9.5 | Класс `Account` с элементами данных `m_name` и `m_balance`, с конструктором и функцией `deposit`, проверяющей данные

Элемент данных `balance`

Банк обслуживает множество счетов, каждый из которых имеет собственный баланс. Каждый объект класса `Account` имеет собственные элементы данных `m_name` и `m_balance`. В строке 39 объявляется «денежный» элемент данных `m_balance` типа `double`¹.

Конструктор с двумя параметрами

В этом классе есть один конструктор и четыре функции. Обычно клиент банка, открывающий счет, сразу же вносит деньги, поэтому конструктор (в строках 11–14) получает второй параметр `balance` (`double`), который представляет первоначальный баланс. Мы не объявляли этот конструктор как `explicit`, поскольку его нельзя вызвать только с одним параметром.

Инициализатор элемента данных `m_balance` вызывает функцию `std::max` (заголовок `<algorithm>`), которая инициализирует `m_balance` числом `0.0` или параметром `balance`, если он больше. Это гарантирует, что при создании каждого объекта класса `Account` переменная `m_balance` будет иметь допустимое неотрицательное значение. Позже мы добавим проверку аргументов в теле конструктора и будем использовать исключения для обнаружения недопустимых аргументов.

Функция `deposit`

Функция `deposit` (строки 17–21) получает параметр `amount` типа `double` и не возвращает никакого значения. Инструкция в строках 18–20 гарантирует, что значение параметра `amount` будет прибавлено к `m_balance` только при условии, что оно больше нуля (то есть если сумма на счете корректна).

¹ Напомним, что в финансовых приложениях, предназначенных для практического применения, нельзя использовать тип `double` для представления денежных величин.

Функция `getBalance`

Функция `getBalance` (строки 24–26) позволяет клиентам класса `Account` получать значение `m_balance` из любого конкретного объекта этого класса. Функция имеет возвращаемый тип `double` и пустой список параметров. Как и `getName`, функция `getBalance` объявляется с квалификатором `const`, потому что возврат значения `m_balance` не изменяет и не должен изменять объекты класса `Account`.

Операции с объектами класса `Account`

Функция `main` на илл. 9.6 создает два объекта класса `Account` (строки 8–9). Они инициализируются допустимым (50.00) и недопустимым (-7.00) значениями баланса соответственно. В строках 12–15 мы выводим на экран имя и баланс, хранящиеся в объектах, вызывая их функции `getName` и `getBalance`. Наш класс гарантирует, что первоначальный баланс на каждом счете не может быть меньше нуля. Баланс объекта `account2` равен 0.0, поскольку конструктор отклонил попытку ввода отрицательного числа, заменив его на 0.0.

```
1 // Fig. 9.6: AccountTest.cpp
2 // Отображение и пополнение баланса объектов класса Account
3 #include <fmt/format.h>
4 #include <iostream>
5 #include "Account.h"
6
7 int main() {
8     Account account1{"Jane Green", 50.00};
9     Account account2{"John Blue", -7.00};
10
11     // Выводим на экран первоначальный баланс каждого счета
12     std::cout << fmt::format("account1: {} balance is {:.2f}\n",
13         account1.getName(), account1.getBalance());
14     std::cout << fmt::format("account2: {} balance is {:.2f}\n\n",
15         account2.getName(), account2.getBalance());
16 }
```

```
account1: Jane Green balance is $50.00
account2: John Blue balance is $0.00
```

Илл. 9.6 | Отображение и пополнение баланса объектов класса `Account`

Внесение «денег» на счет клиента банка

В строках 17–21 запрашивается ввод с клавиатуры числа, обозначающего пополнение баланса в объекте `account1`. В строке 22 вызывается функция `deposit` объекта `account1`, прибавляющая к его балансу значение аргумента `amount`. В строках 25–28 на экран выводятся данные из обоих объектов класса `Account`: имя и баланс каждого клиента. Мы видим, что изменился только счет первого клиента (`amount1`).

```
17     std::cout << "Enter deposit amount for account1: "; // Ввод данных
18     double amount;
```

```

19  std::cin >> amount; // Получаем число от пользователя
20  std::cout << fmt::format(
21      "adding ${:.2f} to account1 balance\n\n", amount);
22  account1.deposit(amount); // Прибавляем это число к балансу счета account1
23
24  // Выводим на экран баланс каждого счета
25  std::cout << fmt::format("account1: {} balance is ${:.2f}\n",
26      account1.getName(), account1.getBalance());
27  std::cout << fmt::format("account2: {} balance is ${:.2f}\n\n",
28      account2.getName(), account2.getBalance());
29

```

```

Enter deposit amount for account1: 25.37
adding $25.37 to account1 balance

account1: Jane Green balance is $75.37
account2: John Blue balance is $0.00

```

В строках 30–33 запрашивается число, которое будет прибавлено к балансу счета `account2`. В строке 34 вызывается функция `deposit` объекта `account2`, прибавляющая к его балансу значение аргумента `amount`. В строках 37–40 на экран снова выводятся данные из обоих объектов класса `Amount`. Мы видим, что теперь изменился только счет второго клиента (`amount2`).

```

30  std::cout << "Enter deposit amount for account2: "; // Ввод данных
31  std::cin >> amount; // Получаем число от пользователя
32  std::cout << fmt::format(
33      "adding ${:.2f} to account2 balance\n\n", amount);
34  account2.deposit(amount); // Прибавляем это число к балансу счета account2
35
36  // Выводим на экран баланс каждого счета
37  std::cout << fmt::format("account1: {} balance is ${:.2f}\n",
38      account1.getName(), account1.getBalance());
39  std::cout << fmt::format("account2: {} balance is ${:.2f}\n",
40      account2.getName(), account2.getBalance());
41 }

```

```

Enter deposit amount for account2: 123.45
adding $123.45 to account2 balance

account1: Jane Green balance is $75.37
account2: John Blue balance is $123.45

```

9.7. Отделение интерфейса от реализации на примере класса `Time`

В предыдущих примерах мы помещали определение класса в заголовок для повторного использования, а затем включали заголовок в файл исходного кода с функцией `main`, чтобы создавать объекты этого класса и управлять ими. К со-

жалению, размещение полного определения класса в заголовке раскрывает всю реализацию класса (его внутреннее устройство) клиентам класса. Заголовок — это обычный текстовый файл, который легко открыть и прочитать.

Общепринятая мудрость профессиональных программистов гласит, что для использования объектов любого класса его клиенты (например, функция `main`) должны знать совсем немного:

- ♦ какие функции этих объектов можно вызывать;
- ♦ какие аргументы надо предоставлять каждой функции;
- ♦ какой возвращаемый тип ожидается от каждой функции.

Клиентскому коду не обязательно знать о том, как именно реализованы функции класса. Это еще один пример принципа наименьших привилегий.

Если программист клиентского кода знает, как реализован класс, он может сделать свой код зависимым от каких-то особенностей этой реализации. И если в будущем реализация этого класса изменится, возникнут проблемы. Клиентский код надо писать так, чтобы его не приходилось переделывать в случае изменений в реализации класса. Скрытие деталей реализации класса позволяет ее модернизировать, но при этом исключить (или хотя бы свести к минимуму) необходимость изменений в клиентском коде.

В следующем примере мы создадим и используем объект класса `Time`¹. Мы продемонстрируем две важные концепции разработки программ на C++:

- ♦ Интерфейс класса должен быть отделен от реализации класса.
- ♦ В каждом заголовке необходима директива препроцессора `#pragma once`, запрещающая многократное (более одного раза) включение этого заголовка в какой-либо файл исходного кода. Поскольку каждый класс должен быть определен в программе только один раз, директива `#pragma once` предотвращает ошибки, вызванные повторяющимися определениями.

Модули (C++20) дают новые возможности отделения интерфейса от реализации

Как вы увидите в главе 16, модули, которые появились в C++20², позволяют обойтись без директив препроцессора, в том числе без директивы `#pragma once`. Вы также увидите, что с помощью модулей вы сможете отделять интерфейс от реализации разными способами: или в одном общем файле исходного кода, или в нескольких файлах.

¹ Это учебный пример. Для работы со значениями времени и даты вам не придется писать собственные классы; используйте заголовок `<chrono>` из стандартной библиотеки C++ (<https://en.cppreference.com/w/cpp/chrono>).

² На момент написания книги модули C++20 еще не были полностью реализованы в трех используемых нами компиляторах, поэтому мы рассмотрим модули в отдельной главе.

9.7.1. Интерфейс класса

Интерфейс (interface) определяет и стандартизирует способы взаимодействия человека с какой-либо системой. Например, элементы управления радиоприемником являются интерфейсом между его внутренними компонентами и пользователем. Через этот интерфейс пользователь может выполнять только ограниченный набор операций (таких, как настройка на радиостанцию, регулировка громкости, переключение между диапазонами АМ и FM). Каждая операция может быть реализована по-разному: первый приемник реагирует на нажатие кнопки, второй — на поворот ручки, третий — на голосовую команду. Интерфейс определяет, какие операции приемник разрешает выполнять своим пользователям. Но интерфейс не определяет, как эти операции реализованы внутри приемника.

Аналогично интерфейс класса определяет, какие сервисы этот класс предоставляет своим клиентам и каким образом они должны запрашивать эти сервисы. Но интерфейс класса ничего не говорит о том, как эти сервисы реализованы внутри класса. Открытый (**public**) интерфейс класса состоит из открытых функций класса, которые также называют **открытыми сервисами (public services)** класса. Как вы скоро увидите, для создания интерфейса класса вам достаточно написать такое определение класса, в котором перечислены только прототипы его функций из секции **public**.

9.7.2. Отделение интерфейса класса от реализации класса

Чтобы отделить интерфейс класса от его реализации, мы разделили наш класс **Time** на два файла. В файле **Time.h** (илл. 9.7) определен класс **Time**, а в файле **Time.cpp** (илл. 9.8) определены функции класса **Time**. Такое разделение:

- ♦ позволяет использовать класс многократно;
- ♦ гарантирует, что клиенты класса знают о том, какие функции предоставляет класс, как их вызывать и какие возвращаемые типы данных ожидать;
- ♦ позволяет клиентам игнорировать реализацию (внутреннее устройство) функций класса.

Помимо вышеперечисленных преимуществ, такое разделение ускоряет компиляцию, поскольку файл с определениями функций (файл реализации) можно скомпилировать только один раз и не компилировать повторно, если реализация не изменяется.

По общепринятому соглашению определения функций класса помещаются в файл с расширением **.cpp**, имеющий то же имя, что и заголовок класса (например, **Time**). Некоторые компиляторы поддерживают и другие расширения имен файлов. На илл. 9.9 показана программа с функцией **main**, которая использует объекты нашего класса **Time**.

9.7.3. Определение класса

Заголовок `Time.h` (илл. 9.7) содержит определение класса `Time` (строки 8–17). Вместо определений функций здесь есть только их прототипы (строки 10–12), которые описывают открытый (`public`) интерфейс класса, не раскрывая реализации его функций. Прототип функции `setTime` в строке 10 указывает, что она принимает три аргумента `int` и возвращает значение `void`. Прототипы функций `to24HourString` и `to12HourString` (строки 11–12) указывают, что они не принимают аргументы и возвращают значение `string`. Если бы в классе были конструкторы, их тоже надо было бы объявить в заголовке, как мы сделаем в последующих примерах.

```
1 // Fig. 9.7: Time.h
2 // Определение класса Time
3 // Функции этого класса определены в файле Time.cpp
4 #pragma once // Предотвращаем повторные включения заголовка
5 #include <string>
6
7 // Определение класса Time
8 class Time {
9 public:
10     void setTime(int hour, int minute, int second);
11     std::string to24HourString() const; // 24-часовой формат строки
12     std::string to12HourString() const; // 12-часовой формат строки
13 private:
14     int m_hour{0}; // 0 - 23 (24-часовой формат времени)
15     int m_minute{0}; // 0 - 59
16     int m_second{0}; // 0 - 59
17 };
```

Илл. 9.7 | Определение класса Time

Заголовок по-прежнему содержит скрытые (`private`) элементы данных класса (строки 14–16). Каждый из них инициализируется значением `0` с помощью **внутриклассового инициализатора (in-class initializer)** (C++11). Компилятор должен знать обо всех элементах данных класса, чтобы зарезервировать соответствующий объем памяти для каждого из объектов класса. Включение заголовка `Time.h` в клиентский код предоставляет компилятору всю информацию, которая ему нужна, чтобы правильно вызывать из клиентского кода функции класса `Time`.

В C++ Core Guidelines рекомендуется использовать внутриклассовые инициализаторы, если элементы данных должны быть инициализированы константами¹. В C++ Core Guidelines также рекомендуется, когда это возможно, инициализировать все элементы данных внутриклассовыми инициализаторами и позволить

¹ C++ Core Guidelines, «C.48: Prefer In-Class Initializers to Member Initializers in Constructors for Constant Initializers». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rc-in-class-initializer>.

компилятору создать конструктор по умолчанию. Конструктор по умолчанию, созданный компилятором, может быть более эффективным, чем тот, который вы напишете¹.

Директива `#pragma once`

Большие программы обычно содержат много заголовков, и каждый из них, в свою очередь, может содержать другие заголовки с определениями и объявлениями. Поэтому после обработки всего исходного кода программы препроцессором очень вероятны случайные повторные включения в нее одного и того же заголовка. Если в программе окажется несколько одинаковых определений, это может привести к ошибкам компиляции. Директива `#pragma once` (строка 4) предотвращает включение заголовка `Time.h` в файл исходного кода более одного раза. Это иногда называют **защитой включения заголовков** (**include guard**). В главе 16 мы обсудим другой способ предотвращения таких проблем: с помощью модулей (C++20).

9.7.4. Функции класса `Time`

В файле `Time.cpp` (илл. 9.8) определены функции класса `Time`, объявленные в строках 10–12 заголовка `Time.h` (илл. 9.7). Для функций `to24HourString` и `to12HourString` ключевое слово `const` должно присутствовать как в прототипах функций (илл. 9.7, строки 11–12), так и в определениях функций (илл. 9.8, строки 23 и 29).

```

1 // Fig. 9.8: Time.cpp
2 // Определения функций класса Time
3 #include <fmt/format.h>
4 #include <stdexcept> // Для класса исключений invalid_argument
5 #include <string>
6 #include "Time.h" // Определение класса Time
7
8 // Устанавливаем новое значение времени, используя 24-часовой формат
9 void Time::setTime(int hour, int minute, int second) {
10     // Проверяем часы, минуты, секунды
11     if ((hour < 0 || hour >= 24) || (minute < 0 || minute >= 60) ||
12         (second < 0 || second >= 60)) {
13         throw std::invalid_argument{
14             "hour, minute or second was out of range"};
15     }
16
17     m_hour = hour;
18     m_minute = minute;
19     m_second = second;
20 }
21
```

¹ C++ Core Guidelines, «C.45: Don't Define a Default Constructor That Only Initializes Data Members; Use In-Class Member Initializers Instead». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rc-default>.

```
22 // Возвращаем время в 24-часовом формате (HH:MM:SS)
23 std::string Time::to24HourString() const {
24     return fmt::format("{}:{:02d}:{:02d}", m_hour, m_minute, m_second);
25 }
26
27 // Возвращаем время в 12-часовом формате (HH:MM:SS AM или PM)
28 std::string Time::to12HourString() const {
29     return fmt::format("{}:{:02d}:{:02d} {}",
30         ((m_hour % 12 == 0) ? 12 : m_hour % 12), m_minute, m_second,
31         (m_hour < 12 ? "AM" : "PM"));
32 }
```

Илл. 9.8 | Определения функций класса Time

9.7.5. Включение заголовка класса в файл исходного кода

Чтобы указать, что функции в файле `Time.cpp` являются частью класса `Time`, мы должны включить в этот файл заголовок `Time.h` (илл. 9.8, строка 6). Это позволяет нам использовать имя класса `Time` в файле `Time.cpp` (строки 9, 23 и 28). При компиляции файла `Time.cpp` компилятор использует информацию из `Time.h`, чтобы гарантировать, что:

- ♦ первая строка каждой функции класса соответствует ее прототипу в файле `Time.h`;
- ♦ каждая функция класса знает об элементах данных класса и о других функциях класса.

9.7.6. Оператор разрешения области видимости (::)

Имя каждой функции (строки 9, 23 и 28) начинается с имени класса и оператора `::` (двойного двоеточия), который называется **оператором разрешения области видимости** (**scope resolution operator**). Это «привязывает» функции к определению класса `Time`, которое теперь находится в отдельном файле (илл. 9.7). Префикс `Time::` перед именем функции сообщает компилятору, что функция находится в области видимости класса `Time` и ее имя известно другим элементам класса.

Без префикса `Time::` перед именем каждой функции компилятор рассматривал бы ее как глобальную функцию, не имеющую отношения к классу `Time`. Такие функции, иногда называемые **свободными функциями** (**free functions**), не могут получать доступ к скрытым данным классов или вызывать функции классов без указания объекта. Таким образом, без префикса `Time::` компилятор не смог бы скомпилировать эти функции из-за отсутствия информации о том, что в классе `Time` объявлены переменные `m_hour`, `m_minute` и `m_second`. Возникли бы ошибки компиляции в строках 17–19, 24 и 30–31 (илл. 9.8), поскольку `m_hour`, `m_minute` и `m_second` не объявлены ни как локальные переменные в функциях, ни как глобальные переменные.

9.7.7. Функция setTime и исключения

Функция `setTime` (строки 9–20) — это открытая функция, которая объявляет три параметра `int` и использует их для установки времени. В строках 11–12 все три аргумента проверяются на соответствие допустимым диапазонам. Если условие выполняется, в строках 17–19 значения аргументов присваиваются элементам данных `m_hour`, `m_minute` и `m_second` соответственно. Аргумент `hour` должен быть больше или равен 0 и меньше 24, поскольку 24-часовой формат представляет часы в виде целых чисел от 0 до 23. Аналогично аргументы `minute` и `second` должны быть больше или равны 0 и меньше 60.

Если какой-либо из аргументов выходит за пределы допустимого диапазона, в строках 13–14 функция `setTime` выдает исключение типа `invalid_argument` (заголовок `<stdexcept>`), сообщая клиентскому коду о недопустимом значении аргумента. Как было показано в разделе 6.15, для перехвата исключений и продолжения работы программы можно использовать инструкции `try...catch`, что мы и делаем в следующем примере (илл. 9.9). Инструкция `throw` создает новый объект исключения типа `invalid_argument` и инициализирует его нашей строкой с сообщением об ошибке. После создания объекта исключения инструкция `throw` завершает работу функции `setTime`. Затем исключение возвращается в код, вызвавший функцию `setTime`.

Итак, объект класса `Time` не может содержать недопустимые значения времени, потому что:

- ♦ при создании объекта `Time` вызывается его конструктор по умолчанию и все элементы данных инициализируются числом 0, как указано в строках 14–16 (илл. 9.7), что означает полночь (12 AM);
- ♦ все последующие попытки клиентского кода изменить элементы данных класса контролируются функцией `setTime`.

9.7.8. Функции to24HourString и to12HourString

Функция `to24HourString` (строки 23–25 на илл. 9.8) не принимает аргументы и возвращает строку, в которой время записано в 24-часовом формате, например "13:30:07". Каждый заполнитель `{:02d}` форматирует целое число (`d`) в поле шириной 2 символа (строка 24). Цифра 0 в этом заполнителе сообщает компилятору, что перед каждым значением, содержащим менее двух цифр, надо добавить ноль.

Функция `to12HourString` (строки 28–32) не принимает аргументы и возвращает строку, в которой время записано в 12-часовом формате, например "10:54:27 AM" (AM — до полудня) или "1:27:06 PM" (PM — после полудня). Заполнитель `{:02d}` форматирует значения минут и секунд (`m_minute` и `m_second`) в поле шириной 2 символа, добавляя при необходимости начальный ноль. В строке 30 условный оператор `?:` используется для форматирования значения часов (`m_hour`). Если `m_hour` равно 0 (полночь) или 12 (полдень), отображается число 12; в противном случае мы используем оператор остатка от деления `%`, чтобы получить число от

1 до 11. Условный оператор в строке 31 определяет, что надо добавить в конце: AM или PM.

9.7.9. Неявное встраивание функций

Если функция полностью определена в теле класса (как в наших примерах с классом Account), компилятор, скорее всего, неявно объявит ее как встроенную, или inline-функцию (раздел 5.11). В большинстве случаев это ускоряет работу программы. Но помните, что компилятор может и не делать этого. Оптимизирующий компилятор оставляет за собой право (но не обязан) встраивать даже те функции, которые вы объявили без ключевого слова `inline`, если у него есть доступ к их определениям.

В заголовке класса можно определять только самые простые и стабильные функции, реализация которых вряд ли изменится, а также те, которые должны работать максимально быстро. Дело в том, что каждое изменение в заголовке требует перекомпиляции всех файлов исходного кода, зависящих от этого заголовка. Это занимает очень много времени, если вы пишете большую программу.

9.7.10. Функции класса в сравнении с глобальными функциями

Функции класса `to24HourString` и `to12HourString` не принимают аргументы. Они неявно знают об элементах данных объекта `Time`, из которого вызываются, и могут получать к ним доступ. Это ценное преимущество объектно-ориентированного программирования: обычно функции классов не принимают аргументы или принимают меньше аргументов, чем функции в программах, которые не являются объектно-ориентированными. Это снижает вероятность передачи неправильных аргументов, неправильного количества аргументов или аргументов в неправильном порядке.

9.7.11. Использование класса Time

Когда класс `Time` определен, его можно использовать в качестве типа данных в объявлениях:

```
Time sunset{}; // Объект класса Time (тип объекта - Time)
std::array<Time, 5> arrayOfTimes{}; // Массив из 5 объектов Time
Time& dinnerTimeRef{sunset}; // Ссылка на объект Time
Time* timePtr{&sunset}; // Указатель на объект Time
```

На илл. 9.9 показан пример создания и использования объекта класса `Time`. Отделение интерфейса класса `Time` от реализации его функций не влияет на использование класса клиентским кодом. В строке 8 включен заголовок `Time.h`, поэтому компилятор знает, сколько памяти надо зарезервировать для объекта `t` (строка 17), и может гарантировать, что объекты класса `Time` создаются и обрабатываются в клиентском коде корректно.

```

1 // fig09_09.cpp
2 // Программа для тестирования класса Time
3 // Внимание: этот файл должен быть скомпилирован с Time.cpp
4 #include <fmt/format.h>
5 #include <iostream>
6 #include <stdexcept> // Для класса исключений invalid_argument
7 #include <string_view>
8 #include "Time.h" // Определение класса Time в файле Time.h
9
10 // Отображаем время в 24-часовом и 12-часовом форматах
11 void displayTime(std::string_view message, const Time& time) {
12     std::cout << fmt::format("{}\n24-hour time: {}\n12-hour time: {}\n\n",
13         message, time.to24HourString(), time.to12HourString());
14 }
15
16 int main() {
17     Time t{}; // Создаем объект t класса Time
18
19     displayTime("Initial time:", t); // Отображаем начальное значение t
20     t.setTime(13, 27, 6); // Изменяем время
21     displayTime("After setTime:", t); // Отображаем новое значение t
22
23     // Попробуем установить недопустимое время
24     try {
25         t.setTime(99, 99, 99); // Все три значения ошибочны!
26     }
27     catch (const std::invalid_argument& e) {
28         std::cout << fmt::format("Exception: {}\n\n", e.what());
29     }
30
31     // Отображаем значение t после попытки установить недопустимое время
32     displayTime("After attempting to set an invalid time:", t);
33 }

```

```

Initial time:
24-hour time: 00:00:00
12-hour time: 12:00:00 AM

```

```

After setTime:
24-hour time: 13:27:06
12-hour time: 1:27:06 PM

```

```

Exception: hour, minute and/or second was out of range

```

```

After attempting to set an invalid time:
24-hour time: 13:27:06
12-hour time: 1:27:06 PM

```

Илл. 9.9 | Программа для тестирования класса Time

На протяжении всей программы мы отображаем данные объекта `Time`, используя функцию `displayTime` (строки 11–14), которая вызывает функции класса `Time`:

`to24HourString` и `to12HourString`. В строке 17 создается объект `t`. Напомним, что в классе `Time` не определен пользовательский конструктор, поэтому вызывается конструктор по умолчанию, созданный компилятором. Таким образом, переменным `m_hour`, `m_minute` и `m_second` присваивается значение 0 через их инициализаторы в определении класса `Time`. В строке 19 мы отображаем время в 24-часовом и 12-часовом форматах, чтобы подтвердить правильность инициализации элементов. В строке 20 мы устанавливаем новое время, вызывая функцию `setTime`, а в строке 21 снова отображаем время в обоих форматах.

Вызов функции `setTime` с недопустимыми значениями

Чтобы подтвердить проверку аргументов функцией `setTime`, мы вызываем ее в строке 25 с недопустимыми аргументами 99 для всех трех параметров: `hour`, `minute` и `second`. Мы поместили эту инструкцию в блок `try` (строки 24–26), чтобы перехватывать исключение `invalid_argument`, выдаваемое функцией `setTime`. Когда возникает исключение, оно перехватывается в строках 27–29, а в строке 28 отображается сообщение об ошибке, полученное с помощью функции `what` класса `invalid_argument`. В строке 32 мы снова отображаем время, подтверждая, что вызов функции с ошибочными аргументами не повредил сохраненные в объекте данные.

9.7.12. Размер объекта

Новички в объектно-ориентированном программировании часто думают, что объекты должны быть довольно большими, поскольку они содержат элементы данных и функции класса. Это может показаться логичным. Вы можете представлять себе, что элементы данных и функции хранятся внутри каждого объекта (и наше обсуждение, безусловно, способствует подобному взгляду). Но на самом деле это не так.

Созданный в памяти компьютера объект содержит только элементы данных, но не функции класса. Код функций класса хранится в памяти отдельно от всех объектов класса. Каждому объекту нужны собственные элементы данных, потому что они должны быть независимыми от других объектов. Но код функций идентичен для всех объектов класса, поэтому нет необходимости повторять его в каждом объекте.

9.8. Процесс компиляции и компоновки программы

Часто интерфейс и реализация класса создаются одним программистом, а использует их другой программист, пишущий клиентский код. Например, **программист-разработчик класса** (**class-implementation programmer**), ответственный за создание повторно используемого класса `Time`, пишет заголовок `Time.h` и файл исходного кода `Time.cpp`, содержащий этот заголовок. Затем он передает оба файла программисту клиентского кода. Исходный код повторно используемого класса

часто доступен программистам клиентского кода в виде библиотеки, которую они могут загрузить с веб-сайта, например с портала GitHub.

Для использования класса `Time` программист клиентского кода должен знать только интерфейс этого класса, а также иметь возможность скомпилировать файл `Time.cpp` и затем **скомпоновать (link)** готовое приложение. Поскольку интерфейс класса является частью определения класса в заголовке `Time.h`, программист клиентского кода должен включить этот заголовок в свой файл. Определение класса в заголовке `Time.h` позволяет компилятору убедиться, что клиентский код правильно создает и использует объекты класса `Time`.

В результате компиляции файлов исходного кода создаются файлы **объектного кода (object code)**, из которых **компоновщик (linker)** создает исполняемый файл, то есть готовое приложение. Чтобы получить самое простое приложение с классом `Time`, надо скомпоновать в единое целое:

- ♦ объектный код функции `main` (в которой находится клиентский код);
- ♦ объектный код функций класса `Time`;
- ♦ объектный код тех классов стандартной библиотеки C++ (например, `std::string`), которые использовались программистом класса и программистом клиентского кода.

Результатом компоновки всех компонентов программы из раздела 9.7 является готовое приложение, которое можно запускать для демонстрации работы класса `Time`. Как правило, компилятор или интегрированная среда разработки (IDE) после компиляции исходного кода автоматически вызывают компоновщик, создающий готовое приложение.

Компиляция программ, содержащих более одного файла исходного кода

В разделе 1.2 мы показали, как скомпилировать и запустить приложение, содержащее только один файл исходного кода (`.cpp`). Чтобы скомпилировать и скомпоновать приложение, содержащее несколько файлов исходного кода, выполните следующие действия¹:

- ♦ Если вы работаете в Microsoft Visual Studio, добавьте в свой проект (как показано в разделе 1.2.1) все свои заголовки и файлы исходного кода, из которых состоит программа, затем создайте и запустите проект. Вы можете поместить заголовки в папку **Header Files** проекта, а файлы исходного кода — в папку **Source Files** проекта, но такое разделение файлов характерно только для больших проектов. Программы будут компилироваться и в том случае, если вы поместите все файлы в папку **Source Files**.
- ♦ Если вы работаете в командной строке с компилятором `g++` или `clang++`, перейдите в каталог, содержащий все файлы вашей программы. Затем в команде компиляции перечислите имена всех файлов `.cpp` или используйте параметр `*.cpp` для компиляции всех имеющихся в этом каталоге файлов

¹ В главе 16 мы рассмотрим другой вариант, основанный на модулях C++20.

с расширением `.cpp`. Препроцессор автоматически найдет в этом каталоге все заголовки, относящиеся к вашей программе.

- ♦ Если вы работаете в Apple Xcode, добавьте в свой проект (как показано в разделе 1.2.2) все заголовки и файлы исходного кода, из которых состоит программа, затем создайте и запустите проект.

9.9. Область видимости класса и доступ к элементам класса

Элементы данных и функции класса находятся в области видимости этого класса. Функции, не являющиеся элементами классов (например, `main`), по умолчанию определяются в глобальной области видимости. В пределах области видимости класса для обращения к любому из его элементов достаточно указать его имя, поскольку каждый элемент непосредственно доступен всем функциям класса. Но для обращения к любому элементу за пределами области видимости класса, то есть к открытому (`public`) элементу другого класса, надо перед именем элемента указать **дескриптор (handle)** объекта, содержащего этот элемент. Дескриптором может быть:

- ♦ имя объекта;
- ♦ ссылка на объект;
- ♦ указатель на объект;
- ♦ указатель на элемент класса.

Тип дескриптора помогает компилятору идентифицировать интерфейс (то есть набор функций), доступный клиентам класса через этот дескриптор. В разделе 9.19 вы увидите, что компилятор автоматически добавляет неявный дескриптор (указатель `this`) каждый раз, когда вы обращаетесь из объекта к его элементу данных или к функции его класса.

Операторы «точка» (.) и «стрелка» (->)

Как вы уже знаете, для обращения к элементу объекта можно добавить после имени объекта или ссылки на объект оператор «точка» (`.`), а затем имя элемента. Похожим образом вы можете обращаться к элементу и через указатель на объект. Для этого добавьте после имени указателя оператор «стрелка» (`->`), а затем имя элемента: *ИмяУказателя->ИмяЭлемента*.

Доступ к открытым элементам класса через объекты, ссылки и указатели

Рассмотрим класс `Account`, в котором есть открытая функция `deposit`. Предположим, в нем есть такие объявления:

```
Account account{}; // Объект класса Account
Account& ref{account}; // Ссылка на объект класса Account
Account* ptr{&account}; // Указатель на объект класса Account
```

Вы можете вызвать функцию `deposit`, используя операторы «точка» (.) и «стрелка» (->):

```
account.deposit(123.45); // Вызов функции deposit через имя объекта
ref.deposit(123.45); // Вызов функции deposit через ссылку на объект
ptr->deposit(123.45); // Вызов функции deposit через указатель на объект
```

Напомним, что вы должны использовать ссылки вместо указателей, когда это возможно. Мы продолжим рассматривать указатели в тех случаях, когда они необходимы, а также для того, чтобы вы умели работать с ними, ведь они могут встретиться вам в устаревшем коде.

9.10. Функции доступа и служебные функции

Функции доступа

Функции доступа (access functions) могут получать или отображать данные, но не могут изменять их. Кроме того, они могут проверять различные условия, возвращая `true` или `false`. Такие функции часто называют **предикатными функциями (predicate functions)**. Примером может служить функция `empty` класса `std::array` или `std::vector`. Программа вызывает функцию `empty` перед попыткой прочитать элементы из контейнера¹.

Служебные функции

Служебные функции (utility functions), также называемые **вспомогательными функциями (helper functions)**, — это скрытые (`private`) функции, которые поддерживают работу других функций класса и не предназначены для использования клиентами класса. Как правило, служебная функция содержит код, который при ее отсутствии пришлось бы дублировать в основных функциях класса. Во многих кодовых базах существуют соглашения об именовании скрытых функций класса; например, их имена могут начинаться с символа подчеркивания (`_`).

9.11. Конструкторы с аргументами по умолчанию

На илл. 9.10–9.12 показан усовершенствованный код класса `Time`, содержащий конструктор с аргументами по умолчанию.

9.11.1. Усовершенствованный класс `Time`

Как и другие функции, конструкторы могут иметь аргументы по умолчанию. В строке 11 на илл. 9.10 объявляется конструктор класса `Time`, задающий для

¹ Многие программисты начинают имена предикатных функций со слова «is». Например, полезными предикатными функциями нашего класса `Time` могут быть `isAM` и `isPM`. Таким функциям должен предшествовать атрибут `[[nodiscard]]`, требующий от компилятора убедиться, что возвращаемое значение используется в вызывающем коде, а не игнорируется.

каждого параметра аргумент по умолчанию 0. Конструктор, в котором для всех параметров есть аргументы по умолчанию, является конструктором по умолчанию — он может быть вызван без аргументов¹. В каждом классе может быть только один конструктор по умолчанию. Если изменилось значение по умолчанию какого-либо аргумента, после этого необходимо заново скомпилировать весь клиентский код, чтобы программа по-прежнему работала правильно.

```
1 // Fig. 9.10: Time.h
2 // Класс Time, содержащий конструктор с аргументами по умолчанию
3 // Функции класса определены в Time.cpp
4 #pragma once // Предотвращаем повторные включения заголовка
5 #include <string>
6
7 // Определение класса Time
8 class Time {
9 public:
10    // Это конструктор по умолчанию, так как он может быть вызван без аргументов
11    explicit Time(int hour = 0, int minute = 0, int second = 0);
12
13    // Set-функции (для изменения данных)
14    void setTime(int hour, int minute, int second);
15    void setHour(int hour); // Устанавливаем hour (после проверки)
16    void setMinute(int minute); // Устанавливаем minute (после проверки)
17    void setSecond(int second); // Устанавливаем second (после проверки)
18
19    // Get-функции (для получения данных)
20    int getHour() const; // Возвращаем hour
21    int getMinute() const; // Возвращаем minute
22    int getSecond() const; // Возвращаем second
23
24    std::string to24HourString() const; // 24-часовой формат строки
25    std::string to12HourString() const; // 12-часовой формат строки
26 private:
27    int m_hour{0}; // 0 - 23 (24-часовой формат часов)
28    int m_minute{0}; // 0 - 59
29    int m_second{0}; // 0 - 59
30 };
```

Илл. 9.10 | Класс Time, содержащий конструктор с аргументами по умолчанию

Конструктор класса Time

На илл. 9.11 в строках 9–11 определяется конструктор класса Time, который вызывает функцию setTime для проверки и изменения значений элементов данных. Функция setTime (строки 14–31) гарантирует, что hour находится в диапазоне 0-23, а minute и second — в диапазоне 0-59. Если какой-либо аргумент находится

¹ Далее вы увидите, что концепты (C++20, глава 15) позволяют вам перегружать любую функцию, в том числе функцию класса, для использования с типами, которые соответствуют вашим требованиям.

вне допустимого диапазона, функция `setTime` выдаст исключение и в программе не будет объекта `Time`, потому что конструктор не завершит его создание. В эту версию функции `setTime` мы включили отдельные инструкции `if`, чтобы проверять каждый аргумент и выдавать точные сообщения об ошибке, объясняющие, какой из аргументов находится вне допустимого диапазона. Каждая из функций `setHour`, `setMinute` и `setSecond` (строки 34–40) вызывает функцию `setTime`, передавая ей свой аргумент и текущие значения двух других элементов данных. Например, функция `setHour` передает свой аргумент `hour` для изменения `m_hour`, а также текущие значения `m_minute` и `m_second`.

```
1 // Fig. 9.11: Time.cpp
2 // Определения функций класса Time
3 #include <fmt/format.h>
4 #include <stdexcept>
5 #include <string>
6 #include "Time.h" // Определение класса Time
7
8 // Конструктор класса Time инициализирует каждый элемент данных
9 Time::Time(int hour, int minute, int second) {
10     setTime(hour, minute, second);
11 }
12
13 // Устанавливаем новые значения, используя 24-часовой формат
14 void Time::setTime(int hour, int minute, int second) {
15     // Проверяем hour, minute и second
16     if (hour < 0 || hour >= 24) {
17         throw std::invalid_argument{"hour was out of range"};
18     }
19
20     if (minute < 0 || minute >= 60) {
21         throw std::invalid_argument{"minute was out of range"};
22     }
23
24     if (second < 0 || second >= 60) {
25         throw std::invalid_argument{"second was out of range"};
26     }
27
28     m_hour = hour;
29     m_minute = minute;
30     m_second = second;
31 }
32
33 // Устанавливаем значение hour
34 void Time::setHour(int hour) {setTime(hour, m_minute, m_second);}
35
36 // Устанавливаем значение minute
37 void Time::setMinute(int minute) {setTime(m_hour, minute, m_second);}
38
39 // Устанавливаем значение second
40 void Time::setSecond(int second) {setTime(m_hour, m_minute, second);}
41
```

```
42 // Возвращаем значение hour
43 int Time::getHour() const {return m_hour;}
44
45 // Возвращаем значение minute
46 int Time::getMinute() const {return m_minute;}
47
48 // Возвращаем значение second
49 int Time::getSecond() const {return m_second;}
50
51 // Возвращаем время в 24-часовом формате (HH:MM:SS)
52 std::string Time::to24HourString() const {
53     return fmt::format("{:02d}:{:02d}:{:02d}",
54         getHour(), getMinute(), getSecond());
55 }
56
57 // Возвращаем время в 12-часовом формате (HH:MM:SS AM или PM)
58 std::string Time::to12HourString() const {
59     return fmt::format("{:02d}:{:02d} {}",
60         (getHour() % 12 == 0) ? 12 : getHour() % 12,
61         getMinute(), getSecond(), (getHour() < 12 ? "AM" : "PM"));
62 }
```

Илл. 9.11 | Определения функций класса Time

В C++ Core Guidelines есть много рекомендаций по созданию конструкторов. Мы рассмотрим некоторые из них в следующих двух главах. Если класс содержит **инвариант класса (class invariant)**¹, то есть его элементы данных должны иметь заранее заданные значения или диапазоны значений (как в нашем классе Time), то в этом классе должен быть определен конструктор, который проверяет аргументы и в случае недопустимого значения выдает исключение, предотвращающее создание объекта^{2,3,4}.

Тестирование усовершенствованного класса Time

Функция main на илл. 9.12 инициализирует пять объектов класса Time:

- ✦ со всеми тремя аргументами по умолчанию, заданными неявным вызовом конструктора (строка 16);

¹ «Инвариант класса». Был доступен: 15 сентября 2023 г. https://ru.wikipedia.org/wiki/Инвариант_класса.

² C++ Core Guidelines, «C.40: Define a Constructor if a Class Has an Invariant». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rc-ctor>.

³ C++ Core Guidelines, «C.41: A Constructor Should Create a Fully Initialized Object». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rc-complete>.

⁴ C++ Core Guidelines, «C.42: If a Constructor Cannot Construct a Valid Object, Throw an Exception». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rc-throw>.

- ♦ с одним аргументом (строка 17);
- ♦ с двумя аргументами (строка 18);
- ♦ с тремя аргументами (строка 19);
- ♦ с тремя недопустимыми аргументами (строка 29).

Программа отображает данные каждого объекта в 24-часовом и 12-часовом форматах. Для объекта `t5` (строка 29) программа выводит сообщение об ошибке, поскольку аргументы конструктора находятся вне допустимого диапазона. Переменная `t5` никогда не будет представлять полностью созданный объект в этой программе, потому что исключение выдается во время выполнения конструктора.

```

1 // fig09_12.cpp
2 // Конструктор с аргументами по умолчанию
3 #include <fmt/format.h>
4 #include <iostream>
5 #include <stdexcept>
6 #include <string>
7 #include "Time.h" // Определение класса Time в файле Time.h
8
9 // Отображаем время в 24-часовом и 12-часовом форматах
10 void displayTime(std::string_view message, const Time& time) {
11     std::cout << fmt::format("{}\n24-hour time: {}\n12-hour time: {}\n\n",
12         message, time.to24HourString(), time.to12HourString());
13 }
14
15 int main() {
16     const Time t1{}; // Все аргументы по умолчанию
17     const Time t2{2}; // Указываем hour; minute и second - по умолчанию
18     const Time t3{21, 34}; // Указываем hour и minute; second - по умолчанию
19     const Time t4{12, 25, 42}; // Указываем все три аргумента
20
21     std::cout << "Constructed with:\n\n";
22     displayTime("t1: all arguments defaulted", t1);
23     displayTime("t2: hour specified; minute and second defaulted", t2);
24     displayTime("t3: hour and minute specified; second defaulted", t3);
25     displayTime("t4: hour, minute and second specified", t4);
26
27     // Пробуем создать объект t5 с недопустимыми аргументами
28     try {
29         const Time t5{27, 74, 99}; // Все аргументы ошибочны!
30     }
31     catch (const std::invalid_argument& e) {
32         std::cerr << fmt::format("t5 not created: {}\n", e.what());
33     }
34 }

```

Constructed with:

```

t1: all arguments defaulted
24-hour time: 00:00:00

```

```
12-hour time: 12:00:00 AM
t2: hour specified; minute and second defaulted
24-hour time: 02:00:00
12-hour time: 2:00:00 AM

t3: hour and minute specified; second defaulted
24-hour time: 21:34:00
12-hour time: 9:34:00 PM

t4: hour, minute and second specified
24-hour time: 12:25:42
12-hour time: 12:25:42 PM

t5 not created: hour was out of range
```

Илл. 9.12 | Конструктор с аргументами по умолчанию

Рекомендации разработчикам на примере *get*-функций, *set*-функций и конструктора класса *Time*

В теле класса *Time* неоднократно вызываются его *set*-функции и *get*-функции. В частности, конструктор (илл. 9.11, строки 9–11) вызывает *setTime*, а функции *to24HourString* и *to12HourString* вызывают функции *getHour*, *getMinute* и *getSecond* (строки 54 и 60–61). Во всех этих случаях мы получаем прямой доступ к скрытым (*private*) данным класса.

Во внутреннем представлении класса мы храним время в трех числах *int*, занимающих 12 байт памяти в системах с 4-байтовыми числами этого типа. Предположим, нам нужно представить время иначе: в виде количества прошедших с полуночи секунд, чтобы обойтись всего одним числом *int*, занимающим 4 байта. Для этого достаточно изменить только тела функций, которые напрямую обращаются к скрытым элементам данных. В этом классе изменения затронут только тела функции *setTime*, а также *set*-функций и *get*-функций для *m_hour*, *m_minute* и *m_second*. При этом не придется изменять ни конструктор класса, ни функции *24hourString* и *12hourString*, поскольку они не обращаются к данным напрямую.

Дублирование инструкций в нескольких функциях или конструкторах затрудняет модификацию представления данных внутри класса. Та реализация конструктора класса *Time* и функций *to24HourString* и *to12HourString*, которая показана в этом примере, снижает вероятность ошибок при изменении реализации класса.

У опытных программистов есть важный принцип: надо избегать повторения фрагментов кода. Этот принцип называется «Don't repeat yourself» («Не повторяйтесь»)¹. Если какой-то фрагмент кода должен выполняться в нескольких местах программы, надо поместить его в функцию, которая может быть вызвана конструктором

¹ «Don't Repeat Yourself». Был доступен: 15 сентября 2023 г. https://ru.wikipedia.org/wiki/Don't_repeat_yourself.

класса и другими функциями. Это упрощает тестирование и модификацию кода, а также снижает вероятность ошибок при внесении любых изменений.

Конструктор может вызывать другие функции класса, но при этом нужна осторожность. Конструктор инициализирует объект, поэтому элементы данных, используемые в вызываемой функции, могут оказаться еще не инициализированными. Обращения к элементам данных до их инициализации приводят к логическим ошибкам.

Объявление элементов данных скрытыми (`private`) и последующий контроль доступа к ним (особенно перед изменением данных) с помощью открытых функций помогают обеспечить целостность данных. Все преимущества целостности данных не появятся сами собой, если вы просто объявите элементы данных скрытыми. Вы должны обеспечить еще и проверку корректности всех значений.

9.11.2. Перегрузка конструкторов и делегирование конструкторов (C++11)

В разделе 5.15 мы рассматривали перегруженные функции. Конструкторы и другие функции класса тоже можно перегружать. Перегруженные конструкторы позволяют инициализировать объекты с разными типами аргументов и (или) с разным количеством аргументов. Чтобы перегрузить конструктор, напишите прототип и определение для каждой перегруженной версии. Это также относится к перегруженным функциям класса.

На илл. 9.10–9.12 конструктор класса `Time` имел аргументы по умолчанию для всех параметров. Вместо этого конструктора мы могли бы определить четыре перегруженных конструктора со следующими прототипами:

```
Time(); // Аргументы по умолчанию для m_hour, m_minute и m_second: 0
explicit Time(int hour); // Аргументы по умолчанию для m_minute и m_second: 0
Time(int hour, int minute); // Аргументы по умолчанию для m_second: 0
Time(int hour, int minute, int second); // Нет аргументов по умолчанию
```

Конструктор может вызывать для выполнения своих задач не только другие функции класса, но и другие конструкторы своего класса. Вызывающий конструктор называется **делегирующим конструктором (delegating constructor)** (C++11), так как он делегирует свою работу другому конструктору. В C++ Core Guidelines рекомендуется определять общий код перегруженных конструкторов в одном конструкторе, а затем вызывать его из делегирующих конструкторов¹. До C++11 общий код помещали в служебную функцию, вызываемую всеми конструкторами.

Выше мы показали прототипы четырех перегруженных конструкторов класса `Time`. Первые три из них могут делегировать работу одному конструктору с тремя аргументами `int`, передавая 0 в качестве значения по умолчанию для отсутствующих

¹ C++ Core Guidelines, «C.51: Use Delegating Constructors to Represent Common Actions for all Constructors of a Class». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rc-delegating>.

параметров. Для этого можно использовать инициализатор элементов данных с именем класса:

```
Time::Time() : Time{0, 0, 0} {}  
Time::Time(int hour) : Time{hour, 0, 0} {}  
Time::Time(int hour, int minute) : Time{hour, minute, 0} {}
```

9.12. Деструкторы

Деструктор (destructor) — это специальная функция класса, у которой может не быть параметров или возвращаемого типа. Имя деструктора класса состоит из символа «тильда» (~) и последующего имени класса, например ~Time. Это соглашение об именовании легко запоминается, потому что тильда — это оператор побитового дополнения, как вы увидите в следующей главе. В каком-то смысле деструктор является дополнением конструктора.

Деструктор класса неявно вызывается, когда объект уничтожается. Обычно это происходит, когда поток управления программы покидает ту область видимости, в которой был создан объект. Сам деструктор не удаляет объект из памяти. Он выполняет **завершающую обработку (termination housekeeping)**, например закрытие файла и очистку других используемых объектом ресурсов, прежде чем занимаемая объектом память будет освобождена для последующего использования.

Мы еще не определяли ни один деструктор в наших классах. Тем не менее в каждом классе C++ обязательно есть один (и только один) деструктор. Если программист не определил деструктор явно, компилятор создает для класса деструктор по умолчанию, вызывающий деструкторы любых элементов данных класса¹. В главе 12 мы объясним, почему из деструкторов не должны выдаваться исключения.

9.13. Вызов конструкторов и деструкторов

Конструкторы неявно вызываются, когда объекты создаются, а деструкторы неявно вызываются, когда объекты выходят из области видимости. Порядок этих вызовов зависит от области видимости объектов. Обычно деструкторы вызываются в обратном порядке по сравнению с вызовами соответствующих конструкторов, но как вы увидите на илл. 9.13–9.15, глобальные и статические (**static**) объекты могут изменять порядок вызова деструкторов.

Конструкторы и деструкторы глобальных объектов

Конструкторы глобальных объектов, то есть объектов, определенных в глобальной области видимости (также называемой глобальным пространством имен), вызы-

¹ В главе 10 вы увидите, что деструктор по умолчанию также уничтожает объекты класса, созданные путем наследования.

ваются до выполнения любой другой функции в этом же файле исходного кода. Порядок выполнения конструкторов глобальных объектов между несколькими файлами не регламентируется, поэтому глобальные объекты в отдельных файлах не должны зависеть друг от друга. Когда функция `main` завершается инструкцией `return` или закрывающей фигурной скобкой, деструкторы таких объектов вызываются в обратном порядке по сравнению с вызовами соответствующих конструкторов. Для экстренного завершения программы в случае фатальной неустранимой ошибки часто используется функция `exit`, вызывающая деструкторы глобальных объектов¹. Функция `abort`, в отличие от `exit`, прерывает программу немедленно, не допуская даже выполнения деструкторов или какого-либо другого завершающего кода. Как правило, вызов функции `abort` означает аварийное прекращение работы программы².

Конструкторы и деструкторы нестатических локальных объектов

Конструктор нестатического локального объекта вызывается, когда поток управления программы достигает определения этого объекта. Соответствующий деструктор объекта вызывается, когда поток управления программы выходит из области видимости объекта, то есть когда блок, в котором определен этот объект, завершается (нормально или из-за исключения). Деструкторы локальных объектов не вызываются, если программа завершается вызовом функции `exit` или `abort`.

Конструкторы и деструкторы статических локальных объектов

Конструктор статического (`static`) локального объекта вызывается только один раз, когда поток управления программы впервые достигает определения этого объекта. Соответствующий деструктор объекта вызывается, когда завершается функция `main` или вызывается функция `exit`. Глобальные и статические объекты уничтожаются в обратном порядке по сравнению с их созданием. Деструкторы статических объектов не вызываются, если программа завершается вызовом функции `abort`.

Демонстрация порядка вызова конструкторов и деструкторов

На илл. 9.13–9.15 мы показываем порядок вызова конструкторов и деструкторов для глобальных, локальных и локальных статических объектов класса `CreateAndDestroy` (илл. 9.13 и 9.14). Этот пример предназначен исключительно для учебных целей.

В коде на илл. 9.13 объявляется класс `CreateAndDestroy`. В строках 13–14 объявляются элементы данных класса: целое число (`m_ID`) и строка (`m_message`), которые идентифицируют каждый объект в выходных данных программы.

¹ «`std::exit`». Был доступен: 15 сентября 2023 г. <https://en.cppreference.com/w/cpp/utility/program/exit>.

² «`std::abort`». Был доступен: 15 сентября 2023 г. <https://en.cppreference.com/w/cpp/utility/program/abort>.

```

1 // Fig. 9.13: CreateAndDestroy.h
2 // Определение класса CreateAndDestroy
3 // Функции класса определены в CreateAndDestroy.cpp
4 #pragma once // Предотвращаем повторные включения заголовка
5 #include <string>
6 #include <string_view>
7
8 class CreateAndDestroy {
9 public:
10     CreateAndDestroy(int ID, std::string_view message); // Конструктор
11     ~CreateAndDestroy(); // Деструктор
12 private:
13     int m_ID; // Идентификационный номер объекта
14     std::string m_message; // Описание объекта
15 };

```

Илл. 9.13 | Определение класса CreateAndDestroy

Конструкторы и деструкторы (илл. 9.14) выводят на экран сообщения, чтобы показать, в каком порядке они вызываются. В деструкторе условное выражение (строка 18) проверяет, имеет ли уничтожаемый объект значение `m_ID`, равное 1 или 6. Если условие истинно, выводится символ новой строки для наглядности выходных данных.

```

1 // Fig. 9.14: CreateAndDestroy.cpp
2 // Определения функций класса CreateAndDestroy
3 #include <fmt/format.h>
4 #include <iostream>
5 #include "CreateAndDestroy.h" // Определение класса CreateAndDestroy
6
7 // Конструктор задает идентификационный номер и описание объекта
8 CreateAndDestroy::CreateAndDestroy(int ID, std::string_view message)
9     : m_ID{ID}, m_message{message} {
10     std::cout << fmt::format("Object {} constructor runs {}\n",
11         m_ID, m_message);
12 }
13
14 // Деструктор
15 CreateAndDestroy::~~CreateAndDestroy() {
16     // Для наглядности выводим символ новой строки, если m_ID равно 1 или 6
17     std::cout << fmt::format("{}Object {} destructor runs {}\n",
18         (m_ID == 1 || m_ID == 6 ? "\n" : ""), m_ID, m_message);
19 }

```

Илл. 9.14 | Определения функций класса CreateAndDestroy

В коде на илл. 9.15 сначала определяется объект в глобальной области видимости (строка 9). Его конструктор вызывается перед выполнением любых инструкций в функции `main`, а его деструктор вызывается при завершении программы после деструкторов всех объектов с автоматической продолжительностью хранения.


```

1 // fig09_15.cpp
2 // Порядок вызова конструкторов
3 // и деструкторов
4 #include <iostream>
5 #include "CreateAndDestroy.h" // Определение класса CreateAndDestroy
6
7 void create(); // Прототип
8
9 const CreateAndDestroy first{1, "(global before main)"}; // Глобальный объект
10
11 int main() {
12     std::cout << "\nMAIN FUNCTION: EXECUTION BEGINS\n";
13     const CreateAndDestroy second{2, "(local in main)"};
14     static const CreateAndDestroy third{3, "(local static in main)"};
15
16     create(); // Вызываем функцию для создания объектов
17
18     std::cout << "\nMAIN FUNCTION: EXECUTION RESUMES\n";
19     const CreateAndDestroy fourth{4, "(local in main)"};
20     std::cout << "\nMAIN FUNCTION: EXECUTION ENDS\n";
21 }
22
23 // Функция для создания объектов
24 void create() {
25     std::cout << "\nCREATE FUNCTION: EXECUTION BEGINS\n";
26     const CreateAndDestroy fifth{5, "(local in create)"};
27     static const CreateAndDestroy sixth{6, "(local static in create)"};
28     const CreateAndDestroy seventh{7, "(local in create)"};
29     std::cout << "\nCREATE FUNCTION: EXECUTION ENDS\n";
30 }

```

Object 1 constructor runs (global before main)

MAIN FUNCTION: EXECUTION BEGINS

Object 2 constructor runs (local in main)

Object 3 constructor runs (local static in main)

CREATE FUNCTION: EXECUTION BEGINS

Object 5 constructor runs (local in create)

Object 6 constructor runs (local static in create)

Object 7 constructor runs (local in create)

CREATE FUNCTION: EXECUTION ENDS

Object 7 destructor runs (local in create)

Object 5 destructor runs (local in create)

MAIN FUNCTION: EXECUTION RESUMES

Object 4 constructor runs (local in main)

MAIN FUNCTION: EXECUTION ENDS

Object 4 destructor runs (local in main)

Object 2 destructor runs (local in main)

```
Object 6 destructor runs (local static in create)
Object 3 destructor runs (local static in main)
Object 1 destructor runs (global before main)
```

Илл. 9.15 | Порядок вызова конструкторов и деструкторов

Функция `main` (строки 11–21) определяет три объекта. Объекты `second` (строка 13) и `fourth` (строка 19) являются локальными, а объект `third` (строка 14) является статическим локальным. Конструктор каждого объекта вызывается, когда поток управления программы достигает точки, в которой этот объект определен. Когда поток управления программы достигает конца функции `main`, деструкторы объектов `fourth` и `second` вызываются в обратном порядке по сравнению с вызовами их конструкторов. Объект `third` является статическим, поэтому он существует до завершения программы. Деструктор объекта `third` вызывается перед деструктором глобального объекта `first`, но после уничтожения нестатических локальных объектов.

Функция `create` (строки 24–30) определяет три объекта: локальные автоматические объекты `fifth` (строка 26) и `seventh` (строка 28) и статический локальный объект `sixth` (строка 27). Когда функция `create` завершается, деструкторы объектов `seventh` и `fifth` вызываются в обратном порядке по сравнению с вызовами их конструкторов. Поскольку объект `sixth` является статическим, он существует до завершения программы. Деструктор объекта `sixth` вызывается перед деструкторами объектов `third` и `first`, но после уничтожения всех остальных нестатических объектов. В качестве упражнения добавьте еще один вызов функции `create` — вы увидите, что конструктор статического объекта `sixth` вызывается только один раз при первом вызове функции `create`.

9.14. Коварная ловушка: возврат ссылки или указателя на скрытый элемент данных

Ссылка на объект является псевдонимом для имени объекта, поэтому ее можно использовать в левой части инструкции присваивания. В этом контексте ссылка представляет собой вполне приемлемое значение *lvalue*, которое можно изменять.

Функция класса может возвращать ссылку на скрытый (`private`) элемент данных. Если возвращаемое значение является константной (`const`) ссылкой, как у функции `getName` нашего класса `Account` ранее в этой главе, эта ссылка является неизменяемым значением *lvalue* и не может быть использована для изменения данных. Но если возвращаемое значение является неконстантной ссылкой, могут возникнуть ошибки, которые трудно обнаружить.

На илл. 9.16–9.18 показан пример с упрощенным классом `Time`, демонстрирующий риск возврата ссылки на скрытый элемент данных. Функция `badSetHour` класса `Time` (объявленная на илл. 9.16 в строке 12 и определенная на илл. 9.17 в стро-

ках 30–33) возвращает ссылку `int&` на элемент данных `m_hour`. Такой возврат ссылки делает результат вызова функции `badSetHour` псевдонимом для скрытого элемента данных `hour`. Этот вызов функции можно использовать как скрытый элемент данных, в том числе как значение *value* в инструкции присваивания. Таким образом, клиенты класса могут по своему желанию перезаписывать скрытые данные класса. Аналогичная проблема возникла бы, если бы функция возвращала указатель на скрытый элемент данных.

```

1 // Fig. 9.16: Time.h
2 // Определение класса Time
3 // Функции класса определены в Time.cpp
4
5 // Предотвращаем повторные включения заголовка
6 #pragma once
7
8 class Time {
9 public:
10     void setTime(int hour, int minute, int second);
11     int getHour() const;
12     int& badSetHour(int hour); // Опасный возврат ссылки!
13 private:
14     int m_hour{0};
15     int m_minute{0};
16     int m_second{0};
17 };

```

Илл. 9.16 | Определение класса `Time`

```

1 // Fig. 9.17: Time.cpp
2 // Определения функций класса Time
3 #include <stdexcept>
4 #include "Time.h" // Определение класса Time
5
6 // Установка нового значения времени в 24-часовом формате
7 void Time::setTime(int hour, int minute, int second) {
8     // Проверяем корректность hour, minute и second
9     if (hour < 0 || hour >= 24) {
10         throw std::invalid_argument{"hour was out of range"};
11     }
12
13     if (minute < 0 || minute >= 60) {
14         throw std::invalid_argument{"minute was out of range"};
15     }
16
17     if (second < 0 || second >= 60) {
18         throw std::invalid_argument{"second was out of range"};
19     }
20
21     m_hour = hour;
22     m_minute = minute;

```

```

23     m_second = second;
24 }
25
26 // Возвращаем значение hour
27 int Time::getHour() const {return m_hour;}
28
29 // Это неправильно: возврат ссылки на скрытый элемент данных!
30 int& Time::badSetHour(int hour) {
31     setTime(hour, m_minute, m_second);
32     return m_hour; // Опасный возврат ссылки!
33 }

```

Илл. 9.17 | Определения функций класса Time

В коде на илл. 9.18 объявляется объект `t` класса `Time` (строка 9) и ссылка `hourRef` (строка 12), которую мы инициализируем ссылкой, возвращенной в результате вызова `t.badSetHour(20)`. В строках 14–15 мы выводим на экран значение `hourRef`, демонстрируя нарушение инкапсуляции класса.

```

1 // fig09_18.cpp
2 // Открытая функция класса,
3 // возвращающая ссылку на скрытые данные
4 #include <iostream>
5 #include <fmt/format.h>
6 #include "Time.h" // Определение класса Time
7
8 int main() {
9     Time t{}; // Создаем объект класса Time
10
11     // Инициализируем hourRef ссылкой, которую возвратила функция badSetHour
12     int& hourRef{t.badSetHour(20)}; // 20 - это допустимое значение
13
14     std::cout << fmt::format(
15         "Valid hour before modification: {}\n", hourRef);
16     hourRef = 30; // Присваиваем элементу недопустимое значение!
17     std::cout << fmt::format(
18         "Invalid hour after modification: {}\n\n", t.getHour());
19
20     // Опасно: вызов функции, возвращающей ссылку, может использоваться
21     // как lvalue! ПЛОХОЙ СТИЛЬ ПРОГРАММИРОВАНИЯ!!!
22     t.badSetHour(12) = 74; // Присваиваем элементу недопустимое значение!
23
24     std::cout << "After using t.badSetHour(12) as an lvalue, "
25         << fmt::format("hour is: {}\n", t.getHour());
26 }

```

```

Valid hour before modification: 20
Invalid hour after modification: 30
After using t.badSetHour(12) as an lvalue, hour is: 74

```

Илл. 9.18 | Открытая функция класса, возвращающая ссылку на скрытые данные

Никакие инструкции в функции `main` не должны иметь доступа к скрытым данным в объекте класса `Time`. Инструкция в строке 16 присваивает элементу данных `hour` недопустимое значение 30 через ссылку `hourRef`. В строках 17–18 мы вызываем функцию `getHour`, чтобы показать, что эта инструкция изменила скрытый элемент данных объекта `t`. В строке 22 вызов функции `badSetHour` используется в качестве значения *value* и через возвращаемую этой функцией ссылку элементу данных `hour` присваивается недопустимое значение 74. В строках 24–25 мы снова вызываем функцию `getHour`, чтобы показать, что инструкция в строке 22 изменила скрытый элемент данных объекта `t`.

Возврат ссылки или указателя на скрытый (`private`) элемент данных нарушает инкапсуляцию класса, делая клиентский код зависимым от представления данных в классе. Лишь в некоторых случаях это может быть уместно. Мы рассмотрим один из таких случаев, когда напомним класс `myArray` в разделе 11.6.

9.15. Оператор присваивания по умолчанию

Оператор присваивания (`=`) может присвоить объект другому объекту того же типа. **Оператор присваивания по умолчанию (default assignment operator)**¹, сгенерированный компилятором, копирует каждый элемент данных из правого операнда в соответствующий элемент данных в левом операнде. На илл. 9.19 и 9.20 определяется класс `Date`. В строке 15 на илл. 9.21 объект `date1` класса `Date` присваивается объекту `date2` класса `Date`. Это означает, что элементы `m_year`, `m_month` и `m_day` объекта `date2` получают значения `m_year`, `m_month` и `m_day` соответственно, взятые из объекта `date1`.

```

1 // Fig. 9.19: Date.h
2 // Определение класса Date. Функции класса определены в Date.cpp
3 #pragma once // Предотвращаем повторные включения заголовка
4 #include <string>
5
6 // Определение класса Date
7 class Date {
8 public:
9     Date(int year, int month, int day);
10     std::string toString() const;
11 private:
12     int m_year;
13     int m_month;
14     int m_day;
15 };

```

Илл. 9.19 | Определение класса `Date`

¹ Точнее, это оператор копирующего присваивания по умолчанию. В главе 11 мы рассмотрим различие между оператором копирующего присваивания и оператором перемещающего присваивания.

```

1 // Fig. 9.20: Date.cpp
2 // Определения функций класса Date
3 #include <fmt/format.h>
4 #include <string>
5 #include "Date.h" // Определение класса Date
6
7 // Конструктор класса Date (должен проверять допустимость значений)
8 Date::Date(int year, int month, int day)
9     : m_year{year}, m_month{month}, m_day{day} {}
10
11 // Возвращаем строку с датой в формате yyyy-mm-dd
12 std::string Date::toString() const {
13     return fmt::format("{}-{:02d}-{:02d}", m_year, m_month, m_day);
14 }

```

Илл. 9.20 | Определения функций класса Date

```

1 // fig09_21.cpp
2 // Присваивание одного объекта другому
3 // с помощью оператора присваивания по умолчанию
4 #include <fmt/format.h>
5 #include <iostream>
6 #include "Date.h" // Определение класса Date
7 using namespace std;
8
9 int main() {
10     const Date date1{2006, 7, 4};
11     Date date2{2022, 1, 1};
12
13     std::cout << fmt::format("date1: {}\ndate2: {}\n\n",
14         date1.toString(), date2.toString());
15     date2 = date1; // Используем оператор присваивания по умолчанию
16     std::cout << fmt::format("After assignment, date2: {}\n",
17         date2.toString());
18 }

```

```

date1: 2006-07-04
date2: 2022-01-01
After assignment, date2: 2006-07-04

```

Илл. 9.21 | Присваивание одного объекта другому с помощью оператора присваивания по умолчанию

Копирующие конструкторы

Функции могут получать объекты в качестве аргументов, а также возвращать объекты. По умолчанию каждый объект при этом передается по значению, то есть функция передает или возвращает копию объекта. В таких случаях C++ создает новый объект и вызывает **копирующий конструктор (copy constructor)**, чтобы скопировать все данные из исходного объекта в новый. Для всех классов,

которые мы до сих пор рассматривали, компилятор предоставляет копирующий конструктор по умолчанию, который копирует каждый элемент из исходного объекта в соответствующий элемент нового объекта¹.

9.16. Константные объекты и константные функции класса

Рассмотрим применение принципа наименьших привилегий к объектам. Некоторые объекты никогда не должны изменяться, поэтому их надо объявлять как константные (с квалификатором `const`). Любая попытка изменить константный объект приводит к ошибке компиляции. Инструкция

```
const Time noon{12, 0, 0};
```

объявляет константный объект `noon` класса `Time` и инициализирует его значениями, соответствующими полудню (12 PM). Вы можете создавать константные и неконстантные объекты одного и того же класса.

C++ запрещает вызов функции константного объекта, если она сама не является константной. Поэтому объявляйте с квалификатором `const` каждую функцию, не изменяющую объект, для которого она вызывается.

Конструктор должен изменять объект в процессе инициализации. Деструктор должен выполнять завершающую обработку до того, как память объекта будет освобождена и передана операционной системе. Поэтому объявление конструктора или деструктора с квалификатором `const` приведет к ошибке компиляции. Неизменяемость каждого константного объекта обеспечивается с момента его создания на протяжении всего времени его существования.

Использование константных и неконстантных функций класса

Наша следующая программа (илл. 9.22) использует почти такой же класс `Time`, как на илл. 9.10 и 9.11, но с одним отличием: мы убрали квалификатор `const` в прототипе и в определении функции `to12HourString`, чтобы вызвать ошибку компиляции. Мы создаем два объекта класса `Time`: неконстантный объект `wakeup` (строка 6) и константный объект `noon` (строка 7). Программа пытается вызвать неконстантные функции для объекта `noon`: `setHour` (строка 11) и `to12HourString` (строка 15). В обоих случаях компилятор выдает сообщение об ошибке. Программа также иллюстрирует три другие комбинации объектов и функций:

- ✦ вызов неконстантной функции для неконстантного объекта (строка 10);
- ✦ вызов константной функции для неконстантного объекта (строка 12);
- ✦ вызов константной функции для константного объекта (строки 13–14).

¹ В главе 11 мы рассмотрим те случаи, когда компилятор заменяет копирующий конструктор перемещающим конструктором.

После кода программы показаны сообщения об ошибках, возникающих при вызове неконстантной функции для константного объекта. Мы добавили пустые строки для наглядности.

```
1 // fig09_22.cpp
2 // Константные объекты и константные функции класса
3 #include "Time.h" // Определение класса Time
4
5 int main() {
6     Time wakeUp{6, 45, 0}; // Неконстантный объект
7     const Time noon{12, 0, 0}; // Константный объект
8
9     // Сочетания объектов и функций
10    wakeUp.setHour(18); // Неконстантный объект, неконстантная функция
11    noon.setHour(12); // Константный объект, неконстантная функция!!!
12    wakeUp.getHour(); // Неконстантный объект, константная функция
13    noon.getMinute(); // Константный объект, константная функция
14    noon.to24HourString(); // Константный объект, константная функция
15    noon.to12HourString(); // Константный объект, неконстантная функция!!!
16 }
```

Сообщения компилятора clang++ об ошибках:

```
fig09_22.cpp:11:4: error: 'this' argument to member function 'setHour' has type
'const Time', but function is not marked const
    noon.setHour(12); // const non-const
    ^
./Time.h:15:9: note: 'setHour' declared here
    void setHour(int hour); // set hour (after validation)
    ^
fig09_22.cpp:15:4: error: 'this' argument to member function 'to12HourString'
has type 'const Time', but function is not marked const
    noon.to12HourString(); // const non-const
    ^
./Time.h:25:16: note: 'to12HourString' declared here
    std::string to12HourString(); // 12-hour time format string
    ^
2 errors generated.
```

Илл. 9.22 | Константные объекты и константные функции класса

Конструктор должен быть неконстантной функцией класса, но он может инициализировать константный объект (илл. 9.22, строка 7). Напомним, что в определении конструктора класса `Time` (илл. 9.11) вызывается неконстантная функция `setTime` для инициализации объекта класса `Time`. Вызов неконстантной функции конструктора для константного объекта разрешен. Объект не будет константным, пока конструктор не завершит его инициализацию.

В строке 15 на илл. 9.22 возникает ошибка компиляции, несмотря на то что функция `to12HourString` класса `Time` не изменяет объект, из которого она вызывается.

Этого факта недостаточно: вы должны явно объявить функцию с квалификатором `const`, чтобы такой вызов был разрешен компилятором.

9.17. Композиция: объекты как элементы классов

Предположим, что объект `AlarmClock` должен знать, в какое время надо подать сигнал будильника. Почему бы не включить в этот класс объект `Time` в качестве элемента? Такая возможность повторного использования кода называется **композицией** (**composition**), или **агрегацией** (**aggregation**). Элементами класса могут быть объекты других классов¹. Вы использовали композицию в примерах этой главы с классом `Account`, который содержал объект класса `string` в качестве элемента данных.

Вы уже знаете, как передавать аргументы конструктору. Теперь рассмотрим, как конструктор класса передает аргументы конструкторам объектов-элементов класса, используя инициализаторы. В следующей программе с классами `Date` (илл. 9.23 и 9.24) и `Employee` (илл. 9.25 и 9.26) мы покажем пример композиции. Класс `Employee` (илл. 9.25) содержит скрытые элементы данных `m_firstName`, `m_lastName`, `m_birthDate` и `m_hireDate`. Элементы этого класса `m_birthDate` и `m_hireDate` являются объектами класса `Date`, который содержит скрытые элементы данных `m_year`, `m_month` и `m_day`.

```

1 // Fig. 9.23: Date.h
2 // Определение класса Date; функции класса определены в Date.cpp
3 #pragma once // Предотвращаем повторные включения заголовка
4 #include <string>
5
6 class Date {
7 public:
8     static const int monthsPerYear{12}; // Месяцы в году
9     Date(int year, int month, int day);
10    std::string toString() const; // Дата в формате yyyy-mm-dd
11    ~Date(); // Реализация показывает, когда вызывается деструктор
12 private:
13     int m_year; // Любой год
14     int m_month; // 1-12 (месяц)
15     int m_day; // 1-31 в зависимости от месяца
16
17     // Служебная функция, проверяющая соответствие дня, месяца и года
18     bool checkDay(int day) const;
19 };

```

Илл. 9.23 | Определение класса `Date`

¹ Как вы увидите в главе 10, при создании класса можно скопировать его атрибуты и поведение из другого класса — это называется наследованием.

```

1 // Fig. 9.24: Date.cpp
2 // Определения функций класса Date
3 #include <array>
4 #include <fmt/format.h>
5 #include <iostream>
6 #include <stdexcept>
7 #include "Date.h" // Определение класса Date
8
9 // Конструктор проверяет допустимость значения month и вызывает
10 // служебную функцию checkDay для проверки допустимости значения day
11 Date::Date(int year, int month, int day)
12     : m_year{year}, m_month{month}, m_day{day} {
13     if (m_month < 1 || m_month > monthsPerYear) { // Проверяем значение month
14         throw std::invalid_argument{"month must be 1-12"};
15     }
16
17     if (!checkDay(day)) { // Проверяем значение day
18         throw std::invalid_argument{
19             "Invalid day for current month and year"};
20     }
21
22     // Сообщение показывает момент вызова конструктора объекта Date
23     std::cout << fmt::format("Date object constructor: {}\n", toString());
24 }
25
26 // Получаем дату в формате yyyy-mm-dd
27 std::string Date::toString() const {
28     return fmt::format("{}-{:02d}-{:02d}", m_year, m_month, m_day);
29 }
30
31 // Сообщение показывает момент вызова деструктора объекта Date
32 Date::~Date() {
33     std::cout << fmt::format("Date object destructor: {}\n", toString());
34 }
35
36 // Служебная функция, проверяющая допустимость значения day
37 // для заданных month и year (учитывает високосные годы)
38 bool Date::checkDay(int day) const {
39     // Элемент 0 игнорируется
40     static const std::array daysPerMonth{
41         0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
42
43     // Проверяем допустимость значения day для заданного month
44     if (1 <= day && day <= daysPerMonth.at(m_month)) {
45         return true;
46     }
47
48     // Проверка на високосный год (29 февраля)
49     if (m_month == 2 && day == 29 && (m_year % 400 == 0 ||
50         (m_year % 4 == 0 && m_year % 100 != 0))) {
51         return true;

```

```

52 }
53
54 return false; // Недопустимое значение day для заданных m_month и m_year
55 }

```

Илл. 9.24 | Определения функций класса Date

```

1 // Fig. 9.25: Employee.h
2 // Определение класса Employee, демонстрирующего композицию
3 // Функции класса определены в Employee.cpp
4 #pragma once // Предотвращаем повторные включения заголовка
5 #include <string>
6 #include <string_view>
7 #include "Date.h" // Определение класса Date
8
9 class Employee {
10 public:
11     Employee(std::string_view firstName, std::string_view lastName,
12             const Date& birthDate, const Date& hireDate);
13     std::string toString() const;
14     ~Employee(); // Для демонстрации порядка выполнения деструкторов
15 private:
16     std::string m_firstName; // Композиция: объект-элемент
17     std::string m_lastName; // Композиция: объект-элемент
18     Date m_birthDate; // Композиция: объект-элемент
19     Date m_hireDate; // Композиция: объект-элемент
20 };

```

Илл. 9.25 | Определение класса Employee, демонстрирующего композицию

```

1 // Fig. 9.26: Employee.cpp
2 // Определения функций класса Employee
3 #include <fmt/format.h>
4 #include <iostream>
5 #include "Employee.h" // Определение класса Employee
6 using namespace std;
7
8 // Конструктор использует список инициализаторов для передачи значений
9 // инициализаторов конструкторам объектов-элементов
10 Employee::Employee(std::string_view firstName, std::string_view lastName,
11                   const Date &birthDate, const Date &hireDate)
12     : m_firstName{firstName}, m_lastName{lastName},
13       m_birthDate{birthDate}, m_hireDate{hireDate} {
14     // Сообщение показывает момент вызова конструктора объекта Employee
15     std::cout << fmt::format("Employee object constructor: {} {}\\n",
16                               m_firstName, m_lastName);
17 }
18
19 // Получаем строковое представление объекта Employee
20 std::string Employee::toString() const {

```

```
21 return fmt::format("{} { } Hired: { } Birthday: { }", m_lastName,  
22 m_firstName, m_hireDate.toString(), m_birthday.toString());  
23 }  
24  
25 // Сообщение показывает момент вызова деструктора объекта Employee  
26 Employee::~Employee() {  
27     cout << fmt::format("Employee object destructor: { }, { }\n",  
28 m_lastName, m_firstName);  
29 }
```

Илл. 9.26 | Определения функций класса `Employee`

Список инициализаторов конструктора класса `Employee`

Согласно прототипу конструктора класса `Employee` (илл. 9.25, строки 11–12), конструктор имеет четыре параметра: `firstName`, `lastName`, `birthDate` и `hireDate`. В определении этого конструктора (илл. 9.26, строки 12–13) первые два параметра передаются через инициализаторы элементов в конструктор класса `string` для элементов данных `FirstName` и `LastName`. Последние два параметра передаются через список инициализаторов в конструктор класса `Date` для элементов данных `birthDate` и `hireDate`. Порядок перечисления инициализаторов в списке не имеет значения. Элементы данных создаются в том порядке, в каком они объявлены в классе `Employee`. В C++ Core Guidelines рекомендуется перечислять инициализаторы элементов в том же порядке, в каком они объявлены в классе¹.

Копирующий конструктор класса `Date` по умолчанию

Изучая код класса `Date` (илл. 9.23), обратите внимание, что он не предоставляет конструктор с параметром `Date`. Почему же инициализатор в конструкторе `Employee` может инициализировать объекты `m_birthDate` и `m_hireDate`, передавая их конструкторам объекты класса `Date`? Напомним из раздела 9.15, что компилятор предоставляет каждому классу копирующий конструктор по умолчанию, который копирует каждый элемент данных из аргумента конструктора в соответствующий элемент данных инициализируемого объекта. В главе 11 вы узнаете, как определять свои собственные копирующие конструкторы.

Тестирование классов `Date` и `Employee`

В программе, показанной на илл. 9.27, мы создаем два объекта `Date` (строки 9–10) и передаем их в качестве аргументов конструктору объекта `Employee`, созданного в строке 11. Когда создается объект `Employee`, его конструктор выполняет:

- ♦ два вызова конструктора класса `string` (строка 12 на илл. 9.26);
- ♦ два вызова копирующего конструктора класса `Date` по умолчанию (строка 13 на илл. 9.26).

¹ C++ Core Guidelines, «C.47: Define and Initialize Member Variables in the Order of Member Declaration». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rc-order>.

В строке 13 на илл. 9.27 мы выводим на экран данные объекта `Employee`, подтверждающие корректность инициализации.

```

1 // fig09_27.cpp
2 // Пример композиции: объект, содержащий объекты-элементы
3 #include <fmt/format.h>
4 #include <iostream>
5 #include "Date.h" // Определение класса Date
6 #include "Employee.h" // Определение класса Employee
7
8 int main() {
9     const Date birth{1987, 7, 24};
10    const Date hire{2018, 3, 12};
11    const Employee manager{"Sue", "Green", birth, hire};
12
13    std::cout << fmt::format("\n{}\n", manager.toString());
14 }
```

```

Date object constructor: 1987-07-24
Date object constructor: 2018-03-12
Employee object constructor: Sue Green

Green, Sue Hired: 2018-03-12 Birthday: 1987-07-24

Employee object destructor: Green, Sue
Date object destructor: 2018-03-12
Date object destructor: 1987-07-24
Date object destructor: 2018-03-12
Date object destructor: 1987-07-24
```

Илл. 9.27 | Пример композиции: объект, содержащий объекты-элементы

Когда создаются объекты класса `Date` (строки 9–10), вызываемый конструктор класса `Date` (строки 11–24 на илл. 9.24) выводит на экран сообщения, демонстрирующие эти вызовы (см. первые две строки выходных данных). Однако в строке 11 на илл. 9.27 происходят два вызова копирующего конструктора класса `Date` (строка 13 на илл. 9.26), которые не отображаются в выходных данных этой программы. Дело в том, что копирующий конструктор класса `Date` создается компилятором и не содержит никаких инструкций вывода на экран.

Каждый из классов `Date` и `Employee` содержит деструктор (строки 32–34 на илл. 9.24 и строки 26–29 на илл. 9.26 соответственно), который выводит сообщение при уничтожении объекта этого класса. Деструкторы помогают нам показать порядок уничтожения объектов: объекты класса `Date` уничтожаются после объекта класса `Employee`.

Обратите внимание на последние четыре строки выходных данных на илл. 9.27. Последние две строки — это сообщения от деструктора класса `Date`, работающего с объектами `hire` (илл. 9.27, строка 10) и `birth` (строка 9) соответственно. Выходные данные подтверждают, что все три объекта, созданные в функции `main`,

уничтожаются в обратном порядке по сравнению с их созданием. Сообщение от деструктора объекта `Employee` находится на пятой строке снизу. Четвертая и третья строки снизу – это сообщения от деструкторов объектов класса `Employee`: `m_hireDate` (илл. 9.25, строка 19) и `m_birthDate` (строка 18).

Эти выходные данные подтверждают уничтожение объекта `Employee` в порядке «снаружи внутрь». Деструктор объекта `Employee` запускается первым (см. пятую снизу строку в выходных данных). Затем объекты-элементы уничтожаются в обратном порядке по сравнению с их созданием. Деструктор класса `string` не содержит инструкций вывода, поэтому мы не видим процесс уничтожения объектов `firstName` и `lastName`.

Что произойдет, если не использовать список инициализаторов?

Если вы не инициализировали объект-элемент явно, то для инициализации неявно вызывается его конструктор по умолчанию. При отсутствии конструктора по умолчанию возникает ошибка компиляции. Значения, установленные конструктором по умолчанию, можно впоследствии изменить с помощью *set*-функций, но при таком подходе компьютер будет делать дополнительную работу и инициализация займет больше времени. Рекомендуется сразу инициализировать объекты-элементы инициализаторами, устраняя тем самым лишнюю нагрузку на компьютер, вызванную их повторной инициализацией.

9.18. Дружественные функции и классы

Дружественной функцией (friend function) по отношению к какому-либо классу называют такую функцию, которая имеет доступ к открытым и скрытым элементам этого класса. Другими класса могут быть:

- ✦ автономные функции,
- ✦ классы (и, следовательно, все их функции);
- ✦ некоторые функции других классов.

В этом разделе мы покажем вам простой пример дружественной функции. В главе 11 мы рассмотрим дружественные функции, которые перегружают операторы для работы с объектами пользовательских классов. Вы увидите, что иногда функцию класса нельзя использовать для определения некоторых перегруженных операторов.

Объявление дружественных отношений

Чтобы объявить дружественную классу функцию, поместите в определение этого класса ее прототип с предшествующим ключевым словом `friend`. Чтобы объявить все функции существующего класса `classTwo` дружественными классу `ClassOne`, поместите в определение класса `ClassOne` такое объявление:

```
friend class ClassTwo;
```

Правила дружбы

Основные правила:

- ♦ Дружба предоставляется, а не забирается. Чтобы класс В стал другом класса А, класс А должен объявить класс В своим другом.
- ♦ Дружба не симметрична. Если класс А является другом класса В, из этого не следует, что класс В является другом класса А.
- ♦ Дружба не передается. Если класс А является другом класса В, а класс В является другом класса С, из этого не следует, что класс А является другом класса С.

Игнорирование спецификаторов доступа

Спецификаторы доступа `public`, `protected` (глава 10) и `private` не применяются к объявлениям друзей класса, поэтому такие объявления можно размещать в любом месте определения класса. Мы предпочитаем размещать объявления друзей класса в начале определения класса, без какого-либо спецификатора доступа перед ними.

Изменение скрытых данных класса дружественной функцией

В следующей программе (илл. 9.28) функция `modifyX` определяется как дружественная к классу `Count` (строка 9). Таким образом, функция `modifyX` может изменять скрытый элемент данных `m_x`.

```

1 // fig09_28.cpp
2 // Дружественная функция получает доступ к скрытым элементам данных класса
3 #include <fmt/format.h>
4 #include <iostream>
5 #include "fmt/format.h" // Или в C++20: #include <format>
6
7 // Определение класса Count
8 class Count {
9     friend void setX(Count& c, int value); // Объявляем дружественную функцию
10 public:
11     int getX() const {return m_x;}
12 private:
13     int m_x{0};
14 };
15
16 // Функция modifyX может изменять скрытые данные класса Count,
17 // поскольку она объявлена как дружественная к этому классу (строка 8)
18 void modifyX(Count& c, int value) {
19     c.m_x = value; // Допустимо, так как modifyX = это друг класса Count
20 }
21
22 int main() {
23     Count counter{}; // Создаем объект класса Count
24

```

```
25  std::cout << fmt::format("Initial counter.m_x: {}\n", counter.getX());
26  modifyX(counter, 8); // Изменяем значение скрытого элемента
27  std::cout << fmt::format("counter.m_x after modifyX: {}\n",
28      counter.getX());
29 }
```

```
Initial counter.m_x: 0
counter.m_x after modifyX: 8
```

Илл. 9.28 | Дружественная функция получает доступ к скрытым элементам данных класса

Функция `modifyX` (строки 18–20) — это автономная (свободная) функция, не входящая в класс `Count`. Когда мы вызываем из `main` функцию `modifyX` для изменения объекта `counter` класса `Count` (строка 26), мы должны передать ей этот объект в качестве аргумента. Функция `modifyX` получает доступ к скрытому элементу данных `m_x` (строка 19), потому что она объявлена как дружественная классу `Count` (строка 9). Без этого объявления мы получили бы сообщения об ошибках, указывающие, что у функции `modifyX` нет доступа к скрытому элементу данных `m_x`.

9.19. Указатель `this`

Во время работы программы набор функций каждого класса существует в памяти компьютера только в одном экземпляре. Тем не менее объектов каждого класса может быть сколько угодно. Каким же образом вызванная функция находит среди многих объектов именно тот, с элементами данных которого ей надо работать? Она обращается к объекту через указатель `this` (ключевое слово C++), инициализированный неявным аргументом: адресом конкретного объекта. Он передается всем нестатическим¹ функциям объекта.

Использование указателя `this` для предотвращения конфликтов имен

Функции класса используют указатель `this` неявно (как мы делали до сих пор) или явно, чтобы обращаться к элементам данных и к другим функциям объекта. Одно из явных применений указателя `this` заключается в предотвращении конфликтов имен между элементами данных класса и параметрами конструктора или функций класса. Если в функции есть локальная переменная и элемент данных с одинаковым именем, локальная переменная **скрывает** (*hide*), или **затеняет** (*shadow*), собой элемент данных. Использование в теле функции этого имени означает обращение к локальной переменной, а не к элементу данных.

В таких случаях для доступа к элементу данных надо квалифицировать его имя с помощью `this->`. Например, если в классе `Account` есть элемент с именем `name`, к нему можно обратиться из функции `setName`:

¹ В разделе 9.20 мы рассмотрим статические элементы класса и объясним, почему не происходит неявной передачи указателя `this` статическим функциям класса.


```
void Account::setName(std::string_view name) {
    this->name = name; // this-> служит для доступа к элементу данных объекта
}
```

Здесь `this->name` — это элемент данных с именем `name`. Впрочем, для предотвращения конфликтов имен рекомендуется начинать все имена элементов данных (в отличие от локальных переменных) с префикса `m_`, как мы делали во всех пользовательских классах.

Типы указателей `this`

Тип указателя `this` зависит от типа объекта и от того, является ли константной (`const`) та функция, в которой он используется:

- ✦ В неконстантной функции класса `Time` указатель `this` имеет тип `Time*` — указатель на объект класса `Time`.
- ✦ В константной функции класса `Time` указатель `this` имеет тип `const Time*` — константный указатель на объект класса `Time`.

9.19.1. Неявное и явное использование указателя `this` для доступа к элементам данных объекта

На илл. 9.29 показан пример, демонстрирующий неявное и явное использование указателя `this` в функции класса для отображения скрытого элемента данных `m_x` объекта `Test`. В разделе 9.19.2 и в главе 11 мы покажем еще несколько полезных примеров кода с указателем `this`.

```
1 // fig09_29.cpp
2 // Использование указателя this для обращения к элементам данных объекта
3 #include <fmt/format.h>
4 #include <iostream>
5
6 class Test {
7 public:
8     explicit Test(int value);
9     void print() const;
10 private:
11     int m_x{0};
12 };
13
14 // Конструктор
15 Test::Test(int value) : m_x{value} {} // Инициализируем m_x значением value
16
17 // Отображаем m_x, используя указатель this неявно, а затем явно
18 // Круглые скобки вокруг *this нужны для согласования приоритетов операторов
19 void Test::print() const {
20     // Неявно используем указатель this для доступа к элементу m_x
21     std::cout << fmt::format(" m_x = {}\n", m_x);
```

```

22
23 // Явно используем указатель this и оператор "стрелка"
24 // для доступа к элементу m_x
25 std::cout << fmt::format(" this->m_x = {}\n", this->m_x);
26
27 // Явно используем разыменованный указатель this
28 // и оператор "точка" для доступа к элементу m_x
29 std::cout << fmt::format("( *this ).m_x = {}\n", ( *this ).m_x);
30 }
31
32 int main() {
33     const Test testObject{12}; // Создаем и инициализируем объект testObject
34     testObject.print();
35 }

```

```

    x = 12
  this->x = 12
  (*this).x = 12

```

Илл. 9.29 | Использование указателя this для обращения к элементам данных объекта

Функция `print` (строки 19–30) сначала отображает значение `m_x`, неявно используя указатель `this` (строка 21) — указывается только имя элемента данных. Затем в функции `print` используются разные формы записи для доступа к `m_x` через указатель `this`:

- ♦ `this->m_x` (строка 25);
- ♦ `(*this).m_x` (строка 29).

Круглые скобки вокруг `*this` (строка 29) обязательны, поскольку оператор «точка» (`.`) имеет более высокий приоритет, чем оператор разыменования указателя (`*`). Без круглых скобок выражение `*this.m_x` было бы вычислено как `*(this.m_x)`. Оператор «точка» нельзя использовать с указателем, так как это вызовет ошибку компиляции.

9.19.2. Использование указателя this для каскадного вызова функций

Указатель `this` также применяют для **каскадного вызова функций класса** (**cascaded member-function calls**), то есть для последовательного вызова нескольких функций в одной инструкции, как в строке 11 на илл. 9.32. Программа, показанная на илл. 9.30–9.32, содержит измененные функции класса `Time`: `setTime`, `setHour`, `setMinute` и `setSecond`. Каждая из них возвращает ссылку на объект класса `Time`, для которого вызывается. Эта ссылка позволяет выполнять каскадный вызов функций. На илл. 9.31 последняя инструкция в функции `setTime` возвращает ссылку на `*this` (строка 30). Каждая из функций `setHour`, `setMinute` и `setSecond` вызывает `setTime` и возвращает `Time&`.

```
1 // Fig. 9.30: Time.h
2 // Класс Time, измененный для каскадного вызова функций
3 #pragma once // Предотвращаем повторные включения заголовка
4 #include <string>
5
6 class Time {
7 public:
8     // Конструктор по умолчанию, так как он может быть вызван без аргументов
9     explicit Time(int hour = 0, int minute = 0, int second = 0);
10
11     // Set-функции
12     Time& setTime(int hour, int minute, int second);
13     Time& setHour(int hour); // Устанавливаем hour (после проверки)
14     Time& setMinute(int minute); // Устанавливаем minute (после проверки)
15     Time& setSecond(int second); // Устанавливаем second (после проверки)
16
17     int getHour() const; // Возвращаем hour
18     int getMinute() const; // Возвращаем minute
19     int getSecond() const; // Возвращаем second
20     std::string to24HourString() const; // 24-часовой формат времени
21     std::string to12HourString() const; // 12-часовой формат времени
22 private:
23     int m_hour{0}; // 0 - 23 (24-часовой формат часов)
24     int m_minute{0}; // 0 - 59
25     int m_second{0}; // 0 - 59
26 };
```

Илл. 9.30 | Класс Time, измененный для каскадного вызова функций

```
1 // Fig. 9.31: Time.cpp
2 // Определения функций класса Time
3 #include <fmt/format.h>
4 #include <stdexcept>
5 #include "Time.h" // Определение класса Time
6
7 // Конструктор инициализирует каждый элемент данных
8 Time::Time(int hour, int minute, int second) {
9     setTime(hour, minute, second);
10 }
11
12 // Устанавливаем новое время в 24-часовом формате
13 Time& Time::setTime(int hour, int minute, int second) {
14     // Проверяем допустимость значений hour, minute и second
15     if (hour < 0 || hour >= 24) {
16         throw std::invalid_argument{"hour was out of range"};
17     }
18
19     if (minute < 0 || minute >= 60) {
20         throw std::invalid_argument{"minute was out of range"};
21     }
22
23     if (second < 0 || second >= 59) {
24         throw std::invalid_argument{"second was out of range"};
25     }
26
27     m_hour = hour;
28     m_minute = minute;
29     m_second = second;
30 }
```

```
21     }
22
23     if (second < 0 || second >= 60) {
24         throw std::invalid_argument{"second was out of range"};
25     }
26
27     m_hour = hour;
28     m_minute = minute;
29     m_second = second;
30     return *this; // Для каскадного вызова функций
31 }
32
33 // Устанавливаем значение hour
34 Time& Time::setHour(int hour) {
35     return setTime(hour, m_minute, m_second);
36 }
37
38 // Устанавливаем значение minute
39 Time& Time::setMinute(int minute) {
40     return setTime(m_hour, minute, m_second);
41 }
42
43 // Устанавливаем значение second
44 Time& Time::setSecond(int second) {
45     return setTime(m_hour, m_minute, second);
46 }
47
48 // Получаем значение hour
49 int Time::getHour() const {return m_hour;}
50
51 // Получаем значение minute
52 int Time::getMinute() const {return m_minute;}
53
54 // Получаем значение second
55 int Time::getSecond() const {return m_second;}
56
57 // Возвращаем время в 24-часовом формате (HH:MM:SS)
58 std::string Time::to24HourString() const {
59     return fmt::format("{:02d}:{:02d}:{:02d}",
60         getHour(), getMinute(), getSecond());
61 }
62
63 // Возвращаем время в 12-часовом формате (HH:MM:SS AM или PM)
64 std::string Time::to12HourString() const {
65     return fmt::format("{:}:{:02d}:{:02d} {}",
66         ((getHour() % 12 == 0) ? 12 : getHour() % 12),
67         getMinute(), getSecond(), (getHour() < 12 ? "AM" : "PM"));
68 }
```

Илл. 9.31 | Определения функций класса Time, измененного для каскадного вызова функций

На илл. 9.32 мы создаем объект `t` класса `Time` (строка 9), а затем используем его в каскадном вызове функций (строки 11 и 19).

```

1 // fig09_32.cpp
2 // Каскадный вызов функций через указатель this
3 #include <fmt/format.h>
4 #include <iostream>
5 #include "Time.h" // Определение класса Time
6 using namespace std;
7
8 int main() {
9     Time t{}; // Создаем объект класса Time
10
11     t.setHour(18).setMinute(30).setSecond(22); // Каскадный вызов функций
12
13     // Отображаем время в 24-часовом и 12-часовом формате
14     std::cout << fmt::format("24-hour time: {}\n12-hour time: {}\n\n",
15                             t.to24HourString(), t.to12HourString());
16
17     // Каскадный вызов функций
18     std::cout << fmt::format("New 12-hour time: {}\n",
19                             t.setTime(20, 20, 20).to12HourString());
20 }

```

```

24-hour time: 18:30:22
12-hour time: 6:30:22 PM

New 12-hour time: 8:20:20 PM

```

Илл. 9.32 | Каскадный вызов функций через указатель `this`

Почему работает возврат `*this` в качестве ссылки? Оператор «точка» (`.`) группируется слева направо, поэтому инструкция в строке 11

```
t.setHour(18).setMinute(30).setSecond(22);
```

прежде всего вызывает функцию `t.setHour(18)`, которая возвращает ссылку на объект `t` (уже измененный) в качестве значения этого вызова функции. Оставшееся выражение интерпретируется так:

```
t.setMinute(30).setSecond(22);
```

Теперь вызывается функция `t.setMinute(30)`, которая возвращает ссылку на объект `t` (измененный повторно). Оставшееся выражение интерпретируется так:

```
t.setSecond(22);
```

В строке 19 на илл. 9.32 тоже используется каскадный вызов¹. Мы не можем добавить в каскад другую функцию класса `Time` после вызова функции `to12HourString`,

¹ Этот конкретный каскадный вызов предназначен только для учебных целей. Вызовы функций, изменяющие объекты, надо помещать в отдельные инструкции, чтобы избежать проблем с порядком выполнения.

потому что она не возвращает `Time&`. Однако мы могли бы добавить функцию класса `string`, потому что `to12HourString` возвращает объект `string`. В главе 11 представлены практические примеры каскадных вызовов функций, например, в инструкциях `cout` с оператором `<<` и в инструкциях `cin` с оператором `>>`.

9.20. Статические элементы класса: данные и функции, общие для всех объектов класса

Есть исключение из правила, согласно которому каждый объект имеет собственную копию элементов данных. Иногда требуется, чтобы некоторые переменные были общими для всех объектов класса. Для этой цели, а также по некоторым другим причинам используются **статические** (**static**) элементы данных (объявленные с ключевым словом `static`). Статическая переменная «работает на весь класс», то есть содержит данные, доступные каждому объекту класса. Статические элементы данных позволяют экономить память в тех случаях, когда все объекты класса могут совместно использовать единственный экземпляр данных.

Преимущества статических данных

Рассмотрим необходимость статических данных на простом примере. Предположим, что мы пишем игру с марсианами (объектами класса `Martian`) и другими космическими существами. Каждый `Martian` агрессивен и готов атаковать других существ, но только когда он уверен, что в данный момент на игровом поле действуют не менее пяти объектов `Martian`. А если их меньше пяти, каждый `Martian` становится трусливым. Таким образом, количество действующих марсиан (значение переменной `martianCount`) должно быть известно каждому из них. Предположим, что мы включим элемент данных `martianCount` во все объекты класса `Martian`. В этом случае у каждого марсианина будет собственная копия этой переменной. При создании и уничтожении каждого объекта `Martian` нам придется обновлять `martianCount` во всех объектах этого класса. Каждый объект `Martian` должен будет хранить в себе постоянно меняющееся количество объектов `Martian`. Из-за этого память компьютера будет занята бесполезными копиями `martianCount`, а для их синхронизации потребуется много процессорного времени. Вместо этого мы объявим элемент данных `martianCount` статическим (**static**), чтобы он стал общим для всех объектов класса `Martian`. Тогда каждый `Martian` получит доступ к `martianCount` как к собственному элементу данных, но в программе будет только один экземпляр `martianCount`. Это экономит память. Мы включим в конструктор класса `Martian` инструкцию инкремента статической переменной `martianCount`, а в деструктор — инструкцию ее декремента. Это гораздо проще, чем выполнять в процессе игры многократные инструкции инкремента и декремента для множества объектов.

Инициализация и область видимости статических элементов данных

Статические элементы данных имеют область видимости класса. Статический элемент данных должен быть инициализирован только один раз. Статические

элементы данных всех основных (фундаментальных) типов инициализируются по умолчанию значением 0. Каждый константный статический элемент данных (`static const`) может иметь внутриклассовый инициализатор. Начиная с C++17, внутриклассовые инициализаторы можно использовать и для неконстантных элементов статических данных, добавляя в объявление ключевое слово `inline` (илл. 9.33). Если статический элемент данных является объектом класса, содержащего конструктор по умолчанию, этот элемент данных не надо инициализировать явно, потому что будет вызван его конструктор по умолчанию.

Доступ к статическим элементам данных

Статические элементы класса существуют даже при отсутствии объектов этого класса. Для доступа к открытым (`public`) статическим элементам данных и функциям класса просто указывайте перед именем нужного вам элемента имя класса и оператор разрешения области видимости (`::`). Например, если статическая переменная `martianCount` из класса `Martian` является открытой, к ней можно получить доступ по полному имени `Martian::martianCount`, даже если никаких объектов `Martian` не было создано.

Доступ к скрытым (`private`) и защищенным (`protected`, глава 10) статическим элементам класса обычно осуществляется через открытые функции класса или дружественные функции. Чтобы при отсутствии объектов класса получать доступ к его скрытым статическим (`private static`) и защищенным статическим (`protected static`) элементам данных, определите открытую статическую (`static`) функцию класса и вызывайте ее, добавляя перед ее именем имя класса и оператор разрешения области видимости. Статическая функция выполняет задачи для всего класса, а не для отдельных объектов этого класса.

Пример использования статических элементов данных

В следующем примере кода мы продемонстрируем скрытый статический элемент данных `m_count`, инициализированный значением 0 (илл. 9.33, строка 22), и открытую статическую функцию `getCount` (илл. 9.33, строка 16). Статический элемент данных также можно инициализировать в файле реализации класса. Например, мы могли бы поместить в файл `Employee.cpp` (илл. 9.34) после включения заголовка `Employee.h` такую инструкцию:

```
int Employee::count{0};
```

```
1 // Fig. 9.33: Employee.h
2 // Определение класса Employee со статическим элементом данных,
3 // отслеживающим количество объектов класса Employee
4 #pragma once // Предотвращаем повторные включения заголовка
5 #include <string>
6 #include <string_view>
7
8 class Employee {
9 public:
10     Employee(std::string_view firstName, std::string_view lastName);
```

```

11 ~Employee(); // Деструктор
12 const std::string& getFirstName() const; // Возвращаем имя сотрудника
13 const std::string& getLastName() const; // Возвращаем фамилию сотрудника
14
15 // Статическая функция класса
16 static int getCount(); // Возвращаем количество созданных объектов
17 private:
18     std::string m_firstName;
19     std::string m_lastName;
20
21 // Статические данные
22 inline static int m_count{0}; // Количество созданных объектов
23 };

```

Илл. 9.33 | Определение класса `Employee` со статическим элементом данных, отслеживающим количество объектов класса `Employee`

На илл. 9.34 в строке 10 определяется статическая функция `getCount`. Обратите внимание, что в этом объявлении нет ключевого слова `static`, которое не может применяться к определению элемента вне определения класса. В этой программе элемент данных `m_count` является счетчиком объектов класса `Employee`, имеющих в памяти в данный момент. Когда объекты класса `Employee` существуют, их функции могут обращаться к элементу `m_count`, как показано в конструкторе (строка 16) и в деструкторе (строка 25).

```

1 // Fig. 9.34: Employee.cpp
2 // Определения функций класса Employee
3 #include <fmt/format.h>
4 #include <iostream>
5 #include "Employee.h" // Определение класса Employee
6 using namespace std;
7
8 // Определение статической функции, возвращающей количество созданных
9 // объектов класса Employee (она объявлена как static в Employee.h)
10 int Employee::getCount() {return m_count;}
11
12 // Конструктор инициализирует нестатические элементы данных и
13 // выполняет инкремент статического счетчика
14 Employee::Employee(string_view firstName, string_view lastName)
15     : m_firstName(firstName), m_lastName(lastName) {
16     ++m_count; // Увеличиваем статический счетчик количества сотрудников
17     std::cout << fmt::format("Employee constructor called for {} {}\n",
18         m_firstName, m_lastName);
19 }
20
21 // Деструктор выполняет декремент статического счетчика
22 Employee::~Employee() {
23     std::cout << fmt::format("~Employee() called for {} {}\n",
24         m_firstName, m_lastName);
25     --m_count; // Уменьшаем статический счетчик количества сотрудников

```



```

26 }
27
28 // Возвращаем имя сотрудника
29 const string& Employee::getFirstName() const {return m_firstName;}
30
31 // Возвращаем фамилию сотрудника
32 const string& Employee::getLastName() const {return m_lastName;}

```

Илл. 9.34 | Определения функций класса Employee

В коде на илл. 9.35 статическая функция `getCount` используется, чтобы отслеживать количество имеющихся в памяти объектов класса `Employee` в разные моменты выполняемой программы. Программа вызывает функцию `Employee::getCount()`:

- ♦ до создания каких-либо объектов класса `Employee` (строка 11);
- ♦ после создания двух объектов класса `Employee` (строка 23);
- ♦ после уничтожения всех объектов класса `Employee` (строка 34).

Когда объекты класса `Employee` находятся в области видимости, вы можете вызывать функцию `getCount` и для них тоже. Например, в строке 23 вы могли бы использовать такой вызов:

```
e1.getCount()
```

Или такой:

```
e2.getCount()
```

Каждый из этих вызовов возвращает текущее значение статического элемента `m_count` класса `Employee`.

```

1 // fig09_35.cpp
2 // Отслеживание количества объектов класса в статическом элементе класса
3 #include <fmt/format.h>
4 #include <iostream>
5 #include "Employee.h" // Определение класса Employee
6
7 int main() {
8     // Объекты еще не созданы; вызываем статическую функцию getCount
9     // через имя класса и оператор разрешения области видимости
10    std::cout << fmt::format("Initial employee count: {}\n",
11        Employee::getCount()); // Используем имя класса
12
13    // В следующем блоке мы будем создавать объекты класса Employee
14    // и уничтожать их до завершения функции main
15    {
16        const Employee e1{"Susan", "Baker"};
17        const Employee e2{"Robert", "Jones"};
18
19        // Есть два объекта; снова вызываем статическую функцию getCount

```

```

20 // через имя класса и оператор разрешения области видимости
21 std::cout << fmt::format(
22     "Employee count after creating objects: {}\n\n",
23     Employee::getCount());
24
25 std::cout << fmt::format("Employee 1: {} {}\nEmployee 2: {} {}\n\n",
26     e1.getFirstName(), e1.getLastName(),
27     e2.getFirstName(), e2.getLastName());
28 }
29
30 // Нет ни одного объекта; снова вызываем статическую функцию getCount
31 // через имя класса и оператор разрешения области видимости
32 std::cout << fmt::format(
33     "Employee count after objects are deleted: {}\n",
34     Employee::getCount());
35 }

```

```

Initial employee count: 0
Employee constructor called for Susan Baker
Employee constructor called for Robert Jones
Employee count after creating objects: 2

Employee 1: Susan Baker
Employee 2: Robert Jones

~Employee() called for Robert Jones
~Employee() called for Susan Baker
Employee count after objects are deleted: 0

```

Илл. 9.35 | Отслеживание количества объектов класса в статическом элементе класса

В строках 15–28 определяется вложенный блок в функции `main`. Напомним, что локальные переменные существуют до тех пор, пока существует область видимости, в которой они определены. Во вложенном блоке мы создаем два объекта класса `Employee` (строки 16–17). При создании каждого из них конструктор выполняет инкремент статического элемента данных `count` класса `Employee`. Когда программа достигает строки 28, оба объекта уничтожаются. Для этого вызываются их деструкторы, каждый из которых выполняет декремент статического элемента данных `count` класса `Employee`.

Дополнительные сведения о статических функциях

Функцию класса надо объявлять как статическую, если она не обращается к нестатическим элементам данных или к нестатическим функциям класса. Статическая функция не имеет указателя `this`, поскольку статические элементы данных и статические функции существуют независимо от любых объектов класса. Указатель `this` должен ссылаться на конкретный объект, но статическую функцию можно вызывать даже при отсутствии объектов ее класса. Использование указателя `this` в статической функции класса вызовет ошибку компиляции.

Статическую функцию класса нельзя объявлять как константную. Квалификатор `const` означает, что функция не может изменять содержимое объекта. Но статические функции существуют и работают независимо от любых объектов класса. Объявление статической функции с квалификатором `const` вызовет ошибку компиляции.

9.21. Агрегаты

Определение агрегатов сформулировано в разделе 9.4.1 стандарта C++:

<http://wg21.link/n4861>

Согласно этому документу, **aggrerat (aggregate)** — это массив или класс, который:

- ♦ не имеет пользовательских конструкторов;
- ♦ не имеет скрытых (`private`) или защищенных (`protected`, глава 10) нестатических элементов данных;
- ♦ не имеет виртуальных функций (глава 10);
- ♦ не имеет скрытых, защищенных или виртуальных базовых классов (глава 10).

Требование об отсутствии пользовательских конструкторов было введено в C++20 для того, чтобы исключить возможность инициализации агрегатного объекта в обход пользовательского конструктора¹.

Вы можете определить агрегат как класс, в котором все данные являются открытыми (`public`). Агрегатом может быть обычный класс или структура. Структура (`struct`) — это класс, который по умолчанию содержит только открытые элементы. Следующая структура определяет агрегатный тип с именем `Record`, содержащий четыре открытых элемента данных:

```
struct Record {  
    int account;  
    string first;  
    string last;  
    double balance;  
};
```

В C++ Core Guidelines рекомендуется использовать `class`, а не `struct`, если хотя бы один элемент данных или хотя бы одна функция не должны быть открытыми².

¹ «Prohibit Aggregates with User-Declared Constructors». Был доступен: 15 сентября 2023 г. <http://wg21.link/p1008>.

² C++ Core Guidelines, «C.8: Use class Rather Than struct if Any Member Is Non-Public». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rc-class>.

9.21.1. Инициализация агрегата

Вы можете инициализировать объект агрегатного типа `Record` таким образом:

```
Record record{100, "Brian", "Blue", 123.45};
```

В C++11 нельзя было использовать скобочную инициализацию для объекта агрегатного типа, если любой из его нестатических элементов данных объявлялся с внутриклассовым инициализатором. Например, приведенная выше инструкция привела бы к ошибке компиляции, если бы тип `Record` был определен со значением по умолчанию для `balance`:

```
struct Record {  
    int account;  
    std::string first;  
    std::string last;  
    double balance{0.0};  
};
```

Начиная с C++14, этого ограничения нет. Кроме того, в вашем списке инициализаторов объекта агрегатного типа может быть меньше инициализаторов, чем элементов данных в объекте, например:

```
Record record{0, "Brian", "Blue"};
```

В этом случае все остальные элементы данных будут инициализированы автоматически:

- ✦ Элементы данных с внутриклассовыми инициализаторами получают их значения (в предыдущем примере элемент `balance` в объекте `record` получит значение `0.0`).
- ✦ Элементы данных без внутриклассовых инициализаторов будут инициализированы пустыми фигурными скобками (`{}`). Это означает, что переменные основных (фундаментальных) типов получают значение `0`, переменные типа `bool` — значение `false`, а объекты — значения по умолчанию.

9.21.2. Назначенные инициализаторы (C++20)

Начиная с C++20, агрегаты поддерживают **назначенные инициализаторы** (**designated initializers**), в которых вы можете указывать по именам, какие элементы данных надо инициализировать. Используя предыдущее определение структуры `Record`, вы можете инициализировать объект `record` таким образом:

```
Record record{.first{"Sue"}, .last{"Green"}};
```

В этом случае будут явно инициализированы только два элемента данных из четырех. Перед каждым явно указанным именем элемента ставится точка (`.`). Все указанные вами идентификаторы должны быть перечислены в том же порядке, в каком они объявлены в агрегатном типе. Предыдущая инструкция инициализи-

рует элементы `first` и `last` значениями `"Sue"` и `"Green"` соответственно. Остальные элементы получают значения по умолчанию:

- ♦ `account` получит значение `0`;
- ♦ `balance` получит значение из объявления типа, в данном случае — `0.0`.

Другие преимущества назначенных инициализаторов¹

Добавление новых элементов данных к агрегатному типу не приведет к нарушению существующих инструкций, использующих назначенные инициализаторы. Все новые элементы, которые не инициализированы явно, просто получают значения по умолчанию. Кроме того, назначенные инициализаторы улучшают совместимость с языком C, в котором они используются, начиная с версии C99.

9.22. Готовые объекты: сериализация в формате JSON

Современные программы все чаще работают «в облаке», то есть их фрагменты распределяются по интернету. Многие приложения, которыми вы ежедневно пользуетесь, взаимодействуют через интернет с **облачными сервисами (cloud-based services)**, для работы которых обычно требуются огромные кластеры компьютерных ресурсов (таких, как процессоры, память, жесткие диски и базы данных).

Сервис, предоставляющий пользователям доступ через интернет, называется **веб-сервисом (web service)**. Многие пользовательские приложения взаимодействуют с веб-сервисами, обмениваясь объектами в формате JSON. **JSON (JavaScript Object Notation)** — это основанный на тексте формат обмена данными, представляющий каждый передаваемый объект в виде набора пар *имя — значение*. Текст в этом формате может прочитать как компьютер, так и человек. В последние годы JSON стал наиболее предпочтительным форматом данных для передачи объектов на разных платформах.

Формат данных JSON

Каждый объект JSON содержит список пар *имя — значение*, разделенный запятыми и заключенный в фигурные скобки. Например, из трех таких пар может состоять запись о клиенте в базе данных банка:

```
{"account": 100, "name": "Jones", "balance": 24.98}
```

JSON также поддерживает массивы. Массив записывается в виде заключенного в квадратные скобки списка значений, разделенных запятыми. Например, так выглядит массив из трех целых чисел в формате JSON:

```
[100, 200, 300]
```

¹ «Designated Initialization». Был доступен: 15 сентября 2023 г. <http://wg21.link/p0329r0>.

JSON может хранить в объектах и массивах:

- ♦ строки в кавычках (например, "Jones");
- ♦ числа (например, 100 или 24.98);
- ♦ логические значения JSON (представленные как true или false);
- ♦ null (отсутствие значения);
- ♦ массивы, содержащие любые допустимые значения JSON;
- ♦ другие объекты JSON.

Массив JSON может содержать элементы одного или разных типов.

Сериализация

Преобразование объекта в другой формат для сохранения в виде файла или для передачи через интернет называется **сериализацией (serialization)**. Восстановление объекта из сериализованных данных называется **десериализацией (deserialization)**. JSON — это один из нескольких общепринятых форматов сериализации. Другим распространенным форматом является XML (eXtensible Markup Language).

Безопасность сериализации

В некоторых языках программирования встроены собственные механизмы и форматы сериализации. Десериализация объектов, преобразованных в такие форматы, может создавать опасные проблемы. По данным Open Web Application Security Project (OWASP), собственные механизмы сериализации «могут быть использованы для вредоносных воздействий при обработке непроверенных данных. Были обнаружены воздействия на десериализаторы, позволяющие осуществлять атаки типа «отказ в обслуживании», «контроль доступа» и «удаленное выполнение кода»¹. OWASP также сообщает, что можно значительно снизить риск атак, отказавшись от имеющихся в языках программирования собственных форматов сериализации в пользу форматов «чистых данных», таких как JSON или XML.

Заголовочная библиотека cereal

Заголовочная библиотека `cereal`² предназначена для сериализации и десериализации объектов в формате JSON (который мы продемонстрируем), а также в формате XML и в двоичных форматах. Эта библиотека поддерживает основные (фундаментальные) типы данных и может обрабатывать большинство типов стандартной библиотеки C++, если вы включите в программу соответствующие заголовки `cereal` для каждого типа. Как вы увидите в следующем разделе, библио-

¹ «Deserialization Cheat Sheet». Был доступен: 15 сентября 2023 г. https://cheatsheetseries.owasp.org/cheatsheets/Deserialization_Cheat_Sheet.html.

² © 2017. У. Шейн Грант (W. Shane Grant) и Рэндольф Вурхис (Randolph Voorhies), «cereal — A C++11 library for serialization». <http://uscilab.github.io/cereal>.

тека `cereal` поддерживает и пользовательские типы. Документация к библиотеке `cereal` опубликована здесь:

<https://uscilab.github.io/cereal/index.html>

Для вашего удобства мы добавили библиотеку `cereal` к примерам кода из этой книги в папку `libraries`. Вы должны указать в параметрах IDE или компилятора папку `include` этой библиотеки, как и в предыдущих примерах с готовыми объектами.

9.22.1. Сериализация вектора объектов, содержащего открытые данные

Начнем с сериализации объектов, содержащих только открытые данные. В следующей программе, разделенной на части для удобства обсуждения (илл. 9.36), мы:

- ♦ создаем вектор (`vector`) объектов типа `Record` и отображаем его содержимое;
- ♦ используем библиотеку `cereal` для сериализации этого вектора в виде текстового файла;
- ♦ десериализуем содержимое этого файла обратно в вектор объектов `Record` и отображаем его содержимое.

Для сериализации в формате JSON нужен заголовок `json.hpp` (строка 3). Для сериализации объекта типа `std::vector` нужен заголовок `vector.hpp` (строка 4).

```

1 // fig09_36.cpp
2 // Сериализация и десериализация объектов с помощью библиотеки cereal
3 #include <cereal/archives/json.hpp>
4 #include <cereal/types/vector.hpp>
5 #include <fmt/format.h>
6 #include <fstream>
7 #include <iostream>
8 #include <vector>
9

```

Илл. 9.36 | Сериализация и десериализация объектов с помощью библиотеки `cereal`

Агрегатный тип `Record`

`Record` — это агрегатный тип данных, определенный как структура (`struct`). Напомним, что все данные в агрегате должны быть открытыми. В структуре это требование выполняется по умолчанию.

```

10 struct Record {
11     int account{};
12     std::string first{};
13     std::string last{};
14     double balance{};
15 };
16

```

Функция `serialize` для работы с объектами `Record`

Библиотека `cereal` предоставляет несколько способов сериализации объектов. Если тип объектов, которые вы хотите сериализовать, содержит только открытые (`public`) данные, вам достаточно определить шаблон функции `serialize` (строки 19–25) с двумя параметрами. Первый параметр — архив (`Archive`), куда будет помещен результат сериализации, а второй параметр — объект вашего типа (`Record`)¹. Мы будем вызывать функцию `serialize` и для сериализации, и для десериализации объектов `Record`. Шаблон функции позволяет вам выбирать между форматами сериализации и десериализации (JSON, XML или двоичные форматы), передавая объект соответствующего типа архива `cereal`. Библиотека предоставляет реализации архива для каждого случая.

```
17 // Шаблон функции serialize для сериализации объектов Record в объект Archive
18 // и для десериализации из объекта Archive в объекты Record
19 template <typename Archive>
20 void serialize(Archive& archive, Record& record) {
21     archive(cereal::make_nvp("account", record.account),
22           cereal::make_nvp("first", record.first),
23           cereal::make_nvp("last", record.last),
24           cereal::make_nvp("balance", record.balance));
25 }
26
```

Каждый тип архива `cereal` имеет перегруженный оператор «круглые скобки», позволяющий использовать имя архива в качестве имени функции (строки 21–24). В зависимости от выполняемого действия — сериализации или десериализации объектов `Record` — эта функция будет:

- ♦ либо выводить содержимое объектов `Record` в указанный поток;
- ♦ либо записывать сериализованные данные из указанного потока, создавая объекты `Record`.

Каждый вызов `cereal::make_nvp`², например в строке 21:

```
cereal::make_nvp("account", record.account)
```

предназначен для сериализации. Он создает пару *имя* — *значение*, в которой первый аргумент (в данном случае `"account"`) будет именем, а второй аргумент (в данном случае `record.account` типа `int`) будет значением. Давать осмысленные имена значениям не обязательно, но такие имена делают выходные данные в формате JSON более понятными для человека, как вы скоро увидите. Если вы не дали значениям имена, библиотека `cereal` использует стандартные имена, например `value0`, `value1` и т. д.

¹ Функцию `serialize` также можно определить как шаблон функции класса с единственным параметром `Archive`: https://uscilab.github.io/cereal/serialization_functions.html.

² Имя `make_nvp` — это сокращение от «make name-value pair» («создать пару имя — значение»).

Функция `displayRecords`

Мы включили в программу функцию `displayRecords`, чтобы показать содержимое объектов `Record` до сериализации и после десериализации. Функция выводит на экран элементы вектора, который она получает в качестве аргумента:

```
27 // Отображаем объекты вектора
28 void displayRecords(const std::vector<Record>& records) {
29     for (const auto& r : records) {
30         std::cout << fmt::format("{} {} {} {:.2f}\n",
31             r.account, r.first, r.last, r.balance);
32     }
33 }
34
```

Создание объектов `Record` для их последующей сериализации

В функции `main` в строках 36–39 мы создаем вектор с именем `records` и инициализируем его двумя объектами агрегатного типа `Record` (строки 37 и 38). С помощью СТАД компилятор автоматически определяет тип элементов вектора по типам из списков инициализации объектов `Record`. В строке 42 мы выводим на экран содержимое вектора, чтобы убедиться, что объекты `Record` инициализированы правильно.

```
35 int main() {
36     std::vector records{
37         Record{100, "Brian", "Blue", 123.45},
38         Record{200, "Sue", "Green", 987.65}
39     };
40
41     std::cout << "Records to serialize:\n";
42     displayRecords(records);
43 }
```

```
Records to serialize:
100 Brian Blue 123.45
200 Sue Green 987.65
```

Сериализация объектов `Record` с помощью класса `cereal::JSONOutputArchive`

Класс `cereal::JSONOutputArchive` сериализует данные в формате JSON в указанный поток, например в стандартный поток вывода или в файловый поток. В строке 45 мы пробуем открыть файл `records.json` для записи. Если файл удалось открыть, в строке 46 создается объект `cereal::JSONOutputArchive` с именем `archive`. Мы инициализируем его объектом `ofstream` с именем `output`, чтобы записать в файл преобразованные в формат JSON данные. В строке 47 объект `archive` используется для вывода пары *имя — значение*, где имя — `"records"`, а значение — наш вектор `records`. В процессе сериализации вектора сериализуются

все его элементы, то есть выполняется вызов функции `serialize` (строки 19–25) для каждого объекта `Record` в векторе.

```
44 // Сериализуем вектор объектов Records в формате JSON и сохраняем в файле
45 if (std::ofstream output{"records.json"}) {
46     cereal::JSONOutputArchive archive(output);
47     archive(cereal::make_nvp("records", records)); // Сериализуем records
48 }
49
```

Содержимое файла `records.json`

После выполнения инструкции в строке 47 файл `records.json` содержит следующие данные в формате JSON:

```
{
  "records": [
    {
      "account": 100,
      "first": "Brian",
      "last": "Blue",
      "balance": 123.45
    },
    {
      "account": 200,
      "first": "Sue",
      "last": "Green",
      "balance": 987.65
    }
  ]
}
```

Внешние фигурные скобки обозначают весь документ JSON. На иллюстрации мы выделили темно-серым прямоугольником пару *имя — значение* с именем `"records"`, значением которой является массив JSON. Он состоит из двух объектов JSON, которые мы выделили внутренними прямоугольниками (каждый объект JSON заключается в фигурные скобки). Таким образом, в этом массиве хранятся преобразованные в формат JSON элементы вектора `records`, который мы сериализовали в строке 47. Каждый объект JSON в массиве содержит четыре пары *имя — значение*, сериализованные функцией `serialize` из соответствующего объекта `Record`.

Десериализация объектов `Record` с помощью класса `cereal::JSONInputArchive`

Теперь мы десериализуем данные из файла и запишем их в новый вектор объектов `Record`. Чтобы библиотека `cereal` могла воссоздавать объекты в памяти, ей нужен доступ к конструктору по умолчанию для каждого типа. Библиотека будет вызывать конструктор для создания объекта, а затем заполнять нужными

значениями элементы данных этого объекта. Для структур, как и для классов, компилятор тоже предоставляет открытый (**public**) конструктор по умолчанию, если пользовательский конструктор не определен.

```

50 // Десериализуем данные JSON из файла в вектор объектов Records
51 if (std::ifstream input{"records.json"}) {
52     cereal::JSONInputArchive archive(input);
53     std::vector<Record> deserializedRecords{};
54     archive(deserializedRecords); // Десериализуем records
55     std::cout << "\nDeserialized records:\n";
56     displayRecords(deserializedRecords);
57 }
58 }

```

```

Deserialized records:
100 Brian Blue 123.45
200 Sue Green 987.65

```

Класс `cereal::JSONInputArchive` десериализует данные в формате JSON из указанного потока. В строке 51 мы пробуем открыть файл `records.json` для чтения. Если файл удалось открыть, в строке 52 создается объект `cereal::JSONInputArchive` с именем `archive`. Мы инициализируем его объектом `ifstream` с именем `input`, чтобы `archive` мог прочитать из файла `records.json` данные в формате JSON. В строке 53 мы создаем пустой вектор объектов `Record`, в который будем записывать данные из файла. В строке 54 объект `archive` используется для десериализации данных из файла в объект `deserializedRecords`. В процессе десериализации вектора десериализуются все его элементы. Для этого снова вызывается функция `serialize` (строки 19–25), но поскольку `archive` является объектом `cereal::JSONInputArchive`, каждый вызов функции `serialize` считывает данные из одного объекта JSON, создает объект `Record` и записывает в него эти данные.

9.22.2. Сериализация вектора объектов, содержащего скрытые данные

Объекты со скрытыми (**private**) данными тоже можно сериализовать. Для этого надо объявить функцию `serialize` как дружественную классу, чтобы она получила доступ к его скрытым данным. Чтобы продемонстрировать сериализацию скрытых данных, мы взяли копию предыдущей программы (илл. 9.36) и внесли в нее соответствующие изменения (илл. 9.37), в том числе заменили определение агрегата `Record` определением класса `Record` в строках 12–35.

```

1 // fig09_37.cpp
2 // Сериализация и десериализация объектов, содержащих скрытые данные
3 #include <cereal/archives/json.hpp>
4 #include <cereal/types/vector.hpp>
5 #include <fmt/format.h>
6 #include <fstream>
7 #include <iostream>

```

```
8 #include <string>
9 #include <string_view>
10 #include <vector>
11
12 class Record {
13     // Объявляем функцию serialize дружественной для доступа к скрытым данным
14     template<typename Archive>
15     friend void serialize(Archive& archive, Record& record);
16
17 public:
18     // Конструктор
19     explicit Record(int account = 0, std::string_view first = "",
20         std::string_view last = "", double balance = 0.0)
21         : m_account{account}, m_first{first},
22         m_last{last}, m_balance{balance} {}
23
24     // Get-функции класса
25     int getAccount() const {return m_account;}
26     const std::string& getFirst() const {return m_first;}
27     const std::string& getLast() const {return m_last;}
28     double getBalance() const {return m_balance;}
29
30 private:
31     int m_account{};
32     std::string m_first{};
33     std::string m_last{};
34     double m_balance{};
35 };
36
37 // Шаблон функции serialize для сериализации объектов Record в объект Archive
38 // и для десериализации из объекта Archive в объекты Record
39 template <typename Archive>
40 void serialize(Archive& archive, Record& record) {
41     archive(cereal::make_nvp("account", record.m_account),
42         cereal::make_nvp("first", record.m_first),
43         cereal::make_nvp("last", record.m_last),
44         cereal::make_nvp("balance", record.m_balance));
45 }
46
47 // Отображаем объекты вектора
48 void displayRecords(const std::vector<Record>& records) {
49     for (const auto& r : records) {
50         std::cout << fmt::format("{} {} {} {:.2f}\n", r.getAccount(),
51             r.getFirst(), r.getLast(), r.getBalance());
52     }
53 }
54
55 int main() {
56     std::vector records{
57         Record{100, "Brian", "Blue", 123.45},
58         Record{200, "Sue", "Green", 987.65}
59     };
```

```

60
61     std::cout << "Records to serialize:\n";
62     displayRecords(records);
63
64     // Сериализуем вектор объектов Records в формате JSON и сохраняем в файле
65     if (std::ofstream output("records2.json")) {
66         cereal::JSONOutputArchive archive(output);
67         archive(cereal::make_nvp("records", records)); // Сериализуем records
68     }
69
70     // Десериализуем данные JSON из файла в вектор объектов Records
71     if (std::ifstream input("records2.json")) {
72         cereal::JSONInputArchive archive(input);
73         std::vector<Record> deserializedRecords{};
74         archive(deserializedRecords); // Десериализуем records
75         std::cout << "\nDeserialized records:\n";
76         displayRecords(deserializedRecords);
77     }
78 }

```

Илл. 9.37 | Сериализация и десериализация объектов, содержащих скрытые данные

Класс `Record` содержит конструктор (строки 19–22), *get*-функции для получения данных (строки 25–28) и скрытые данные (строки 31–34). Отметим два ключевых момента:

- ♦ В строках 14–15 объявляется шаблон функции `serialize`, дружественной для этого класса. Дружественная функция `serialize` получает прямой доступ к скрытым элементам данных класса: `account`, `first`, `last` и `balance`.
- ♦ Все параметры конструктора имеют аргументы по умолчанию, что позволяет библиотеке `cereal` использовать его в качестве конструктора по умолчанию при десериализации объектов `Record`.

Функция `serialize` (строки 39–45) обращается к скрытым элементам данных класса `Record`, а функция `displayRecords` (строки 48–53) использует *get*-функции класса для доступа к скрытым данным и для их отображения на экране. Функция `main` здесь такая же, как в разделе 9.22.1. Она дает такие же результаты, поэтому мы не приводим выходные данные.

9.23. Итоги

В этой главе вы создавали пользовательские классы и объекты этих классов, а затем вызывали их функции для выполнения нужных действий. Вы объявляли элементы данных класса, предназначенные для хранения данных в объектах, и определяли функции класса, предназначенные для обработки этих данных. Вы узнали, как использовать конструктор класса, чтобы присваивать начальные значения элементам данных ваших объектов.

На примере учебного класса `Time` мы показали целый ряд дополнительных возможностей. Вы спроектировали класс, отделив его интерфейс от его реализации. Вы обращались к элементам данных объекта через указатель на объект с помощью оператора «стрелка». Вы узнали, что функции класса имеют область видимости класса: имя любой функции класса известно только другим функциям этого класса, за исключением тех случаев, когда клиент класса вызывает ее через имя объекта, ссылку на объект класса, указатель на объект класса или оператор разрешения области видимости. Мы рассмотрели функции доступа (обычно применяемые для получения данных и для проверки истинности условий) и служебные функции (скрытые функции класса, которые поддерживают работу открытых функций).

Вы узнали, что конструктор может содержать аргументы по умолчанию, что позволяет вызывать его несколькими способами. Вы также узнали, что любой конструктор, который может быть вызван без аргументов, является конструктором по умолчанию. Мы показали, как распределять код между конструкторами с помощью делегирующих конструкторов. Мы рассмотрели деструкторы, выполняющие завершающую обработку перед уничтожением объектов, и продемонстрировали порядок вызова конструкторов и деструкторов объекта.

На примере неправильного кода мы показали проблемы, которые возникают, когда функция класса возвращает ссылку или указатель на скрытый элемент данных, нарушая инкапсуляцию класса. Мы также показали, что объекты одного типа можно присваивать друг другу с помощью оператора присваивания по умолчанию.

Вы узнали, как определять константные объекты и функции класса для защиты от некорректного изменения объектов, обеспечивая тем самым соблюдение принципа наименьших привилегий. Вы познакомились с понятием композиции, благодаря которой элементами класса могут быть объекты других классов. Вы научились объявлять и использовать дружественные функции.

Теперь вы знаете, что указатель `this` передается в качестве неявного аргумента всем нестатическим функциям класса, позволяя им обращаться к элементам данных нужного объекта и к другим нестатическим функциям класса. Мы явно использовали указатель `this` для доступа к элементам класса и для каскадного вызова функций класса. Мы объяснили необходимость статических элементов данных и функций класса, а затем показали, как их объявлять и использовать.

Вы познакомились с понятием агрегатных типов данных и с назначенными инициализаторами (C++20) для них. В очередном примере работы с готовыми объектами вы выполнили сериализацию объектов в формате JSON (JavaScript Object Notation) с помощью библиотеки `cereal`.

В следующей главе мы продолжим обсуждение классов и познакомим вас с наследованием. Вы увидите, что классы могут наследовать свои атрибуты и поведение от общего для них класса, который называется базовым. Затем мы расширим тему наследования и введем понятие полиморфизма. Эта объектно-ориентированная концепция позволяет писать программы, в которых единообразно обрабатываются объекты разных классов, связанных наследованием.

ООП: наследование и динамический полиморфизм

В этой главе вы:

- узнаете традиционные и современные идиомы¹ наследования, познакомитесь с базовыми и производными классами;
- познакомитесь с порядком вызова конструкторов и деструкторов в иерархиях наследования;
- узнаете, как динамический полиморфизм упрощает программирование и способствует расширяемости систем;
- познакомитесь с ключевым словом `override`, предназначенным для переопределения виртуальных функций базового класса;
- научитесь запрещать спецификатором `final` переопределение функций;
- научитесь запрещать спецификатором `final` использование класса в качестве базового;
- научитесь наследованию с абстрактными и конкретными классами;
- узнаете, как в C++ реализованы ресурсоемкие виртуальные функции и динамическое связывание;
- используете идиому неvirtуального интерфейса (NVI) с открытыми неvirtуальными функциями и со скрытыми или защищенными виртуальными функциями;
- используете интерфейсы для создания более гибких систем на основе динамического полиморфизма;
- реализуете метод динамического полиморфизма без иерархии классов с помощью `std::variant` и `std::visit`.

¹ Идиома языка программирования — устойчивая синтаксическая конструкция, объединенная общей целью. — *Примеч. ред.*

10.1. Введение

В этой главе продолжается изучение объектно-ориентированного программирования (ООП), вводятся понятия наследования и динамического полиморфизма. С помощью **наследования (inheritance)** вы будете создавать классы, которые обладают функциональностью готовых (ранее созданных) классов, но дают вам возможность расширять и адаптировать эту функциональность под свои задачи.

Создавая класс, вы можете указать, что он должен **наследовать (inherit)** элементы другого класса. Этот существующий класс называется **базовым классом (base class)**, а ваш новый класс называется **производным классом (derived class)**. В некоторых языках программирования, таких как Java и C#, есть термины **суперкласс (superclass)** и **подкласс (subclass)**, обозначающие базовый и производный классы соответственно¹.

Отношения «содержит» и «является»

Вы должны различать отношения «содержит» (имеет) и «является» (представляет собой):

- ♦ Отношение **«содержит» («has-a»)** — это композиция (раздел 9.17). Объект содержит в качестве своих элементов один или несколько объектов других классов. Например, автомобиль содержит руль, содержит педаль тормоза, содержит двигатель, содержит коробку передач и т. д.
- ♦ Отношение **«является» («is-a»)** — это наследование. Объект производного класса может рассматриваться как объект, имеющий тип базового класса. Например, автомобиль является транспортным средством, то есть демонстрирует поведение и атрибуты транспортного средства.

Динамический полиморфизм

Мы объясним и покажем на примерах, что такое динамический полиморфизм с иерархиями наследования². **Динамический полиморфизм (runtime polymorphism)** позволяет удобно «программировать вообще», а не «программировать в частности». Программы могут обрабатывать объекты разных классов, связанных наследованием, как если бы все эти объекты имели один тип базового класса. Вы увидите, что полиморфный код (который мы сначала реализуем с помощью наследования и виртуальных функций) обращается к объектам через указатели базового класса или ссылки базового класса.

¹ Иногда программисты называют базовые классы родительскими, а производные — дочерними. — *Примеч. ред.*

² Позже вы увидите, что полиморфизм не обязательно реализуется с помощью иерархий наследования. Мы рассмотрим статический полиморфизм в главе 15.

Простое расширение программ

С помощью динамического полиморфизма вы можете проектировать и внедрять большие программы, функциональность которых легко расширяется по мере необходимости. Вы можете добавлять новые классы, почти не внося изменений в основные части вашей программы, при условии, что эти новые классы входят в иерархию наследования вашей программы. Вы можете модифицировать только тот код, в котором потребуется прямое взаимодействие с новыми классами. Предположим, что от класса `Vehicle` наследуется новый класс `Car`. Вам нужно написать только этот новый класс и код, который создает объекты класса `Car` и добавляет их в систему. После этого новый класс просто «подключится» к программе, а весь основной код, обрабатывающий объекты класса `Vehicle`, может остаться прежним.

Динамический полиморфизм и виртуальные функции «под капотом» компилятора

В этой главе мы рассмотрим, как устроены в языке C++ динамический полиморфизм, виртуальные функции и динамическое связывание. Стандарт C++ не определяет, каким образом они должны быть реализованы в компиляторе. Мы подробно расскажем об одном из возможных вариантов их реализации.

Мы начнем с традиционного введения в наследование и динамический полиморфизм, чтобы вы познакомились с основными идеями на базовом и среднем уровне и понимали, как работают эти технологии. Далее в этой главе (и в последующих главах) мы рассмотрим современные идиомы программирования.

Идиома неvirtуального интерфейса

В частности, мы покажем вам другой подход к динамическому полиморфизму: **идиому неvirtуального интерфейса** (**non-virtual interface idiom**, или сокращенно **NVI**), требующую, чтобы каждая функция базового класса предназначалась только для одной из двух целей:

- ♦ или эта функция вызывается клиентским кодом для выполнения задачи;
- ♦ или это функция, которую могут настраивать (адаптировать для своих целей) производные классы.

Настраиваемые функции — это внутренние детали реализации классов, облегчающие обслуживание и расширение цифровых систем без необходимости внесения изменений в код клиентских приложений.

Интерфейсы и внедрение зависимостей

Чтобы объяснить механику наследования, мы сосредоточили первые примеры этой главы на **наследовании реализации** (**implementation inheritance**), которое в основном используется для определения тесно связанных классов со многими одинаково реализованными элементами данных и функциями. Этот стиль наследования десятилетиями широко практиковался в объектно-ориентированном

программировании. Но со временем выявились его слабые стороны. Язык C++ используется для создания критически важных для бизнеса цифровых систем, в том числе очень больших. Эмпирические результаты многолетнего опыта создания систем, основанных на наследовании реализации, показывают, что их сложно модифицировать.

По этой причине мы затем проведем **рефакторинг (refactoring)**, то есть доработку одного из наших примеров: заменим наследование реализации на **наследование интерфейса (interface inheritance)**. Базовый класс не будет предоставлять никаких деталей реализации. Вместо этого он будет содержать «заполнители», которые сообщают производным классам, какие функции они должны реализовать. А производные классы будут предоставлять детали реализации: элементы данных и реализованные функции классов. В рамках этого примера мы представим **внедрение зависимостей (dependency injection)**: добавим в класс указатель на объект, обеспечивающий нужное поведение объектов класса (в нашем примере — вычисление зарплаты сотрудника). Этот указатель может быть переопределен. Например, если сотрудник получает повышение, мы можем изменить расчет зарплаты, просто перенаправив указатель на соответствующий объект. Вы поймете, почему этот доработанный код более удобен для модификации.

Динамический полиморфизм с помощью `std::variant` и `std::visit`

Затем мы реализуем динамический полиморфизм для объектов классов, которые не связаны наследованием и не имеют виртуальных функций. Для этого мы будем использовать шаблон класса `std::variant` (C++17) и функцию `std::visit` стандартной библиотеки. Вызов общей функциональности объектов, типы которых не связаны между собой, называется **утиной типизацией (duck typing)**.

Множественное наследование

Мы продемонстрируем **множественное наследование (multiple inheritance)**, которое позволяет классу наследовать элементы нескольких базовых классов. Мы рассмотрим потенциальные проблемы множественного наследования и обсудим их решения с помощью виртуального наследования.

Другие способы реализации полиморфизма

Наконец, мы рассмотрим другие методы динамического полиморфизма и несколько методов статического полиморфизма на основе шаблонов. Статический полиморфизм подробнее рассматривается в главе 15. Там же вы узнаете о введенных в C++20 **концептах (concepts)**, которые делают ненужными некоторые устаревшие методы программирования и расширяют инструментарий для решения ряда задач на этапе компиляции с помощью шаблонов. Мы также дадим ссылки на дополнительные ресурсы, которые помогут вам, если вы захотите глубже изучить эти темы.

Цель главы

Цель этой главы — познакомить вас с основными принципами наследования и динамического полиморфизма, чтобы вы оценили пользу современных идиом и методов полиморфизма, которые упрощают модификацию больших программ и повышают их быстродействие.

10.2. Базовые и производные классы

В следующей таблице приведены примеры базовых и производных классов. Базовые классы, как правило, являются более **общими (general)**, а производные классы — более **конкретными (specific)**.

Базовый класс	Производные классы
Student	GraduateStudent, UndergraduateStudent
Shape	Circle, Triangle, Rectangle, Sphere, Cube
Loan	CarLoan, HomeImprovementLoan, MortgageLoan
Vehicle	Car, Motorcycle, Boat
Employee	Faculty, Staff
Account	CheckingAccount, SavingsAccount

Каждый объект производного класса является также и объектом типа его базового класса. Один базовый класс может иметь много производных классов. Таким образом, то множество объектов, которое представляет базовый класс, почти всегда больше множества объектов, которые представляет производный класс. Например, базовый класс **Vehicle** представляет все транспортные средства, включая легковые и грузовые автомобили, лодки, самолеты, велосипеды и т. д. Для сравнения, производный класс **Car** представляет только легковые автомобили, то есть небольшое подмножество **Vehicle**.

10.2.1. Иерархия класса **CommunityMember**

Наследование естественным образом формирует иерархии классов. Базовый класс существует в иерархических отношениях с производными классами. Когда классы входят в иерархии наследования, они становятся связанными с другими классами¹. Класс может быть:

- ♦ базовым классом, предоставляющим элементы другим классам;
- ♦ производным классом, наследующим элементы от других классов;
- ♦ базовым и производным одновременно.

¹ Мы покажем, что тесно связанные классы могут затруднять модификацию систем. Мы рассмотрим альтернативы, позволяющие избежать тесной связи.

На следующей UML-диаграмме показан простой пример иерархии наследования. Такие диаграммы иллюстрируют отношения между классами в иерархии:



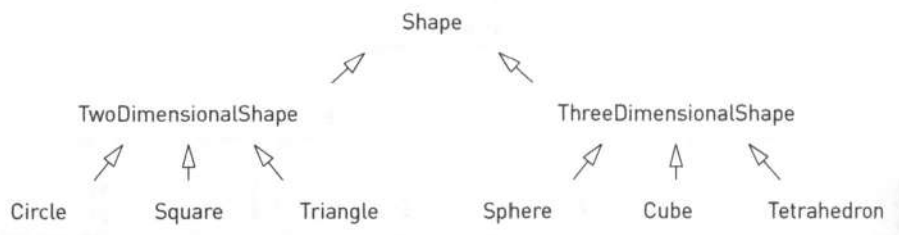
Иерархия университетского сообщества может состоять из нескольких тысяч объектов класса `CommunityMember`, таких как `Employee`, `Student` и `Alumnus`. В этой иерархии `Employee` — это `Faculty` или `Staff`, а `Faculty` — это `Administrator` или `Teacher`. Некоторые администраторы одновременно являются и преподавателями, то есть относятся к классу `AdministratorTeacher`. При одиноким (простом) наследовании производный класс наследуется от одного базового класса. При множественном наследовании производный класс наследуется от двух или более базовых классов. В этой иерархии есть пример множественного наследования: класс `AdministratorTeacher`.

Каждая стрелка на диаграмме обозначает связь «является» («is-a»). На основании этих стрелок мы можем утверждать, что «`Employee` является `CommunityMember`», а «`Teacher` является `Faculty`». `CommunityMember` — это **прямой базовый класс (direct base class)** для `Employee`, `Student` и `Alumnus`. `CommunityMember` — это **косвенный базовый класс (indirect base class)** для всех остальных классов иерархии. Косвенный базовый класс находится в иерархии как минимум на два уровня выше его производных классов.

Вы можете пройти по стрелкам вверх на несколько уровней, применяя отношение «является». Таким образом, `AdministratorTeacher` является `Administrator` (а также является `Teacher`), является `Faculty`, является `Employee` и является `CommunityMember`.

10.2.2. Иерархия класса `Shape` и открытое наследование

Давайте рассмотрим иерархию наследования класса `Shape`, представляющего геометрические фигуры. Эта иерархия начинается с базового класса `Shape`:



Классы `TwoDimensionalShape` и `ThreeDimensionalShape` происходят от `Shape`, поэтому `TwoDimensionalShape` является `Shape` и `ThreeDimensionalShape` тоже является `Shape`. Третий уровень иерархии конкретизирует классы `TwoDimensionalShape` и `ThreeDimensionalShape`. Вы можете проследить за стрелками, чтобы определить прямые и косвенные отношения «является». Например, `Triangle` является `TwoDimensionalShape` и `Shape`, а `Sphere` является `ThreeDimensionalShape` и `Shape`.

Следующее объявление класса указывает, что `TwoDimensionalShape` наследуется от `Shape`:

```
class TwoDimensionalShape : public Shape
```

Это **открытое наследование** (**public inheritance**), которое мы будем чаще всего использовать в этой главе. При открытом наследовании открытые (**public**) элементы базового класса становятся открытыми (**public**) элементами производного класса, а защищенные (**protected**) элементы базового класса становятся защищенными (**protected**) элементами производного класса. Защищенные элементы класса мы обсудим позже. Хотя из производных классов нет прямого доступа к скрытым (**private**) элементам базового класса, эти элементы тоже наследуются и являются частями объектов производных классов. Производный класс может оперировать скрытыми элементами базового класса с помощью унаследованных открытых и защищенных функций базового класса, если эти функции предоставляют такую возможность. Мы также рассмотрим скрытое (**private**) и защищенное (**protected**) наследование (раздел 10.16).

Наследование подходит не для всех отношений между классами. Иногда более уместно отношение «содержит» («**has-a**»). Например, если у нас есть классы `Employee`, `BirthDate` и `PhoneNumber`, было бы ошибкой сказать, что «`Employee` является `BirthDate`» или «`Employee` является `PhoneNumber`». В этом случае `Employee` содержит `BirthDate` и `PhoneNumber`.

Объекты базового класса и объекты производного класса можно рассматривать как аналогичные — их общие свойства выражены в элементах базового класса. Позже в этой главе мы рассмотрим примеры, использующие преимущества этой взаимосвязи.

10.3. Взаимосвязь между базовым и производным классами

В этом разделе мы продемонстрируем взаимосвязь между базовым и производным классами, используя иерархию наследования классов, представляющих две категории сотрудников в программе расчета зарплаты:

- ♦ сотрудникам базового класса выплачивается фиксированная еженедельная зарплата (оклад);
- ♦ сотрудникам производного класса, работающим в отделе продаж, выплачивается фиксированная еженедельная зарплата (оклад), а также комиссионное вознаграждение (в процентах от их продаж).

10.3.1. Объявление и использование класса `SalariedEmployee`

Давайте рассмотрим определение класса `SalariedEmployee` (илл. 10.1–10.2). В его заголовке (илл. 10.1) объявлены открытые функции класса, также называемые открытыми сервисами (*services*):

- ♦ конструктор (строка 9);
- ♦ функция `earnings` (строка 17);
- ♦ функция `toString` (строка 18);
- ♦ открытые *get*-функции и *set*-функции, оперирующие элементами данных `m_name` и `m_salary` (объявленными в строках 20–21).

Функция `setSalary` (илл. 10.2, строки 22–28) проверяет свой аргумент перед изменением элемента данных `m_salary`.

```
1 // Fig. 10.1: SalariedEmployee.h
2 // Определение класса SalariedEmployee
3 #pragma once // Предотвращаем повторные включения заголовка
4 #include <string>
5 #include <string_view>
6
7 class SalariedEmployee {
8 public:
9     SalariedEmployee(std::string_view name, double salary);
10
11     void setName(std::string_view name);
12     std::string getName() const;
13
14     void setSalary(double salary);
15     double getSalary() const;
16
```

Окончание ⇨

```

17 double earnings() const;
18 std::string toString() const;
19 private:
20     std::string m_name{};
21     double m_salary{0.0};
22 };

```

Илл. 10.1 | Определение класса SalariedEmployee

```

1 // Fig. 10.2: SalariedEmployee.cpp
2 // Определения функций класса SalariedEmployee
3 #include <fmt/format.h>
4 #include <stdexcept>
5 #include "SalariedEmployee.h" // Определение класса SalariedEmployee
6
7 // Конструктор
8 SalariedEmployee::SalariedEmployee(std::string_view name, double salary)
9     : m_name{name} {
10     setSalary(salary);
11 }
12
13 // Записываем имя сотрудника
14 void SalariedEmployee::setName(std::string_view name) {
15     m_name = name; // Нужна проверка
16 }
17
18 // Возвращаем имя сотрудника
19 std::string SalariedEmployee::getName() const {return m_name;}
20
21 // Записываем оклад
22 void SalariedEmployee::setSalary(double salary) {
23     if (salary < 0.0) {
24         throw std::invalid_argument("Salary must be >= 0.0");
25     }
26
27     m_salary = salary;
28 }
29
30 // Возвращаем оклад
31 double SalariedEmployee::getSalary() const {return m_salary;}
32
33 // Рассчитываем зарплату
34 double SalariedEmployee::earnings() const {return getSalary();}
35
36 // Возвращаем строковое представление объекта SalariedEmployee
37 std::string SalariedEmployee::toString() const {
38     return fmt::format("name: {} salary: {:.2f}\n", getName(),
39         getSalary());
40 }

```

Илл. 10.2 | Определения функций класса SalariedEmployee

Конструктор класса `SalariedEmployee`

Конструктор класса (илл. 10.2, строки 8–11) использует список инициализаторов для инициализации элемента данных `m_name`, предназначенного для имени сотрудника. Мы могли бы проверить это имя (например, убедиться, что оно имеет приемлемую длину). Конструктор вызывает функцию `setSalary` для проверки и инициализации элемента данных `m_salary`.

Функции `earnings` и `toString` класса `SalariedEmployee`

Функция `earnings` (строка 34) вызывает функцию `getSalary` и возвращает результат. Функция `toString` (строки 37–40) возвращает строку, содержащую информацию об объекте `SalariedEmployee`.

Тест-драйв класса `SalariedEmployee`

На илл. 10.3 показана программа для тест-драйва класса `SalariedEmployee`. В строке 9 создается объект класса `SalariedEmployee` с именем `employee`. В строках 12–14 демонстрируются `get`-функции созданного объекта. В строке 16 функция `setSalary` используется для изменения значения `m_salary` объекта `employee`. Затем в строках 17–18 вызывается функция `toString` для получения и вывода обновленных данных из объекта `employee`. Наконец, в строке 21 отображается обновленное значение зарплаты сотрудника.

```
1 // fig10_03.cpp
2 // Тест-драйв класса SalariedEmployee
3 #include <fmt/format.h>
4 #include <iostream>
5 #include "SalariedEmployee.h" // Определение класса SalariedEmployee
6
7 int main() {
8     // Создаем объект класса SalariedEmployee
9     SalariedEmployee employee{"Sue Jones", 300.0};
10
11     // Получаем данные из объекта
12     std::cout << "Employee information obtained by get functions:\n"
13               << fmt::format("name: {}\nsalary: ${:.2f}\n", employee.getName(),
14                             employee.getSalary());
15
16     employee.setSalary(500.0); // Изменяем оклад
17     std::cout << "\nUpdated employee information from function toString:\n"
18               << employee.toString();
19
20     // Выводим на экран зарплату сотрудника
21     std::cout << fmt::format("\nearnings: ${:.2f}\n", employee.earnings());
22 }
```

```
Employee information obtained by get functions:
name: Sue Jones
salary: $300.00
```



```
Updated employee information from function toString:
name: Sue Jones
salary: $500.00
earnings: $500.00
```

Илл. 10.3 | Тест-драйв класса `SalariedEmployee`

10.3.2. Создание иерархии наследования: `SalariedEmployee` и `SalariedCommissionEmployee`

Теперь давайте создадим класс `SalariedCommissionEmployee` (илл. 10.4–10.5), унаследованный от `SalariedEmployee` (илл. 10.1–10.2). В этом примере класс `SalariedCommissionEmployee` наследует открытые (`public`) элементы класса `SalariedEmployee`. В дополнение к ним класс `SalariedCommissionEmployee` содержит элементы данных `m_grossSales` и `m_commissionRate` (илл. 10.4, строки 22–23), которые мы перемножим для расчета комиссионного вознаграждения. В строках 13–17 объявляются открытые `get`-функции и `set`-функции, которые оперируют элементами данных `m_grossSales` и `m_commissionRate` класса `SalariedCommissionEmployee`.

```
1 // Fig. 10.4: SalariedCommissionEmployee.h
2 // Класс SalariedCommissionEmployee, производный от SalariedEmployee
3 #pragma once // Предотвращаем повторные включения заголовка
4 #include <string>
5 #include <string_view>
6 #include "SalariedEmployee.h"
7
8 class SalariedCommissionEmployee : public SalariedEmployee {
9 public:
10     SalariedCommissionEmployee(std::string_view name, double salary,
11         double grossSales, double commissionRate);
12
13     void setGrossSales(double grossSales);
14     double getGrossSales() const;
15
16     void setCommissionRate(double commissionRate);
17     double getCommissionRate() const;
18
19     double earnings() const;
20     std::string toString() const;
21 private:
22     double m_grossSales{0.0};
23     double m_commissionRate{0.0};
24 };
```

Илл. 10.4 | Определение класса `SalariedCommissionEmployee`, унаследованного от `SalariedEmployee`

Наследование

Двоеточие (:) в строке 8 означает наследование, а слово `public` справа от него означает, что наследование открытое. При открытом наследовании открытые элементы базового класса становятся открытыми элементами производного класса, а защищенные (мы рассмотрим их позже) — защищенными. Класс `SalariedCommissionEmployee` наследует все элементы класса `SalariedEmployee`, кроме конструктора и деструктора. Конструкторы и деструкторы специфичны для каждого класса, поэтому в производных классах должны быть собственные конструкторы и деструкторы¹.

Функции класса `SalariedCommissionEmployee`

Класс `SalariedCommissionEmployee` содержит открытые функции (открытые сервисы):

- ♦ собственный конструктор (строки 10–11);
- ♦ `setGrossSales`, `getGrossSales`, `setCommissionRate`, `getCommissionRate`, `earnings` и `toString` (строки 13–20);
- ♦ открытые функции, унаследованные от класса `SalariedEmployee`.

Обратите внимание: этих унаследованных функций нет в исходном коде класса `SalariedCommissionEmployee`, тем не менее они являются частями этого класса. Объект `SalariedCommissionEmployee` также содержит скрытые элементы класса `SalariedEmployee`, но они недоступны напрямую в производном классе. Вы можете получить к ним доступ только через унаследованные открытые (или защищенные) функции класса `SalariedEmployee`.

Реализация `SalariedCommissionEmployee`

На илл. 10.5 показана реализация функций класса `SalariedCommissionEmployee`. Каждый конструктор производного класса должен вызывать конструктор базового класса. Мы делаем это явно с помощью **инициализатора базового класса (base-class initializer)** (строка 11), который передает аргументы конструктору базового класса. В конструкторе класса `SalariedCommissionEmployee` (строки 8–15) в строке 11 вызывается конструктор класса `SalariedEmployee` для инициализации унаследованных элементов данных аргументами `name` и `salary`. Инициализатор базового класса вызывает конструктор базового класса по имени. Функции `setGrossSales` (строки 18–24) и `setCommissionRate` (строки 32–41) проверяют свои аргументы перед изменением элементов `m_grossSales` и `m_commissionRate`.

¹ Обычно конструктор производного класса просто передает свои аргументы соответствующему конструктору базового класса без дополнительных действий.

```
1 // Fig. 10.5: SalariedCommissionEmployee.cpp
2 // Определения функций класса SalariedCommissionEmployee
3 #include <fmt/format.h>
4 #include <stdexcept>
5 #include "SalariedCommissionEmployee.h"
6
7 // Конструктор
8 SalariedCommissionEmployee::SalariedCommissionEmployee(
9     std::string_view name, double salary, double grossSales,
10     double commissionRate)
11     : SalariedEmployee{name, salary} { // Вызываем конструктор базового класса
12
13     setGrossSales(grossSales); // Объем продаж
14     setCommissionRate(commissionRate); // Комиссионная ставка
15 }
16
17 // Записываем объем продаж
18 void SalariedCommissionEmployee::setGrossSales(double grossSales) {
19     if (grossSales < 0.0) {
20         throw std::invalid_argument("Gross sales must be >= 0.0");
21     }
22
23     m_grossSales = grossSales;
24 }
25
26 // Возвращаем объем продаж
27 double SalariedCommissionEmployee::getGrossSales() const {
28     return m_grossSales;
29 }
30
31 // Возвращаем комиссионную ставку
32 void SalariedCommissionEmployee::setCommissionRate(
33     double commissionRate) {
34
35     if (commissionRate <= 0.0 || commissionRate >= 1.0) {
36         throw std::invalid_argument(
37             "Commission rate must be > 0.0 and < 1.0");
38     }
39
40     m_commissionRate = commissionRate;
41 }
42
43 // Получаем комиссионную ставку
44 double SalariedCommissionEmployee::getCommissionRate() const {
45     return m_commissionRate;
46 }
47
48 // Рассчитываем зарплату с помощью функции SalariedEmployee::earnings()
49 double SalariedCommissionEmployee::earnings() const {
50     return SalariedEmployee::earnings() +
51         getGrossSales() * getCommissionRate();
52 }
```

```
52 }
53
54 // Возвращаем строковое представление объекта SalariedCommissionEmployee
55 std::string SalariedCommissionEmployee::toString() const {
56     return fmt::format(
57         "{}gross sales: ${:.2f}\\ncommission rate: {:.2f}\\n",
58         SalariedEmployee::toString(), getGrossSales(), getCommissionRate());
59 }
```

Илл. 10.5 | Определения функций класса `SalariedCommissionEmployee`

Функция `earnings` класса `SalariedCommissionEmployee`

Функция `earnings` класса `SalariedCommissionEmployee` (строки 49–52) переопределяет функцию `earnings` класса `SalariedEmployee` (илл. 10.2, строка 34) для расчета зарплаты сотрудников отдела продаж. В строке 50 в версии класса `SalariedCommissionEmployee` мы используем выражение:

```
SalariedEmployee::earnings()
```

чтобы получить значение зарплаты, основанное на окладе, а затем прибавить к нему комиссионное вознаграждение для расчета итоговой суммы.

Для вызова переопределенной функции базового класса из производного класса надо поместить перед ее именем имя базового класса и оператор разрешения области видимости (`::`). Префикс `SalariedEmployee::` нужен, чтобы избежать бесконечной рекурсии. Кроме того, вызывая функцию класса `SalariedEmployee` из функции класса `SalariedCommissionEmployee`, мы избегаем дублирования кода¹.

Функция `toString` класса `SalariedCommissionEmployee`

Аналогично функция `toString` класса `SalariedCommissionEmployee` (илл. 10.5, строки 55–59) переопределяет функцию `toString` класса `SalariedEmployee` (илл. 10.2, строки 37–40). Новая версия возвращает строку, содержащую:

- ♦ результат вызова `SalariedEmployee::toString()` (илл. 10.5, строка 58);
- ♦ объем продаж и комиссионную ставку.

Тест-драйв класса `SalariedCommissionEmployee`

На илл. 10.6 показано создание объекта `employee` класса `SalariedCommissionEmployee` (строка 9). В строках 12–16 мы отображаем данные объекта `employee`, вызывая его `get`-функции для получения значений элементов данных. В строках 18–19 используются функции `setGrossSales` и `setCommissionRate` для изменения значений `m_grossSales` и `m_commissionRate` соответственно. Затем в строках 20–21

¹ С одной стороны, отсутствие дубликата кода является преимуществом. С другой стороны, такой подход создает связь между базовым и производными классами, что может затруднить модификацию крупномасштабных систем.

вызывается функция `toString`, чтобы отобразить обновленные данные объекта `employee`. Наконец, в строке 24 отображается обновленная зарплата.

```

1 // fig10_06.cpp
2 // Тест-драйв класса SalariedCommissionEmployee
3 #include <fmt/format.h>
4 #include <iostream>
5 #include "SalariedCommissionEmployee.h"
6
7 int main() {
8     // Создаем объект класса SalariedCommissionEmployee
9     SalariedCommissionEmployee employee("Bob Lewis", 300.0, 5000.0, .04);
10
11     // Получаем данные из объекта
12     std::cout << "Employee information obtained by get functions:\n"
13         << fmt::format("{}: {}\n{}: {:.2f}\n{}: {:.2f}\n{}: {:.2f}\n",
14             "name", employee.getName(), "salary", employee.getSalary(),
15             "gross sales", employee.getGrossSales(),
16             "commission", employee.getCommissionRate());
17
18     employee.setGrossSales(8000.0); // Изменяем объем продаж
19     employee.setCommissionRate(0.1); // Изменяем комиссионную ставку
20     std::cout << "\nUpdated employee information from function toString:\n"
21         << employee.toString();
22
23     // Выводим на экран зарплату сотрудника
24     std::cout << fmt::format("\nearnings: ${:.2f}\n", employee.earnings());
25 }

```

Employee information obtained by get functions:

name: Bob Lewis
 salary: \$300.00
 gross sales: \$5000.00
 commission: 0.04

Updated employee information from function toString:

name: Bob Lewis
 salary: \$300.00
 gross sales: \$8000.00
 commission rate: 0.10
 earnings: \$1100.00

Илл. 10.6 | Тест-драйв класса `SalariedCommissionEmployee`

Конструктор производного класса должен вызывать конструктор базового класса

Компилятор выдаст ошибку, если конструктор класса `SalariedCommissionEmployee` явно не вызывает конструктор класса `SalariedEmployee`. В этом случае компилятор попытается вызвать конструктор класса `SalariedEmployee` по умолчанию, которого не существует, поскольку в базовом классе явно определен пользовательский

конструктор. Если базовый класс предоставляет конструктор по умолчанию, конструктор производного класса может неявно его вызывать.

Дополнительные сведения о конструкторах в производных классах

Конструктор производного класса должен вызывать конструктор базового класса с требуемыми аргументами, иначе возникает ошибка компиляции. Это гарантирует, что унаследованные скрытые элементы базового класса, к которым производный класс не может получить прямой доступ, будут инициализированы. Инициализаторы данных производного класса обычно размещаются в списке инициализаторов после одного или нескольких инициализаторов базового класса.

Из производного класса нет прямого доступа к скрытым элементам базового класса

C++ накладывает строгие ограничения на доступ к скрытым элементам данных. Даже производный класс, тесно связанный со своим базовым классом, не может получить прямой доступ к его скрытым данным. Например, скрытый элемент данных `m_salary` класса `SalariedEmployee` хоть и является частью каждого объекта класса `SalariedCommissionEmployee`, но недоступен напрямую функциям класса `SalariedCommissionEmployee`. При попытке прямого доступа к скрытым данным компилятор выдаст сообщение об ошибке. Например, компилятор GNU g++ сообщит:

```
'double SalariedEmployee::m_salary' is private within this context
```

Однако, как показано на илл. 10.5, функции класса `SalariedCommissionEmployee` могут обращаться напрямую к открытым элементам, унаследованным от класса `SalariedEmployee`.

Заголовок базового класса в заголовке производного класса

Обратите внимание, что мы включили заголовок базового класса в заголовок производного класса (илл. 10.4, строка 6). Это необходимо по следующим причинам:

- ♦ Для того чтобы выполнить наследование (илл. 10.4, строка 8), компилятору требуется определение базового класса из `SalariedEmployee.h`.
- ♦ Компилятор рассчитывает размер объекта по определению класса. Чтобы создать объект, компилятор должен знать определение класса для резервирования необходимого объема памяти. Размер объекта производного класса зависит от элементов данных, явно объявленных в определении класса, а также от элементов данных, унаследованных от прямых и косвенных базовых классов¹. Включение определения базового класса позволяет компилятору зарезервировать память с учетом всех элементов данных, влияющих на итоговый размер объекта.

¹ Все объекты класса совместно используют одну копию функций класса, которая хранится отдельно от объектов и не учитывается при расчете их размера.

- ✦ Определение базового класса позволяет компилятору проверить правильность использования производным классом унаследованных элементов базового класса. Например, компилятор использует прототипы функций базового класса для проверки вызовов унаследованных функций базового класса из функций производного класса.

Наследование реализации предотвращает повторы кода

При наследовании реализации базовый класс объявляет элементы данных и функции, общие для всех классов иерархии. Когда надо изменить какой-то из этих общих элементов, вы вносите изменения только в базовый класс. Затем производные классы наследуют эти изменения и должны быть перекомпилированы. Без наследования вам пришлось бы вносить изменения во все файлы исходного кода, содержащие одинаковые фрагменты.

10.4. Конструкторы и деструкторы в производных классах

Порядок вызовов конструкторов

Когда создается объект производного класса, начинается цепочка вызовов конструкторов. Перед выполнением своих собственных задач конструктор производного класса вызывает конструктор базового класса: или явно, через инициализатор элемента базового класса, или неявно, вызывая конструктор базового класса по умолчанию. Последний конструктор в этой цепочке — конструктор класса на вершине иерархии. Он завершает выполнение первым, а конструктор самого нижнего производного класса завершает выполнение последним.

Каждый конструктор базового класса инициализирует элементы данных базового класса, наследуемые производными классами. В иерархии классов `SalariedEmployee` — `SalariedCommissionEmployee`, которую мы рассмотрели ранее, при создании объекта `SalariedCommissionEmployee`:

- ✦ конструктор этого объекта сразу вызывает конструктор базового класса `SalariedEmployee`;
- ✦ класс `SalariedEmployee` является базовым классом в этой иерархии, поэтому выполняется его конструктор, инициализирующий унаследованные объектом `SalariedCommissionEmployee` элементы данных `m_name` и `m_salary`;
- ✦ конструктор класса `SalariedEmployee` возвращает управление конструктору объекта класса `SalariedCommissionEmployee` для инициализации элементов данных `m_grossSales` и `m_commissionRate`.

Порядок вызовов деструкторов

Когда уничтожается объект производного класса, программа вызывает деструктор этого объекта, начиная цепочку вызовов деструкторов. Деструкторы выполняются в обратном порядке по сравнению с конструкторами. Когда вызывается деструктор

объекта производного класса, он выполняется и вызывает деструктор следующего класса в иерархии. Это повторяется, пока не будет вызван деструктор базового класса на вершине иерархии.

Конструкторы и деструкторы составных объектов

Предположим, мы создаем объект производного класса, причем базовый и производный классы содержат объекты других классов. Когда создается объект производного класса:

- ♦ выполняются конструкторы для объектов-элементов базового класса в том порядке, в каком они объявлены;
- ♦ затем выполняется конструктор базового класса;
- ♦ затем выполняются конструкторы для объектов-элементов производного класса в том порядке, в каком эти объекты объявлены в производном классе;
- ♦ затем выполняется конструктор производного класса.

Деструкторы для элементов данных вызываются в обратном порядке по сравнению с соответствующими конструкторами.

10.5. Принцип динамического полиморфизма на примере объектов в видеоигре

Предположим, мы пишем видеоигру со следующими производными классами: `Martian`, `Venusian`, `Plutonian`, `SpaceShip` и `LaserBeam`. Каждый из этих производных классов наследуется от базового класса `SpaceObject` с функцией `draw` и реализует эту функцию по-своему.

Менеджер экрана

Программа, управляющая выводом на экран (менеджер экрана), работает с вектором (`vector`) указателей `SpaceObject`, указывающих на объекты разных классов. Чтобы обновить изображение, менеджер экрана периодически отправляет каждому объекту одно и то же сообщение через функцию `draw`. Каждый объект реагирует по-своему, например:

- ♦ `Martian` рисует себя в виде красного существа с антеннами;
- ♦ `SpaceShip` рисует себя в виде серебристой летающей тарелки;
- ♦ `LaserBeam` рисует себя в виде ярко-красного луча.

Одно и то же сообщение `draw` вызывает разные формы поведения — это и есть **полиморфизм (polymorphism)**¹.

¹ Это пример динамического полиморфизма (*dynamic polymorphism*), который также называется полиморфизмом времени выполнения (*runtime polymorphism*), так как выбор поведения происходит во время выполнения программы. — *Примеч. ред.*

Добавление новых классов в систему

Полиморфный менеджер экрана позволяет легко добавлять в программу новые классы с минимальными изменениями кода. Предположим, вы хотите добавить в игру объекты `Mercurian`. Вы создаете класс `Mercurian`, который наследуется от `SpaceObject` и определяет собственную функцию `draw`, а затем добавляете адрес объекта этого класса в вектор указателей `SpaceObject`. Код менеджера экрана вызывает функцию `draw` одинаково для всех перечисленных в векторе объектов, независимо от их типа. Таким образом, новые объекты `Mercurian` просто «подключаются» к программе.

Благодаря динамическому полиморфизму, вы можете практически без изменений в программе (кроме создания объектов новых классов) добавлять классы по мере необходимости, в том числе те, которые вы не предусмотрели заранее, при разработке программы.

Благодаря динамическому полиморфизму, вы можете работать с общими понятиями, а C++ будет заботиться о конкретных деталях. Вы можете обеспечить правильное поведение объектов, даже не зная их типов, если они принадлежат к одной иерархии классов и обращения к ним выполняются через ссылки или указатели их базовых классов¹.

Динамический полиморфизм способствует расширяемости программ. Полиморфный код создается независимо от конкретных типов объектов, которым вы отправляете сообщения. Таким образом, новые типы объектов, способные обрабатывать существующие сообщения, просто «подключаются» к программе без изменения основной части кода. Для использования новых типов вам надо только модифицировать клиентский код, создающий новые объекты.

10.6. Отношения между объектами в иерархии наследования

В разделе 10.3 мы создали иерархию классов `SalariedEmployee` — `SalariedCommissionEmployee`. Давайте подробнее рассмотрим отношения, существующие в иерархии наследования. На нескольких примерах мы покажем, как направлять указатели базовых и производных классов на объекты базовых и производных классов, а затем вызывать через эти указатели функции для операций с объектами:

- ♦ В разделе 10.6.1 мы присвоим адрес объекта производного класса указателю базового класса, а затем покажем, что при вызове функции через указатель базового класса вызывается функциональность базового класса в объекте производного класса. Тип дескриптора определяет, какая из функций будет вызвана.
- ♦ В разделе 10.6.2 мы присвоим адрес объекта базового класса указателю производного класса, чтобы вызвать ошибку компиляции. Мы обсудим сообще-

¹ Далее мы рассмотрим такие формы динамического полиморфизма и статического полиморфизма, которые не зависят от иерархий наследования.

ние об ошибке и объясним, почему компилятор не разрешает такие присваивания.

- ♦ В разделе 10.6.3 мы присвоим адрес объекта производного класса указателю базового класса, а затем убедимся, что через указатель базового класса можно вызывать только функции, существующие в базовом классе. При попытке вызвать через указатель базового класса функцию, которая есть только в производном классе, возникнет ошибка компиляции.
- ♦ В разделе 10.7 мы представим вам **виртуальные функции (virtual functions)**, чтобы показать, как динамический полиморфизм реализуется с помощью указателей базового класса, направленных на объекты производного класса. Мы присвоим адрес объекта производного класса указателю базового класса и вызовем через этот указатель функции производного класса — это и есть полиморфное поведение.

Перечисленные примеры демонстрируют открытое (`public`) наследование, при котором объект производного класса может также рассматриваться и как объект базового класса. В результате вы получаете много интересных возможностей. Например, вы можете создать вектор указателей базового класса, которые указывают на объекты производных классов многих разных типов. Компилятор допускает это, потому что каждый объект производного класса является также и объектом своего базового класса.

С другой стороны, нельзя рассматривать объект базового класса как объект производного класса. Например, объект класса `SalariedEmployee` не является объектом класса `SalariedCommissionEmployee`. В классе `SalariedEmployee` нет элементов данных `m_grossSales`, `m_commissionRate` и соответствующих *set*-функций и *get*-функций. Отношение «является» применимо только в одну сторону: от производного класса к его прямому базовому классу и косвенным базовым классам.

10.6.1. Вызов функций базового класса из объектов производного класса

В программе, показанной на илл. 10.7, мы повторно используем классы `SalariedEmployee` и `SalariedCommissionEmployee` из разделов 10.3.1–10.3.2. Этот пример демонстрирует присваивание указателей базового и производного классов объектам базового и производного классов. Первые два случая логичны и просты:

- ♦ мы направляем указатель базового класса на объект базового класса и вызываем функциональность базового класса;
- ♦ мы направляем указатель производного класса на объект производного класса и вызываем функциональность производного класса.

Затем мы демонстрируем отношение «является» между производным и базовым классами, направляя указатель базового класса на объект производного класса и показывая, что функциональность базового класса доступна в объекте производного класса.

```
1 // fig10_07.cpp
2 // Присваивание адресов объектов базового и производного классов
3 // указателям базового и производного классов
4 #include <fmt/format.h>
5 #include <iostream>
6 #include "SalariedEmployee.h"
7 #include "SalariedCommissionEmployee.h"
8
9 int main() {
10     // Создаем объект базового класса
11     SalariedEmployee salaried{"Sue Jones", 500.0};
12
13     // Создаем объект производного класса
14     SalariedCommissionEmployee salariedCommission{
15         "Bob Lewis", 300.0, 5000.0, .04};
16
17     // Выводим на экран объекты salaried и salariedCommission
18     std::cout << fmt::format("{}:\n{}\n{}\n",
19         "DISPLAY BASE-CLASS AND DERIVED-CLASS OBJECTS",
20         salaried.toString(), // Функция toString базового класса
21         salariedCommission.toString()); // Функция toString производного класса
22
23     // Направляем указатель базового класса на объект базового класса
24     SalariedEmployee* salariedPtr(&salaried);
25     std::cout << fmt::format("{}:\n{}\n{}\n",
26         "CALLING TOSTRING WITH BASE-CLASS POINTER TO",
27         "BASE-CLASS OBJECT INVOKES BASE-CLASS FUNCTIONALITY",
28         salariedPtr->toString()); // Версия из базового класса
29
30     // Направляем указатель производного класса на объект производного класса
31     SalariedCommissionEmployee* salariedCommissionPtr(&salariedCommission);
32
33     std::cout << fmt::format("{}:\n{}\n{}\n",
34         "CALLING TOSTRING WITH DERIVED-CLASS POINTER TO",
35         "DERIVED-CLASS OBJECT INVOKES DERIVED-CLASS FUNCTIONALITY",
36         salariedCommissionPtr->toString()); // Версия из производного класса
37
38     // Направляем указатель базового класса на объект производного класса
39     salariedPtr = &salariedCommission;
40     std::cout << fmt::format("{}:\n{}\n{}\n",
41         "CALLING TOSTRING WITH BASE-CLASS POINTER TO DERIVED-CLASS",
42         "OBJECT INVOKES BASE-CLASS FUNCTIONALITY",
43         salariedPtr->toString()); // Версия из базового класса
44 }
```

DISPLAY BASE-CLASS AND DERIVED-CLASS OBJECTS:

name: Sue Jones
salary: \$500.00

name: Bob Lewis
salary: \$300.00
gross sales: \$5000.00

```
commission rate: 0.04

CALLING TOSTRING WITH BASE-CLASS POINTER TO
BASE-CLASS OBJECT INVOKES BASE-CLASS FUNCTIONALITY:
name: Sue Jones
salary: $500.00

CALLING TOSTRING WITH DERIVED-CLASS POINTER TO
DERIVED-CLASS OBJECT INVOKES DERIVED-CLASS FUNCTIONALITY:
name: Bob Lewis
salary: $300.00
gross sales: $5000.00
commission rate: 0.04

CALLING TOSTRING WITH BASE-CLASS POINTER TO DERIVED-CLASS
OBJECT INVOKES BASE-CLASS FUNCTIONALITY:
name: Bob Lewis
salary: $300.00
```

Илл. 10.7 | Присваивание адресов объектов базового и производного классов указателям базового и производного классов

Напомним, что объект `SalariedCommissionEmployee` является объектом `SalariedEmployee`, но с добавлением комиссионного вознаграждения за продажи. Функция `earnings` класса `SalariedCommissionEmployee` (илл. 10.5, строки 49–52) **переопределяет** (*redefines*) версию класса `SalariedEmployee` (илл. 10.2, строка 34), чтобы прибавить к окладу комиссионное вознаграждение. Функция `toString` класса `SalariedCommissionEmployee` (илл. 10.5, строки 55–59) переопределяет версию класса `SalariedEmployee` (илл. 10.2, строки 37–40), возвращая ту же информацию и комиссионное вознаграждение сотрудника.

Создание объектов и отображение их данных

На илл. 10.7 в строке 11 создается объект `SalariedEmployee`, а в строках 14–15 создается объект `SalariedCommissionEmployee`. В строках 20 и 21 имя каждого объекта используется для вызова его функции `toString`.

Направление указателя базового класса на объект базового класса

В строке 24 указатель `salariedPtr` инициализируется адресом объекта базового класса `salaried`. В строке 28 этот указатель используется для вызова функции `toString` объекта `salaried` из базового класса `SalariedEmployee`.

Направление указателя производного класса на объект производного класса

В строке 31 указатель `salariedCommissionPtr` инициализируется адресом объекта производного класса `salariedCommission`. В строке 36 этот указатель используется для вызова функции `toString` объекта `salariedCommission` из производного класса `SalariedCommissionEmployee`.

Направление указателя базового класса на объект производного класса

В строке 39 адрес объекта производного класса `salariedCommission` присваивается указателю базового класса `salariedPtr`. Такое «пересечение» разрешено, потому что объект производного класса является также и объектом своего базового класса. В строке 43 этот указатель используется для вызова функции `toString`. Несмотря на то что указатель базового класса указывает на объект производного класса, вызывается функция базового класса. Объем продаж и комиссионная ставка не отображаются, поскольку они не являются элементами данных базового класса.

Выходные данные этой программы показывают, что выбор вызываемой функции зависит от типа указателя (или от типа ссылки, как вы скоро увидите), через который она вызывается, а не от типа объекта, для которого она вызывается. В разделе 10.7 мы покажем, что виртуальные функции позволяют выбирать нужную функциональность в зависимости от типа объекта, — это важный аспект реализации динамического полиморфного поведения.

10.6.2. Направление указателя производного класса на объект базового класса

Теперь давайте попробуем направить указатель производного класса на объект базового класса (илл. 10.8). В строке 7 мы создаем объект класса `SalariedEmployee`. В строке 11 мы инициализируем указатель `SalariedCommissionEmployee` адресом объекта `salaried` базового класса. Компилятор сообщает об ошибке, поскольку `SalariedEmployee` не является `SalariedCommissionEmployee`.

```
1 // fig10_08.cpp
2 // Присваивание адреса объекта базового класса указателю производного класса!
3 #include "SalariedEmployee.h"
4 #include "SalariedCommissionEmployee.h"
5
6 int main() {
7     SalariedEmployee salaried("Sue Jones", 5000.0);
8
9     // Направляем указатель производного класса на объект базового класса
10    // Ошибка: SalariedEmployee не является SalariedCommissionEmployee!
11    SalariedCommissionEmployee* salariedCommissionPtr(&salaried);
12 }
```

Сообщение компилятора Microsoft Visual C++ об ошибке:

```
fig10_08.cpp(11,63): error C2440: 'initializing': cannot convert from
'SalariedEmployee *' to 'SalariedCommissionEmployee *'
```

Илл. 10.8 | Ошибочное присваивание адреса объекта базового класса указателю производного класса

Подумайте, что было бы, если бы компилятор разрешил такое присваивание. Через указатель `SalariedCommissionEmployee` можно вызвать любую функцию этого

класса (в том числе `setGrossSales` и `setCommissionRate`) для объекта, на который направлен указатель, то есть для объекта `salaried` базового класса. Но в объекте `SalariedEmployee` нет ни функций `setGrossSales` и `setCommissionRate`, ни элементов данных `m_grossSales` и `m_commissionRate`. Из-за такого присваивания возникли бы проблемы, поскольку функции `setGrossSales` и `setCommissionRate` предполагали бы, что элементы данных `m_grossSales` и `m_commissionRate` расположены «на своих обычных местах» внутри объекта `SalariedCommissionEmployee`. Но в памяти объекта `SalariedEmployee` нет этих элементов данных, поэтому функции `setGrossSales` и `setCommissionRate` повредили бы другие данные в памяти.

10.6.3. Вызовы функций производного класса через указатель базового класса

Компилятор позволяет вызывать через указатель базового класса только функции базового класса. Если указатель базового класса указывает на объект производного класса, то при обращении к функции, существующей только в производном классе, возникает ошибка компиляции. На илл. 10.9 показаны сообщения компилятора при вызове функции производного класса через указатель базового класса (строки 22–24).

```

1 // fig10_09.cpp
2 // Вызовы функций, существующих только в производном классе,
3 // через указатель базового класса!
4 #include <string>
5 #include "SalariedEmployee.h"
6 #include "SalariedCommissionEmployee.h"
7
8 int main() {
9     SalariedCommissionEmployee salariedCommission{
10         "Bob Lewis", 300.0, 5000.0, .04};
11
12     // Направляем указатель базового класса на объект производного класса
13     SalariedEmployee* salariedPtr{&salariedCommission};
14
15     // Вызываем функции базового класса для объекта производного класса
16     // через указатель базового класса
17     std::string name{salariedPtr->getName()};
18     double salary{salariedPtr->getSalary()};
19
20     // Вызываем функции, существующие только в производном классе, для объекта
21     // производного класса через указатель базового класса (запрещено!)
22     double grossSales{salariedPtr->getGrossSales()};
23     double commissionRate{salariedPtr->getCommissionRate()};
24     salariedPtr->setGrossSales(8000.0);
25 }
```

Сообщения компилятора GNU C++ об ошибках:

```
fig10_09.cpp: In function 'int main()':
```

```

fig10_09.cpp:22:35: error: 'class SalariedEmployee' has no member named
'getGrossSales'
    22 | double grossSales{salariedPtr->getGrossSales()};
        |                               ^~~~~~
fig10_09.cpp:23:39: error: 'class SalariedEmployee' has no member named
'getCommissionRate'
    23 | double commissionRate{salariedPtr->getCommissionRate()};
        |                               ^~~~~~
fig10_09.cpp:24:17: error: 'class SalariedEmployee' has no member named
'setGrossSales'
    24 | salariedPtr->setGrossSales(8000.0);
        |               ^~~~~~

```

Илл. 10.9 | Ошибочные вызовы функций, существующих только в производном классе, через указатель базового класса

В строках 9–10 создается объект класса `SalariedCommissionEmployee`. В строке 13 указатель базового класса `salariedPtr` инициализируется адресом объекта производного класса `salariedCommission`. Это разрешено, потому что `SalariedCommissionEmployee` является `SalariedEmployee`.

В строках 17–18 функции базового класса вызываются через указатель базового класса. Эти вызовы разрешены, потому что `SalariedCommissionEmployee` наследует эти функции.

Мы знаем, что `salariedPtr` указывает на объект `SalariedCommissionEmployee`. В строках 22–24 мы пробуем вызывать три функции, которые есть только в `SalariedCommissionEmployee`: `getGrossSales`, `getCommissionRate` и `setGrossSales`. Компилятор сообщает об ошибках, поскольку этих функций нет в базовом классе `SalariedEmployee`. Через указатель `salariedPtr` можно вызывать только функции базового класса `SalariedEmployee`.

10.7. Виртуальные функции и виртуальные деструкторы

В разделе 10.6.1 мы создали указатель базового класса `SalariedEmployee`, указывающий на объект производного класса `SalariedCommissionEmployee`, и использовали его для вызова функции `toString`. В этом случае была вызвана функция класса `SalariedEmployee`. Как же вызвать функцию `toString` производного класса через указатель базового класса?

10.7.1. Зачем нужны виртуальные функции

Предположим, что классы `Circle`, `Triangle`, `Rectangle` и `Square` — производные от базового класса `Shape`. Они рисуют на экране свои объекты с помощью функции `draw`, но ее реализация в каждом классе немного отличается. В программе, которая выводит на экран много разных фигур, удобно рассматривать все фигуры оди-

наково: как объекты базового класса `Shape`. Тогда мы сможем нарисовать любую фигуру, просто вызвав функцию `draw` через указатель базового класса. Во время выполнения программа сама выберет нужную функцию `draw` (функцию `draw` из соответствующего производного класса), основываясь на типе объекта, на который направлен указатель базового класса. Это динамическое полиморфное поведение. При вызове виртуальной (`virtual`) функции автоматически выбирается функция соответствующего класса в зависимости от типа объекта, к которому происходит обращение по указателю (или по ссылке), а не от типа этого указателя (или ссылки).

10.7.2. Объявление виртуальной функции

Чтобы реализовать динамическое полиморфное поведение, мы объявляем функцию базового класса `draw` виртуальной (`virtual`)¹, а затем переопределяем ее в каждом производном классе для отображения соответствующей фигуры. Переопределенная функция в производном классе должна иметь ту же сигнатуру, что и функция в базовом классе. Чтобы объявить виртуальную функцию, перед ее прототипом добавляется ключевое слово `virtual`. Пример прототипа функции `draw` в базовом классе `Shape`:

```
virtual void draw() const;
```

Этот прототип объявляет, что `draw` — виртуальная функция, которая не принимает аргументы и ничего не возвращает. Функция `draw` объявлена как `const`, потому что она не должна изменять объект `Shape`, для которого вызывается. Виртуальные функции не обязательно должны объявляться как `const`, они могут получать аргументы и возвращать значения при необходимости. Если функция объявлена виртуальной, она становится виртуальной во всех классах, прямо или косвенно производных от данного базового класса. Если производный класс не переопределяет виртуальную функцию базового класса, он просто наследует реализацию виртуальной функции от своего базового класса.

10.7.3. Вызов виртуальной функции

Если виртуальная функция вызывается

- ♦ через указатель базового класса на объект производного класса (например, `shapePtr->draw()`)
- ♦ или через ссылку базового класса на объект производного класса (например, `shapeRef.draw()`),

то программа выберет во время выполнения соответствующую функцию производного класса, основываясь на типе объекта, а не на типе указателя или ссылки.

¹ В некоторых языках программирования, в том числе Java и Python, все функции (методы) работают подобно виртуальным функциям C++. В разделе 10.10 мы покажем, что виртуальные функции немного снижают быстродействие и увеличивают потребление памяти. C++ позволяет вам выбирать, какие функции должны быть виртуальными, исходя из требований к быстродействию ваших приложений.

Если выбор нужной функции происходит во время выполнения программы, это называется **динамическим связыванием (dynamic binding)**, или **поздним связыванием (late binding)**.

Когда виртуальная функция вызывается через имя объекта с оператором «точка» (например, `squareObject.draw()`), оптимизирующий компилятор может выбрать функцию заранее, на этапе компиляции. Это называется **статическим связыванием (static binding)**. Виртуальная функция, которая будет вызвана, — это функция, определенная для класса этого объекта.

10.7.4. Виртуальные функции в иерархии `SalariedEmployee`

Давайте рассмотрим, как виртуальные функции могут обеспечить полиморфизм в нашей иерархии. Мы внесли только два изменения в заголовок каждого класса, чтобы получить полиморфное поведение, которое вы увидите на илл. 10.10. В заголовке класса `SalariedEmployee` (илл. 10.1) мы изменили прототипы функций `earnings` и `toString` (строки 17–18):

```
double earnings() const;
std::string toString() const;
```

Теперь к ним добавлено ключевое слово `virtual`:

```
virtual double earnings() const;
virtual std::string toString() const;
```

В заголовке класса `SalariedCommissionEmployee` (илл. 10.4) мы изменили прототипы функций `earnings` и `toString` (строки 19–20):

```
double earnings() const;
std::string toString() const;
```

Теперь к ним добавлено ключевое слово `override` (обсуждается после илл. 10.10):

```
double earnings() const override;
std::string toString() const override;
```

Функции `earnings` и `toString` класса `SalariedEmployee` являются виртуальными, поэтому их версии из производного класса `SalariedCommissionEmployee` переопределяют версии из класса `SalariedEmployee`. Мы не меняем реализации функций в классах `SalariedEmployee` и `SalariedCommissionEmployee`. Мы повторно используем версии, показанные на илл. 10.2 и 10.5.

Динамическое полиморфное поведение

Теперь, если мы присвоим указатель базового класса `SalariedEmployee` объекту производного класса `SalariedCommissionEmployee` и используем этот указатель для вызова функций `earnings` или `toString`, эти функции объектов производного класса будут вызваны полиморфно. Код на илл. 10.10 демонстрирует это полиморфное поведение. Сначала в строках 10–21 создаются объекты классов `SalariedEmployee` и `SalariedCommissionEmployee`, затем имена этих объектов ис-

пользуются для отображения результатов вызова функции `toString`. Это поможет подтвердить результаты динамического связывания далее в программе. В строках 27–38 снова показывается, что:

- ♦ указатель `SalariedEmployee`, направленный на объект `SalariedEmployee`, может использоваться для вызова функциональности `SalariedEmployee`;
- ♦ указатель `SalariedCommissionEmployee`, направленный на объект `SalariedCommissionEmployee`, может использоваться для вызова функциональности `SalariedCommissionEmployee`.

В строке 41 указатель базового класса `salariedPtr` направляется на объект производного класса `salariedCommission`. В строке 48 вызывается функция `toString` через указатель базового класса. Как вы можете видеть в выходных данных, вызывается функция `toString` объекта производного класса `salariedCommission`. Объявление функции класса как виртуальной и ее вызов через указатель или ссылку базового класса приводит к тому, что программа во время выполнения выбирает вызываемую функцию в зависимости от типа объекта.

```

1 // fig10_10.cpp
2 // Полиморфизм, виртуальные функции и динамическое связывание
3 #include <fmt/format.h>
4 #include <iostream>
5 #include "SalariedEmployee.h"
6 #include "SalariedCommissionEmployee.h"
7
8 int main() {
9     // Создаем объект базового класса
10    SalariedEmployee salaried{"Sue Jones", 500.0};
11
12    // Создаем объект производного класса
13    SalariedCommissionEmployee salariedCommission{
14        "Bob Lewis", 300.0, 5000.0, .04};
15
16    // Выводим данные, используя статическое связывание
17    std::cout << fmt::format("{}\n{}:\n{}\n{}\n",
18        "INVOKING TOSTRING FUNCTION ON BASE-CLASS AND",
19        "DERIVED-CLASS OBJECTS WITH STATIC BINDING",
20        salaried.toString(), // Статическое связывание
21        salariedCommission.toString()); // Статическое связывание
22
23    std::cout << "INVOKING TOSTRING FUNCTION ON BASE-CLASS AND\n"
24        << "DERIVED-CLASS OBJECTS WITH DYNAMIC BINDING\n\n";
25
26    // Направляем указатель базового класса на объект базового класса
27    SalariedEmployee* salariedPtr(&salaried);
28    std::cout << fmt::format("{}\n{}:\n{}\n",
29        "CALLING VIRTUAL FUNCTION TOSTRING WITH BASE-CLASS POINTER",
30        "TO BASE-CLASS OBJECT INVOKES BASE-CLASS FUNCTIONALITY",
31        salariedPtr->toString()); // Версия из базового класса
32
33

```

```

33 // Направляем указатель производного класса на объект производного класса
34 SalariedCommissionEmployee* salariedCommissionPtr{&salariedCommission};
35 std::cout << fmt::format("{}\n{}:\n{}\n",
36     "CALLING VIRTUAL FUNCTION TOSTRING WITH DERIVED-CLASS POINTER",
37     "TO DERIVED-CLASS OBJECT INVOKES DERIVED-CLASS FUNCTIONALITY",
38     salariedCommissionPtr->toString()); // Версия из производного класса
39
40 // Направляем указатель базового класса на объект производного класса
41 salariedPtr = &salariedCommission;
42
43 // Динамический полиморфизм: вызываем SalariedCommissionEmployee
44 // через указатель базового класса на объект производного класса
45 std::cout << fmt::format("{}\n{}:\n{}\n",
46     "CALLING VIRTUAL FUNCTION TOSTRING WITH BASE-CLASS POINTER",
47     "TO DERIVED-CLASS OBJECT INVOKES DERIVED-CLASS FUNCTIONALITY",
48     salariedPtr->toString()); // Версия из производного класса
49 }

```

INVOKING TOSTRING FUNCTION ON BASE-CLASS AND
DERIVED-CLASS OBJECTS WITH STATIC BINDING:

name: Sue Jones
salary: \$500.00

name: Bob Lewis
salary: \$300.00
gross sales: \$5000.00
commission rate: 0.04

INVOKING TOSTRING FUNCTION ON BASE-CLASS AND
DERIVED-CLASS OBJECTS WITH DYNAMIC BINDING

CALLING VIRTUAL FUNCTION TOSTRING WITH BASE-CLASS POINTER
TO BASE-CLASS OBJECT INVOKES BASE-CLASS FUNCTIONALITY:

name: Sue Jones
salary: \$500.00

CALLING VIRTUAL FUNCTION TOSTRING WITH DERIVED-CLASS POINTER
TO DERIVED-CLASS OBJECT INVOKES DERIVED-CLASS FUNCTIONALITY:

name: Bob Lewis
salary: \$300.00
gross sales: \$5000.00
commission rate: 0.04

CALLING VIRTUAL FUNCTION TOSTRING WITH BASE-CLASS POINTER
TO DERIVED-CLASS OBJECT INVOKES DERIVED-CLASS FUNCTIONALITY:

name: Bob Lewis
salary: \$300.00
gross sales: \$5000.00
commission rate: 0.04

Илл. 10.10 | Полиморфизм: вызов виртуальной функции производного класса через указатель базового класса на объект производного класса

Когда `salariedPtr` указывает на объект `SalariedEmployee`, вызывается функция `toString` класса `SalariedEmployee` (строка 31). Когда `salariedPtr` указывает на объект `SalariedCommissionEmployee`, вызывается функция `toString` класса `SalariedCommissionEmployee` (строка 48). Таким образом, один и тот же вызов функции `toString` через указатели базового класса на различные объекты принимает много форм (в данном случае две формы). Это и есть полиморфизм.

Виртуальные функции не следует вызывать из конструкторов и деструкторов

При вызове виртуальной функции из конструктора или деструктора базового класса выбирается ее версия из базового класса, даже если конструктор или деструктор базового класса вызывается при создании или уничтожении объекта производного класса. Это не то поведение, которое вы ожидаете от виртуальных функций, поэтому в C++ Core Guidelines рекомендуется не вызывать виртуальные функции из конструкторов и деструкторов¹.

Ключевое слово `override` (C++11)

Объявление функций `earnings` и `toString` класса `SalariedCommissionEmployee` с ключевым словом `override` требует от компилятора проверить наличие в базовом классе виртуальной функции с идентичной сигнатурой. Если ее там нет, компилятор выдает сообщение об ошибке. Это гарантирует, что вы переопределите соответствующую функцию базового класса. Это также предотвращает случайное сокрытие функции базового класса с тем же именем, но с другой сигнатурой. Итак, чтобы избежать ошибок, добавляйте ключевое слово `override` в прототип каждой функции производного класса, переопределяющей виртуальную функцию базового класса.

В C++ Core Guidelines поясняется, что:

- ♦ ключевое слово `virtual` определяет новую виртуальную функцию в иерархии;
- ♦ ключевое слово `override` указывает, что функция производного класса переопределяет виртуальную функцию базового класса.

Поэтому вы должны использовать `virtual` или `override` в прототипе каждой виртуальной функции².

¹ «C.82: Don't Call Virtual Functions in Constructors and Destructors». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rc-ctor-virtual>.

² «C.128: Virtual Functions Should Specify Exactly One of `virtual`, `override`, or `final`». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rhoverride>.

10.7.5. Виртуальные деструкторы

В C++ Core Guidelines рекомендуется включать виртуальный деструктор в каждый класс, содержащий виртуальные функции¹. Это помогает предотвращать трудно выявляемые ошибки в тех случаях, когда в производном классе есть пользовательский деструктор. Если в классе нет деструктора, компилятор генерирует его автоматически, но этот деструктор не будет виртуальным. Поэтому в современном C++ определение виртуального деструктора для большинства классов выглядит примерно так:

```
virtual ~SalariedEmployee() = default;
```

При такой форме записи деструктор объявляется виртуальным, но компилятор все равно генерирует для класса деструктор по умолчанию (= default).

10.7.6. Финальные функции и классы

До C++11 производный класс мог переопределять любую виртуальную функцию базового класса. Начиная с C++11, виртуальную функцию можно объявить в прототипе как финальную (final):

возвращаемый тип имяФункции(параметры) final;

Финальная функция не может быть переопределена ни в одном производном классе. В многоуровневой иерархии классов это гарантирует, что определение такой функции будет использоваться всеми последующими прямыми и косвенными производными классами.

До C++11 любой класс мог стать базовым классом в любой точке иерархии. В C++11 и более поздних версиях можно объявить класс финальным (final), чтобы запретить его использование в качестве базового класса:

```
class MyClass final {
// Тело класса
};
```

или:

```
class DerivedClass : public BaseClass final {
// Тело класса
};
```

Попытка переопределить финальную функцию или выполнить наследование от финального класса приведет к ошибке компиляции.

Если виртуальная функция объявлена финальной, это дает преимущество в быстрой работе. Компилятор оптимизирует исполняемый код, когда знает, что вир-

¹ «C.35: A Base Class Destructor Should Be Either public and virtual, or protected and Non-virtual». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rc-dtor-virtual>.

туальная функция наверняка не будет переопределяться. Например, компилятор может уже во время компиляции выбрать нужную функцию из нескольких¹. Такая оптимизация называется **девиртуализацией (devirtualization)**².

Иногда компилятор девиртуализирует вызовы даже тех виртуальных функций, которые объявлены без спецификатора `final`. Например, компилятор может распознать во время компиляции тип объекта, который будет использоваться во время выполнения. В этом случае уже на этапе компиляции возможна оптимизация вызова соответствующей виртуальной функции.

10.8. Абстрактные классы и чистые виртуальные функции

Иногда требуется определить класс, из которого никогда не будут создаваться объекты. Такой **абстрактный класс (abstract class)** определяет открытый (`public`) интерфейс для производных классов в иерархии. Поскольку абстрактные классы используются в качестве базовых классов в иерархиях наследования, их называют **абстрактными базовыми классами (abstract base classes)**. Их нельзя использовать для создания объектов, потому что абстрактные классы содержат не все необходимые для этого элементы. Недостающие элементы надо определить в производных классах, чтобы получить возможность создания объектов производных классов. Мы рассмотрим программы с абстрактными классами в разделах 10.9, 10.12 и 10.14.

Классы, из которых можно создавать объекты, называются **конкретными классами (concrete classes)**. Они определяют или наследуют реализации каждой функции, определенной в базовом классе. Хороший пример абстрактного класса — базовый класс `Shape` из раздела 10.2. Там же мы показали на диаграмме абстрактный базовый класс `TwoDimensionalShape` с конкретными производными классами `Circle`, `Square` и `Triangle` и абстрактный базовый класс `ThreeDimensionalShape` с конкретными производными классами `Sphere`, `Cube` и `Tetrahedron`. Абстрактные базовые классы являются слишком обобщенными для определения объектов. Например, если кто-то попросит вас «нарисовать двумерную фигуру», какую фигуру вы нарисуете? Конкретные классы предоставляют все необходимые уточнения, позволяющие создавать объекты.

10.8.1. Чистые виртуальные функции

Класс становится абстрактным, если в нем объявлена одна или несколько **чистых виртуальных функций (pure virtual functions)**. Такие функции обозначаются символами «`= 0`» в прототипе, например:

```
virtual void draw() const = 0; // Чистая виртуальная функция
```

¹ Мэтт Годболт (Matt Godbolt), «Optimizations in C++ Compilers», 12 ноября 2019 г. Был доступен: 15 сентября 2023 г. <https://queue.acm.org/detail.cfm?id=3372264>.

² В стандарте C++ описан ряд возможных оптимизаций, но они необязательны. Оптимизация выполняется по усмотрению разработчиков компилятора.

Здесь символы «= 0» называются **спецификатором чистой функции (pure specifier)**. Чистая виртуальная функция не предоставляет реализацию. Каждый конкретный производный класс должен переопределить чистые виртуальные функции базового класса конкретными реализациями. Если производный класс не переопределяет чистую виртуальную функцию своего базового класса, он тоже будет абстрактным. Для сравнения, обычная виртуальная функция имеет реализацию, предоставляющую каждому производному классу выбор: переопределить функцию или наследовать реализацию базового класса. Абстрактный класс может содержать элементы данных и конкретные функции. Как будет показано в разделе 10.12, абстрактный класс, в котором все функции являются чистыми виртуальными, называется **чистым абстрактным классом (pure abstract class)** или **интерфейсом (interface)**.

Чистая виртуальная функция используется в тех случаях, когда реализовать функцию в базовом классе невозможно, а во всех конкретных производных классах — необходимо. В нашем примере с космической игрой не имеет смысла включать в базовый класс `SpaceObject` реализацию функции `draw`. Невозможно нарисовать «просто космический объект», не зная, какой конкретно объект имеется в виду.

Хотя нельзя создавать объекты абстрактного базового класса, можно объявлять указатели и ссылки с типом абстрактного базового класса. Они могут обозначать объекты любых конкретных классов, производных от абстрактного базового класса. Такие указатели и ссылки обычно используются для полиморфного управления объектами производных классов во время выполнения.

10.8.2. Драйверы устройств: полиморфизм в операционных системах

Полиморфизм очень эффективен при реализации многоуровневых программных систем. Например, операционная система должна учитывать, что каждый тип физического устройства ввода-вывода может работать иначе, чем другие. Несмотря на это, сообщения ввода-вывода, передающие данные между операционной системой и устройствами, должны быть одинаковыми для всей группы устройств. Сообщение о выводе данных, отправленное объекту драйвера устройства, должно интерпретироваться в контексте этого конкретного драйвера, с учетом того, как он взаимодействует с устройством. Но само это сообщение («скопировать несколько байтов из памяти в устройство») не отличается от других сообщений операционной системы, выводящих данные в другие устройства.

Рассмотрим объектно-ориентированную операционную систему, в которой есть совместимый со всеми драйверами интерфейс, основанный на абстрактном базовом классе. От этого класса наследуются производные классы, которые выполняют примерно одинаковые задачи. Открытые (`public`) функции, входящие в драйверы устройств, являются чистыми виртуальными функциями абстрактного базового класса. Реализации этих функций находятся в производных классах, соответствующих конкретным драйверам.

Эта архитектура позволяет легко добавлять в операционную систему новые устройства. Пользователь может просто подключить устройство к разъему ком-

пьютера и установить его драйвер. После этого операционная система будет «общаться» с новым устройством через открытые функции драйвера. Эти функции аналогичны функциям других драйверов, так как основаны на абстрактном базовом классе всей группы драйверов.

10.9. Полиморфизм в программе расчета зарплаты

Рассмотрим использование абстрактных классов и полиморфизма для выполнения расчетов зарплаты двух типов сотрудников. Мы создадим простую иерархию сотрудников для решения следующей задачи:

Компания еженедельно платит зарплату сотрудникам, которые делятся на две категории:

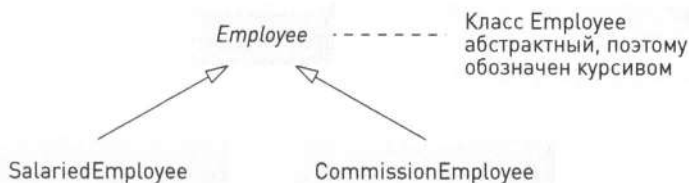
- 1) *сотрудники, получающие фиксированную зарплату (оклад);*
- 2) *сотрудники, получающие в качестве зарплаты определенный процент от их продаж.*

Компания хочет внедрить написанную на C++ программу расчета зарплаты.

Многие иерархии начинаются с абстрактного базового класса, от которого наследуются финальные (*final*) производные классы. Мы будем использовать абстрактный класс **Employee**, который представляет обобщенного сотрудника и определяет «интерфейс» иерархии, то есть набор функций, которые должны быть у всех производных классов (представляющих разные категории сотрудников), чтобы программа могла вызывать эти функции для любых объектов этих классов.

Независимо от способа расчета зарплаты, у каждого сотрудника есть имя. Для хранения имени сотрудника мы определим скрытый элемент данных *m_name* в абстрактном базовом классе **Employee**.

Конечные классы, которые наследуются непосредственно от базового класса **Employee**, — это **SalariedEmployee** и **CommissionEmployee**. Каждый из них является последним звеном в иерархии и не может быть базовым классом. Следующая диаграмма показывает иерархию наследования для нашей программы расчета зарплаты. Имя абстрактного класса **Employee** выделено курсивом, как это принято на таких UML-диаграммах.



Производный класс может наследовать от базового класса интерфейс и (или) реализацию. В тех иерархиях, которые предназначены для **наследования интерфейса (interface inheritance)**, конкретные определения функциональности обычно находятся внизу. На вершине иерархии находится базовый класс, объявляющий одну или несколько функций, которые должны быть определены и реализованы в каждом из производных классов.

В следующих разделах мы реализуем иерархию класса `Employee`. В разделах 10.9.1–10.9.3 определяются все три класса иерархии: абстрактный и два конкретных. В разделе 10.9.4 показана тестовая программа, которая создает объекты конкретных классов и обрабатывает их по принципу динамического полиморфизма.

10.9.1. Создание абстрактного базового класса `Employee`

Класс `Employee` (илл. 10.11–10.12) предоставляет функции `earnings` и `toString`, а также *get*-функции и *set*-функции, оперирующие элементом данных `m_name`. Функция `earnings`, безусловно, применима в целом ко всем объектам `Employee`, но каждый расчет зарплаты зависит от конкретного класса. Таким образом, мы объявляем `earnings` чистой виртуальной функцией в базовом классе `Employee`, потому что ее реализация по умолчанию не имеет смысла в контексте базового класса, без дополнительной информации, от которой зависит размер зарплаты.

Каждый производный класс должен переопределить функцию `earnings` с подходящей реализацией. При расчете зарплаты сотрудника программа присваивает адрес соответствующего объекта указателю базового класса `Employee`, а затем вызывает функцию `earnings` этого объекта.

Тестовая программа содержит вектор указателей `Employee`. Каждый из них указывает на объект, который является `Employee`, то есть на какой-либо объект конкретного класса, производного от `Employee`. Программа перебирает элементы вектора и вызывает функцию `earnings` для каждого сотрудника. C++ обрабатывает эти вызовы полиморфно. Поскольку `earnings` определена в базовом классе как чистая виртуальная функция, ее должен переопределить каждый производный класс, который «хочет» быть конкретным классом.

Функция `toString` класса `Employee` возвращает строку с именем сотрудника. Каждый класс, производный от `Employee`, переопределяет функцию `toString`, чтобы она возвращала не только имя сотрудника, но и дополнительные сведения о нем. Функция `toString` каждого производного класса может вызывать функцию `earnings`, хотя `earnings` — это чистая виртуальная функция в классе `Employee`. Учитывая, что в каждом конкретном классе гарантированно есть реализация функции `earnings`, ее можно вызывать даже из функции `toString` класса `Employee`. Вызывая функцию `toString` через указатель или через ссылку во время выполнения, вы всегда вызываете ее для конкретного объекта производного класса.

В следующей таблице вы видите соответствие между тремя классами нашей иерархии и функциями `earnings` и `toString`.

	earnings	toString
Employee	<i>pure virtual</i>	name: <i>m_name</i>
Salaried- Employee	<i>m_salary</i>	name: <i>m_name</i> salary: <i>m_salary</i>
Commission- Employee	<i>m_commissionRate * m_grossSales</i>	name: <i>m_name</i> gross sales: <i>m_grossSales</i> commission rate: <i>m_commissionRate</i>

Курсивом показано, какие значения объектов используются в функциях `earnings` и `toString`. Мы отметили, что в классе `Employee` функция `earnings` является *pure virtual*, потому что этот класс не предоставляет ее реализацию. Каждый производный класс переопределяет эту функцию, предоставляя соответствующую реализацию. Мы не перечислили *get*-функции и *set*-функции базового класса `Employee`, поскольку производные классы их не переопределяют. Каждая из этих функций наследуется и используется производными классами в исходном виде.

Заголовок класса `Employee`

Рассмотрим заголовок класса `Employee` (илл. 10.11). В нем есть открытые функции:

- ♦ конструктор, принимающий имя сотрудника в качестве аргумента (строка 9);
- ♦ виртуальный деструктор по умолчанию (C++11) (строка 10);
- ♦ *set*-функция для изменения имени сотрудника (строка 12);
- ♦ *get*-функция для получения имени сотрудника (строка 13);
- ♦ чистая виртуальная функция `earnings` (строка 16);
- ♦ виртуальная функция `toString` (строка 17).

Напомним, что мы объявляем функцию `earnings` как чистой виртуальной, потому что для определения способа расчета зарплаты надо знать категорию сотрудника. Каждый конкретный производный класс предоставляет реализацию функции `earnings`. Используя указатель или ссылку базового класса `Employee`, программа может вызывать функцию `earnings` для объекта любого конкретного класса, производного от `Employee`.

```

1 // Fig. 10.11: Employee.h
2 // Абстрактный базовый класс Employee
3 #pragma once // Предотвращаем повторные включения заголовка
4 #include <string>
5 #include <string_view>
6
7 class Employee {
8 public:
9     explicit Employee(std::string_view name);

```

```

10 virtual ~Employee() = default; // Компилятор генерирует вирт. деструктор
11
12 void setName(std::string_view name);
13 std::string getName() const;
14
15 // Чистая виртуальная функция делает класс Employee абстрактным базовым
16 virtual double earnings() const = 0; // Чистая виртуальная функция
17 virtual std::string toString() const; // Виртуальная функция
18 private:
19     std::string m_name;
20 };

```

Илл. 10.11 | Абстрактный базовый класс Employee

Определения функций класса Employee

На илл. 10.12 показаны определения функций класса Employee. Виртуальная функция `earnings` здесь не реализована, а реализация виртуальной функции `toString` (строки 17–19) будет переопределена в каждом производном классе. Эти переопределенные функции `toString` будут вызывать функцию `toString` базового класса Employee, чтобы получать `string` с общей информацией для всех классов в иерархии (с именем сотрудника).

```

1 // Fig. 10.12: Employee.cpp
2 // Определения функций абстрактного базового класса Employee
3 // Внимание: чистые виртуальные функции не определены
4 #include <fmt/format.h>
5 #include "Employee.h" // Определение класса Employee
6
7 // Конструктор
8 Employee::Employee(std::string_view name) : m_name{name} {} // Пустое тело
9
10 // Записываем имя
11 void Employee::setName(std::string_view name) {m_name = name;}
12
13 // Получаем имя
14 std::string Employee::getName() const {return m_name;}
15
16 // Возвращаем строковое представление данных Employee
17 std::string Employee::toString() const {
18     return fmt::format("name: {}", getName());
19 }

```

Илл. 10.12 | Файл реализации класса Employee

10.9.2. Создание конкретного производного класса SalariedEmployee

Класс `SalariedEmployee` (илл. 10.13–10.14) является производным от класса `Employee` (илл. 10.13, строка 8). В классе `SalariedEmployee` есть открытые функции:

- ✦ конструктор, принимающий имя сотрудника и оклад в качестве аргументов (строка 10);
- ✦ виртуальный деструктор по умолчанию (C++11) (строка 11);
- ✦ *set*-функция для записи нового неотрицательного значения в элемент данных `m_salary` (строка 13) и *get*-функция для возврата значения `m_salary` (строка 14);
- ✦ переопределенная виртуальная функция `earnings`, вычисляющая зарплату для объекта `SalariedEmployee` (строка 17);
- ✦ переопределенная виртуальная функция `toString` (строка 18), возвращающая строковое представление объекта `SalariedEmployee`.

```

1 // Fig. 10.13: SalariedEmployee.h
2 // Класс SalariedEmployee, производный от Employee
3 #pragma once
4 #include <string> // Класс string стандартной библиотеки C++
5 #include <string_view>
6 #include "Employee.h" // Определение класса Employee
7
8 class SalariedEmployee final : public Employee {
9 public:
10     SalariedEmployee(std::string_view name, double salary);
11     virtual ~SalariedEmployee() = default; // Виртуальный деструктор
12
13     void setSalary(double salary);
14     double getSalary() const;
15
16     // Ключевое слово override указывает на переопределение
17     double earnings() const override; // Расчет зарплаты
18     std::string toString() const override; // Строковое представление
19 private:
20     double m_salary{0.0};
21 };

```

Илл. 10.13 | Заголовок класса `SalariedEmployee`

Определения функций класса `SalariedEmployee`

На илл. 10.14 показаны функции класса `SalariedEmployee`:

- ✦ Конструктор передает аргумент `name` конструктору класса `Employee` (строка 9) для инициализации скрытого элемента, недоступного напрямую в производном классе.
- ✦ Функция `earnings` (строка 27) переопределяет чистую виртуальную функцию `earnings` класса `Employee`, предоставляя ее конкретную реализацию, которая возвращает значение зарплаты за неделю. Если бы эта функция не была переопределена, класс `SalariedEmployee` унаследовал бы чистую виртуальную функцию `earnings` класса `Employee` и был бы абстрактным.

- ♦ Функция `toString` (строки 30–33) переопределяет функцию `toString` класса `Employee`. Если бы эта функция не была переопределена, она была бы унаследована от класса `Employee` и возвращала бы строку, содержащую только имя сотрудника. Переопределенная функция `toString` класса `SalariedEmployee` возвращает результат вызова `Employee::toString()` и значение зарплаты.

В заголовке класса `SalariedEmployee` функции `earnings` и `toString` объявлены с ключевым словом `override`, что дает уверенность в их правильном переопределении. Напомним, что они являются виртуальными в базовом классе `Employee`, поэтому остаются виртуальными во всей иерархии классов.

```

1 // Fig. 10.14: SalariedEmployee.cpp
2 // Определения функций класса SalariedEmployee
3 #include <fmt/format.h>
4 #include <stdexcept>
5 #include "SalariedEmployee.h" // Определение класса SalariedEmployee
6
7 // Конструктор
8 SalariedEmployee::SalariedEmployee(std::string_view name, double salary)
9     : Employee{name} {
10     setSalary(salary);
11 }
12
13 // Записываем оклад
14 void SalariedEmployee::setSalary(double salary) {
15     if (salary < 0.0) {
16         throw std::invalid_argument("weekly salary must be >= 0.0");
17     }
18
19     m_salary = salary;
20 }
21
22 // Возвращаем оклад
23 double SalariedEmployee::getSalary() const {return m_salary;}
24
25 // Рассчитываем зарплату,
26 // переопределяя чистую виртуальную функцию earnings класса Employee
27 double SalariedEmployee::earnings() const {return getSalary();}
28
29 // Возвращаем строковое представление объекта SalariedEmployee
30 std::string SalariedEmployee::toString() const {
31     return fmt::format("{}\n(): ${:.2f}", Employee::toString(),
32         "salary", getSalary());
33 }

```

10.9.3. Создание конкретного производного класса `CommissionEmployee`

Класс `CommissionEmployee` (илл. 10.15–10.16) наследуется от класса `Employee` (илл. 10.15, строка 8). Файл реализации функций (илл. 10.16) содержит:

- ♦ конструктор (строки 8–12), который принимает имя сотрудника, объем продаж и комиссионную ставку, а затем передает имя конструктору класса `Employee` (строка 9) для инициализации унаследованных элементов данных;
- ♦ *set*-функции (строки 15–21 и 27–34) для записи новых значений в элементы данных `m_grossSales` и `m_commissionRate`;
- ♦ *get*-функции (строки 24 и 37–39) для возврата значений `m_grossSales` и `m_commissionRate`;
- ♦ переопределенную функцию `earnings` (строки 42–44), вычисляющую зарплату сотрудника;
- ♦ переопределенную функцию `toString` (строки 47–51), возвращающую результат вызова `Employee::toString()`, объем продаж и комиссионную ставку.

```
1 // Fig. 10.15: CommissionEmployee.h
2 // Класс CommissionEmployee, производный от Employee
3 #pragma once
4 #include <string>
5 #include <string_view>
6 #include "Employee.h" // Определение класса Employee
7
8 class CommissionEmployee final : public Employee {
9 public:
10     CommissionEmployee(std::string_view name, double grossSales,
11                        double commissionRate);
12     virtual ~CommissionEmployee() = default; // Виртуальный деструктор
13
14     void setGrossSales(double grossSales);
15     double getGrossSales() const;
16
17     void setCommissionRate(double commissionRate);
18     double getCommissionRate() const;
19
20     // Ключевое слово override указывает на переопределение
21     double earnings() const override; // Расчет зарплаты
22     std::string toString() const override; // Строковое представление
23 private:
24     double m_grossSales{0.0};
25     double m_commissionRate{0.0};
26 };
```

```
1 // Fig. 10.16: CommissionEmployee.cpp
2 // Определения функций класса CommissionEmployee
3 #include <fmt/format.h>
4 #include <stdexcept>
5 #include "CommissionEmployee.h" // Определение класса CommissionEmployee
6
7 // Конструктор
8 CommissionEmployee::CommissionEmployee(std::string_view name,
9     double grossSales, double commissionRate) : Employee{name} {
10     setGrossSales(grossSales);
11     setCommissionRate(commissionRate);
12 }
13
14 // Записываем объем продаж
15 void CommissionEmployee::setGrossSales(double grossSales) {
16     if (grossSales < 0.0) {
17         throw std::invalid_argument("Gross sales must be >= 0.0");
18     }
19
20     m_grossSales = grossSales;
21 }
22
23 // Возвращаем объем продаж
24 double CommissionEmployee::getGrossSales() const {return m_grossSales;}
25
26 // Записываем комиссионную ставку
27 void CommissionEmployee::setCommissionRate(double commissionRate) {
28     if (commissionRate <= 0.0 || commissionRate >= 1.0) {
29         throw std::invalid_argument(
30             "Commission rate must be > 0.0 and < 1.0");
31     }
32
33     m_commissionRate = commissionRate;
34 }
35
36 // Возвращаем комиссионную ставку
37 double CommissionEmployee::getCommissionRate() const {
38     return m_commissionRate;
39 }
40
41 // Рассчитываем зарплату
42 double CommissionEmployee::earnings() const {
43     return getGrossSales() * getCommissionRate();
44 }
45
46 // Возвращаем строковое представление объекта CommissionEmployee
47 std::string CommissionEmployee::toString() const {
48     return fmt::format("{}\n{}: ${:.2f}\n{}: {:.2f}", Employee::toString(),
49         "gross sales", getGrossSales(),
50         "commission rate", getCommissionRate());
51 }
```

10.9.4. Демонстрация полиморфных операций с объектами

Программа, предназначенная для тестирования нашей иерархии (илл. 10.17), создает объекты конкретных классов `SalariedEmployee` и `CommissionEmployee` и сначала обращается к этим объектам по их именам, а затем методом динамического полиморфизма: через вектор указателей базового класса `Employee`. Объекты создаются в строках 16–17. В строках 20–23 выводится информация о каждом сотруднике и его зарплате. В этих строках обращение к объектам выполняется через имена переменных, поэтому компилятор может определить во время компиляции тип каждого объекта, чтобы выбрать, какие из функций `toString` и `earnings` должны быть вызваны.

```
1 // fig10_17.cpp
2 // Обработка объектов класса, производного от Employee, по именам переменных,
3 // а затем полиморфно: через указатели и ссылки базового класса
4 #include <fmt/format.h>
5 #include <iostream>
6 #include <vector>
7 #include "Employee.h"
8 #include "SalariedEmployee.h"
9 #include "CommissionEmployee.h"
10
11 void virtualViaPointer(const Employee* baseClassPtr); // Прототип
12 void virtualViaReference(const Employee& baseClassRef); // Прототип
13
14 int main() {
15     // Создаем объекты производного класса
16     SalariedEmployee salaried{"John Smith", 8000.0};
17     CommissionEmployee commission{"Sue Jones", 10000, .06};
18
19     // Выводим данные каждого объекта Employee
20     std::cout << "EMPLOYEES PROCESSED INDIVIDUALLY VIA VARIABLE NAMES\n";
21     << fmt::format("{}\n{{:.2f}}\n\n{{:.2f}}\n\n",
22         salaried.toString(), "earned $", salaried.earnings(),
23         commission.toString(), "earned $", commission.earnings());
24
25     // Создаем и инициализируем вектор указателей базового класса
26     std::vector<Employee*> employees{&salaried, &commission};
27
28     std::cout << "EMPLOYEES PROCESSED POLYMORPHICALLY VIA"
29         << " DYNAMIC BINDING\n\n";
30
31     // Вызываем virtualViaPointer для вывода данных каждого
32     // объекта Employee с помощью динамического связывания
33     std::cout << "VIRTUAL FUNCTION CALLS MADE VIA BASE-CLASS POINTERS\n";
34
35     for (const Employee* employeePtr : employees) {
36         virtualViaPointer(employeePtr);
37     }
38
39     // Вызываем virtualViaReference для вывода данных каждого
40     // объекта Employee с помощью динамического связывания
41     std::cout << "VIRTUAL FUNCTION CALLS MADE VIA BASE-CLASS REFERENCES\n";
```



```

42
43   for (const Employee* employeePtr : employees) {
44       virtualViaReference(*employeePtr); // Разыменованный указатель!
45   }
46 }
47
48 // Вызываем виртуальные функции toString и earnings класса Employee
49 // через указатель базового класса с помощью динамического связывания
50 void virtualViaPointer(const Employee* baseClassPtr) {
51     std::cout << fmt::format("{}\nearned {:.2f}\n\n",
52         baseClassPtr->toString(), baseClassPtr->earnings());
53 }
54
55 // Вызываем виртуальные функции toString и earnings класса Employee
56 // через ссылку базового класса с помощью динамического связывания
57 void virtualViaReference(const Employee& baseClassRef) {
58     std::cout << fmt::format("{}\nearned {:.2f}\n\n",
59         baseClassRef.toString(), baseClassRef.earnings());
60 }

```

EMPLOYEES PROCESSED INDIVIDUALLY VIA VARIABLE NAMES

```

name: John Smith
salary: $800.00
earned $800.00

```

```

name: Sue Jones
gross sales: $10000.00
commission rate: 0.06
earned $600.00

```

EMPLOYEES PROCESSED POLYMORPHICALLY VIA DYNAMIC BINDING

VIRTUAL FUNCTION CALLS MADE VIA BASE-CLASS POINTERS

```

name: John Smith
salary: $800.00
earned $800.00

```

```

name: Sue Jones
gross sales: $10000.00
commission rate: 0.06
earned $600.00

```

VIRTUAL FUNCTION CALLS MADE VIA BASE-CLASS REFERENCES

```

name: John Smith
salary: $800.00
earned $800.00

```

```

name: Sue Jones
gross sales: $10000.00
commission rate: 0.06
earned $600.00

```

Илл. 10.17 | Обработка объектов класса, производного от Employee, через имена переменных, а затем полиморфно: через указатели и ссылки базового класса

Создание вектора указателей

В строке 26 создается и инициализируется вектор `employees`, содержащий два указателя типа `Employee`: на объекты `salaried` и `commission`. Компилятор позволяет инициализировать элементы вектора адресами этих объектов, поскольку `SalariedEmployee` является `Employee` и `CommissionEmployee` тоже является `Employee`.

Функция `virtualViaPointer`

В строках 35–37 перебираются элементы вектора `employees`, при этом функция `virtualViaPointer` (строки 50–53) поочередно вызывается с каждым элементом в качестве аргумента. Она получает в свой параметр `baseClassPtr` тот адрес, который хранится в выбранном элементе вектора `employees`, и вызывает через этот указатель виртуальные функции `toString` и `earnings`. У функции `virtualViaPointer` нет никакой информации о типе объекта (`SalariedEmployee` или `CommissionEmployee`), она знает только о базовом классе `Employee`. Программа по очереди направляет указатель `baseClassPtr` на разные конкретные объекты производных классов, поэтому компилятор не знает, какие функции конкретных классов должны быть вызваны через `baseClassPtr`. Компилятор должен разрешить выбор вызова во время выполнения программы с использованием динамического связывания. Во время выполнения программы каждый вызов виртуальной функции приводит к корректному вызову подходящей функции для того объекта, на который указывает `baseClassPtr`. Выходные данные программы показывают, что функции каждого класса вызываются правильно и выводят корректные данные. Получение данных о зарплате методом динамического полиморфизма дает тот же результат, что и прямой доступ к данным в строках 22 и 23.

Функция `virtualViaReference`

В строках 43–45 перебираются элементы вектора `employees`, при этом функция `virtualViaReference` (строки 57–60) поочередно вызывается с каждым его элементом в качестве аргумента. Она получает в свой параметр `baseClassRef` (типа `const Employee&`) ссылку на объект, которая образуется путем разыменования указателя, хранящегося в выбранном элементе вектора `employees` (строка 44). При каждом вызове этой функции вызываются виртуальные функции `toString` и `earnings` через `baseClassRef`. Вы можете убедиться, что полиморфная обработка данных работает и со ссылками базового класса. Каждый вызов виртуальной функции приводит к корректному вызову подходящей функции для того объекта, на который ссылается `baseClassRef`. Это еще один пример динамического связывания.

Итак, через ссылки базового класса мы получили такие же выходные данные, как двумя предыдущими способами: через указатели базового класса и через имена переменных (со статическим связыванием).

10.10. Динамический полиморфизм, виртуальные функции и динамическое связывание «под капотом» компилятора

Давайте «поднимем капот» компилятора C++ и посмотрим на внутреннюю реализацию динамического полиморфизма, виртуальных функций и динамического связывания. Мы рассмотрим один из возможных вариантов реализации, чтобы вы лучше поняли, как все это работает. Важнее всего, чтобы вы учитывали повышенную нагрузку на компьютер. Динамический полиморфизм требует дополнительных ресурсов: памяти и процессорного времени. Поэтому в одних случаях он вам поможет, а в других случаях его надо избегать. Например, классы стандартной библиотеки C++ должны работать очень быстро, поэтому они, как правило, реализованы без виртуальных функций.

Сначала мы расскажем о структурах данных, которые компилятор создает для поддержки динамического полиморфизма. Вы увидите, что для этого нужна тройная косвенность, то есть три уровня указателей. Затем мы покажем, как эти структуры данных используются во время выполнения программы для работы виртуальных функций и для динамического связывания.

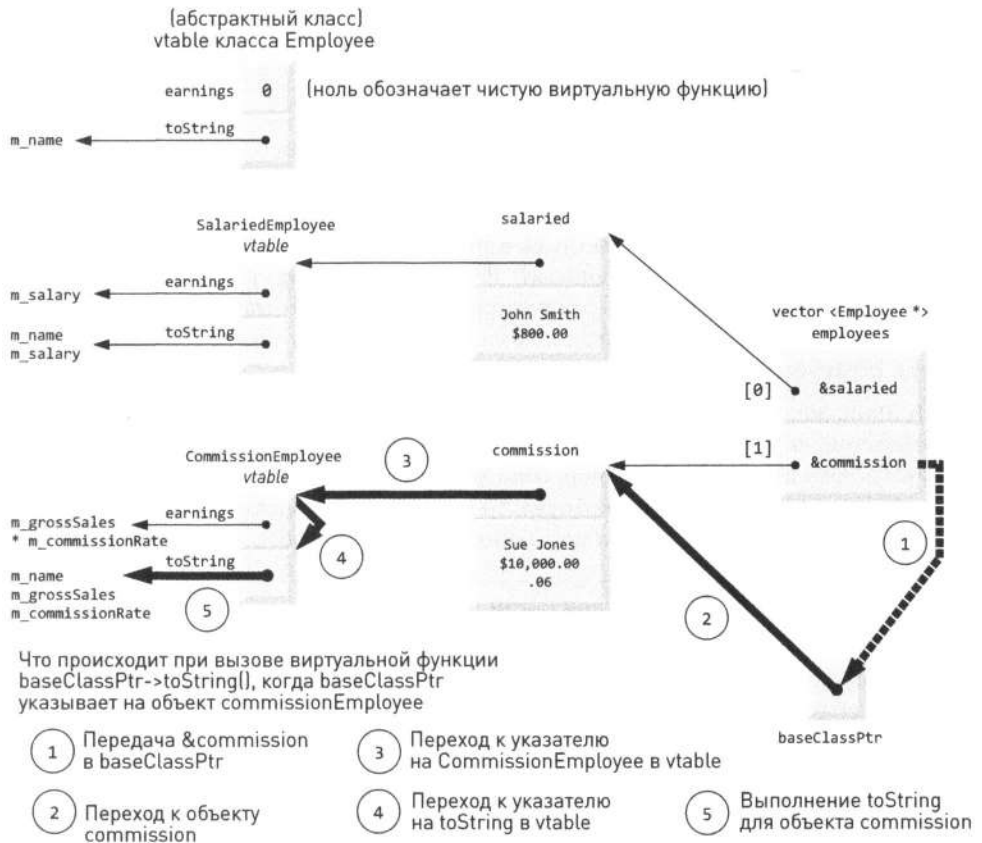
Таблицы виртуальных функций

Когда в C++ компилируется класс с одной или несколькими виртуальными функциями, для него создается **таблица виртуальных функций** (**virtual function table**, или сокращенно **vtable**). Она содержит указатели на виртуальные функции класса. Каждый такой указатель хранит адрес памяти, с которого начинается машинный код, выполняющий задачу функции. Вы уже знаете, что имя массива неявно преобразуется в адрес первого элемента массива. Так и имя функции неявно преобразуется в начальный адрес кода функции.

Для динамического связывания выполняемая программа обращается к виртуальной таблице класса, чтобы при каждом вызове виртуальной функции выбрать реализацию, подходящую для объекта соответствующего класса. В левом столбце на илл. 10.18 показаны виртуальные таблицы наших классов `Employee`, `SalariedEmployee` и `CommissionEmployee`.

Таблица виртуальных функций класса `Employee`

Первый указатель на функцию в таблице виртуальных функций класса `Employee` содержит `0`, поскольку `earnings` — это чистая виртуальная функция без реализации. Второй указатель указывает на функцию `toString`, возвращающую строку с именем сотрудника. На этой иллюстрации мы для экономии места сократили выходные данные каждой функции `toString`. Любой класс, в таблице виртуальных функций которого есть одна или несколько чистых виртуальных функций (представленных значением `0`), является абстрактным классом. Для сравнения, `SalariedEmployee` и `CommissionEmployee` — это конкретные классы, поэтому в их таблицах виртуальных функций нет `nullptr`.



Илл. 10.18 | Как устроены вызовы виртуальных функций

Таблица виртуальных функций класса SalariedEmployee

Указатель на функцию earnings в виртуальной таблице класса SalariedEmployee указывает на переопределенную этим классом функцию earnings, возвращающую зарплату сотрудника. Класс SalariedEmployee также переопределяет функцию toString, поэтому соответствующий указатель ссылается на функцию toString из класса SalariedEmployee, возвращающую имя и зарплату сотрудника.

Таблица виртуальных функций класса CommissionEmployee

Указатель на функцию earnings в виртуальной таблице класса CommissionEmployee указывает на функцию earnings из класса CommissionEmployee, возвращающую произведение объема продаж сотрудника и комиссионной ставки. Указатель функции toString ссылается на версию этой функции из класса CommissionEmployee, возвращающую имя сотрудника, комиссионную ставку и объем продаж. Как и в слу-

чае с классом `SalariedEmployee`, обе функции переопределяют функции базового класса `Employee`.

Наследование конкретных виртуальных функций

Каждый конкретный класс в нашей программе предоставляет реализацию виртуальной функции `earnings` и функции `toString`. Учитывая, что `earnings` — это чистая виртуальная функция, все прямые производные классы от класса `Employee` должны ее реализовать, чтобы стать конкретными. Но прямым производным классам не обязательно реализовывать функцию `toString`, чтобы стать конкретными, — они могут наследовать реализацию функции `toString` от класса `Employee`. В нашем случае оба производных класса переопределяют функцию `toString` из класса `Employee`.

Если производный класс в иерархии наследует функцию, не переопределяя ее, то в таблице виртуальных функций этого класса хранится указатель на унаследованную реализацию. Например, если мы решим не переопределять функцию `toString` в классе `CommissionEmployee`, то в таблице этого класса будет указатель на ту же версию `toString`, что и в таблице класса `Employee`.

Три уровня указателей для реализации полиморфизма

Динамический полиморфизм может быть реализован с помощью элегантной структуры данных, включающей три уровня указателей. Мы рассмотрели первый уровень: указатели в таблицах виртуальных функций. Каждый такой указатель содержит адрес кода, который должен выполняться при вызове соответствующей виртуальной функции.

В среднем столбце диаграммы показан второй уровень указателей. Создавая объект с одной или несколькими виртуальными функциями, компилятор «прикрепляет» к нему указатель на таблицу виртуальных функций класса. Обычно этот указатель находится в начале объекта, но возможны и другие варианты в зависимости от компилятора. На диаграмме вы видите эти указатели в объектах классов `SalariedEmployee` и `CommissionEmployee` (илл. 10.17). Рядом показаны значения элементов данных каждого объекта.

Третий уровень указателей содержит адреса объектов, для которых будут вызываться виртуальные функции. В правом столбце диаграммы показан вектор `employees`, содержащий эти указатели.

Теперь рассмотрим, как выполняется типичный вызов виртуальной функции. Предположим, что из функции `virtualViaPointer` выполняется вызов `baseClassPtr->toString()` (илл. 10.17, строка 52). Предположим, что `baseClassPtr` в данный момент указывает на `employees[1]`, то есть на адрес объекта `commission` в векторе. При компиляции этой инструкции компилятор видит, что вызов выполняется через указатель базового класса, а функция `toString` является виртуальной. Поэтому компилятор находит `toString` во второй строке таблицы виртуальных функций и берет из этой строки адрес машинного кода функции.

Сгенерированный компилятором код выполнит следующие операции (их порядковые номера указаны в кружках на илл. 10.18):

1. Возьмет из вектора `employees` элемент с индексом 1 (адрес объекта `commission`) и передаст его в качестве аргумента функции `virtualViaPointer`. Ее параметр `baseClassPtr` станет указателем на объект `commission`.
2. Разыменует этот указатель для перехода к объекту `commission`, который начинается с указателя на таблицу виртуальных функций класса `CommissionEmployee`.
3. Разыменует этот указатель для перехода к таблице виртуальных функций класса `CommissionEmployee`.
4. Получит указатель на функцию `toString` из второй строки таблицы виртуальных функций класса `CommissionEmployee`¹.
5. Выполнит функцию `toString` для объекта `commission`, которая возвратит строку с именем сотрудника, объемом продаж и комиссионной ставкой.

Структуры данных на диаграмме с таблицами виртуальных функций могут показаться сложными. Но компилятор выполняет все эти сложные операции, скрывая их от вас, поэтому с точки зрения программиста применять динамический полиморфизм очень просто. Но не забывайте, что при каждом вызове виртуальной функции выполняются операции разыменования указателей и обращения к памяти, требующие дополнительного процессорного времени. А таблицы виртуальных функций и добавляемые к объектам указатели на них требуют дополнительной памяти.

Динамический полиморфизм, обычно реализуемый в C++ с помощью виртуальных функций и динамического связывания, очень эффективен. Вы можете использовать его в большинстве программ, не опасаясь существенной нагрузки на компьютер. И лишь в некоторых случаях эта дополнительная нагрузка будет играть решающую роль. Она может оказаться чрезмерной в программах реального времени, которые должны работать максимально быстро, а также в программах с огромным количеством маленьких объектов, по сравнению с которыми дополнительные указатели были бы слишком большими.

10.11. Идиомы не виртуального интерфейса

Идиомы не виртуального интерфейса (non-virtual interface idiom, или сокращенно NVI)^{2,3} — это еще один способ реализовать динамический полиморфизм на основе иерархий классов. Херб Саттер (Herb Sutter) впервые предложил NVI в своей

¹ На практике операции 3 и 4 могут быть объединены в одной машинной инструкции.

² «Non-Virtual Interface (NVI) Pattern». Был доступен: 15 сентября 2023 г. https://en.wikipedia.org/wiki/Non-virtual_interface_pattern.

³ Мариус Банчила (Marius Bancila), *Modern C++ Programming Cookbook: Master C++ Core Language and Standard Library Features, with over 100 Recipes, Updated to C++20*. Бирмингем: издательство Packt, 2020, с. 562–567.

статье «Virtuality»¹. В этом разделе мы будем следовать четырем рекомендациям Саттера по реализации иерархий классов:

1. «Делайте интерфейсы неvirtуальными, используя метод шаблонов» — это паттерн объектно-ориентированного проектирования. Саттер объясняет, что открытая (`public`) виртуальная функция служит двум целям: описывает часть интерфейса класса и позволяет переопределять себя в производных классах для настройки поведения. Он рекомендует, чтобы каждая функция служила только какой-либо одной из этих целей.
2. «Делайте виртуальные функции скрытыми (`private`)». Производный класс может переопределять скрытые виртуальные функции своего базового класса. Скрытие виртуальной функции гарантирует, что она служит только одной цели: позволяет переопределять себя в производных классах для настройки поведения. Невиртуальная функция в базовом классе внутренне вызывает скрытую виртуальную функцию как деталь реализации.
3. «Только если производным классам надо вызывать базовую реализацию виртуальной функции, делайте эту виртуальную функцию защищенной (`protected`)». Это позволяет производным классам не только переопределять виртуальную функцию базового класса, но и пользоваться ее реализацией в базовом классе, чтобы избежать повторения кода в производных классах.
4. «Деструктор базового класса должен быть либо открытым и виртуальным, либо защищенным и неvirtуальным». Здесь мы снова сделаем деструктор базового класса `Employee` открытым и виртуальным. В главе 11 вы увидите, что открытые виртуальные деструкторы важны при удалении динамически выделяемых объектов производного класса через указатели базового класса.

Статья Саттера демонстрирует каждую рекомендацию, подчеркивая необходимость отделения интерфейса класса от его реализации, что является хорошей практикой программирования.

Рефакторинг класса `Employee` с идиомой неvirtуального интерфейса

Давайте доработаем класс `Employee` (илл. 10.19–10.20), основываясь на рекомендациях Саттера. Большая часть кода останется такой же, как в разделе 10.9, поэтому мы сосредоточимся на описании изменений. Перечислим основные изменения в заголовке (илл. 10.19):

- ♦ Открытые функции `earnings` (строка 15) и `toString` (строка 16) больше не являются виртуальными, а функция `earnings`, как вы скоро увидите, будет реализована в классе `Employee`. Теперь каждая из функций `earnings` и `toString` служит только одной цели: позволяет клиентскому коду получать данные из объектов класса `Employee` (размер зарплаты и строковое представление). В прежних версиях эти функции служили еще и в качестве

¹ Херб Саттер (Herb Sutter), «Virtuality», C/C++ Users Journal, том 19, № 9, сентябрь 2001 г. Был доступен: 15 сентября 2023 г. <http://www.gotw.ca/publications/mull18.htm>.

точек настройки для производных классов, но теперь для этого предназначены новые функции.

- ✦ Мы добавили защищенную (**protected**) виртуальную функцию **getString** (строка 18), которая будет точкой настройки для производных классов. Эта функция будет вызываться неvirtуальной функцией **toString**. Защищенные элементы базового класса доступны производным классам. Как вы скоро увидите, функции **getString** производных классов будут и вызывать, и переопределять функцию **getString** класса **Employee**.
- ✦ Мы добавили скрытую (**private**) чистую виртуальную функцию **getPay** (строка 21), которая будет точкой настройки для производных классов. Эта функция будет вызываться неvirtуальной функцией **earnings**. Производные классы будут переопределять функцию **getPay** для настройки расчетов заработной платы.

```

1 // Fig. 10.19: Employee.h
2 // Абстрактный базовый класс Employee
3 #pragma once // Предотвращаем повторные включения заголовка
4 #include <string>
5 #include <string_view>
6
7 class Employee {
8 public:
9     Employee(std::string_view name);
10    virtual ~Employee() = default;
11
12    void setName(std::string_view name);
13    std::string getName() const;
14
15    double earnings() const; // Невиртуальная функция
16    std::string toString() const; // Невиртуальная функция
17 protected:
18    virtual std::string getString() const; // Виртуальная функция
19 private:
20    std::string m_name;
21    virtual double getPay() const = 0; // Чистая виртуальная функция
22 };

```

Илл. 10.19 | Абстрактный базовый класс **Employee**

Перечислим основные изменения в файле реализации функций класса **Employee** (илл. 10.20):

- ✦ Функция **earnings** (строка 17) теперь предоставляет конкретную реализацию, возвращающую результат вызова скрытой чистой виртуальной функции **getPay**. Этот вызов разрешен, потому что во время выполнения программы функция **earnings** будет вызываться для объекта конкретного класса, производного от **Employee**.

- ✦ Функция `toString` (строка 20) теперь возвращает результат вызова защищенной виртуальной функции `getString`.
- ✦ Мы добавили определение новой защищенной функции `getString`, возвращающей строковое представление объекта по умолчанию, содержащее имя сотрудника. Эта функция является защищенной, поэтому производные классы могут переопределять ее, а также вызывать ее, чтобы получить ту часть строкового представления, которая относится к базовому классу.

```

1 // Fig. 10.20: Employee.cpp
2 // Определения функций абстрактного базового класса Employee
3 // Внимание: для чистых виртуальных функций нет определений
4 #include <fmt/format.h>
5 #include "Employee.h" // Определение класса Employee
6
7 // Конструктор
8 Employee::Employee(std::string_view name) : m_name{name} {} // Пустое тело
9
10 // Записываем имя сотрудника
11 void Employee::setName(std::string_view name) {m_name = name;}
12
13 // Получаем имя сотрудника
14 std::string Employee::getName() const {return m_name;}
15
16 // Открытая неvirtуальная функция, возвращающая зарплату сотрудника
17 double Employee::earnings() const {return getPay();}
18
19 // Открытая неvirtуальная функция, возвращающая строковое представление
    объекта Employee
20 std::string Employee::toString() const {return getString();}
21
22 // Защищенная виртуальная функция, которую производные классы могут
    переопределять и вызывать
23 std::string Employee::getString() const {
24     return fmt::format("name: {}", getName());
25 }
```

Илл. 10.20 | Определения функций абстрактного базового класса `Employee`

Доработанный класс `SalariedEmployee`

Перечислим основные изменения в классе `SalariedEmployee` (илл. 10.21–10.22):

- ✦ В заголовке этого класса (илл. 10.21) больше нет прототипов функций `earnings` и `toString`. Эти открытые неvirtуальные функции базового класса теперь наследуются от класса `Employee`.
- ✦ В секции `private` теперь объявлены переопределения скрытой чистой виртуальной функции базового класса `getPay` и защищенной виртуальной функции `getString` (илл. 10.21, строки 19–20). Функция `getString` клас-

са `SalariedEmployee` является скрытой, потому что это финальный (`final`) класс, который не может стать базовым.

- ♦ В файле реализации функций (илл. 10.22) теперь есть переопределенная функция `getPay` (строка 27), возвращающая размер зарплаты, и переопределенная функция `getString` (строки 30–33), возвращающая строковое представление объекта `SalariedEmployee`. Обратите внимание, что функция `getString` класса `SalariedEmployee` вызывает защищенную функцию `getString` класса `Employee`, чтобы получить соответствующую часть строкового представления.

```

1 // Fig. 10.21: SalariedEmployee.h
2 // Класс SalariedEmployee, производный от Employee
3 #pragma once
4 #include <string> // Класс string стандартной библиотеки C++
5 #include <string_view>
6 #include "Employee.h" // Определение класса Employee
7
8 class SalariedEmployee final : public Employee {
9 public:
10     SalariedEmployee(std::string_view name, double salary);
11     virtual ~SalariedEmployee() = default; // Виртуальный деструктор
12
13     void setSalary(double salary);
14     double getSalary() const;
15 private:
16     double m_salary{0.0};
17
18     // Ключевое слово override указывает на переопределение
19     double getPay() const override; // Расчет зарплаты
20     std::string getString() const override; // Строковое представление
21 };

```

Илл. 10.21 | Класс `SalariedEmployee`, производный от класса `Employee`

```

1 // Fig. 10.22: SalariedEmployee.cpp
2 // Определения функций класса SalariedEmployee
3 #include <fmt/format.h>
4 #include <stdexcept>
5 #include "SalariedEmployee.h" // Определение класса SalariedEmployee
6
7 // Конструктор
8 SalariedEmployee::SalariedEmployee(std::string_view name, double salary)
9     : Employee(name) {
10     setSalary(salary);
11 }
12
13 // Записываем оклад
14 void SalariedEmployee::setSalary(double salary) {
15     if (salary < 0.0) {
16         throw std::invalid_argument("Weekly salary must be >= 0.0");

```

```

17     }
18
19     m_salary = salary;
20 }
21
22 // Возвращаем оклад
23 double SalariedEmployee::getSalary() const {return m_salary;}
24
25 // Рассчитываем зарплату;
26 // переопределяем чистую виртуальную функцию getPay класса Employee
27 double SalariedEmployee::getPay() const {return getSalary();}
28
29 // Возвращаем строковое представление объекта SalariedEmployee
30 std::string SalariedEmployee::getString() const {
31     return fmt::format("{}\n{}: ${:.2f}", Employee::getString(),
32         "salary", getSalary());
33 }

```

Илл. 10.22 | Определения функций класса `SalariedEmployee`

Доработанный класс `CommissionEmployee`

Перечислим основные изменения в классе `CommissionEmployee` (илл. 10.23–10.24):

- ♦ В заголовке этого класса (илл. 10.23) больше нет прототипов функций `earnings` и `toString`. Эти открытые не виртуальные функции базового класса теперь наследуются от класса `Employee`.
- ♦ В секции `private` теперь объявлены переопределения скрытой чистой виртуальной функции базового класса `getPay` и защищенной виртуальной функции `getString` (илл. 10.23, строки 24–25). Опять же, функция `getString` является скрытой, потому что это финальный (`final`) класс, который не может стать базовым.
- ♦ В файле реализации функций (илл. 10.24) теперь есть переопределенная функция `getPay` (строки 42–44), возвращающая результат расчета комиссионного вознаграждения, и переопределенная функция `getString` (строки 47–51), возвращающая строковое представление объекта `CommissionEmployee`. Функция `getString` вызывает защищенную функцию `getString` класса `Employee`, чтобы получить соответствующую часть строкового представления.

```

1 // Fig. 10.23: CommissionEmployee.h
2 // Класс CommissionEmployee, производный от класса Employee
3 #pragma once
4 #include <string>
5 #include <string_view>
6 #include "Employee.h" // Определение класса Employee
7
8 class CommissionEmployee final : public Employee {
9 public:
10     CommissionEmployee(std::string_view name, double grossSales,

```

```

11     double commissionRate);
12     virtual ~CommissionEmployee() = default; // Виртуальный деструктор
13
14     void setGrossSales(double grossSales);
15     double getGrossSales() const;
16
17     void setCommissionRate(double commissionRate);
18     double getCommissionRate() const;
19 private:
20     double m_grossSales{0.0};
21     double m_commissionRate{0.0};
22
23     // Ключевое слово override указывает на переопределение
24     double getPay() const override; // Расчет зарплаты
25     std::string getString() const override; // Строковое представление
26 };

```

Илл. 10.23 | Класс `CommissionEmployee`, производный от класса `Employee`

```

1 // Fig. 10.24: CommissionEmployee.cpp
2 // Определения функций класса CommissionEmployee
3 #include <fmt/format.h>
4 #include <stdexcept>
5 #include "CommissionEmployee.h" // Определение класса CommissionEmployee
6
7 // Конструктор
8 CommissionEmployee::CommissionEmployee(std::string_view name,
9     double grossSales, double commissionRate) : Employee{name} {
10     setGrossSales(grossSales);
11     setCommissionRate(commissionRate);
12 }
13
14 // Записываем объем продаж
15 void CommissionEmployee::setGrossSales(double grossSales) {
16     if (grossSales < 0.0) {
17         throw std::invalid_argument("Gross sales must be >= 0.0");
18     }
19
20     m_grossSales = grossSales;
21 }
22
23 // Возвращаем объем продаж
24 double CommissionEmployee::getGrossSales() const {return m_grossSales;}
25
26 // Записываем комиссионную ставку
27 void CommissionEmployee::setCommissionRate(double commissionRate) {
28     if (commissionRate <= 0.0 || commissionRate >= 1.0) {
29         throw std::invalid_argument(
30             "Commission rate must be > 0.0 and < 1.0");
31     }
32 }

```

```

33     m_commissionRate = commissionRate;
34 }
35
36 // Возвращаем комиссионную ставку
37 double CommissionEmployee::getCommissionRate() const {
38     return m_commissionRate;
39 }
40
41 // Рассчитываем зарплату
42 double CommissionEmployee::getPay() const {
43     return getGrossSales() * getCommissionRate();
44 }
45
46 // Возвращаем строковое представление объекта CommissionEmployee
47 std::string CommissionEmployee::getString() const {
48     return fmt::format(
49         "{}\n({}: {:.2f})\n({}: {:.2f})", Employee::getString(),
50         "gross sales", getGrossSales(),
51         "commission rate", getCommissionRate());
52 }

```

Илл. 10.24 | Определения функций класса `CommissionEmployee`

Динамический полиморфизм в иерархии `Employee` с идиомой неvirtуального интерфейса

Тестовая программа для этого примера идентична предыдущей (илл. 10.17), поэтому на илл. 10.25 мы показываем только выходные данные. Как видите, они тоже идентичны выходным данным предыдущей программы (илл. 10.17). Это подтверждает, что код остается полиморфным даже с защищенными и скрытыми виртуальными функциями базового класса. Наш клиентский код теперь вызывает только неvirtуальные функции. Тем не менее каждый производный класс обеспечивает требуемое поведение, переопределяя защищенные и скрытые виртуальные функции базового класса. Теперь виртуальные функции являются деталями реализации внутри иерархии классов, скрытыми от программиста клиентского кода. Мы можем изменять эти реализации виртуальных функций (и, возможно, даже их сигнатуры) без изменения клиентского кода!

EMPLOYEES PROCESSED INDIVIDUALLY VIA VARIABLE NAMES

```

name: John Smith
salary: $800.00
earned $800.00

```

```

name: Sue Jones
gross sales: $10000.00
commission rate: 0.06
earned $600.00

```

EMPLOYEES PROCESSED POLYMORPHICALLY VIA DYNAMIC BINDING

VIRTUAL FUNCTION CALLS MADE VIA BASE-CLASS POINTERS

name: John Smith
salary: \$800.00
earned \$800.00

name: Sue Jones
gross sales: \$10000.00
commission rate: 0.06
earned \$600.00

VIRTUAL FUNCTION CALLS MADE VIA BASE-CLASS REFERENCES

name: John Smith
salary: \$800.00
earned \$800.00

name: Sue Jones
gross sales: \$10000.00
commission rate: 0.06
earned \$600.00

Илл. 10.25 | Обработка объектов класса, производного от `Employee`, со статическим связыванием, а затем полиморфно: с динамическим связыванием

10.12. Наследование интерфейса вместо наследования реализации¹

Наследование реализации в основном используется для определения родственных классов со множеством одинаковых элементов данных и функций. Такое наследование создает тесно связанные классы, в которых элементы данных и функции базового класса наследуются в производных классах. Изменения в базовом классе напрямую влияют на все соответствующие производные классы.

Тесные связи между классами могут затруднять модификацию иерархии. Подумайте, как бы вы могли дополнить иерархию класса `Employee` из раздела 10.9 для поддержки пенсионных накоплений. В США есть разные планы пенсионных накоплений, например «401(k)» или «IRA». Вы могли бы добавить в класс `Employee` чистую виртуальную функцию `makeRetirementDeposit` и определить соответствующие производные классы: `SalariedEmployeeWith401k`, `SalariedEmployeeWithIRA`, `CommissionEmployeeWith401k`, `CommissionEmployeeWithIRA` и т. д., в каждом из которых будет собственная реализация чистой виртуальной функции `makeRetirementDeposit`. Очевидно, что при таком подходе вы получите много производных классов, поэтому реализация и обслуживание этой иерархии окажется непростой задачей.

¹ Эрих Гамма и др., «Приемы объектно-ориентированного проектирования». Также обсуждается: Джошуа Блох (Joshua Bloch), «Effective Java». Upper Saddle River, Нью-Джерси: Эддисон-Уэсли, 2008.

Маленькие иерархии, находящиеся под контролем одного программиста, как правило, легче модифицировать, чем большие иерархии, поддерживаемые командами программистов. Это верно даже для иерархий с тесно связанными классами, которые образуются при наследовании реализации.

Переосмысление иерархий Employee: композиция и внедрение зависимостей

Год за годом программисты пишут многочисленные статьи и посты в блогах о проблемах тесных связей в иерархиях классов (как в иерархии Employee из раздела 10.9). В следующем примере мы выполним рефакторинг нашей иерархии Employee. Теперь модель расчета зарплаты не будет жестко встроена в иерархию¹.

Чтобы добиться этого, мы используем композицию и внедрение зависимостей: добавим в класс указатель на объект, от которого зависит требуемое поведение объектов этого класса. В нашем примере поведение — это способ расчета зарплаты каждого сотрудника. Мы определим новый класс Employee, содержащий указатель на объект класса CompensationModel с функциями earnings и toString. У этого класса Employee не будет производных классов: мы объявим его финальным (final). Затем мы определим производные от CompensationModel классы, реализующие две модели расчета зарплаты сотрудников:

- ♦ фиксированная зарплата (оклад);
- ♦ комиссионное вознаграждение, зависящее от объема продаж.

Конечно, мы можем аналогично определить и любые другие модели расчета зарплаты.

Наследование интерфейса дает самые гибкие возможности

Для создания моделей расчета зарплаты мы будем использовать **наследование интерфейса (interface inheritance)**. Каждый конкретный класс CompensationModel будет производным от класса, содержащего только чистые виртуальные функции. Класс, в котором все функции являются чистыми виртуальными, называется **чистым абстрактным классом (pure abstract class)**, или **интерфейсом (interface)**. В C++ Core Guidelines рекомендуется отдавать предпочтение наследованию от чистых абстрактных классов, а не от классов с деталями реализации^{2,3,4,5}.

¹ Мы благодарим Брайана Гетца (Brian Goetz), архитектора языка Java, который предложил рассматриваемую в этом разделе архитектуру, когда рецензировал нашу книгу «Java How to Program».

² «I.25: Prefer Abstract Classes as Interfaces to Class Hierarchies». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Ri-abstract>.

³ «C.121: If a Base Class Is Used as an Interface, Make It a Pure Abstract Class». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rh-abstract>.

⁴ «C.122: Use Abstract Classes As Interfaces When Complete Separation of Interface and Implementation Is Needed». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rh-separation>.

⁵ «C.129: When Designing a Class Hierarchy, Distinguish Between Implementation Inheritance and Interface Inheritance». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rh-kind>.

В интерфейсах обычно нет никаких элементов данных. В отличие от наследования реализации, наследование интерфейса может потребовать от вас больше работы, поскольку конкретные классы должны предоставлять и элементы данных, и реализации чистых виртуальных функций, даже если все они похожи или идентичны во многих конкретных классах. Зато, как мы покажем в конце раздела, этот подход дает вам больше гибкости благодаря отсутствию тесных связей между классами.

Интерфейсы упрощают модификацию сложного программного обеспечения. В разделе 10.8.2 мы объяснили это на хорошем примере с драйверами устройств для операционных систем.

10.12.1. Переосмысление иерархии класса `Employee`: интерфейс `CompensationModel`

Рассмотрим иерархию класса `Employee` из раздела 10.9 с композицией и интерфейсом. Можно сказать, что каждый объект `Employee` содержит `CompensationModel`. На илл. 10.26 вы видите определение интерфейса `CompensationModel`, который является чистым абстрактным классом, поэтому не имеет файла `.cpp`. В этом классе есть сгенерированный компилятором виртуальный деструктор и две чистые виртуальные функции:

- ♦ `earnings` — для расчета зарплаты сотрудника на основе его `CompensationModel`;
- ♦ `toString` — для создания строкового представления `CompensationModel`.

Любой класс, унаследованный от `CompensationModel` и переопределяющий его чистые виртуальные функции, является классом `CompensationModel`, реализующим этот интерфейс.

```
1 // Fig. 10.26: CompensationModel.h
2 // Интерфейс (чистый абстрактный класс) CompensationModel
3 #pragma once // Предотвращаем повторные включения заголовка
4 #include <string>
5
6 class CompensationModel {
7 public:
8     virtual ~CompensationModel() = default; // Сгенерированный деструктор
9     virtual double earnings() const = 0; // Чистая виртуальная функция
10    virtual std::string toString() const = 0; // Чистая виртуальная функция
11 };
```

Илл. 10.26 | Интерфейс (чистый абстрактный класс) `CompensationModel`

10.12.2. Класс `Employee`

На илл. 10.27 показан наш новый класс `Employee`. Каждый объект `Employee` содержит указатель на соответствующую реализацию `CompensationModel` (строка 16). Обратите внимание, что этот класс объявлен финальным (`final`), поэтому его нельзя использовать в качестве базового.

```
1 // Fig. 10.27: Employee.h
2 // Класс Employee содержит CompensationModel
3 #pragma once // Предотвращаем повторные включения заголовка
4 #include <string>
5 #include <string_view>
6 #include "CompensationModel.h"
7
8 class Employee final {
9 public:
10     Employee(std::string_view name, CompensationModel* modelPtr);
11     void setCompensationModel(CompensationModel *modelPtr);
12     double earnings() const;
13     std::string toString() const;
14 private:
15     std::string m_name{};
16     CompensationModel* m_modelPtr{}; // Указатель на объект реализации
17 };
```

Илл. 10.27 | Класс Employee содержит CompensationModel

В коде, показанном на илл. 10.28, определены функции класса Employee:

- ♦ Конструктор (строки 10–11) инициализирует имя сотрудника и направляет свой указатель CompensationModel на объект, реализующий интерфейс CompensationModel. Этот метод известен как **внедрение конструктора (constructor injection)**. Конструктор получает указатель (или ссылку) на другой объект и сохраняет его в создаваемом объекте¹.
- ♦ Функция setCompensationModel (строки 15–17) позволяет клиентскому коду изменять CompensationModel сотрудника, направляя указатель m_modelPtr на другой объект CompensationModel. Этот метод известен как **внедрение свойств (property injection)**.
- ♦ Функция earnings (строки 20–22) определяет зарплату сотрудника, вызывая из соответствующей реализации CompensationModel функцию earnings через указатель m_modelPtr.
- ♦ Функция toString (строки 25–27) создает строковое представление объекта Employee (с именем сотрудника и сведениями о зарплате), вызывая из соответствующей реализации CompensationModel функцию toString через указатель m_modelPtr.

Внедрение конструктора и внедрение свойств — это две формы **внедрения зависимостей (dependency injection)**. Вы определяете часть поведения объекта, предоставляя указатель или ссылку на другой объект, от которого зависит это

¹ При внедрении зависимостей объект CompensationModel должен существовать дольше объекта Employee, чтобы предотвратить логическую ошибку во время выполнения, известную как висячий указатель. Мы обсуждаем висячие указатели в разделе 11.6.

поведение¹. В нашем примере поведение, позволяющее рассчитывать зарплату и генерировать строковое представление, зависит от `CompensationModel`.

```

1 // Fig. 10.28: Employee.cpp
2 // Определения функций класса Employee
3 #include <fmt/format.h>
4 #include <string>
5 #include "CompensationModel.h"
6 #include "Employee.h"
7
8 // Внедряем конструктор для инициализации
9 // указателя CompensationModel на реализацию CompensationModel
10 Employee::Employee(std::string_view name, CompensationModel* modelPtr)
11     : m_name{name}, m_modelPtr{modelPtr} {}
12
13 // Set-функция выполняет внедрение свойств для направления
14 // указателя CompensationModel на новую реализацию CompensationModel
15 void Employee::setCompensationModel(CompensationModel* modelPtr) {
16     m_modelPtr = modelPtr;
17 }
18
19 // Используем CompensationModel для расчета зарплаты
20 double Employee::earnings() const {
21     return m_modelPtr->earnings();
22 };
23
24 // Возвращаем строковое представление объекта Employee
25 std::string Employee::toString() const {
26     return fmt::format("{}\n{}", m_name, m_modelPtr->toString());
27 }

```

Илл. 10.28 | Определения функций класса `Employee`

10.12.3. Реализации `CompensationModel`

Теперь мы определим модели расчета зарплаты для разных категорий сотрудников, то есть реализации классов, производных от класса `CompensationModel`. Объекты этих классов будут внедрены в объекты `Employee` для расчета зарплаты каждой категории сотрудников.

Salaried — класс, производный от `CompensationModel`

Модель расчета зарплаты класса `Salaried` (илл. 10.29–10.30) предназначена для сотрудников, получающих фиксированную зарплату (оклад). Этот класс содержит элемент данных `m_salary` и переопределяет функции `earnings` и `toString` из интерфейса `CompensationModel`. В строке 7 класс `Salaried` объявлен финальным (`final`), поэтому он является последним звеном в иерархии `CompensationModel`, его нельзя использовать в качестве базового.

¹ «Dependency Injection». Был доступен: 15 сентября 2023 г. https://en.wikipedia.org/wiki/Dependency_injection.

```

1 // Fig. 10.29: Salaried.h
2 // Класс Salaried реализует интерфейс CompensationModel
3 #pragma once
4 #include <string>
5 #include "CompensationModel.h" // Определение класса CompensationModel
6
7 class Salaried final : public CompensationModel {
8 public:
9     explicit Salaried(double salary);
10    double earnings() const override;
11    std::string toString() const override;
12 private:
13    double m_salary{0.0};
14 };

```

Илл. 10.29 | Класс Salaried реализует интерфейс CompensationModel

```

1 // Fig. 10.30: Salaried.cpp
2 // Определения функций класса Salaried
3 #include <fmt/format.h>
4 #include <stdexcept>
5 #include "Salaried.h" // Определение класса Salaried
6
7 // Конструктор
8 Salaried::Salaried(double salary) : m_salary{salary} {
9     if (m_salary < 0.0) {
10         throw std::invalid_argument("Weekly salary must be >= 0.0");
11     }
12 }
13
14 // Переопределяем чистую виртуальную функцию earnings
15 double Salaried::earnings() const {return m_salary;}
16
17 // Переопределяем чистую виртуальную функцию toString
18 std::string Salaried::toString() const {
19     return fmt::format("salary: ${:.2f}", m_salary);
20 }

```

Илл. 10.30 | Определения функций класса Salaried

Commission — класс, производный от CompensationModel

Модель оплаты труда класса **Commission** (илл. 10.31–10.32) предназначена для сотрудников отдела продаж, получающих в качестве зарплаты комиссионное вознаграждение, зависящее от объема продаж. Этот класс содержит элементы данных `m_grossSales` и `m_commissionRate` и переопределяет функции `earnings` и `toString` из интерфейса **CompensationModel**. Как и класс **Salaried**, класс **Commission** объявлен финальным (строка 7), поэтому является последним звеном в иерархии **CompensationModel**, его нельзя использовать в качестве базового.

```

1 // Fig. 10.31: Commission.h
2 // Класс Commission реализует интерфейс CompensationModel
3 #pragma once
4 #include <string>
5 #include "CompensationModel.h" // Определение класса CompensationModel
6
7 class Commission final : public CompensationModel {
8 public:
9     Commission(double grossSales, double commissionRate);
10    double earnings() const override;
11    std::string toString() const override;
12 private:
13    double m_grossSales{0.0};
14    double m_commissionRate{0.0};
15 };

```

Илл. 10.31 | Класс Commission реализует интерфейс CompensationModel

```

1 // Fig. 10.32: Commission.cpp
2 // Определения функций класса Commission
3 #include <fmt/format.h>
4 #include <stdexcept>
5 #include "Commission.h" // Определение класса Commission
6
7 // Конструктор
8 Commission::Commission(double grossSales, double commissionRate)
9     : m_grossSales{grossSales}, m_commissionRate{commissionRate} {
10
11 if (m_grossSales < 0.0) {
12     throw std::invalid_argument("Gross sales must be >= 0.0");
13 }
14
15 if (m_commissionRate <= 0.0 || m_commissionRate >= 1.0) {
16     throw std::invalid_argument(
17         "Commission rate must be > 0.0 and < 1.0");
18 }
19 }
20
21 // Переопределяем чистую виртуальную функцию earnings
22 double Commission::earnings() const {
23     return m_grossSales * m_commissionRate;
24 }
25
26 // Переопределяем чистую виртуальную функцию toString
27 std::string Commission::toString() const {
28     return fmt::format("gross sales: ${:.2f}; commission rate: {:.2f}",
29         m_grossSales, m_commissionRate);
30 }

```

Илл. 10.32 | Определения функций класса Commission

10.12.4. Тест-драйв новой иерархии

Итак, мы создали интерфейс `CompensationModel` и две реализации производных классов: две модели расчета зарплаты. Теперь мы создадим объекты класса `Employee` и инициализируем их конкретными реализациями `CompensationModel` (илл. 10.33).

```
1 // fig10_33.cpp
2 // Расчет зарплаты на основе реализаций CompensationModel
3 #include <fmt/format.h>
4 #include <iostream>
5 #include <vector>
6 #include "Employee.h"
7 #include "Salaried.h"
8 #include "Commission.h"
9
10 int main() {
11     // Создаем объекты, представляющие сотрудников и модели расчета зарплаты
12     Salaried salaried{800.0};
13     Employee salariedEmployee{"John Smith", &salaried};
14
15     Commission commission{10000, .06};
16     Employee commissionEmployee{"Sue Jones", &commission};
17
18     // Создаем и инициализируем вектор объектов Employee
19     std::vector<Employee> employees{salariedEmployee, commissionEmployee};
20
21     // Выводим на экран строковые представления объектов Employee
22     for (const Employee& employee : employees) {
23         std::cout << fmt::format("{}\nearned: {:.2f}\n\n",
24             employee.toString(), employee.earnings());
25     }
26 }
```

```
John Smith
salary: $800.00
earned: $800.00

Sue Jones
gross sales: $10000.00; commission rate: 0.06
earned: $600.00
```

Илл. 10.33 | Расчет зарплаты на основе реализаций `CompensationModel`

В строке 12 создается объект класса `Salaried` (`salaried`). В строке 13 создается объект `salariedEmployee`, в который внедрена модель расчета фиксированной зарплаты: указатель на объект `salaried` является вторым аргументом конструктора. В строке 15 создается объект класса `Commission` (`commission`). В строке 16 создается объект `commissionEmployee`, в который внедрена модель расчета комиссионной зарплаты: указатель на объект `commission` является вторым аргументом конструктора¹.

¹ В этом примере объекты `CompensationModel` переживут свои соответствующие объекты `Employee`, поэтому исключен риск возникновения висячего указателя.

В строке 19 создается вектор объектов `Employee`, инициализированный объектами `salariedEmployee` и `commissionEmployee`. Наконец, в строках 22–25 перебираются элементы вектора и на экран выводится строковое представление каждого объекта `Employee` и зарплата обоих сотрудников.

10.12.5. Преимущества внедрения зависимостей

Простота модификации при изменениях моделей расчета зарплаты

Поскольку все модели расчета зарплаты — это отдельные классы, реализующие один и тот же интерфейс `CompensationModel`, мы можем легко вносить в систему изменения по мере необходимости. Предположим, что в компании появились новые модели оплаты труда. Для каждого из них достаточно определить новый производный от `CompensationModel` класс с соответствующей функцией `earnings`.

Простота модификации при переходе сотрудников между отделами

Архитектура программы, основанная на интерфейсах, композиции и внедрении зависимостей, является более гибкой, чем прежняя, описанная в разделе 10.9. Там в случае изменения категории сотрудника вам пришлось бы изменить тип объекта, представляющего этого сотрудника. Для этого надо было бы создать новый объект соответствующего класса, производного от `Employee`, и переместить данные о сотруднике в новый объект. А благодаря внедрению зависимостей достаточно вызвать функцию `setCompensationModel` и перенаправить указатель с одной реализации `CompensationModel` на другую.

Простота модификации при расширении класса `Employee`

Архитектура программы, основанная на интерфейсах, композиции и внедрении зависимостей, упрощает расширение класса `Employee`. Например, мы решим добавить поддержку пенсионных планов «401(k)» и «IRA». В этом случае мы можем сказать, что каждый объект `Employee` содержит («has-a») `RetirementPlan`. Сначала мы определим интерфейс `RetirementPlan` с функцией `makeRetirementDeposit`, а также его реализации в производных классах. После этого надо будет внести в класс `Employee` лишь небольшие изменения:

- ♦ добавить элемент данных, указывающий на `RetirementPlan`;
- ♦ добавить аргумент конструктора для инициализации указателя на `RetirementPlan`;
- ♦ добавить функцию `setRetirementPlan`, позволяющую изменять пенсионный план сотрудника.

10.13. Полиморфизм на основе шаблона класса `std::variant` и функции `std::visit`

Итак, мы реализовали динамический полиморфизм методами наследования реализации и наследования интерфейса. Оба метода требуют наличия иерархии

классов. А можно ли реализовать динамический полиморфизм применительно к объектам классов, не связанных между собой? Да, для этого в стандартной библиотеке (C++17) есть шаблон класса `std::variant` и функция `std::visit` из заголовка `<variant>`^{1,2,3,4}. Особенность этого метода заключается в том, что вы должны заранее знать все типы, которые понадобятся вашей программе для полиморфной обработки, — их называют **замкнутым набором типов (closed set of types)**. Объект `std::variant` может хранить только один объект любого типа, указанного при создании объекта `std::variant`. Как вы скоро увидите, доступ к функциям объектов, хранящихся в объектах `std::variant`, осуществляется через функцию `std::visit`.

Для демонстрации полиморфизма на основе шаблона класса `std::variant` мы доработаем пример из раздела 10.12. Классы останутся почти такими же, как прежде, поэтому мы укажем только на отличия.

Класс Salaried

Класс `Salaried` (илл. 10.34–10.35) определяет модель расчета фиксированной зарплаты для объектов класса `Employee`. Единственное отличие этого класса от предыдущего (описанного в разделе 10.12) заключается в том, что теперь класс `Salaried` не является производным. Таким образом, его функции `earnings` и `toString` не переопределяют виртуальные функции базового класса.

```

1 // Fig. 10.34: Salaried.h
2 // Класс Salaried
3 #pragma once
4 #include <string>
5
6 class Salaried {
7 public:
8     Salaried(double salary);
9     double earnings() const;
10    std::string toString() const;
11 private:
12    double m_salary{0.0};
13 };

```

Илл. 10.34 | Класс Salaried

¹ Невин Либер (Nevin Liber), «The Many Variants of std::variant». Был доступен: 15 сентября 2023 г. <https://www.youtube.com/watch?v=JUxhwf7gYLg>.

² «std::variant». Был доступен: 15 сентября 2023 г. <https://en.cppreference.com/w/cpp/utility/variant>.

³ Бартоломей Филипек (Bartłomiej Filipek), «Runtime Polymorphism with std::variant and std::visit». Был доступен: 15 сентября 2023 г. <https://www.bfilipek.com/2020/04/variant-virtualpolymorphism.html>.

⁴ Бартоломей Филипек, «Everything You Need to Know About std::variant from C++17». Был доступен: 15 сентября 2023 г. <https://www.bfilipek.com/2018/06/variant.html>.

```
1 // Fig. 10.35: Salaried.cpp
2 // Определения функций класса Salaried
3 #include <fmt/format.h>
4 #include <stdexcept>
5 #include "Salaried.h" // Определение класса Salaried
6
7 // Конструктор
8 Salaried::Salaried(double salary) : m_salary{salary} {
9     if (m_salary < 0.0) {
10         throw std::invalid_argument("Weekly salary must be >= 0.0");
11     }
12 }
13
14 // Рассчитываем зарплату
15 double Salaried::earnings() const {return m_salary;}
16
17 // Возвращаем строковое представление объекта
18 std::string Salaried::toString() const {
19     return fmt::format("salary: {:.2f}", m_salary);
20 }
```

Илл. 10.35 | Определения функций класса `Salaried`

Класс `Commission`

Класс `Commission` (илл. 10.36–10.37) определяет модель расчета зарплаты для тех объектов класса `Employee`, для которых рассчитывается комиссионное вознаграждение в зависимости от объема продаж. Как и класс `Salaried`, класс `Commission` не является производным. Таким образом, его функции `earnings` и `toString` не переопределяют виртуальные функции базового класса, как в разделе 10.12.

```
1 // Fig. 10.36: Commission.h
2 // Класс Commission
3 #pragma once
4 #include <string>
5
6 class Commission {
7 public:
8     Commission(double grossSales, double commissionRate);
9     double earnings() const;
10     std::string toString() const;
11 private:
12     double m_grossSales{0.0};
13     double m_commissionRate{0.0};
14 };
```

Илл. 10.36 | Класс `Commission`

```

1 // Fig. 10.37: Commission.cpp
2 // Определения функций класса Commission
3 #include <fmt/format.h>
4 #include <stdexcept>
5 #include "Commission.h" // Определение класса Commission
6
7 // Конструктор
8 Commission::Commission(double grossSales, double commissionRate)
9     : m_grossSales{grossSales}, m_commissionRate{commissionRate} {
10
11     if (m_grossSales < 0.0) {
12         throw std::invalid_argument("Gross sales must be >= 0.0");
13     }
14
15     if (m_commissionRate <= 0.0 || m_commissionRate >= 1.0) {
16         throw std::invalid_argument(
17             "Commission rate must be > 0.0 and < 1.0");
18     }
19 }
20
21 // Рассчитываем зарплату
22 double Commission::earnings() const {
23     return m_grossSales * m_commissionRate;
24 }
25
26 // Возвращаем строковое представление объекта
27 std::string Commission::toString() const {
28     return fmt::format("gross sales: ${:.2f}; commission rate: {:.2f}",
29         m_grossSales, m_commissionRate);
30 }

```

Илл. 10.37 | Определения функций класса `Commission`

Определение класса `Employee`

Как и в разделе 10.12, каждый объект `Employee` (илл. 10.38) содержит модель расчета зарплаты (строка 21). Однако в этом примере модель расчета представляет собой объект `std::variant`, содержащий либо объект класса `Commission`, либо объект класса `Salaried`, — обратите внимание, что это объект, а не указатель на объект. В строке 11 есть объявление `using`:

```
using CompensationModel = std::variant<Commission, Salaried>;
```

Оно объявляет псевдоним `CompensationModel` для типа `std::variant`:

```
std::variant<Commission, Salaried>
```

Такие объявления `using` (C++11) называются **объявлениями псевдонимов (alias declarations)**. Они позволяют создавать удобные имена для сложных типов, таких как `std::variant`, которые потенциально могут иметь много параметров. В любой момент времени объект нашего типа `std::variant` может хранить либо объект `Commission`, либо объект `Salaried`. В строке 21 мы используем псевдоним

`CompensationModel`, определяющий объект `std::variant`, в котором хранится модель расчета зарплаты.

```
1 // Fig. 10.38: Employee.h
2 // Класс Employee содержит CompensationModel
3 #pragma once // Предотвращаем повторные включения заголовка
4 #include <string>
5 #include <string_view>
6 #include <variant>
7 #include "Commission.h"
8 #include "Salaried.h"
9
10 // Определяем удобное имя для типа std::variant
11 using CompensationModel = std::variant<Commission, Salaried>;
12
13 class Employee {
14 public:
15     Employee(std::string_view name, CompensationModel model);
16     void setCompensationModel(CompensationModel model);
17     double earnings() const;
18     std::string toString() const;
19 private:
20     std::string m_name{};
21     CompensationModel m_model; // Внимание: это не указатель!
22 };
```

Илл. 10.38 | Класс `Employee` содержит `CompensationModel`

Типобезопасное объединение

В языках C и C++ **объединение (union)**¹ — это объект, в котором могут поочередно храниться объекты разных типов. Все элементы объединения совместно используют одну и ту же область памяти, поэтому объединение может содержать в любой момент времени не более одного объекта и требует достаточного объема памяти для самого большого из своих возможных элементов. Объект типа `std::variant` часто называют **типовым безопасным объединением (type-safe union)**².

Конструктор класса `Employee` и функция `setCompensationModel`

Функции класса `Employee` (илл. 10.39) выполняют те же задачи, что и аналогичные функции на илл. 10.28, но с несколькими изменениями. Конструктор класса `Employee` (строки 8–9) и функция `setCompensationModel` (строки 12–14) получают объект `CompensationModel`. В отличие от архитектуры с композицией и внедрением зависимостей, в объекте `std::variant` хранится не указатель на объект, а сам объект.

¹ «Union Declaration». Был доступен: 15 сентября 2023 г. <https://en.cppreference.com/w/cpp/language/union>.

² «std::variant». Был доступен: 15 сентября 2023 г. <https://en.cppreference.com/w/cpp/utility/variant>.

```

1 // Fig. 10.39: Employee.cpp
2 // Определения функций класса Employee
3 #include <fmt/format.h>
4 #include <string>
5 #include "Employee.h"
6
7 // Конструктор
8 Employee::Employee(std::string_view name, CompensationModel model)
9     : m_name{name}, m_model{model} {}
10
11 // Изменяем CompensationModel
12 void Employee::setCompensationModel(CompensationModel model) {
13     m_model = model;
14 }
15
16 // Возвращаем оклад
17 double Employee::earnings() const {
18     auto getEarnings{[](const auto& model){return model.earnings();}};
19     return std::visit(getEarnings, m_model);
20 }
21
22 // Возвращаем строковое представление объекта Employee
23 std::string Employee::toString() const {
24     auto getString{[](const auto& model){return model.toString();}};
25     return fmt::format("{}\n{}", m_name, std::visit(getString, m_model));
26 }

```

Илл. 10.39 | Определения функций класса Employee

Вызов функций `earnings` и `toString` класса `Employee` через функцию `std::visit`

Ключевое различие между полиморфизмом на основе иерархии классов и полиморфизмом на основе `std::variant` заключается в том, что объект `std::variant` не может вызывать функции хранимого объекта. Для вызова этих функций вам надо использовать функцию `std::visit` стандартной библиотеки. Рассмотрим строки 18–19 в функции `earnings`:

```

auto getEarnings{[](const auto& model){return model.earnings();}};
return std::visit(getEarnings, m_model);

```

В строке 18 определяется переменная `getEarnings` и инициализируется обобщенным лямбда-выражением, которое получает ссылку на объект (`model`) и вызывает его функцию `earnings`. Это лямбда-выражение может вызывать функцию `earnings` любого объекта, которая не принимает аргументы и возвращает значение. В строке 19 лямбда-выражение `getEarnings` и объект `m_model` передаются в функцию `std::visit`. Функция `std::visit` передает `m_model` в лямбда-выражение и возвращает результат вызова функции `earnings` объекта `m_model`.

Аналогично в функции `toString` класса `Employee` (строка 24) создается лямбда-выражение, возвращающее результат вызова функции `toString` некоторого объекта.

В строке 25 вызывается функция `std::visit`, которая передает `m_model` в лямбда-выражение и возвращает результат вызова функции `toString` объекта `m_model`.

Тест-драйв динамического полиморфизма на основе `std::variant` и `std::visit`

В функции `main`, в строках 11–12 (илл. 10.40), создаются два объекта класса `Employee`. Первый хранит в своем `std::variant` объект класса `Salaried`, а второй — объект класса `Commission`. Выражение в строке 11

```
Salaried{800.0}
```

создает объект `Salaried`, который немедленно используется для инициализации `CompensationModel` объекта `Employee`. Аналогично выражение в строке 12

```
Commission{10000.0, .06}
```

создает объект `Commission`, который немедленно используется для инициализации `CompensationModel` объекта `Employee`.

```
1 // fig10_40.cpp
2 // Расчет зарплаты для разных категорий сотрудников
3 #include <fmt/format.h>
4 #include <iostream>
5 #include <vector>
6 #include "Employee.h"
7 #include "Salaried.h"
8 #include "Commission.h"
9
10 int main() {
11     Employee salariedEmployee{"John Smith", Salaried{800.0}};
12     Employee commissionEmployee{"Sue Jones", Commission{10000.0, .06}};
13
14     // Создаем и инициализируем вектор объектов Employee
15     std::vector employees{salariedEmployee, commissionEmployee};
16
17     // Выводим на экран данные о сотрудниках и их зарплате
18     for (const Employee& employee : employees) {
19         std::cout << fmt::format("{}\nearned: {:.2f}\n\n",
20             employee.toString(), employee.earnings());
21     }
22 }
```

```
John Smith
salary: $800.00
earned: $800.00
```

```
Sue Jones
gross sales: $10000.00; commission rate: 0.06
earned: $600.00
```

Илл. 10.40 | Расчет зарплаты для разных категорий сотрудников

В строке 15 создается вектор объектов `Employee`. В строках 18–21 перебираются элементы вектора, при этом вызываются функции `earnings` и `toString` каждого объекта. Мы реализовали полиморфную обработку и получили такие же результаты, как в разделе 10.12. Возможность вызывать функциональность, общую для многих объектов, не связанных иерархией классов, часто называют **утиной типизацией** (*duck typing*):

Если это ходит как утка и крикает как утка, то это должно быть уткой¹.

Иными словами, если объект имеет соответствующие функции и его тип является элементом `std::variant`, то объект будет работать в этом коде.

10.14. Множественное наследование

Мы рассмотрели одиночное наследование, при котором класс является производным только от одного базового класса. Но C++ поддерживает и множественное наследование: класс может наследовать элементы данных от двух и более базовых классов. Множественное наследование — это сложный инструмент, предназначенный только для опытных программистов. Некоторые проблемы множественного наследования настолько сложны, что новые языки программирования, такие как Java и C#, поддерживают только одиночное наследование². Нужно быть крайне осторожным при проектировании системы с множественным наследованием. Его не следует использовать, если можно обойтись одиночным наследованием и (или) композицией.

При множественном наследовании в нескольких базовых классах могут оказаться элементы данных и функции с одинаковыми именами. Это создает проблему неоднозначной идентификации при компиляции. На сайте isocpp.org, в разделе часто задаваемых вопросов, рекомендуется выполнять множественное наследование только от чистых абстрактных базовых классов, чтобы избежать этой проблемы и других проблем, которые мы обсудим в этом разделе, а затем в разделе 10.14.1³.

Пример множественного наследования

Рассмотрим пример множественного наследования реализации (илл. 10.41–10.45). Класс `Base1` (илл. 10.41) содержит:

- ♦ скрытый элемент данных типа `int` (`m_value`; строка 11);
- ♦ конструктор (строка 8), устанавливающий значение `m_value`;
- ♦ открытую функцию `getData` (строка 9), возвращающую значение `m_value`.

¹ «Утиная типизация». Был доступен: 15 сентября 2023 г. https://ru.wikipedia.org/wiki/Утиная_типизация.

² Точнее, Java и C# поддерживают только одиночное наследование реализации, допуская множественное наследование интерфейса.

³ «Inheritance — Multiple and Virtual Inheritance». Был доступен: 15 сентября 2023 г. <https://isocpp.org/wiki/faq/multiple-inheritance>.

```
1 // Fig. 10.41: Base1.h
2 // Пример множественного наследования, Base1.h
3 #pragma once
4
5 // Определение класса Base1
6 class Base1 {
7 public:
8     explicit Base1(int value) : m_value{value} {}
9     int getData() const {return m_value;}
10 private: // Доступ для производных классов - через функцию getData
11     int m_value;
12 };
```

Илл. 10.41 | Пример множественного наследования, Base1.h

Класс **Base2** (илл. 10.42) похож на класс **Base1**, но его скрытый элемент данных имеет тип `char` (`m_letter`, строка 11). Подобно классу **Base1**, класс **Base2** содержит открытую функцию `getData`, но она возвращает значение переменной `m_letter`.

```
1 // Fig. 10.42: Base2.h
2 // Пример множественного наследования, Base2.h
3 #pragma once
4
5 // Определение класса Base2
6 class Base2 {
7 public:
8     explicit Base2(char letter) : m_letter{letter} {}
9     char getData() const {return m_letter;}
10 private: // Доступ для производных классов - через функцию getData
11     char m_letter;
12 };
```

Илл. 10.42 | Пример множественного наследования, Base2.h

Класс **Derived** (илл. 10.43–10.44) наследуется от классов **Base1** и **Base2**. Этот производный класс содержит:

- ♦ скрытый элемент данных `m_real` типа `double` (илл. 10.43, строка 18);
- ♦ конструктор для инициализации всех данных класса **Derived**;
- ♦ открытую функцию `getReal`, возвращающую значение `m_real`;
- ♦ открытую функцию `toString`, возвращающую строковое представление объекта **Derived**.

```
1 // Fig. 10.43: Derived.h
2 // Определение класса Derived, унаследованного от
3 // двух базовых классов: Base1 и Base2
4 #pragma once
5 #include <iostream>
6 #include <string>
```

```

7 #include "Base1.h"
8 #include "Base2.h"
9 using namespace std;
10
11 // Определение класса Derived
12 class Derived : public Base1, public Base2 {
13 public:
14     Derived(int value, char letter, double real);
15     double getReal() const;
16     std::string toString() const;
17 private:
18     double m_real; // Скрытый элемент данных производного класса
19 };

```

Илл. 10.43 | Пример множественного наследования, Derived.h

```

1 // Fig. 10.44: Derived.cpp
2 // Определения функций класса Derived
3 #include <fmt/format.h> // Или в C++20: #include <format>
4 #include "Derived.h"
5
6 // Конструктор для вызова конструкторов классов Base1 и Base2
7 Derived::Derived(int value, char letter, double real)
8     : Base1{value}, Base2{letter}, m_real{real} {}
9
10 // Возвращаем значение m_real
11 double Derived::getReal() const {return m_real;}
12
13 // Выводим на экран элементы данных Derived
14 std::string Derived::toString() const {
15     return fmt::format("int: {}; char: {}; double: {}",
16         Base1::getData(), Base2::getData(), getReal());
17 }

```

Илл. 10.44 | Пример множественного наследования, Derived.cpp

Для множественного наследования (илл. 10.43) мы добавляем после имени производного класса и двоеточия (`Derived :`) список базовых классов, разделенный запятыми (строка 12). Обратите внимание, что на илл. 10.44 конструктор производного класса явно вызывает конструкторы каждого базового класса (`Base1` и `Base2`), используя синтаксис инициализации элементов (строка 8). Конструкторы базового класса вызываются в порядке наследования. Если список инициализаторов явно не вызывает конструктор базового класса, будет неявно вызван конструктор базового класса по умолчанию.

Решение проблемы одноименных функций базовых классов при множественном наследовании

Функция `toString` (строки 14–17) возвращает строковое представление объекта производного класса `Derived`, используя все *get*-функции производного класса.

Здесь могла бы возникнуть проблема неоднозначности, ведь объект `Derived` содержит две функции `getData`: одна из них унаследована от класса `Base1`, другая — от класса `Base2`. Но мы легко решили эту проблему с помощью оператора разрешения области видимости. Вызов функции `Base1::getData()` возвращает значение переменной, унаследованной от класса `Base1` (`m_value`), а вызов `Base2::getData()` возвращает значение переменной, унаследованной от класса `Base2` (`m_letter`).

Тест-драйв иерархии с множественным наследованием

На илл. 10.45 показана программа для тестирования нашей иерархии классов (илл. 10.41–10.44). В строке 10 создается объект `base1` класса `Base1` и инициализируется целым числом 10. В строке 11 создается объект `base2` класса `Base2` и инициализируется символом 'Z'. В строке 12 создается объект `derived` класса `Derived` и инициализируется значениями 7, 'A' и 3.5 соответственно.

```

1 // fig10_45.cpp
2 // Пример множественного наследования, тест-драйв
3 #include <fmt/format.h>
4 #include <iostream>
5 #include "Base1.h"
6 #include "Base2.h"
7 #include "Derived.h"
8
9 int main() {
10     Base1 base1{10}; // Объект базового класса Base1
11     Base2 base2{'Z'}; // Объект базового класса Base2
12     Derived derived{7, 'A', 3.5}; // Объект производного класса Derived
13
14     // Выводим на экран элементы данных всех объектов
15     std::cout << fmt::format("{}: {}\n{}: {}\n{}: {}\n\n",
16         "Object base1 contains", base1.getData(),
17         "Object base2 contains the character", base2.getData(),
18         "Object derived contains", derived.toString());
19
20     // Выводим на экран элементы данных объекта производного класса
21     // с помощью оператора разрешения области видимости
22     std::cout << fmt::format("{}\n{}: {}\n{}: {}\n\n",
23         "Data members of Derived can be accessed individually:",
24         "int", derived.Base1::getData(),
25         "char", derived.Base2::getData(),
26         "double", derived.getReal());
27
28     std::cout << "Derived can be treated as an object"
29         << " of either base class:\n";
30
31     // Обрабатываем объект Derived как объект Base1
32     Base1* base1Ptr = &derived;
33     std::cout << fmt::format("base1Ptr->getData() yields {}\n",
34         base1Ptr->getData());
35
36     // Обрабатываем объект Derived как объект Base2

```



```

37  Base2* base2Ptr = &derived;
38  std::cout << fmt::format("base2Ptr->getData() yields {}\n",
39      base2Ptr->getData());
40 }

```

```

Object base1 contains: 10
Object base2 contains the character: Z
Object derived contains: int: 7; char: A; double: 3.5

```

Data members of Derived can be accessed individually:

```

int: 7
char: A
double: 3.5

```

Derived can be treated as an object of either base class:

```

base1Ptr->getData() yields 7
base2Ptr->getData() yields A

```

Илл. 10.45 | Пример множественного наследования, тест-драйв

В строках 15–18 на экран выводятся данные каждого объекта. Для каждого из объектов `base1` и `base2` вызывается его функция `getData`. Несмотря на одинаковые имена функций, эти вызовы не являются неоднозначными. В строке 16 компилятор знает, что `base1` является объектом класса `Base1`, поэтому вызывается функция `getData` класса `Base1`. В строке 17 компилятор знает, что `base2` является объектом класса `Base2`, поэтому вызывается функция `getData` класса `Base2`. В строке 18 вызывается функция `toString` объекта `derived` для вывода на экран его данных.

В строках 22–26 данные объекта `derived` выводятся снова, но уже с помощью `get`-функций класса `Derived`. Здесь тоже могла бы возникнуть проблема неоднозначности, поскольку этот объект содержит две функции с именем `getData`: из класса `Base1` и из класса `Base2`. Выражение:

```
derived.Base1::getData()
```

получает значение `m_value`, унаследованное от класса `Base1`, а выражение:

```
derived.Base2::getData()
```

получает значение `m_letter`, унаследованное от класса `Base2`.

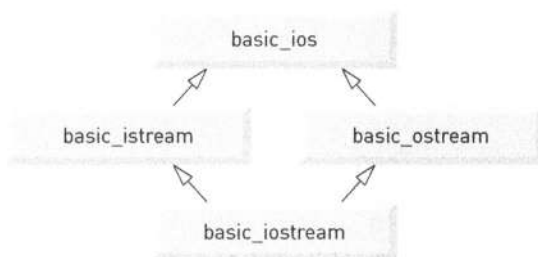
Отношение «является» при множественном наследовании

Отношение «является» («is-a»), свойственное одиночному наследованию, применимо и к множественному наследованию. В строке 32 мы присваиваем адрес объекта `derived` указателю `base1Ptr` типа `Base1`. Это разрешено, потому что объект `Derived` является объектом `Base1`. В строке 34 мы вызываем функцию `getData` класса `Base1` через указатель `base1Ptr`, чтобы получить значение элемента данных, унаследованного производным классом от класса `Base1`. В строке 37 мы присваиваем адрес объекта `derived` указателю `base2Ptr` типа `Base2`. Это разрешено, потому

что объект `Derived` является объектом `Base2`. В строке 39 мы вызываем функцию `getData` класса `Base2` через указатель `base2Ptr`, чтобы получить значение элемента данных, унаследованного производным классом от класса `Base2`.

10.14.1. Ромбовидное наследование

В предыдущем примере мы рассмотрели множественное наследование, при котором один класс может быть производным от двух и более классов. Например, множественное наследование, показанное на следующей диаграмме, используется в стандартной библиотеке C++ для формирования класса `basic_iostream`:



Класс `basic_ios` является базовым для классов `basic_istream` и `basic_ostream`, которые формируются путем одиночного наследования. Класс `basic_iostream` является производным от `basic_istream` и от `basic_ostream`. Это позволяет объектам класса `basic_iostream` обеспечивать функциональность классов `basic_istream` и `basic_ostream`. Показанное на этой диаграмме наследование и иерархиях с множественным наследованием называется **ромбовидным наследованием (diamond inheritance)**.

Классы `basic_istream` и `basic_ostream` являются производными от класса `basic_ios`, поэтому у класса `basic_iostream` есть потенциальная проблема. Он может наследовать от `basic_ios` две копии элементов данных: одну — через класс `basic_istream`, а другую — через класс `basic_ostream`. Эта неоднозначность может вызвать ошибку компиляции, если компилятор не будет знать, какую копию элементов данных класса `basic_ios` надо использовать. Давайте рассмотрим, как эту проблему решают виртуальные базовые классы.

Ошибки компиляции, возникающие при неоднозначности в ромбовидном наследовании

На илл. 10.46 приведен пример неоднозначности, которая может возникнуть при ромбовидном наследовании. Класс `Base` (строки 7–10) содержит чистую виртуальную функцию `print` (строка 9). Классы `DerivedOne` (строки 13–17) и `DerivedTwo` (строки 20–24) открыто наследуют и переопределяют функцию `print` класса `Base`. Каждый из классов `DerivedOne` и `DerivedTwo` содержит **подобъект базового класса (base-class subobject)** — элемент класса `Base`.

```
1 // fig10_46.cpp
2 // Попытка полиморфного вызова функции, унаследованной
3 // от каждого из двух базовых классов
4 #include <iostream>
5
6 // Определение класса Base
7 class Base {
8 public:
9     virtual void print() const = 0; // Чистая виртуальная функция
10 };
11
12 // Определение класса DerivedOne
13 class DerivedOne : public Base {
14 public:
15     // Переопределяем функцию print
16     void print() const override {std::cout << "DerivedOne\n";}
17 };
18
19 // Определение класса DerivedTwo
20 class DerivedTwo : public Base {
21 public:
22     // Переопределяем функцию print
23     void print() const override {std::cout << "DerivedTwo\n";}
24 };
25
26 // Определение класса Multiple
27 class Multiple : public DerivedOne, public DerivedTwo {
28 public:
29     // Квалифицируем версию функции print
30     void print() const override {DerivedTwo::print();}
31 };
32
33 int main() {
34     Multiple both{}; // Создаем объект класса Multiple
35     DerivedOne one{}; // Создаем объект класса DerivedOne
36     DerivedTwo two{}; // Создаем объект класса DerivedTwo
37     Base* array[3]{}; // Создаем массив указателей базового класса
38
39     array[0] = &both; // Ошибка неоднозначности!
40     array[1] = &one;
41     array[2] = &two;
42
43     // Полиморфно вызываем функцию print
44     for (int i{0}; i < 3; ++i) {
45         array[i] ->print();
46     }
47 }
```

Сообщение компилятора Visual C++ об ошибке:

```
fig10_46.cpp(39,20): error C2594: '=': ambiguous conversions from 'Multiple *'
to 'Base *'
```

Илл. 10.46 | Попытка полиморфного вызова функции, унаследованной от каждого из двух базовых классов

Класс `Multiple` (строки 27–31) является производным от классов `DerivedOne` и `DerivedTwo`. Функция `print` класса `Multiple` переопределена для вызова функции `print` класса `DerivedTwo` (строка 30). Обратите внимание, что мы должны были квалифицировать вызов функции `print`. В данном случае мы использовали имя класса `DerivedTwo`, чтобы указать, какую версию функции `print` следует вызывать.

В функции `main` (строки 33–47) объявляются объекты классов `Multiple` (строка 34), `DerivedOne` (строка 35) и `DerivedTwo` (строка 36). В строке 37 объявляется традиционный массив указателей базового класса. Каждому элементу массива присваивается адрес объекта (строки 39–41). Ошибка возникает, когда адрес объекта `both` (объекта класса `Multiple`) присваивается элементу `array[0]`. Объект `both` содержит два подобъекта класса `Base`. Компилятор не знает, на какой из подобъектов должен указывать элемент `array[0]`, поэтому генерирует ошибку «ambiguous conversion».

10.14.2. Устранение конфликта подобъектов с помощью виртуального наследования базового класса

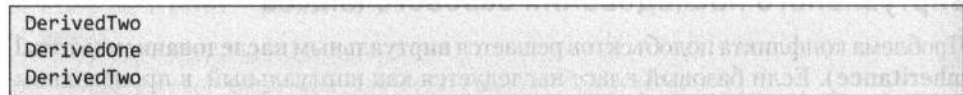
Проблема конфликта подобъектов решается **виртуальным наследованием** (**virtual inheritance**). Если базовый класс наследуется как виртуальный, в производном классе появится только один подобъект. На илл. 10.47 показана модифицированная программа с виртуальными базовыми классами.

```

1 // fig10_47.cpp
2 // Использование виртуальных базовых классов
3 #include <iostream>
4
5 // Определение класса Base
6 class Base {
7 public:
8     virtual void print() const = 0; // Чистая виртуальная функция
9 };
10
11 // Определение класса DerivedOne
12 class DerivedOne : virtual public Base {
13 public:
14     // Переопределяем функцию print
15     void print() const override {std::cout << "DerivedOne\n";}
16 };
17
18 // Определение класса DerivedTwo
19 class DerivedTwo : virtual public Base {
20 public:
21     // Переопределяем функцию print
22     void print() const override {std::cout << "DerivedTwo\n";}
23 };
24
25 // Определение класса Multiple
26 class Multiple : public DerivedOne, public DerivedTwo {
27 public:

```

```
28 // Квалифицируем версию функции print
29 void print() const override {DerivedTwo::print();}
30 };
31
32 int main() {
33     Multiple both; // Создаем объект класса Multiple
34     DerivedOne one; // Создаем объект класса DerivedOne
35     DerivedTwo two; // Создаем объект класса DerivedTwo
36     Base* array[3];
37
38     array[0] = &both; // Теперь разрешено
39     array[1] = &one;
40     array[2] = &two;
41
42     // Полиморфно вызываем функцию print
43     for (int i = 0; i < 3; ++i) {
44         array[i]->print();
45     }
46 }
```



Илл. 10.47 | Использование виртуальных базовых классов

Ключевое изменение заключается в том, что классы `DerivedOne` (строка 12) и `DerivedTwo` (строка 19) наследуются от класса `Base` с ключевым словом `virtual`. Поскольку они являются производными от `Base`, каждый из них содержит подобъекты класса `Base`. Преимущество виртуального наследования еще не очевидно, пока класс `Multiple` не становится производным от классов `DerivedOne` и `DerivedTwo` (строка 26). Поскольку каждый из базовых классов унаследован как виртуальный, компилятор гарантирует, что класс `Multiple` наследует только один подобъект `Base`. Это предотвращает ошибку неоднозначности, показанную на илл. 10.46. Теперь компилятор разрешает неявное преобразование указателя производного класса (`&both`) в указатель базового класса `array[0]` в функции `main` (строка 38). Инструкция `for` в строках 43–45 полиморфно вызывает функцию `print` для каждого объекта.

Конструкторы в иерархиях множественного наследования с виртуальными базовыми классами

Реализация иерархии с виртуальными базовыми классами упрощается, если для базовых классов используются конструкторы по умолчанию. В предыдущем примере (илл. 10.46–10.47) используется конструктор по умолчанию, созданный компилятором. Если виртуальный базовый класс предоставляет конструктор, требующий аргументов, реализации производных классов становятся сложнее. Конечный производный класс должен явно вызывать конструктор виртуального базового класса. По этой причине рассмотрите возможность использования

конструкторов по умолчанию для виртуальных базовых классов, чтобы производным классам не приходилось явно вызывать пользовательские конструкторы виртуальных базовых классов¹.

10.15. Защищенные элементы класса

В главе 9 были представлены спецификаторы доступа `public` и `private`. Открытые (`public`) элементы базового класса доступны в этом классе, а также в любом месте программы, где есть доступ к какому-либо объекту этого класса или одного из его производных классов. Скрытые (`private`) элементы базового класса доступны только в теле этого класса, а также в дружественных ему классах и функциях. Элементы, объявленные со спецификатором доступа `protected`, называются **защищенными** (`protected`). Этот спецификатор дает промежуточный уровень доступа между `public` и `private`. Защищенные элементы доступны в базовом классе, в его элементах, в дружественных ему классах и функциях, а также в классах и функциях, дружественных любому его производным классам.

При открытом наследовании все открытые и защищенные элементы базового класса сохраняют свой первоначальный уровень доступа, когда становятся элементами производного класса:

- ♦ открытые элементы базового класса становятся открытыми элементами производного класса;
- ♦ защищенные элементы базового класса становятся защищенными элементами производного класса.

Скрытые элементы базового класса недоступны за его пределами. Производные классы могут получить доступ к скрытым элементам базового класса только через открытые или защищенные функции, унаследованные от базового класса. Функции производного класса могут ссылаться на унаследованные от базового класса открытые и защищенные элементы по их именам.

Проблемы защищенных элементов данных

Желательно избегать защищенных элементов данных, потому что они могут стать причиной серьезных проблем:

- ♦ Объекту производного класса не обязательно использовать `set`-функцию для доступа к защищенному элементу базового класса. Таким образом, появляется возможность присвоить элементу недопустимое значение, после чего объект перейдет в некорректное состояние. Например, если элемент данных `m_grossSales` класса `CommissionEmployee` будет иметь спецификатор доступа

¹ «Inheritance — Multiple and Virtual Inheritance — What special considerations do I need to know about when I use virtual inheritance?». Был доступен: 15 сентября 2023 г. <https://isocpp.org/wiki/faq/multiple-inheritance#virtual-inheritance-abcs>.

`protected` (и этот класс не будет объявлен финальным), объект производного класса сможет присвоить элементу данных `m_grossSales` отрицательное значение.

- ♦ Функции производного класса обычно реализуются с зависимостью от данных базового класса. Производные классы должны зависеть только от тех функций базового класса, которые не являются скрытыми. Если мы изменим имя защищенного элемента данных базового класса, нам придется изменить каждый производный класс, который напрямую ссылается на этот элемент. Такие программы называют **хрупкими (fragile)** или **ломкими (brittle)**. Даже маленькое изменение в базовом классе может «сломать» все производные классы — это так называемая проблема хрупкого базового класса¹. Она является основной причиной, почему в C++ Core Guidelines рекомендуется избегать защищенных данных². Вы должны иметь возможность вносить изменения в базовый класс без необходимости изменять его производные классы.

В большинстве случаев предпочтительно использовать скрытые элементы данных, чтобы стимулировать правильную разработку программного обеспечения. Если элементы данных базового класса объявлены скрытыми, вы можете изменять реализацию базового класса без необходимости изменять реализацию производных классов. Это упрощает поддержку, модификацию и отладку вашего кода.

Производный класс может изменять состояние скрытых элементов данных базового класса только через унаследованные функции базового класса, которые не являются скрытыми. Если производный класс может получить доступ к скрытым элементам данных базового класса, то классы, производные от этого производного класса, тоже могут получить доступ к этим элементам данных, поэтому теряются преимущества инкапсуляции.

Защищенные функции базового класса

Одно из применений защищенных элементов данных — определение функций базового класса, которые не должны быть доступны в клиентском коде, но должны быть доступны в производных классах. В этом случае можно переопределить в производном классе защищенную виртуальную функцию базового класса и вызывать ее исходную версию (версию базового класса) из реализации производного класса³. Мы показали эту возможность в примере из раздела 10.11.

¹ «Хрупкий базовый класс». Был доступен: 15 сентября 2023 г. https://ru.wikipedia.org/wiki/Хрупкий_базовый_класс.

² C++ Core Guidelines, «C.133: Avoid protected Data». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rh-protected>.

³ Херб Саттер, «Virtuality». Был доступен: 15 сентября 2023 г. <http://www.gotw.ca/publications/mill18.htm>.

10.16. Открытое, защищенное и скрытое наследование

Вы можете выбрать между открытым, защищенным и скрытым наследованием, но открытое наследование является самым распространенным, а защищенное встречается крайне редко. В таблице приведена информация о доступности элементов данных базового класса в производном классе для каждого типа наследования.

Спецификатор доступа к элементам базового класса	Открытое (public) наследование	Защищенное (protected) наследование	Скрытое (private) наследование
public	public в производном классе. Прямой доступ из функций класса, дружественных функций и сторонних функций.	protected в производном классе. Прямой доступ из функций класса и дружественных функций.	private в производном классе. Прямой доступ из функций класса и дружественных функций.
protected	protected в производном классе. Прямой доступ из функций класса и дружественных функций.	protected в производном классе. Прямой доступ из функций класса и дружественных функций.	private в производном классе. Прямой доступ из функций класса и дружественных функций.
private	Нет доступа в производном классе. Доступ из функций класса и дружественных функций через открытые или защищенные функции класса, унаследованные от базового класса.	Нет доступа в производном классе. Доступ из функций класса и дружественных функций через открытые или защищенные функции класса, унаследованные от базового класса.	Нет доступа в производном классе. Доступ из функций класса и дружественных функций через открытые или защищенные функции класса, унаследованные от базового класса.

При открытом (public) наследовании:

- ♦ открытые элементы базового класса становятся открытыми элементами производного класса;
- ♦ защищенные элементы базового класса становятся защищенными элементами производного класса.

Скрытые элементы базового класса никогда не доступны напрямую в производном классе, но доступ к ним возможен через унаследованные открытые и защищенные функции базового класса, предназначенные для доступа к скрытым элементам.

При защищенном (`protected`) наследовании открытые и защищенные элементы базового класса становятся защищенными элементами производного класса. При скрытом (`private`) наследовании открытые и защищенные элементы базового класса становятся скрытыми элементами производного класса.

Для производных классов, созданных путем скрытого (`private`) и защищенного (`protected`) наследования, не действует отношение «является» применительно к их базовым классам, поскольку открытые элементы базовых классов недоступны для клиентского кода производных классов.

Наследование по умолчанию в классах и структурах

Как классы, так и структуры (рассмотренные в разделе 9.21) могут определять новые типы данных. Однако между классом и структурой есть два ключевых различия. Первое различие заключается в том, что по умолчанию элементы класса являются скрытыми, а элементы структуры являются открытыми. Другое отличие заключается в спецификаторе наследования по умолчанию. Например, следующее определение класса использует по умолчанию скрытое наследование:

```
class Derived : Base {  
    // ...  
};
```

Для сравнения, следующее определение структуры использует по умолчанию открытое наследование:

```
struct Derived : Base {  
    // ...  
};
```

10.17. Другие методы динамического полиморфизма и статический полиморфизм

В этой главе мы сосредоточились на наследовании, а также на нескольких методах динамического полиморфизма. Теперь мы кратко рассмотрим другие методы динамического полиморфизма, а также основы статического полиморфизма (**static polymorphism**), также называемого **полиморфизмом времени компиляции (compile-time polymorphism)**^{1,2}. Эти идиомы и методы предназначены для

¹ «C++: Static Polymorphism». Был доступен: 15 сентября 2023 г. https://en.wikipedia.org/wiki/C%2B%2B#Static_polymorphism.

² Катерина Бондаренко (Kateryna Bondarenko). «Static Polymorphism in C++». Был доступен: 15 сентября 2023 г. <https://medium.com/@kateolenya/static-polymorphism-in-c-9c1ae27a945b>.

опытных разработчиков и достаточно сложны. Некоторые из них мы продолжим рассматривать и в последующих главах. Дополнительную информацию по темам этого раздела вы найдете в соответствующих сносках.

10.17.1. Другие методы динамического полиморфизма

Для реализации динамического полиморфизма мы сначала использовали наследование и виртуальные функции. Затем мы использовали шаблон класса `std::variant` и функцию `std::visit` для реализации динамического полиморфизма с набором заранее известных типов, но без наследования и виртуальных функций. В этом разделе мы рассмотрим другие методы динамического полиморфизма.

Идиома концепта времени выполнения

Идиома концепта времени выполнения (*runtime concept idiom*)^{1,2} была предложена Шоном Парентом³. Она отделяет логику динамической полиморфной обработки от обрабатываемых типов. Эти типы не обязательно должны входить в иерархию классов, поэтому они не должны переопределять виртуальные функции базового класса.

Продолжим тему видеоигры из раздела 10.5. Менеджер экрана многократно удаляет и перерисовывает на экране игровые объекты. С точки зрения идиомы концепта времени выполнения «концепт»⁴ — это функция `draw`, которая должна быть реализована для каждого типа изображаемых объектов. Но вместо того чтобы применять это требование с помощью базового класса, содержащего виртуальную функцию `draw`, идиома концепта времени выполнения использует несвязанные классы и утиную типизацию. Мы можем определить отдельные классы `Martian`, `Venusian`, `Plutonian`, `SpaceShip` и `LaserBeam`. В идиоме концепта времени выполнения все функции `draw` этих классов будут реализованы как перегруженные глобальные функции. У каждой функции будет аргумент: ссылка на тип объекта, который надо нарисовать. Такой подход устраняет тесные связи между классами в иерархиях наследования, затрудняющие внесение изменений в эти иерархии.

Чтобы реализовать полиморфное поведение, идиома концепта времени выполнения использует продуманную систему шаблонов классов, наследования и скрытых виртуальных функций для вызова автономных перегруженных функций `draw`. При этом шаблоны, наследование и виртуальные функции являются деталями реализации, скрытыми от программиста клиентского кода.

¹ Шон Парент (Sean Parent), «Inheritance Is the Base Class of Evil». Был доступен: 15 сентября 2023 г. <https://www.youtube.com/watch?v=b1hUE5uUFOA>.

² Шон Парент, «Better Code: Runtime Polymorphism». Был доступен: 15 сентября 2023 г. <https://www.youtube.com/watch?v=QGcVXgEVMJgNDC>.

³ «Sean Parent». Был доступен: 15 сентября 2023 г. <https://sean-parent.stlab.cc>.

⁴ Не следует путать с концептами C++20, о которых вы узнаете в главе 15.

Динамический полиморфизм со стиранием типов

В языке C++ **стирание типов (type erasure)**¹ — это основанный на шаблонах метод реализации полиморфизма с помощью **утиной типизации (duck typing)**. Шаблоны позволяют вам удалять (стирать) определенные типы из кода, который вы пишете. В отличие от метода на основе `std::variant` (раздел 10.13), метод стирания типов не требует заранее знать все типы, которые должны обрабатываться полиморфно. Идиома концепта времени выполнения использует «под капотом» стирание типов.

Метод стирания типов стал популярным, поэтому были разработаны библиотеки C++ с открытым исходным кодом, упрощающие написание программ с утиной типизацией. В частности, заслуживают внимания:

- ✦ библиотека **Folly**² с открытым исходным кодом, в которую входит шаблон класса `Poly`³, позволяющий реализовать полиморфизм на основе стирания типов;
- ✦ библиотека **Dyno**^{4,5}, которую разработал Луи Дионн (Louis Dionne), гуру шаблонов C++, также позволяющая реализовать полиморфизм на основе стирания типов.

10.17.2. Методы статического полиморфизма

В современном C++ есть важная тенденция: выполнять как можно больше работы на этапе компиляции, чтобы повысить надежность проверки типов и быстродействие программ^{6,7}. В главах 13 и 14 мы рассмотрим некоторые шаблоны классов стандартной библиотеки, а в главе 15 — программирование пользовательских шаблонов. Программисты часто используют шаблоны для того, чтобы генерировать на этапе компиляции код, способный работать со многими типами данных, — он генерируется для каждого типа отдельно. Пользовательские шаблоны и шаблоны функций используются для реализации статического полиморфизма. Благодаря шаблонам, компилятор может выявлять ошибки еще

¹ Артур О'Двайр (Arthur O'Dwyer), «What Is Type Erasure?». Был доступен: 15 сентября 2023 г. <https://quuxplusone.github.io/blog/2019/03/18/what-is-type-erasure>.

² «Folly Open-Source Library». Был доступен: 15 сентября 2023 г. <https://github.com/facebook/folly>.

³ «folly/Poly.h». Был доступен: 15 сентября 2023 г. <https://github.com/facebook/folly/blob/master/folly/docs/Poly.md>.

⁴ «Dyno: Runtime Polymorphism Done Right». Был доступен: 15 сентября 2023 г. <https://github.com/ldionne/dyno>.

⁵ Луи Дионн, «Runtime Polymorphism: Back to the Basics». Был доступен: 15 сентября 2023 г. <https://www.youtube.com/watch?v=gVGtNFg4ay0>.

⁶ «C++ Core Guidelines — Per: Performance». Был доступен: 15 сентября 2023 г. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#S-performance>.

⁷ «Big Picture Issues — What's the Big Deal with Generic Programming?». Был доступен: 15 сентября 2023 г. <https://isocpp.org/wiki/faq/big-picture#generic-paradigm>.

до начала выполнения программы и оптимизировать машинный код для повышения быстродействия¹. В этом разделе мы кратко упомянем несколько методов статического полиморфизма.

Неудавшаяся подстановка не является ошибкой (SFINAE)

Шаблоны поддерживают статический полиморфизм: вы можете генерировать по шаблону класса или функции код, специализированный для работы с одним или несколькими заданными типами. Шаблоны часто предъявляют определенные требования к типам, обязательные для генерации кода. Рассмотрим наш шаблон функции `maximum` из раздела 5.16. Он сравнивает полученные аргументы оператором `>`, поэтому совместим только с теми типами, к которым применим этот оператор. Если мы предоставим этому шаблону функции аргументы, несовместимые с оператором `>`, компилятор сообщит об ошибке. **SFINAE (substitution failure is not an error — неудавшаяся подстановка не является ошибкой)**^{2,3,4} — это термин, означающий, что компилятор отбрасывает неподходящие варианты кода в процессе выбора вызываемой функции. Если компилятор нашел подходящий вариант, код считается корректным. SFINAE не позволяет компилятору выдавать многочисленные сообщения об ошибках с первой же неудачной попытки специализации шаблона. В главе 15 мы рассмотрим SFINAE в контексте концептов C++20^{5,6}.

Концепты C++20

Концепты C++20^{7,8,9} упрощают программирование с шаблонами, позволяя вам задавать ограничения для типов, которые могут использоваться для специализации шаблонов. Компилятор может проверить, удовлетворяет ли тип заданным для

¹ «Compile Time vs Run Time Polymorphism in C++ Advantages/Disadvantages». Был доступен: 15 сентября 2023 г. <https://stackoverflow.com/questions/16875989/compile-time-vs-run-time-polymorphism-in-c-advantages-disadvantages>.

² «Substitution Failure Is Not an Error». Был доступен: 15 сентября 2023 г. https://en.wikipedia.org/wiki/Substitution_failure_is_not_an_error.

³ Бартломей Филипек, «Notes on C++ SFINAE, Modern C++ and C++20 Concepts». Был доступен: 15 сентября 2023 г. <https://www.bfilipek.com/2016/02/notes-on-c-sfinae.html>.

⁴ Давид Вандевурд (Daveed Vandevoorde) и Николай М. Хосуттис (Nicolai M. Josuttis), «C++ Templates: The Complete Guide». Издательство Addison-Wesley Professional, 2002.

⁵ Мариус Банчила (Marius Bancila), «Concepts versus SFINAE-Based Constraints». Был доступен: 15 сентября 2023 г. <https://mariusbancila.ro/blog/2019/10/04/concepts-versus-sfinae-basedconstraints>.

⁶ Бартломей Филипек, «Notes on C++ SFINAE, Modern C++ and C++20 Concepts».

⁷ Саар Раз (Saar Raz), «C++20 Concepts: A Day in the Life». Был доступен: 15 сентября 2023 г. <https://www.youtube.com/watch?v=qawSiMIXtE4>.

⁸ «Constraints and Concepts». Был доступен: 15 сентября 2023 г. <https://en.cppreference.com/w/cpp/language/constraints>.

⁹ «Concepts library». Был доступен: 15 сентября 2023 г. <https://en.cppreference.com/w/cpp/concepts>.

шаблона ограничениям, прежде чем использовать шаблон для генерации кода. В главе 15 мы покажем возможности концептов C++20¹.

Диспетчеризация тегов

Диспетчеризация тегов (tag dispatch)² — это основанный на шаблонах метод, в котором компилятор выбирает для вызова нужную версию перегруженной функции, учитывая не только параметры типов шаблона, но и свойства этих типов. Бьерн Страуструп в своей статье «Concepts: The Future of Generic Programming» называет свойства типов «концептами», а диспетчеризацию тегов — перегрузкой концептов, или перегрузкой на основе концептов³. Затем он рассказывает о том, как заменить диспетчеризацию тегов введенными в C++20 концептами.

10.17.3. Другие концепции полиморфизма

Вас также могут заинтересовать следующие методы:

- ♦ **Двойная диспетчеризация (double dispatch)**^{4,5}, также называемая **шаблоном посетителя (visitor pattern)**⁶, — это метод динамического полиморфизма, основанный на том, что нужная функция выбирается в зависимости от актуальных типов двух объектов в момент вызова. Двойную диспетчеризацию можно реализовать с помощью `std::variant` и `std::visit`⁷.
- ♦ В презентации «Dynamic Polymorphism with Metaclasses and Code Injection» (CPPCON 2020) Сай Бренд, эксперт Microsoft по разработке языка C++, рассказывает о передовых методах динамического полиморфизма, использующих планируемые нововведения в версиях C++23 и C++26⁸.

¹ Мариус Банчила (Marius Bancila), «Concepts versus SFINAE-based constraints».

² «Generic Programming: Tag Dispatching». Был доступен: 15 сентября 2023 г. https://www.boost.org/community/generic_programming.html#tag_dispatching.

³ Бьерн Страуструп, «Concepts: The Future of Generic Programming (Section 6)». Был доступен: 15 сентября 2023 г. https://www.stroustrup.com/good_concepts.pdf.

⁴ «Double Dispatch». Был доступен: 15 сентября 2023 г. https://en.wikipedia.org/wiki/Double_dispatch.

⁵ Барат Каннан (Barath Kannan), «Generalised Double Dispatch». Был доступен: 15 сентября 2023 г. <https://www.youtube.com/watch?v=nNqiBasCab4>.

⁶ «Visitor Pattern». 11 января 2022 г. https://en.wikipedia.org/wiki/Visitor_pattern.

⁷ Вишал Човатия (Vishal Chovatiya), «Double Dispatch in C++: Recover Original Type of the Object Pointed by Base Class Pointer». Был доступен: 15 сентября 2023 г. <http://www.vishalchovatiya.com/double-dispatch-in-cpp>.

⁸ Сай Бренд (Sy Brand), «Dynamic Polymorphism with Metaclasses and Code Injection». Был доступен: 15 сентября 2023 г. https://www.youtube.com/watch?v=8c6BAQcYF_E.

10.18. Итоги

В этой главе мы продолжили тему объектно-ориентированного программирования (ООП), рассказав об основах наследования и динамического полиморфизма. Теперь вы умеете создавать производные классы, наследующие элементы базовых классов, а затем настраивать и дополнять их. Мы провели различие между отношением композиции «содержит» («has-a») и отношением наследования «является» («is-a»).

На примере иерархий наследования мы объяснили и продемонстрировали принципы динамического полиморфизма. Вы научились писать программы, которые единообразно обрабатывают объекты разных классов, входящих в одну иерархию, как если бы все они были объектами базового класса иерархии. Вы научились обращаться к этим объектам через указатели и ссылки базового класса.

Мы показали, что динамический полиморфизм помогает разрабатывать и писать большие программы, которые легко расширять, добавляя новые классы почти без изменений в основных частях кода. Мы детально объяснили, как динамический полиморфизм может быть реализован «под капотом» компилятора C++.

Мы представили вам идиому неvirtуального интерфейса (NVI). Согласно этой идиоме каждая функция базового класса может быть либо открытой неvirtуальной функцией, которую клиентский код может вызывать для выполнения задачи, либо скрытой (или защищенной) виртуальной функцией, которую могут переопределять производные классы. Вы узнали, что при применении этой идиомы виртуальные функции представляют собой внутренние детали реализации, скрытые от клиентского кода, что упрощает поддержку и модификацию программ.

Мы рассказали о наследовании реализации, а затем отметили сложность поддержки и модификации такого кода, подтвержденную многолетней практикой. Затем мы доработали пример программы для расчета зарплаты, применив наследование интерфейса, чтобы базовый класс содержал только чистые виртуальные функции, реализуемые в производных конкретных классах. Этот пример был основан на композиции и внедрении зависимостей: мы включили в класс указатель на объект, от которого зависит поведение объектов этого класса. Затем мы объяснили, как этот подход, основанный на интерфейсах, облегчает модификацию программ.

Далее мы рассмотрели еще один метод реализации динамического полиморфизма, позволяющий обрабатывать объекты не связанных наследованием классов с помощью утиной типизации, шаблона класса `std::variant` и функции `std::visit`.

Мы продемонстрировали множественное наследование, обсудили его потенциальные проблемы и показали, как они решаются виртуальным наследованием. Мы представили вам спецификатор `protected`, дающий функциям и дружественным классам производного класса доступ к защищенным элементам базового класса. Мы сравнили три типа наследования: открытое, защищенное и скрытое — и их влияние на доступность элементов базового класса в производном классе.

Наконец, мы упомянули другие методы динамического полиморфизма и несколько методов статического полиморфизма на основе шаблонов, предоставив ссылки на ресурсы для разработчиков, желающих углубиться в эту тему.

Изучив эту главу, вы освоили принципы наследования и динамического полиморфизма на основе виртуальных функций. Теперь вы сможете лучше оценить другие идиомы и методы полиморфизма, которые могут упрощать модификацию программ и ускорять их работу. Некоторые из этих современных идиом мы рассмотрим в последующих главах.

В главе 11 мы продолжим тему объектно-ориентированного программирования. Мы расскажем о перегрузке операторов, которая позволяет стандартным операторам языка C++ работать с объектами пользовательских типов. Например, вы узнаете, как перегрузить оператор << для вывода всех элементов массива без явного применения инструкции цикла. Вы будете использовать умные указатели, чтобы управлять динамически выделяемой памятью, гарантируя ее освобождение после использования. Мы представим вам ряд специальных функций класса и объясним, как их правильно применять. Мы рассмотрим семантику перемещения, которая позволяет передавать ресурсы (прежде всего динамически выделяемую память) вышедших из области видимости объектов другим объектам, чтобы экономить память и повышать быстродействие.